O'REILLY®

9th Edition

# Java in a Nutshell

## A Desktop Quick Reference



Benjamin J. Evans,
Jason Clark &
David Flanagan

# Java in a Nutshell

A Desktop Quick Reference

NINTH EDITION

**Benjamin J. Evans, Jason Clark, and David Flanagan**




Beijing • Boston • Farnham • Sebastopol • Tokyo

## Java in a Nutshell

by Benjamin J. Evans, Jason R. Clark, and David Flanagan

Copyright © 2026 Benjamin J. Evans and Jason Clark. All rights reserved.

Published by O’Reilly Media, Inc., 141 Stony Circle, Suite 195, Santa Rosa, CA 95401.

O’Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (*https://oreilly.com*). For more information, contact our corporate/institutional sales department: 800-998-9938 or *corporate@oreilly.com*.

Acquisitions Editor: Louise Corrigan

Development Editor: Melissa Potter

Production Editor: Ashley Stussy

Copyeditor: Piper Content Partners

Proofreader: Tim Stewart

Indexer: WordCo Indexing Services, Inc.

Cover Designer: Karen Montgomery

Cover Illustrator: Karen Montgomery

Interior Designer: David Futato

Interior Illustrator: Kate Dullea

August 2026: Ninth Edition

### Revision History for the Ninth Edition

- 2026-08-28: First Release

See https://oreilly.com/catalog/errata.csp?isbn=9798341669543 for release details.

The O’Reilly logo is a registered trademark of O’Reilly Media, Inc. *Java in a Nutshell*, the cover image, and related trade dress are trademarks of O’Reilly Media, Inc.

The views expressed in this work are those of the authors, and do not represent the publisher’s views. While the publisher and the authors have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the authors disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology

this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

979-8-341-66954-3

[LSI]

## Dedication

*This book is dedicated to all who teach peace and resist violence.*

## Foreword

It is a period of stable progress and continued change in the Java ecosystem.

The eighth edition of *Java in a Nutshell* focused on Java 17, the first long-term support version since Java 11 three years earlier. Since that time, the cadence of LTS releases has shortened to every two years and held steady up through Java 25, the target version for this ninth edition.

The early trends around LTS versions have also held true. Interim releases, while useful for keeping a predictable schedule and allowing features to incubate, are rarely deployed to production. The two-year release cycle has proven to be short enough for seeing valuable updates to the platform, but not so harried a pace that teams are discouraged from keeping up.

With the release of Java 25, the language continues moving forward with a mix of incremental language-level improvement like continued investment in pattern matching, and new platform capabilities such as virtual threads.

Java performance continues to improve, and Java 25 is the fastest release yet.

In all, this is a great time to be joining (or returning to) application development in Java. Looking forward, the future still holds major changes that will alter the character of Java development in fundamental ways.

Once again, in working on this new edition of a classic text, if we have preserved the feel of *Java in a Nutshell*, while updating it to bring it to the attention of a new generation of developers, then we shall be well satisfied.

***Ben Evans, Barcelona, Spain, 2026 Jason Clark, Portland, Oregon, 2026***

# Preface

This book is a desktop Java reference, designed to sit faithfully by your keyboard while you program. Part I, "Introducing Java", is a fast-paced, "no-fluff" introduction to the Java programming language and the core runtime aspects of the Java platform. Part II, "Working with the Java Platform", is a reference section that blends elucidation of core concepts with examples of important core APIs. The book covers Java 25, but we recognize that some shops may not have adopted it yet—so where possible we call out when specific features were introduced. We use Java 25 syntax throughout, including `var` and lambda expressions.

## Changes in the Ninth Edition

The eighth edition of this book covered Java 17, whereas this edition covers Java 25. The updated release process of Java has continued since Java 9, with certain releases now badged as long-term support releases. So, Java 25 is the current LTS release at the time of this writing, with Java 21 and 17 preceding it.

With the ninth edition we have tried to update the concept of what it means to be a "Nutshell" guide. The modern Java developer needs to know more than just syntax and APIs. As the Java environment has matured, such topics as concurrency, object-oriented design, memory, and the Java type system have all grown in importance for all developers.

In this edition, we have taken the approach that only the most recent versions of Java are likely to be of interest to the majority of Java developers, so we usually only call out when new features arrived if they entered the language recently.

For example, the module system (that arrived with Java 9) is still likely to be new for at least some developers, and it represents a major change. However, it is also something of an advanced topic and is in some ways separate from the rest of the language, so we have restricted our treatment of it to a single chapter (Chapter 14).

## Contents of This Book

The first six chapters document the Java language and the Java platform—they should all be considered essential reading. The book is biased toward the Oracle/OpenJDK (Open Java Development Kit) implementation of Java but not overly so. Developers working with other Java environments will still find plenty to occupy them. Part I includes:

*Chapter 1, "Introduction to the Java Environment"*

> This chapter is an overview of the Java language and the Java platform. It explains the important features and benefits of Java, along with history and comparison to other languages. We touch on Java security and answer some criticisms of Java.

*Chapter 2, "Java Syntax from the Ground Up"*

This chapter explains the details of the Java programming language. It is a long and detailed chapter that does not assume substantial programming experience. Experienced Java programmers can use it as a language reference. Programmers with substantial experience with languages such as C and C++, Python, or JavaScript should be able to pick up Java syntax quickly by reading this chapter; beginning programmers with only a modest amount of experience should be able to learn Java programming by studying this chapter carefully, although it is best read in conjunction with an introductory text (such as O'Reilly's *Head First Java* by Kathy Sierra, Bert Bates, and Trisha Gee).

*Chapter 3, "Your First Java Programs"*

This chapter describes the growing set of ways that are available for running Java code. It covers the interactive JShell, starting directly from source files, and the more traditional compiled forms of Java.

*Chapter 4, "Object-Oriented Programming in Java"*

This chapter describes how the basic Java syntax documented in Chapter 2 is used to write simple object-oriented programs using classes and objects in Java. The chapter assumes no prior experience with object-oriented programming. It can be used as a tutorial by new programmers or as a reference by experienced Java programmers.

*Chapter 5, "The Java Type System"*

This chapter builds on the basic description of object-oriented programming in Java and introduces the other aspects of Java's type system, such as generic types, enumerated types, and annotations. With this more complete picture, we can discuss the biggest change from Java 8—the arrival of lambda expressions.

*Chapter 6, "Introduction to Object-Oriented Design in Java"*

This chapter is an overview of some basic techniques used in the design of well-engineered object-oriented programs, and it briefly touches on the topic of design patterns and their use in software engineering.

These first six chapters teach you the Java language and get you up and running with the most important concepts of the Java platform. Part II is all about how to get real programming work done in the Java environment. It contains plenty of examples and is designed to complement the cookbook approach found in some other texts. This part includes:

*Chapter 7, "Programming and Documentation Conventions"*

This chapter documents important and widely adopted Java programming conventions. It also explains how you can make your Java code self-documenting by including specially formatted documentation comments.

*Chapter 8, "Working with Java Collections"*

This chapter introduces Java's standard collections libraries. These contain data structures that are vital to the functioning of virtually every Java program—such as `List`, `Map`, and `Set`. The `Stream` abstraction and the relationship between lambda expressions and the collections are explained in detail.

*Chapter 9, "Java and Memory"*

This chapter explains how the Java Virtual Machine manages memory on behalf of the programmer, and how memory and visibility are closely interconnected with Java's support for concurrent programming and threads.

*Chapter 10, "Handling Common Data Formats"*

This chapter discusses how to use Java to work effectively with very common data formats, such as text, numbers, and temporal (date and time) information.

*Chapter 11, "File Handling and I/O"*

This chapter covers several different approaches to file access—from the more classic approach found in older versions of Java to more modern and even asynchronous styles. The chapter concludes with a short introduction to networking with the core Java platform APIs.

*Chapter 12, "Concurrency in Java"*

This chapter covers the basics of concurrent programming in Java. It introduces threads, the fundamental primitives for concurrency in Java. It examines concepts around locking and data integrity which are crucial to safety in concurrent programming. The JDK provides many helpful libraries, for both storing data and managing execution, and these classes are introduced.

*Chapter 13, "Classloading, Reflection, and Method Handles"*

This chapter introduces the subtle art of metaprogramming in Java—first introducing the concept of metadata about Java types, and then turning to the subject of classloading and how Java's security model is linked to the dynamic loading of types. The chapter concludes with some applications of classloading and the relatively new feature of method handles.

*Chapter 14, "Java Platform Modules"*

This chapter describes the Java Platform Module System (JPMS), the major feature that was introduced as part of Java 9, and provides an introduction to the wide-ranging changes that it brings.

*Appendix A, "Future Java"*

This appendix covers Java beyond version 25, including ongoing research and development projects to enhance the language and JVM.

*Appendix B, “Platform Tools”*

Oracle’s JDK (as well as OpenJDK) includes a number of useful Java development tools, most notably the Java interpreter and the Java compiler. This chapter documents those tools, as well as new tools for working with modular Java.

## Related Books

O’Reilly publishes an entire series of books on Java programming, including several companion books to this one:

*Learning Java* *by Marc Loy, Patrick Niemeyer, and Daniel Leuck*

This book is a comprehensive tutorial introduction to Java and includes topics such as XML and client-side Java programming.

*Java 8 Lambdas* *by Richard Warburton*

This book documents the lambda expressions that were introduced in Java 8 in detail and introduces concepts of functional programming that may be unfamiliar to some Java programmers.

*Head First Java* *by Kathy Sierra, Bert Bates, and Trisha Gee*

This book uses a unique approach to teaching Java. Developers who think visually often find it a great accompaniment to a traditional Java book.

You can find a complete list of Java books from O’Reilly at *https://oreil.ly/OzOfe*.

## Conventions Used in This Book

The following typographical conventions are used in this book:

*Italic*

Indicates new terms, URLs, email addresses, filenames, and file extensions

`Constant width`

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords

**`Constant width bold`**

Shows commands or other text that should be typed literally by the user

> **TIP**
>
> This element signifies a tip or suggestion.

> **NOTE**
>
> This element signifies a general note.

> **WARNING**
>
> This element indicates a warning or caution.

## Using Code Examples

Supplemental material (code examples, exercises, etc.) is available for download at *https://github.com/kittylyst/javanut9*.

If you have a technical question or a problem using the code examples, please send email to *support@oreilly.com*.

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but generally do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: "*Java in a Nutshell*, Ninth Edition, by Benjamin J. Evans, Jason Clark, and David Flanagan (O'Reilly). Copyright 2026 Benjamin J. Evans and Jason Clark, 979-8-341-66954-3."

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at *permissions@oreilly.com*.

## O'Reilly Online Learning

> **NOTE**
>
> For more than 40 years, O'Reilly Media has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O’Reilly’s online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O’Reilly and 200+ other publishers. For more information, visit *https://oreilly.com*.

## How to Contact Us

Please address comments and questions concerning this book to the publisher:


O’Reilly Media, Inc.

141 Stony Circle, Suite 195

Santa Rosa, CA 95401

800-889-8969 (in the United States or Canada)

707-827-7019 (international or local)

707-829-0104 (fax)

*support@oreilly.com*

*https://oreilly.com/about/contact.html*


We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at *https://oreil.ly/java-in-a-nutshell-9e*.

For news and information about our books and courses, visit *https://oreilly.com*.

Find us on LinkedIn: *https://linkedin.com/company/oreilly*.

Watch us on YouTube: *https://youtube.com/oreillymedia*.

## Acknowledgments


Melissa Potter was the editor of the ninth edition. Her guidance and many useful contributions were a great help throughout the process.

Special thanks are due to Dan Heidinga, Ashutosh Mehra, and Aleksey Shipilëv.

Our technical reviewers for this edition were Sunny Patel, Chris Devers, Chris Davies, Tony Mancill, and Viktor Danylenko.

# Part I. Introducing Java

Part I is an introduction to the Java language and the Java platform. These chapters provide enough information for you to get started using Java right away:


Chapter 1, “Introduction to the Java Environment”

Chapter 2, “Java Syntax from the Ground Up”

Chapter 3, “Your First Java Programs”

Chapter 4, “Object-Oriented Programming in Java”

Chapter 5, “The Java Type System”

Chapter 6, “Introduction to Object-Oriented Design in Java”

# Chapter 1. Introduction to the Java Environment

Welcome to Java in 2026.

You may be coming to the Java world from another tradition, or maybe this is your first taste of computer programming. Whatever road you may have traveled to get here, welcome—we're glad you've arrived.

Java is a powerful, general-purpose programming environment. It is one of the most widely used programming environments in the world and has been exceptionally successful in business and enterprise computing for over 30 years.

In this chapter, we'll set the scene by describing the Java *language* (which programmers write their applications in), the Java *execution environment* (known as the "Java Virtual Machine," which actually runs those applications), and the Java *ecosystem* (which provides a lot of the value of the programming environment to development teams).

All three of these concepts (language, execution environment, and ecosystem) are habitually referred to just as "Java," with the precise usage inferred from context. In practice, they are such connected ideas that this isn't as confusing as it might first seem.

We'll briefly cover the lifecycle of a Java program, then clear up some common questions about Java, and then move on to discuss the differences between Java and other environments. As Java has been around for many years, some criticisms have been made. We'll address those criticisms and then briefly talk about the history of the Java language and virtual machine and explain how that relates to the release model for today's Java.

Let's get started!

## The Language, the JVM, and the Ecosystem

The base Java programming environment has been around since the late 1990s. It is composed of the Java language and the supporting runtime, the Java Virtual Machine (JVM). The third leg—the Java ecosystem beyond the standard library that ships with Java—is provided by third parties, such as open source projects and Java technology vendors.

At the time that Java was initially developed, this split was considered novel, but trends in software development in the intervening years have made it more commonplace. Notably, Microsoft's .NET environment, announced a few years after Java, adopted a very similar approach to platform architecture.

One important difference between Microsoft's .NET platform and Java is that Java was always conceived as a relatively open ecosystem of multiple vendors, albeit led by a steward who owns

the technology. Throughout Java's history, these vendors have both cooperated and competed on aspects of Java technology.

The current steward of Java is Oracle Corporation (which acquired Sun Microsystems, the originator of Java). Other corporations, such as IBM, Red Hat, Amazon, Microsoft, SAP, Azul Systems, and BellSoft, are also involved in producing implementations of standardized Java technologies.

The standardization in the ecosystem is one of the main reasons for Java's success, as it means there are specifications for the technologies that make up the environment. These standards give the developer and consumer confidence that the technology will be compatible with other components, even if they come from a different technology vendor.

> **TIP**
>
> The reference implementation of Java is the open source OpenJDK (Open Java Development Kit), which many of these companies collaborate and base their shipping products upon.

Java originally comprised several different, but related, environments and specifications, such as Java Mobile Edition (Java ME),[1] Java Standard Edition (Java SE), and Java Enterprise Edition (Java EE).[2] Generally speaking, if someone says "Java" without any further clarification, they mean Java SE.

In this book, we'll cover only Java SE, version 25, with historical notes related to when certain features were introduced into the platform. This is necessary because Java versions tend to have a long lifespan. For example, there are still plenty of deployed applications (probably at least 25% at time of writing) that are running on Java 8—a version that came out in 2014.

We will have more to say about standardization and versions later, so let's move on to discuss the Java language and JVM as separate but related concepts.

## What Is the Java Language?

Java programs are written as source code in the Java language. This is a human-readable programming language, which is strictly class based and object-oriented. The language syntax is deliberately modeled on that of C and C++, and it was explicitly intended to be familiar to programmers coming from those languages, which were the dominant languages at the time Java was created.

> **NOTE**
>
> Although the source code is similar to C++, in practice Java includes features and a managed runtime that has much more in common with dynamic languages.

Java is considered to be relatively easy to read and write (if occasionally a bit verbose). It has a rigid grammar and simple program structure and is intended to be easy to learn and to teach. It is built on industry experience with languages like C++ and tries to remove complex features (to help reduce "unnecessary cleverness") while preserving "what works" from previous programming languages.

The Java language is governed by the Java Language Specification (JLS), which defines how a conforming implementation must behave.

Overall, Java is intended to provide a stable, solid base for companies to develop business-critical applications. As a programming language, it has a relatively conservative design and a slow rate of change. These properties are a conscious attempt to serve the goal of protecting the investment that organizations have made in Java technology.

The language has undergone gradual revision (but no complete rewrites) since its inception in 1995. That means that some of Java's original design choices, which were expedient in the late 1990s, are still affecting the language today—see Chapters 2 and 4 for more details.

On the other hand, in the last 10 or so years, Java has modernized its language syntax to address concerns about verbosity and provide features more familiar to programmers coming from other popular languages.

For example, in 2014, Java 8 added the most radical changes seen in the language for almost a decade. Features like *lambda expressions* and the introduction of the Streams API were enormously popular and forever changed the way Java developers write code.

> **TIP**
>
> If you're returning to Java programming after some time away, you should be pleasantly surprised by the quality-of-life improvements provided for developers in recent versions.

As we'll discuss later in this chapter, the Java project has transitioned to a new release model. In this new model, Java versions are released every six months, but only certain versions (8, 11, 17, 21, and 25) are considered eligible for *long-term support* (LTS). All other versions are supported for only 6 months and have not seen widespread adoption by development teams.

## What Is the JVM?

The JVM is a program that provides the runtime environment necessary for Java programs to execute. Java programs cannot run unless there is a JVM available for the appropriate hardware and OS platform we wish to execute on.

Fortunately, the JVM has been ported to run on a large number of hardware environments—anything from a set-top box or Blu-ray player to a huge mainframe will probably have a JVM available for it. The JVM has its own specification, the Java Virtual Machine Specification, and every implementation must conform to the rules of the specification. When new hardware types

arrive in the mainstream market, then it is likely that companies or individuals interested in the hardware will start a project to port OpenJDK to the new chip.

A recent example of this was the arrival of OpenJDK to Apple's M-series chips (which are a form of AArch64). Red Hat had already ported the JVM to Linux/AArch64 in 2014, and then Azul led a Java Enhancement Proposal (JEP) to bring OpenJDK to macOS/AArch64, which arrived with Java 17. Microsoft engineers provided early public collaboration and branch and pull-request work to complete the native build and run portions and later helped with backports to Java 11.

Java programs can be started in several ways, but the simplest (and oldest) method is to start from a command line:

```
java <arguments> <program name>
```

This brings up the JVM as an operating system process that provides the Java runtime environment and then executes our program in the context of the freshly started (and empty) virtual machine.

It is important to understand that when the JVM takes in a Java program for execution, the program is not provided as Java language source code. Instead, the Java language source must be compiled into a form known as Java bytecode. Java bytecode is then supplied to the JVM in a format called *class files* (which always have a *.class* extension). We can contrast this with Python, where the script is run directly by the Python interpreter.[3]

> **TIP**
>
> The Java platform has always emphasized backward compatibility, and code written for Java 1.0 will still run on today's JVMs without modification or recompilation.

The JVM provides an *execution environment* for the program. It starts an interpreter for the bytecode form of the program that steps through one bytecode instruction at a time. However, production-quality JVMs also provide a special compiler that operates while the Java program is running. This compiler (known as a JIT, or *just-in-time compiler*) will accelerate the important parts of the program by replacing them with equivalent compiled (and heavily optimized) machine code.

You should also be aware that both the JVM and the user program are capable of spawning additional threads of execution, so a user program may have many functions running simultaneously.

The original design of the JVM was built on many years of experience with earlier programming environments, notably C and C++, so we can think of it as having several different goals—all intended to make life easier for the programmer:

- Create a standard execution environment for application code to run inside

- Facilitate secure and reliable code execution (as compared to C/C++)
- Take low-level memory management out of the hands of developers
- Provide a cross-platform execution environment

These objectives are often mentioned together in discussions of the platform.

We've already mentioned the first of these goals when we discussed the JVM and its bytecode interpreter—it functions as the container for application code.

We'll discuss the second and third goals in Chapter 9, when we talk about how the Java environment deals with memory management.

The fourth goal, sometimes called *write once, run anywhere* (*WORA*), is the property that Java class files can be moved from one execution platform to another, and they will run unaltered, provided a JVM is available.

This means that a Java program can be developed (and converted to class files) on a machine running macOS on an M-series chip, and then the class files can be moved to Linux or Microsoft Windows on Intel hardware (or other platforms) and the Java program will run without any further action required.

> **NOTE**
>
> The Java environment has been very widely ported, including to platforms that are very different from mainstream platforms like Linux, macOS, and Windows. In this book, we use the phrase "most implementations" to indicate the platforms that the majority of developers are likely to encounter; macOS, Windows, Linux, BSD Unix, and the like are all considered "mainstream platforms" and count within "most implementations."

In addition to these four primary goals, there is another aspect of the JVM's design that is not always recognized or discussed—it uses runtime information to self-manage.

Software research in the 1970s and 1980s revealed that the runtime behavior of programs has a large number of interesting and useful patterns that cannot be deduced at compile time. The JVM was the first truly mainstream programming environment to use the results of this research.

It collects runtime information to make better decisions about how to execute code. That means that the JVM can monitor and optimize a program running on it in a manner not possible for platforms without this capability.

A key example is the runtime fact that not all parts of a Java program are equally likely to be called during the lifetime of the program—some portions will be called far more often than others. Most Java platform implementations take advantage of this phenomenon with a JIT compiler, as we mentioned earlier.[4]

In the HotSpot JVM (which was the JVM that Sun first shipped as part of Java 1.3 and is still in use today), the JVM first identifies which parts of the program are called most often—the "hot methods." Then the JVM compiles these hot methods directly into machine code, bypassing the

JVM interpreter.

The JVM uses the available runtime information to deliver higher performance than would be possible from purely interpreted execution. In fact, the optimizations that the JVM uses now in many cases produce performance that surpasses compiled code from languages such as C, C++, Go, or Rust.

The standard that describes how a properly functioning JVM must behave is called the *JVM Specification*.

## What Is the Java Ecosystem?

The Java language is easy to learn and contains relatively few abstractions compared to other programming languages. The JVM provides a solid, portable, high-performance base for Java (or other languages) to execute on. Taken together, these two connected technologies provide a foundation that businesses can feel confident about when choosing where to base their development efforts.

The benefits of Java do not end there, however. Since Java's inception, an extremely large ecosystem of third-party libraries and components has grown up. This means that a development team can benefit hugely from the existence of connectors and drivers for practically every technology imaginable—both proprietary and open source.

> **TIP**
>
> Java's build tools (the most popular two are called Maven and Gradle) agree on the dependency formats and standards, and the Maven Central infrastructure serves as a source for libraries, in the same way that the npm registry and PyPI do in the JavaScript and Python communities.

In the modern technology ecosystem, it is now rare indeed to find a technology component that does *not* offer a Java connector. From databases (both traditional relational and NoSQL), to AI and machine learning, to every type of enterprise monitoring system, to messaging systems, to Internet of Things (IoT)—everything integrates with Java.

It is this fact that has been a major driver of adoption of Java technologies by enterprises and larger companies. Development teams have been able to unlock their potential by making use of preexisting libraries and components. This has promoted developer choice and encouraged open, best-of-breed architectures with Java technology cores.

> **NOTE**
>
> Google's Android environment is sometimes thought of as being "based on Java." However, the picture is actually rather more complicated. Android code is written in Java (or, more usually, the Kotlin language) but uses a different compiler, not `javac`, to convert to a different file format for a virtual machine that is *not* the JVM.

The combination of a rich ecosystem and a first-rate virtual machine with an open standard for program binaries makes the Java platform a very attractive execution target. In fact, there are a large number of non-Java languages that target the JVM and also interoperate with Java (which allows them to piggyback off the platform's success). These languages include Kotlin, JRuby, Scala, Clojure, and many others. While all of them are small compared to Java, they have distinct niches within the Java world and provide a source of innovation and healthy competition to Java.

## The Lifecycle of a Java Program

To better understand how Java code is compiled and executed, and the difference between Java and other types of programming environments, consider the pipeline in Figure 1-1, which represents the traditional way that Java programs are created, compiled, and executed.

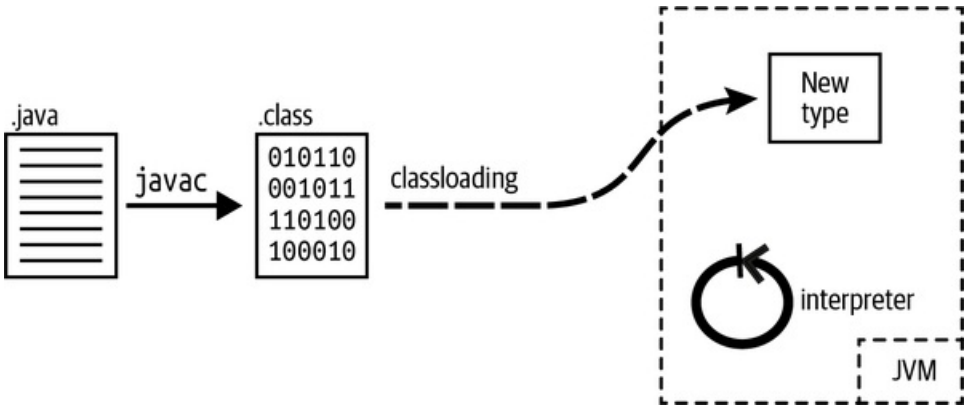



*Figure 1-1. How Java code is compiled and loaded*

This starts with Java source and passes it through the `javac` program to produce class files—which contain the source code compiled to Java bytecode. The class file is the smallest unit of functionality the platform will deal with and the only way to get new code into a running program.

New class files are onboarded via the classloading mechanism (see Chapter 13 for a lot more detail on how classloading works). This makes the new code (represented as a type) available to the interpreter for execution, and execution begins in the `main()` method.

> **TIP**
>
> A major question that many people have is how to analyze the performance of Java programs and optimize their execution if necessary. This is a major topic, and interested readers should consult a specialist text, such as *Optimizing Cloud Native Java* (O'Reilly).

As the program runs, self-management systems may kick in, such as a JIT compiler or garbage collectors (to manage memory automatically). Further classloading is also very possible.

## Frequently Asked Questions

In this section, we'll discuss some of the most frequently asked questions about Java and the lifecycle of programs written in the Java environment.

### What is a virtual machine?

When developers are first introduced to the concept of a virtual machine (VM), they sometimes think of it as "a computer inside a computer" or "a computer simulated in software." It's then easy to imagine bytecode as "machine code for the CPU of the internal computer" or "machine code for a simulated processor." However, this simple intuition can be misleading, and instead it's better to think of a VM as an "internal execution environment" that delegates any functions requiring resources (e.g., memory or input/output [I/O]) to the host machine in which it is a guest.

### What is bytecode?

JVM bytecode is actually not very similar to machine code that would run on a real hardware processor. Instead, computer scientists would call bytecode a type of *intermediate representation*—a midpoint between source code and machine code.

### Is javac a compiler?

Both compilers and `javac` generate binary output may that appear superficially similar—neither is human-readable. Generally speaking, compilers usually produce machine code in the form of object files (such as Windows `.dll` files, or Unix `.so` files), which can then be linked to produce executable binaries for the local machine architecture, whereas `javac` produces machine-independent bytecode.

In computer science terms, `javac` is most similar to the *front half* of a compiler—it creates the intermediate representation that can either be interpreted or used to produce machine code later (e.g., by the JIT compiler).

However, because creating class files is a separate build-time step that resembles compilation in C/C++/Go/Rust, many developers consider running `javac` to be compilation. In this book, we will use the terms *source code compiler* or *javac compiler* to mean the production of class files by `javac`.

We will reserve *compilation* as a standalone term to mean JIT compilation—as it's JIT compilation that actually produces machine code.

### Why is it called "bytecode"?

The instruction code (opcode) is just a single byte (some operations also have parameters that follow them in the bytestream), so there are only 256 possible instructions. In practice, some are unused—about 200 are in use, but some of them aren't emitted by recent versions of `javac`.

### Is bytecode optimized?

In the early days of the platform, `javac` produced heavily optimized bytecode. This turned out

to be a mistake.

With the advent of JIT compilation, the important methods are going to be compiled to very fast machine code. It's therefore very important to make the job of the JIT compiler easier—as there are much bigger gains available from using JIT compilation than there are from optimizing bytecode, which will still have to be interpreted.

### Is bytecode really machine independent? What about things like endianness?

The format of bytecode is always the same, regardless of what type of machine it was created on. This includes the byte ordering (sometimes called *endianness*) of the machine. For readers who are interested in the details, bytecode is always big-endian.

### Is Java an interpreted language?

The JVM is basically an interpreter (with JIT compilation to give it a big performance boost). However, most interpreted languages directly interpret programs from source form (usually by constructing an abstract syntax tree from the input source file). The JVM interpreter, on the other hand, requires class files—which, of course, require a separate source code compilation step with `javac`.

In fact, the modern version of many languages that were traditionally interpreted (such as JavaScript, Python, PHP, and Ruby) now also have JIT compilers, so the divide between "interpreted" and "compiled" languages is increasingly blurred. Once again, Java's design decisions have been validated by their adoption in other programming environments.

### Can other languages run on the JVM?

Yes. The JVM can run any valid class file, so this means that non-Java languages can run on the JVM in several ways. First, they could have a source code compiler (similar to `javac`) that produces class files, which would run on the JVM just like Java code (this is the approach taken by languages like Kotlin and Scala).

Alternatively, a non-Java language could implement an interpreter and runtime in Java and then interpret the source form of their language directly. This second option is the approach taken by languages like JRuby (but JRuby has a very sophisticated runtime that is capable of *secondary JIT compilation* in some circumstances).

## Comparing Java to Other Languages

In this section, we'll briefly highlight some differences between the Java platform and other programming environments you may be familiar with.

### Java Compared to Python

- Java is statically typed; Python is dynamically typed (with optional, *gradual* typing).

- Java is an object-oriented (OO) language with functional programming (FP) features; Python is a hybrid OO/procedural language with some FP support.
- Java and Python both have a bytecode format—Java uses JVM class files; Python uses Python bytecode (PBC)—but this is really an implementation detail that most developers don't have to care about.
- Java's bytecode has extensive static checks; Python's bytecode does not.
- Java is multithreaded; Python allows only one thread to execute Python bytecode at once (the Global Interpreter Lock, or GIL)—this is changing slowly, but it is a huge project within Python.

## Java Compared to JavaScript

- Java is statically typed; JavaScript is dynamically typed.
- Java uses class-based objects; JavaScript is prototype based (the JavaScript keyword `class` is syntactic sugar).
- Java provides good object encapsulation; JavaScript does not.
- Java has namespaces; JavaScript does not.
- Java is multithreaded; JavaScript is not.

## Java Compared to C

- Java is object-oriented; C is procedural.
- Java is portable as class files; C needs to be recompiled.
- Java provides extensive instrumentation as part of the runtime.
- Java has no pointers and no equivalent of pointer arithmetic.
- Java provides automatic memory management via garbage collection (GC).
- Java currently has no ability to lay out memory at a low level (no structs).
- Java has no preprocessor.

## Java Compared to C++

- Java has a simplified object model compared to C++.
- Java's method dispatch is virtual by default.
- Java is always pass-by-value (but one of the only possibilities for Java values is object references).

- Java does not support full multiple inheritance.
- Java's generics are less powerful (but also less dangerous) than C++ templates.
- Java has no operator overloading.

## Answering Some Criticisms of Java

Java has had a long history in the public eye and, as a result, has attracted its fair share of criticism over the years. Some of this negative press can be attributed to some technical shortcomings combined with rather overzealous marketing in the first versions of Java.

Some criticisms have, however, entered technical folklore despite no longer being very accurate. In this section, we'll look at some common grumbles and the extent to which they're true for modern versions of the platform.

### Overly Verbose

The Java core language has sometimes been criticized as overly verbose. Even simple Java statements such as `Object o = new Object();` seem to be repetitious—the type `Object` appears on both the left and right side of the assignment. Critics point out that this is essentially redundant, that other languages do not need this duplication of type information, and that many languages support features (e.g., type inference) that remove it.

The counterpoint to this argument is that Java was designed from the start to be easy to read (code is read more often than written) and that many programmers, especially novices, find the extra type information helpful when reading code.

Java is widely used in enterprise environments, which often have separate dev and ops teams. The extra verbosity can often be a blessing when you are responding to an outage call, or when you need to maintain and patch code that was written by developers who have long since moved on.

In recent versions of Java, the language designers have attempted to respond to some of these points by finding places where the syntax can become less verbose and by making better use of type information. For example:

```
// Files helper methods
byte[] contents =
  Files.readAllBytes(Paths.get("/home/ben/myFile.bin"));

// Diamond syntax for repeated type information
List<String> l = new ArrayList<>();

// Local variables can be type inferred
var threadPool = Executors.newScheduledThreadPool(2);

// Lambda expressions simplify Runnables
threadPool.submit(() -> { System.out.println("On ThreadPool"); });
```

However, Java's overall philosophy is to make changes to the language only very slowly and carefully, so the pace of these changes may not satisfy detractors completely.

## Slow to Change

The original Java language is now over 30 years old and has not undergone a complete and total revision in that time. Many other languages (e.g., Microsoft's C#) have released backward-incompatible versions in the same period, and some developers criticize Java for not doing likewise.

Furthermore, the Java language has sometimes come under fire for being slow to adopt language features that are now commonplace in other languages.

The conservative approach to language design that Oracle (and originally Sun) has taken is an attempt to avoid imposing the costs and externalities of misfeatures on a very large user base. Many Java shops have made major investments in the technology, and the language designers have taken seriously the responsibility of not disrupting the existing user and install base.

Each new language feature needs to be very carefully considered—not only in isolation but in terms of how it will interact with all the existing features of the language. New features can sometimes have impacts beyond their immediate scope—and Java is widely used in very large codebases, where there are more potential places for an unexpected interaction to manifest.

It is almost impossible to remove a feature that turns out to be incorrect after it has shipped. Java has a couple of misfeatures (such as the serialization mechanism) that have been all but impossible to remove safely without impacting the install base. The language designers have taken the view that extreme caution is required when evolving the language.

Having said that, the new language features that have arrived in recent versions are a significant step toward addressing the most common complaints about missing features, and they should cover many of the idioms that developers have been asking for.

## Performance Problems

The Java platform is still sometimes criticized for being slow—but of all the criticisms that are leveled at the platform, this is probably the one that is least justified. It is a genuine myth about the platform.

Release 1.3 of Java introduced the HotSpot Virtual Machine and its JIT compiler. Since then, there have been over 25 years of continual innovation and improvement in the virtual machine and its performance. The Java platform is now blazingly fast, regularly winning performance benchmarks on popular frameworks, and even beating native-compiled C and C++ under the right circumstances.

Criticism in this area appears to be largely caused by a folk memory that Java was slow at some point in the past. Some of the larger, more sprawling architectures that Java has been used within may also have contributed to this impression.

The truth is that any large architecture will require benchmarking, analysis, and performance tuning to get the best out of it—and Java is no exception.

The core of the platform—language and JVM—was and remains one of the fastest general-use environments available to the developer.

One area that still has a lot of misconceptions is developers' understanding of Java's GC, and this deserves a mention here (we'll do a full treatment of GC in Chapter 9). Many universities teach an introduction to GC as part of their curriculum, but it is the very simplest form of the algorithm and doesn't reflect in any way the techniques used in a modern production garbage collector.

A modern GC (Java/JVM is best in class, but other modern languages also have pretty good implementations) gives you safe and flexible automatic memory management at extremely low cost. GC is not a significant performance concern for most server-side workloads anymore, apart from very specific niche use cases.

## Insecure

Some people have historically criticized Java's record of security vulnerabilities.

Many of these vulnerabilities involved the desktop and GUI components of the Java system and wouldn't affect websites or other server-side code written in Java.

The truth is that Java has been designed from the ground up with security in mind; this gives it a great advantage over many other existing systems and platforms. The Java security architecture was designed by security experts and has been studied and probed by many other security experts since the platform's inception. The consensus is that the architecture itself is strong and robust, without any security holes in the design (at least none that have been discovered yet).

Fundamental to the design of the security model is that bytecode is heavily restricted in what it can express—there is no way, for example, to directly address memory. This cuts out entire classes of security problems that have plagued languages like C and C++. Furthermore, the VM goes through a process known as *bytecode verification* whenever it loads an untrusted class, which removes a further large class of problems (see Chapter 13 for more about bytecode verification).

Despite all this, however, no system can guarantee 100% security, and Java is no exception.

While the design is still theoretically robust, the implementation of the security architecture is another matter, and there is a long history of security flaws being found and patched in particular implementations of Java. In all likelihood, security flaws will continue to be discovered (and patched) in Java VM implementations.

All programming platforms have security issues at times, and many other languages have a comparable history of security vulnerabilities that have been significantly less well publicized. For practical server-side coding, Java remains perhaps the most secure general-purpose platform currently available, especially when kept patched and up-to-date.

### Too Corporate

The Java platform is extensively used by corporate and enterprise developers. The perception that it is too corporate is therefore not surprising—Java has often been perceived as lacking the "freewheeling" style of languages that are deemed to be more community oriented.

In truth, Java has always been, and remains, a very widely used language for community and free or open source software development. It is one of the most popular languages for projects hosted on GitHub and other project-hosting sites. Not only that, but the Java community is regularly held up as one of the real strengths of the ecosystem—with user groups, conferences, journals, and all of the most visible signs of an active and healthy user community.

Finally, the most widely used implementation of the language itself is based on OpenJDK—which is itself an open source project with a vibrant and growing community.

## A Brief History of Java and the JVM

As we mentioned at the start of the chapter, one of the facts of life in the Java ecosystem is that multiple versions of Java are supported at any one time. Not only that, but because Java is a very stable platform, many applications do not upgrade their Java version often.

This means that professional Java programmers are very likely to encounter older platform versions, so it can help to have an overview of the history of different versions and when various features were introduced.

*Java 1.0 (1996)*

> This was the first public version of Java. It contained just 212 classes organized in eight packages.

*Java 1.1 (1997)*

> This release of Java more than doubled the size of the Java platform. This release introduced "inner classes" and the first version of the Reflection API.

*Java 1.2 (1998)*

> This was a very significant release of Java; it tripled the size of the Java platform again. This release marked the first appearance of the Java Collections API (with sets, maps, and lists). The many new features in the 1.2 release led Sun to rebrand the platform as "the Java 2 Platform." The term *Java 2* was simply a trademark, however, and not an actual version number for the release.

*Java 1.3 (2000)*

> This was primarily a maintenance release, focused on bug fixes, stability, and performance improvements. This release also introduced the HotSpot Java Virtual Machine, which is still in use today, although it has been heavily modified and improved since then.

*Java 1.4 (2002)*

This was another fairly big release, adding important new functionality such as a higher-performance, low-level I/O API; regular expressions for text handling; XML and XSLT libraries; SSL support; a logging API; and cryptography.

*Java 5 (2004)*

This large release of Java introduced a number of changes to the core language itself, including generic types, enumerated types (enums), annotations, varargs methods, autoboxing, and a new `for` loop. These changes were considered significant enough to change the major version number and to start numbering as major releases. This release included 3,562 classes and interfaces in 166 packages. Notable additions included utilities for concurrent programming, a remote management framework, and classes for the remote management and instrumentation of the Java VM itself.

*Java 6 (2006)*

This release was also largely a maintenance and performance release. It introduced the Compiler API, expanded the usage and scope of annotations, and provided bindings to allow scripting languages to interoperate with Java. There were also a large number of internal bug fixes and improvements to the JVM and the Swing GUI technology.

*Java 7 (2011)*

The first release of Java under Oracle's stewardship included a number of major upgrades to the language and platform and was the first release to be based on the open source reference implementation (OpenJDK). The introduction of `try`-with-resources and the NIO.2 API enabled developers to write much safer and less error-prone code for handling resources and I/O. The Method Handles API provided a simpler and safer alternative to reflection; in addition, it opened the door for `invokedynamic` (the first new bytecode since version 1.0 of Java).

*Java 8 (2014) (retrospectively made LTS)*

This was a huge release—potentially the most significant change to the language since Java 5 (or possibly ever). The introduction of lambda expressions provided the ability to significantly enhance the productivity of developers, the Collections were updated to make use of lambdas, and the machinery required to achieve this marked a fundamental change in Java's approach to object-orientation. Other major updates included a new date and time API and major updates to the concurrency libraries.

*Java 9 (2017)*

Significantly delayed, this release introduced the new platform modularity feature, which allows Java applications to be packaged into deployment units and modularize the platform

runtime. Other changes included a new default GC algorithm, a new API for handling processes, and some changes to the way that frameworks can access the internals. This release also changed the release cycle itself and introduced LTS releases—which are the only ones to have gained market traction since then. Accordingly, we only record the LTS releases beyond this point.

*Java 11 (September 2018) (LTS)*

This release was the first modular Java to be considered an LTS release. It adds a few new features that are directly visible to the developer—primarily improved support for type inference (`var`), Java Development Kit (JDK) Flight Recorder (JFR), and the new HTTP/2 API. There were some additional internal changes and substantial performance improvements, but this LTS release was primarily intended for stabilization after Java 9.

*Java 17 (September 2021) (LTS)*

This release included important changes to Java's OO model (Sealed classes, Records, and Nestmates) as well as Switch Expressions, Text Blocks, and the first version of language Pattern Matching. The JVM had additional performance improvements and better support for running in containers. The internal upgrades continued, and two new GCs were added.

*Java 21 (September 2023) (LTS)*

The arrival of *virtual threads* (vthreads) had been long awaited, providing new concurrency capabilities (very similar to Go's "goroutines" construct). The expansion of Pattern Matching to `switch` represented a major step forward, drawing together incremental improvements over several releases.

*Java 25 (September 2025) (LTS)*

Java 25 is the current LTS version, offering major improvements on the new concurrency capabilities provided by vthreads and incremental additions to Pattern Matching. This version also offers significant performance enhancements.

The current situation with LTS versions can be somewhat confusing for newcomers, so let's take a more in-depth look at Java's release model.

## Java's Release Model

A new version of Java is released every six months, but only certain releases have support available beyond their *release window*, which is the half-year period until the next release comes out. These longer-lived releases are referred to as LTS, and as it stands, Java 11, 17, 21, and 25 are regarded as LTS (with 8 retrospectively added)—although 11 is now out of mainstream support. Non-LTS releases are also known as *Feature Releases*.

> **NOTE**
>
> LTS has a dual meaning: for Oracle customers it means that paid support is available for a multiyear period, while other JDK providers (including Red Hat, Microsoft, Amazon, etc.) have de facto adopted the same versions as those for which backported security and other fixes will be made publicly available—free of charge—as certified OpenJDK binaries. Some OpenJDK vendors also offer paid support options.

The industry, as a whole, has not chosen to adopt a six-month Java upgrade cycle for various reasons, and so in practice, the LTS versions are the only ones that are likely to be deployed into production. However, most of the OpenJDK providers do diligently publish binaries for all Java releases, even those that will not be supported beyond the six-month window.

From this model there arises a dichotomy for this book: new features arrive every six months but are not widely deployed by teams until the next LTS, which complicates the task of writing about specific Java versions. This is further complicated by the concept of *Incubating* and *Preview* features, which are used to experiment with new APIs and new language features, respectively, before they are finalized and become a standard part of the language.

The solution we have chosen is to target new editions of this book at LTS versions (this edition covers Java 25 LTS) and to cover final features only in the main part of the book; discussion of Incubating and Preview features will be confined to appendixes, with forward references where upcoming changes may alter the current circumstances.

As a final note on this subject, you should note that Java 8 is now considered a "classic" release, and teams are strongly encouraged to migrate from it to one of the modern LTS versions. In addition, Java 11 is effectively in maintenance-only mode and does not receive much attention apart from any critical security patches that may arise.

## Summary

In this introductory chapter, we've placed Java in context within the overall landscape and history of programming languages. We've compared the language to other popular alternatives, taken a first look at the basic anatomy of how a Java program is compiled and executed, and tried to dispel some of the popular myths about Java.

The next chapter covers Java's language syntax—primarily from a *bottom-up* perspective, focusing on the individual basic units of lexical syntax and building upward. If you are already familiar with the syntax of a language similar to Java (such as JavaScript, C, or C++), you may choose to skim or skip this chapter and refer to it when you encounter any syntax that is unfamiliar to you.

---

1 Java ME was an older standard for feature phones and first-generation smartphones. Android and iOS dominate the market on phones today, and Java ME is no longer being updated.

2 Java EE has now been transferred to the Eclipse Foundation, where it continues its life as the Jakarta EE project.

[3] Technically speaking, Python too has bytecode (as does Node.js), but this is an implementation detail that is essentially used for performance reasons. You do not copy the Python bytecode from the build machine to the production environment, as you would do with Java class files.

[4] JIT compilation is not a required part of the Java specifications, and implementations exist that do not include it, such as the Zero build of OpenJDK, which is used on iOS.

# Chapter 2. Java Syntax from the Ground Up

This chapter is fairly dense but should provide a comprehensive introduction to Java syntax. It is written primarily for readers who are new to the language but have some previous programming experience. Determined novices with no prior programming experience may also find it useful. If you already know Java, you should find it a useful language reference. The chapter includes some comparisons of Java to JavaScript, Python, C, and C++ for the benefit of programmers coming from those languages.

> **TIP**
>
> If you want to skip ahead and start writing simple Java programs straightaway, you can always jump ahead to Chapter 3 and then use this chapter as a reference that you can look back at as you work your way through that material.

In this chapter, we'll document the syntax of Java programs by starting at the very lowest level of Java syntax and building from there, moving on to increasingly higher orders of structure. We'll cover:

- The characters used to write Java programs and the encoding of those characters.
- Literal values, identifiers, and other tokens that constitute a Java program.
- The data types that Java can manipulate.
- The operators used in Java to group individual tokens into larger expressions.
- Statements, which group expressions and other statements to form logical chunks of Java code.
- Methods, which are named collections of Java statements that can be invoked by other Java code.
- Classes, which are collections of methods and fields. Classes are the central program element in Java and form the basis of object-oriented programming. Chapter 4 is devoted entirely to a discussion of classes and objects.
- Packages, which are collections of related classes.
- Java programs, which consist of one or more interacting classes that may be drawn from one or more packages.

The syntax of most programming languages is complex, and Java is no exception. In general, it

is not possible to document all elements of a language without referring to other elements that have not yet been discussed. For example, it is not really possible to explain in a meaningful way the operators and statements supported by Java without referring to objects. But it is also not possible to document objects thoroughly without referring to the operators and statements of the language. The process of learning Java, or any language, is therefore an iterative one.

## Java Programs from the Top Down

Before we begin our bottom-up exploration of Java syntax, let's take a moment for a top-down overview of a Java program. Java programs consist of one or more files, or *compilation units*, of Java source code. Near the end of the chapter, we describe the structure of a Java file and explain how to compile and run a Java program.

Each compilation unit begins with an optional `package` declaration followed by zero or more `import` declarations. These declarations specify the namespace within which the compilation unit will define names and the namespaces from which the compilation unit imports names. We'll see `package` and `import` again later in this chapter in "Packages and the Java Namespace".

The optional `package` and `import` declarations are typically followed by zero or more *reference type* definitions. We will meet the full variety of possible reference types in Chapters 4 and 5, but for now, we should note that these are most often either `class` or `interface` definitions.

Within the definition of a reference type, we will encounter *members* such as *fields*, *methods*, and *constructors*. Methods are the most important kind of member. Methods are blocks of Java code composed of *statements*.

With these basic terms defined, let's start by approaching a Java program from the bottom up by examining the basic units of syntax—often referred to as *lexical tokens*.

## Lexical Structure

This section explains the lexical structure of a Java program. It starts with a discussion of the Unicode character set in which Java programs are written. It then covers the tokens that constitute a Java program, explaining comments, identifiers, reserved words, literals, and so on.

### The Unicode Character Set

Java programs are written using Unicode. You can use Unicode characters anywhere in a Java program, including comments and identifiers such as variable names. Unlike the 7-bit ASCII character set, which is useful only for English, and the 8-bit ISO Latin-1 character set, which is useful only for major Western European languages, the Unicode character set can represent virtually every written language in common use on the planet.

> **TIP**
>
> If, for some reason, you don't want to work with Unicode directly, you can embed Unicode characters into your Java programs using the special Unicode escape sequence `\u`*xxxx*—that is, a backslash and a lowercase u, followed by four hexadecimal characters. For example, `\u0020` is the space character, and `\u03c0` is the character π.

Java has invested a large amount of time and engineering effort in ensuring that its Unicode support is first class. If your business application needs to deal with global users, especially in non-Western markets, then the Java platform is a great choice. Java also has support for multiple encodings and character sets, in case applications need to interact with legacy non-Java applications that do not speak Unicode.

## Case Sensitivity and Whitespace

Java is a case-sensitive language. Its keywords are written in lowercase and must always be used that way. That is, `While` and `WHILE` are not the same as the `while` keyword. Similarly, if you declare a variable named `i` in your program, you may not refer to it as `I`.

> **WARNING**
>
> In general, relying on case sensitivity to distinguish identifiers is a terrible idea. The more similar identifiers there are, the more difficult the code is to read and understand. Do not use it in your own code, and in particular never give an identifier the same name as a keyword but differently cased.

Java ignores spaces, tabs, newlines, and other whitespace, except when they appear within quoted characters and string literals. Programmers typically use whitespace to format and indent their code for easy readability, but it has no influence on the program's behavior as indents do in Python. You will see common indentation conventions in this book's code examples.

## Comments

Comments are natural-language text intended for human readers of a program. They are ignored by the Java compiler. Java supports four types of comments. The first type is a single-line comment, which begins with the characters `//` and continues until the end of the current line. For example:

```
int i = 0;   // Initialize the loop variable
```

The second kind of comment is a multiline comment. It begins with the characters `/*` and continues, over any number of lines, until the characters `*/`. Any text between the `/*` and the `*/` is ignored by `javac`. Although this style of comment is typically used for multiline comments, it also can be used for single-line comments, or even within a single line. This type of comment cannot be nested (i.e., one `/* */` comment cannot appear within another). When writing

multiline comments, programmers often use extra * characters to make the comments stand out. Here is a typical multiline comment:

```
/*
 * First, establish a connection to the server.
 * If the connection attempt fails, quit right away.
 */
```

The third type of comment is a special case of the second. If a comment begins with /**, it is regarded as a special *doc comment*. Like regular multiline comments, doc comments end with */ and cannot be nested. When you write a Java class, you expect other programmers to use doc comments to embed documentation about the class and each of its methods directly into the source code. A program named javadoc extracts these comments and processes them to create online documentation for your class. A doc comment can contain HTML tags and can use additional syntax understood by javadoc. For example:

```
/**
 * Upload a file to a web server.
 *
 * @param file The file to upload.
 * @return <tt>true</tt> on success,
 *         <tt>false</tt> on failure.
 * @author David Flanagan
 */
```

The fourth and most recently added type of comment is another *doc comment* formatted using per-line comment markers ///. These function similarly to the multiline doc comments but provide support for the popular Markdown format in the documentation. Markdown doc comments are also extracted by javadoc. The preceding example in Markdown format would look like this:

```
/// Upload a file to a web server.
///
/// @param file The file to upload.
/// @return `true` on success,
///         `false` on failure.
/// @author David Flanagan
```

See Chapter 7 for more information on the doc comment syntax, and Appendix B for more information on the javadoc program.

Comments may appear between any tokens of a Java program but may not appear within a token. In particular, comments may not appear within double-quoted string literals. A comment within a string literal simply becomes a literal part of that string.

## Reserved Words

The following words are reserved in Java (they are part of the syntax of the language and may

not be used to name variables, classes, and so forth):

```
abstract   const      final        int          public       throw
assert     continue   finally      interface    return       throws
boolean    default    float        long         short        transient
break      do         for          native       static       true
byte       double     goto         new          strictfp     try
case       else       if           null         super        void
catch      enum       implements   package      switch       volatile
char       extends    import       private      synchronized while
class      false      instanceof   protected    this
```

Of these, `true`, `false`, and `null` are technically literals.

Note that `const` and `goto` are reserved but aren't actually used in the language (instead being "reserved for future use") and that `interface` has an additional variant form—`@interface`, which is used when defining types known as *annotations*. Some of the reserved words (notably `final` and `default`) have a variety of meanings depending on context.

Other keywords exist that are not reserved in general and are known as *contextual keywords*:

```
exports   opens    requires     uses       module    permits
sealed    var      non-sealed   provides   to        with
open      record   transitive   yield
```

`var` indicates a local variable that should be type-inferred. `sealed`, `non-sealed`, and `record` are used when defining classes, which we'll meet in Chapter 4. `yield` appears within `switch` expressions, which we'll meet later in this chapter, while the remaining contextual keywords deal with modules, the syntax and use of which are covered in Chapter 14.

## Identifiers

An *identifier* is simply a name given to some part of a Java program, such as a class, a method within a class, or a variable declared within a method. Identifiers may be of any length and may contain letters and digits drawn from the entire Unicode character set. An identifier may not begin with a digit.

In general, identifiers may not contain punctuation characters. Exceptions include the dollar sign ($) as well as other Unicode currency symbols such as `£` and `¥`.

> **TIP**
>
> Currency symbols are intended for use in automatically generated source code, such as code produced by `javac`. By avoiding the use of currency symbols in your own identifiers, you don't have to worry about collisions with automatically generated identifiers.

The ASCII underscore (_) also deserves special mention. Originally, the underscore could be freely used as an identifier or part of one. From Java 17 through 21, the underscore may not be

used as an identifier. From Java 22 onward, _ is allowed as an unnamed variable, the value of which cannot be read. Examples showing the use of unnamed variables will be highlighted throughout the chapter where applicable.

The usual Java convention is to name variables using *camel case*. This means that the first letter of a variable should be lowercase but the first letter of any other words in the identifier should be uppercase.

Formally, the characters allowed at the beginning of and within an identifier are defined by the methods `isJavaIdentifierStart()` and `isJavaIdentifierPart()` of the class `java.lang.Character`.

The following are examples of legal identifiers:

```
i    x1    theCurrentTime    current    獺
```

Note in particular the example of a UTF-8 identifier, 獺. This is the Kanji character for “otter” and is perfectly legal as a Java identifier. The use of non-ASCII identifiers is unusual in programs predominantly written by Westerners, but it’s sometimes seen.

## Literals

*Literals* are sequences of source characters that directly represent constant values that appear as is in Java source code. They include integer and floating-point numbers, single characters within single quotes, strings of characters within double quotes, and the reserved words `true`, `false`, and `null`. For example, the following are all literals:

```
1    1.0    '1'    1L    "one"    true    false    null
```

The syntax for expressing numeric, character, and string literals is detailed in “Primitive Data Types”.

## Punctuation

Java also uses a number of punctuation characters as tokens. The Java Language Specification divides these characters (somewhat arbitrarily) into two categories, separators and operators. The 12 separators are:

| | | |
|---|---|---|
| `( )` | `{ }` | `[ ]` |
| `...` | `@` | `::` |
| `;` | `,` | `.` |

The 38 operators are:

| | | | | | |
|---|---|---|---|---|---|
| + | – | * | / | % | & |
| += | -= | *= | /= | %= | &= |
| = | == | != | < | <= | > |
| ! | ~ | && | \|\| | ++ | -- |

We'll see separators throughout the book and will cover each operator individually in "Expressions and Operators".

## Primitive Data Types

Java supports eight basic data types known as *primitive types*, as described in Table 2-1. The primitive types include a boolean type, a character type, four integer types, and two floating-point types. The four integer types and the two floating-point types differ in the number of bits that represent them and therefore in the range of numbers they can represent. Note that the size of these types is the notional size in the Java language. Different JVM implementations may use more actual space to hold these values due to padding, alignment, and the like.

*Table 2-1. Java primitive data types*

| Type | Contains | Default | Size | Range |
|---|---|---|---|---|
| `boolean` | `true` or `false` | `false` | 1 bit | NA |
| `char` | Unicode character | `\u0000` | 16 bits | `\u0000` to `\uFFFF` |
| `byte` | Signed integer | 0 | 8 bits | –128 to 127 |
| `short` | Signed integer | 0 | 16 bits | –32768 to 32767 |
| `int` | Signed integer | 0 | 32 bits | –2147483648 to 2147483647 |
| `long` | Signed integer | 0 | 64 bits | –9223372036854775808 t 9223372036854775807 |
| `float` | IEEE 754 | 0.0 | 32 bits | 1.4E–45 to 3.4028235E+3 |

| | | | | |
|---|---|---|---|---|
| | floating point | | | |
| `double` | IEEE 754 floating point | 0.0 | 64 bits | 4.9E–324 to 1.7976931348623157E+3 |

The next section summarizes these primitive data types. In addition to these primitive types, Java supports nonprimitive data types known as *reference types*, which are introduced in “Reference Types”.

## The boolean Type

The `boolean` type represents truth values. This type has only two possible values, representing the two Boolean states: on or off, yes or no, true or false. Java reserves the words `true` and `false` to represent these two Boolean values.

Programmers coming to Java from other languages (especially JavaScript, Python, or C) should note that Java is much stricter about its Boolean values than other languages are; in particular, a `boolean` is neither an integral nor an object type, and incompatible values cannot be used in place of a `boolean`. In other words, you cannot take shortcuts such as the following in Java:

```
Object o = new Object();
int i = 1;

if (o) {     // Invalid!
  while(i) { // Also invalid!
    // ...
  }
}
```

Instead, Java forces you to write cleaner code by explicitly stating the comparisons you want:

```
if (o != null) {
  while(i != 0) {
    // ...
  }
}
```

## The char Type

The `char` type represents Unicode characters. Java has a slightly unique approach to representing characters—`javac` accepts identifiers and literals as UTF-8 (a variable-width encoding) in input. However, internally, Java represents chars in a fixed-width encoding—either a 16-bit encoding (before Java 9) or as ISO-8859-1 (an 8-bit encoding, used for Western European languages, also called Latin-1) if possible (Java 9 and later).

This distinction between external and internal representation does not normally need to concern the developer. In most cases, all that is required is to remember the rule that to include a

character literal in a Java program, simply place it between single quotes (apostrophes):

```
char c = 'A';
```

You can, of course, use Unicode characters as character literals with the `\u` Unicode escape sequence. In addition, Java supports a number of other escape sequences that make it easy both to represent commonly used nonprinting ASCII characters, such as `newline`, and to escape certain punctuation characters that have special meaning in Java. For example:

```
char tab = '\t', nul = '\000', aleph = '\u05D0', backslash = '\\';
```

Table 2-2 lists the escape characters that can be used in `char` literals. These characters can also be used in string literals, which are covered in the next section.

*Table 2-2. Java escape characters*

| Escape sequence | Character value |
|---|---|
| `\b` | Backspace |
| `\t` | Horizontal tab |
| `\n` | Newline |
| `\f` | Form feed |
| `\r` | Carriage return |
| `\"` | Double quote |
| `\'` | Single quote |
| `\\` | Backslash |
| `\`*xxx* | The Latin-1 character with the encoding *xxx*, where *xxx* is an octal (base 8) number between 000 and 377. The forms `\`*x* and `\`*xx* are also legal, as in `\0`, but are not recommended because they can cause difficulties in string constants where the escape sequence is followed by a regular digit. This form is generally discouraged in favor of the `\uxxxx` form. |
| `\u`*xxxx* | |

| | |
|---|---|
| | The Unicode character with encoding *xxxx*, where *xxxx* is four hexadecimal digits. Unicode escapes can appear anywhere in a Java program, not only in character and string literals. |

`char` values can be converted to and from the various integral types, and the `char` data type is a 16-bit integral type. Unlike `byte`, `short`, `int`, and `long`, however, `char` is an unsigned type and may receive values only in the range of 0 to 65535. The `Character` class defines a number of useful `static` methods for working with characters, including `isDigit()`, `isJavaLetter()`, `isLowerCase()`, and `toUpperCase()`.

The Java language and its `char` type were designed with Unicode in mind. The Unicode standard is evolving, however, and each new version of Java adopts a new version of Unicode. Java 11 uses Unicode 10.0.0, Java 17 uses Unicode 13.0, and Java 25 uses Unicode 16.0.

A complication in recent Unicode releases is the introduction of characters whose encodings, or *codepoints*, do not fit in 16 bits. These supplementary characters, which are mostly infrequently used Han (Chinese) ideographs, occupy 21 bits and cannot be represented in a single `char` value. Instead, you must use an `int` value to hold the codepoint of a supplementary character, or you must encode it into a so-called "surrogate pair" of two `char` values.

If you do anticipate having to process characters that do not fit into a `char`, methods have been added to the `Character`, `String`, and related classes for working with text using `int` codepoints.

### String literals

In addition to the `char` type, Java also has a data type for working with strings of text (usually simply called *strings*). The `String` type is a class, however, and is not one of the primitive types of the language. However, because strings are so commonly used, Java does have syntax for including string values literally in a program.

The simplest form of a `String` literal consists of arbitrary text within double quotes (as opposed to the single quotes for `char` literals). For example:

```
"Hello World"
"'This' is a string!"
```

However, because `String` is a reference type, string literals are described in more detail later in this chapter in "String literals". In addition, Chapter 10 contains more details on some of the ways you can work with `String` objects in Java.

## Integer Types

The integer types in Java are `byte`, `short`, `int`, and `long`. As shown in Table 2-1, these four types differ only in the number of bits and, therefore, in the range of numbers each type can represent. All integral types represent signed numbers; there is no `unsigned` keyword as there

is in C and C++.

Literals for each of these types are written exactly as you would expect: as a sequence of decimal digits, optionally preceded by a minus sign.[1] Digits in any of these literals may be separated by an underscore (_) for better readability. Here are some legal integer literals:

```
0
1
123
9_000
-42000
```

Integer literals are 32-bit values (and so are taken to be the Java type `int`) unless they end with the character `L` or `l`, in which case they are 64-bit values (and are understood to be the Java type `long`):

```
1234        // An int value
1234l       // A long value
0xffL       // Another long value
```

Integer literals can also be expressed in hexadecimal, binary, or octal notation. A literal that begins with `0x` or `0X` is taken as a hexadecimal number, using the letters `A` to `F` (or `a` to `f`) as the additional digits required for base-16 numbers.

Integer binary literals start with `0b` and may, of course, feature only the digits 1 or 0. Use of the underscore separator in binary literals is very common, as binary literals can be very long.

Java also supports octal (base-8) integer literals. These literals begin with a leading `0` and cannot include the digits 8 or 9. They are not often used and should be avoided unless needed. Legal hexadecimal, binary, and octal literals include:

```
0xff              // Decimal 255, expressed in hexadecimal
0377              // The same number, expressed in octal (base 8)
0b0010_1111       // Decimal 47, expressed in binary
0xBAADFADE        // Haircut advice in hexadecimal
```

Integer arithmetic in Java never produces an overflow or an underflow error when you exceed the range of a given integer type. Instead, numbers just wrap around. For example, let's look at an overflow:

```
byte b1 = 127, b2 = 1;        // Largest byte is 127
byte sum = (byte)(b1 + b2);   // Sum wraps to -128, the smallest byte
```

and the corresponding underflow behavior:

```
byte b3 = -128, b4 = 5;        // Smallest byte is -128
byte sum2 = (byte)(b3 - b4);   // Sum wraps to a large byte value, 123
```

Neither the Java compiler nor the JVM warns you in any way when this sort of overflow or

underflow occurs.

Instead, when doing integer arithmetic, you must ensure that the type you are using has a sufficient range for the purposes you intend. Integer division by zero and modulo by zero, on the other hand, are illegal and cause an `ArithmeticException` to be thrown. (We'll see more about exceptions soon in "Checked and Unchecked Exceptions").

Each integer type has a corresponding wrapper class: `Byte`, `Short`, `Integer`, and `Long`. Each of these classes defines `MIN_VALUE` and `MAX_VALUE` constants that describe the range of the type. Each class also provides a static `valueOf()` method that is the preferred approach for explicitly creating an instance of the wrapper class from a primitive value. Java also has a feature called *autoboxing* that means that explicit creation of wrapper objects can usually be avoided.

> **WARNING**
>
> While the wrapper classes also have plain constructors that take the primitives as arguments, these constructors are deprecated and should be avoided in favor of the `valueOf()` methods.

The wrapper classes also define useful static methods, such as `Byte.parseByte()` and `Integer.parseInt()`, for converting strings to integer values.

## Floating-Point Types

Real numbers in Java are represented by the `float` and `double` data types. As shown in Table 2-1, `float` is a 32-bit, single-precision, floating-point value, and `double` is a 64-bit, double-precision, floating-point value. Both types adhere to the IEEE 754 standard, which specifies both the format of the numbers and the behavior of arithmetic for the numbers.

Floating-point values can be included literally in a Java program as an optional string of digits, followed by a decimal point and another string of digits. Here are some examples:

```
123.45
0.0
.01
```

Floating-point literals can also use exponential, or scientific, notation, in which a number is followed by the letter `e` or `E` (for exponent) and another number. This second number represents the power of 10 by which the first number is multiplied. For example:

```
1.2345E02     // 1.2345 * 10^2 or 123.45
1e-6          // 1 * 10^-6 or 0.000001
6.02e23       // Avogadro's Number: 6.02 * 10^23
```

Floating-point literals are `double` values by default. To include a `float` value literally in a program, follow the number with `f` or `F`:

```
double d = 6.02E23;
float f = 6.02e23f;
```

Floating-point literals cannot be expressed in hexadecimal, binary, or octal notation.

### FLOATING-POINT REPRESENTATIONS

Most real numbers, by their very nature, cannot be represented exactly in any finite number of bits. Thus, it is important to remember that `float` and `double` values are only approximations of the numbers they are meant to represent. A `float` is a 32-bit approximation, which results in at least six significant decimal digits, and a `double` is a 64-bit approximation, which results in at least 15 significant digits. In Chapter 10, we will cover floating-point representations in more detail.

When a floating-point computation underflows the representable significance of a `float` or a `double`, a zero value results. For example, imagine repeatedly dividing the double value `1.0` by `2.0` (e.g., in a `while` loop). In mathematics, no matter how often we perform the division, the result will never become equal to zero. However, in a floating-point representation, after enough divisions, the result will eventually be so small as to be indistinguishable from zero.

In addition to representing ordinary numbers, the `float` and `double` types can also represent some special values: positive and negative infinity, positive and negative zero, and NaN.

The infinity values result when a floating-point computation produces a value that overflows the representable range of a `float` or `double`. The Java floating-point types make a distinction between positive zero and negative zero, depending on the direction from which the underflow occurred. In practice, positive and negative zero behave pretty much the same. Finally, the last special floating-point value is NaN, which stands for “Not a Number.” The NaN value results when an illegal floating-point operation, such as 0.0/0.0, is performed.

Here are examples of statements that result in these special values:

```
double inf = 1.0/0.0;            // Infinity
double neginf = -1.0/0.0;        // Negative infinity
double negzero = -1.0/inf;       // Negative zero
double NaN = 0.0/0.0;            // Not a Number
```

The `float` and `double` primitive types have corresponding classes, named `Float` and `Double`. Each of these classes defines the following useful constants: `MIN_VALUE`, `MAX_VALUE`, `NEGATIVE_INFINITY`, `POSITIVE_INFINITY`, and `NaN`. Much like the integer wrapper classes, the floating-point wrappers also have a static `valueOf()` for constructing instances.

### NOTE

Java floating-point types can handle overflow to infinity and underflow to zero and have a special NaN value. This

means floating-point arithmetic never throws exceptions, even when performing illegal operations, like dividing zero by zero or taking the square root of a negative number.

The infinite floating-point values behave as you would expect. Adding or subtracting any finite value to or from infinity, for example, yields infinity. Negative zero behaves almost identically to positive zero, and, in fact, the `==` equality operator reports that negative zero is equal to positive zero. One way to distinguish negative zero from positive, or regular, zero is to divide by it: `1.0/0.0` yields positive infinity, but `1.0` divided by negative zero yields negative infinity. Finally, because NaN is Not a Number, the `==` operator says that it is not equal to any other number, nor is NaN equal to itself!

```
double NaN = 0.0/0.0;              // Not a Number
NaN == NaN;                        // false
Double.isNaN(NaN);                 // true
```

To check whether a `float` or `double` value is NaN, you must use the `Float.isNaN()` and `Double.isNaN()` methods.

## Primitive Type Conversions

Java allows conversions between integer values and floating-point values. In addition, because every character corresponds to a number in the Unicode encoding, `char` values can be converted to and from the integer and floating-point types. In fact, `boolean` is the only primitive type that cannot be converted to or from another primitive type in Java.

There are two basic types of conversions. A *widening conversion* occurs when a value of one type is converted to a wider type—one that has a larger range of legal values. For example, Java performs widening conversions automatically when you assign an `int` literal to a `double` variable or a `char` literal to an `int` variable.

Narrowing conversions are another matter, however. A *narrowing conversion* occurs when a value is converted to a type that is not as wide as it is. Narrowing conversions are not always safe: it is reasonable to convert the `int` value 13 to a `byte`, for example, but it is not reasonable to convert 13,000 to a `byte`, because `byte` can hold only numbers between –128 and 127. Because you can lose data in a narrowing conversion, `javac` complains when you attempt any narrowing conversion, even if the value being converted would in fact fit in the narrower range of the specified type:

```
int i = 13;
// byte b = i;    // Incompatible types: possible lossy
                  // conversion from int to byte
```

The one exception to this rule is that you can assign an integer literal (an `int` value) to a `byte` or `short` variable if the literal falls within the range of the variable:

```
byte b = 13;
```

If you need to perform a narrowing conversion and are confident you can do so without losing data or precision, you can force Java to perform the conversion using a language construct known as a *cast*. Perform a cast by placing the name of the desired type in parentheses before the value to be converted. For example:

```
int i = 13;
byte b = (byte) i;   // Force the int to be converted to a byte
i = (int) 13.456;    // Force this double literal to the int 13
```

Casts of primitive types are most often used to convert floating-point values to integers. When you do this, the fractional part of the floating-point value is simply truncated (i.e., the floating-point value is rounded toward zero, not toward the nearest integer). The static methods `Math.round()`, `Math.floor()`, and `Math.ceil()` perform other types of rounding.

The `char` type acts like an integer type in most ways, so a `char` value can be used anywhere an `int` or `long` value is required. Recall, however, that the `char` type is *unsigned*, so it behaves differently from the `short` type, even though both are 16 bits wide:

```
short s = (short) 0xffff; // These bits represent the number -1
char c = '\uffff';        // The same bits, as a Unicode character
int i1 = s;               // Converting the short to an int yields -1
int i2 = c;               // Converting the char to an int yields 65535
```

Table 2-3 shows which primitive types can be converted to which other types and how the conversion is performed. The letter *N* in the table means that the conversion cannot be performed. The letter *Y* means that the conversion is a widening conversion and is therefore performed automatically and implicitly by Java. The letter *C* means that the conversion is a narrowing conversion and requires an explicit cast.

Finally, the notation *Y** means that the conversion is an automatic widening conversion, but some of the least significant digits of the value may be lost in the conversion. This can happen when you are converting an `int` or `long` to a floating-point type—see the table for details. The floating-point types have a larger range than the integer types, so any `int` or `long` can be represented by a `float` or `double`. However, the floating-point types are approximations of numbers and cannot always hold as many significant digits as the integer types (see Chapter 10 for some more detail about floating-point numbers).

*Table 2-3. Java primitive type conversions*

| | **Convert to:** | | | | |
|---|---|---|---|---|---|
| **Convert from:** | **`boolean`** | **`byte`** | **`short`** | **`char`** | **`int`** |

| | | | | | |
|---|---|---|---|---|---|
| `boolean` | - | N | N | N | N |
| `byte` | N | - | Y | C | Y |
| `short` | N | C | - | C | Y |
| `char` | N | C | C | - | Y |
| `int` | N | C | C | C | - |
| `long` | N | C | C | C | C |
| `float` | N | C | C | C | C |
| `double` | N | C | C | C | C |

## Expressions and Operators

So far in this chapter, we've learned about the primitive types that Java programs can manipulate and seen how to include primitive values as *literals* in a Java program. We've also used *variables* as symbolic names that represent, or hold, values. These literals and variables are the tokens out of which Java programs are built.

An *expression* is the next higher level of structure in a Java program. The JVM *evaluates* an expression to compute its value. The very simplest expressions are called *primary expressions* and consist of literals and variables. So, for example, the following are all expressions:

```
1.7         // A floating-point literal
true        // A Boolean literal
sum         // A variable
```

When the JVM evaluates a literal expression, the resulting value is the literal itself. When the JVM evaluates a variable expression, the resulting value is the value stored in the variable.

Primary expressions are not very interesting. More complex expressions are made by using *operators* to combine primary expressions. For example, the following expression uses the assignment operator to combine two primary expressions—a variable and a floating-point literal—into an assignment expression:

```
sum = 1.7
```

But operators are not used with just primary expressions; they can also be used with expressions at any level of complexity. The following are all legal expressions:

```
sum = 1 + 2 + 3 * 1.2 + (4 + 8)/3.0
sum/Math.sqrt(3.0 * 1.234)
(int)(sum + 33)
```

## Operator Summary

The kinds of expressions you can write in a programming language depend entirely on the set of operators available to you. Java has a wealth of operators, but to work effectively with them, you must understand two important concepts: *precedence* and *associativity*. These concepts—and the operators themselves—are explained in more detail in the following sections.

### Precedence

The P column of Table 2-4 specifies the *precedence* of each operator. Precedence specifies the order in which operations are performed. Operations that have higher precedence are performed before those with lower precedence. For example, consider this expression:

```
a + b * c
```

The multiplication operator has higher precedence than the addition operator, so `a` is added to the product of `b` and `c`, just as we expect from elementary mathematics. Operator precedence can be thought of as a measure of how tightly operators bind to their operands. The higher the number, the more tightly they bind.

Default operator precedence can be overridden through the use of parentheses that explicitly specify the order of operations. The previous expression can be rewritten to specify that the addition should be performed before the multiplication:

```
(a + b) * c
```

The default operator precedence in Java was chosen for compatibility with C; the designers of C chose this precedence so that most expressions can be written naturally without parentheses. Only a few common Java idioms require parentheses. Examples include:

```
// Class cast combined with member access
((Integer) o).intValue();

// Assignment combined with comparison
while((line = in.readLine()) != null) { ... }

// Bitwise operators combined with comparison
if ((flags & (PUBLIC | PROTECTED)) != 0) { ... }
```

### Associativity

Associativity is a property of operators that defines how to evaluate expressions that would otherwise be ambiguous. This is particularly important when an expression involves several operators that have the same precedence.

Most operators are left-to-right associative, which means that the operations are performed from left to right. The assignment and unary operators, however, have right-to-left associativity. The A column of Table 2-4 specifies the associativity of each operator or group of operators. The value L means left to right, and R means right to left.

The additive operators are all left-to-right associative, so the expression `a+b-c` is evaluated from left to right: `(a+b)-c`. Unary operators and assignment operators are evaluated from right to left. Consider this complex expression:

```
a = b += c = -~d
```

This is evaluated as follows:

```
a = (b += (c = -(~d)))
```

As with operator precedence, operator associativity establishes a default order of evaluation for an expression. This default order can be overridden through the use of parentheses. However, the default operator associativity in Java has been chosen to yield a natural expression syntax. Despite this, adding parentheses can often result in code that is easier to read and be understood by a larger audience, even if the parentheses aren't actually modifying the rules of precedence or associativity.

### Operator summary table

Table 2-4 summarizes the operators available in Java. The Prec and Assoc columns of the table specify the precedence and associativity, respectively, of each group of related operators. The table is ordered from highest precedence to lowest. Use this table as a quick reference for operators (especially their precedence) when required.

*Table 2-4. Java operators*

| Prec | Assoc | Operator | Operand type(s) | Operation performed |
|---|---|---|---|---|
| 16 | Left | . | object, member | Object member access |
| | | `[ ]` | array, int | Array element access |
| | | `( args )` | method, arglist | Method invocation |
| | | `++, --` | variable | Post-increment, post-decrement |

| | | | | |
|---|---|---|---|---|
| 15 | Right | `++`, `--` | variable | Pre-increment, pre-decrement |
| | | `+`, `-` | number | Unary plus, unary minus |
| | | `~` | integral | Bitwise complement |
| | | `!` | boolean | Boolean NOT |
| 14 | Right | `new` | class, arglist | Object creation |
| | | `( type )` | type, any | Cast (type conversion) |
| 13 | Left | `*`, `/`, `%` | number, number | Multiplication, division, remainder |
| 12 | Left | `+`, `-` | number, number | Addition, subtraction |
| | | `+` | string, any | String concatenation |
| 11 | Left | `<<` | integral, integral | Left shift |
| | | `>>` | integral, integral | Right shift with sign extension |
| | | `>>>` | integral, integral | Right shift with zero extension |
| 10 | Left | `<`, `<=` | number, number | Less than, less than or equal |
| | | `>`, `>=` | number, number | Greater than, greater than or equal |
| | | `instanceof` | reference, type | Type comparison |
| 9 | Left | `==` | primitive, primitive | Equal (have identical values) |

| | | | | |
|---|---|---|---|---|
| | | != | primitive, primitive | Not equal (have different values) |
| | | == | reference, reference | Equal (refer to same object) |
| | | != | reference, reference | Not equal (refer to different objects) |
| 8 | Left | & | integral, integral | Bitwise AND |
| | | & | boolean, boolean | Boolean AND |
| 7 | Left | ^ | integral, integral | Bitwise XOR |
| | | ^ | boolean, boolean | Boolean XOR |
| 6 | Left | \| | integral, integral | Bitwise OR |
| | | \| | boolean, boolean | Boolean OR |
| 5 | Left | && | boolean, boolean | Conditional AND |
| 4 | Left | \|\| | boolean, boolean | Conditional OR |
| 3 | Right | ? : | boolean, any | Conditional (ternary) operator |
| 2 | Right | = | variable, any | Assignment |
| | | *=, /=, %=, | variable, any | Assignment with operation |

| | | | | |
|---|---|---|---|---|
| | | +=, -=, <<=, | | |
| | | >>=, >>>=, | | |
| | | &=, ^=, \|= | | |
| 1 | Right | -> | arglist, method body | lambda expression |

### Operand number and type

The fourth column of Table 2-4 specifies the number and type of the operands expected by each operator. Some operators operate on only one operand; these are called *unary operators*. For example, the unary minus operator changes the sign of a single number:

```
-n              // The unary minus operator
```

Most operators, however, are binary operators that operate on two operand values. The – operator actually comes in both forms:

```
a - b           // The subtraction operator is a binary operator
```

Java also defines one ternary operator, often called the *conditional operator*. It is like an `if` statement inside an expression. Its three operands are separated by a question mark and a colon; the second and third operands must be convertible to the same type:

```
x > y ? x : y  // Ternary expression; evaluates to larger of x and y,
               // or to y if x and y are equal
```

In addition to expecting a certain number of operands, each operator also expects particular types of operands. The fourth column of the table lists the operand types. Some of the codes used in that column require further explanation:

*Number*

An integer, floating-point value, or character (i.e., any primitive type except `boolean`). Auto-unboxing (see "Boxing and Unboxing Conversions") means that the wrapper classes (such as `Character`, `Integer`, and `Double`) for these types can be used in this context as well.

*Integer*

A `byte`, `short`, `int`, `long`, or `char` value (`long` values are not allowed for the array access operator `[ ]`). With auto-unboxing, `Byte`, `Short`, `Integer`, `Long`, and

`Character` values are also allowed.

*Reference*

An object or array.

*Variable*

A variable or anything else, such as an array element, to which a value can be assigned.

### Return type

Just as every operator expects its operands to be of specific types, each operator produces a value of a specific type. For example, the arithmetic, increment, and decrement return a `double` if at least one of the operands is a `double`. They return a `float` if at least one of the operands is a `float`. They return a `long` if at least one of the operands is a `long`. Otherwise, they return an `int`, even if both operands are `byte`, `short`, or `char` types that are narrower than `int`.

The comparison, equality, and Boolean operators always return `boolean` values. Each assignment operator returns whatever value it assigned, which is of a type compatible with the variable on the left side of the expression. The conditional operator returns the value of its second or third argument (which must both be convertible to the same type).

### Side effects

Every operator computes a value based on one or more operand values. Some operators, however, have *side effects* in addition to their basic evaluation. If an expression contains side effects, evaluating it changes the state of a Java program in such a way that evaluating the expression again may yield a different result.

For example, the `++` increment operator has the side effect of incrementing a variable. The expression `++a` increments the variable `a` and returns the newly incremented value. If this expression is evaluated again, the value will be different. The various assignment operators also have side effects. For example, the expression `a*=2` can also be written as `a=a*2`. The value of the expression is the value of `a` multiplied by 2, but the expression has the side effect of storing that value back into `a`.

The method invocation operator `()` has side effects if the invoked method has side effects. Some methods, such as `Math.sqrt()`, simply compute and return a value without side effects of any kind. Typically, however, methods do have side effects. Finally, the `new` operator has the profound side effect of creating a new object.

### Order of evaluation

When the JVM evaluates an expression, it performs the various operations in an order specified by the parentheses in the expression, the precedence of the operators, and the associativity of the operators. Before any operation is performed, however, the JVM first evaluates the operands of the operator. (The exceptions are the `&&`, `||`, and `?:` operators, which do not always evaluate all

their operands.) The JVM always evaluates operands in order from left to right. This matters if any of the operands are expressions that contain side effects. Consider this code, for example:

```
int a = 2;
int v = ++a + ++a * ++a;
```

Although the multiplication is performed before the addition, the operands of the `+` operator are evaluated first. As the operands of `++` are both `++a`, these are evaluated to `3` and `4`, and so the expression evaluates to `3 + 4 * 5`, or `23`—a great example of where parentheses could be usefully deployed to make the meaning clear.

## Arithmetic Operators

The arithmetic operators can be used with integers, floating-point numbers, and even characters (i.e., they can be used with any primitive type other than `boolean`). If either of the operands is a floating-point number, floating-point arithmetic is used; otherwise, integer arithmetic is used. This matters because integer arithmetic and floating-point arithmetic differ in the way division is performed and in the way underflows and overflows are handled, for example. The arithmetic operators are:

*Addition (`+`)*

The `+` operator adds two numbers. As we'll see shortly, the `+` operator can also be used to concatenate strings. If either operand of `+` is a string, the other one is converted to a string as well. Be sure to use parentheses when you want to combine addition with concatenation. For example:

```
System.out.println("Total: " + 3 + 4);   // Prints "Total: 34", not 7!
```

The `+` operator can also be used in unary form to explicitly express a positive number, such as `+42`.

*Subtraction (`-`)*

When the `-` operator is used as a binary operator, it subtracts its second operand from its first. For example, `7-3` evaluates to `4`. The `-` operator can also perform unary negation.

*Multiplication (`*`)*

The `*` operator multiplies its two operands. For example, `7*3` evaluates to `21`.

*Division (`/`)*

The `/` operator divides its first operand by its second. If both operands are integers, the result is an integer, and any remainder is lost. If either operand is a floating-point value, however, the result is a floating-point value. When you divide two integers, division by zero throws an

`ArithmeticException`. For floating-point calculations, however, division by zero simply yields an infinite result or `NaN`:

```
7/3          // Evaluates to 2
7/3.0f       // Evaluates to 2.333333f
7/0          // Throws an ArithmeticException
7/0.0        // Evaluates to positive infinity
0.0/0.0      // Evaluates to NaN
```

*Modulo (%)*

The `%` operator computes the first operand modulo the second operand (i.e., it returns the remainder when the first operand is divided by the second operand an integral number of times). For example, `7%3` is `1`. The sign of the result is the same as the sign of the first operand. While the modulo operator is typically used with integer operands, it also works for floating-point values. For example, `4.3%2.1` evaluates to `0.1`. When you are operating with integers, trying to compute a value modulo zero causes an `ArithmeticException`. When you are working with floating-point values, anything modulo `0.0` evaluates to `NaN`, as does infinity modulo anything.

*Unary minus (–)*

When the `–` operator is used as a unary operator—that is, before a single operand—it performs unary negation. In other words, it converts a positive value to an equivalently negative value, and vice versa.

## String Concatenation Operator

In addition to adding numbers, the `+` operator (and the related `+=` operator) also concatenates, or joins, strings. If either of the operands to `+` is a string, the operator converts the other operand to a string. For example:

```
// Prints "Quotient: 2.3333333"
System.out.println("Quotient: " + 7/3.0f);
```

As a result, you must be careful to put any addition expressions in parentheses when combining them with string concatenation. If you do not, the addition operator is interpreted as a concatenation operator.

Java has built-in string conversions for all primitive types. An object is converted to a string by invoking its `toString()` method. Some classes define custom `toString()` methods so that objects of that class can easily be converted to strings in this way. Sadly, not all classes return friendly results when converted to strings. For example, the built-in `toString()` for an array

doesn't return a useful string representation of its contents, only information about the array object itself.

## Increment and Decrement Operators

The `++` operator increments its single operand, which must be a variable, an element of an array, or a field of an object, by one. The expression value of this operator depends on its position relative to the operand. When used before the operand, where it is known as the *pre-increment* operator, it increments the operand and evaluates to the incremented value of that operand. When used after the operand, where it is known as the *post-increment* operator, it increments its operand but evaluates to the value of that operand before it was incremented.

For example, the following code sets both `i` and `j` to 2:

```
i = 1;
j = ++i;
```

But these lines set `i` to 2 and `j` to 1:

```
i = 1;
j = i++;
```

Similarly, the `--` operator decrements its single numeric operand, which must be a variable, an element of an array, or a field of an object, by one. Like the `++` operator, the behavior of `--` depends on its position relative to the operand. When used before the operand, it decrements the operand and returns the decremented value. When used after the operand, it decrements the operand but returns the *undecremented* value.

The expressions `x++` and `x--` are equivalent to `x = x + 1` and `x = x – 1`, respectively, except that when you are using the increment and decrement operators, `x` is evaluated only once. If `x` is itself an expression with side effects, this makes a big difference. For example, these two expressions are not equivalent, as the second form increments `i` twice:

```
a[i++]++;             // Increments an element of an array

// Adds 1 to an array element and stores new value in another element
a[i++] = a[i++] + 1;
```

These operators, in both prefix and postfix forms, are most commonly used to increment or decrement the counter that controls a loop.

This somewhat subtle behavior was taken from C and C++. However, an increasing number of programmers prefer to avoid using the increment and decrement operators altogether, preferring to use explicit code. This view is motivated by the large number of bugs that historically were caused by incorrect usage of the operators.

## Comparison Operators

The comparison operators consist of the equality operators that test values for equality or inequality and the relational operators used with ordered types (numbers and characters) to test for greater than and less than relationships. Both types of operators yield a `boolean` result, so they are typically used with `if` statements, the ternary conditional operator, or `while` and `for` loops to make branching and looping decisions. For example:

```
if (o != null) ...;           // The not equals operator
while(i < a.length) ...;      // The less than operator
```

Java provides the following equality operators:

*Equals (==)*

The `==` operator evaluates to `true` if its two operands are equal, and `false` otherwise. With primitive operands, it tests whether the operand values themselves are identical. For operands of reference types, however, it tests whether the operands refer to the same object or array. In other words, it does not test the equality of two distinct objects or arrays. In particular, note that you cannot test two distinct strings for equality with this operator.

> **WARNING**
>
> If you experiment comparing strings via `==`, you may sometimes see results that suggest it works properly. This is a side effect of Java's internal caching of strings, and is not to be trusted. The only reliable way to compare strings (or any other reference type for that matter) for equality is the `equals()` method. More discussion of object equality can be found in "equals()".

If `==` is used to compare two numeric or character operands that are not of the same type, the narrower operand is converted to the type of the wider operand before the comparison is done. For example, when you are comparing a `short` to a `float`, the `short` is first converted to a `float` before the comparison is performed. For floating-point numbers, the special negative zero value tests equal to the regular, positive zero value. Also, the special `NaN` (Not a Number) value is not equal to any number, nor is it equal to itself. To test whether a floating-point value is `NaN`, use the `Float.isNaN()` or `Double.isNaN()` method.

*Not equals (`!=`)*

The `!=` operator is exactly the opposite of the `==` operator. It evaluates to `true` if its two primitive operands have different values or if its two reference operands refer to different objects or arrays. Otherwise, it evaluates to `false`.

The relational operators can be used with numbers and characters but not with `boolean` values, objects, or arrays because those types are not ordered.

Java provides the following relational operators:

*Less than (<)*

Evaluates to `true` if the first operand is less than the second

*Less than or equal (<=)*

Evaluates to `true` if the first operand is less than or equal to the second

*Greater than (>)*

Evaluates to `true` if the first operand is greater than the second

*Greater than or equal (>=)*

Evaluates to `true` if the first operand is greater than or equal to the second

## Boolean Operators

As we've just seen, the comparison operators compare their operands and yield a `boolean` result, which is often used in branching and looping statements. In order to make branching and looping decisions based on conditions more interesting than a single comparison, you can use the Boolean (or logical) operators to combine multiple comparison expressions into a single, more complex expression. The Boolean operators require their operands to be `boolean` values, and they evaluate to `boolean` values. The operators are:

*Conditional AND (&&)*

This operator performs a Boolean AND operation on its operands. It evaluates to `true` if and only if both of its operands are `true`. If either or both operands are `false`, it evaluates to `false`. For example:

```
if (x < 10 && y > 3) ... // If both comparisons are true
```

This operator (and all the Boolean operators except the unary `!` operator) have a lower precedence than the comparison operators. Thus, it is perfectly legal to write a line of code like the one just shown. However, some programmers prefer to use parentheses to make the order of evaluation explicit:

```
if ((x < 10) && (y > 3)) ...
```

You should use whichever style you find easier to read, or the one in use by your wider team.

This operator is called a conditional AND because it conditionally evaluates its second operand. If the first operand evaluates to `false`, the value of the expression is `false`, regardless of the value of the second operand. Therefore, to increase efficiency, the JVM

takes a shortcut and skips the second operand. The second operand is not guaranteed to be evaluated, so you must use caution when using this operator with expressions that have side effects. On the other hand, the conditional nature of this operator allows us to write Java expressions such as the following:

```
if (data != null && i < data.length && data[i] != -1)
    ...
```

The second and third comparisons in this expression would cause errors if the first or second comparisons evaluated to `false`. Fortunately, we don't have to worry about this because of the conditional behavior of the && operator.

*Conditional OR (||)*

This operator performs a Boolean OR operation on its two `boolean` operands. It evaluates to `true` if either or both of its operands are `true`. If both operands are `false`, it evaluates to `false`. Like the && operator, `||` does not always evaluate its second operand. If the first operand evaluates to `true`, the value of the expression is `true`, regardless of the value of the second operand. Thus, the operator simply skips the second operand in that case.

*Boolean NOT (!)*

This unary operator changes the `boolean` value of its operand. If applied to a `true` value, it evaluates to `false`, and if applied to a `false` value, it evaluates to `true`. It is useful in expressions like these:

```
if (!found) ...          // found is a boolean declared somewhere
while (!c.isEmpty()) ... // The isEmpty() method returns a boolean
```

Because `!` is a unary operator, it has a high precedence and often must be used with parentheses:

```
if (!(x > y && y > z))
```

*Boolean AND (&)*

When used with `boolean` operands, the & operator behaves like the && operator, except that it always evaluates both operands, regardless of the value of the first operand. This operator is almost always used as a bitwise operator with integer operands, however, and many Java programmers would not even recognize its use with `boolean` operands as legal Java code.

*Boolean OR (|)*

This operator performs a Boolean OR operation on its two `boolean` operands. It is like the `||` operator, except that it always evaluates both operands, even if the first one is `true`. The `|` operator is almost always used as a bitwise operator on integer operands; its use with `boolean` operands is very rare.

*Boolean XOR (^)*

When used with `boolean` operands, this operator computes the exclusive OR (XOR) of its operands. It evaluates to `true` if exactly one of the two operands is `true`. In other words, it evaluates to `false` if both operands are `false` or if both operands are `true`. Unlike the `&&` and `||` operators, this one must always evaluate both operands. The `^` operator is much more commonly used as a bitwise operator on integer operands. With `boolean` operands, this operator is equivalent to the `!=` operator.

## Bitwise and Shift Operators

The bitwise and shift operators are low-level operators that manipulate the individual bits that make up an integer value. The bitwise operators are not commonly used in modern Java except for low-level work (e.g., network programming). They are used for testing and setting individual flag bits in a value. To understand their behavior, you must understand binary (base-2) numbers and the two's complement format used to represent negative integers.

You cannot use these operators with floating-point, `boolean`, array, or object operands. When used with `boolean` operands, the `&`, `|`, and `^` operators perform a different operation, as described in the previous section.

If either of the arguments to a bitwise operator is a `long`, the result is a `long`. Otherwise, the result is an `int`. If the left operand of a shift operator is a `long`, the result is a `long`; otherwise, the result is an `int`. The operators are:

*Bitwise complement (~)*

The unary `~` operator is known as the *bitwise complement*, or *bitwise NOT*, operator. It inverts each bit of its single operand, converting 1s to 0s and 0s to 1s. For example:

```
byte b = ~12;           // ~00001100 == 11110011 or -13 decimal
flags = flags & ~f;     // Clear flag f in a set of flags
```

*Bitwise AND (&)*

This operator combines its two integer operands by performing a Boolean AND operation on their individual bits. The result has a bit set only if the corresponding bit is set in both operands. For example:

```
10 & 7                  // 00001010 & 00000111 == 00000010 or 2
if ((flags & f) != 0)   // Test whether flag f is set
```

When used with `boolean` operands, & is the infrequently used Boolean AND operator described earlier.

*Bitwise OR (|)*

This operator combines its two integer operands by performing a Boolean OR operation on their individual bits. The result has a bit set if the corresponding bit is set in either or both of the operands. It has a zero bit only where both corresponding operand bits are zero. For example:

```
10 | 7                  // 00001010 | 00000111 == 00001111 or 15
flags = flags | f;      // Set flag f
```

When used with `boolean` operands, | is the infrequently used Boolean OR operator described earlier.

*Bitwise XOR (^)*

This operator combines its two integer operands by performing a Boolean XOR (exclusive OR) operation on their individual bits. The result has a bit set if the corresponding bits in the two operands are different. If the corresponding operand bits are both 1s or both 0s, the result bit is a 0. For example:

```
10 ^ 7              // 00001010 ^ 00000111 == 00001101 or 13
```

When used with `boolean` operands, ^ is the seldom used Boolean XOR operator.

*Left shift (<<)*

The << operator shifts the bits of the left operand left by the number of places specified by the right operand. High-order bits of the left operand are lost, and zero bits are shifted in from the right. Shifting an integer left by *n* places is equivalent to multiplying that number by $2^n$. For example:

```
10 << 1    // 0b00001010 << 1 == 00010100 = 20 = 10*2
7 << 3     // 0b00000111 << 3 == 00111000 = 56 = 7*8
-1 << 2    // 0xFFFFFFFF << 2 == 0xFFFFFFFC = -4 = -1*4
           // 0xFFFF_FFFC == 0b1111_1111_1111_1111_1111_1111_1111_1100
```

If the left operand is a `long`, the right operand should be between 0 and 63. Otherwise, the left operand is taken to be an `int`, and the right operand should be between 0 and 31. If either of these ranges is exceeded, you may see unintuitive wrapping behavior from these operators.

*Signed right shift (>>)*

The >> operator shifts the bits of the left operand to the right by the number of places specified by the right operand. The low-order bits of the left operand are shifted away and are lost. The high-order bits shifted in are the same as the original high-order bit of the left operand. In other words, if the left operand is positive, 0s are shifted into the high-order bits. If the left operand is negative, 1s are shifted in instead. This technique is known as *sign extension*; it is used to preserve the sign of the left operand. For example:

```
10 >> 1      // 00001010 >> 1 == 00000101 = 5 = 10/2
27 >> 3      // 00011011 >> 3 == 00000011 = 3 = 27/8
-50 >> 2     // 11001110 >> 2 == 11110011 = -13 != -50/4
```

If the left operand is positive and the right operand is *n*, the >> operator is the same as integer division by $2^n$.

*Unsigned right shift (>>>)*

This operator is like the >> operator, except that it always shifts zeros into the high-order bits of the result, regardless of the sign of the lefthand operand. This technique is called *zero extension*; it is appropriate when the left operand is being treated as an unsigned value (despite the fact that Java integer types are all signed). These are examples:

```
0xff >>> 4    // 11111111 >>> 4 == 00001111 = 15  = 255/16
-50 >>> 2     // 0xFFFFFFCE >>> 2 == 0x3FFFFFF3 = 1073741811
```

## Assignment Operators

The assignment operators store, or assign, a value into a piece of the computer's memory—often referred to as a *storage location*. The left operand must evaluate to an appropriate local variable, array element, or object field.

> **NOTE**
>
> The lefthand side of an assignment expression is sometimes called an *lvalue*. In Java it must refer to some assignable storage (i.e., memory that can be written to).

The righthand side (the *rvalue*) can be any value of a type compatible with the variable. An assignment expression evaluates to the value that is assigned to the variable. More importantly, however, the expression has the side effect of actually performing the assignment—storing the *rvalue* in the *lvalue*.

> **TIP**
>
> Unlike all other binary operators, the assignment operators are right-associative, which means that the assignments in `a=b=c` are performed right to left, as follows: `a=(b=c)`.

The basic assignment operator is `=`. Do not confuse it with the equality operator, `==`. To keep these two operators distinct, we recommend that you read `=` as "is assigned the value."

In addition to this simple assignment operator, Java also defines 11 other operators that combine assignment with the five arithmetic operators and the six bitwise and shift operators.

For example, the `+=` operator reads the value of the left variable, adds the value of the right operand to it, stores the sum back into the left variable as a side effect, and returns the sum as the value of the expression.

Thus, the expression `x+=2` is almost the same as `x=x+2`. The difference between these two expressions is that when you use the `+=` operator, the left operand is evaluated only once. This makes a difference when that operand has a side effect. Consider the following two expressions, which are not equivalent:

```
a[i++] += 2;
a[i++] = a[i++] + 2;
```

The general form of these combination assignment operators is:

*lvalue op= rvalue*

This is equivalent (unless there are side effects in *lvalue*) to:

*lvalue = lvalue op rvalue*

The available operators are:

```
+=    -=    *=    /=    %=    // Arithmetic operators plus assignment

&=    |=    ^=                // Bitwise operators plus assignment

<<=   >>=   >>>=              // Shift operators plus assignment
```

The most commonly used operators are `+=` and `-=`, although `&=` and `|=` can also be useful when you are working with `boolean` or bitwise flags. For example:

```
i += 2;          // Increment a loop counter by 2
```

```
c -= 5;          // Decrement a counter by 5
flags |= f;      // Set a flag f in an integer set of flags
flags &= ~f;     // Clear a flag f in an integer set of flags
```

## The Conditional Operator

The conditional operator `?:` is a somewhat obscure ternary (three-operand) operator inherited from C. It allows you to embed a conditional within an expression. You can think of it as the operator version of the `if/else` statement. The first and second operands of the conditional operator are separated by a question mark (`?`), while the second and third operands are separated by a colon (:). The first operand must evaluate to a `boolean` value. The second and third operands can be of any type, but they must be convertible to the same type.

The conditional operator starts by evaluating its first operand. If it is `true`, the operator evaluates its second operand and uses that as the value of the expression. On the other hand, if the first operand is `false`, the conditional operator evaluates and returns its third operand. The conditional operator never evaluates both its second and third operand, so be careful when using expressions with side effects with this operator. Examples of this operator are:

```
int max = (x > y) ? x : y;
String name = (value != null) ? value : "unknown";
```

Note that the `?:` operator has lower precedence than all other operators except the assignment operators, so parentheses are not usually necessary around the operands of this operator. Many programmers find conditional expressions easier to read if the first operand is placed within parentheses, however. This is especially true because the conditional `if` statement always has its conditional expression written within parentheses.

## The instanceof Operator

The `instanceof` operator is intimately bound up with objects and the operation of the Java type system. If this is your first look at Java, it may be preferable to skim this definition and return to this section after you have a decent grasp of Java's objects.

`instanceof` requires an object or array value as its left operand and the name of a reference type as its right operand. In its basic form, it evaluates to `true` if the object or array is an *instance* of the specified type; it returns `false` otherwise. If the left operand is `null`, `instanceof` always evaluates to `false`. If an `instanceof` expression evaluates to `true`, it means that you can safely cast and assign the left operand to a variable of the type of the right operand.

The `instanceof` operator can be used only with reference types and objects, not primitive types and values. Examples of `instanceof` are:

```
// True: all strings are instances of String
"string" instanceof String
```

```
// True: strings are also instances of Object, including the empty
// string
"" instanceof Object

// False: null is never an instance of anything
null instanceof String

Object o = new int[] {1,2,3};
o instanceof int[]   // True: the array value is an int array

o instanceof byte[]  // False: the array value is not a byte array
o instanceof Object  // True: all arrays are instances of Object

// Use instanceof to make sure that it is safe to cast an object
if (object instanceof Account) {
  Account a = (Account) object;
}
```

Historically, using `instanceof` was discouraged in favor of other more object-oriented solutions we'll see in Chapter 6. Java's increasing adoption of pattern matching, though, is changing attitudes about this operator. `instanceof` is especially well suited to the common scenarios around receiving data in unpredictable formats through an API. It is often a pragmatic option these days rather than a last resort.

## Pattern Matching and Type Patterns

From Java 17 onward, `instanceof` has an extended form that condenses a common pattern—checking `instanceof` with a cast to the appropriate type within a conditional. This is a simple form of a language feature known as *pattern matching*, and it expresses this by including a variable after the reference type—if `instanceof` sees the type is compatible, the variable is assigned the cast object, like this:

```
if (object instanceof Account a) {
  // variable a is available in this scope
}
```

The `Account a` portion of the `instanceof` is a construct known as a *type pattern*.

A type pattern's most basic form is defined by a type identifier, and a variable that will be populated with the object if it matches the type:

```
TypeName variable
```

If the variable is unused in the following block, it can be excluded with the unnamed variable _. Any number of instances of _ can be used in a given scope without conflict, as their values are inaccessible.

```
TypeName _
```

In addition to working with `instanceof`, type patterns also have some powerful features when combined with `switch` expressions and Java's `record` classes—as we'll see in "Pattern Matching".

## Special Operators

Java has six language constructs (plus one special case) that are sometimes considered operators and sometimes considered simply part of the basic language syntax. These "operators" were included in Table 2-4 to show their precedence relative to the other true operators. The use of these language constructs is detailed elsewhere in this book, but it is described briefly here so that you can recognize them in code examples:

*Member access (`.`)*

An *object* is a collection of data and methods that operate on that data; the data fields and methods of an object are called its *members*. The dot (.) operator accesses these members. If `o` is an expression that evaluates to an object reference (or a class name), and `f` is the name of a field of the class, `o.f` evaluates to the value contained in that field. If `m` is the name of a method, `o.m` refers to that method and allows it to be invoked using the `()` operator shown later.

*Array element access (`[]`)*

An *array* is a numbered list of values. Each element of an array can be referred to by its number, or *index*. The `[ ]` operator allows you to refer to the individual elements of an array. If `a` is an array, and `i` is an expression that evaluates to an `int`, `a[i]` refers to one of the elements of `a`. Unlike other operators that work with integer values, this operator restricts array index values to be of type `int` or narrower.

*Method invocation (`. ()`)*

A *method* is a named collection of Java code that can be run, or *invoked*, by following the name of the method with zero or more comma-separated expressions contained within parentheses. The values of these expressions are the *arguments* to the method. The method processes the arguments and optionally returns a value that becomes the value of the method invocation expression. If `o.m` is a method that expects no arguments, the method can be invoked with `o.m()`. If the method expects three arguments, for example, it can be invoked with an expression such as `o.m(x, y, z)`. `o` is referred to as the *receiver* of the method—if `o` is an object, then it is said to be the *receiver object*. Before the JVM invokes a method, it evaluates each of the arguments to be passed to the method. These expressions are guaranteed to be evaluated in order from left to right (which matters if any of the arguments have side effects).

*Object creation (`new`)*

In Java, objects are created with the `new` operator, which is followed by the type of the

object to be created and a parenthesized list of arguments to be passed to the object *constructor*. A constructor is a special block of code that initializes a newly created object, so the object creation syntax is similar to the Java method invocation syntax. For example:

```
new ArrayList<String>();
new Account("Jason", 0.0, 42);
```

### *Array creation (`new`)*

Arrays are a special case of objects, and they too are created with the `new` operator, with a slightly different, special case syntax. The keyword is followed by the type of the array to be created and the size of the array encased in square brackets. The size of an array is an `int`, so the maximum number of elements of a Java array is `Integer.MAX_VALUE`, or 2,147,483,647:

```
new int[5];
```

In some circumstances, arrays can also be created using the *array literal* syntax:

```
new int[] {1, 2, 3, 4, 5}
```

### *Lambda expression (`->`)*

A *lambda expression* represents the creation of an object with an associated piece of executable Java code (which is essentially a method body). It consists of a method argument list (zero or more comma-separated expressions contained within parentheses) followed by the lambda *arrow* operator followed by a block of Java code. If the block of code comprises just a single statement, then the usual curly braces to denote block boundaries can be omitted. If the lambda takes only a single argument, the parentheses around the argument can be omitted.

### *Type conversion or casting (`()`)*

As we've already seen, parentheses can also be used as an operator to perform narrowing type conversions, or casts. The first operand of this operator is the type to be converted to; it is placed between the parentheses. The second operand is the value to be converted; it follows the parentheses. For example:

```
(byte) 28          // An integer literal cast to a byte type
(int) (x + 3.14f)  // A floating-point sum value cast to an integer
(String)h.get(k)   // A generic object cast to a string
```

## Statements

A *statement* is a basic unit of execution in the Java language—it expresses a single piece of intent by the programmer. Unlike expressions, Java statements do not have a value. Statements also typically contain expressions and operators (especially assignment operators) and are frequently executed for the side effects that they cause.

Many of the statements defined by Java are flow-control statements, such as conditionals and loops, that can alter the default, linear order of execution in well-defined ways. Table 2-5 summarizes the statements defined by Java.

*Table 2-5. Java statements*

| Statement | Purpose | Syntax |
|---|---|---|
| *expression* | side effects | `variable = expr ; expr ++; method (); new Type ( );` |
| *compound* | group statements | `{ statements }` |
| *empty* | do nothing | `;` |
| *labeled* | name a statement | `label : statement` |
| *variable* | declare a variable | `[final] type name [= value ] [, name [= value ]] …;` |
| `if` | conditional | `if ( expr ) statement [ else statement ]` |
| `switch` | conditional | `switch ( expr ) { [ case expr : statements ] … [ default: statements ]}` |
| `switch` | conditional expression | `switch ( expr ) { [ case expr , [ expr …] -> expr ;] … [ default -> expr ;] }` |
| `while` | loop | `while ( expr ) statement` |
| `do` | loop | `do statement while ( expr );` |
| `for` | simplified loop | `for ( init ; test ; increment ) statement` |

| | | |
|---|---|---|
| *foreach* | collection iteration | `for ( variable : iterable ) statement` |
| `break` | exit block | `break [ label ] ;` |
| `continue` | restart loop | `continue [ label ] ;` |
| `return` | end method | `return [ expr ] ;` |
| `synchronized` | critical section | `synchronized ( expr ) { statements }` |
| `throw` | throw exception | `throw expr ;` |
| `try` | handle exception | `try { statements } [ catch ( type name ) { statements } ] … [ finally { statements } ]` |
| `try` | handle exception, closing resources | `try ([ variable = expr ]) { statements } [ catch ( type name ) { statements } ] … [ finally { statements } ]` |
| `assert` | verify invariant | `assert invariant [ error ];` |

## Expression Statements

As we saw earlier in the chapter, certain types of Java expressions have side effects. In other words, they do not only evaluate to some value; they also change the program state in some way. You can use any expression with side effects as a statement simply by following it with a semicolon. The legal types of expression statements are assignments, increments and decrements, method calls, and object creation. For example:

```
a = 1;                              // Assignment
x *= 2;                             // Assignment with operation
i++;                                // Post-increment
--c;                                // Pre-decrement
System.out.println("statement");    // Method invocation
```

## Compound Statements

A *compound statement* is any number and kind of statements grouped together within curly braces. You can use a compound statement anywhere a statement is required by Java syntax:

```
for(int i = 0; i < 10; i++) {
```

```
  a[i]++;           // Body of this loop is a compound statement.
  b[i]--;           // It consists of two expression statements
}                   // within curly braces.
```

## The Empty Statement

An *empty statement* in Java is written as a single semicolon. The empty statement doesn't do anything, but the syntax is occasionally useful. For example, you can use it to indicate an empty loop body in a `for` loop:

```
for(int i = 0; i < 10; a[i++]++)  // Increment array elements
  /* empty */;                    // Loop body is empty statement
```

## Labeled Statements

A *labeled statement* is simply a statement that you have given a name by prepending an identifier and a colon to it. Labels are used by the `break` and `continue` statements. For example:

```
rowLoop: for(int r = 0; r < rows.length; r++) {       // Labeled loop
  colLoop: for(int c = 0; c < columns.length; c++) {  // Another one
    break rowLoop;                                    // Use a label
  }
}
```

Labels are not often required on statements, but some programmers find that targeted use of labels can significantly enhance readability of complex loops.

## Local Variable Declaration Statements

A *local variable*, often simply called a *variable*, is a symbolic name for a location to store a value that is defined within a method or compound statement. All variables must be declared before they can be used; this is done with a variable declaration statement. Because Java is a statically typed language, a variable declaration specifies the type of the variable, and only values of that type can be stored in the variable.

In its simplest form, a variable declaration specifies a variable's type and name:

```
int counter;
String s;
```

A variable declaration can also include an *initializer*, an expression that specifies an initial value for the variable. For example:

```
int i = 0;
String s = readLine();
int[] data = {x+1, x+2, x+3}; // Array initializers are discussed later
```

The Java compiler does not allow you to use a local variable that has not been initialized, so it is

usually convenient to combine variable declaration and initialization into a single statement. The initializer expression need not be a literal value or a constant expression that can be evaluated by the compiler; it can be an arbitrarily complex expression whose value is computed when the program is run.

If a variable has an initializer, then the programmer can use a special syntax to ask the compiler to automatically work out the type, if it is possible to do so:

```
var i = 0;          // type of i inferred as int
var s = readLine(); // type of s inferred as String
```

This can be a useful syntax, but it is potentially harder to read. Our second example, for instance, requires that you know that the return type of `readLine()` is `String` to know what type will be inferred for `s`.

For this reason, throughout the text we only use `var` in examples when the initializer makes the type completely redundant. As you learn the Java language, this may be a reasonable policy to follow while you become familiar with the Java type system.

> **NOTE**
>
> Large language models (LLMs) will tend to produce code that uses explicit types more frequently than the `var` forms. This is partly due to the text corpus of explicitly typed code being much larger, and partly due to the lack of reasoning capability present in LLM-based text generation.

A single variable declaration statement can declare and initialize more than one variable, but all variables must be of the same explicitly declared type. Variable names and optional initializers are separated from each other with commas:

```
int i, j, k;
float x = 1.0f, y = 1.0f;
String question = "Really Quit?", response;
```

Variable declaration statements can begin with the `final` keyword. This modifier specifies that once an initial value is defined for the variable, that value is never allowed to change:

```
final String greeting = getLocalLanguageGreeting();
```

We will have more to say about the `final` keyword later on, especially when talking about the design of classes and the immutable style of programming (as well as the difference between `final` and the idea of a constant present in some other languages).

Local variables also allow the unnamed variable _. This can be useful for keeping code consistent when we may care about some but not all return values from a method:

```
int i = sideEffect(); // we want this value
```

```
int j = sideEffect(); // and this one too
int _ = sideEffect(); // but we don't care here
```

> **TIP**
>
> There isn't a case where _ is required for local variables—any *rvalue* can stand as a statement on its own, so this is only useful if it enhances readability in formatting. We'll see other locations where unnamed variables are more directly useful.

Java variable declaration statements can appear anywhere in Java code; they are not restricted to the beginning of a method or block of code. Local variable declarations can also be integrated with the *initialize* portion of a `for` loop, as we'll discuss shortly.

Local variables can be used only within the method or block of code in which they are defined. This is called their *scope* or *lexical scope*:

```
void method() {           // A method definition
  int i = 0;              // Declare variable i
  while (i < 10) {        // i is in scope here
    int j = 0;            // Declare j; the scope of j begins here
    i++;                  // i is in scope here; increment it
  }                       // j is no longer in scope;
  System.out.println(i);  // i is still in scope here
}                         // The scope of i ends here
```

## The if/else Statement

The `if` statement is a fundamental control statement that allows Java to make decisions, or, more precisely, to execute statements conditionally. The `if` statement has an associated expression and statement. If the expression evaluates to `true`, the JVM executes the statement. If the expression evaluates to `false`, the JVM skips the statement.

> **NOTE**
>
> Java allows the expression to be of the wrapper type `Boolean` instead of the primitive type `boolean`. In this case, the wrapper object is automatically unboxed.

Here is an example `if` statement:

```
if (username == null)     // If username is null,
  username = "John Doe";  // use a default value
```

Although they look extraneous, the parentheses around the expression are a required part of the syntax for the `if` statement, unlike in some languages like Python. As we already saw, a block of statements enclosed in curly braces is itself a statement, so we can write `if` statements that look like this as well:

```
if ((address == null) || (address.equals(""))) {
  address = "[undefined]";
  System.out.println("WARNING: no address specified.");
}
```

An `if` statement can include an optional `else` keyword that is followed by a second statement. In this form of the statement, the expression is evaluated, and, if it is `true`, the first statement is executed. Otherwise, the second statement is executed. For example:

```
if (username != null)
  System.out.println("Hello " + username);
else {
  username = askQuestion("What is your name?");
  System.out.println("Hello " + username + ". Welcome!");
}
```

When you use nested `if/else` statements, some caution is required to ensure that the `else` clause goes with the appropriate `if` statement. Consider the following lines:

```
if (i == j)
  if (j == k)
    System.out.println("i equals k");
else
  System.out.println("i doesn't equal j");    // WRONG!!
```

In this example, the inner `if` statement forms the single statement allowed by the syntax of the outer `if` statement. That seems a reasonable thing to say—but unfortunately, it is not clear which `if` the `else` goes with.

In this example, the indentation hint is wrong—Java is not Python.

The rule the compiler follows is that an `else` clause like this is associated with the nearest `if` statement. Properly indented, this code looks like this:

```
if (i == j)
  if (j == k)
    System.out.println("i equals k");
  else
    System.out.println("i doesn't equal j");    // WRONG!!
```

This is still legal code, but it is probably not what the programmer had in mind.

When working with nested `if` statements, you should use curly braces to make your code easier to read.

Here is a better way to write the code:

```
if (i == j) {
  if (j == k)
    System.out.println("i equals k");
}
else {
```

```
    System.out.println("i doesn't equal j");
  }
```

This sort of confusion prompts a common recommendation (shared by the authors) to always include curly braces with `if` statements for clarity—even with single lines. Although not required by the language, it makes code easier to understand surprisingly often.

### The else if clause

The `if/else` statement is useful for testing a condition and choosing between two statements or blocks of code to execute. But what about when you need to choose between several blocks of code? This is typically done with an `else if` clause, which is not really new syntax but a common idiomatic usage of the standard `if/else` statement. It looks like this:

```
if (n == 1) {
  // Execute code block #1
}
else if (n == 2) {
  // Execute code block #2
}
else if (n == 3) {
  // Execute code block #3
}
else {
  // If all else fails, execute block #4
}
```

There is nothing special about this code. It is just a series of `if` statements, where each `if` is part of the `else` clause of the previous statement. Using the `else if` idiom is preferable to, and more legible than, writing these statements out in their fully nested form:

```
if (n == 1) {
  // Execute code block #1
}
else {
  if (n == 2) {
    // Execute code block #2
  }
  else {
    if (n == 3) {
      // Execute code block #3
    }
    else {
      // If all else fails, execute block #4
    }
  }
}
```

## The switch Statement

An `if` statement causes a branch in the flow of a program's execution. You can use multiple `if` statements, as shown in the previous section, to perform a multiway branch. This is not always

the best solution, however, especially when all of the branches depend on the value of a single variable.

In this case, the repeated `if` statements may seriously hamper readability, especially if the code has been refactored over time or features multiple levels of nested `if`.

A better solution is to use a `switch` construction.

This syntax was originally inherited from the C programming language, and its basic form is not nearly as elegant as other parts of Java. Recent Java versions include an expression variant of `switch` that we'll examine in a moment, and which is often a superior choice. However, that alternative won't erase the long history of `switch` statements in the language (or the large amount of code that uses them), so it's essential to get to grips with it.

> **NOTE**
>
> A `switch` statement starts with an expression whose type is an `int`, `short`, `char`, `byte` (or their wrapper type), `String`, or an enum (see Chapter 5 for more on enumerated types).

This expression is followed by a block of code in curly braces that contains various entry points that correspond to possible values for the expression. For example, the following `switch` statement is equivalent to the repeated `if` and `else/if` statements shown in the previous section:

```
switch(n) {
  case 1:                           // Start here if n == 1
    // Execute code block #1
    break;                          // Stop here
  case 2:                           // Start here if n == 2
    // Execute code block #2
    break;                          // Stop here
  case 3:                           // Start here if n == 3
    // Execute code block #3
    break;                          // Stop here
  default:                          // If all else fails...
    // Execute code block #4
    break;                          // Stop here
}
```

As you can see from the example, the various entry points into a `switch` statement are labeled either with the keyword `case`, followed by an integer value and a colon, or with the special `default` keyword, followed by a colon. When a `switch` statement executes, the JVM computes the value of the expression in parentheses and then looks for a `case` label that matches that value. If it finds one, the JVM starts executing the block of code at the first statement following the `case` label. If it does not find a `case` label with a matching value, the JVM starts execution at the first statement following a special-case `default`: label. Or, if there is no `default`: label, the JVM skips the body of the `switch` statement altogether.

Note the use of the `break` keyword at the end of each `case` in the previous code. The `break` statement is described later in this chapter, but, in this example, it causes the JVM to exit the body of the `switch` statement. The `case` clauses in a `switch` statement specify only the starting point of the desired code. The individual `case`s are not independent blocks of code, and they do not have any implicit ending point.

> **WARNING**
>
> You must explicitly specify the end of each `case` with a `break` or related statement. In the absence of `break` statements, a `switch` statement begins executing code at the first statement after the matching `case` label and continues executing statements until it reaches the end of the block. The control flow will *fall through* into the next `case` label and continue executing, rather than exit the block.[2]

On rare occasions, it is useful to write code like this that falls through from one `case` label to the next, but 99% of the time you should be careful to end every `case` and `default` clause with a statement that causes the `switch` statement to stop executing. Normally you use a `break` statement, but `return` and `throw` also work.

As a consequence of this default fall-through, a `switch` statement can have more than one `case` clause labeling the same statement. Consider the `switch` statement in the following method:

```
boolean parseYesOrNoResponse(char response) {
  switch(response) {
    case 'y':
    case 'Y':
      return true;
    case 'n':
    case 'N':
      return false;
    default:
      throw new IllegalArgumentException("Response must be Y or N");
  }
}
```

The `switch` statement and its `case` labels have some important restrictions. The expression associated with a `switch` statement must be either `byte`, `char`, `short`, `int` (or their wrappers), or a reference type. The floating-point and `boolean` types are not supported, and neither is `long`, even though `long` is an integer type.

Each `case` label must be either a constant value, a constant expression the compiler can evaluate, or a type pattern. A `case` label cannot contain a runtime expression involving variables or method calls.

Constant-valued labels were shown in the preceding example, so let’s look at a type pattern example. These are formed of an expected object type and a variable to assign the value to upon successful matching. This is another application of *pattern matching*, which we’ve already seen

with `instanceof` (see "Pattern Matching and Type Patterns"). The unnamed variable _ is allowed (and, indeed, encouraged) if the value is unused:

```
Object o = "string";
switch(o) {
  case String s:
    System.out.println("String: " + s);
    break;
  case Integer _:
    System.out.println("Some number")
    break;
  default:
    throw new IllegalArgumentException("Unexpected type");
}
```

The `case` label values must be within the range of the data type used for the `switch` expression. If using type patterns, matching must be exhaustive, which we'll discuss at more length in the section on `switch` expressions (subtly different from `switch` *statements*).

Finally, it is not legal to have two or more `case` labels with the same value or more than one `default` label.

With all those caveats in play, let's look at how the new `switch` expression makes for a cleaner experience.

## The switch Expression

A frequent problem with the classic `switch` statement arises when capturing a variable value:

```
Boolean yesOrNo = null;
switch(input) {
  case "y":
  case "Y":
    yesOrNo = true;
    break;
  case "n":
  case "N":
    yesOrNo = false;
    break;
  default:
    throw new IllegalArgumentException("Response must be Y or N");
}
```

For the variable to be available after the `switch`, it must be declared outside the statement and provided an initial value. Then each `case` must be certain to set the variable. However, we have no guarantees, and in code with more branches than this simple example, it's an easy thing to miss and introduce a bug.

The `switch` expression is explicitly designed to address these and other faults. The first LTS version that supports switch expressions is Java 17—it's not available in 8 or 11. As the name calls out, it's an *expression*—one of the more syntactically complex ones in the language—and

as such results in a value:

```
boolean yesOrNo = switch(input) {
  case "y" -> true;
  case "Y" -> true;
  case "N" -> false;
  case "n" -> false;
  default -> throw new IllegalArgumentException("Y or N");
};
```

Much like the `switch` statement, each `case` here evaluates the input against its value. After an `->`, you provide the resulting value for the switch expression as a whole. In this example, we're assigning that to our variable `yesOrNo`, which no longer needs to be the nullable wrapper type.

Our code as written here hides one of the protections that the `switch` expressions is giving us. If we remove the `default` clause, the compiler will give us an error because the expression is not *exhaustive*—that is, it does not cover all possible values. This requirement to explicitly cover all options (or provide a `default` fallback) is a nice little safety feature so programmers don't miss critical cases:

```
boolean yesOrNo = switch(input) {
  case "y" -> true;
  case "Y" -> true;
  case "N" -> false;
  case "n" -> false;
};

// Compiler error:
//   the switch expression does not cover all possible input values
```

Java 21 introduced `case null`, which was necessary now that a `switch` can take a reference type as an input.

> **WARNING**
>
> For backward compatibility, the `default` label will not match on `null` but instead raises a `NullPointerException`. Changing this ran the risk of altering the behavior of existing `switch` expressions, something the Java platform makes serious efforts to avoid.

Switch expressions do not fall through like the statement form. To support multiple values evaluating to the same result, each `case` can take a comma-separated list of values instead of just a single value:

```
boolean yesOrNo = switch(input) {
  case "y", "Y" -> true;
  case "n", "N" -> false;
  default -> throw new IllegalArgumentException("Y or N");
};
```

Like `switch` statements, we can use a type pattern instead of a constant value in our `switch` expressions. Note that constants and type patterns may not be mixed in a single `switch` of either flavor:

```
Object o = "string";
var message = switch(o) {
  case String s -> "Hello String";
  case Integer i -> "Count to " + i;
  default -> "Just some object";
};
```

Our desired result can't always be expressed as a single value or method call. To support that, curly braces may introduce a statement. However, the statement must end with either `yield` to exit the switch with a value, a `return` leaving the entire enclosing method, or a `throw` with an exception:[3]

```
boolean yesOrNo = switch(input) {
  case "y", "Y" -> { System.out.println("Yup"); yield true; }
  case "n", "N" -> { System.out.println("Nope"); yield false; }
  case "escape" -> return; // Assumes containing method is void
  default -> throw new IllegalArgumentException("Y or N");
};
```

If we don't make use of the result from our `switch` expression, we can drop the curly braces while avoiding unexpected side effects and fall-through:

```
switch(input) {
  case "y", "Y" -> System.out.println("Sure");
  case "n", "N" -> System.out.println("Nope");
  default -> throw new IllegalArgumentException("Y or N");
}
```

## The while Statement

The `while` statement is a basic statement that allows Java to perform repetitive actions—or, to put it another way, it is one of Java's primary *looping constructs*. It has the following syntax:

```
while (expression)
  statement
```

The `while` statement works by first evaluating the *`expression`*, which must result in a `boolean` or `Boolean` value. If the value is `false`, the JVM skips the *`statement`* associated with the loop and moves to the next statement in the program. If it is `true`, however, the *`statement`* that forms the body of the loop is executed, and the *`expression`* is reevaluated. Again, if the value of *`expression`* is `false`, the JVM moves on to the next statement in the program; otherwise, it executes the *`statement`* again. This cycle continues while the *`expression`* remains `true` (i.e., until it evaluates to `false`), at which point the

`while` statement ends, and the JVM moves on to the next statement. You can create an infinite loop with the syntax `while(true)`.

Here is an example `while` loop that prints the numbers 0 to 9:

```
int count = 0;
while (count < 10) {
   System.out.println(count);
   count++;
}
```

As you can see, the variable `count` starts off at 0 in this example and is incremented each time the body of the loop runs. Once the loop has executed 10 times, the expression becomes `false` (i.e., `count` is no longer less than 10), the `while` statement finishes, and the Java JVM can move to the next statement in the program. Most loops have a counter variable like `count`. The variable names `i`, `j`, and `k` are commonly used as loop counters, although you should use more descriptive names if it makes your code easier to understand.

## The do Statement

A `do` loop is much like a `while` loop, except that the loop expression is tested at the bottom of the loop rather than at the top. This means that the body of the loop is always executed at least once. The syntax is:

```
do
   statement
while (expression);
```

Notice a couple of differences between the `do` loop and the more ordinary `while` loop. First, the `do` loop requires both the `do` keyword to mark the beginning of the loop and the `while` keyword to mark the end and introduce the loop condition. Also, unlike the `while` loop, the `do` loop is terminated with a semicolon. This is because the `do` loop ends with the loop condition rather than simply ending with a curly brace that marks the end of the loop body. The following `do` loop prints the same output as the `while` loop just discussed:

```
int count = 0;
do {
   System.out.println(count);
   count++;
} while(count < 10);
```

The `do` loop is much less commonly used than its `while` cousin because, in practice, it is unusual to encounter a situation where you are sure you always want a loop to execute at least once.

## The for Statement

The `for` statement provides a looping construct that is often more convenient than the `while` and `do` loops. The `for` statement takes advantage of a common looping pattern. Most loops have a counter, or state variable of some kind, that is initialized before the loop starts, tested to determine whether to execute the loop body, and then incremented or updated somehow at the end of the loop body before the test expression is evaluated again. The *`initialize`*, *`test`*, and *`update`* steps are the three crucial manipulations of a loop variable, and the `for` statement makes these three steps an explicit part of the loop syntax:

```
for(initialize; test; update) {
    statement
}
```

This `for` loop is basically equivalent to the following `while` loop:

```
initialize;
while (test) {
   statement;
   update;
}
```

Placing the *`initialize`*, *`test`*, and *`update`* expressions at the top of a `for` loop makes it especially easy to understand what the loop is doing, and it prevents mistakes such as forgetting to initialize or update the loop variable. The JVM discards the values of the *`initialize`* and *`update`* expressions, so to be useful, these expressions must have side effects. *`initialize`* is typically an assignment expression, while *`update`* is usually an increment, decrement, or some other assignment.

The following `for` loop prints the numbers 0 to 9, just as the previous `while` and `do` loops have done:

```
int count;
for(count = 0 ; count < 10 ; count++)
   System.out.println(count);
```

Notice how this syntax places all the important information about the loop variable on a single line, making it very clear how the loop executes. Placing the *`update`* expression in the `for` statement itself also simplifies the body of the loop to a single statement; we don't even need to use curly braces to produce a statement block.

The `for` loop supports some additional syntax that makes it even more convenient to use. Because many loops use their loop variables only within the loop, the `for` loop allows the *`initialize`* expression to be a full variable declaration so that the variable is scoped to the body of the loop and is not visible outside of it. For example:

```
for(int count = 0 ; count < 10 ; count++)
  System.out.println(count);
```

Furthermore, the `for` loop syntax does not restrict you to writing loops that use only a single variable. Both the *`initialize`* and *`update`* expressions of a `for` loop can use a comma to separate multiple initializations and update expressions. For example:

```
for(int i = 0, j = 10 ; i < 10 ; i++, j--)
  sum += i * j;
```

Even though all the examples so far have counted numbers, `for` loops are not restricted to loops that count numbers. For example, you might use a `for` loop to iterate through the elements of a linked list:

```
for(Node n = listHead; n != null; n = n.nextNode())
  process(n);
```

The *`initialize`*, *`test`*, and *`update`* expressions of a `for` loop are all optional; only the semicolons that separate the expressions are required. If the *`test`* expression is omitted, it is assumed to be `true`. Thus, you can write an infinite loop as `for(;;)`.

## The foreach Variant

Java's `for` loop works well for primitive types, but it is needlessly clunky for handling collections of objects. Instead, an alternative syntax known as a *foreach* loop is used for handling collections of objects that need to be looped over.

> **NOTE**
>
> Despite its name, the foreach loop does not have a keyword foreach; instead, it is common to read the colon as "in"—as in "for each `name` in `studentNames`."

The foreach loop uses the keyword `for` followed by an opening parenthesis, a variable declaration (without initializer), a colon, an expression, a closing parenthesis, and finally the statement (or block) that forms the body of the loop:

```
for( declaration : expression )
     statement
```

For the `while`, `do`, and `for` loops, we've shown an example that prints 10 numbers. The foreach loop can do this too, but it needs a collection to iterate over. In order to loop 10 times (to print out 10 numbers), we need an array or other collection with 10 elements. Here's code we can use:

```
// Prime numbers to print with a loop
int[] primes = new int[] { 2, 3, 5, 7, 11, 13, 17, 19, 23, 29 };
for(int n : primes)
  System.out.println(n);
```

### What foreach cannot do

The foreach is different from the `while`, `for`, or `do` loops, because it hides the loop counter or `Iterator` from you. This is a very powerful idea, as we'll see when we discuss lambda expressions, but there are some algorithms that cannot be expressed very naturally with a foreach loop.

For example, suppose you want to print the elements of an array as a comma-separated list. To do this, you need to print a comma after every element of the array except the last, or equivalently, before every element of the array except the first. With a traditional `for` loop, the code might look like this:

```
for(int i = 0; i < words.length; i++) {
  if (i > 0)
    System.out.print(", ");
  System.out.print(words[i]);
}
```

This is a very straightforward task, but you simply cannot do it with foreach without keeping track of additional state. The problem is that the foreach loop doesn't give you a loop counter or any other way to tell if you're on the first iteration, the last iteration, or somewhere in between.

> **NOTE**
>
> A similar issue exists when you're using foreach to iterate through the elements of a collection. Just as a foreach loop over an array has no way to obtain the array index of the current element, a foreach loop over a collection has no way to obtain the `Iterator` object that is being used to itemize the elements of the collection.

Here are some other things you cannot do with a foreach-style loop:

- Iterate backward through the elements of an array or `List`
- Use a single loop counter to access the same-numbered elements of two distinct arrays
- Iterate through the elements of a `List` using calls to its `get()` method rather than calls to its iterator

## The break Statement

A `break` statement causes the JVM to skip immediately to the end of a containing statement. We have already seen the `break` statement used with the `switch` statement. The `break` statement is most often written as simply the keyword `break` followed by a semicolon:

```
break;
```

When used in this form, it causes the JVM to immediately exit the innermost containing `while`, `do`, `for`, or `switch` statement. For example:

```
int index = -1;  // -1 indicates not found
for(int i = ....)
for(int i = 0; i < data.length; i++) {
  if (data[i] == target) {  // When we find what we're looking for,
    index = i;              // remember where we found it
    break;                  // and stop looking!
  }
}   // The JVM goes here after executing break
```

The `break` statement can also be followed by the name of a containing labeled statement. When used in this form, `break` causes the Java JVM to immediately exit the named block, which can be any kind of statement, not just a loop or `switch`. For example:

```
TESTFORNULL: if (data != null) {
  for(int row = 0; row < numrows; row++) {
    for(int col = 0; col < numcols; col++) {
      if (data[row][col] == null) {
        break TESTFORNULL;          // treat the array as undefined.
      }
    }
  }
}  // Execution goes here after executing break TESTFORNULL
```

## The continue Statement

While a `break` statement exits a loop, a `continue` statement quits the current iteration of a loop and starts the next one. `continue`, in both its unlabeled and labeled forms, can be used only within a `while`, `do`, or `for` loop. When used without a label, `continue` causes the innermost loop to start a new iteration. For example:

```
for(int i = 0; i < data.length; i++) {  // Loop through data.
  if (data[i] == -1) {                  // If a data value is missing,
    continue;                           // skip to the next iteration.
  }
  process(data[i]);                     // Process the data value.
}
```

When used with a label that is the name of a containing loop, it causes the named loop to start a new iteration. For example, let's suppose that we want to be sure that each row of a matrix has at least one non-`null` entry. We can use a labeled `continue` with a nested `for` loop:

```
EACHROW: for(int row = 0; row < numrows; row++) {
  for(int col = 0; col < numcols; col++) {
    if (data[row][col] != null) {
      continue EACHROW;
    }
  }
}
```

`while`, `do`, and `for` loops differ slightly in the way that `continue` starts a new iteration:

- With a `while` loop, the JVM simply returns to the top of the loop, tests the loop condition again, and, if it evaluates to `true`, executes the body of the loop again.
- With a `do` loop, the JVM jumps to the bottom of the loop, where it tests the loop condition to decide whether to perform another iteration of the loop.
- With a `for` loop, the JVM jumps to the top of the loop, where it first evaluates the *`update`* expression and then evaluates the *`test`* expression to decide whether to loop again. As you can see from the examples, the behavior of a `for` loop with a `continue` statement is different from the behavior of the "basically equivalent" `while` loop presented earlier; *`update`* gets evaluated in the `for` loop but not in the equivalent `while` loop.

## The return Statement

A `return` statement tells the JVM to stop executing the current method. If the method is declared to return a value, the `return` statement must be followed by an expression matching the method's return type. The value of the expression becomes the return value of the method. For example, the following method computes and returns the square of a number:

```
double square(double x) { // A method to compute x squared
  return x * x;           // Compute and return a value
}
```

Some methods are declared `void` to indicate that they do not return any value. The JVM runs methods like this by executing their statements one by one until it reaches the end of the method. After executing the last statement, the JVM returns implicitly. Sometimes, however, a `void` method has to return explicitly before reaching the last statement. In this case, it can use the `return` statement by itself, without any expression. For example, the following method prints, but does not return, the square root of its argument. If the argument is a negative number, it returns without printing anything:

```
void printSquareRoot(double x) {
  if (x < 0)
    return;                          // If x is negative, return
  System.out.println(Math.sqrt(x)); // Print the square root of x
}                                    // Method end: return implicitly
```

## The synchronized Statement

Java has always provided support for multithreaded programming. We cover this in some detail later on (especially in Chapter 12); however, be aware that concurrency is difficult to get right and has a number of subtleties.

In particular, when working with multiple threads, you must often take care to prevent multiple threads from modifying an object simultaneously in a way that might corrupt the object's state. Java provides the `synchronized` statement to help the programmer prevent corruption. The

syntax is:

```
synchronized ( expression ) {
   statements
}
```

*`expression`* is an expression that must evaluate to an object (including arrays). *`statements`* constitute the code of the section that could cause damage and must be enclosed in curly braces.

> **NOTE**
>
> In Java, the protection of object state (i.e., data) is the primary concern of the concurrency primitives. This is unlike some other languages, where the exclusion of threads from *critical sections* (i.e., code) is the main focus.

Before executing the statement block, the JVM first obtains an exclusive lock on the object or array specified by *`expression`*. It holds the lock until it is finished running the block, then releases it. While a thread holds the lock on an object, no other thread can obtain that lock.

As well as the block form, `synchronized` can also be used as a method modifier in Java. When applied to a method, the keyword indicates that the entire method is treated as `synchronized`.

For a `synchronized` instance method, Java obtains an exclusive lock on the class instance. (Class and instance methods are discussed in Chapter 4.) It can be thought of as a `synchronized (this) { ... }` block that covers the entire method.

A `static synchronized` method (a class method) causes Java to obtain an exclusive lock on the class (technically the class object corresponding to the type) before executing the method.

## The throw Statement

An *exception* is a signal that indicates some sort of exceptional condition or error has occurred. To *throw* an exception is to signal an exceptional condition. To *catch* an exception is to *handle* it —to take whatever actions are necessary to recover from it (and Java provides some special syntax for defining handlers, as we will see).

In Java, the `throw` statement is used to throw an exception:

```
throw expression;
```

The *`expression`* must evaluate to an `Exception` object that describes the exception or error that has occurred. We'll talk more about types of exceptions shortly; for now, all you need to know is that an exception:

- Is an object
- Has a type that is a subclass of `Exception`

- Has a slightly specialized role in Java's syntax
- Can be of two different types: *checked* or *unchecked*

Here is some example code that throws an exception:

```
public static double factorial(int x) {
  if (x < 0)
    throw new IllegalArgumentException("x must be >= 0");
  double fact;
  for(fact = 1.0; x > 1; fact *= x, x--)
    /* empty */ ;          // Note use of the empty statement
  return fact;
}
```

When the JVM executes a `throw` statement, it immediately stops normal program execution and starts looking for an exception handler that can catch, or handle, the exception. Exception handlers are written with the `try/catch/finally` statement, which is described in the next section.

The JVM first looks at the enclosing block of code to see if it has an associated exception handler. If so, it exits that block of code and starts running the exception-handling code associated with the block. After running the exception handler, the JVM continues execution at the statement immediately following the handler code.

If the enclosing block of code does not have an appropriate exception handler, the JVM checks the next higher enclosing block of code in the method. This continues until a handler is found. If the method does not contain an exception handler that can handle the exception thrown by the `throw` statement, the JVM stops running the current method and returns to the caller. Now the JVM starts looking for an exception handler in the blocks of code of the calling method.

In this way, exceptions propagate up through the execution structure of Java methods (or up the *call stack* of the JVM). If the exception is never caught, it propagates all the way up to the `main()` method of the program. If it is not handled in that method, the JVM prints an error message, prints a *stack trace* to indicate where the exception occurred, and then exits the program.

## The try/catch/finally Statement

Java has two slightly different exception-handling mechanisms. The classic form is the `try/catch/finally` statement. The `try` clause of this statement establishes a block of code for exception handling. This `try` block is followed by zero or more `catch` clauses, each of which is a block of statements designed to handle specific exceptions. Each `catch` block can handle multiple different exceptions—to indicate that a `catch` block should handle multiple exceptions, we use the `|` symbol to separate the different exceptions a `catch` block should handle. The `catch` clauses are followed by an optional `finally` block that contains cleanup code guaranteed to be executed regardless of what happens in the `try` block.

### TRY BLOCK SYNTAX

Both the `catch` and `finally` clauses are optional, but every `try` block must either declare some automatically managed resources (the `try`-with-resources construct we'll see in "The try-with-resources Statement") or be accompanied by a `catch`, a `finally`, or both. The `try`, `catch`, and `finally` blocks all begin and end with curly braces. These are a required part of the syntax and cannot be omitted, even if the clause contains only a single statement.

The following code illustrates the syntax and purpose of the `try/catch/finally` statement:

```
try {
   // Normally this code runs from the top of the block to the bottom
   // without problems. But it can sometimes throw an exception,
   // either directly with a throw statement or indirectly by calling
   // a method that throws an exception.
}
catch (SomeException e1) {
   // This block contains statements that handle an exception object
   // of type SomeException or a subclass of that type. Statements in
   // this block can refer to that exception object by the name e1.
}
catch (AnotherException | YetAnotherException e2) {
   // This block contains statements that handle an exception of
   // type AnotherException or YetAnotherException, or a subclass of
   // either of those types. Statements in this block refer to the
   // exception object they receive by the name e2.
}
finally {
   // This block contains statements that are always executed
   // after we leave the try clause, regardless of whether we leave it:
   //   1) normally, after reaching the bottom of the block;
   //   2) because of a break, continue, or return statement;
   //   3) with an exception that is handled by a catch clause above;
   //   4) with an uncaught exception that has not been handled.
   // If the try clause calls System.exit(), however, the JVM
   // exits before the finally clause can be run.
}
```

#### try

The `try` clause simply establishes a block of code that either has its exceptions handled or needs special cleanup code to be run when it completes or terminates for any reason (including an *abnormal termination*). The `try` clause by itself doesn't do anything interesting; it is the `catch` and `finally` clauses that do the exception-handling and cleanup operations.

#### catch

A `try` block can be followed by zero or more `catch` clauses that specify code to handle various types of exceptions. Each `catch` clause is declared with a single argument that specifies the types of exceptions the clause can handle (possibly using the special `|` syntax to indicate that

the `catch` block can handle more than one type of exception) and also provides a name the clause can use to refer to the exception object it is currently handling. _ can be used for the name of the exception if the value is unused in the handler. Any type that a `catch` block wishes to handle must be some subclass of `Throwable`.

When an exception is thrown, the JVM looks for a `catch` clause with an argument that matches the same type as the exception object or a superclass of that type. The JVM invokes the first such `catch` clause it finds. The code within a `catch` block should take whatever action is necessary to cope with the exceptional condition. If the exception is a `java.io.FileNotFoundException`, for example, you might handle it by asking the user to check their spelling and try again.

It is not required to have a `catch` clause for every possible exception; in some cases, the correct response is to allow the exception to propagate up and be caught by the invoking method. In other cases, such as a programming error signaled by `NullPointerException`, the correct response is probably not to catch the exception at all but to allow it to propagate and have the JVM exit with a stack trace and an error message.

### finally

The `finally` clause is generally used to clean up after the code in the `try` clause (e.g., close files and shut down network connections). The `finally` clause is useful because it is guaranteed to be executed if any portion of the `try` block is executed, regardless of how the code in the `try` block completes. In fact, the only way a `try` clause can exit without allowing the `finally` clause to be executed is by invoking the `System.exit()` method, which causes the JVM to stop running.

In the normal case, control reaches the end of the `try` block and then proceeds to the `finally` block, which performs any necessary cleanup. If control leaves the `try` block because of a `return`, `continue`, or `break` statement, the `finally` block is executed before control transfers to its new destination.

If an exception occurs in the `try` block and there is an associated `catch` block to handle the exception, control transfers first to the `catch` block and then to the `finally` block. If there is no local `catch` block to handle the exception, control transfers first to the `finally` block and then propagates up to the nearest containing `catch` clause that can handle the exception.

#### CONTROL TRANSFER WITH FINALLY

If a `finally` block itself transfers control with a `return`, `continue`, `break`, or `throw` statement or by calling a method that throws an exception, the currently executing control flow is abandoned, and this new control transfer is processed.

This means that if a `finally` clause throws an exception, that exception replaces any exception that was in the process of being thrown, as this example shows:

```
try {
  throw new UnsupportedOperationException("Not implemented");
} finally {
  throw new RuntimeException("Finally sends its regards");
}
```

If a `finally` clause issues a `return` statement, the method returns normally, even if an exception has been thrown and has not yet been handled.

`try` and `finally` can be used together without exceptions or any `catch` clauses. In this case, the `finally` block is simply cleanup code that is guaranteed to be executed, regardless of any `break`, `continue`, or `return` statements within the `try` clause.

## The try-with-resources Statement

The standard form of a `try` block is very general, but there is a common set of circumstances that requires developers to be very careful when writing `catch` and `finally` blocks. These circumstances are when operating with resources that need to be cleaned up or closed when they are no longer needed.

Java provides a very useful mechanism for automatically closing resources that require cleanup. This is known as `try`-with-resources, or TWR. We discuss TWR in detail in "try-with-resources Revisited", but for completeness, let's introduce the syntax now. The following example shows how to open a file using the `FileInputStream` class (which results in an object that will require cleanup):

```
try (InputStream is = new FileInputStream("/Users/ben/details.txt")) {
  // ... process the file
}
```

This new form of `try` takes parameters that are all objects that require cleanup.[4] These objects are scoped to this `try` block and are then cleaned up automatically no matter how this block is exited. The developer does not need to write any `catch` or `finally` blocks—the Java compiler automatically inserts correct cleanup code.

All new code that deals with resources should be written in the TWR style—it is considerably less error prone than manually writing `catch` blocks and does not suffer from the problems that plague techniques such as finalization.

## The assert Statement

An `assert` statement is an attempt to provide a capability to verify design assumptions in Java code. An *assertion* consists of the `assert` keyword followed by a boolean expression that the programmer believes should always evaluate to `true`. By default, assertions are not enabled, and the `assert` statement does not actually do anything.

It is possible to enable assertions as a debugging tool, however; when this is done, the `assert`

statement evaluates the expression. If it is indeed `true`, `assert` does nothing. On the other hand, if the expression evaluates to `false`, the assertion fails, and the `assert` statement throws a `java.lang.AssertionError`.

> **TIP**
>
> Outside of the core JDK libraries, the `assert` statement is *extremely* rarely used. It turns out to be too inflexible for testing most applications and is not often used by ordinary developers. Instead, developers use ordinary testing libraries, such as JUnit.

The `assert` statement may include an optional second expression, separated from the first by a colon. When assertions are enabled and the first expression evaluates to `false`, the value of the second expression is taken as an error code or error message and is passed to the `AssertionError()` constructor. The full syntax of the statement is:

```
assert assertion;
```

or:

```
assert assertion : errorcode;
```

To use assertions effectively, you must also be aware of a couple of fine points. First, remember that your programs will normally run with assertions disabled and only sometimes with assertions enabled. This means that you should be careful not to write assertion expressions that contain side effects.

> **WARNING**
>
> You should never throw `AssertionError` from your own code, as it may have unexpected results in future versions of the platform.

If an `AssertionError` is thrown, it indicates that one of the programmer's assumptions has not held up. This means that the code is being used outside of the parameters for which it was designed, and it cannot be expected to work correctly. In short, there is no plausible way to recover from an `AssertionError`, and you should not attempt to catch it (unless you catch it at the top level simply so that you can display the error in a more user-friendly fashion).

### Enabling assertions

For efficiency, it does not make sense to test assertions each time code is executed—`assert` statements encode assumptions that should always be true. Thus, by default, assertions are disabled, and `assert` statements have no effect. The assertion code remains compiled in the class files, however, so it can always be enabled for diagnostic or debugging purposes. You can

enable assertions, either across the board or selectively, with command-line arguments to the JVM.

To enable assertions in all classes except for system classes, use the `-ea` argument. To enable assertions in system classes, use `-esa`. To enable assertions within a specific class, use `-ea` followed by a colon and the class name:

```
java -ea:com.example.sorters.MergeSort com.example.sorters.Test
```

To enable assertions for all classes in a package and in all of its subpackages, follow the `-ea` argument with a colon, the package name, and three dots:

```
java -ea:com.example.sorters... com.example.sorters.Test
```

You can disable assertions in the same way, using the `-da` argument. For example, to enable assertions throughout a package and then disable them in a specific class or subpackage, use:

```
java -ea:com.example.sorters... -da:com.example.sorters.QuickSort
java -ea:com.example.sorters... -da:com.example.sorters.plugins..
```

Finally, it is possible to control whether or not assertions are enabled or disabled at classloading time. If you use a custom classloader (see Chapter 13 for details on custom classloading) in your program and want to turn on assertions, you may be interested in these methods.

# Methods

A *method* is a named sequence of Java statements that can be invoked by other Java code. When a method is invoked, it is passed zero or more values known as *arguments*. The method performs some computations and, optionally, returns a value.

As described earlier in "Expressions and Operators", a method invocation is an expression that is evaluated by the JVM. Because method invocations can have side effects, however, they also can be used as expression statements. This section does not discuss method invocation (or method calling) but instead describes how to define methods.

## Defining Methods

You already know how to define the body of a method; it is simply an arbitrary sequence of statements enclosed within curly braces. What is more interesting about a method is its *signature*.[5] The signature specifies:

- The name of the method
- The number, order, type, and name of the parameters used by the method
- The type of the value returned by the method

- The checked exceptions that the method can throw (the signature may also list unchecked exceptions, but these are not required)
- Various method modifiers that provide additional information about the method

A method signature defines everything you need to know about a method before calling it. It is the method *specification* and defines the API for the method. To use the Java platform's online API reference, you need to know how to read a method signature. And, to write Java programs, you need to know how to define your own methods, each of which begins with a method signature.

A method signature looks like this:

```
[ modifiers ] type name (paramlist) [ throws exceptions ]
```

The signature (the method specification) is followed by the method body (the method implementation), which is simply a sequence of Java statements enclosed in curly braces. If the method is *abstract* (see Chapter 4), the implementation is omitted, and the method body is replaced with a single semicolon.

The signature of a method may also include type variable declarations—such methods are known as *generic methods*. Generic methods and type variables are discussed in Chapter 5.

> **TIP**
>
> Annotations are an interesting special case (see Chapter 5 for more on annotations)—they can be thought of as halfway between a method modifier and additional supplementary type information.

Here are some example method definitions, which begin with the signature and are followed by the method body:

```
// This method is passed an array of strings and has no return value.
public static void main(String[] args) {
  if (args.length > 0) System.out.println("Hello " + args[0]);
  else System.out.println("Hello world");
}

// This method is passed two double arguments and returns a double.
static double distanceFromOrigin(double x, double y) {
  return Math.sqrt(x * x + y * y);
}

// This method is abstract, which means it has no body.
// Note that it may throw exceptions when invoked.
protected abstract String readText(File f, String encoding)
  throws FileNotFoundException, UnsupportedEncodingException;
```

The `main()` method is known as an *entry point*, and execution always starts from an entry point. All executable Java programs have a method with this name, with a few different

permissible signatures (see Chapter 3 for more details on exactly how entry points work).

*`modifiers`* are zero or more special modifier keywords, separated from one another by spaces. A method might be declared with the `public` and `static` modifiers, for example. The allowed modifiers and their meanings are described in the next section.

The *`type`* in a method signature specifies the return type of the method. If the method does not return a value, *`type`* must be `void`. If a method is declared with a non-`void` return type, it must include a `return` statement that returns a value of or convertible to the declared type.

A *constructor* is a block of code, similar to a method, that is used to initialize newly created objects. As we'll see in Chapter 4, constructors are defined in a very similar way to methods, except that their signatures do not include this *`type`* specification and must be named the same as the class.

The *`name`* of a method follows the specification of its modifiers and type. Method names, like variable names, are Java identifiers and, like all Java identifiers, may contain letters in any language represented by the Unicode character set. It is legal, and often quite useful, to define more than one method with the same name, as long as each version of the method has a different parameter list. Defining multiple methods with the same name is called *method overloading*.

For example, the `System.out.println()` method we've seen already is an overloaded method. One method by this name prints a string, and other methods by the same name print the values of the various primitive types. The Java compiler decides which method to call based on the type of the argument passed to the method.

> **TIP**
>
> Unlike some other languages, notably JavaScript, Java does not have anonymous methods. Instead, Java 8 introduced lambda expressions, which are similar to anonymous methods but which the Java runtime automatically converts to an expression that creates an object with a suitable method—see "Lambda Expressions" for more details.

When you are defining a method, the name of the method is always followed by the method's parameter list, which must be enclosed in parentheses. The parameter list defines zero or more arguments that are passed to the method. The parameter specifications, if there are any, each consist of a type and a name, and the specifications are separated from one another by commas (if there are multiple parameters). When a method is invoked, the argument values it is passed must match the number, type, and order of the parameters specified in this method signature line. The values passed need not have exactly the same type as specified in the signature, but they must be convertible to those types without casting.

> **NOTE**
>
> When a Java method expects no arguments, its parameter list is simply `()`, not `(void)`. Java does not regard `void` as a type—C and C++ programmers in particular should pay heed.

Java allows the programmer to define and invoke methods that accept a variable number of arguments, using a syntax known colloquially as *varargs*. Varargs are covered in detail later in this chapter.

The final part of a method signature is the `throws` clause, which is used to list the *checked exceptions* that a method can throw. Checked exceptions are a category of exception classes that must be listed in the `throws` clauses of methods that can throw them.

If a method uses the `throw` statement to throw a checked exception, the method must declare that it can throw that exception. The method must also declare that it can throw an exception if it calls some other method that throws a checked exception and the calling method does not explicitly catch that exception.

A method declares that it can throw one or more checked exceptions by placing the `throws` keyword after the argument list and following it by the name of the exception class or classes it can throw. If a method does not throw any checked exceptions, it does not use the `throws` keyword. If a method throws more than one type of checked exception, separate the names of the exception classes from one another with commas.

## Method Modifiers

The modifiers of a method consist of zero or more modifier keywords from the following list:

`abstract`

> An `abstract` method is a specification without an implementation. The curly braces and Java statements that would normally constitute the body of the method are replaced with a single semicolon, and it is a syntax error to try to declare a method body for an abstract method. A class that includes an `abstract` method must itself be declared `abstract`. Such a class is incomplete and cannot be instantiated (see Chapter 4).

`default`

> A `default` method may be defined only on an interface. All classes implementing the interface receive the default method unless they override it directly. Implementing interfaces in classes is explored thoroughly in Chapter 4.

`final`

> A `final` method may not be overridden or hidden by a subclass, which makes it amenable to compiler optimizations that are not possible for regular methods. All `private` methods are implicitly `final`, as are all methods of any class that is declared `final`.

`native`

> The `native` modifier specifies that the method implementation is written in some "native" language such as C and is provided externally to the Java program, using the Java Native Interface (JNI). Like `abstract` methods, `native` methods have no body: the curly braces

are replaced with a semicolon.

### IMPLEMENTING NATIVE METHODS

When Java was first released, `native` methods were sometimes used for efficiency reasons. That is almost never necessary today. Instead, `native` methods are used to interface Java code to existing libraries written in languages with C-compatible calling conventions (which includes C and C++ and also FORTRAN and Rust). `native` methods are implicitly platform-dependent, and the procedure for linking the implementation with the Java class that declares the method is dependent on the implementation of the JVM. `native` methods are not covered in this book, but more information can be found in the Java Native Interface Specification.

As of Java 25, a potentially superior option is to use the Java Foreign Function and Memory API (FFM) instead of JNI. However, a discussion of FFM is also beyond the scope of this book—but it is useful to know that it exists as a potential alternative even though it is not used for the implementation of `native` methods directly.

`public`, `protected`, `private`

These access modifiers specify whether and where a method can be used outside of the class that defines it. These very important modifiers are explained in Chapter 4.

`static`

A method declared `static` is a *class method* associated with the class itself rather than with an instance of the class (we cover this in more detail in Chapter 4).

`strictfp`

This rarely seen keyword is a no-op from Java 17 onward. For performance reasons, in Java 1.2 the language allowed for subtle deviation from the strict IEEE 754 standard when using certain floating-point acceleration hardware, but these considerations haven't been relevant for many years.

`synchronized`

The `synchronized` modifier makes a method threadsafe. Before a thread can invoke a `synchronized` method, it must obtain a lock on the method's class (for `static` methods) or on the relevant instance of the class (for non-`static` methods). This prevents two threads from executing the method at the same time.

The `synchronized` modifier is an implementation detail and thus is not part of the method specification or API. Good documentation specifies explicitly whether a method is threadsafe; you should not rely on the presence or absence of the `synchronized` keyword when working with multithreaded programs, for reasons we will discuss fully in Chapter 12.

## Checked and Unchecked Exceptions

The Java exception-handling scheme distinguishes between two types of exceptions, known as *checked* and *unchecked* exceptions.

The distinction between checked and unchecked exceptions has to do with the circumstances under which the exceptions could be thrown. Checked exceptions arise in specific, well-defined circumstances and very often are conditions from which the application may be able to partially or fully recover.

For example, consider some code that might find its configuration file in one of several possible directories. If we attempt to open the file from a directory it isn't present in, then a `FileNotFoundException` will be thrown. In our example, we want to catch this exception and move on to try the next possible location for the file. In other words, although the file not being present is an exceptional condition, it is one from which we can recover, and it is an understood and anticipated failure.

On the other hand, in the Java environment there are a set of failures that cannot easily be predicted or anticipated, due to such things as runtime conditions or abuse of library code. There is no good way to predict an `OutOfMemoryError`, for example, and any method that uses objects or arrays can throw a `NullPointerException` if it is passed an invalid `null` argument.

These are the unchecked exceptions—and practically any method can throw an unchecked exception at essentially any time. They are the Java environment's version of Murphy's law: "Anything that can go wrong, will go wrong." Recovery from an unchecked exception is usually very difficult, if not impossible—simply due to their sheer unpredictability.

To figure out whether an exception is checked or unchecked, remember that exceptions are `Throwable` objects and that these fall into two main categories, specified by the `Error` and `Exception` subclasses. Any exception object that is an `Error` is unchecked. There is also a subclass of `Exception` called `RuntimeException`—and any subclass of `RuntimeException` is also an unchecked exception. All other exceptions are checked exceptions.

### Working with checked exceptions

Java has different rules for working with checked and unchecked exceptions. If you write a method that throws a checked exception, you must use a `throws` clause to declare the exception in the method signature. The Java compiler checks to make sure you have declared them in method signatures and produces a compilation error if you have not (that's why they're called *checked exceptions*).

Even if you never throw a checked exception yourself, sometimes you must use a `throws` clause to declare a checked exception. If your method calls a method that can throw a checked exception, you must either include exception-handling code to handle that exception or use `throws` to declare that your method can also throw that exception.

For example, the following method tries to estimate the size of a web page—it uses the standard `java.net` libraries and the class `URL` (we'll meet these in Chapter 11) to contact the web page. It uses methods and constructors that can throw various types of `java.io.IOException` objects, so it declares this fact with a `throws` clause:

```
public static int estimateWebpageSize(String host) throws IOException {
    URL url = new URL("http://"+ host +"/");
    try (InputStream is = url.openStream()) {
        return is.available();
    }
}
```

In fact, the preceding code has a bug: we've misspelled the protocol specifier—there's no such protocol as *htp://*. So, the `estimateHomepageSize()` method will always fail with a `MalformedURLException`.

How do you know if the method you are calling can throw a checked exception? You can look at its method signature to find out. Or, failing that, the Java compiler will tell you (by reporting a compilation error) if you've called a method whose exceptions you must handle or declare.

## Variable-Length Argument Lists

Methods may be declared to accept, and may be invoked with, variable numbers of arguments. Such methods are commonly known as *varargs* methods. The "print formatted" method `System.out.printf()` as well as the related `format()` methods of `String` use varargs, as do a number of important methods from the Reflection API of `java.lang.reflect`.

To declare a variable-length argument list, follow the type of the last argument to the method with three dots (`...`), indicating that this last argument can be repeated zero or more times.[6] For example:

```
public static int max(int first, int... rest) {
  /* body omitted for now */
}
```

Varargs methods are handled purely by the compiler. They operate by converting the variable number of arguments into an array. To the Java runtime, the `max()` method is indistinguishable from this one:

```
public static int max(int first, int[] rest) {
  /* body omitted for now */
}
```

To convert a varargs signature to the "real" signature, simply replace `...` with `[ ]`. Remember that only one ellipsis can appear in a parameter list, and it may only appear on the last parameter in the list.

Let's flesh out the `max()` example a little:

```
public static int max(int first, int... rest) {
  int max = first;
  for(int i : rest) { // legal because rest is actually an array
    if (i > max) max = i;
  }
  return max;
}
```

This `max()` method is declared with two arguments. The first is just a regular `int` value. The second, however, may be repeated zero or more times. All of the following are legal invocations of `max()`:

```
max(0)
max(1, 2)
max(16, 8, 4, 2, 1)
```

Because varargs methods are compiled into methods that expect an array of arguments, invocations of those methods are compiled to include code that creates and initializes such an array. So the call `max(1,2,3)` is compiled to this:

```
max(1, new int[] { 2, 3 })
```

In fact, if you already have method arguments stored in an array, it is perfectly legal for you to pass them to the method that way, instead of writing them out individually. You can treat any `...` argument as if it were declared as an array. The converse is not true, however: you can use varargs method invocation syntax only when the method is actually declared as a varargs method using an ellipsis.

## Introduction to Classes and Objects

Now that we have introduced operators, expressions, statements, and methods, we can finally talk about classes. A *class* is a named collection of fields that holds data values and methods that operate on those values. Classes are just one of five reference types supported by Java (the others are arrays, interfaces, enums, and annotations), but they are the most important type. Classes are thoroughly documented in a chapter of their own (Chapter 4). We introduce them here, however, because they are the next higher level of syntax after methods, and because the rest of this chapter requires a basic familiarity with the concept of a class and the basic syntax for defining a class, instantiating it, and using the resulting *object*.

The most important thing about classes is that they define new data types. For example, you might define a class named `Account` to represent a bank account that holds a balance. The class would define fields to hold data items such as the balance (perhaps represented as a `double`), account holder's name and address (as `String` instances), and methods to manipulate and operate on the account. The `Account` class is a new data type.

When discussing data types, it is important to distinguish between the data type itself and the

values the data type represents. `char` is a data type: it represents Unicode characters. But a `char` value represents a single specific character. A class is a data type; a class value is called an *object*. We use the name *class* because each class defines a type (or kind, or species, or class) of objects. The `Account` class is a data type that represents bank accounts, while an `Account` object represents a single specific account. As you might imagine, classes and their objects are closely linked. In the sections that follow, we will discuss both.

## Defining a Class

Here is a possible definition of the `Account` class we have been discussing:

```
/** Represents a customer bank account */
public class Account {
  public String name;
  public double balance;
  public int accountId;

  // A constructor that initializes the fields
  public Account(String name, double openingBalance, int id) {
    this.name = name;
    this.balance = openingBalance;
    this.accountId = id;
  }
}
```

This class definition is stored in a file named *Account.java* and compiled to a file named *Account.class*, where it is available for use by Java programs and other classes. This class definition is provided here for completeness and to provide context, but don't expect to understand all the details just yet; most of Chapter 4 is devoted to the topic of defining classes.

Keep in mind that you don't have to define every class you want to use in a Java program. The Java platform includes thousands of predefined classes that are guaranteed to be available on every computer that runs that given version of Java.

## Creating an Object

Now that we have defined the `Account` class as a new data type, we can use the following line to declare a variable that holds a reference to an `Account` object:

```
Account a;
```

Declaring a variable to hold an `Account` object reference does not create the object itself, however. To actually create an object in the heap, you must use the `new` operator. This keyword is followed by the object's class (i.e., its type) and an optional argument list in parentheses. These arguments are passed to the constructor for the class, which initializes internal fields in the new object:

```
// Declare variable a and store a reference to new Account object
```

```
Account a = new Account("Jason Clark", 0.0, 42);

// Create some other objects as well
// An object that represents the current time
LocalDateTime d = new LocalDateTime();

// A HashSet object to hold a set of strings
Set<String> words = new HashSet<>();
```

The `new` keyword is by far the most common way to create objects in Java, but a few other ways are also worth mentioning.

First, classes that meet certain criteria are so important that Java defines special literal syntax for creating objects of those types (as we discuss later in this section). Second, Java supports a mechanism that allows programs to load classes and create instances of those classes dynamically. See Chapter 13 for more details. Finally, objects can also be created by deserializing them. An object that has had its state saved, or serialized, usually to a file, can be re-created using the `java.io.ObjectInputStream` class.

## Using an Object

Now that we've seen how to define classes and instantiate them by creating objects, we need to look at the Java syntax that allows us to use those objects. Recall that a class defines a collection of fields and methods. Each object has its own copies of those fields and has access to those methods. We use the dot character (`.`) to access the named fields and methods of an object. For example:

```
Account a = new Account("Jason", 0.0, 42);  // Create an object

double b  = a.balance;                 // Read a field of the object
a.balance = a.balance + 10.0;          // Set the value of a field

String s  = a.toString();              // Access a method of the object
```

This syntax is very common when programming in object-oriented languages, and Java is no exception. Note, in particular, the expression `a.toString()`. This tells the Java compiler to look up a method named `toString` (which is defined by the parent `Object` class of `Account`) and use that method to perform a computation on the object `a`. We'll cover the details of this operation in Chapter 4.

## Object Literals

In our discussion of primitive types, we saw that each primitive type has a literal syntax for including values of the type literally into the text of a program. Java also defines a literal syntax for a few special reference types, as described next.

### String literals

The `String` class represents text as a string of characters. Because programs usually communicate with their users through the written word, the ability to manipulate strings of text is quite important in any programming language. In Java, strings are objects; the data type used to represent text is the `String` class. Modern Java programs usually use more string data than anything else.

Accordingly, because strings are such a fundamental data type, Java allows you to include text literally in programs in one of two formats. Traditional strings are placed between double-quote (`"`) characters, or a newer text block form may be used between sequences of three double-quote characters (`"""`).

A traditional double-quoted string looks like this:

```
String name = "David";
System.out.println("Hello, " + name);
```

Don't confuse the double-quote characters that surround string literals with the single-quote (or apostrophe) characters that surround `char` literals. Note that this differs from many popular scripting languages such as Python and JavaScript where `'` and `"` are interchangeable in defining literal strings.

String literals of either form can contain any of the escape sequences `char` literals can (see Table 2-2). Traditional double-quoted strings require escape sequences to embed double-quote characters or newlines. They also must consist of a single line in our Java code. For example:

```
String story = "\t\"How can you stand it?\" he asked sarcastically.\n";
```

The primary use for text blocks instead of traditional strings is representing multi-line strings. Text blocks start with `"""`, followed by a newline, and end when a concluding `"""` is reached.

Along with their support for multiline strings, text blocks also allow us to use double quotes without escaping. This often makes text blocks much easier to read, particularly when expressing another programming language (such as SQL or HTML) in our Java code:

```
String html = """
              <html>
                <body class="main-body">
                  ...
                </body>
              </html>""";
System.out.println(html);
```

Examining the output from this code reveals one more interesting fact about text blocks regarding indentation. The above prints with `<html>` with no leading spaces.

The compiler finds the smallest indentation across the lines of our text block and strips that many leading spaces from each line. The placement of the closing `"""` also participates in choosing the indent—we could retain the full white space with:

```
String html = """
              <html>
                <body class="main-body">
                  ...
                </body>
              </html>
""";  // As smallest indent (0), this leaves the text block as written

System.out.println(html);
```

Before text blocks were introduced in Java, it was common to break up string literals for easier reading using + to concatenate them. Along with existing in many codebases, this remains a valid technique if your string shouldn't include newlines:

```
// This is illegal
// Traditional string literals cannot break across lines.
String x = "This is a test of the
            emergency broadcast system";

// Common before text blocks
// Still useful if avoiding newlines in the text
String s = "This is a test of the " +
           "emergency broadcast system";
```

Literals, whether traditional or text blocks, are concatenated when your program is compiled, not when it is run, so you do not need to worry about any kind of performance penalty.

### Type literals

The second type that supports its own special object literal syntax is the class named `Class`. Instances of the `Class` class represent a Java data type and contain metadata about the type that is referred to. To include a `Class` object literally in a Java program, follow the name of any data type with `.class`. For example:

```
var typeInt = int.class;
var typeIntArray = int[].class;
var typeAccount = Account.class;
```

We're using the var type-inferred syntax here to simplify the discussion—we will revisit the exact types used here in Chapter 13.

### The null reference

The `null` keyword is a special literal value that is a reference to nothing, or an absence of a reference. The `null` value is unique because it is a member of every reference type. You can assign `null` to variables of any reference type. For example:

```
String s = null;
Account a = null;
```

## Lambda Expressions

Java 8 introduced a major new feature—*lambda expressions*. These are a very common programming language construct and in particular are widely used in the family of languages known as *functional programming languages* (e.g., Lisp, Haskell, and OCaml).

However, the power and flexibility of lambdas goes far beyond just functional languages, and lambdas can be found in almost all modern programming languages and are particularly prevalent in JavaScript.

> **DEFINITION OF A LAMBDA EXPRESSION**
>
> A lambda expression is essentially a function that does not have a name and can be treated as a value in the language (e.g., assigned to a variable). The JVM does not allow code to be defined outside of classes, so in Java this means that a lambda creates an instance of some class with the anonymous method defined on that class.[7]

The syntax for a lambda expression looks like this:

```
( paramlist ) -> { statements }
```

One simple, very traditional example:

```
Runnable r = () -> System.out.println("Hello World");
```

When a lambda expression is used as a value, it is automatically converted to a new object of the correct type for the variable it is being placed into.

> **TIP**
>
> Whenever you see a lambda being declared, you should understand it as an object creation expression—no different than the use of the `new` operator (or a factory method).

This autoconversion and *type inference* is essential to Java's approach to lambda expressions, but it relies upon a proper understanding of Java's type system as a whole. "Nested Types" provides a full explanation of lambda expressions—so for now, it suffices to recognize the syntax for lambdas.

This includes slightly more complex examples such as:

```
ActionListener listener = (e) -> {
  System.out.println("Event fired at: " + e.getWhen());
  System.out.println("Event command: " + e.getActionCommand());
};
```

This code shows how to declare an `ActionListener` as a lambda expression. The body

contains the block of statements that will be run when the listener's method is fired by an event. Note the type inference of the argument `e`, which can be understood to be the event that caused the listener to activate.

# Arrays

An *array* is a special kind of object that holds zero or more primitive values or references. These values are held in the *elements* of the array, which are unnamed variables referred to by their position or *index*. The type of an array is characterized by its *element type*, and all elements of the array must be of that type.

Array elements are numbered starting with zero, and valid indexes range from zero to the number of elements minus one. The array element with index 1, for example, is the *second* element in the array. The number of elements in an array is its *length*. The length of an array is specified when the array is created, and it never changes (unlike Java collections, which we'll see in Chapter 8).

The element type of an array may be any valid Java type, including array types. This means that Java supports arrays of arrays, which provide a kind of multidimensional array capability. Java does not support the matrix-style multidimensional arrays found in some languages.

While Java's Collection API, covered thoroughly in Chapter 8, is often more flexible and feature-rich than basic arrays, arrays remain common throughout the platform, and it's worth understanding the details of using them.

## Array Types

Array types are reference types, just as classes are. Instances of arrays are objects, just as the instances of a class are.[8] Unlike classes, array types do not have to be defined. Simply place square brackets after the element type. For example, the following code declares three variables of array type:

```
byte b;                       // byte is a primitive type
byte[] arrayOfBytes;          // byte[] is an array of byte values
byte[][] arrayOfArrayOfBytes; // byte[][] is an array of byte[]
String[] strings;             // String[] is an array of strings
```

The length of an array is not part of the array type. It is not possible, for example, to declare a method that expects an array of exactly four `int` values. If a method parameter is of type `int[]`, a caller can pass an array with any number (including zero) of elements.

Array types are not classes, but array instances are objects. This means that arrays inherit the methods of `java.lang.Object`. Arrays implement the `Cloneable` interface and override the `clone()` method to guarantee that an array can always be cloned and that `clone()` never throws a `CloneNotSupportedException`. Arrays also implement `Serializable` so that any array can be serialized if its element type can be serialized. Finally, all arrays have a

`public final int` field named `length` that specifies the number of elements in the array.

### Array type widening conversions

Because arrays extend `Object` and implement the `Cloneable` and `Serializable` interfaces, any array type can be widened to any of these three types. But certain array types can also be widened to other array types. If the element type of an array is a reference type `T`, and `T` is assignable to a type `S`, the array type `T[]` is assignable to the array type `S[]`. Note that there are no widening conversions of this sort for arrays of a given primitive type. As examples, the following lines of code show legal array widening conversions:

```
String[] arrayOfStrings;      // Created elsewhere
int[][] arrayOfArraysOfInt;   // Created elsewhere

// String is assignable to Object,
// so String[] is assignable to Object[]
Object[] oa = arrayOfStrings;

// String implements Comparable, so a String[] can
// be considered a Comparable[]
Comparable[] ca = arrayOfStrings;

// An int[] is an Object, so int[][] is assignable to Object[]
Object[] oa2 = arrayOfArraysOfInt;

// All arrays are cloneable, serializable Objects
Object o = arrayOfStrings;
Cloneable c = arrayOfArraysOfInt;
Serializable s = arrayOfArraysOfInt[0];
```

This ability to widen an array type to another array type means that the compile-time type of an array is not always the same as its runtime type.

> **TIP**
>
> This widening is known as *array covariance*, and as we shall see in “Bounded Type Parameters”, it is regarded by modern standards as a historical artifact and a misfeature, because of the mismatch between compile and runtime typing that it exposes.

The compiler must usually insert runtime checks before any operation that stores a reference value into an array element to ensure that the runtime type of the value matches the runtime type of the array element. An `ArrayStoreException` is thrown if the runtime check fails.

## Creating and Initializing Arrays

To create an array value in Java, you use the `new` keyword, just as you do to create an object. Array types don’t have constructors, but you are required to specify a length whenever you create an array. Specify the desired size of your array as a nonnegative integer between square

brackets:

```
// Create a new array to hold 1024 bytes
byte[] buffer = new byte[1024];

// Create an array of 50 references to strings
String[] lines = new String[50];
```

When you create an array with this syntax, each of the array elements is automatically initialized to the same default value that is used for the fields of a class: `false` for `boolean` elements, `\u0000` for `char` elements, `0` for integer elements, `0.0` for floating-point elements, and `null` for elements of reference type.

Array creation expressions can also be used to create and initialize a multidimensional array of arrays. This syntax is somewhat more complicated and is explained later in this section.

### Array initializers

To create an array and initialize its elements in a single expression, omit the array length and follow the square brackets with a comma-separated list of expressions within curly braces. The type of each expression must be assignable to the element type of the array, of course. The length of the array that is created is equal to the number of expressions. It is legal, but not necessary, to include a trailing comma following the last expression in the list. For example:

```
String[] greetings = new String[] { "Hello", "Hi", "Howdy" };
int[] smallPrimes = new int[] { 2, 3, 5, 7, 11, 13, 17, 19, };
```

Note that this syntax allows arrays to be created, initialized, and used without ever being assigned to a variable. In a sense, these array creation expressions are anonymous array literals. Here are examples:

```
// Call a method, passing an anonymous array literal that
// contains two strings
String response = askQuestion("Do you want to quit?",
                              new String[] {"Yes", "No"});

// Call another method with an anonymous array (of anonymous objects)
double d = sumAccounts(new Account[] { new Account("1st", 100.0, 1),
                                       new Account("2nd", 200.0, 2),
                                       new Account("3rd", 300.0, 3) });
```

When an array initializer is part of a variable declaration, you may omit the `new` keyword and element type and list the desired array elements within curly braces:

```
String[] greetings = { "Hello", "Hi", "Howdy" };
int[] powersOfTwo = {1, 2, 4, 8, 16, 32, 64, 128};
```

Array literals are created and initialized when the program is run, not when the program is compiled. Consider the following array literal:

```
int[] perfectNumbers = {6, 28};
```

This is compiled into Java bytecodes that are equivalent to:

```
int[] perfectNumbers = new int[2];
perfectNumbers[0] = 6;
perfectNumbers[1] = 28;
```

The fact that Java does all array initialization at runtime has an important corollary. It means that the expressions in an array initializer may be computed at runtime and need not be compile-time constants. For example:

```
Account[] accounts = { findAccountById(1), findAccountById(2) };
```

## Using Arrays

Once an array has been created, you are ready to start using it. The following sections explain basic access to the elements of an array and cover common idioms of array usage, such as iterating through the elements of an array and copying an array or part of an array.

### Accessing array elements

The elements of an array are variables. When an array element appears in an expression, it evaluates to the value held in the element. And when an array element appears on the lefthand side of an assignment operator (an *lvalue*), a new value is stored into that element. Unlike a normal variable, however, an array element has no name, only a number. Array elements are accessed using a square bracket notation. If `a` is an expression that evaluates to an array reference, you index that array and refer to a specific element with `a[i]`, where `i` is an integer literal or an expression that evaluates to an `int`. For example:

```
// Create an array of two strings
String[] responses = new String[2];
responses[0] = "Yes";  // Set the first element of the array
responses[1] = "No";   // Set the second element of the array

// Now read these array elements
System.out.println(question + " (" + responses[0] + "/" +
                   responses[1] + " ): ");

// Both the array reference and the array index may be more complex
double datum = data.getMatrix()[data.row() * data.numColumns() +
                   data.column()];
```

The array index expression must be of type `int`, or a type that can be widened to an `int`: `byte`, `short`, or even `char`. It is obviously not legal to index an array with a `boolean`, `float`, or `double` value. Remember that the `length` field of an array is an `int`, and so arrays may not have more than `Integer.MAX_VALUE` elements. Indexing an array with an expression of type `long` generates a compile-time error, even if the value of that expression at

runtime would be within the range of an `int`.[9]

### Array bounds

Remember that the first element of an array `a` is `a[0]` , the second element is `a[1]`, and the last is `a[a.length-1]`.

A common bug involving arrays is the use of an index that is too small (a negative index) or too large (greater than or equal to the array `length`). In languages like C or C++, accessing elements before the beginning or after the end of an array yields unpredictable behavior that can vary from invocation to invocation and platform to platform. Such bugs may not always be caught, and if a failure occurs, it may be at some later time. While it is just as easy to write faulty array indexing code in Java, Java guarantees predictable results by checking every array access at runtime. If an array index is too small or too large, Java always throws an `ArrayIndexOutOf Bound sException` before any illegal memory access can occur.

### Iterating arrays

It is common to write loops that iterate through each of the elements of an array in order to perform some operation on it. This is typically done with a `for` loop. The following code, for example, computes the sum of an array of integers:

```
int[] primes = { 2, 3, 5, 7, 11, 13, 17, 19, 23 };
int sumOfPrimes = 0;
for(int i = 0; i < primes.length; i++)
  sumOfPrimes += primes[i];
```

The structure of this `for` loop is idiomatic, and you'll see it frequently. Java also has the foreach syntax that we've already met. The summing code could be rewritten succinctly as follows:

```
// ...
for(int p : primes)
  sumOfPrimes += p;
```

### Copying arrays

Sometimes you want to copy elements from an existing array to another existing array, and `System.arraycopy()` is a static method to perform this common operation.

It can be difficult to use—because it has five arguments to remember: first, pass the source array from which elements are to be copied. Second, pass the index of the start element in that array. Pass the destination array and the destination index as the third and fourth arguments. Finally, as the fifth argument, specify the number of elements to be copied.

`System.arraycopy()` works correctly even for overlapping copies within the same array. For example, if you've "deleted" the element at index `0` from array `a` and want to shift the elements between indexes `1` and `n` down one so that they occupy indexes `0` through `n-1,` you could do this:

```
System.arraycopy(a, 1, a, 0, n);
```

`System.arraycopy()` is designed for efficiency, and you can assume that JVM implementations perform this method using high-speed block copy operations on the underlying hardware.

### Array utilities

The `java.util.Arrays` class contains a number of static utility methods for working with arrays. Most of these methods are heavily overloaded, with versions for arrays of each primitive type and another version for arrays of objects.

The `sort()` and `binarySearch()` methods are particularly useful for sorting and searching arrays. The `equals()` method allows you to compare the contents of two arrays. The `toString()` method is useful when you want to convert array content to a string, such as for debugging or logging output. `copyOf()` is a useful alternative to the `arraycopy()` we've seen before if you're OK with a new array being allocated rather than copying into an existing one.

The `Arrays` class also includes `deepEquals()`, `deepHashCode()`, and `deepToString()` methods that work correctly for multidimensional arrays.

## Multidimensional Arrays

As we've seen, an array type is written as the element type followed by a pair of square brackets. An array of `char` is `char[]`, and an array of arrays of `char` is `char[][]`. When the elements of an array are themselves arrays, we say that the array is *multidimensional*. To work with multidimensional arrays, you need to understand a few additional details.

Imagine that you want to use a multidimensional array to represent a multiplication table:

```
int[][] products;      // A multiplication table
```

Each of the pairs of square brackets represents one dimension, so this is a two-dimensional array. To access a single `int` element of this two-dimensional array, you must specify two index values, one for each dimension. Assuming that this array was actually initialized as a multiplication table, the `int` value stored at any given element would be the product of the two indexes. That is, `products[2][4]` would be 8, and `products[3][7]` would be 21.

To create a new multidimensional array, use the `new` keyword and specify the size of both dimensions of the array. For example:

```
int[][] products = new int[10][10];
```

In some languages, an array like this would be created as a single block of 100 `int` values. Java does not work this way. This line of code does three things:

- Declares a variable named `products` to hold an array of arrays of `int`.
- Creates a 10-element array to hold 10 arrays of `int` (e.g., `int[]`).
- Creates 10 more arrays, each of which is a 10-element array of `int`. It assigns each of these 10 new arrays to the elements of the initial array. The default value of every `int` element of each of these 10 new arrays is 0.

To put this another way, the previous single line of code is equivalent to the following code:

```
int[][] products = new int[10][]; // An array to hold 10 int[] values
for(int i = 0; i < 10; i++) {     // Loop 10 times...
  products[i] = new int[10];      // ...and create 10 arrays
}
```

The `new` keyword automatically performs this additional initialization for you. It works with arrays with more than two dimensions as well:

```
float[][][] globalTemperatureData = new float[360][180][100];
```

When using `new` with multidimensional arrays, you do not have to specify a size for all dimensions of the array, only the leftmost dimension or dimensions. For example, the following two lines are legal:

```
float[][][] globalTemperatureData = new float[360][][];
float[][][] globalTemperatureData = new float[360][180][];
```

The first line creates a single-dimensional array, where each element of the array can hold a `float[][]`. The second line creates a two-dimensional array, where each element of the array is a `float[]`. If you specify a size for only some of the dimensions of an array, however, those dimensions must be the leftmost ones. The following lines are not legal:

```
float[][][] globalTemperatureData = new float[360][][100];  // Error!
float[][][] globalTemperatureData = new float[][180][100];  // Error!
```

Like a one-dimensional array, a multidimensional array can be initialized using an array initializer. Simply use nested sets of curly braces to nest arrays within arrays. For example, we can declare, create, and initialize a 5 × 5 multiplication table like this:

```
int[][] products = { {0, 0, 0, 0, 0},
                     {0, 1, 2, 3, 4},
                     {0, 2, 4, 6, 8},
                     {0, 3, 6, 9, 12},
                     {0, 4, 8, 12, 16} };
```

Or, if you want to use a multidimensional array without declaring a variable, you can use the anonymous initializer syntax:

```
boolean response = bilingualQuestion(question, new String[][] {
                                                    { "Yes", "No" },
                                                    { "Oui", "Non" }});
```

When you create a multidimensional array using the `new` keyword, it is usually good practice to use only *rectangular* arrays, ones in which all the array values for a given dimension have the same size.

# Reference Types

Now that we've covered arrays and introduced classes and objects, we can turn to a more general description of *reference types*. Classes and arrays are two of Java's five kinds of reference types. Classes were introduced earlier and are covered in complete detail, along with *interfaces*, in Chapter 4. Enumerated types and annotation types are reference types introduced in Chapter 5.

This section does not cover specific syntax for any particular reference type but instead explains the general behavior of reference types and illustrates how they differ from Java's primitive types. In this section, the term *object* refers to a value or instance of any reference type, including arrays.

## Reference Versus Primitive Types

Reference types and objects differ substantially from primitive types and their primitive values:

*Eight primitive types are defined by the Java language, and the programmer cannot define new primitive types.*

> Reference types are user-defined, so there is an unlimited number of them. For example, a program might define a class named `Account` and use objects of this newly defined type to store and track user bank accounts.

*Primitive types represent single values.*

> Reference types are aggregate types that hold zero or more primitive values or objects. Our hypothetical `Account` class, for example, might hold a numeric value for the balance, along with identifiers for the account owner. The `char[]` and `Account[]` array types are aggregate types because they hold a sequence of primitive `char` values or `Account` objects.

*Primitive types require between one and eight bytes of memory.*

> When a primitive value is stored in a variable or passed to a method, the computer makes a copy of the bytes that hold the value. Objects, on the other hand, may require substantially more memory. Memory to store an object is dynamically allocated on the heap when the object is created, and this memory is automatically "garbage collected" when the object is no longer needed.

The sharp division between primitives and references has been part of the Java data model since the beginning. However, there is an ongoing effort (Project Valhalla) to evaluate whether this can be reimagined for modern Java—we'll discuss it in Appendix A.

> **TIP**
>
> When an object is assigned to a variable or passed to a method, the memory that represents the object is not copied. Instead, only a reference to that memory is stored in the variable or passed to the method.

References are completely opaque in Java, and the representation of a reference is an implementation detail of the Java runtime. If you are a C programmer, however, you can safely imagine a reference as a pointer or a memory address. Remember, though, that Java programs cannot manipulate references directly in any way.

For example, unlike pointers in C and C++, references cannot be converted to or from integers, and so they cannot be incremented or decremented. C and C++ programmers should also note that Java does not support the address-of operator (`&`) or the dereference operators (`*` and `->`).

## Manipulating Objects and Reference Copies

The following code manipulates primitive `int` values:

```
int x = 42;
int y = x;
```

After these lines execute, the variable `y` contains a copy of the value held in the variable `x`. Inside the JVM, there are two independent copies of the 32-bit integer 42.

Now think about what happens if we run the same basic code but use a reference type instead of a primitive type:

```
Account a = new Account("Jason", 0.0, 42);
Account b = a;
```

After this code runs, the variable `b` holds a copy of the *reference* held in the variable `a`. There is still only one copy of the `Account` object in the JVM, but there are now two copies of the reference to that object. This has some important implications. Suppose the two previous lines of code are followed by this code:

```
System.out.println(a.balance);  // Print out balance of a: 0.0
b.balance = 13.0;               // Now change balance of b
System.out.println(a.balance);  // Print a's balance again: 13.0
```

Because the variables `a` and `b` hold references to the same object, either variable can be used to make changes to the object, and those changes are visible through the other variable as well. As

arrays are a kind of object, the same thing happens with arrays, as illustrated by the following code:

```
// greet holds an array reference
char[] greet = { 'h','e','l','l','o' };
char[] cuss = greet;              // cuss holds the same reference
cuss[4] = '!';                    // Use reference to change an element
System.out.println(greet);        // Prints "hell!"
```

A similar difference in behavior between primitive types and reference types occurs when arguments are passed to methods. Consider the following method:

```
void changePrimitive(int x) {
  while(x > 0) {
    System.out.println(x--);
  }
}
```

When this method is invoked, the method is given a copy of the argument used to invoke the method in the parameter `x`. The code in the method uses `x` as a loop counter and decrements it to zero. Because `x` is a primitive type, the method has its own private copy of this value, so this is a perfectly reasonable thing to do.

On the other hand, consider what happens if we modify the method so that the parameter is a reference type:

```
void changeReference(Account b) {
  while (b.balance > 0) {
    System.out.println(b.balance--);
  }
}
```

When this method is invoked, it is passed a private copy of a reference to an `Account` object. Changes to this reference also change the `Account` object. For example, consider:

```
Account a = new Account("Jason", 3.0, 42);  // Account balance: 3.0
changeReference(a);              // Prints 3,2,1 and modifies the
                                 // Account
System.out.println(a.balance);  // The balance of a is now 0!
```

When the `changeReference()` method is invoked, it is passed a copy of the reference held in variable `a`. Now both the variable `a` and the method parameter `b` hold references to the same object. The method can use its reference to change the contents of the object. Note, however, that it cannot change the contents of the variable `a`. In other words, the method can change the `Account` object beyond recognition, but it cannot change the fact that the variable `a` refers to that object.

## Comparing Objects

We've seen that primitive types and reference types differ significantly in the way they are assigned to variables, passed to methods, and copied. The types also differ in the way they are compared for equality. When used with primitive values, the equality operator (==) simply tests whether two values are identical (i.e., whether they have exactly the same bits). With reference types, however, == compares references, not actual objects. In other words, == tests whether two references refer to the same object; it does not test whether two objects have the same content. Here's an example:

```
String letter = "o";
String s = "hello";              // These two String objects
String t = "hell" + letter;      // contain exactly the same text.
if (s == t) System.out.println("equal"); // But they are not equal!
```

When working with reference types, keep in mind there are two kinds of equality: equality of reference and equality of object. It is important to distinguish between these two kinds of equality. One way to do this is to use the word *identical* when talking about equality of references and the word *equal* when talking about two distinct objects that have the same content. To test two nonidentical objects for equality, pass one of them to the `equals()` method of the other:

```
String letter = "o";
String s = "hello";             // These two String objects
String t = "hell" + letter;     // contain exactly the same text.
if (s.equals(t)) {              // And the equals() method
  System.out.println("equal");  // tells us so.
}
```

All objects inherit an `equals()` method (from `Object`), but the default implementation simply uses == to test for identity of references, not equality of content. A class that wants to allow objects to be compared for equality can define its own version of the `equals()` method. Our `Account` class does not do this, but the `String` class does, as indicated in the code example. You can call the `equals()` method on an array, but it is the same as using the == operator, because arrays always inherit the default `equals()` method that compares references rather than array content. You can compare arrays for equality with the `java.util.Arrays.equals()` convenience method.

## Boxing and Unboxing Conversions

Primitive types and reference types behave quite differently. It is sometimes useful to treat primitive values as objects, and for this reason, the Java platform includes *wrapper classes* for each of the primitive types. `Boolean`, `Byte`, `Short`, `Character`, `Integer`, `Long`, `Float`, and `Double` are immutable, final classes whose instances each hold a single primitive value. These wrapper classes are usually used when you want to store primitive values in collections such as `java.util.List`:

```
// Create a List-of-Integer collection
```

```
List<Integer> numbers = new ArrayList<>();
// Store a wrapped primitive
numbers.add(Integer.valueOf(-1));
// Extract the primitive value
int i = numbers.get(0).intValue();
```

Java allows types of conversions known as *boxing* and *unboxing conversions*. Boxing conversions convert a primitive value to its corresponding wrapper object, and unboxing conversions do the opposite. You may explicitly specify a boxing or unboxing conversion with a cast, but this is unnecessary, as these conversions are automatically performed when you assign a value to a variable or pass a value to a method. Furthermore, unboxing conversions are also automatic if you use a wrapper object when a Java operator or statement expects a primitive value. Because Java performs boxing and unboxing automatically, this language feature is often known as *autoboxing*.

Here are some examples of automatic boxing and unboxing conversions:

```
Integer i = 0;   // int literal 0 boxed to an Integer object
Number n = 0.0f; // float literal boxed to Float and widened to Number
Integer i = 1;   // this is a boxing conversion
int j = i;       // i is unboxed here
i++;             // i is unboxed, incremented, and then boxed up again
Integer k = i+2; // i is unboxed and the sum is boxed up again
i = null;
j = i;           // unboxing here throws a NullPointerException
```

Autoboxing makes dealing with collections much easier as well. Let's look at an example that uses Java's *generics* (a language feature we'll meet properly in "Java Generics") that allows us to restrict what types can be put into lists and other collections:

```
List<Integer> numbers = new ArrayList<>(); // Create a List of Integer
numbers.add(-1);                           // Box int to Integer
int i = numbers.get(0);                    // Unbox Integer to int
```

## Packages and the Java Namespace

A *package* is a named collection of classes, interfaces, and other reference types. Packages serve to group related classes and define a namespace for the classes they contain.

The core classes of the Java platform are in packages whose names begin with `java`. For example, the most fundamental classes of the language are in the package `java.lang`. Various utility classes are in `java.util`. Classes for input and output are in `java.io`, and classes for networking are in `java.net`. Some of these packages contain subpackages, such as `java.lang.reflect` and `java.util.regex`. Extensions to the Java platform that have been standardized by Oracle (or originally Sun) typically have package names that begin with `javax`. Some of these extensions, such as `javax.swing` and its myriad subpackages, were later adopted into the core platform itself. Finally, the Java platform also includes several "endorsed standards," which have packages named after the standards body that created them,

such as `org.w3c` and `org.omg`.

Every class has both a simple name, which is the name given to it in its definition, and a fully qualified name, which includes the name of the package of which it is a part. The `String` class, for example, is part of the `java.lang` package, so its fully qualified name is `java.lang.String`.

This section explains how to place your own classes and interfaces into a package and how to choose a package name that won't conflict with anyone else's package name. Next, it explains how to selectively import type names or static members into the namespace so that you don't have to type the package name of every class or interface you use.

## Package Declaration

To specify the package a class belongs to, you use a `package` declaration. The `package` keyword, if it appears, must be the first token of Java code (i.e., the first thing other than comments and space) in the Java file. The keyword should be followed by the name of the desired package and a semicolon. Consider a Java file that begins with this directive:

```
package org.apache.commons.net;
```

All classes defined by this file are part of the package `org.apache.commons.net`.

If no `package` directive appears in a Java file, all classes defined in that file are part of an unnamed default package. In this case, the qualified and unqualified names of a class are the same.

> **TIP**
>
> The possibility of naming conflicts means that you should not use the default package. As your project grows more complicated, conflicts become almost inevitable—much better to create packages right from the start.

## Globally Unique Package Names

One of the important functions of packages is to partition the Java namespace and prevent name collisions between classes. It is only their package names that keep the `java.util.List` and `java.awt.List` classes distinct, for example. For this to work, however, package names must themselves be distinct. As the developer of Java, Oracle controls all package names that begin with `java`, `javax`, and `sun`.

One common scheme is to use your domain name, with its elements reversed, as the prefix for all your package names. For example, the Apache Project produces a networking library as part of the Apache Commons project. The package name used for the networking library is `org.apache.commons.net`.

Note that these package-naming rules apply primarily to API developers. If other programmers

will be using classes that you develop along with other unknown classes, it is important that your package name be globally unique. On the other hand, if you are developing a Java application and will not be releasing any of the classes for reuse by others, you know the complete set of classes that your application will be deployed with and do not have to worry about unforeseen naming conflicts. In this case, you can choose a package-naming scheme for your own convenience rather than for global uniqueness. One common approach is to use the application name as the main package name (it may have subpackages beneath it).

## Importing Types

When referring to a class or interface in your Java code, you must, by default, use the fully qualified name of the type, including the package name. If you're writing code to manipulate a file and need to use the `File` class of the `java.io` package, you must type `java.io.File`. This rule has three exceptions:

- Types from the package `java.lang` are so important and so commonly used that they can always be referred to by their simple names.
- The code in a type `p.T` may refer to other types defined in the package `p` by their simple names.
- Types that have been *imported* into the namespace with an `import` declaration may be referred to by their simple names.

The first two exceptions are known as *automatic imports*. The types from `java.lang` and the current package are "imported" into the namespace so that they can be used without their package name. Typing the package name of commonly used types that are not in `java.lang` or the current package quickly becomes tedious, and so it is also possible to explicitly import types from other packages into the namespace. This is done with the `import` declaration.

`import` declarations must appear at the start of a Java file, immediately after the `package` declaration, if there is one, and before any type definitions. You may use any number of `import` declarations in a file. An `import` declaration applies to all type definitions in the file (but not to any `import` declarations that follow it).

The `import` declaration has two forms. To import a single type into the namespace, follow the `import` keyword with the name of the type and a semicolon:

```
import java.io.File;    // Now we can type File instead of java.io.File
```

This is known as the "single type `import`" declaration.

The other form of `import` declaration is the "on-demand type `import`." In this form, you specify the name of a package followed by the characters `.*` to indicate that any type from that package may be used without its package name.[10] Thus, if you want to use several other classes from the `java.io` package in addition to the `File` class, you can simply import the entire

package:

```
import java.io.*;   // Use simple names for all classes in java.io
```

This on-demand `import` syntax does not apply to subpackages. If I import the `java.util` package, I must still refer to the `java.util.zip.ZipInputStream` class by its fully qualified name or import it.

Using an on-demand type `import` declaration is not the same as explicitly writing out a single type `import` declaration for every type in the package. It is more like an explicit single type `import` for every type in the package *that you actually use* in your code. This is the reason it's called "on-demand"; types are imported as you use them.

There is also an additional form of `import` that is available when working with modular code—but we'll defer discussing it until Chapter 14 as it's an advanced topic and not relevant for nonmodular code.

### Naming conflicts and shadowing

`import` declarations are invaluable to Java programming. They do expose us to the possibility of naming conflicts, however. Consider the packages `java.util` and `java.awt`. Both contain types named `List`.

`java.util.List` is an important and commonly used interface. The `java.awt` package contains a number of important types that are commonly used in client-side applications, but `java.awt.List` has been superseded and is not one of these important types. It is illegal to import both `java.util.List` and `java.awt.List` in the same Java file. The following single type `import` declarations produce a compilation error:

```
import java.util.List;
import java.awt.List;
```

Using on-demand type imports for the two packages is legal:

```
import java.util.*;  // For collections and other utilities.
import java.awt.*;   // For fonts, colors, and graphics.
```

Difficulty arises, however, if you actually try to use the type `List`. This type can be imported "on demand" from either package, and any attempt to use `List` as an unqualified type name produces a compilation error. The workaround, in this case, is to explicitly specify the package name you want.

Because `java.util.List` is much more commonly used than `java.awt.List`, it is useful to combine the two on-demand type `import` declarations with a single type `import` declaration that serves to disambiguate what we mean when we say `List`:

```
import java.util.*;    // For collections and other utilities.
```

```
import java.awt.*;     // For fonts, colors, and graphics.
import java.util.List; // To disambiguate from java.awt.List
```

With these `import` declarations in place, we can use `List` to mean the `java.util.List` interface. If we actually need to use the `java.awt.List` class, we can still do so as long as we include its package name. There are no other naming conflicts between `java.util` and `java.awt`, and their types will be imported "on-demand" when we use them without a package name.

## Importing Static Members

As well as types, you can import the static members of types using the keywords `import static`. (Static members are explained in Chapter 4. If you are not already familiar with them, you may want to come back to this section later.) Like type `import` declarations, these static `import` declarations come in two forms: single static member `import` and on-demand static member `import`. Suppose, for example, that you are writing a text-based program that sends a lot of output to `System.out`. In this case, you might use this single static member `import` to save yourself typing:

```
import static java.lang.System.out;
```

You can then use `out.println()` instead of `System.out.println()`. Or suppose you are writing a program that uses many of the trigonometric and other functions of the `Math` class. In a program that is clearly focused on numerical methods like this, having to repeatedly type the class name "Math" does not add clarity to your code; it just gets in the way. In this case, an on-demand static member `import` may be appropriate:

```
import static java.lang.Math.*
```

With this `import` declaration, you are free to write concise expressions like `sqrt(abs(sin(x)))` without having to prefix the name of each static method with the class name `Math`.

Another important use of `import static` declarations is to import the names of constants into your code. This works particularly well with enumerated types (see Chapter 5). Suppose, for example, that you want to use the values of this enumerated type in code you are writing:

```
package climate.temperate;
enum Seasons { WINTER, SPRING, SUMMER, AUTUMN };
```

You could import the type `climate.temperate.Seasons` and then prefix the constants with the type name: `Seasons.SPRING`. For more concise code, you could import the enumerated values themselves:

```
import static climate.temperate.Seasons.*;
```

Using static member `import` declarations for constants is generally a better technique than implementing an interface that defines the constants.

### Static member imports and overloaded methods

A static `import` declaration imports a *name*, not any one specific member with that name. Because Java allows method overloading and allows a type to have fields and methods with the same name, a single static member `import` declaration may actually import more than one member. Consider this code:

```
import static java.util.Arrays.sort;
```

This declaration imports the name "sort" into the namespace, not any one of the 18 `sort()` methods defined by `java.util.Arrays`. If you use the imported name `sort` to invoke a method, the compiler will look at the types of the method arguments to determine which method you mean.

It is even legal to import static methods with the same name from two or more different types as long as the methods all have different signatures. Here is one natural example:

```
import static java.util.Arrays.sort;
import static java.util.Collections.sort;
```

You might expect that this code would cause a syntax error. In fact, it does not because the `sort()` methods defined by the `Collections` class have different signatures than all of the `sort()` methods defined by the `Arrays` class. When you use the name "sort" in your code, the compiler looks at the types of the arguments to determine which of the 20 possible imported methods you mean.

## Summary

In this chapter, we've introduced the basic syntax of the Java language. This chapter has taken us from the smallest to the largest elements of Java syntax, from individual characters and tokens to operators, expressions, statements, and methods, and on up to classes and packages.

Due to the interlocking nature of the syntax of programming languages, it is perfectly fine if you don't feel at this point that you have completely grasped all of the syntax of the language. It is by practice that we acquire proficiency in any language, human or computer.

It is also worth observing that some parts of syntax are far more regularly used than others. For example, the `assert` keyword is almost never used, and `strictfp` is a no-op from Java 17 onward. Rather than trying to grasp every aspect of Java's syntax, it is far better to begin to acquire facility in the core aspects of Java and then return to any details of syntax that may still be troubling you.

With this in mind, let's move to the next chapter and writing and running our first Java

programs.

---

1 Technically, the minus sign is an operator that operates on the literal and is not part of the literal itself.

2 This has been the source of numerous bugs in Java programs (and programs written in C and C++, the languages from which the behavior was inherited).

3 `yield` is an example of a context-sensitive keyword. It can be used as an identifier in other contexts, so `var yield = 1;` is perfectly legal Java code (but please don't).

4 Technically, they must all implement the `AutoCloseable` interface.

5 In the Java Language Specification, the term "signature" has a technical meaning that is slightly different from that used here. This book uses a less formal definition of method signature, but for the interested reader, the official form can be found in Section 8.4.2 of the Java Language Specification Java SE 25 Edition.

6 The Unicode ellipsis, … or \u2026, is not a valid character for use in a Java identifier.

7 The name of the defining class will be implementation-dependent, may differ from run to run, and is essentially unknown to the developer.

8 There is a terminology difficulty in discussions of arrays. Unlike with classes and their instances, we use the term "array" for both the array type and the array instance. In practice, it is usually clear from context whether a type or a value is being discussed.

9 If your program needs to address a bigger chunk of memory than 2 GB, then you should use the `ByteBuffer` class or the new FFM API, which we discuss in Appendix A.

10 This is sometimes referred to as a *wildcard* import, but this is something of an abuse of terminology.

# Chapter 3. Your First Java Programs

As we discussed in Chapter 1, Java is traditionally understood as a language that is class-oriented and has a distinct compile-interpret-JIT execution model. However, recent developments have extended and generalized this model in ways that are particularly helpful to newcomers to the Java world.

So, in this chapter we will discuss these new technologies and see how they create a new, and much easier, path to writing your first programs in Java. Let's start by meeting JShell, which is an interactive shell for Java, somewhat similar to the read-evaluate-print loop (REPL) seen in languages like Python, Scala, or Lisp.[1]

## Introduction to JShell

JShell is intended for instruction and exploratory use and, due to the nature of the Java language, is not expected to be as much use to the experienced programmer as similar shells in other languages. In particular, it is not expected that Java will become an REPL-driven language. Instead, it provides a capability to use JShell for a different style of programming, one that complements the traditional use case but also provides new perspectives, especially for working with a new API.

It is very easy to use JShell to explore simple language features, for instance:

- Primitive data types
- Simple numeric operations
- String manipulation basics
- Object types
- Definition of new classes
- Creation of new objects
- Calling of methods on objects

However, in the examples in this section, we're not going to explain each language feature as it comes up—instead we'll just assume some familiarity with another language (e.g., JavaScript or Python) and that the similarities will allow you to see what's going on.

So, if this is your first time using Java, then it might be helpful to refer to Chapter 2 for details of Java's specific syntax as you experiment within JShell. Or come back to this section after you've read some more of the book.

Importantly, and as we'll see later, the Java philosophy of classes and objects is slightly different

from Python's, and both are very different from JavaScript's approach. However, we're going to ignore those differences for now, so just pretend that everything works the same way as it does in your current favorite language.

To start up JShell, we just invoke it from the command line:

```
$ jshell
|  Welcome to JShell -- Version 25
|  For an introduction type: /help intro

jshell>
```

From here, we can enter small pieces of Java code, which are known as *snippets*:

```
jshell> 2 * 3
$1 ==> 6

jshell> var i = 2 * 3
i ==> 6
```

The shell is designed to be a simple working environment, and so it relaxes some of the rules that working Java programmers may expect.

> **TIP**
>
> The verbose mode is particularly useful when you are getting to know JShell—it can be activated by passing the `-v` switch at startup.

Some of the differences between JShell snippets and regular Java include:

- Semicolons are optional in JShell.
- JShell supports a verbose mode.
- JShell has a wider set of default imports than a regular Java program.
- Methods and classes can be redefined within snippets.
- A snippet may not declare a package or a module—everything is placed in an unnamed package controlled by the shell.
- Only public classes may be accessed from JShell.
- Due to package restrictions, it's advisable to ignore access control when defining classes and working within JShell.

It's easy to create simple class hierarchies (e.g., for exploring Java's inheritance and generics) and create objects:

```
jshell> class Pet {}
```

```
|  created class Pet

jshell> class Cat extends Pet {}
|  created class Cat

jshell> var c = new Cat()
c ==> Cat@2ac273d3
```

Tab completion within the shell is also possible, such as for auto-completion of possible methods available on objects:

```
jshell> c.<TAB>
equals(       getClass()    hashCode()    notify()      notifyAll()
toString()    wait(
```

Pressing the Tab key twice with certain input will display documentation for a method:

```
jshell> c.hashCode(<TAB>
Signatures:
int Object.hashCode()

<press tab again to see documentation>
jshell> c.hashCode(<TAB TAB>
int Object.hashCode()
Returns a hash code value for the object. (Full Javadoc follows...)
```

We can also create top-level methods, such as:

```
jshell> int div(int x, int y) {
   ...> return x / y;
   ...> }
|  created method div(int,int)
```

Be careful of Java's type system, especially if you're coming from JavaScript:

```
jshell> div(4,2)
$2 ==> 2

jshell> div(7,2)
$3 ==> 3
```

The `div()` method returns an `int`, which is always a whole number—so the fractional part is simply dropped. Java has other number types, such as `double`, that have a fractional part.

Simple exception backtraces are supported:

```
jshell> div(3,0)
|  Exception java.lang.ArithmeticException: / by zero
|        at div (#2:2)
|        at (#3:1)
```

We can access classes from the JDK:

```
jshell> var ls = List.of("Alpha", "Beta", "Gamma", "Delta", "Epsilon")
ls ==> [Alpha, Beta, Gamma, Delta, Epsilon]

jshell> ls.get(3)
$11 ==> "Delta"

jshell> ls.forEach(s -> IO.println(s.charAt(1)))
l
e
a
e
p
```

If we need other classes from the JDK, we can explicitly ask for them:

```
jshell> import java.time.LocalDateTime

jshell> var now = LocalDateTime.now()
now ==> 2025-10-21T16:32:27.770162
```

The environment also provides JShell control commands, which start with a `/`. It is useful to be aware of some of the most common basic commands:

- **`/help intro`** is the introductory help text.
- **`/help`** is a more comprehensive entry point into the help system.
- **`/vars`** shows which variables are in scope.
- **`/list`** shows the shell history.
- **`/save`** saves the snippets from the current session to a file.
- **`/open`** reads a saved file and loads it into the environment.
- **`/exit`** exits the JShell interface.

For example, the imported classes available within JShell include a lot more than just the basic classes in the `java.lang` package. A substantial list of packages is loaded by JShell during startup and can be seen as the *special imports* that are visible through the **`/list -all`** command:

```
jshell> /list -all

  s1 : import java.io.*;
  s2 : import java.math.*;
  s3 : import java.net.*;
  s4 : import java.nio.file.*;
  s5 : import java.util.*;
  s6 : import java.util.concurrent.*;
  s7 : import java.util.function.*;
  s8 : import java.util.prefs.*;
  s9 : import java.util.regex.*;
 s10 : import java.util.stream.*;
```

Now that we've experimented with some basic Java statements and methods in JShell, let's see how to write a standalone program, which is going to execute a bit like a script, as you might find in Python.

## Simple Standalone Java Programs

Recent additions to the Java programming language are designed so that beginners can write their first programs without needing to understand language features designed for larger programs.

Instead of using a separate dialect of the language, these new features allow you to write simplified declarations for programs constituting just a single class. After becoming more familiar with the language and environment, you can move on to the classic forms that Java has always had—but the simplified versions are always there if you want to use them.

To explain the difference, let's look at a "Hello World" program written in the "classic" Java style, which you may have seen before:

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

There's a lot of boilerplate here, and a lot of Java language concepts that we're exposing all at once, but don't worry about that too much for now; it's about to get easier!

To run this code in the traditional way, we need to save it in a file called *HelloWorld.java*, then compile it with **`javac HelloWorld.java`**, and then run it with **`java HelloWorld`**. This is the process we met in "The Lifecycle of a Java Program".

However, from Java 25 onward we can greatly simplify the whole process of running basic programs. If we save this code into a file called *HW.java* (or any other filename that ends with *.java*):

```
void main() {
    IO.println("Hello, World!");
}
```

and then run it with **`java HW.java`**, then we'll see the same output as before. Under the hood, basically the same processing is occurring, but the new language features and tool enhancements are just making it easier for us.

For a Java source file to be eligible to be run directly like the preceding example, it must have a method called `main()`—this is called the *entry point* of the program and is where execution starts.

Although Java code is always contained in classes, to make things simpler for new programmers,

Java 25 allows classes to be *implicitly declared*. If a Java source file omits the class declaration, then the Java compiler will create a special, private class (and not disclose its name to the user) and use it to contain the code.

A second feature that might surprise programmers who already have some familiarity with Java is the appearance of `IO.println()` instead of the traditional `System.out.println()`. The `java.lang.IO` class is a new arrival with Java 25, and it provides a simplified subset of the functionality of Java's standard input and output streams. It exposes five public static methods—three for printing to console and two for reading user input—and no other functionality.

As we will see, these features are really only useful for simple programs, but it's intended to be a big benefit to developers who are still new to Java. It means that you can get coding in a "scripting style" similar to Python or JavaScript straightaway. Let's reuse an earlier example from JShell to form the basis of a new script:

```
LocalDateTime now = LocalDateTime.now();

void main() {
    IO.println("Start time: " + now);
    IO.println();
    IO.println("Tomorrow: " + now.plusDays(1).toLocalDate());
    IO.println("Yesterday: " + now.minusDays(1).toLocalDate());
    IO.println();
}
```

This script prints out the current date and time and then the date for tomorrow and yesterday.

It uses `now` as, effectively, a global variable (although, in reality, it's a field of our implicit class) that we can use throughout the main body of the program.

> **TIP**
>
> This reduction in mandatory scaffolding in Java 25 is a real win for Java usability. Being able to read, understand, and maintain existing codebases using the older, more verbose style is important, but for anyone starting from scratch today, these shorter, clearer idioms are an obvious win.

For our scripts to be really useful, we need to provide a way to pass parameters into them from the command line. Let's see how to do that in our next example, where you provide the name of a file and the program prints out its length:

```
void main(String[] args) {
    if (args.length == 0) {
        IO.println("No file specified");
        return;
    }
    var path = Path.of(args[0]);
    try {
        var contents = Files.readAllBytes(path);
```

```
            IO.println("Size: " + contents.length);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
```

We've introduced a couple of new language features here. The first is that the arguments to the program appear as an array of strings, and that we have changed the main method declaration to `void main(String[] args)` to allow us to access them.

As we saw in "Arrays", Java uses the hopefully familiar square brackets syntax for arrays, so the first argument provided to the program is `args[0]`.

The second, and potentially more important, point is that because this is a program that takes user input, we now need to spend time looking at the input, checking that it's valid and also protecting the program from possible errors. In this case, it means that the majority of the program is actually validation and error-protection code.

First, we check that the user provided us with an argument, and we exit cleanly if not. Then we generate a `Path` object from the provided filename—note that this is different from some other languages, because in Java we use an object, rather than just a plain string, to represent paths. This is for several reasons, such as helping with platform independence and being able to represent different types of paths (such as remote paths, paths to files contained inside ZIP files, etc.).

With our path object, we will be able to read the contents of the file that the user has specified. However, there are a number of things that could go wrong here—for example, the file might not exist, or it might be corrupt in some way so that we can't open it, or we may not have the correct permissions to open it.

Java handles this possibility with the conception of exceptions, which we met in "Checked and Unchecked Exceptions". In particular, the sorts of errors that we need to deal with here fall into the "could arise due to predictable but exceptional circumstances" category—i.e., they are checked exceptions, and the programmer must indicate how they will deal with them if they arise.

This is done with the `try { ... } catch (Exception x) { ... }` construction. The code within the `try` block runs, and if there is a problem, then an exception is *thrown* (or *raised*) and program execution continues in the `catch` block, where the program can recover from the error.

> **TIP**
>
> JavaScript has a very similar exception capability, and Python has `try: ... except:`. Both languages draw a lot of inspiration from Java's approach to exceptions, so if you've already used exceptions in one of those languages, they work very similarly.

In our case, we can't really do much to recover from the error, so we just print out the detailed error by calling the `printStackTrace()` method and then exit.

The simplified approach that we've met in this section using implicit classes is designed to provide an easy on-ramp. For reference, here are the rules regarding an implicitly declared class:

- It is a final top-level class in the unnamed package.
- It extends `Object` and does not implement any interfaces.
- It has a default constructor with no parameters and no other constructors.
- It has, as its members, the fields and methods in the compact source file.
- It must have a launchable main method; if it does not, a compile-time error is reported.

Don't worry if these don't all make sense yet—they will after reading the next couple of chapters. We're listing them here for developers who are coming back to Java after some time away and are curious about the rules for these new features.

Let's move on to see how to take the next step toward a full-fledged Java program.

## Java Source File Structure

From a practical standpoint, the unit of Java program structure you will be dealing with most often is the *.java* source file. This is the natural progression from implicit to explicitly declared classes. Java's syntax does add some additional overhead, but the extra ceremony quickly pays off for all but the smallest programs.

A Java file is the smallest unit of Java code that can be compiled by the Java compiler. A complete Java file consists of:

- An optional `package` directive
- Zero or more `import` or `import static` directives
- One or more type definitions—classes or other reference types

These elements can be interspersed with comments, of course, but they must appear in this order. This is all there is to a Java file. All Java statements (except the `package` and `import` directives, which are not true statements, and the specialized module descriptors we'll discuss in Chapter 14) must appear within methods, and all methods must appear within a type definition.

A special Java file named `module-info.java` is used only in declaring the structure and visibility of our packages in a modular Java application. These more advanced techniques and syntax are covered in detail in Chapter 14.

Java files have a couple of other important restrictions. First, each file can contain at most one top-level class that is declared `public`. A `public` class is one that is designed for use by other classes in other packages. A class can contain any number of nested or inner classes that are

`public`. We'll see more about the `public` modifier and nested classes in Chapter 4.

> **TIP**
>
> Regardless of whether your classes are `public` or not, it is good programming practice to define only one per file and to give the file the same name as the class.

The second restriction concerns the filename of a Java file. If a Java file contains a `public` class, the name of the file must be the same as the name of the class, with the extension *.java* appended. Therefore, if `Account` is defined as a `public` class, its source code must appear in a file named *Account.java*.

Let's rewrite one of our implicit classes into a full-fledged class, and we'll include the entire code of the class for completeness this time (although on future occasions we will probably elide some details):

```
package javanut.ch03;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;

public class Details {
    void main(String[] args) {
        if (args.length == 0) {
            System.err.println("No file specified");
            System.exit(1);
        }
        var path = Path.of(args[0]);
        try {
            var contents = Files.readAllBytes(path);
            System.out.println("Size: " + contents.length);
        } catch (IOException e) {
            System.err.println(e.getClass().getName() + ": "
              + e.getMessage());
        }
    }
}
```

We're now declaring an explicit public class, `Details`, and so it must live in a file called *Details.java*. It resides in the package `javanut.ch03`, so the file must be located in a directory *javanut/ch03* relative to a Java *source root*.

> **TIP**
>
> One very common way to lay out Java projects is to have a top-level project directory (or folder, in Windows terminology) and then create a source root of *src/main/java*. This is the standard convention for the two most common Java build tools (Apache Maven and Gradle) and so it has become a de facto standard for all Java projects. We recommend adopting it for all of your Java projects.

Next come the imports. When using implicit classes, Java helpfully includes a lot of packages automatically. This is how we were able to access types like `LocalDateTime`, `Path`, and `Files` without any additional ceremony. Now that we're declaring an explicit class, we also need to be explicit about our imports.

The next item is the class declaration, and we're following the best practice of "one top-level class per file." Within the class, we have the `main()` method and that's it, and the code is very similar to what was used in the implicit class. The only real change is that we've migrated away from the use of the `IO` class.

We've made this change because `IO` provides no way of accessing the standard error stream—by design. The idea is that well-behaved programs should send their error messages to the error stream, so this provides a useful indication to the programmer that their program has reached a sufficient size or complexity and that it's time to deal with I/O properly.

In our case, we're now using the `System.out` and `System.err` I/O objects directly, and we've also modified the exception handling to give a more helpful and less verbose error message.

> **NOTE**
>
> When migrating code away from using the `IO` class, best practice is to fully refactor—migrate all usages of the `IO` class to use the stream objects directly, to avoid any possibility of conflicts between the wrapped methods and the lower-level I/O streams.

This code can be run in one of two ways—either with the source code runner, **`java javanut/ch03/Details.java`**, or by first compiling the code **`javac javanut/ch03/Details.java`** and then executing the compiled class file **`java javanut.ch03.Details`**.

In the first case, we're using the source code runner and providing an exact path to the source code file. The second case starts up the JVM directly and gives it a fully qualified class name as an argument. The class name that is passed must contain an entry point (i.e., a suitable `main` method) for the program to be able to start executing.

In our examples so far, we have only used a single source code file—but the source code runner can be used to run programs constituting multiple source code files just as easily.

However, while this capability is helpful for smaller projects, or for developers who are new to Java, for full-scale projects we must eventually get to grips with the second approach, and that means understanding Java's source code compiler, which is the subject of the next section.

## The Source Code Compiler

As we've mentioned several times, when a Java file is compiled, each of the classes it defines is

compiled into a separate class file that contains Java bytecodes to be executed by the JVM. A class file has the same name as the class it defines, with the extension *.class* appended.

The Java runtime knows where the class files for the standard system classes are located and can load them as needed. However, our programs will usually require more than just the classes in the standard library, so the runtime will need to locate our code (and any extra dependencies as well).

When the Java runtime executes a program that wants to use a class named `javanut.ch03.Details`, it knows that the code for that class is located in a directory named *javanut/ch03*, and, by default, it looks in the current directory for a subdirectory of that name.

To tell the runtime to look in locations other than the current directory, you can use the `-classpath` option when invoking Java or set the `CLASSPATH` environment variable. For details, see the documentation for the Java executable, `java`, in Appendix B.

When running `javac`, it is tempting to compare it to the native compiler for languages like C++, Rust, or Go. However, this is not necessarily a particularly useful comparison.

As we discussed in Chapter 1, Java bytecode is not very similar to the machine code that natively compiled programs consist of.

Instead, class files have a relatively simple structure—as well as the code for individual methods in the class, it has sections such as the *constant pool*, which holds constant values in code (for example, names of classes, interfaces, and fields).

This is somewhat similar to the symbol table seen in shared libraries, but when the JVM executes a program, the constant pool table is used to refer to values symbolically rather than having to rely on the precise layout of memory structures at runtime.

Bytecode is versioned, with a major and minor version number (although the latter is not commonly used). In general, new versions of bytecode do not change the instruction set—the only new opcode that's ever been added is `invokedynamic`, which is used to implement lambda expressions and many other modern Java features.[2] Some Java versions have zero changes to the bytecode format, but the class file major number is still incremented along with the Java version for reasons of simplicity.

Java class files are backward compatible, so an old class file should always run on a newer Java version without recompilation. The source code compiler also includes the ability to generate code that targets an earlier class file version, as this is occasionally helpful when building libraries that are intended to run on a range of Java versions.

> **TIP**
>
> Generally speaking, library authors pick an LTS version of Java and use that as their baseline version. As of the start of 2026, an increasing number of authors are standardizing on Java 17 as Java 8 usage declines.

The version numbers are checked to ensure that the version of the JVM is not of a lower version than the one declared in the class file; if these are not compatible, an `UnsupportedClassVersionError` will be thrown, indicating the runtime is a lower version than the compiled class file.

These differences from native object code also extend to the compiler flags provided by `javac`. As Java bytecode is not heavily optimized, there is no need for the optimization and code emission flags that are often seen in C/C++ builds (i.e., no `-Ox` flags). Java does support compiling class files with debug information via the `-g` switch, but this feature is not commonly used.[3] In fact, many Java developers do not even know about the `-g` flag.

There are flags that control the behavior of warnings (`-nowarn` for mandatory warnings only and `-Werror` if all warnings should be treated as errors). This can be a helpful tool (although most professional programmers will have a similar capability as a part of their IDE or build tool). Let's take a look at a simple example that displays some simple flaws that cause compiler warnings:

```
public final strictfp class WarningSigns {
    private static final int ZERO = 0;

    static void warnDivZero() {
        var val = 10 / ZERO;
        IO.println("Val: " + val);
    }

    void warnCasts() {
        var hello = "Hello";
        String s = (String) hello;
        IO.println("Hello: " + s);
    }

    void main() {
        this.warnDivZero();
        warnCasts();
    }
}
```

We can compile this with warnings upgraded to errors to be completely clear about the problems:

```
$ javac -Xlint:all -Werror src/main/java/javanut/ch03/WarningSigns.java
src/main/java/javanut/ch03/WarningSigns.java:3: warning: [strictfp] as
  of release 17, all floating-point expressions are evaluated strictly
  and 'strictfp' is not required
public final strictfp class WarningSigns {
                      ^
src/main/java/javanut/ch03/WarningSigns.java:8: warning: [divzero]
  division by zero
        var val = 10 / ZERO;
                       ^
src/main/java/javanut/ch03/WarningSigns.java:14: warning: [cast]
  redundant cast to String
        String s = (String) hello;
```

```
                  ^
src/main/java/javanut/ch03/WarningSigns.java:19: warning: [static]
  static method should be qualified by type name, WarningSigns,
  instead of by an expression
        this.warnDivZero();
            ^
error: warnings found and -Werror specified
1 error
4 warnings
```

In practice, most professional Java projects do not often interact directly with `javac` very much. Instead, one of the Java build tools will be used instead. However, it's useful to have at least a basic understanding of the source code compiler and its most common flags.

## Executing Full Java Programs

To recap, a Java program consists of a set of interacting class definitions, but not every Java class or Java file defines a program. To create an executable Java program, you must define a class that has a `main()` method to act as an entry point to execution. This can be defined either as a static or an instance method. For a static method, this is traditionally defined as:

```
public static void main(String[] args);
```

This method is passed an array of strings (if required) and returns no value. However, with Java 25, the range of acceptable entry point methods has been greatly broadened. One of the remaining restrictions is that the `main()` method must not be private, as we can see:

```
public class Main1 {
    private static void main() {
        IO.println("Hello World!");
    }
}
```

When we try to run this code, we get an error:

```
$ java nutshell-examples/src/main/java/javanut/ch03/Main1.java
error: can't find main(String[]) or main() method in class:
  javanut.ch03.Main1
```

Note that it is perfectly possible to compile this code, as it's legal to declare a private method called `main()`—it's just not a valid entry point.

For instance methods, there is one more restriction: the class containing the entry point must have a nonprivate constructor (or an implicitly declared default constructor) as the class is instantiated and an object created before the `main()` method is called on the object.

In either case, when `main()` returns, the Java interpreter exits (unless `main()` has created separate threads, in which case the interpreter waits for all those threads to exit).

The general rule is that to run a Java program, you run the Java executable, `java`, specifying the fully qualified name of the class that contains the entry point. Note that you specify the name of the class, *not* the path to the class file that contains the class. You will also need to specify the `-classpath` option (or `-cp`) to tell the interpreter where to look for the classes needed by the program that are outside the Java standard library.

Let's break down the following command into its parts:

```
java -classpath classes/ javanut.ch03.Details filename.txt
```

`java` is the command to run the JVM, `-classpath classes/` tells the interpreter where to look for class files. `javanut.ch03.Details` is the name of the class that defines the `main()` method. Finally, `filename.txt` is a string that is passed to that `main()` method as the single element of an array of `String` objects.

The classpath piece of the command comprises a list of paths separated by a `:` (or `;` on Windows) where classes that the program requires can be found. As an alternative to a directory of classes, a Java archive (JAR) file can be specified. These files have a *.jar* extension by default and are a ZIP archive with some optional additional metadata.

The usual case when running a production Java application is to specify a (potentially long) list of *.jar* files on the classpath, each of which represents a dependency of the application.

> **TIP**
>
> When using a Java build tool, such as Maven, there is often a specialized command to autogenerate the classpath for the user for inclusion in startup scripts.

There is an easier way to run programs, however. If a program and its non-JDK dependencies have been properly bundled into a single *.jar* file (commonly called an *uberjar*), then you can run the program simply by specifying the name of the JAR.

For example, to start up a log analyzer program that has been packaged with all non-JDK dependencies into a single JAR, we would execute a command like this:

```
java -jar /usr/local/log-analyzer/log-analyzer.jar
```

To go even further, some operating systems make JAR files automatically executable. On those systems, you can simply say:

```
/usr/local/log-analyzer/log-analyzer.jar
```

Finally, there is the option of executing a Java *module* (a relatively advanced concept that we will introduce in Chapter 14). For the sake of completeness, we'll include the syntax here, although it isn't something that you need to be too concerned about if this is your first encounter

with Java:

```
java -m mymodule/MyMain  # explicitly specify the MyMain class
java -m mymodule  # the module's default class will be used
```

This will execute the main class `MyMain` in a module `mymodule` or, if no class name is given, the default class in the module (if there is one). In other words, `MyMain` can be used when the default class is not specified by the module, or to override the default value when it has been specified.

## Summary

We have looked at various ways to execute Java code, starting with the JShell interactive environment and then moving on to the source code runner. From there, we introduced the traditional compilation-execution model and gave a primer on the source code compiler. We also covered various improvements made in Java 25 to help newcomers. To round out the chapter, we discussed the different ways to execute Java programs.

These different methods are intended to allow the newcomer to Java to grow at their own speed—starting with JShell and building toward the full-fledged traditional model of Java program execution.

In the next chapter, we will discuss the classes and objects that are so central to Java and the basics of Java's approach to object-oriented programming.

---

1 JShell is not available for Java 8 and earlier.

2 The `jsr` / `ret` combination has also been declared as obsolete and is disallowed in modern bytecode versions, but old classes containing them will still run.

3 This is largely because Java is usually developed in a modern IDE, which has superior debugging capabilities than those provided by, say, `gdb`.

# Chapter 4. Object-Oriented Programming in Java

Now that we've covered fundamental Java syntax, and how to run our programs, we are ready to begin object-oriented programming in Java. All Java programs use objects, and the type of an object is defined by its *class* or *interface*. Every Java program is defined as a class, and nontrivial programs include a number of classes and interface definitions.

This chapter explains how to define new classes (and an important special case, called *record classes* or *records*) and how to do object-oriented programming with them. We also introduce the concept of an interface, but a full discussion of interfaces and Java's type system is deferred until Chapter 5.

> **NOTE**
>
> If you have experience with OO programming, however, be careful. The term "object-oriented" has different meanings in different languages. Don't assume that Java works the same way as your other favorite OO languages. (This is particularly true for JavaScript or Python programmers.)

This is a fairly lengthy chapter, so let's begin with an overview and some definitions.

## Overview of Classes and Records

Classes are the most fundamental structural element of all Java programs. You cannot write Java code without defining a class. All Java statements appear within classes, and all methods are implemented within classes.

### Basic OO Definitions

Here are some important definitions:

*Class*

> A *class* is a collection of data fields that holds values, along with methods that operate on those values. A class defines a new reference type, such as the `Account` type defined in Chapter 2.
>
> The `Account` class defines a type that represents customer accounts within a banking system.

*Object*

An *object* is an *instance* of a class.

An `Account` object is a value of that type: it represents a specific customer bank account.

Objects are often created by *instantiating* a class with the `new` keyword and a constructor invocation, as shown here:

```
Account a = new Account("John Smith", 100, 1144789);
```

Constructors are covered in detail later in this chapter in "Creating and Initializing Objects".

A class definition consists of a *signature* and a *body*. The class signature defines the name of the class (and may also specify other important information). The body of a class is a set of *members* enclosed in curly braces. The members of a class usually include fields and methods and may also include constructors, initializers, and nested types.

Members can be *static* or nonstatic. A static member belongs to the class itself, while a nonstatic member is associated with the instances of a class (see "Fields and Methods").

> **NOTE**
>
> There are four very common kinds of members—class fields, class methods, instance fields, and instance methods. The majority of work done with Java involves interacting with these kinds of members.

The signature of a class may declare that the class *extends* another class. The extended class is known as the *superclass*, and the extension is known as the *subclass*. A subclass *inherits* the members of its superclass and may declare new members or *override* inherited methods with new implementations.

The members of a class may have the *access modifiers* `public`, `protected`, or `private`.[1] These modifiers specify the members' visibility and accessibility to clients and to subclasses. This allows classes to control access to members that are not part of their public API. This ability to hide members enables an object-oriented design (OOD) technique known as *data encapsulation*, which we discuss in "Data Hiding and Encapsulation".

## Records

From Java 17 onward, the Java language also includes support for records—which are a special kind of class that has additional semantic guarantees that general classes do not.

Specifically, a record guarantees that the instance fields precisely define the only meaningful state of an object of that type. This can be expressed as the design principle (or *pattern*) that the record class is a *data carrier* or "just holds fields." Agreeing to this principle imposes constraints on programmers, but it also frees programmers from needing to be explicit about some design details and to focus on more important concerns.

In its simplest form, a record class is defined like this:

```
/** Represents a point in 2-dimensional space */
public record Point(double x, double y) {}
```

There is no need to explicitly declare a constructor or accessor methods for the fields—for a record class, the compiler automatically generates these members and adds them to the class definition. The accessor methods are named exactly the same as the underlying fields they provide access to. Record classes are always implicitly final and cannot be extended.

> **TIP**
>
> It is possible to add additional methods to a record, but it is not necessary to do so if all that is needed is the basic data carrier form.

Instances of records are created and instantiated in the same way as regular classes, and we can call the accessors on the objects we create:

```
// Create a Point object representing (2, -3.5).
// Declare a variable p and store a reference to the new Point object
Point p = new Point(2.0, -3.5);
double x = p.x();                   // Read a field of the object
```

One other aspect of records is that they are always immutable. Once created, the value of a record's fields cannot be altered. This means that there is no need for setter methods for the fields, as they cannot be modified.

The contract that Java records have is that the parameter name (as specified in the record declaration), the field name, and the method name are all identical: if there's a record parameter `x` of type `double`, then the class has a field called `x` of type `double` and an instance method called `x()` that returns `double`.

Records have certain other methods that are also automatically generated by the compiler. We will have more to say about them in Chapter 6 when we discuss how to use records as part of OOD.

## Interfaces

The signature of a class may also declare that the class *implements* one or more interfaces. An *interface* is a reference type similar to a class that defines method signatures but does not usually include method bodies to implement the methods.

The design intent of an interface is to provide a contract—a guarantee that a class definitely implements some group of methods. This is normally done by insisting that classes that implement a given interface provide implementations for all of the methods in the interface.

However, in some cases, interfaces may provide a `default` implementation, which will be

used by all implementing classes that do not provide an implementation of the method. This renders the method optional, as classes do not have to implement the method, but retains the flexibility to allow them to specify their own implementation if they wish.

We can summarize this by saying that a class that implements an interface is required to provide bodies for the interface's nondefault methods. Instances of a class that implement an interface are also instances of the interface type.

## Other Reference Types

Classes and interfaces are the most important of the five fundamental reference types defined by Java. Arrays, enumerated types (or *enums*), and annotation types (usually just called *annotations*) are the other three. Arrays are covered in Chapter 2. Enums are a specialized kind of class, and annotations are a specialized kind of interface—both are discussed later in Chapter 5, along with a full discussion of interfaces.

## Class Definition Syntax

As we saw in "Defining a Class", the simplest form of a class definition consists of the keyword `class` followed by the name of the class and a set of class members within curly braces.

The `class` keyword may be preceded by modifier keywords and annotations. If the class extends another class, the class name is followed by the `extends` keyword and the name of the class being extended. If the class implements one or more interfaces, then the class name or the `extends` clause is followed by the `implements` keyword and a comma-separated list of interface names.

For example, the declaration of the `Integer` class in `java.lang`, which both extends *and* implements, looks like this:

```
public final class Integer extends Number
                           implements Serializable, Comparable {
    // class members go here
}
```

Java also includes the ability to declare *generic classes* that allow an entire family of types to be created from a single class declaration. We will meet this feature, along with its supporting mechanisms (such as *type parameters* and *wildcards*), in Chapter 5.

Class declarations may include modifier keywords. In addition to the access control modifiers (`public`, `protected`, etc.), these include:

`abstract`

> An `abstract` class is one whose implementation is incomplete and cannot be instantiated. Any class with one or more `abstract` methods must be declared `abstract`. Abstract classes are discussed in "Abstract Classes and Methods".

`final`

The `final` modifier specifies that the class may not be extended. A class cannot be declared to be both `abstract` and `final`.

`sealed`

Sealed classes are those that may be extended only by a known set of subclasses. Sealed classes provide a midpoint between `final` classes and the default, which is classes that are open for extension. The use of sealed classes is discussed in more detail in Chapter 6. Note that a class *can* be both `abstract` and `sealed`, and this is in fact a very common use case. Sealed classes are available only in Java 17 and above.

`strictfp`

This is a legacy keyword and can be safely ignored from Java 17 onward.

## Fields and Methods

A class can be viewed as a collection of data (also referred to as *state*) and code to operate on that state (also referred to as *behavior*). The data is stored in fields, and the code is organized into methods.

This section covers fields and methods, the two most important kinds of class members. Fields and methods come in two distinct types: class members (also known as *static members*) are associated with the class itself, while instance members are associated with individual instances of the class (i.e., with objects). This gives us four kinds of members:

- Class fields
- Class methods
- Instance fields
- Instance methods

The simple class definition for the class `Circle`, shown in Example 4-1, contains all four types of members.

*Example 4-1. A simple class and its members*

```
public class Circle {
  // A class field
  public static final double PI = 3.14159;     // A useful constant

  // A class method: just compute a value based on the arguments
  public static double radiansToDegrees(double radians) {
    return radians * 180 / PI;
  }

  // An instance field
```

```
  public double r;                  // The radius of the circle

  // Two instance methods: operate on an object's instance fields

  // Compute the area of the circle
  public double area() {
    return PI * r * r;
  }

  // Compute the circumference of the circle
  public double circumference() {
    return 2 * PI * r;
  }
}
```

> **WARNING**
>
> It is not good practice to declare a `public` instance field (such as `r` in our example) in your classes. It would be much better to have a private field `r` and a method `radius()` (or `r()`) to provide access to it. The reason for this will be explained later, in "Data Hiding and Encapsulation". For now, we use a public field simply to give examples of how to work with instance fields.

The following sections explain all four common kinds of members. First, we cover the declaration syntax for fields. (The syntax for declaring methods is covered later in this chapter in "Data Hiding and Encapsulation".)

## Field Declaration Syntax

Field declaration syntax is much like the syntax for declaring local variables (see Chapter 2) except that field definitions may also include modifiers. The simplest field declaration consists of the field type followed by the field name.

The type may be preceded by zero or more modifier keywords or annotations, and the name may be followed by an equals sign and initializer expression that provides the initial value of the field. If two or more fields share the same type and modifiers, the type may be followed by a comma-separated list of field names and initializers. Here are some valid field declarations:

```
int x = 1;
private String name;
public static final int DAYS_PER_WEEK = 7;
String[] dayNames = new String[DAYS_PER_WEEK];
private int a = 17, b = 37, c = 53;
```

Field declarations can be modified by including zero or more of the following keywords:

`public`, `protected`, `private`

These access modifiers specify whether and where a field can be used outside of the class that defines it.

`static`

If present, this modifier specifies that the field is associated with the defining class itself rather than with each instance of the class. See “Class Fields”.

`final`

This modifier specifies that once the field has been initialized, its value may never be changed. This is a form of immutability, but if the value is a reference—e.g., to a collection object—then this does not imply that the contents of the collection are immutable. Fields that are both `static` and `final` are compile-time constants that `javac` may inline. Note that the `final` keyword can be used on many other language elements, not just fields.

`transient`

This modifier specifies that a field is not part of the persistent state of an object and that it need not be serialized along with the rest of the object. This modifier is very rarely seen.

`volatile`

This modifier indicates that the field has extra semantics for concurrent use by two or more threads. The `volatile` modifier says that the value of a field must always be read from and flushed to main memory, and that it may not be cached by a thread (in a register or CPU cache). See Chapter 12 for more details.

There are some additional details about static fields, which we should address before moving on to discuss methods.

## Class Fields

A *class field* (aka *static field*) is associated with the class in which it is defined rather than with an instance of the class. The following line declares a class field:

```
public static final double PI = 3.14159;
```

This line declares a field of type `double` named `PI` and assigns it a value of 3.14159.

The `static` modifier says that the field is a class field. Class fields are sometimes called static fields because of this `static` modifier. The `final` modifier says that the value of the field cannot be reassigned directly. Because the field `PI` represents a constant, we declare it `final` so that it cannot be changed.

It is a convention in Java (and many other languages) that constants are named with capital letters, which is why our field is named `PI`, not `pi`. Defining constants like this is a common use for class fields, meaning that the `static` and `final` modifiers are often used together. Not all class fields are constants, however. In other words, a field can be declared `static` without

being declared `final`.

> **WARNING**
>
> The use of public fields that are not `final` is a classic sign of badly engineered code (sometimes called a *code smell*)—because multiple threads could update the field and cause behavior that is extremely hard to debug. Beginning Java programmers should not use public fields that are not final. See Chapter 12 for more details.

A public static field is essentially Java's version of a global variable. The names of class fields are qualified by the unique names of the classes that contain them, however. Thus, Java does not suffer from the name collisions that can affect other languages when different modules of code define global variables with the same name.

The key point to understand about a static field is that there is only a single copy of it. This field is associated with the class itself, not with instances of the class. If you look at the various methods of the `Circle` class, you'll see that they use this field. From inside the `Circle` class, the field can be referred to simply as `PI`. Outside the class, however, both class and field names are required to uniquely specify the field. Methods that are not part of `Circle` access this field as `Circle.PI`.

## Class Methods

As with class fields, *class methods* are declared with the `static` modifier, so they are also known as *static methods*:

```
public static double radiansToDegrees(double rads) {
  return rads * 180 / PI;
}
```

This line declares a class method named `radiansToDegrees()`. It has a single parameter of type `double` and returns a `double` value.

Like class fields, class methods are associated with a class, rather than with an object. When invoking a class method from code that exists outside the class, you must specify both the name of the class and the method. For example:

```
// How many degrees is 2.0 radians?
double d = Circle.radiansToDegrees(2.0);
```

If you want to invoke a class method from inside the class in which it is defined, you don't have to specify the class name. You can also shorten the amount of typing required via the use of a static import (as discussed in "Importing Static Members").

Note that the body of our `Circle.radiansToDegrees()` method uses the class field `PI`. A class method can use any class fields and class methods of its own class (or of any other class that is visible to it).

A class method cannot use any instance fields or instance methods because class methods are not associated with an instance of the class. In other words, although the `radiansToDegrees()` method is defined in the `Circle` class, it cannot use the instance part of any `Circle` objects.

As we discussed earlier, a class field is essentially a global variable. In a similar way, a class method is a global method, or global function. Although `radiansToDegrees()` does not operate on `Circle` objects, it is defined within the `Circle` class because it is a utility method that is sometimes useful when you're working with circles, and so it makes sense to package it along with the other functionality of the `Circle` class.

## Instance Fields

Any field declared without the `static` modifier is an *instance field*:

```
public double r;    // The radius of the circle
```

Instance fields are associated with instances of the class, so every `Circle` object we create has its own copy of the `double` field `r`. In our example, `r` represents the radius of a specific circle. Each `Circle` object can have a radius independent of all other `Circle` objects.

Inside a class definition, instance fields are referred to by name alone. You can see an example of this if you look at the method body of the `circumference()` instance method. In code outside the class, the name of an instance field must be prefixed with a reference to the object that contains it. For example, if the variable `c` holds a reference to a `Circle` object, we use the expression `c.r` to refer to the radius of that circle:

```
Circle c = new Circle(); // Create a Circle object; store a ref in c
c.r = 2.0;               // Assign a value to its instance field r
Circle d = new Circle(); // Create a different Circle object
d.r = c.r * 2;           // Make this one twice as big
```

Instance fields are key to object-oriented programming. Instance fields hold the state of an object; the values of those fields make one object distinct from another.

## Instance Methods

An *instance method* operates on a specific instance of a class (an object), and any method not declared with the `static` keyword is automatically an instance method.

Instance methods are the feature that makes object-oriented programming start to get interesting. The `Circle` class defined in Example 4-1 contains two instance methods, `area()` and `circumference()`, that compute and return the area and circumference of the circle represented by a given `Circle` object.

To use an instance method from outside the class in which it is defined, we must prefix it with a reference to the instance that is to be operated on. For example:

```
// Create a Circle object; store in variable c
Circle c = new Circle();
c.r = 2.0;                  // Set an instance field of the object
double a = c.area();        // Invoke an instance method of the object
```

> **NOTE**
>
> This is why it is called object-oriented programming; the object is the focus here, not the method call.

From within an instance method, we naturally have access to all the instance fields that belong to the object the method was called on. Recall that an object is often best considered to be a bundle containing state (represented as the fields of the object) and behavior (the methods to act on that state).

> **NOTE**
>
> The bodies of the `area()` and `circumference()` methods both use the class field `PI`. We saw earlier that class methods can use only class fields and class methods, not instance fields or methods. Instance methods are not restricted in this way: they can use any member of a class, whether it is declared `static` or not.

All instance methods are implemented by using an implicit parameter not shown in the method signature. The implicit argument is named `this`; it holds a reference to the object through which the method is invoked. In our example, the `this` object is a `Circle`.[2]

## How the this Reference Works

The implicit `this` parameter is not shown in method signatures because it is usually not needed; whenever a Java method accesses the instance fields in its class, it is implicit that it is accessing fields in the object referred to by the `this` parameter. The same is true when an instance method invokes another instance method in the same class—it's taken that this means "call the instance method on the current object."

However, you can use the `this` keyword explicitly when you want to make it clear that a method is accessing its own fields and/or methods. For example, we can rewrite the `area()` method to use `this` explicitly to refer to instance fields:

```
public double area() { return Circle.PI * this.r * this.r; }
```

This code also uses the class name explicitly to refer to class field `PI`. In a method this simple, it is not normally necessary to be quite so explicit. In more complicated cases, however, you may sometimes find that it increases the clarity of your code to use an explicit `this` even when it is not strictly required.

In some cases, the `this` keyword *is* required, however. For example, when a method parameter

or local variable in a method has the same name as one of the fields of the class, you must use `this` to refer to the field. This is because the name used alone refers to the method parameter or local variable, as discussed in "Lexical Scoping and Local Variables".

For example, we can add the following method to the `Circle` class:

```
public void setRadius(double r) {
  this.r = r;      // Assign the argument (r) to the field (this.r)
                   // Note that writing r = r is a bug
}
```

Some developers will deliberately choose the names of their method arguments in such a way that they don't clash with field names, so the use of `this` can largely be avoided. However, accessor methods (setters) generated by any of the major Java IDEs will typically use the `this.x = x` style shown here.

A second common usage of `this` is when we're calling a method on another object but we want to pass a reference to the current object. This is often the case in situations like callback handlers, where the other object needs to hold a reference to the current object and then call methods on it later when some criteria is met.

Finally, note that while instance methods can use the `this` keyword, class methods cannot because class methods are not associated with individual objects—attempting to do this will cause a compiler error.

## Creating and Initializing Objects

Now that we've covered fields and methods, let's move on to other important members of a class. In particular, we'll look at constructors—these are class members whose job is to initialize the fields of a class as new instances of the class are created.

Take another look at how we've been creating `Circle` objects:

```
Circle c = new Circle();
```

This can easily be read as creating a new instance of `Circle`, by calling something that looks a bit like a method. In fact, `Circle()` is an example of a *constructor*. This is a member of a class that has the same name as the class and a body like a method.

Here's how a constructor works. The `new` operator indicates that we need to create a new instance of the class. First of all, memory is allocated (in the Java heap) to hold the new object instance. Then, the constructor body is called, with any arguments that have been specified. The constructor uses these arguments to do whatever initialization of the new object is necessary.

Every class in Java has at least one constructor, and its purpose is to perform any necessary initialization for a new object. If the programmer does not explicitly define a constructor for a class, the `javac` compiler automatically creates a constructor (called the *default constructor*)

that takes no arguments and performs no special initialization. The `Circle` class seen in Example 4-1 used this mechanism to automatically declare a constructor.

## Defining a Constructor

There is some obvious initialization we could do for our `Circle` objects, so let's define a constructor. Example 4-2 shows a new definition for `Circle` that contains a constructor that lets us specify the radius of a new `Circle` object. We've also taken the opportunity to make the field `r` protected (to prevent access to it from arbitrary objects).

*Example 4-2. A constructor for the Circle class*

```
public class Circle {
    public static final double PI = 3.14159;  // A constant
    // An instance field that holds the radius of the circle
    private double r;

    // The constructor: initialize the radius field
    public Circle(double r) { this.r = r; }

    // The instance methods: compute values based on the radius
    public double circumference() { return 2 * PI * r; }
    public double area() { return PI * r * r; }
    public double radius() { return r; }
}
```

When we relied on the default constructor supplied by the compiler, we had to write code like this to initialize the radius explicitly:

```
Circle c = new Circle();
c.r = 0.25;
```

With the new constructor, the initialization becomes part of the object creation step:

```
Circle c = new Circle(0.25);
```

This allows us to make the field `r` private, and this often allows it to also be marked `final`, which further protects it from inadvertent modification. In our current example, we haven't marked the field as `final`, but in practice it is—because the field is private (and so cannot be modified from outside the `Circle` class) and none of the methods on `Circle` contain code that modifies `r`. So, we could, in fact, also add the `final` modifier here—and it is good practice to do so whenever possible.

Here are some basics regarding naming, declaring, and writing constructors:

- The constructor name is always the same as the class name.
- A constructor is declared without a return type (not even the `void` placeholder).
- The body of a constructor is the code that initializes the object. You can think of this as setting up the contents of the `this` reference.

- A constructor does not return `this` (or any other value).
- If we declare an explicit constructor like this one, then we now must specify a radius when declaring a new `Circle` object.

One point requires a little more clarification: if we declare *any* constructor, then the default constructor is no longer automatically created. This is because the compiler's convention is that an explicitly declared constructor indicates that the programmer has taken responsibility for defining what parameters are necessary for object creation.

## Defining Multiple Constructors

Sometimes you want to initialize an object in a number of different ways, depending on what is most convenient in a particular circumstance. For example, we might want to initialize the radius of a circle to a specified value or a reasonable default value. Here's how we can define two constructors for `Circle`:

```
public Circle() { r = 1.0; }
public Circle(double r) { this.r = r; }
```

Because our `Circle` class has only a single instance field, we can't initialize it in too many ways, of course. But in more complex classes, it is often convenient to define a variety of constructors.

It is perfectly legal to define multiple constructors for a class, as long as each constructor has a different parameter list. The compiler determines which constructor you wish to use based on the number and type of arguments you supply. This ability to define multiple constructors is analogous to method overloading.

## Invoking One Constructor from Another

A specialized use of the `this` keyword arises when a class has multiple constructors; it can be used from a constructor to invoke one of the other constructors of the same class. In other words, we can rewrite the two previous `Circle` constructors as follows:

```
// This is the basic constructor: initialize the radius
public Circle(double r) { this.r = r; }
// This constructor uses this() to invoke the constructor above
public Circle() { this(1.0); }
```

This is a useful technique in complex situations where multiple constructors for the same class would otherwise have to duplicate a lot of the same initialization code.

## Field Defaults and Initializers

The fields of a class do not necessarily require initialization. If their initial values are not specified, the fields are automatically initialized to a default value—`false`, `\u0000`, `0`, `0.0`,

or `null`, depending on their type (see Table 2-1 for more details). These default values are specified by the Java Language Specification and apply to both instance fields and class fields.

> **TIP**
>
> The default values are essentially the “natural” interpretation of the zero bit pattern for each type.

If the default field value is not appropriate for your field, you can instead explicitly provide a different initial value. For example:

```
public static final double PI = 3.14159;
public double r = 1.0;
```

Field declarations are not part of any method. Instead, the Java compiler generates initialization code for the field automatically and puts it into all the constructors for the class. The initialization code is inserted into a constructor before the author’s constructor begins in the order in which it appears in the source code, which means that a field initializer can use the initial values of any fields declared before it.

Consider the following code excerpt, which shows a constructor and two instance fields of a hypothetical class:

```
public class SampleClass {
  public int len = 10;
  public int[] table = new int[len];

  public SampleClass() {
    for(int i = 0; i < len; i = i + 1) {
        table[i] = i;
    }
  }

  // The rest of the class is omitted...
}
```

In this case, the code generated by `javac` for the constructor is actually equivalent to:

```
public SampleClass() {
  len = 10;
  table = new int[len];
  for(int i = 0; i < len; i = i + 1) {
      table[i] = i;
  }
}
```

This raises the question—if instance fields are initialized in the constructor, then where are class fields initialized? These fields are associated with the class, even if no instances of the class are ever created. Logically, this means they need to be initialized even before a constructor is called.

To support this, `javac` generates a class initialization method automatically for every class. Class fields are initialized in the body of this method, which is invoked exactly once before the class is first used (when the class is first loaded by the JVM).

As with instance field initialization, class field initialization expressions are inserted into the class initialization method in the order in which they appear in the source code. This means that the initialization expression for a class field can use the class fields declared before it.

The class initialization method is an internal method that is hidden from Java programmers. In the class file, it bears the name `<clinit>` (and you could see this method by, for example, examining the class file with `javap`—see Appendix B for more details on how to use `javap` to do this).

So far, we've seen that objects can be initialized through the initialization expressions for their fields and by arbitrary code in their constructors. A class has a class initialization method (which is like a constructor), but we cannot explicitly define the body of this method in Java, although it is perfectly legal to do so in bytecode.

Java does however allow us to express class initialization with a construct known as a *static initializer*. A static initializer is simply the keyword `static` followed by a block of code in curly braces. A static initializer can appear in a class definition anywhere a field or method definition can appear. For example, consider the following code that performs some nontrivial initialization for two class fields:

```
// We can draw the outline of a circle using trigonometric functions
// Trigonometry is slow, though, so we pre-compute a bunch of values
public class TrigCircle {
  // Here are our static lookup tables and their own initializers
  private static final int NUMPTS = 500;
  private static double sines[] = new double[NUMPTS];
  private static double cosines[] = new double[NUMPTS];

  // Here's a static initializer that fills in the arrays
  static {
    double x = 0.0;
    double delta_x = (Circle.PI / 2)/(NUMPTS - 1);
    for(int i = 0, x = 0.0; i < NUMPTS; i = i + 1, x += delta_x) {
      sines[i] = Math.sin(x);
      cosines[i] = Math.cos(x);
    }
  }
  // The rest of the class is omitted...
}
```

A class can have any number of static initializers. The body of each initializer block is incorporated into the class initialization method, along with any static field initialization expressions. A static initializer is like a class method in that it cannot use the `this` keyword or any instance fields or instance methods of the class.

## Record Constructors

Record classes, available from Java 17 onward, implicitly define one constructor: the canonical constructor defined by the parameter list. There may be circumstances, however, when developers need to provide additional (aka *auxiliary*) constructors for record classes—for example, to provide default values for some of the record parameters, as in:

```
public record Point(double x, double y) {
    /** Constructor simulates default parameters */
    public Point() {
        this(0.0, 0.0);
    }
}
```

Records also provide for another refinement to class constructors: the *compact constructor*. This is used when some sort of validation or other checking code is helpful for creating valid record objects. For example:

```
/** Represents a point in 2-dimensional space */
public record Point(double x, double y) {
    /** Compact constructor provides validation */
    public Point {
        if (Double.isNaN(x) || Double.isNaN(y)) {
            throw new IllegalArgumentException("Illegal NaN");
        }
    }
}
```

Note that in the compact constructor syntax, the parameter list does not need to be repeated (since it is inferred from the record declaration) and the parameters (in our example, `x` and `y`) are already in scope. Compact constructors, like the canonical constructor, also implicitly initialize the fields from the parameter values.

## Subclasses and Inheritance

The `Circle` defined earlier is a simple class that distinguishes circle objects only by their radii. Suppose, instead, that we want to represent circles that have both a size and a position. For example, a circle of radius 1.0 centered at point (0, 0) in the Cartesian plane is different from the circle of radius 1.0 centered at point (1, 2). To do this, we need a new class, which we'll call `PositionedCircle`.

We'd like to add the ability to represent the position of a circle without losing any of the existing functionality of the `Circle` class. We do this by defining `PositionedCircle` as a subclass of `Circle` so that `PositionedCircle` *inherits* the fields and methods of its superclass, `Circle`. The ability to add functionality to a class by subclassing, or extending, is central to the object-oriented programming paradigm.

### Extending a Class

In Example 4-3, we show how we can implement `PositionedCircle` as a subclass of the `Circle` class.

*Example 4-3. Extending the `Circle` class*

```
package javanut.ch04.shapes; // Specify a package for the class

public final class PositionedCircle extends Circle {     // The class is
    // still public. We automatically inherit the fields and methods of
    // Circle, so we only have to put the new stuff here.
    // New instance fields that store the center point of the circle
    private final double cx, cy;

    // The nondefault constructor
    public PositionedCircle(double r, double x, double y) {
        this.cx = x;    // Initialize the instance field cx
        this.cy = y;    // Initialize the instance field cy
        super(r);       // Invoke superclass constructor Circle()
    }

    public double getCenterX() {
        return cx;
    }

    public double getCenterY() {
        return cy;
    }

    // The area() and circumference() methods are inherited from Circle
    // A new instance method checks whether a point is inside the circle
    // Note that it uses the inherited instance field r
    public boolean isInside(double x, double y) {
        double dx = x - cx, dy = y - cy;
                                        // Distance from center
        double distance = Math.sqrt(dx * dx + dy * dy);
                                        // Pythagorean theorem
        return (distance < r);
                                        // Returns true or false
    }
}
```

Note the use of the keyword `extends` in the first line of Example 4-3. This keyword tells Java that `PositionedCircle` extends, or subclasses, `Circle`, meaning that it inherits the fields and methods of that class.

The definition of the `isInside()` method shows field inheritance; this method uses the field `r` (defined by the `Circle` class) as if it were defined right in `PositionedCircle` itself. `PositionedCircle` also inherits the methods of `Circle`. Therefore, if we have a `PositionedCircle` object referenced by variable `pc`, we can say:

```
double ratio = pc.circumference() / pc.area();
```

This works just as if the `area()` and `circumference()` methods were defined in `PositionedCircle` itself.

Another feature of subclassing is that every `PositionedCircle` object is also a perfectly legal `Circle` object. If `pc` refers to a `PositionedCircle` object, we can assign it to a `Circle` variable and forget all about its extra positioning capabilities:

```
// Unit circle at the origin
PositionedCircle pc = new PositionedCircle(1.0, 0.0, 0.0);
Circle c = pc;     // Assigned to a Circle variable without casting
```

This assignment of a `PositionedCircle` object to a `Circle` variable can be done without a cast. As we discussed in Chapter 2, a conversion like this is always legal. The value held in the `Circle` variable `c` is still a valid `PositionedCircle` object, but the compiler cannot know this for sure, so it doesn't allow us to do the opposite (narrowing) conversion without a cast:

```
// Narrowing conversions require a cast (and a JVM runtime check)
PositionedCircle pc2 = (PositionedCircle) c;
boolean inside = ((PositionedCircle) c).isInside(0.0, 0.0);
```

This distinction is covered in more detail in "Nested Types", where we talk about the distinction between the compile and runtime type of an object.

You may have noticed that `PositionedCircle` was declared with the `final` modifier. This means that it cannot be extended or subclassed (`java.lang.String` is another example of a `final` class). Declaring a class `final` prevents unwanted extensions to the class: if you invoke a method on a `String` object, you know that the method is the one defined by the `String` class itself, even if the `String` is passed to you from some unknown outside source.

In general, many of the classes that Java developers create should be `final`. Think carefully about whether it will make sense to allow other (possibly unknown) code to extend your classes—if it doesn't, then disallow the mechanism by declaring your classes `final`.

## Superclasses, Object, and the Class Hierarchy

In our example, `PositionedCircle` is a subclass of `Circle`. We can also say that `Circle` is the superclass of `PositionedCircle`. The superclass of a class is specified in its `extends` clause, and a class may have only a single direct superclass:

```
public class PositionedCircle extends Circle { ... }
```

Every class the programmer defines has a superclass. If the superclass is not specified with an `extends` clause, then the superclass is taken to be the class `java.lang.Object`.

As a result, the `Object` class is special for a couple of reasons:

- It is the only class in Java that does not have a superclass.
- All Java classes inherit (directly or indirectly) the methods of `Object`.

Because every class (except `Object`) has a superclass, classes in Java form a class hierarchy, which can be represented as a tree with `Object` at its root.

> **NOTE**
>
> `Object` has no superclass, but every other class has exactly one superclass. A class cannot extend more than one superclass; see Chapter 5 for more information on how to achieve a similar result using interfaces.

Figure 4-1 shows a partial class hierarchy diagram including our `Circle` and `PositionedCircle` classes, as well as some of the standard classes from the Java API.

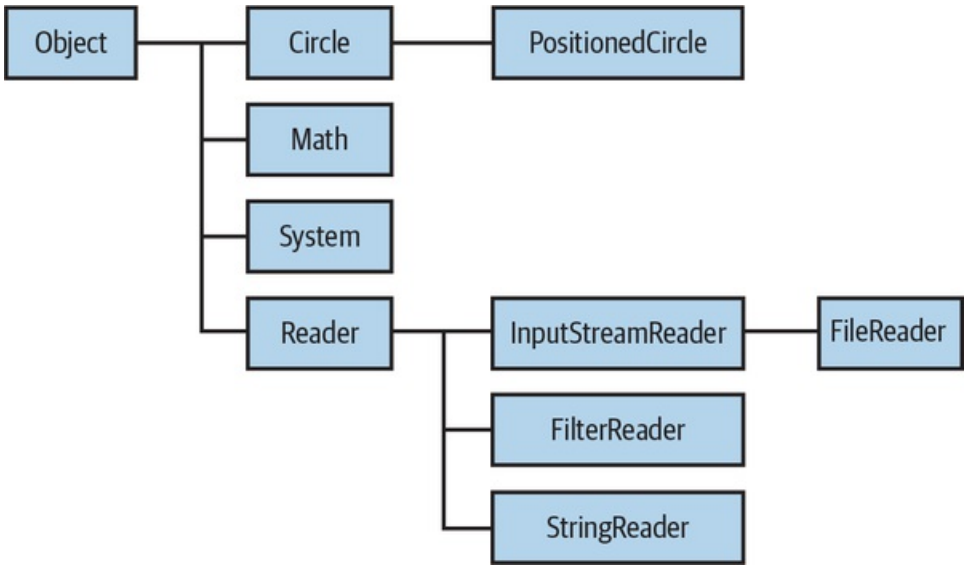



*Figure 4-1. A class hierarchy diagram*

## Subclass Constructors

Look again at the `PositionedCircle()` constructor from Example 4-3:

```
public PositionedCircle(double r, double x, double y) {
  this.cx = x;    // Initialize the instance field cx
  this.cy = y;    // Initialize the instance field cy
  super(r);       // Invoke the constructor of the superclass, Circle()
}
```

Although this constructor explicitly initializes the `cx` and `cy` fields newly defined by `PositionedCircle`, it relies on the superclass `Circle()` constructor to initialize the inherited fields of the class. To invoke the superclass constructor, our constructor calls `super()`.

`super` is a reserved word in Java. One of its main uses is to invoke the constructor of a superclass from within a subclass constructor. This use is analogous to the use of `this()` to invoke one constructor of a class from within another constructor of the same class.

Invoking a constructor using `super()` is subject to the same restrictions as using `this()`:

- `super()` can be used in this way only within a constructor.
- The arguments passed to `super()` must match the parameters of the superclass constructor.
- If the superclass defines more than one constructor, `super()` can be used to invoke any one of them, depending on the arguments passed.

The use of `super()` leads to the concept of *constructor chaining*, which we will discuss next.

## Constructor Chaining

Java guarantees that the constructor of a class is called whenever an instance of that class is created. It also guarantees that the constructor is called whenever an instance of any subclass is created. In order to guarantee this second point, Java must ensure that every constructor calls its superclass constructor.

> **WARNING**
>
> In Java versions before 25, the call to the superclass constructor must appear as the first statement within the constructor, even before local variable declarations. This has been relaxed as of Java 25, but this may trip you up when working with codebases that use earlier versions than 25.

Consider what happens when we create a new instance of the `PositionedCircle` class:

1. First, the fields of `PositionedCircle` are set (this is sometimes called a *constructor prologue*).
2. Next, the constructor explicitly calls `super(r)` to invoke a `Circle` constructor.
3. That `Circle()` constructor implicitly calls `super()` to invoke the constructor of its superclass, `Object` (`Object` has only one constructor).
4. At this point, we've reached the top of the hierarchy and constructor bodies start to run.
5. The body of the `Object` constructor runs first. It does nothing user-visible.
6. When it returns, the body of the `Circle()` constructor runs.
7. Finally, when the call to `super(r)` returns, any remaining statements of the `PositionedCircle()` constructor would execute, but there aren't any (this would be called a *constructor epilogue*).

What all this means is that constructor calls are chained; any time an object is created, a sequence of constructors is invoked, from subclass to superclass on up to `Object` at the root of the class hierarchy.

## The Default Constructor

There is one missing piece in the previous description of constructor chaining. If a constructor does not invoke a superclass constructor, Java does so implicitly.

If a class is declared without any constructor, Java implicitly adds a constructor to the class. This default constructor does nothing but invoke the superclass constructor. For example, if we don't declare a constructor for the `PositionedCircle` class, Java implicitly inserts this constructor:

```
public PositionedCircle() { super(); }
```

Classes declared `public` are given `public` constructors. All other classes are given a default constructor that is declared without any visibility modifier; such a constructor has default visibility.

One very important point is that if a class declares constructors that take parameters but does not define a no-argument constructor, then all its subclasses must define constructors that explicitly invoke a constructor with the necessary arguments.

> **NOTE**
>
> If you are creating a `public` class that should not be publicly instantiated, declare at least one non-`public` constructor to prevent the insertion of a default `public` constructor.

Classes that should never be instantiated (such as `java.lang.Math` or `java.lang.System`) should define only a `private` no-argument constructor. Such a constructor can never be invoked from outside the class, and it prevents the automatic insertion of the default constructor. The overall effect is that the class will never be instantiated, as it is not instantiated by the class itself and no other class has the correct access.[3]

## Hiding Superclass Fields

Let's imagine that our `PositionedCircle` class needs to know the distance between the center of the circle and the origin (0, 0). We can add another instance field to hold this value:

```
public double r;
```

Adding the following line to the constructor computes the value of the field:

```
this.r = Math.sqrt(cx * cx + cy * cy);  // Pythagorean theorem
```

But wait; this new field `r` has the same name as the radius field `r` in the `Circle` superclass. When this happens, we say that the field `r` of `PositionedCircle` *hides* the field `r` of `Circle`. (This is a contrived example, of course: the new field really should be called

`distanceFromOrigin`.)

> **WARNING**
>
> In code that you write, you should avoid declaring fields with names that hide superclass fields. It is almost always a sign of bad code.

With this new definition of `PositionedCircle`, the expressions `r` and `this.r` both refer to the field of `PositionedCircle`. How, then, can we refer to the field `r` of `Circle` that holds the radius of the circle? A special syntax for this uses the `super` keyword:

```
r        // Refers to the PositionedCircle field
this.r   // Refers to the PositionedCircle field
super.r  // Refers to the Circle field
```

Another way to refer to a hidden field is to cast `this` (or any instance of the class) to the appropriate superclass and then access the field:

```
((Circle) this).r   // Refers to field r of the Circle class
```

This casting technique is particularly useful when you need to refer to a hidden field defined in a class that is not the immediate superclass. Suppose, for example, that classes `A`, `B`, and `C` all define a field named `x` and that `C` is a subclass of `B`, which is a subclass of `A`. Then, in the methods of class `C`, you can refer to these different fields as follows:

```
x               // Field x in class C
this.x          // Field x in class C
super.x         // Field x in class B
((B)this).x     // Field x in class B
((A)this).x     // Field x in class A
super.super.x   // Illegal; does not refer to x in class A
```

> **NOTE**
>
> You cannot refer to a hidden field `x` in the superclass of a superclass with `super.super.x`. This is not legal syntax.

Similarly, if you have an instance `c` of class `C`, you can refer to the three fields named `x` like this:

```
c.x             // Field x of class C
((B)c).x        // Field x of class B
((A)c).x        // Field x of class A
```

So far, we've been discussing instance fields. Class fields can also be hidden, but this is heavily discouraged. To see why, let's consider an example and suppose that the implementer of

`PositionedCircle` decides that the `Circle.PI` field does not have enough decimal places. They can define their own class field `PI`:

```
public static final double PI = 3.14159265358979323846;
```

Now code in `PositionedCircle` can use this more accurate value with the expressions `PI` or `PositionedCircle.PI`. It can also refer to the old, less accurate value with the expressions `super.PI` and `Circle.PI`.

> **TIP**
>
> You can use the `super` syntax to refer to the hidden value of the field, but this is never necessary, as you can always refer to a class field by prepending the name of the desired class.

However, there is a problem: the `area()` and `circumference()` methods inherited by `PositionedCircle` are defined in the `Circle` class, so they use the value `Circle.PI`. That value is hidden now by `PositionedCircle.PI`, but in order to make use of it, the methods will have to be overridden. This strongly works against the design spirit of OO programming, and so the practice of hiding class fields should be avoided.

## Overriding Superclass Methods

When a class defines an instance method using the same name, return type, and parameters as a method in its superclass, that method *overrides* the method of the superclass. When the method is invoked for an object of the class, it is the new definition of the method that is called, not the old definition from the superclass.

> **TIP**
>
> The return type of the overriding method may be a subclass of the return type of the original method (instead of being exactly the same type). This is known as a *covariant return*.

Method overriding is an important and useful technique in object-oriented programming. `PositionedCircle` doesn't override either of the methods defined by `Circle`.

> **WARNING**
>
> Don't be tempted to consider subclassing `Circle` with a class like `Ellipse`—this would actually violate a core principle of object-oriented development (the Liskov Substitution Principle, which we will meet later in this chapter and discuss further in Chapter 5).

Instead, let's look at a different example that does work with method overriding:

```
public class Car {
    public static final double LITERS_PER_100KM = 8.9;

    protected double topSpeed;

    protected double fuelTankCapacity;

    private int doors;

    public Car(double topSpeed, double fuelTankCapacity,
               int doors) {
        this.topSpeed = topSpeed;
        this.fuelTankCapacity = fuelTankCapacity;
        this.doors = doors;
    }

    public double getTopSpeed() {
        return topSpeed;
    }

    public int getDoors() {
        return doors;
    }

    public double getFuelTankCapacity() {
        return fuelTankCapacity;
    }

    public double range() {
        return 100 * fuelTankCapacity / LITERS_PER_100KM;
    }
}
```

This is a bit more complex, but it will illustrate the concepts behind overriding. Along with the `Car` class, we also have a specialized class, `SportsCar`. This has several differences: it has a 50-liter fuel tank and comes only in a two-door version. It may also have a much higher top speed than the regular form, but if the top speed rises above 200 km/h, then the fuel efficiency of the car suffers, and as a result the overall range of the car starts to decrease:

```
public class SportsCar extends Car {

    private double efficiency;

    public SportsCar(double topSpeed) {
        super(topSpeed, 50.0, 2);
        if (topSpeed > 200.0) {
            efficiency = 200.0 / topSpeed;
        } else {
            efficiency = 1.0;
        }
    }

    public double getEfficiency() {
        return efficiency;
    }

    @Override
```

```
    public double range() {
        return 100 * fuelTankCapacity * efficiency / LITER_PER_100KM;
    }

}
```

The upcoming discussion of method overriding considers only instance methods.

Class (aka `static`) methods behave quite differently, and they cannot be overridden. Just like fields, class methods can be hidden by a subclass but not overridden.

As noted earlier in this chapter, it is good programming style to always prefix a class method invocation with the name of the class in which it is defined. If you consider the class name part of the class method name, the two methods have different names, so nothing is actually hidden at all.

> **NOTE**
>
> The code example for the `SportsCar` includes the syntax construct `@Override`. This is known as an *annotation*, and we shall meet this piece of Java syntax properly in Chapter 5.

Before we go any further with the discussion of method overriding, you should understand the difference between method overriding and method overloading. As we discussed in Chapter 2, method overloading refers to the practice of defining multiple methods (in the same class) that have the same name but different parameter lists.

On the other hand, a method overrides a method in its superclass when the instance method uses the same name, return type, and parameter list as a method in its superclass. These two features are very different from each other, so don't get them confused.

### Overriding is not hiding

Although Java treats the fields and methods of a class analogously in many ways, method overriding is not at all like field hiding. You can refer to hidden fields simply by casting an object to an instance of the appropriate superclass, but you cannot invoke overridden instance methods with this technique. The following code illustrates this crucial difference:

```
class A {                           // Define a class named A
  int i = 1;                        // An instance field
  int f() { return i; }             // An instance method
  static char g() { return 'A'; }   // A class method
}

class B extends A {                 // Define a subclass of A
  int i = 2;                        // Hides field i in class A
  int f() { return -i; }            // Overrides method f in class A
  static char g() { return 'B'; }   // Hides class method g() in class A
}

public class OverrideTest {
```

```
  public static void main(String args[]) {
    B b = new B();              // Creates a new object of type B
    System.out.println(b.i);    // Refers to B.i; prints 2
    System.out.println(b.f());  // Refers to B.f(); prints -2
    System.out.println(b.g());  // Refers to B.g(); prints B
    System.out.println(B.g());  // A better way to invoke B.g()

    A a = (A) b;                // Casts b to an instance of class A
    System.out.println(a.i);    // Now refers to A.i; prints 1
    System.out.println(a.f());  // Still refers to B.f(); prints -2
    System.out.println(a.g());  // Refers to A.g(); prints A
    System.out.println(A.g());  // A better way to invoke A.g()
  }
}
```

While this difference between method overriding and field hiding may seem surprising at first, a little thought makes the purpose clear.

Suppose we are manipulating a bunch of `Car` and `SportsCar` objects and store them in an array of type `Car[]`. We can do this because `SportsCar` is a subclass of `Car`, so all `SportsCar` objects are legal `Car` objects.

When we loop through the elements of this array, we don't have to know or care whether the element is actually a `Car` or a `SportsCar`. What we do care about very much, however, is that the correct value is computed when we invoke the `range()` method of any element of the array. In other words, we don't want to use the formula for the range of a car when the object is actually a sports car!

All we really want is for the objects we're computing the ranges of to "do the right thing"—the `Car` objects should use their definition of how to compute their own range, and the `SportsCar` objects should use the definition that is correct for them.

Seen in this context, it is not surprising that Java handles method overriding differently from field hiding.

### Virtual method lookup

If we have a `Car[]` array that holds both `Car` and `SportsCar` objects at the same time, how does Java know whether to call the `range()` method of the `Car` class or the `SportsCar` class for any given item in the array? The contents of the array can change at runtime, so `javac` (the source code compiler) cannot know this at compilation time.

Instead, `javac` creates bytecode that uses *virtual* method lookup at runtime. When the interpreter runs the code, it looks up the appropriate `range()` method to call for each of the objects in the array. That is, when the interpreter interprets the expression `o.range()`, it checks the actual runtime type of the object referred to by the variable `o` and then finds the `range()` method that is appropriate for that type.

**NOTE**

Some other languages (such as C# or C++) do not do virtual lookup by default and instead have a `virtual` keyword that programmers must explicitly use if they want subclasses to be able to override a method.

This is another way of approaching the concept of method overriding, which we discussed earlier. If the version of the `range()` method associated with the static type of `o` was used, without the runtime (aka virtual) lookup, then overriding would not work properly.

Virtual method lookup is the default for Java instance methods. See Chapter 5 for more details about compile-time and runtime types and how they affect virtual method lookup.

### Invoking an overridden method

We've seen the important differences between method overriding and field hiding. Nevertheless, the Java syntax for invoking an overridden method is quite similar to the syntax for accessing a hidden field: both use the `super` keyword. The following code illustrates:

```
class A {
  int i = 1;             // An instance field hidden by subclass B
  int f() { return i; } // An instance method overridden by subclass B
  void g() {}            // An instance method that is *not* overridden
                         // by subclass B
}

class B extends A {
  int i;                    // This field hides i in A
  int f() {                 // This method overrides f() in A
    i = super.i + 1;        // It can retrieve A.i like this
    return super.f() + i;   // It can invoke A.f() like this
  }
}
```

Recall that when you use `super` to refer to a hidden field, it is the same as casting `this` to the superclass type and accessing the field through it. Using `super` to invoke an overridden method, however, is not the same as casting the `this` reference. In other words, in the previous code, the expression `super.f()` is not the same as `((A)this).f()`.

When the interpreter invokes an instance method with the `super` syntax, a modified form of virtual method lookup is performed. The first step, as in regular virtual method lookup, is to determine the actual class of the object through which the method is invoked. Normally, the runtime search for an appropriate method definition would begin with this class.

When a method is invoked with the `super` syntax, however, the search begins at the superclass of the class. If the superclass implements the method directly, that version of the method is invoked. If the superclass inherits the method, the inherited version of the method is invoked.

Note that the `super` keyword invokes the most immediately overridden version of a method. Let's look at an example:

Suppose class `A` has a subclass `B` that has a subclass `C` and that all three classes define the same

method `f()`, but only `A` and `C` define `g()`:

```
class C extends B {
    int f() {
        return 17 + super.f();
    }

    void g() {
        super.g();
    }
}
```

The method `C.f()` can invoke the method `B.f()`, which it overrides directly, with `super.f()`. But there is no way for `C.f()` to invoke `A.f()` directly: `super.super.f()` is not legal Java syntax. Of course, if `C.f()` invokes `B.f()`, it is reasonable to suppose that `B.f()` might also invoke `A.f()`.

However, calling `super.g()` invokes `A.g()` since `B` does not declare `g()`. This kind of chaining is relatively common with overridden methods: it is a way of augmenting the behavior of a method without replacing the method entirely.

> **NOTE**
>
> Don't confuse the use of `super` to invoke an overridden method with the `super()` method call used in a constructor to invoke a superclass constructor. Although they both use the same keyword, these are two different syntaxes.

It is also important to remember that `super` can be used to invoke an overridden method only from within the class that overrides it. Given a reference to a `SportsCar` object `e`, there is no way for a program that uses `e` to invoke the `range()` method defined by the `Car` class on `e`.

## Sealed Classes

Until this point, we have encountered only two possibilities for class inheritance:

- Unrestricted ability to subclass (which is the default and has no keyword associated with it)
- Complete prevention of subclassing with the `final` keyword applied to a class

As of Java 17, there is a third possibility, which is denoted by the `sealed` keyword. A *sealed class* is one that can be subclassed but only by a specific list of known classes. This is done by using the `permits` keyword to enumerate the list of possible subclasses up front, when the sealed class is declared, like this:

```
// In Shape.java
public abstract sealed class Shape permits Circle, Triangle {
    // ...
```

```
}

// In Circle.java
public final class Circle extends Shape {
    // ...
}

// In Triangle.java
public final class Triangle extends Shape {
    // ...
}
```

In this example, we have declared both `Circle` and `Triangle` as `final`, so they cannot be further subclassed. This is the usual approach, but it is also possible to declare a subtype of a sealed class as either `sealed` (with a further set of permitted subclasses) or as `non-sealed`, which restores the default Java behavior of unrestricted subclassing.

For this reason, it is a compile-time error to try to subclass a sealed class without providing one of the three sealing modifiers: there is no default behavior here.

> **NOTE**
>
> The introduction of `non-sealed` is the first example of a *hyphenated keyword* that's been seen in Java.

In this example we've used an abstract sealed base class (`Shape`). This is not always necessary, but it is often a good practice, as it means that any instances of the type that we encounter are known to be one of the "leaf types," such as `Circle` or `Triangle`. We will meet abstract classes properly later in the chapter.

Although sealed classes are new with Java 17, we expect that many developers will adopt them quickly—along with records, they represent a "missing concept" that fits very naturally into Java's view of OO. We will have more to say about this in Chapter 6 when we discuss aspects of OOD related to sealed types.

## Data Hiding and Encapsulation

We started this chapter by describing a class as a collection of data and methods. One of the most important object-oriented techniques we haven't discussed so far is hiding the data within the class and making it available only through the methods.

This technique is known as *encapsulation* because it contains the data (and internal methods) safely inside the "capsule" of the class, where it can be accessed only by trusted users (i.e., the methods of the class).

Why would you want to do this? The most important reason is to hide the internal implementation details of your class. If you prevent programmers from relying on those details, you can safely modify the implementation without worrying that you will break existing code

that uses the class.

> **TIP**
>
> You should always encapsulate your code. It is almost always impossible to reason through and ensure the correctness of code that hasn't been well-encapsulated, especially in multithreaded environments (and essentially all Java programs are multithreaded).

Another reason for encapsulation is to protect your class against accidental or willful misuse. A class often contains a number of interdependent fields that must be in a consistent state. If you allow programmers (including yourself) to manipulate those fields directly, they may change one field without changing important related fields, leaving the class in an inconsistent state. If instead the programmer has to call a method to change the field, that method can be sure to do everything necessary to keep the state consistent. Similarly, if a class defines certain methods for internal use only, hiding these methods prevents users of the class from calling them.

Here's another way to think about encapsulation: when all the data for a class is hidden, the methods define the only possible operations that can be performed on objects of that class.

Once you have carefully tested and debugged your methods, you can be confident that the class will work as expected. On the other hand, if all the fields of the class can be directly manipulated, the number of possibilities you have to test becomes unmanageable.

> **NOTE**
>
> This idea can be carried to a very powerful conclusion, as we will see in "Safe Java Programming" when we discuss the *safety* of Java programs (which differs from the concept of *type safety* of the Java programming language).

Other, secondary, reasons to hide fields and methods of a class include the following:

- Internal fields and methods that are visible outside the class just clutter up the API. Keeping visible fields to a minimum keeps your class tidy and therefore easier to use and understand.
- If a method is visible to the users of your class, you have to document it. Save yourself time and effort by hiding it instead.

## Access Control

Java defines access control rules that can restrict members of a class from being used outside the class. In a number of examples in this chapter, you've seen the `public` modifier used in field and method declarations. This `public` keyword, along with `protected` and `private` (and one other special one) are *access control modifiers*; they specify the access rules for the field or method.

### Access to modules

The biggest change to arrive in Java 9 was Java platform modules. These are a grouping of code that is larger than a single package and intended as the future way to deploy code for reuse. As Java is often used in large applications and environments, the arrival of modules was intended to make it easier to build and manage enterprise codebases.

The modules technology is an advanced topic, and if Java is one of the first programming languages you have encountered, you should not try to learn it until you have gained some language proficiency. An introductory treatment of modules is provided in Chapter 14, and we defer discussing the access control impact of modules until then.

### Access to packages

Access control on a per-package basis is not directly part of the core Java language and instead is provided by the Java Modules mechanism. In the normal course of programming, access control is usually done at the level of classes and members of classes—a package that has been loaded is always accessible to code defined within the same package.

### Access to classes

By default, top-level classes are accessible within the package in which they are defined. However, if a top-level class is declared `public`, it is freely accessible.[4]

It wouldn't make sense for top-level classes to be declared `private` or `protected`, so default and `public` are the only two options for top-level classes.

However, in "Nested Types", we'll meet nested classes. These are classes that can be defined as members of other classes, and so they obey the member access control rules and can be declared with any of the access control modifiers.

### Access to members

The members of a class are always accessible within the body of the class. By default, members are also accessible throughout the package in which the class is defined. This default level of access is often called *package access*.

It is one of four possible levels of access. The other three levels are defined by the `public`, `protected`, and `private` modifiers. Here is some example code that uses these modifiers:

```
public class Laundromat {    // People can use this class.
  private Laundry[] dirty;   // They cannot use this internal field,
  public void wash() { ... } // but they can use these public methods
  public void dry() { ... }  // to manipulate the internal field.
  // A subclass might want to tweak this field
  protected int temperature;
}
```

These access rules apply to members of a class:

- All the fields and methods of a class can always be used within the body of the class itself.
- If a member of a class is declared `private`, the member is only accessible within the class itself or nested types within the class. This is the most restrictive type of access control. Notably, `private` fields are *not* available in subclasses.
- If a member of a class is declared `protected`, it is accessible to all classes within the package (the same as the default package accessibility) and also within the body of any subclass of the class, regardless of the package in which that subclass is defined.
- If a member of a class is declared with the `public` modifier, it means that the member is accessible anywhere the containing class is accessible. This is the least restrictive type of access control.
- If a member of a class is not declared with any of these modifiers, it has *default* access (sometimes called *package* access), and it is accessible to code within all classes that are defined in the same package but inaccessible outside of the package.

Of these, `protected` access is perhaps the most complex and requires some more elaboration.

> **WARNING**
>
> Default access is *more* restrictive than `protected`—as default access does not allow access by subclasses outside the package.

Suppose class `A` declares a `protected` field `x` and is extended by a class `B`, which is defined in a different package (this last point is important). Class `B` inherits the `protected` field `x`, and its code can access that field in the current instance of `B` or in any other instances of `B` that the code can refer to. This does not mean, however, that the code of class `B` can start reading the protected fields of arbitrary instances of `A`.

Let's look at this language detail in code. Here's the definition for `A`:

```
package javanut.ch04.access;

public class A {
    protected final String name;

    public A(String named) {
        name = named;
    }

    public String getName() {
        return name;
    }
}
```

Here's the definition for B:

```
package javanut.ch04.different;

import javanut.ch04.access.A;

public class B extends A {

    public B(String named) {
        super(named);
    }

    @Override
    public String getName() {
        return "B: " + name;
    }
}
```

> **NOTE**
>
> Java packages do not "nest," so `javanut.ch04.different` is just a different package from `javanut.ch04`; it is not contained "inside it" or related to it in any way.

However, if we try to add this new method to B, we will get a compilation error, because instances of B do not have access to arbitrary instances of A:

```
    public String examine(A a) {
        return "B sees: " + a.name;
    }
```

If we change the method to this:

```
    public String examine(B b) {
        return "B sees another B: " + b.name;
    }
```

then the compiler is happy, because instances of the same exact type can always see one another's fields, regardless of access modifiers. If B was in the same package as A, then any instance of B could read any `protected` field of any instance of A because `protected` fields are visible to every class in the same package.

### Access control and inheritance

The Java specification states the following:

- A subclass inherits all the instance fields and instance methods of its superclass accessible to it.
- If the subclass is defined in the same package as the superclass, it inherits all non-`private` instance fields and methods.

- If the subclass is defined in a different package, it inherits all `protected` and `public` instance fields and methods.
- `private` fields and methods are never inherited; neither are class fields or class methods.
- Constructors are not inherited (instead, they are chained, as described earlier in this chapter).

Some programmers are confused by the statement that a subclass does not inherit the inaccessible fields and methods of its superclass. Let us be explicit: every instance of a subclass includes a complete instance of the superclass within it, including all private fields and methods. When you create an instance of a subclass, memory is allocated for all `private` fields defined by the superclass; however, the subclass does not have access to these fields directly.

This existence of potentially inaccessible members seems to be in conflict with the statement that the members of a class are always accessible within the body of the class. To clear up this confusion, we define *inherited members* as those superclass members that are accessible.

Then the correct statement about member accessibility is "All inherited members and all members defined in this class are accessible." An alternative way of saying this is:

- A class inherits *all* instance fields and instance methods (but not constructors) of its superclass.
- The body of a class can always access all the fields and methods it declares itself. It can also access the *accessible* fields and members it inherits from its superclass.

### Member access summary

We summarize the member access rules in Table 4-1.

*Table 4-1. Class member accessibility*

| | Member visibility | | | |
|---|---|---|---|---|
| **Accessible to** | **Public** | **Protected** | **Default** | **Private** |
| Defining class | Yes | Yes | Yes | Yes |
| Class in same package | Yes | Yes | Yes | No |
| Subclass in different package | Yes | Yes | No | No |
| Nonsubclass different | Yes | No | No | No |

package

There are a few generally observed rules about what parts of a Java program should use each visibility modifier. It is important that even beginning Java programmers follow these rules:

- Use `public` only for methods and constants that form part of the public API of the class. The only acceptable usage of `public` fields is for constants or immutable objects, and so they must also be declared `final`.
- Use `protected` for fields and methods that aren't required by most programmers using the class but that may be of interest to anyone creating a subclass as part of a different package.
- Use the default package visibility for fields and methods that are internal implementation details but are used by cooperating classes in the same package.
- Use `private` for fields and methods that are used only inside the class and should be hidden everywhere else.

If you are not sure whether to use `protected`, package, or `private` accessibility, start with `private`. If this is overly restrictive, you can always relax the access restrictions slightly (or provide accessor methods, in the case of fields).

> **WARNING**
>
> In effect, `protected` members are technically part of the exported API of a class. They must be documented just like any other part of the API.

This is especially important for designing APIs because increasing access restrictions is not a backward compatible change and can break code that relies on access to those members.

## Data Accessor Methods

In the original `Circle` example, we declared the radius to be a `public` field. It might seem that the `Circle` class is one in which it may be reasonable to keep that field publicly accessible; it is a simple enough class, with no dependencies between its fields.

However, even this near-trivial example has problems that we should seek to avoid. For example, our current implementation of the class allows a `Circle` object to have a negative radius, and circles with negative radii simply should not exist. As long as the radius is stored in a `public` field, any programmer can set the field to any value they want, no matter how unreasonable.

The only solution is to restrict the programmer's direct access to the field and define `public` methods that provide indirect access to the field. Providing `public` methods to read and write a

field is not the same as making the field itself `public`. The crucial difference is that methods can run code—e.g., to perform error and consistency checking.

In Example 4-4, we show how we might change the definition of `Circle` to perform such checking.

This version of `Circle` declares the `r` field to be `protected` and defines accessor methods named `getRadius()` and `setRadius()` to read and write the field value while enforcing the restriction on negative radius values. Because the `r` field is `protected`, it is still directly accessible to subclasses.

*Example 4-4. The `Circle` class using data hiding and encapsulation*

```
package javanut.ch04.shapes; // Specify a package for the class

public class Circle {     // The class is still public
    // This is a generally useful constant, so we keep it public
    public static final double PI = 3.14159;

    protected double r;     // Radius is hidden from arbitrary classes

    // The nondefault constructor
    public Circle(double r) {
        if (r < 0.0) {
            throw new IllegalArgumentException("illegal negative radius");
        }
        this.r = r;
    }

    // Public data accessor methods
    public double getRadius() { return r; }
    public void setRadius(double r) {
        if (r < 0.0) {
            throw new IllegalArgumentException("illegal negative radius");
        }
        this.r = r;
    }

    // Methods to operate on the instance field
    public double area() { return PI * r * r; }
    public double circumference() { return 2 * PI * r; }
}
```

We have defined the `Circle` class within a package named `javanut.ch04.shapes`. The field `r` is `protected` so any other classes in the `javanut.ch04.shapes` package have direct access to that field and can set it however they like.

The assumption here is that all classes within the `javanut.ch04.shapes` package trust one another not to abuse their privileged level of access to one another's implementation details. For example, these classes could well have been written by the same author or a closely cooperating group of authors.

**NOTE**

One set of common (but older) conventions in Java—known as *Java Beans* conventions—is that data accessor methods begin with the prefixes *get* and *set*. But if the field being accessed is of type `boolean`, the `get()` method may be replaced with an equivalent method that begins with *is*—the accessor method for a `boolean` field named `readable` is typically called `isReadable()` instead of `getReadable()`.

Let's move on to discuss abstract methods, a very common OO technique in Java.

## Abstract Classes and Methods

In Example 4-4, we declared our `Circle` class to be part of a package named `shapes`. Suppose we plan to implement a number of shape classes: `Rectangle`, `Square`, `Hexagon`, `Triangle`, and so on. We can give these shape classes our two basic `area()` and `circumference()` methods.

Now, to make it easy to work with an array of shapes, it would be helpful if all our shape classes had a common superclass, `Shape`.

If we structure our class hierarchy this way, every shape object, regardless of the actual type of shape it represents, can be assigned to variables, fields, or array elements of type `Shape`. We want the `Shape` class to encapsulate whatever features all our shapes have in common (e.g., the `area()` and `circumference()` methods). But our generic `Shape` class doesn't represent any specific kind of shape, so it cannot define practical implementations of the methods.

Java handles this situation with *abstract methods*. We can define a method without implementing it by declaring the method with the `abstract` modifier. An `abstract` method has no body; it simply has a signature definition followed by a semicolon.[5]

**NOTE**

The `ClassLoader` class that we will meet in Chapter 13 is a good example of an `abstract` class that does not have any `abstract` methods.

Here are the rules about `abstract` methods and the `abstract` classes that contain them:

- Any class with an `abstract` method is automatically `abstract` itself and must be declared as such. To fail to do so is a compilation error.
- An `abstract` class cannot be instantiated.
- A subclass of an `abstract` class can be instantiated only if it overrides each of the `abstract` methods of its superclass and provides an implementation (i.e., a method body) for all of them. Such a class is often called a *concrete* subclass, to emphasize the fact that it is not `abstract`.
- If a subclass of an `abstract` class does not implement all the `abstract` methods it

inherits, that subclass is itself `abstract` and must be declared as such.

- `static`, `private`, and `final` methods cannot be `abstract`, because these types of methods cannot be overridden by a subclass. Similarly, a `final` class cannot contain any `abstract` methods.
- A class can be declared `abstract` even if it does not actually have any `abstract` methods. Declaring such a class `abstract` indicates that the implementation is somehow incomplete and is meant to serve as a base class for one or more subclasses that complete the implementation.
- It is perfectly legitimate, and in fact quite common, for classes to be declared both `abstract` and `sealed`. This becomes increasingly popular as a pattern with the arrival of type patterns in Java.

Let's look at an example of how these rules work. If we define the `Shape` class to have `abstract area()` and `circumference()` methods, any subclass of `Shape` is required to provide implementations of these methods so that it can be instantiated. In other words, every `Shape` object is guaranteed to have implementations of these methods defined.

Example 4-5 shows how this might work. It defines an `abstract Shape` class and a concrete subclass of it. You should also imagine that the `Circle` class from Example 4-4 has been modified so that it `extends Shape`.

*Example 4-5. An abstract class and concrete subclass*

```
public abstract class Shape {
    public abstract double area();             // Abstract methods: note
    public abstract double perimeter();   // semicolon instead of body.
}

public class Rectangle extends Shape {
    // Instance data
    protected double w, h;

    // Constructor
    public Rectangle(double w, double h) {
        this.w = w;  this.h = h;
    }

    // Accessor methods
    public double getWidth() { return w; }
    public double getHeight() { return h; }

    // Implementation of abstract methods
    public double area() { return w * h; }
    public double perimeter() { return 2 * (w + h); }
}
```

Each `abstract` method in `Shape` has a semicolon right after its parentheses. Method declarations of this sort have no curly braces, and no method body is defined.

Note that we could have declared the class `Shape` as a sealed class, but we have deliberately

chosen not to. This is so other programmers can define their own shape classes as new subclasses of `Shape`, should they wish to.

Using the classes defined in Example 4-5, we can now write code such as:

```
Shape[] shapes = new Shape[3];        // Create an array to hold shapes
shapes[0] = new Circle(2.0);          // Fill in the array
shapes[1] = new Rectangle(1.0, 3.0);
shapes[2] = new Rectangle(4.0, 2.0);

double totalArea = 0;
for(int i = 0; i < shapes.length; i += 1) {
    totalArea += shapes[i].area();   // Compute the area of the shapes
}
```

Notice two important points here:

- Subclasses of `Shape` can be assigned to elements of an array of `Shape`. No cast is necessary. This is another example of a widening reference type conversion (discussed in Chapter 2).
- You can invoke the `area()` and `perimeter()` methods for any `Shape` object, even though the `Shape` class does not define a body for these methods. When you do this, the method to be invoked is found using virtual lookup, which we discussed earlier. In our case, this means that the area of a circle is computed using the method defined by `Circle`, and the area of a rectangle is computed using the method defined by `Rectangle`.

## Reference Type Conversions

Object references can be converted between different reference types. As with primitive types, reference type conversions can be widening conversions (allowed automatically by the compiler) or narrowing conversions that require a cast (and possibly a runtime check). In order to understand reference type conversions, you need to understand that reference types form a hierarchy, usually called the *class hierarchy*.

Every Java class (except `Object`) *extends* exactly one other class, known as its *superclass*. A class inherits the fields and methods of its superclass and then defines its own additional fields and methods. The special class named `Object` serves as the *root* of the class hierarchy in Java. Thus, all Java classes extend `Object` either directly or indirectly. The `Object` class defines a number of special methods that are therefore inherited (or overridden) by all objects.

The predefined `String` class and the `Account` class we discussed earlier in this chapter both extend `Object`. We can say that all `String` objects are also `Object` objects. We can also say that all `Account` objects are `Object` objects. The opposite is not true, however. We cannot say that every `Object` is a `String` because, as we've just seen, some `Object` objects are `Account` objects.

With this simple understanding of the class hierarchy, we can define the rules of reference type conversion:

- An object reference cannot be converted to an unrelated type. The Java compiler does not allow you to convert a `String` to an `Account`, for example, even if you use a cast operator.
- An object reference can be converted to the type of its superclass or of any ancestor class. This is a widening conversion, so no cast is required. For example, a `String` value can be assigned to a variable of type `Object` or passed to a method where an `Object` parameter is expected.

> **NOTE**
>
> No conversion is actually performed; the object is simply treated as if it were an instance of the superclass. This is a simple form of the Liskov Substitution Principle, named for Barbara Liskov, the computer scientist who first explicitly formulated it.

- An object reference can be converted to the type of a subclass, but this is a narrowing conversion and requires a cast. The Java compiler provisionally allows this kind of conversion, but the Java interpreter checks at runtime to make sure it is valid. Only cast a reference to the type of a subclass if you are sure, based on the logic of your program, that the object is actually an instance of the subclass. If it is not, the interpreter throws a `ClassCastException`. For example, if we assign a `String` reference to a variable of type `Object`, we can later cast the value of that variable back to type `String`:

```
Object o = "string";    // Widening conversion from String
                        // to Object later in the program...
String s = (String) o;  // Narrowing conversion from Object
                        // to String
```

Arrays are objects and follow some conversion rules of their own. First, any array can be converted to an `Object` value through a widening conversion. A narrowing conversion with a cast can convert such an object value back to an array. Here's an example:

```
// Widening conversion from array to Object
Object o = new int[] {1, 2, 3};

// Narrowing conversion back to array type
int[] a = (int[])o;
```

In addition to converting an array to an object, we can convert an array to another type of array if the "base types" of the two arrays are reference types that can themselves be converted. For example:

```
// Here is an array of strings.
String[] strings = new String[] { "hi", "there" };
```

```
// A widening conversion to CharSequence[] is allowed because String
// can be widened to CharSequence
CharSequence[] sequences = strings;
// The narrowing conversion back to String[] requires a cast.
strings = (String[]) sequences;

// This is an array of arrays of strings
String[][] s = new String[][] { strings };
// It cannot be converted to CharSequence[] because String[] cannot be
// converted to CharSequence: the number of dimensions don't match

sequences = s;  // This line will not compile
// s can be converted to Object or Object[], because all array types
// (including String[] and String[][]) can be converted to Object.
Object[] objects = s;
```

Note that these array conversion rules apply only to arrays of objects and arrays of arrays. An array of a primitive type cannot be converted to any other array type, even if the primitive base type can be converted:

```
// Can't convert int[] to double[] even though
// int can be widened to double
// This line causes a compilation error
double[] data = new int[] {1, 2, 3};
// This line is legal, however,
// because int[] can be converted to Object
Object[] objects = new int[][] {{1, 2},{3, 4}};
```

## Modifier Summary

As we've seen, classes, interfaces, and their members can be declared with one or more *modifiers*—keywords such as `public`, `static`, and `final`. Let's conclude this chapter by listing the Java modifiers, explaining what types of Java constructs they can modify, and explaining what they do. Table 4-2 has the details; you can also refer to "Overview of Classes and Records", "Field Declaration Syntax", and "Method Modifiers".

*Table 4-2. Java modifiers*

| Modifier | Used on | Meaning |
|---|---|---|
| `abstract` | Class | The class cannot be instantiated and may contain unimplemented methods. |
| | Interface | All interfaces are `abstract`. The modifier is optional in interface declarations. |
| | Method | No body is provided for the method; it is provided by a subclass. The signature is followed by a semicolon. The |

| | | |
|---|---|---|
| | | enclosing class must also be `abstract`. |
| `default` | Method | Implementation of this interface method is optional. The interface provides a default implementation for classes that elect not to implement it. See Chapter 5 for more details. |
| `final` | Class | The class cannot be subclassed. |
| | Method | The method cannot be overridden. |
| | Field | The field cannot have its value changed. `static final` fields are compile-time constants when assigned at declaration. |
| | Variable | A local variable, method parameter, or exception parameter cannot have its value changed. |
| `native` | Method | The method is implemented in some platform-dependent way (often in C). No body is provided; the signature is followed by a semicolon. |
| `non-sealed` | Class | The class inherits from a sealed type but itself has unrestricted open inheritance. |
| <None> (package) | Class | A non-`public` class is accessible only in its package. |
| | Interface | A non-`public` interface is accessible only in its package. |
| | Member | A member that is not `private`, `protected`, or `public` has package visibility and is accessible only within its package. |
| `private` | Member | The member is accessible only within the class that defines it. |
| `protected` | Member | The member is accessible only within the package in which it is defined and within subclasses. |
| `public` | Class | The class is accessible anywhere its package is. |

| | | |
|---|---|---|
| | Interface | The interface is accessible anywhere its package is. |
| | Member | The member is accessible anywhere its class is. |
| `sealed` | Class | The class can be subclassed only by a specified list of subclasses, as given by the `permits` clause. If the `permits` clause is missing, the class can be subclassed only by classes within the same compilation unit. |
| `static` | Class | An inner class declared `static` is a top-level class not associated with a member of the containing class. See Chapter 5 for more details. |
| | Method | A `static` method is a class method. It is not passed an implicit `this` object reference. It can be invoked through the class name. |
| | Field | A `static` field is a class field. There is only one instance of the field, regardless of the number of class instances created. It can be accessed through the class name. |
| | Initializer | The initializer is run when the class is loaded rather than when an instance is created. |
| `strictfp` | Class | This modifier is extremely rarely used and is a no-op in Java 17 and later. |
| | Method | This modifier is extremely rarely used and is a no-op in Java 17 and later. |
| `synchronized` | Method | The method makes nonatomic modifications to the class or instance, so care must be taken to ensure that two threads cannot modify the class or instance at the same time. For a `static` method, a lock for the class is acquired before executing the method. For a non-`static` method, a lock for the specific object instance is acquired. See Chapter 12 for more details. |
| `transient` | Field | The field is not part of the persistent state of the object and should not be serialized with the object. Used with object serialization; see `java.io.ObjectOutputStream`. |

| | | |
|---|---|---|
| `volatile` | Field | The field can be accessed by unsynchronized threads, so certain optimizations must not be performed on it. This modifier is used for certain operations in multithreaded code. See Chapter 12 for more details. |

## Summary

Java, like all object-oriented languages, has its own model of how OO should work. In this chapter we have met the basic concepts of this model: static typing, fields, methods, inheritance, access control, encapsulation, overloading, overriding, and sealing. To become a proficient Java programmer, you will need to gain proficiency in handling all of these concepts and understand the relationship between them and how they interact.

The next two chapters are devoted to exploring these features further and understanding how the basic aspects of OOD in Java arise directly from this relatively small set of basic concepts.

---

1 There is also the default, aka package, visibility that we will meet later.

2 In Java bytecode, the `this` object is passed as the 0th parameter to the instance method call, and the "real" method parameters are 1-indexed.

3 Noninstantiable classes are used to group static fields and methods together into a logical unit.

4 Modules have a role to play in determining exactly when public classes are accessible, but in the absence of modules, `public` means "accessible everywhere."

5 An `abstract` method in Java is something like a pure virtual function in C++ (i.e., a virtual function that is declared `= 0`). In C++, a class that contains a pure virtual function is called an abstract class and cannot be instantiated. The same is true of Java classes that contain `abstract` methods.

# Chapter 5. The Java Type System

In this chapter, we move beyond basic object-oriented programming with classes and into the additional concepts required to work effectively with Java's static type system.

> **NOTE**
>
> A *statically typed* language is one in which variables have definite types, and where it is a compile-time error to assign a value of an incompatible type to a variable. Languages that check type compatibility at runtime are called *dynamically typed*.

Java is a fairly classic example of a statically typed language. JavaScript is an example of a dynamically typed language that allows any variable to store any type of value. Python started off as dynamically typed but is adding the ability to selectively adopt aspects of static typing—an approach that some people call *gradual typing*.

The Java type system involves not only classes and primitive types but also other kinds of reference types that are related to the basic concept of a class but that differ in some way and are usually treated in a special way by `javac` or the JVM.

We have already met arrays and classes, two of Java's most widely used kinds of reference types. This chapter starts by discussing another very important kind of reference type —*interfaces*. We then move on to discuss Java's *generics*, which have a major role to play in Java's type system. With these topics under our belts, we can discuss the differences between compile-time and runtime types in Java.

To complete the full picture of Java's reference types, we look at specialized kinds of classes and interfaces—known as *enums* and *annotations*. We conclude the chapter by looking at *lambda expressions* and *nested types* and then draw the threads together by discussing Java's type system as both type-inferred and nominal.

Let's get started by taking a look at interfaces—probably the most important of Java's reference types after classes and a key building block for the rest of Java's type system.

## Interfaces

In Chapter 4, we met the idea of inheritance. We also saw that a Java class can inherit only from a single class. This is quite a big restriction on the kinds of object-oriented programs that we want to build. The designers of Java knew this, but they also wanted to ensure that Java's approach to object-oriented programming was less complex and error-prone than, for example, that of C++ (we will revisit this topic in Chapter 6 and show a concrete example).

The solution that they chose was to introduce the concept of an interface to Java. Like a class, an

*interface* defines a new reference type. As its name implies, an interface is intended to represent only an API—so it provides a description of a type and the methods (and signatures) that classes that *implement* that API must provide.

In general, a Java interface does not provide any implementation code for the methods that it describes. These methods are considered *mandatory*—any class that wishes to implement the interface must provide an implementation of these methods.

However, an interface may wish to mark that some API methods are optional and that implementing classes do not need to implement them if they choose not to. This is done with the `default` keyword—and the interface must provide an implementation of these optional methods, which will be used by any implementing class that elects not to implement them.

> **NOTE**
>
> The ability to have optional methods in interfaces was not always part of the language—it was introduced in Java 8 (2014). See "Records and Interfaces" for a full description of how optional (also called *default*) methods work.

It is not possible to directly instantiate an interface and create a member of the interface type. Instead, a class must *implement* the interface to provide the necessary method bodies.

Any instances of the implementing class are *compatible with* both the type defined by the class and the type defined by the interface. This means that the instances may be substituted at any point in the code that requires an instance of either the class type or the interface type. This extends the Liskov principle that we met in "Reference Type Conversions".

Another way of saying this is that two objects that do not share the same class or superclass may still both be compatible with the same interface type if both objects are instances of classes that implement the interface.

One very common use case for interfaces that can be particularly effective is in testing code. Interfaces can be used to create an implementation that provides a *test double* (such as a dummy, stub, or mock) to allow developers to focus on only the parts of the code they're interested in testing and avoid the full system complexity during testing.

## Defining an Interface

An interface definition is somewhat like a class definition in which all the (mandatory) methods are abstract and the keyword `class` has been replaced with `interface`. For example, this code shows the definition of an interface named `Positioned` (a `Shape` class, such as those defined in Chapter 4, might implement this interface if it wants to allow the coordinates of its center to be queried):

```
interface Positioned {
  double getCenterX();
  double getCenterY();
}
```

A number of restrictions apply to the members of an interface:

- All mandatory methods of an interface are implicitly `abstract` and must have a semicolon in place of a method body. The `abstract` modifier is allowed but by convention is usually omitted.
- An interface defines a public API. By convention, members of an interface are implicitly `public`, and it is conventional to omit the unnecessary `public` modifier.
- It is a compile-time error to try to define a `protected` method in an interface.
- An interface may not define any instance fields. Fields are an implementation detail, and an interface is a specification, not an implementation. The only fields allowed in an interface definition are constants that are declared both `static` and `final`.
- An interface cannot be instantiated, so it may not define a constructor.
- Interfaces may contain nested types. Any such types are implicitly `public` and `static`. See “Nested Types” for a full description of nested types.
- Interfaces may contain `static` methods.
- Interfaces may contain `private` methods.

Note that early versions of Java did not allow static or private methods in interfaces, which is widely believed to have been a design flaw in the Java language.

## Extending Interfaces

Interfaces may extend other interfaces, and, like a class definition, an interface definition indicates this by including an `extends` clause. When one interface extends another, it inherits all the methods and constants of its superinterface and can define new methods and constants. Unlike classes, however, the `extends` clause of an interface definition may include more than one superinterface. For example, here are some interfaces that extend other interfaces:

```
interface Positionable extends Positioned {
  void setCenter(double x, double y);
}
interface Transformable extends Positionable, Scalable, Rotatable {}
```

An interface that extends more than one interface inherits all the methods and constants from each of those interfaces and can define its own additional methods and constants. A class that implements such an interface must implement the abstract methods defined directly by the interface, as well as all the abstract methods inherited from all the superinterfaces.

In general, a good design principle is for each interface to be limited in scope to only those methods that it should care about—this is known as *interface segregation* and is a good code hygiene practice, especially in larger codebases.

## Implementing an Interface

Just as a class uses `extends` to specify its superclass, it can use `implements` to name one or more interfaces it supports. The `implements` keyword can appear in a class declaration following the `extends` clause. It should be followed by a comma-separated list of interfaces that the class implements.

When a class declares an interface in its `implements` clause, it is saying that it provides an implementation (i.e., a body) for each mandatory method of that interface. If a class implements an interface but does not provide an implementation for every mandatory interface method, it inherits those unimplemented `abstract` methods from the interface and must itself be declared `abstract`. If a class implements more than one interface, it must implement every mandatory method of each interface it implements (or be declared `abstract`).

The following code shows how to define a `PositionedRectangle` class. This is similar to the example in Chapter 4, but instead of extending `Rectangle` and adding extra functionality, here we explicitly say we are implementing the `Positioned` interface:

```
public final class PositionedRectangle extends Rectangle
    implements Positioned {

    private final double cx;
    private final double cy;

    public PositionedRectangle(double cx, double cy,
                               double w, double h) {
        this.cx = cx;
        this.cy = cy;
        super(w, h);
    }

    // We inherit all the methods of Rectangle but must
    // provide implementations of all the Positioned methods.
    public double getCenterX() { return cx; }
    public double getCenterY() { return cy; }
}
```

Suppose we implement `PositionedCircle` and `PositionedSquare` just as we have implemented this `PositionedRectangle` class. Each class extends `Shape`, so instances of the classes can be treated as instances of the `Shape` class, as we saw earlier. Because each class implements the `Positioned` interface, instances can also be treated as instances of that type. The following code demonstrates how objects can be members of both a class type and an interface type:

```
var shapes = new Shape[2];      // Create an array to hold shapes

// Create some centered shapes, and store them in the Shape[]
// No cast necessary: these are all compatible assignments
shapes[0] = new PositionedCircle(1.0, 1.0, 1.0);
shapes[1] = new PositionedRectangle(2.3, 4.5, 3, 4);
```

```
// Compute average area of the shapes and
// average distance from the origin
double totalArea = 0;
double totalDistance = 0;
for(int i = 0; i < shapes.length; i += 1) {
  totalArea += shapes[i].area();   // Compute the area of the shapes

  // We're using the "safe instanceof" pattern to ensure the shape has
  // a position
  if (shapes[i] instanceof Positioned p) { // The shape is a
                                           // Positioned shape
    double cx = p.getCenterX();    // Get coordinates of the center
    double cy = p.getCenterY();    // And compute distance from origin
    totalDistance += Math.sqrt(cx * cx + cy * cy);
  }
}
IO.println("Average area: " + totalArea / shapes.length);
IO.println("Average distance: " + totalDistance / shapes.length);
```

Another way of saying this is that interfaces are data types in Java, just like classes. When a class implements an interface, instances of that class can be assigned to variables of the interface type.

Don't interpret this example to imply that you must assign a `PositionedRectangle` object to a `Positioned` variable before you can invoke the `getCenterX()` method, or to a `Shape` variable before invoking the `area()` method. Instead, because the `PositionedRectangle` class defines `getCenterX()` and inherits `area()` from its `Rectangle` superclass, you can always invoke these methods.

As we could see by examining the bytecode (e.g., by using the `javap` tool we will meet in Appendix B), the JVM calls the `getCenterX()` method slightly differently depending on whether the local variable holding the shape is of the type `PositionedRectangle` or `Positioned`, but this is not a distinction that matters most of the time when you're writing Java code.

## Records and Interfaces

Records, being a special case of classes, can implement interfaces, just like any other class. The body of the record must contain implementation code for all of the mandatory methods of the interface, and it may contain overriding implementations for any of the default methods of the interface.

Let's look at an example as applied to the `Point` record we met in the last chapter. Given an interface defined like this:

```
interface Translatable {
    Translatable deltaX(double dx);
    Translatable deltaY(double dy);
    Translatable delta(double dx, double dy);
}
```

then we can update the `Point` type like this:

```
public record Point(double x, double y) implements Translatable {
    public Translatable deltaX(double dx) {
        return delta(dx, 0.0);
    }

    public Translatable deltaY(double dy) {
        return delta(0.0, dy);
    }

    public Translatable delta(double dx, double dy) {
        return new Point(x + dx, y + dy);
    }
}
```

Note that because records are immutable, it is not possible to mutate instances in place, and so, if we need a modified object, we have to create one and return it explicitly. This implies that not every interface will be suitable for implementation by a record type. For example, the `Positionable` interface contains a setter method (which returns `void`), and this carries the implication that classes that implement `Positionable` will mutate instances to update their position, which is not possible for records.

This is an important aspect of OOD, and we'll discuss it more in Chapter 6.

## Sealed Interfaces

We met the `sealed` keyword in the last chapter, as applied to classes. It can also be applied to interfaces, like this:

```
sealed interface Rotate90 permits Circle, Rectangle {
    void clockwise();
    void antiClockwise();
}
```

In our current data model, we don't want to deal with the complexity of allowing other shapes to have rotational behavior, so we restrict the interface so that it can be implemented only by the two simplest cases: circles and rectangles. This sealed interface represents the capability for a shape to be rotated by 90 degrees, and the declaration also contains a `permits` clause that specifies the only types—`Circle` and `Rectangle`—that are allowed to implement this interface.

We can modify the `Circle` class from Chapter 4 like this to accommodate the new rotation capability:

```
public final class Circle extends Shape implements Rotate90 {
    // ...

    @Override
    public void clockwise() {
        // No-op, circles are rotation-invariant
    }
```

```
    @Override
    public void antiClockwise() {
        // No-op, circles are rotation-invariant
    }

    // ...
}
```

whereas the `Rectangle` has been modified like this:

```
public final class Rectangle extends Shape implements Rotate90 {
    // ...

    @Override
    public void clockwise() {
        // Swap width and height
        double tmp = w;
        w = h;
        h = tmp;
    }

    @Override
    public void antiClockwise() {
        // Swap width and height
        double tmp = w;
        w = h;
        h = tmp;
    }

    // ...
}
```

There is also an interesting interplay between sealed interfaces and records, which we will discuss in Chapter 6.

## Default Methods

From Java 8 onward, it is possible to declare methods in interfaces that include an implementation. In this section, we'll discuss these methods, which should be understood as optional methods in the API the interfaces represent—they're usually called *default methods*. Let's start by looking at the reasons why we need the default mechanism in the first place.

### Backward compatibility

The Java platform has always been very committed to backward compatibility. This means that code that was written (or even compiled) for an earlier version of the platform must continue to work with later releases of the platform. This principle allows development groups to have a high degree of confidence that an upgrade of their JDK or Java Runtime Environment (JRE) will not break currently working applications.

Backward compatibility is a great strength of the Java platform, but to achieve it, some constraints are placed on the platform. One of them is that interfaces may not have new

mandatory methods added to them in a new release of the interface.

For example, let's suppose that we want to update the `Positionable` interface with the ability to add a top-right bounding point as well as a center:

```
public interface Positionable extends Positioned {
  void setCenter(double x, double y);
  void setUpperRightCorner(double x, double y);
  double getUpperRightX();
  double getUpperRightY();
}
```

With this new definition, if we try to use this new interface with code developed for the old, it just won't work, as the existing code is missing the mandatory methods `setUpperRightCorner()`, `getUpperRightX()`, and `getUpperRightY()`.

> **NOTE**
>
> You can see this effect quite easily in your own code. Compile a class file that depends on an interface. Then add a new mandatory method to the interface and try to run the program with the new version of the interface, together with your old class file. You should see the program crash with a `NoClassDefError`.

This limitation was a concern for the designers of Java 8—as one of their goals was to be able to upgrade the core Java Collections libraries and introduce methods that used lambda expressions.

To solve this problem, a new mechanism was needed, essentially to allow interfaces to evolve by allowing new methods to be added without breaking backward compatibility.

### Implementation of default methods

Adding new methods to an interface without breaking backward compatibility requires the interface writer to provide some implementation for the older implementations of the interface so that they can continue to work.

> **NOTE**
>
> A default method (sometimes called an *optional method*) can be added to any interface. This must include an implementation, called the *default implementation*, which is written inline in the interface definition.

The basic behavior of a default method is:

- An implementing class may (but is not required to) implement the default method.
- If an implementing class implements the default method, then the implementation in the class is used.
- If no other implementation can be found, then the default implementation is used.

An example default method is the `sort()` method. It was added to the interface `java.util.List` in JDK 8 and is defined as:

```
// The <E> syntax is Java's way of writing a generic type - see
// the next section for full details. If you aren't familiar with
// generics, just ignore that syntax for now.
interface List<E> {
  // Other members omitted

  default void sort(Comparator<? super E> c) {
    Collections.<E>sort(this, c);
  }
}
```

Thus, any object that implements `List` has an instance method `sort()` that can be used to sort the list using a suitable `Comparator`. As the return type is `void`, we might expect that this is an in-place sort, and this is indeed the case.

One consequence of default methods is that when implementing multiple interfaces, it's possible that two or more interfaces may contain a default method with a completely identical name and signature.

For example:

```
interface Vocal {
  default void call() {
    System.out.println("Hello!");
  }
}

interface Caller {
  default void call() {
    Switchboard.placeCall(this);
  }
}

public class Person implements Vocal, Caller {
  // ... which default is used?
}
```

These two interfaces have very different default semantics for `call()` and could cause a potential implementation clash—a *colliding default method*. In versions of Java prior to 8, this could not occur, as the language permitted only single inheritance of implementation. The introduction of default methods means that Java now permits a limited form of *multiple inheritance* (but only of method implementations). Java still does not permit (and has no plans to add) multiple inheritance of object state.

**TIP**

In some other languages, notably C++, the problem of possibly colliding multiple inheritance is known as *diamond inheritance*.

Default methods have a simple set of rules to help resolve any potential ambiguities:

- If a class implements multiple interfaces in such a way as to cause a potential clash of default method implementations, the implementing class must override the clashing method and provide a definition of what is to be done.
- Syntax is provided to allow the implementing class to simply call one of the interface default methods if that is what is desired

However, either way, the ambiguity must be resolved, and a class that implements both interfaces *must* define any colliding methods:

```
public class Person implements Vocal, Caller {

    public void call() {
        // Can do our own thing
        // or delegate to either interface
        // e.g.,
        // Vocal.super.call();
        // or
        // Caller.super.call();
    }
}
```

When working with default methods, we should be aware that there is a slightly restricted set of operations we can perform from within a default method. We can:

- Call another method present in the interface's public API (whether mandatory or optional); some implementation for such methods is guaranteed to be available.
- Call a private method on the interface (Java 9 and up).
- Call a static method, whether on the interface or defined elsewhere.
- Use the `this` reference (e.g., as an argument to method calls).

The biggest takeaway from these restrictions is that even with default methods, Java interfaces still lack meaningful state; we cannot alter or store state within the interface.

There is also a very small, unavoidable usage issue that may arise in the case of evolving interfaces with colliding methods. This is essentially a side effect of the design of default methods and is worth a quick look.

Consider the case where a class implements two separate interfaces `A` and `B` with version numbers `a.0` and `b.0`, respectively, that do not have default methods. If at a later time the interfaces change and adopt a default implementation of a method, then compile-time errors can occur.

For example, if version `a.1` introduces a default method in `A`, then the implementing class will pick up the implementation when run with the new version of the dependency. If version `b.1` of

`B` also introduces the same method, it can cause a collision:

- If `B` introduces the method as a mandatory (i.e., abstract) method, then the implementing class continues to work—both at compile time and at runtime, because the default implementation from `A` is used.
- If `B` introduces the method as a default method, then this is not safe and the implementing class will fail both at compile and at runtime, if previously compiled classes were used.

This minor issue is very much a corner case and in practice is considered a very small price to pay in order to have usable default methods in the language.

Default methods have had a profound impact on the way that Java practitioners approach object-oriented programming. When combined with the rise of lambda expressions, they have upended many previous conventions of Java coding; we will discuss this in detail in the next chapter.

## Marker Interfaces

Occasionally it is useful to define an interface that is entirely empty. A class can implement this interface simply by naming it in its `implements` clause without having to implement any methods. In this case, any instances of the class become valid instances of the interface as well and can be cast to the type.

Java code can check whether an object is an instance of the interface using the `instanceof` operator, so this technique is a useful way to provide additional information about an object. It can be thought of as providing additional, auxiliary type information about a class.

> **TIP**
>
> Marker interfaces are much less widely used than they once were. Java's *annotations* (which we shall meet very soon) have largely replaced them due to their much greater flexibility at conveying extended type information.

The interface `java.util.RandomAccess` is an example of a marker interface: `java.util.List` implementations use this interface to advertise that they provide fast random access to the elements of the list. For example, `ArrayList` implements `RandomAccess`, while `LinkedList` does not. Algorithms that care about the performance of random-access operations can test for `RandomAccess` like this:

```
// Before sorting the elements of a long arbitrary list, we may want
// to make sure that the list allows fast random access.  If not,
// it may be quicker to make a random-access copy of the list before
// sorting it. Note that this is not necessary when using
// java.util.Collections.sort().
List l = List.of(1, 2);  // Some arbitrary list we're given
if (l.size() > 2 && !(l instanceof RandomAccess)) {
    l = new ArrayList(l);
```

```
}
sortListInPlace(l);
```

As we will see later, Java's type system is very tightly coupled to the names that types have—an approach called *nominal typing*. A marker interface is a great example of this: it has nothing at all *except* a name.

# Java Generics

One of the great strengths of the Java platform is the standard library it ships. Java's standard library provides a great deal of useful functionality—and in particular robust implementations of common data structures. These implementations are relatively simple to develop with and are well documented.

The libraries are known as the Java Collections, and we'll spend a big chunk of Chapter 8 discussing them. For a more complete treatment, see *Java Generics and Collections* by Maurice Naftalin and Philip Wadler with Stuart Marks (O'Reilly).

Although they were still very useful, the earliest versions of the collections had a fairly major limitation: the data structure (sometimes called the *container*) essentially obscured the type of the data being stored in it.

> **NOTE**
>
> Data hiding and encapsulation are great principles of object-oriented programming, but in the case of opaque container types, they can cause a lot of significant problems for the developer.

Let's kick off the section by demonstrating the problem and showing how the introduction of *generic types* solved it and made life much easier for Java developers.

## Introduction to Generics

If we want to build a collection of `Shape` instances, we can use a `List` to hold them:

```
List shapes = new ArrayList();   // Create a List to hold shapes

// Create some shapes, and store them in the list
shapes.add(new Circle(1.0));

// This is legal Java - but it's a very bad design choice
shapes.add(new Rectangle(2.5, 2.0));

// List::get() returns Object, so to get back a Circle we must cast
Circle c = (Circle)shapes.get(0);

// Next line causes a runtime failure
Circle c = (Circle)shapes.get(1);
```

A problem with this code stems from the requirement to perform a cast to get the shape objects back out in a usable form—the `List` doesn't know what type of objects it contains. Not only that, but it's actually possible to put different types of objects into the same container, and everything will work fine until an illegal cast is used and the program crashes.

What we really want is a form of `List` that understands what type it contains. Then, `javac` could detect when an illegal argument was passed to the methods of `List` and cause a compilation error, rather than deferring the potential problem to runtime.

> **NOTE**
>
> Collections that have all elements of the same type are called *homogeneous*, while collections that can have elements of potentially different types are called *heterogeneous* (sometimes called *mystery meat* collections).

Java provides a simple syntax to cater to homogeneous collections. To indicate that a type is a container that holds instances of another reference type, we enclose the *payload* type that the container holds within angle brackets:

```
// Create a List-of-Circle
List<Circle> shapes = new ArrayList<Circle>();

// Create a circle, and store it in the list
shapes.add(new Circle(1.0));

// Next line will cause a compilation error
shapes.add(new Rectangle(2.5, 2.0));

// List<Circle>::get() returns a Circle, no cast needed
Circle c = shapes.get(0);
```

This syntax ensures that a large category of unsafe code is caught by the compiler, before it gets anywhere near runtime. This is, of course, the whole point of static type systems—to use compile-time knowledge to help eliminate runtime problems wherever possible.

The resulting types, which combine an enclosing container type and a payload type, are usually called *generic types*, and they are declared like this:

```
interface Box<T> {
  void box(T t);
  T unbox();
}
```

This indicates that the `Box` interface is a general construct, which can hold any type of payload. It isn't really a complete interface by itself—it's more like a general description of a whole family of interfaces, one for each type that can be used in place of `T`.

## Generic Types and Type Parameters

We've seen how to use a generic type to provide enhanced program safety by using compile-time knowledge to prevent simple type errors. In this section, let's dig deeper into the properties of generic types.

The syntax `<T>` has a special name, *type parameter*, and another name for a generic type is a *parameterized type*. This should convey the sense that the container type (e.g., `List`) is parameterized by another type (the payload type). When we write a type like `Map<String, Integer>`, we are assigning concrete values to the type parameters.

When we define a type that has parameters, we need to do so in a way that does not make assumptions about the type parameters. So the `List` type is declared in a generic way as `List<E>`, and the type parameter `E` is used all the way through to stand as a placeholder for the actual type that programmers will use for the payload when they use the `List` data structure.

> **TIP**
>
> A type parameter always stands in for a reference type of some sort. It is not possible to use a primitive type as a value for a type parameter.[1]

The type parameter can be used in the signatures and bodies of methods as though it is a real type, for example:

```
interface List<E> extends Collection<E> {
  boolean add(E e);
  E get(int index);
  // other methods omitted
}
```

Note how the type parameter `E` can be used as a parameter for both return types and method arguments. We don't assume that the payload type has any specific properties and make only the basic assumption of consistency—that the type we put in is the same type that we will later get back out.

> **TIP**
>
> Generics can be nested to any level—so it is entirely possible to have a "list-of-lists-of-String," or `List<List<String>>`

This enhancement has effectively introduced a new kind of type to Java's type system. By combining the container type with the value of the type parameter, we are making new types—in fact new families of types.

## Diamond Syntax

When we create an instance of a generic type, the righthand side of the assignment statement repeats the value of the type parameter. This is usually unnecessary, as the compiler can deduce the values of the type parameters. In modern versions of Java, we can leave out the repeated type values in what is called *diamond syntax*.

Let's look at an example of how to use diamond syntax, by rewriting one of our earlier examples:

```
// Create a List-of-Circle using diamond syntax
List<Circle> shapes = new ArrayList<>();
```

This is a small improvement in the verbosity of the assignment statement—we've managed to save a few characters of typing. We'll return to the topic of type inference several times later in this chapter.

## Type Erasure

In "Default Methods", we discussed the Java platform's strong preference for backward compatibility. The addition of generics in Java 5 was another example where backward compatibility was an issue for a new language feature.

The central question was how to make a type system that allowed older, nongeneric collection classes to be used alongside newer, generic collections. The design decision was to achieve this by the use of casts:

```
List someThings = getSomeThings();
// Unsafe cast, but we know that the
// contents of someThings are really strings
List<String> myStrings = (List<String>)someThings;
```

This means that `List` and `List<String>` are compatible as types, at least at some level. Java achieves this compatibility by *type erasure*. This means that generic type parameters are visible only at compile time—they are stripped out by `javac` and are not reflected in the bytecode.[2]

> **WARNING**
>
> The nongeneric type `List` is usually called a *raw type*. It is still perfectly legal Java to work with the raw form of types, even for types that are now generic. This is almost always a sign of poor-quality code, however.

The mechanism of type erasure gives rise to a difference in the type system seen by `javac` and that seen by the JVM—we will discuss this fully in "Compile and Runtime Typing".

Type erasure also prohibits some other definitions, which would otherwise seem legal. In this code, we want to count the orders as represented in two slightly different data structures:

```
// Won't compile
```

```
interface OrderCounter {
  // Name maps to list of order numbers
  int totalOrders(Map<String, List<String>> orders);

  // Name maps to total orders made so far
  int totalOrders(Map<String, Integer> orders);
}
```

This seems like perfectly legal Java code, but it will not compile. The issue is that although the two methods seem like normal overloads, after type erasure, the signature of both methods becomes:

```
  int totalOrders(Map orders);
```

All that is left after type erasure is the raw type of the container—in this case, `Map`. The runtime would be unable to distinguish between the methods by signature, and so the language specification makes this syntax illegal.

In general, we would strongly advise beginning Java programmers to avoid using raw types—although the generic forms may seem like extra complexity, the extra safety and clarity quickly pays off.

## Bounded Type Parameters

Consider a simple generic box:

```
public class Box<T> {
    protected T value;

    public void box(T t) {
        value = t;
    }

    public T unbox() {
        T t = value;
        value = null;
        return t;
    }
}
```

This is a useful abstraction, but suppose we want to have a restricted form of box that holds only numbers. Java allows us to achieve this by using a *bound* on the type parameter. This is the ability to restrict the types that can be used as the value of a type parameter, for example:

```
public class NumberBox<T extends Number> extends Box<T> {
    public int intValue() {
        return value.intValue();
    }
}
```

The type bound `T extends Number` ensures that `T` can be substituted only with a type that is

compatible with the type `Number`. As a result of this, the compiler knows that `value` will definitely have a method `intValue()` available on it.

> **NOTE**
>
> Notice that because the `value` field has protected access, it can be accessed directly in the subclass.

If we attempt to instantiate `NumberBox` with an invalid value for the type parameter, the result will be a compilation error:

```
NumberBox<Integer> ni = new NumberBox<>(); // This compiles fine

NumberBox<Object> no = new NumberBox<>(); // Won't compile
```

In general, type bounds can be used to write better generic code and libraries. With practice, some fairly complex constructions can be built, for example:

```
public class ComparingBox<T extends Comparable<T>> extends Box<T>
                          implements Comparable<ComparingBox<T>> {
    @Override
    public int compareTo(ComparingBox<T> o) {
        if (value == null)
            return o.value == null ? 0 : -1;
        return value.compareTo(o.value);
    }
}
```

The definition might seem daunting, but the `ComparingBox` is really just a `Box` that contains a `Comparable` value. The type also extends the comparison operation to the `ComparingBox` type itself, just by comparing the contents of the two boxes. The method implementation relies on the fact that the box contents are `Comparable`, whereas the method would be called by someone wanting to compare two `ComparingBox` objects (e.g., when sorting a `List<ComparingBox<T>>`).

One minor word of caution about using raw types when working with a type bound: the type bound can be evaded, but in doing so, the code is left vulnerable to a runtime exception:

```
// Compiles
NumberBox n = new NumberBox();
// This is very dangerous
n.box(new Object());
// Runtime error
System.out.println(n.intValue());
```

The call to `intValue()` fails with a `java.lang.ClassCastException`—as `javac` has inserted an unconditional cast of `value` to `Number` before calling the method.

We should also note that this does at least give a warning. For example, JShell gives the

following:

```
|  Warning:
|  unchecked call to box(T) as a member of the raw type Box
|  n.box(new Object());
|  ^-----------------^
```

Let's move on to see how the design of Java's generics contains the solution to a (very old) problem in the type system.

## Introducing Covariance

In the earliest versions of Java, before the collections libraries were even introduced, the language had been forced to confront a deep-seated type system design issue.

Put simply, the question is this:

> *Should an array of strings be compatible with a variable of type array-of-object?*

In other words, should this code be legal?

```
String[] words = {"Hello World!"};
Object[] objects = words;
```

Without this, then even simple methods like `Arrays::sort` would have been very difficult to write in a useful way, as this would not work as expected:

```
Arrays.sort(Object[] a);
```

The method declaration would work only for the type `Object[]` and not for any other array type. As a result of these complications, the very first version of the Java Language Specification determined that:

> *If a value of type `C` can be assigned to a variable of type `P`, then a value of type `C[]` can be assigned to a variable of type `P[]`.*

To restate this slightly: an array's assignment syntax *varies with* the base type that it holds—that is, arrays are *covariant*.

This design decision is rather unfortunate, as it leads to immediate negative consequences:

```
String[] words = {"Hello", "World!"};
Object[] objects = words;

// Oh, dear, runtime error
objects[0] = new Integer(42);
```

The assignment to `objects[0]` attempts to store an `Integer` into a piece of storage that is expecting to hold a `String`. This obviously will not work and will throw an

`ArrayStoreException`.

> **WARNING**
>
> The usefulness of covariant arrays led to them being seen as a necessary evil in the very early days of the platform, despite the hole in the static type system that the feature exposes.

However, more recent research on modern open source codebases indicates that array covariance is extremely rarely used and is a language misfeature.[3] You should avoid it when writing new code.

When considering the behavior of generics in the Java platform, a very similar question can be asked: "Is `List<String>` a subtype of `List<Object>`?" That is, can we write this:

```
// Is this legal?
List<Object> objects = new ArrayList<String>();
```

At first glance, this seems entirely reasonable—`String` is a subclass of `Object`, so we know that any `String` element in our collection is also a valid `Object`.

However, consider the following code (which is just the array covariance code translated to use `List`):

```
// Is this legal?
List<Object> objects = new ArrayList<String>();

// What do we do about this?
objects.add(new Object());
```

As the type of `objects` was declared to be `List<Object>`, then it should be legal to add an `Object` instance to it. However, since the actual instance holds strings, then trying to add an `Object` would not be compatible, and so this would fail at runtime.

This would have changed nothing from the case of arrays, and so the resolution is to realize that although this is legal:

```
Object o = new String("X");
```

that does not mean that the corresponding statement for generic container types is also true, and as a result:

```
// Won't compile
List<Object> objects = new ArrayList<String>();
```

Another way of saying this is that `List<String>` is *not* a subtype of `List<Object>`, or that generic types are *invariant*, not *covariant*. This is related to the topic of wildcards, which we

will now discuss.

## Wildcards

A parameterized class, such as `ArrayList<T>`, is not *instantiable*; we cannot create instances of them. This is because `<T>` is just a type parameter, or merely a placeholder for a genuine type. It is only when we provide a concrete value for the type parameter (e.g., `ArrayList<String>`) that the class becomes fully formed, and we can create objects of that type.

This poses a problem if the type that we want to work with is unknown at compile time. Fortunately, the Java type system is able to accommodate this concept. It does so by having an explicit concept of the *unknown type*, which is represented as `<?>`. This is the simplest example of Java's *wildcard types*.

We can write expressions that involve the unknown type:

```
ArrayList<?> mysteryList = unknownList();
Object o = mysteryList.get(0);
```

This is perfectly valid Java: `ArrayList<?>` is a complete type that a variable can have, unlike `ArrayList<T>`. We don't know anything about `mysteryList`'s payload type, but that may not be a problem for our code.

For example, when we get an item out of `mysteryList`, it has a completely unknown type. However, we can be sure that the object is assignable to `Object`—because all valid values of a generic type parameter are reference types, and all reference values can be assigned to a variable of type `Object`.

On the other hand, when we're working with the unknown type, there are some limitations on its use. For example, this code will not compile:

```
// Won't compile
mysteryList.add(new Object());
```

The reason for this is simple: we don't know what the payload type of `mysteryList` is! For example, if `mysteryList` was really an instance of `ArrayList<String>`, then we wouldn't expect to be able to put an `Object` into it.[4]

The only value that we know we can always insert into a container is `null`, since we know that `null` is a possible value for any reference type. This isn't that useful, and for this reason, the Java Language Specification also rules out instantiating a container object with the unknown type as payload, for example:

```
// Won't compile
List<?> unknowns = new ArrayList<?>();
```

The unknown type may seem to be of limited utility, but one very important use for it is as a starting point for resolving the covariance question. We can use the unknown type if we want to have a subtyping relationship for containers, like this:

```
// Perfectly legal
List<?> objects = new ArrayList<String>();
```

This means that `List<String>` *is* a subtype of `List<?>`—although when we use an assignment like the preceding one, we have lost some type information. For example, the return type of `objects.get()` is now effectively `Object`.

> **NOTE**
>
> For any value of the type parameter `T`, `List<?>` is not a subtype of the type `List<T>`.

The unknown type sometimes confuses developers—provoking questions like, "Why wouldn't you just use `Object` instead of the unknown type?" However, as we've seen, the need to have subtyping relationships between generic types essentially requires us to have a notion of the unknown type.

### Bounded wildcards

In fact, Java's wildcard types extend beyond just the unknown type, with the concept of *bounded wildcards*.

These are used to describe the inheritance hierarchy of a mostly unknown type—effectively making statements like, for example, "I don't know anything about this type, except that it must implement `List`."

This would be written as `? extends List` in the type parameter. This provides a useful lifeline to programmers. Instead of being restricted to the totally unknown type, they know that at least the capabilities of the type bound are available.

> **WARNING**
>
> The `extends` keyword is always used, regardless of whether the constraining type is a class or interface type.

This is an example of a concept called *type variance*, which is the general theory of how inheritance between container types relates to the inheritance of their payload types.

*Type covariance*

> This means that the container types have the same relationship with one another as the payload types do. This is expressed using the `extends` keyword.

*Type contravariance*

> This means that the container types have the inverse relationship to one another as the payload types. This is expressed using the `super` keyword.

These ideas tend to appear when discussing container types. For example, if `Cat` extends `Pet`, then `List<Cat>` is a subtype of `List<? extends Pet>`, and so:

```
List<Cat> cats = new ArrayList<Cat>();
List<? extends Pet> pets = cats;
```

However, this differs from the array case, because type safety is maintained in the following way:

```
pets.add(new Cat()); // won't compile
pets.add(new Pet()); // won't compile
cats.add(new Cat());
```

The compiler cannot prove that the storage pointed at by `pets` is capable of storing a `Cat`, and so it rejects the call to `add()`. However, as `cats` definitely points at a list of `Cat` objects, then it must be acceptable to add a new one to the list.

As a result, it is very commonplace to see these types of generic constructions with types that act as producers or consumers of payload types.

For example, when the `List` is acting as a *producer* of `Pet` objects, then the appropriate keyword is `extends`:

```
Pet p = pets.get(0);
```

Note that for the producer case, the payload type appears as the return type of the producer method.

For a container type that is acting purely as a *consumer* of instances of a type, we would use the `super` keyword, and we would expect to see the payload as the type of a method argument.

> **NOTE**
>
> This is codified in the *Producer Extends, Consumer Super* (PECS) principle coined by Joshua Bloch. It can be understood as "a producer is allowed to produce something more specific than `T`, which is expressed as `<? extends T>`, while a consumer is allowed to accept something more general than `T`, i.e., `<? super T>`."

As we will discuss in Chapter 8, both covariance and contravariance appear throughout the Java Collections. They exist largely to ensure that the generics just "do the right thing" and behave in a manner that should not surprise the developer.

## Generic Methods

A *generic method* is a method that is able to take instances of any reference type.

Let's look at an example. In Java, the comma is used to allow multiple declarations in a single line (usually referred to as a *compound declaration*). Other languages, such as JavaScript or C, have a comma operator that is much more general.

For instance, the JavaScript comma operator (`,`) evaluates both expressions provided to it (from left to right) and returns the value of the last expression. The aim is to create a compound expression in which multiple expressions are evaluated, with the compound expression's value being the value of the rightmost of its member expressions. Note that any side effects from evaluating the expressions to the comma are always triggered, unlike in a short-circuiting logic operator.

Java's comma is much more restrictive than this, by design. This is because the comma in other languages can lead to hard-to-understand code and can be a fantastic source of bugs. However, if we did want to emulate the behavior of the comma operator from some other language, we could do so by creating a generic method:

```
// Note that this class is not generic
public final class Utils {
  private Utils() {}

  public static <T> T comma(T a, T b) {
    return b;
  }
}
```

Calling the method `Utils.comma()` will cause the values of the expressions `a` and `b` to be computed, and any side effects to be triggered, before the method call, which is the behavior we want.

However, notice that even though a type parameter is used in the definition of the method, the class it is defined in (`Utils`) is not generic. Instead, we see that a new syntax is used to indicate that the method can be used freely and that the return type is the same as the argument.

Let's look at another example, from the Java Collections library. In the `ArrayList<E>` class, we can find a method to create a new array object from an `ArrayList` instance:

```
@SuppressWarnings("unchecked")
public <T> T[] toArray(T[] a) {
    if (a.length < size)
        // Make a new array of a's runtime type, but my contents:
        return (T[]) Arrays.copyOf(elementData, size, a.getClass());
    System.arraycopy(elementData, 0, a, 0, size);
    if (a.length > size)
        a[size] = null;
    return a;
}
```

This method uses the low-level `arraycopy()` method to do the actual work, and it is an instance method because it uses the `elementData` and `size` fields of the list object being copied to an array.

> **NOTE**
>
> The `toArray()` method provides one half of a bridge API between the collections and Java's original arrays. The other half of the API—moving from arrays to collections—involves a few additional subtleties, as we will discuss in Chapter 8.

If we look at the class definition for `ArrayList`, we can see that the type parameter is `<E>`, not `<T>`, and the type parameter `<E>` does not appear at all in the definition of the generic method `toArray()`. Generic methods aren't required to use the same type parameter as their enclosing class—and, in fact, generic methods can appear in nongeneric classes.

## Compile and Runtime Typing

Consider an example piece of code:

```
List<String> l = new ArrayList<>();
System.out.println(l);
// Outputs "[]'
```

We can ask the following question: what is the type of `l`? The answer to that question depends on whether we consider `l` at compile time (i.e., the type seen by `javac`) or at runtime (as seen by the JVM).

`javac` will see the type of `l` as `List` of `String` and will use that type information to carefully check for syntax errors, such as an attempted `add()` of an illegal type.

Conversely, the JVM will see `l` as an object of type `ArrayList`, as we can see from the output of the `println()` statement. The runtime type of `l` is a raw type due to type erasure.

The compile-time and runtime types are therefore slightly different from each other. The slightly strange thing is that in some ways, the runtime type is both more *and* less specific than the compile-time type.

The runtime type is less specific than the compile-time type because the type information about the payload type is gone—it has been erased, and the resulting runtime type is just a raw type.

The compile-time type of `l` is less specific than the runtime type because we don't know exactly what concrete type `l` will be; all we know is that it will be of a type compatible with `List`.[5]

The differences between compile-time and runtime typing sometimes confuse new Java programmers, but the distinction quickly comes to be seen as a normal part of working in the language.

### Using and Designing Generic Types

When working with Java's generics, it can be helpful to think in terms of two levels of understanding:

*Practitioner*

> A practitioner needs to use existing generic libraries and to build some fairly simple generic classes. At this level, the developer should also understand the basics of type erasure, as several Java syntax features are confusing without at least an awareness of the runtime handling of generics.

*Designer*

> The designer of new libraries that use generics needs to understand much more about the capabilities of generics. There are some nastier parts of the specification, including a full understanding of wildcards and advanced topics such as "capture-of" error messages.

Java generics are one of the most complex parts of the language specification with a lot of potential corner cases. Not every developer needs to fully understand this part of the language, at least not on their first encounter with this part of Java's type system.

## Enums and Annotations

We have already met records, but Java has additional specialized forms of classes and interfaces used to fulfill specific roles in the type system. They are known as *enumerated types* and *annotation types*, or normally just *enums* and *annotations*.

### Enums

Enums are a variation on classes that have limited functionality and the specific semantic meaning that the type has a small number of possible permitted values.

For example, suppose we want to define a type to represent the primary colors of red, green, and blue, and we want these to be the only possible values of the type. We can do this by using the `enum` keyword:

```
public enum PrimaryColor {
  // The ; is not required at the end of the list of instances
  RED, GREEN, BLUE
}
```

The only available instances of the type `PrimaryColor` can then be referenced as static fields: `PrimaryColor.RED`, `PrimaryColor.GREEN`, and `PrimaryColor.BLUE`.

> **NOTE**

> In other languages, such as C++, the role of enum types is fulfilled by using constant integers, but Java's approach provides better type safety and more flexibility.

As enums are specialized classes, enums can have member fields and methods. If they do have a body (consisting of fields or methods), then the semicolon at the end of the list of instances is required, and the list of enum constants must precede the methods and fields.

For example, suppose that we want to have an enum that encompasses the suits of standard playing cards. We can achieve this by using an enum that takes a value as a parameter, like this:

```
public enum Suit {
    // ; at the end of list required for enums with parameters
    HEART('♥'),
    CLUB('♣'),
    DIAMOND('♦'),
    SPADE('♠');

    private char symbol;
    private char letter;

    public char getSymbol() {
        return symbol;
    }

    public char getLetter() {
        return letter;
    }

    private Suit(char symbol) {
        this.symbol = symbol;
        this.letter = switch (symbol) {
            case '♥' -> 'H';
            case '♣' -> 'C';
            case '♦' -> 'D';
            case '♠' -> 'S';
            default -> throw new RuntimeException("Illegal:" + symbol);
        };
    }
}
```

The parameters (only one of them in this example) are passed to the constructor to create the individual enum instances. As the enum instances are created by the Java runtime, and can't be instantiated from outside, the constructor is declared as private.

Enums have some special properties:

- All (implicitly) extend `java.lang.Enum`.
- May not be generic.
- May implement interfaces.
- Cannot be extended.

- May have abstract methods only if all enum values provide an implementation body.
- May not be directly instantiated by `new`.

Enums are used when we want to express that there are only a finite set of legal values of a given type. In "OOD Using Sealed Types", we will discuss how to express exclusive, exhaustive choice represented at the type level.

## Annotations

Annotations are a specialized kind of interface that, as the name suggests, annotate some part of a Java program.

For example, consider the `@Override` annotation. You may have seen it on some methods in some of the earlier examples and may have asked the following question: what does it do?

The short (and perhaps surprising) answer is that it does nothing at all.

The longer (and less flippant) answer is that, like all annotations, it has no direct effect but instead acts as additional information about the method that it annotates; in this case, it denotes that a method overrides a superclass method.

This acts as a useful hint to compilers and integrated development environments (IDEs)—if a developer has misspelled the name of a method intended to be an override of a superclass method, then the presence of the `@Override` annotation on the misspelled method (which does not, in that case, override anything) alerts the compiler to the fact that something is not right.

Annotations, as originally conceived, were not supposed to alter program semantics; instead, they were to provide optional metadata. In its strictest sense, this means that they should not affect program execution and instead should only provide information for compilers and other pre-execution phases.

In practice, modern Java applications make heavy use of annotations, and this now includes many use cases that essentially render the annotated classes useless without additional runtime support.

For example, classes bearing annotations defined (often outside of the JDK and in popular frameworks) such as `@Inject`, `@Autowired`, or `@Test` cannot realistically be used outside of a suitable execution container, such as an application server or test runner. As a result, it is difficult to argue that such annotations do not violate the "no semantic meaning" rule.

The platform defines a small number of basic annotations in `java.lang`. The original set were `@Override`, `@Deprecated`, and `@SuppressWarnings`, which were used to indicate that a method was overridden or deprecated or that it generated some compiler warnings that should be suppressed.

These were augmented by a small number of additions—such as `@SafeVarargs` in Java 7 (which provides extended warning suppression for varargs methods) and `@FunctionalInterface` in Java 8.

This last example deserves some special attention, as it indicates an interface can be used as a target for a lambda expression. As we will see, it is a useful marker annotation although not mandatory.

Annotations have some special properties, compared to regular interfaces:

- All (implicitly) extend `java.lang.annotation.Annotation`.
- May not be generic.
- May not extend any other interface.
- May only define zero-arg methods.
- May not define methods that throw exceptions.
- Have restrictions on the return types of methods.
- Can have a default return value for methods.

In practice, annotations do not typically have a great deal of functionality and instead are a fairly simple language concept.

In this section, we've met Java's enum and annotation types. Let's move on to consider the next important part of Java's type system: lambda expressions.

## Lambda Expressions

One of the most eagerly anticipated features of Java 8 was the introduction of lambda expressions (frequently referred to as just *lambdas*).

This major upgrade to the Java platform was driven by five goals, in roughly descending order of priority:

- More expressive programming
- Better libraries
- Concise code
- Improved programming safety
- Potentially increased data parallelism

Lambdas have three key aspects that help define the essential nature of the feature:

- They allow small bits of code to be written inline as literals in a program.
- They relax the strict grammar of Java code by using type inference.
- They facilitate a more functional style of programming Java.

As we saw in Chapter 2, the syntax for a lambda expression is to take a list of parameters (the types of which are typically inferred) and to attach that to a method body, like this:

```
(p, q) -> { /* method body */ }
```

This can provide a very compact way to represent what is effectively a single method. Just as we saw in Chapter 3, Java always works with classes, but the language provides capabilities to infer boilerplate where possible.

In this case, the core concept is that the lambda expression represents the creation of an object of a specific type. The type of the instance that is created is known as the *target type* of the lambda, and it has a method defined on it that matches the method definition provided in the lambda.

For the inference and boilerplate-reduction to work properly, only certain types are eligible to be the target of a lambda.

These *target types* are also called *functional interfaces*, and they must:

- Be interfaces
- Have exactly one mandatory (i.e., abstract) method (but may have other methods that are default)

Some developers also like to use the term *single abstract method* (or SAM) type to refer to the interface type that the lambda is converted into. This draws attention to the fact that to be usable by the lambda expression mechanism, an interface must have only a single nondefault method.

> **NOTE**
>
> A lambda expression has almost all the component parts of a method, with the obvious exception that a lambda doesn't have a name. In fact, many developers like to think of lambdas as "anonymous methods."

As a result, this means that the single line of code:

```
Runnable r  = () -> System.out.println("Hello");
```

does not result in the execution of the `println()` but instead creates an object, which is assigned to a variable `r`, of type `Runnable`. This object, `r`, will execute the `println()` statement, but only when `r.run()` is called, and not until then.

## Lambda Expression Conversion

When `javac` encounters a lambda expression, it interprets it as the body of a method with a specific signature—but which method?

To resolve this question, `javac` looks at the surrounding code. To be legal Java code, the lambda expression must satisfy the following properties:

- The lambda must appear where an instance of an interface type is expected.
- The expected interface type should have exactly one mandatory method.
- The expected interface method should have a signature that exactly matches that of the lambda expression.

If this is the case, then an instance is created of a type that implements the expected interface and uses the lambda body as the implementation for the mandatory method.

This slightly complex conversion approach comes from the desire to keep Java's type system as purely *nominative* (based on names) rather than the other possibilities that exist in the type systems of some other programming languages. Java's attitude to lambdas reflects that emphasis on nominal typing—the lambda expression is said to be *converted* to an instance of the correct interface type (which is, of course, named).

Before the arrival of lambdas, the only way to approximate the lambda coding style was to use *anonymous classes*, which we will discuss later in this chapter. However, since Java 8, lambdas have proved to be very popular with Java programmers and now have almost completely taken over the role of anonymous classes.

> **NOTE**
>
> Despite the surface similarity between lambda expressions and anonymous classes, lambdas are *not* simply syntactic sugar over anonymous classes. In fact, lambdas are implemented using method handles (which we will meet in Chapter 13) and a special JVM bytecode called `invokedynamic`.

From this discussion, we can see that lambda expressions, despite being added to Java rather than being built in from the start, have been specifically designed to fit into Java's existing type system.

Let's consider an example of lambda conversion—the `list()` method of the `java.io.File` class. This method lists the files in a directory. Before it returns the list, though, it passes the name of each file to a `FilenameFilter` object that the programmer must supply. This `FilenameFilter` object accepts or rejects each file and is a SAM type defined in the `java.io` package:

```
@FunctionalInterface
public interface FilenameFilter {
    boolean accept(File dir, String name);
}
```

The type `FilenameFilter` carries the `@FunctionalInterface` to indicate that it is a suitable type to be used as the target type for a lambda. However, this annotation is not required, and any type that meets the requirements (by being an interface and a SAM type) can be used as a target type.

This is because the JDK and the existing corpus of Java code already had a huge number of SAM types available before Java 8 was released. To require potential target types to carry the annotation would have prevented lambdas from being easily retrofitted to existing code.

> **TIP**
>
> In code that you write, you should always try to indicate when your types are usable as target types, which you can do by adding the `@FunctionalInterface` annotation to them. This aids readability and can provide a small amount of additional safety as well—if `@FunctionalInterface` is included on a type that doesn't have exactly one abstract method, the compiler will fail the build.

Here's how we can define a `FilenameFilter` class to list only those files whose names end with *.java*, using a lambda:

```
File dir = new File("/src");      // The directory to list

String[] filelist = dir.list((d, fName) -> fName.endsWith(".java"));
```

For each file in the list, the block of code in the lambda expression is evaluated. If the method returns `true` (which happens if the filename ends in *.java*), then the file is included in the output—which ends up in the array `filelist`.

This design principle (aka pattern), where a block of code is used to test whether an element of a container matches a condition, and to return only the elements that pass the condition, is called a *filter idiom*. It is one of the standard techniques of functional programming, which we will discuss in more depth presently.

## Method References

Recall that we can think of lambda expressions as objects representing methods that don't have names. Now, consider this lambda expression:

```
// In real code this would probably be
// shorter because of type inference
(MyObject myObj) -> myObj.toString()
```

The type `FilenameFilter` carries the `@FunctionalInterface` to indicate that it is a suitable type to be used as the target type for a lambda. This will be autoconverted to an implementation of a `@FunctionalInterface` type that has a single nondefault method that takes a single `MyObject` and returns a `String`—specifically, the string obtained by calling `toString()` on the instance of `MyObject`. However, this seems like excessive boilerplate, and so Java 8 provides a syntax for making this easier to read and write:

```
MyObject::toString
```

This shorthand, known as a *method reference*, uses an existing method as a lambda expression. The method reference syntax is completely equivalent to the previous form expressed as a lambda. It can be thought of as using an existing method but ignoring the name of the method, so it can be used as a lambda and then autoconverted in the usual way.

Java defines four types of method references, which are equivalent to four slightly different lambda expression forms (see Table 5-1).

*Table 5-1. Method references*

| Name | Method reference | Equivalent lambda |
|---|---|---|
| Unbound | `Trade::getPrice` | `trade -> trade.getPrice()` |
| Bound | `out::println` | `s -> out.println(s)` |
| Static | `IO::println` | `s -> IO.println(s)` |
| Constructor | `Trade::new` | `price -> new Trade(price)` |

The form we originally introduced can be seen to be an *unbound method reference*. When we use an unbound method reference, it is equivalent to a lambda that is expecting an instance of the type that contains the method reference—in Table 5-1 that is a `Trade` object.

It is called an unbound method reference because the receiver object needs to be supplied (as the first argument to the lambda) when the method reference is used. That is, we are going to call `getPrice()` on some `Trade` object, but the supplier of the method reference has not defined which one. That is left up to the user of the reference.

By contrast, a *bound method reference* always includes the receiver as part of the instantiation of the method reference. In Table 5-1, the receiver is `out` so, when the reference is used, the `println()` method will always be called on `out`, and the parameter of the lambda will be used as the method parameter to `println()`.[6]

We will discuss use cases for method references versus lambda expressions in more detail in the next chapter.

## Lambdas and Functional Programming

Java is fundamentally an object-oriented language. However, since the arrival of lambda expressions, it has become much easier to write code that is closer to the functional approach.

**NOTE**

There's no single definition of exactly what constitutes a *functional language*—but there is at least consensus that it should at a minimum have the ability to represent a function as a value that can be put into a variable.

Java has always (since version 1.1) been able to represent functions via inner classes (see "Lambdas versus nested classes"), but the syntax was complex and lacking in clarity. Lambda expressions greatly simplify that syntax, and so it is only natural that more developers will be seeking to use aspects of functional programming in their Java code.

The first taste of functional programming that Java developers are likely to encounter are three basic idioms that are remarkably useful:

`map()`

The map idiom is used with lists and list-like containers. The idea is that a function is passed in that is applied to each element in the collection, and a new collection is created that consists of the results of applying the function to each element in turn. This means that a map idiom converts a collection of one type to a collection of potentially a different type.

`filter()`

We have already met an example of the filter idiom, when we discussed how to replace an anonymous implementation of `FilenameFilter` with a lambda. The filter idiom is used for producing a new subset of a collection, based on some selection criteria. Note that in functional programming, it is normal to produce a new collection rather than modifying an existing one in place.

`reduce()`

The reduce idiom has several different guises. It is an aggregation operation, which can be called *fold*, *accumulate*, or *aggregate*, as well as *reduce*. The basic idea is to take an initial value and an aggregation (or reduction) function and apply the reduction function to each element in turn, building up a final result for the whole collection by making a series of intermediate results—similar to a "running total"—as the reduce operation traverses the collection.

Java has full support for these key functional idioms (and several others). The implementation is explained in some depth in Chapter 8, where we discuss Java's data structures and collections, and in particular the *stream* abstraction, which makes all of this possible.

Let's conclude this note with some words of caution, specifically that Java is perhaps best regarded as having support for "slightly functional programming." It is not an especially functional language, nor does it try to be. Some particular aspects of Java that weigh against any claims to being a functional language include:

- Java has no structural types, which means no "true" function types. Every lambda is automatically converted to the appropriate target type.

- Type erasure causes problems for functional programming—type safety can be lost for higher-order functions.
- Java is inherently mutable (as we'll discuss in Chapter 6)—mutability is often regarded as highly undesirable for functional languages.
- Java does not have "variable arity generics"—only functions with up to two arguments can be represented by the interfaces in `java.util.function`.
- The Java collections are imperative, not functional. Collections must be converted to streams to use a functional style.

Despite this, easy access to the basics of functional programing—especially idioms such as map, filter, and reduce—has proved to be a huge step forward for the Java community. Many of these techniques were possible using patterns like callbacks and handlers, but the syntax was always quite cumbersome, as you had to explicitly define a completely new type even when you needed to express only a single line of code in a callback.

By focusing on providing the most basic functional idioms first, Java developers have gradually been introduced to a slightly more functional style. Since then, the OpenJDK and Java language communities have continued to deliver additional, incremental improvements—such as sealed types and pattern matching.

The overall intent is to provide "enough functional programming capabilities to be useful," and while Java will probably never be comparable to languages like Haskell or Scala, it seems likely that a large majority of Java developers will never need or miss the more advanced functional capabilities provided by those languages.

## Lexical Scoping and Local Variables

A local variable is defined within a block of code that defines its *scope*, and, outside of that scope, a local variable cannot be accessed and ceases to exist. Only code within the curly braces that define the boundaries of a block can use local variables defined in that block. This type of scoping is known as *lexical scoping*, and it defines a section of source code within which a variable can be used.

It is common for programmers to think of such a scope as *temporal* instead—that is, to think of a local variable as existing from the time the JVM begins executing the block until the time that control exits the block. This is usually a reasonable way to think about local variables and their scope. However, lambda expressions (and anonymous and local classes, which we will meet later) have the ability to bend or break this intuition.

This can cause effects that some developers initially find surprising. Because lambdas can use local variables, they can contain copies of values from lexical scopes that no longer exist. This can be seen in the following code:

```
public interface IntHolder {
    public int getValue();
```

```
}

public class Weird {
    public static void main(String[] args) {
        IntHolder[] holders = new IntHolder[10];
        for (int i = 0; i < 10; i++) {
            final int fi = i;

            holders[i] = () -> {
                return fi;
            };
        }
  // The lambda is now out of scope, but we have 10 valid instances
  // of the class the lambda has been converted to in our array.
  // The local variable fi is not in our scope here, but is still
  // in scope for the getValue() method of each of those 10 objects.
  // So call getValue() for each object and print it out.
  // This prints the digits 0 to 9.
        for (int i = 0; i < 10; i++) {
            System.out.println(holders[i].getValue());
        }
    }
}
```

Each instance of a lambda has an automatically created private copy of each of the final local variables it uses, so, in effect, it has its own private copy of the scope that existed when it was created. This is sometimes referred to as a *captured* variable.[7]

Lambdas that capture variables like this are referred to as *closures*, and the variables are said to have been *closed over*.

> **WARNING**
>
> Other programming languages may have a slightly different definition of a closure. In fact, some theorists would dispute that Java's mechanism counts as a closure because, technically, it is the contents of the variable (a value) and not the variable itself that is captured.

In practice, the preceding closure example is more verbose than it needs to be in two separate ways:

- The lambda has an explicit scope `{}` and `return` statement.
- The variable `fi` is explicitly declared `final`.

The compiler `javac` helps with both of these.

Lambdas that return the value of only a single expression need not include a scope or `return`; instead, the body of the lambda is just the expression without the need for curly braces. In our example, we have explicitly included the braces and `return` statement to spell out that the lambda is defining its own scope.

In Java 7 and earlier, there were two hard requirements when closing over a variable:

- The captures must not be modified after they have been captured (e.g., after the lambda).
- The captured variables must be declared `final`.

However, since Java 8, `javac` can analyze the code and detect whether the programmer attempts to modify the captured variable after the scope of the lambda. If not, then the `final` qualifier on the captured variable can be omitted (such a variable is said to be *effectively final*). If the `final` qualifier is omitted, then it is a compile-time error to attempt to modify a captured variable after the lambda's scope.

The reason for this is that Java implements closures by copying the bit pattern of the contents of the variable into the scope created by the closure. Further changes to the contents of the closed-over variable would not be reflected in the copy contained in closure scope, so the design decision was made to make such changes illegal and a compile-time error.

These assists from `javac` mean that we can rewrite the inner loop of the preceding example to the very compact form:

```
for (int i = 0; i < 10; i++) {
    int fi = i;
    holders[i] = () -> fi;
}
```

Closures are very useful in some styles of programming, and different programming languages define and implement closures in different ways.

Modern Java applications use lambda expressions to provide closure, but, before lambdas were available, Java used local classes and anonymous classes to capture state and implement closures. Let's move on to discuss this, and other aspects of inner types.

## Nested Types

The classes, interfaces, and enum types we have seen so far in this book have all been defined as *top-level types*. This means that they are direct members of packages, defined independently of other types. However, type definitions can also be nested within other type definitions. These *nested types*, commonly known as *inner classes*, are a powerful feature of the Java language.

In general, nested types are used for two separate purposes, both related to encapsulation. First, a type may be nested because it needs especially unfettered access to the internals of another type. By being a nested type, it has access in the same way that member variables and methods do. This means that nested types have privileged access and can be thought of as "slightly bending the rules of encapsulation."

Another way of thinking about this use case of nested types is that they are types that are somehow tied together with another type. This means that they don't really have a completely independent existence as an entity and only coexist.

Alternatively, a type may only be required for a very specific reason and in a very small section of code. This means that it should be tightly localized, as it is really part of the implementation detail.

In older versions of Java, the only way to do this was with a nested type, such as an anonymous implementation of an interface. In practice, since Java 8, this second use case has substantially been taken over by lambda expressions. The use of anonymous types as closely localized types has dramatically declined as a result, although it still persists for some cases.

Types can be nested within another type in four ways:

*Static member types*

> A static member type is any type defined as a `static` member of another type. Nested interfaces, enums, records, and annotations are always static (even if you don't use the keyword).

*Nonstatic member classes*

> A *nonstatic member type* is simply a member type that is not declared `static`. Only classes can be nonstatic member types.

*Local classes*

> A local class is a class that is defined and visible only within a block of Java code. Interfaces, enums, and annotations may not be defined locally.

*Anonymous classes*

> An anonymous class is a kind of local class that has no meaningful name useful to humans; it is merely an arbitrary name assigned by the compiler, which programmers should not use directly. Interfaces, enums, and annotations cannot be defined anonymously.

The term *nested types*, while correct and precise, is not widely used by developers. Instead, most Java programmers use the much vaguer term *inner class*. Depending on the situation, this can refer to a nonstatic member class, local class, or anonymous class but not a static member type, with no real way to distinguish between them.

Fortunately, although the terminology for describing nested types is not always clear, the syntax for working with them is, and it is usually apparent from context which kind of nested type is being discussed.

Let's move on to describe each of the four kinds of nested types in greater detail. Each section describes the features of the nested type, the restrictions on its use, and any special Java syntax used with the type.

## Static Member Types

A *static member type* is much like a regular top-level type. For convenience, however, it is

nested within the namespace of another type. Static member types have the following basic properties:

- A static member type is like the other static members of a class: static fields and static methods.
- A static member type is not associated with any instance of the containing class (i.e., there is no `this` object).
- A static member type can access (only) the `static` members of the class that contains it.
- A static member type has access to all the `static` members (including any other static member types) of its containing type.
- Nested interfaces, enums, and annotations are implicitly static, whether or not the `static` keyword appears.
- Any type nested within an interface or annotation is also implicitly `static`.
- Static member types may be defined within top-level types or nested to any depth within other static member types.

Let's look at a quick example of the syntax for static member types. Example 5-1 shows a helper interface defined as a static member of a containing interface, in this case, the `Map` interface from the Java Collections (which we'll meet properly in Chapter 8).

*Example 5-1. Defining and using a static member interface*

```
public interface Map<K, V> {
    // ...

    Set<Map.Entry<K, V>> entrySet();

    // All nested interfaces are automatically static
    interface Entry<K, V> {
        K getKey();
        V getValue();
        V setValue(V value);

        // other members elided
    }

    // other members elided
}
```

When used by an external class, `Entry` will be referred to by its hierarchical name `Map.Entry`.

It provides a good example of a nested type that needs special access to implementation details of its enclosing type, because an `Entry` object represents a key-value pair, i.e., an association between a key and a (current) value, which is a concept that is closely tied to the usage of the `Map` data structure and not altogether independent.

### Features of static member types

A static member type has access to all static members of its containing type, including `private` members. The reverse is true as well: the methods of the containing type have access to all members of a static member type, including the `private` members. A static member type even has access to all the members of any other static member types, including the `private` members of those types. A static member type can use any other static member without qualifying its name with the name of the containing type.

Top-level types can be declared as either `public` or package-private (if they're declared without the `public` keyword). But declaring top-level types as `private` and `protected` wouldn't make a great deal of sense—`protected` would just mean the same as package-private, and a `private` top-level class would be unable to be accessed by any other type.

Static member types, on the other hand, are members and so can use any access control modifiers that other members of the containing type can. These modifiers have the same meanings for static member types as they do for other members of a type.

Under most circumstances, the `Outer.Inner` syntax for class names provides a helpful reminder that the inner class is interconnected with its containing type. However, the Java language does permit you to use the `import` directive to directly import a static member type:

```
import java.util.Map.Entry;
```

You can then reference the nested type without including the name of its enclosing type (e.g., just as `Entry`).

> **NOTE**
>
> You can also use the `import static` directive to import a static member type. See "Packages and the Java Namespace" in Chapter 2 for details on `import` and `import static`.

However, importing a nested type can obscure the fact that that type is closely associated with its containing type (which is usually important information), and as a result it is not commonly done.

## Nonstatic Member Classes

A *nonstatic member class* is a class that is declared as a member of a containing class or enumerated type without the `static` keyword:

- If a static member type is analogous to a class field or class method, a nonstatic member class is analogous to an instance field or instance method.
- An instance of a nonstatic member class is always associated with an instance of the enclosing type.

- Only classes can be nonstatic member types.
- The code of a nonstatic member class has access to all the fields and methods (both `static` and non-`static`) of its enclosing type.
- Several Java syntax features exist specifically to work with the enclosing instance of a nonstatic member class.

Example 5-2 shows how a member class can be defined and used. This example shows a `LinkedStack` example: it defines a nested interface that describes the nodes of the linked list underlying the stack, and a nested class to allow enumeration of the elements on the stack. The member class defines an implementation of the `java.util.Iterator` interface.

*Example 5-2. An iterator implemented as a member class*

```
import java.util.Iterator;

public class LinkedStack {

    // Our static member interface
    public interface Linkable {
        public Linkable getNext();
        public void setNext(Linkable node);
    }

    // The head of the list
    private Linkable head;

    // Method bodies omitted here
    public void push(Linkable node) { ... }
    public Linkable pop() { ... }

    // This method returns an Iterator object for this LinkedStack
    public Iterator<Linkable> iterator() { return new LinkedIterator(); }

    // Here is the implementation of the Iterator interface,
    // defined as a nonstatic member class.
    protected class LinkedIterator implements Iterator<Linkable> {
        Linkable current;

        // The constructor uses a private field of the containing class
        public LinkedIterator() { current = head; }

        // The following three methods are defined
        // by the Iterator interface
        public boolean hasNext() {  return current != null; }

        public Linkable next() {
            if (current == null)
              throw new java.util.NoSuchElementException();
            Linkable value = current;
            current = current.getNext();
            return value;
        }

        public void remove() { throw new UnsupportedOperationException(); }
    }
```

```
}
```

Notice how the `LinkedIterator` class is nested within the `LinkedStack` class. `LinkedIterator` is a helper class used only within `LinkedStack`, so having it defined close to where it is used by the containing class makes for a clean design.

### Features of member classes

Like instance fields and instance methods, every instance of a nonstatic member class is associated with an instance of the class in which it is defined. This means that the code of a member class has access to all the instance fields and instance methods (as well as the `static` members) of the containing instance, including any that are declared `private`.

This crucial feature was already illustrated in Example 5-2. Here is the `LinkedStack.LinkedIterator()` constructor again:

```
public LinkedIterator() { current = head; }
```

This single line of code sets the `current` field of the inner class to the value of the `head` field of the containing class. The code works as shown, even though `head` is declared as a `private` field in the containing class.

A nonstatic member class, like any member of a class, can be assigned one of the standard access control modifiers. In Example 5-2, the `LinkedIterator` class is declared `protected`, so it is inaccessible to code (in a different package) that uses the `LinkedStack` class but is accessible to any class that subclasses `LinkedStack`.

A nonstatic member class cannot have the same name as any containing class. This is an important rule, one that is *not* shared by fields and methods.

### Syntax for member classes

The most important feature of a member class is that it can access the instance fields and methods in its containing object.

If we want to use explicit references, and make use of `this`, then we have to use a special syntax for explicitly referring to the containing instance of the `this` object. For example, if we want to be explicit in our constructor, we can use the following syntax:

```
public LinkedIterator() { this.current = LinkedStack.this.head; }
```

The general syntax is *`classname.this`*, where *`classname`* is the name of a containing class. Note that member classes can themselves contain member classes, nested to any depth.

However, no member class can have the same name as any containing class, so the use of the enclosing class name prepended to `this` is a perfectly general way to refer to any containing instance. Another way of saying this is that the syntax construction `EnclosingClass.this` is an unambiguous way of referring to the containing instance as an *uplevel reference*.

## Local Classes

A *local class* is declared locally within a block of Java code rather than as a member of a class. Only classes may be defined locally: interfaces, enumerated types, and annotation types must be top-level or static member types. Typically, a local class is defined within a method, but it can also be defined within a static initializer or instance initializer of a class.

Just as all blocks of Java code appear within class definitions, all local classes are nested within containing blocks. For this reason, although local classes share many of the features of member classes, it is usually more appropriate to think of them as an entirely separate kind of nested type.

The defining characteristic of a local class is that it is local to a block of code. Like a local variable, a local class is valid only within the scope defined by its enclosing block. Example 5-3 illustrates how we can modify the `iterator()` method of the `LinkedStack` class so it defines `LinkedIterator` as a local class instead of a member class.

By doing this, we move the definition of the class even closer to where it is used and hopefully improve the clarity of the code even further. For brevity, Example 5-3 shows only the `iterator()` method, not the entire `LinkedStack` class that contains it.

*Example 5-3. Defining and using a local class*

```
// This method returns an Iterator object for this LinkedStack
public Iterator<Linkable> iterator() {
    // Here's the definition of LinkedIterator as a local class
    class LinkedIterator implements Iterator<Linkable> {
        Linkable current;

        // The constructor uses a private field of the containing class
        public LinkedIterator() { current = head; }

        // The following three methods are defined
        // by the Iterator interface
        public boolean hasNext() {  return current != null; }

        public Linkable next() {
            if (current == null)
              throw new java.util.NoSuchElementException();
            Linkable value = current;
            current = current.getNext();
            return value;
        }

        public void remove() { throw new UnsupportedOperationException(); }
    }

    // Create and return an instance of the class we just defined
    return new LinkedIterator();
}
```

### Features of local classes

Local classes have the following interesting features:

- Like member classes, local classes are associated with a containing instance and can

access any members, including `private` members, of the containing class.

- In addition to accessing fields defined by the containing class, local classes can access any local variables, method parameters, or exception parameters that are in the scope of the local method definition and are declared `final`.

Local classes are subject to the following restrictions:

- The name of a local class is defined only within the block that defines it; it can never be used outside that block. (Note, however, that instances of a local class created within the scope of the class can continue to exist outside of that scope. This situation is described in more detail later in this section.)
- Local classes cannot be declared `public`, `protected`, `private`, or `static`.
- A local class, like a member class, cannot have the same name as any of its enclosing classes.
- As noted earlier, a local class can close over the local variables, method parameters, and even exception parameters that are in its scope but only if those variables or parameters are effectively `final`.

### Scope of a local class

In discussing nonstatic member classes, we saw that a member class can access any members inherited from superclasses and any members defined by their containing classes.

The same is true for local classes, but local classes can also behave like lambdas and access effectively `final` local variables and parameters. Example 5-4 illustrates the different kinds of fields and variables that may be accessible to a local class (or a lambda, for that matter).

*Example 5-4. Fields and variables available to a local class*

```
class A { protected char a = 'a'; }
class B { protected char b = 'b'; }

public class C extends A {
  private char c = 'c';         // Private fields visible to local class
  public static char d = 'd';
  public void createLocalObject(final char e)
  {
    final char f = 'f';
    int i = 0;                  // i not final; not usable by local class
    class Local extends B {     // Local classes do not extend their
                                // containing class; Local doesn't extend C
      char g = 'g';
      public void printVars()
      {
        // All of these fields and variables are accessible to this class
        System.out.println(g);  // (this.g) g is a field of this class
        System.out.println(f);  // f is a final local variable
        System.out.println(e);  // e is a final local parameter
        System.out.println(d);  // (C.this.d) d field of containing class
        System.out.println(c);  // (C.this.c) c field of containing class
```

```
            System.out.println(b);  // b is inherited by this class
            System.out.println(a);  // a is inherited by the containing class
        }
    }
    Local l = new Local();      // Create an instance of the local class
    l.printVars();              // and call its printVars() method.
  }
}
```

Local classes have quite a complex scoping structure, therefore. To see why, notice that instances of a local class can have a lifetime that extends past the time that the JVM exits the block where the local class is defined.

In other words, if you create an instance of a local class, that instance does not automatically go away when the JVM finishes executing the block that defines the class. So, even though the definition of the class was local, instances of that class can escape the place they were defined.

Local classes, therefore, behave like lambdas in many regards, although the use case of local classes is more general than that of lambdas. However, in practice, the extra generality is rarely required, and lambdas are preferred wherever possible.

## Anonymous Classes

An *anonymous class* is a local class without a name. It is defined and instantiated in a single expression using the `new` operator. While a local class definition is a statement in a block of Java code, an anonymous class definition is an expression, which means that it can be included as part of a larger expression, such as a method call.

> **NOTE**
>
> For the sake of completeness, we cover anonymous classes here, but for most use cases, lambda expressions (see "Lambda Expressions") are now the preferred option. See Chapter 6 for details as to when it's appropriate to choose an anonymous class versus a lambda expression.

Consider Example 5-5, which shows the `LinkedIterator` class implemented as an anonymous class within the `iterator()` method of the `LinkedStack` class. Compare it with Example 5-4, which shows the same class implemented as a local class.

*Example 5-5. An enumeration implemented with an anonymous class*

```
public Iterator<Linkable> iterator() {
    // The anonymous class is defined as part of the return statement
    return new Iterator<Linkable>() {
        Linkable current;
        // Replace constructor with an instance initializer
        { current = head; }

        // The following three methods are defined
        // by the Iterator interface
        public boolean hasNext() {  return current != null; }
        public Linkable next() {
```

```
            if (current == null)
              throw new java.util.NoSuchElementException();
            Linkable value = current;
            current = current.getNext();
            return value;
        }
        public void remove() { throw new UnsupportedOperationException(); }
    };  // Note the required semicolon. It terminates the return statement.
}
```

As you can see, the syntax for defining an anonymous class and creating an instance of that class uses the `new` keyword, followed by the name of a type and a class body definition in curly braces. If the name following the `new` keyword is the name of a class, the anonymous class is a subclass of the named class. If the name following `new` specifies an interface, as in the two previous examples, the anonymous class implements that interface and extends `Object`.

> **NOTE**
>
> The syntax for anonymous classes deliberately does not include any way to specify an `extends` clause, an `implements` clause, or a name for the class.

Because an anonymous class has no name, it is not possible to define a constructor for it within the class body. This is one of the basic restrictions on anonymous classes. Any arguments you specify between the parentheses following the superclass name in an anonymous class definition are implicitly passed to the superclass constructor.

Anonymous classes are commonly used to subclass simple classes that do not take any constructor arguments, so the parentheses in the anonymous class definition syntax are often empty. An anonymous class is just a kind of local class, so anonymous classes and local classes share the same restrictions.

An anonymous class cannot define any `static` fields, methods, or classes, except for `static final` constants. Interfaces, enumerated types, and annotation types cannot be defined anonymously. Also, like local classes, anonymous classes cannot be `public`, `private`, `protected`, or `static`.

The syntax for defining an anonymous class combines definition with instantiation, similar to a lambda expression. Using an anonymous class instead of a local class is not appropriate if you need to create more than a single instance of the class each time the containing block is executed.

## Describing the Java Type System

At this point, we have met all major aspects of Java's powerful type system, and so it is possible for us to describe and characterize it.

The most important and obvious characteristics of Java's type system are that it is:

- Static
- Not single-rooted
- Capable of a moderate amount of type inference
- Nominal

We've already discussed the first two of these properties, so let's add some brief commentary before moving on to consider the third and fourth points in more depth.

Static typing, which is the most easily and widely recognized of the four aspects, means that in Java every piece of data storage (such as variables, fields, etc.) has a type, and that type is declared when the storage is first introduced. It is a compile-time error to try to put an incompatible value into storage that does not support it.

That Java's type system is not single-rooted is also immediately clear: Java has both primitive types and reference types. Every object in Java belongs to a class, and every class, except `Object`, has a single parent. This means that the set of classes in any Java program forms a tree structure with `Object` at the root.

However, not all data in Java is objects, and there is no inheritance relationship between any of the primitive types and `Object`. As a result, the overall graph of Java classes consists of a large tree of reference types and eight disjoint, isolated points that correspond to the primitives. This leads to the need to use wrapper types, such as `Integer`, to represent primitive values as objects where necessary (such as in the Java Collections).[8]

## Type Inference

One of the common criticisms of Java (which we discussed back in Chapter 1) is that it has historically tended toward a verbose coding style, with code like `Object o = new Object();` being a fairly common occurrence. This piece of code obviously repeats type information, although it does provide extra visual clues regarding the type of `o`, which may make it easier for some beginning programmers.

As a response to this criticism, a number of changes have been made to the Java language to reduce this verbosity, by using type inference to allow the programmer to omit type information that the source code compiler can safely deduce.

Possibly the first step on Java's type inference journey came with the introduction of generic methods in Java 5, where in almost all cases the type parameter of the generic method can be omitted by the programmers. In fact, this is so common and automatic to most Java programmers that it doesn't necessarily even register as a form of type inference. We also met "diamond syntax" earlier in this chapter, which allows us to avoid repeating type parameters, for example, when constructing new objects of specific generic types.

Similarly, lambda expressions make use of type inference to strip away all unnecessary typing. This is especially important because lambdas have a design intent to them, and so being able to

see the actual behavior of the lambda clearly is essential to the way that Java developers are intended to use lambdas.

These efforts are best characterized as gradual, careful, and conservative, and they have taken place over a large number of versions.

The next major step came with the appearance of the pseudo-keyword `var`, which is perhaps one of the most visible signs of source code targeting Java 9 or later. This represents a new language feature properly known as *Local Variable Type Inference* (LVTI), and, in the simplest case, it allows us to write code such as:

```
var cat = "Cat";
```

This simple case infers the type of the variable `cat` to be `String` and displays the general principle that the intent of `var` is to allow the types of variables to be inferred.

The implementation achieves this by making `var` a reserved type name rather than a keyword. This means that code can still use `var` as a variable, method, or package name without being affected by the new syntax. However, code that has previously used `var` as the name of a type will have to be recompiled.

> **WARNING**
>
> Using contextual keywords (like `var`) as variable names, while allowed for compatibility, is strongly discouraged. `var var = "var";` is indeed a valid Java statement, but it ought to be viewed with deep suspicion and flagged during code review.

As well as the simple cases, when type parameters and generics are involved, then more complex possibilities arise. For example, we can use type inference like this (LVTI):

```
var ls = new ArrayList<String>();
```

or like this (diamond syntax):

```
List<String> ls = new ArrayList<>();
```

These two possibilities encode slightly different information—the type equations are over-determined, but a minimum amount of information is still required to appear somewhere.

The payload of a parameterized type must be declared somewhere—either on the left (variable) or the right (value). For LVTI we have to provide the full payload type on the value, and we can no longer infer it using diamond syntax.

In other words, in the first case we infer the type of `ls` to be the concrete class type `ArrayList<String>`, and in the second we declare it to be the interface type `List<String>`.

In general, LVTI is designed to reduce verbosity and to make programmers coming to Java from other languages, especially JavaScript and .NET, feel more comfortable. However, it does carry the risk that overuse will potentially obscure the intent of the code being written, so it should be used appropriately. For this reason, we recommend consulting the LVTI style guide for advice on best practices for using type inference in your own code.

As it exists now, type inference has been added to the language slowly and cautiously. It is possible to evolve this feature further still and add more powerful capabilities, but this comes at the cost of extra complexity, and the current design opinion of the language architects is that this trade-off is not worth it at the current moment.

Let's move on to look at the final aspect of Java's type system that also requires slightly more detailed discussion.

## Nominal Typing

In Java, each type has a name. In the normal course of Java programming, this will be a simple string of letters (and sometimes numbers) that has some semantic meaning that reflects the purpose of the type. This approach is known as *nominal typing*.

Not all languages have purely nominal typing; for example, some languages can express the idea that "this type has a method with a certain signature" without needing to explicitly refer to the name of the type, sometimes known as a *structural type*.

For example, in Python, you can call `len()` on any object that defines a `__len__()` method. Of course, Python is a dynamically typed language and so will throw a runtime exception if the call to `len()` cannot be made. However, it is also possible to express a similar idea in statically typed languages, such as Scala.

Java, on the other hand, has no way to express this idea without using an interface, which, of course, has a name. Java also maintains type compatibility based strictly on inheritance and implementation. Let's look at an example:

```
@FunctionalInterface
public interface MyRunnable {
    void run();
}
```

The interface `MyRunnable` has a single method that exactly matches that of `Runnable`. However, the two interfaces have no inheritance or other relationship to each other, and so code like this:

```
MyRunnable myR = () -> System.out.println("Hello");
Runnable r = (Runnable)myR;
r.run();
```

will compile cleanly but will fail with a `ClassCastException` at runtime. The fact that a `run()` method with an identical signature exists on both interfaces is not considered, and in fact

the program never even makes it to the point where `run()` would be called: it fails on the previous line where the cast is attempted.

Another important point is that the entire construction of Java's lambda expressions, and especially the use of target typing to a functional interface, is to ensure that lambdas fit into the nominal typing approach. For example, consider an interface such as:

```
@FunctionalInterface
public interface MyIntProvider {
    int run() throws InterruptedException;
}
```

then a lambda expression that yields a constant, e.g., `() -> 42`, can be used in a number of different ways:

```
MyIntProvider prov       = () -> 42;
Supplier<Integer> sup    = () -> 42;
Callable<Integer> callMe = () -> 42;
```

From this, we can see that the expression `() -> 42` is, by itself, incomplete. Java lambdas rely upon type inference, and so we need to see the expression in context with its target type for it to be meaningful. When combined with a target type, the lambda's class type is "an unknown-at-compile-time implementation of the target interface," and the programmer must use the interface type as the type of the lambda.

Beyond lambdas, there are some other corner cases of nominal typing in Java, such as anonymous classes—but even these types still have names. However, the type names of anonymous types are automatically generated by the compiler and are specially chosen to be usable by the JVM but not accepted by the Java source code compiler. For advanced use cases, the JVM also supports *hidden classes*—but these are an advanced feature and outside the scope of our current discussion.

There is one other corner case that we should consider to complete the picture.

## An Aside: Nondenotable Types

As well as the simple cases we've already met, `var` actually permits programming constructs that were not possible before. To see the differences, let's consider the fact that `javac` has always permitted a very limited form of type inference:

```
public class Test {
    public static void main(String[] args) {
        (new Object() {
            public void bar() {
                System.out.println("bar!");
            }
        }).bar();
    }
}
```

The code will compile and run, printing out `bar!`. This slightly counterintuitive result occurs because `javac` preserves enough type information about the anonymous class (i.e., that it has a `bar()` method) for just long enough that the compiler can conclude that the call to `bar()` is valid.[9]

The problem with this form of type inference is that it has no real practical applications: the type of "Object-with-a-bar-method" exists within the compiler, but the type is impossible to express as the type of a variable—it is not a *denotable type*. This means that before `var`, the existence of this type is restricted to a single expression and cannot be used in a larger scope.

With the arrival of LVTI, however, the type of variables does not always need to be made explicit. Instead, we can use `var` to allow us to preserve the static type information by avoiding denoting the type.

This means we can now modify our example and write:

```
var o = new Object() {
    public void bar() {
        System.out.println("bar!");
    }
};

o.bar();
```

This has allowed us to preserve the true type of `o` beyond a single expression. The type of `o` cannot be denoted, and so it cannot appear as the type of either a method parameter or return type. This means the type is still limited to only a single method, but it is still useful to express some constructions that would be awkward or impossible otherwise.

This use of `var` as a "magic type" allows the programmer to preserve type information for each distinct usage of `var`, in a way that is somewhat reminiscent of bounded wildcards from Java's generics, and more advanced usages of `var` with nondenotable types are possible.

## Summary

By examining Java's type system, we have been able to build up a clear picture of the worldview that the Java platform has about data types. Java's type system can be characterized as:

*Static*

> All Java variables have types that are known at compile time.

*Nominal*

> The name of a Java type is of paramount importance. Java does not permit structural types and has only limited support for nondenotable types.

*Object/imperative*

Java code is object-oriented, and all code must live inside methods, which must live inside classes. However, Java's primitive types prevent full adoption of the "everything is an object" worldview.

*Slightly functional*

Java provides support for some of the more common functional idioms but more as a convenience to programmers than anything else.

*Type-inferred*

Java is optimized for readability (even by novice programmers) and prefers to be explicit but uses type inference to reduce boilerplate where it does not impact the legibility of the code.

*Strongly backward compatible*

Java is primarily a business-focused language, and backward compatibility and protection of existing codebases are very high priorities.

*Type erased*

Java permits parameterized types, but this information is not available at runtime.

Java's type system has evolved (albeit slowly and cautiously) over the years—and is now on par with the type systems of other mainstream programming languages. Lambda expressions, along with default methods, represent the greatest transformation since the advent of Java 5 and the introduction of generics, annotations, and related innovations.

Default methods represent a major shift in Java's approach to object-oriented programming—perhaps the biggest since the language's inception. From Java 8 onward, interfaces can contain implementation code. This fundamentally changes Java's nature. Previously a single-inherited language, Java is now multiply inherited (but only for behavior—there is still no multiple inheritance of state).

Despite all of these innovations, Java's type system is not (and is not intended to be) equipped with the power of the type systems of languages such as Scala or Haskell. Instead, Java's type system is strongly biased in favor of simplicity, readability, and a simple learning curve for newcomers.

Java has also benefited enormously from the approaches to types developed in other languages over the last 10 years. Scala's example of a statically typed language that nevertheless achieves much of the feel of a dynamically typed language through the use of type inference has been a good source of ideas for features to add to Java, even though the languages have quite different design philosophies.

One remaining question is whether the modest support for functional idioms that lambda expressions provide in Java is sufficient for the majority of Java programmers.

It remains to be seen how many Java programmers require the added power—and attendant

complexity—that comes from an advanced (and much less nominal) type system such as Scala's, or whether the "slightly functional programming" introduced in Java 8 (e.g., *map*, *filter*, *reduce*, and their peers) and progressively extended since then will suffice for most developers' needs.

The details we have learned about the Java type system are intellectually interesting, but they still need to be formed into sound system designs, and this is the subject of our next chapter, which will conclude Part I.

---

1 There is currently ongoing research on ways to revisit this possibility in Project Valhalla, which we discuss in Appendix A.

2 Some small traces of generics remain, which can be seen at runtime via reflection.

3 Raoul-Gabriel Urma and Janina Voigt, "Using the OpenJDK to Investigate Covariance in Java," *Java Magazine* (May/June 2012): 44–47.

4 This is another version of the storage problems we saw with covariance.

5 In this specific case, the compiler could infer the concrete type, but it's easy to construct examples where the concrete type cannot be known until runtime (e.g., a method that randomly returns a `LinkedList` or an `ArrayList`). The language designers therefore chose not to complicate the compiler by special-casing the trivial case.

6 The variable `out` should be understood to be `System.out` or some other `PrintStream` object—but it is an instance, and there is nothing static happening here.

7 Captured lexical variables are not directly related to the "capture-of" concept that appears in advanced Java generics, which were mentioned earlier.

8 This is recognized as a weakness in the current type system, and various possible approaches to improving it are being discussed in the context of Project Valhalla, which we will discuss in Appendix A.

9 In fact, this edge case has been known in the Java community since at least 2009.

# Chapter 6. Introduction to Object-Oriented Design in Java

In this chapter, we will consider various techniques relevant to object-oriented design (OOD) in Java.

We'll look at how to work with Java's objects, covering the key methods of `Object`, aspects of OOD, and the use of exception handling schemes. Throughout the chapter, we will introduce some *design patterns*—essentially best practices for solving some very common situations that arise in software design. Toward the end of the chapter, we'll also consider *safe* programs—those that are designed so as not to become inconsistent over time.

> **NOTE**
>
> This chapter is intended to showcase some examples of a complex topic and a few underlying principles. This is by no means a complete treatment, and we encourage you to consult additional resources, such as *Effective Java* by Joshua Bloch.

Before we get underway, let's take a moment to discuss an OO term that we are deliberately *not* using. Many resources and books make extensive use of the term *polymorphism* (which is Greek for "many forms") when discussing OO programming. However, although the general idea seems initially useful, on closer inspection it loses its appeal.

The word is intended to cover a wide range of language mechanisms (such as dynamic dispatch or generics), but using the word does nothing to explain the actual language feature being discussed. This leads to an overcomplication of basic mechanisms like method overloading and overriding while also giving the false sense that there is some important underlying principle that connects them.

There is also a big disconnect between the way that OO and functional languages use the word *polymorphism*. As Java is increasingly a language that uses both OO and FP techniques, our view is that it's better to focus on the specifics of what programmers actually do instead of introducing rather academic jargon of dubious utility.

Let's get started with our treatment of modern OO design in Java by considering the subject of method calling and passing conventions, and the nature of Java's values.

## Java Values

Java's values, and their relationship to the type system, are quite straightforward. Java has two types of values: primitives and object references. There are precisely eight different primitive

types in Java, and new primitive types cannot be defined by the programmer.

A major difference between primitive values and references is that primitive values cannot be altered; the value `2` is always the same value. By contrast, the contents of object references can usually be changed—often referred to as *mutation* of object contents.

Also note that variables can contain values of only the appropriate type. In particular, variables of reference type always contain a reference to the memory location holding the object—they do not contain the object contents directly. This means that in Java there is no equivalent of a dereference operator or a `struct`.

Java tries to simplify a concept that often confused C++ programmers: the difference between "contents of an object" and "reference to an object." Unfortunately, it's not possible to completely hide the difference, and so it is necessary for the programmer to understand how reference values work in the platform.

## Is Java "Pass-by-Reference"?

Java handles objects by reference, which means that Java objects always remain in the heap and Java bytecode manipulates references to them—the objects cannot be loaded onto the stack directly.

However, we must not confuse this with the phrase *pass-by-reference*, a term used to describe the method-calling conventions of certain programming languages. In a pass-by-reference language, values—even primitive values—are not passed directly to methods. Instead, methods always receive references to passed values, and if the method modifies the values held in its parameters, those modifications are visible when the method returns, even for primitive types.

Java does *not* do this; it is a "pass-by-value" language. When a method takes a reference type as a parameter, then the value that is passed is a (bitwise) copy of the reference as a value. However, this is not the same as pass-by-reference—if Java were a pass-by-reference language, then a reference type passed to a method would be passed as a reference to the reference.

> **TIP**
>
> You could describe Java as a "pass-by-bit-pattern" language—we always pass the bits, whether they represent a primitive such as an `int` or a reference to an object (which will be passed as the address of the object in the Java heap).

The fact that Java is pass-by-value can be demonstrated very simply, e.g., by running the following code:

```
public void manipulate(Circle circle) {
    circle = new Circle(3);
}

Circle c = new Circle(2);
```

```
System.out.println("Radius: " + c.getRadius());
manipulate(c);
System.out.println("Radius: " + c.getRadius());
```

This outputs `Radius: 2` twice and thus shows that even after the call to `manipulate()`, the value contained in variable `c` is unaltered—it is still holding a reference to a `Circle` object of radius 2. If Java were a pass-by-reference language, it would instead be holding a reference to a `Circle` with a radius of 3.

If we're scrupulously careful about the distinction and about referring to object references as one of Java's possible kinds of values, then we won't be confused by Java's calling conventions. Be careful—some older texts are ambiguous on this point. We will meet this concept of Java's values again when we discuss memory and garbage collection in Chapter 9.

## Constants

Recall that, in Java, a constant is a `static final` field. This combination of modifiers gives a single value (per class) with the given name, and it is initialized as soon as the class is loaded and then cannot be changed.

> **NOTE**
>
> Unlike languages like C++ or Rust, the `final` qualifier does *not* form part of the type—as it would for `const` in C++. Instead, for fields (and local variables), `final` is an instruction to the compiler that "this location can be written to only once."

By convention, Java's constants are named in all capitals, using *snake case*, for example, `NETWORK_SERVER_SOCKET`,[1] as opposed to the *camel case* (or *camelCase*) convention, such as `networkServerSocket` for a regular field.

There are essentially three different subcases of constants that can appear:

`public` *constants*

: These form part of the public API of the class.

`private` *constants*

: These are used when the constant is an internal implementation detail for this class only.

*Package-level constants*

: These have no additional access keyword and are used when the constant is an internal implementation detail that needs to be seen by different classes within the same package.

The final case might arise, for example, when client and server classes implement a network protocol whose details (such as the port number to connect to and listen on) are captured in a set

of symbolic constants.

As discussed earlier, an alternative approach is for constants to appear in an interface definition. Any class that implements an interface inherits the constants it defines and can use them as if they were defined directly in the class itself. This has the advantage that there is no need to prefix the constants with the name of the interface.

However, this can become overcomplicated, and so a preferred approach is to define constants (as either public or package-level) in a class (called something like `Constants` or `Utils`) and then import the constants from their defining class with the `import static` declaration. See "Packages and the Java Namespace" for details.

## Important Common Methods

As we've noted, all classes extend, directly or indirectly, `java.lang.Object`. This class defines a number of useful methods, some of which were designed to be overridden by classes you write. Example 6-1 shows a class that overrides these methods. The sections that follow this example document the default implementation of each method and explain why you might want to override it.

Note that this example is for demonstration purposes only; in reality, we would probably represent classes like `Circle` as records, and a lot of these methods would be implemented automatically by the compiler (we will see examples of this later in this chapter).

*Example 6-1. A class that overrides important Object methods*

```
// This class represents a circle with immutable position and radius.
public class Circle implements Comparable<Circle> {
    // These fields hold the coordinates of the center and the radius.
    // They are private for data encapsulation and final for immutability
    private final int x, y, r;

    // The basic constructor: initialize the fields to specified values
    public Circle(int x, int y, int r) {
        if (r < 0) throw new IllegalArgumentException("negative radius");
        this.x = x; this.y = y; this.r = r;
    }

    // This is a "copy constructor" - a useful alternative to clone()
    // as we will discuss in more detail presently
    public Circle(Circle original) {
        x = original.x;   // Just copy the fields from the original
        y = original.y;
        r = original.r;
    }

    // Public accessor methods for the private fields.
    // These are part of data encapsulation.
    public int getX() { return x; }
    public int getY() { return y; }
    public int getR() { return r; }

    // Return a string representation
```

```
    @Override public String toString() {
        return String.format("center=(%d,%d); radius=%d", x, y, r);
    }

    // Test for equality with another object
    @Override public boolean equals(Object o) {
        // Identical references?
        if (o == this) return true;
        // Correct type and non-null?
        if (!(o instanceof Circle)) return false;
        Circle that = (Circle) o;                  // Cast to our type
        if (this.x == that.x && this.y == that.y && this.r == that.r)
            return true;                           // If all fields match
        else
            return false;                          // If fields differ
    }

    // A hash code allows an object to be used in a hash table.
    // Equal objects must have equal hash codes. Unequal objects are
    // allowed to have equal hash codes, but we try to avoid that.
    // We must override this method because we also override equals().
    @Override public int hashCode() {
        int result = 17;          // This hash code algorithm is from
        result = 37 * result + x; // Effective Java, by Joshua Bloch
        result = 37 * result + y;
        result = 37 * result + r;
        return result;
    }

    // This method is defined by the Comparable interface. Compare
    // this Circle to that Circle.  Return a value < 0 if this < that
    // Return 0 if this == that. Return a value > 0 if this > that.
    // Circles are ordered top to bottom, left to right, then by radius
    public int compareTo(Circle that) {
        // Higher circles have bigger y
        long result = (long)that.y - this.y;
        // If same, compare l-to-r
        if (result == 0) result = (long)this.x - that.x;
        // If same, compare radius
        if (result == 0) result = (long)this.r - that.r;

        // We have to use a long value for subtraction because the
        // differences between a large positive and large negative
        // value could overflow an int. But we can't return the long,
        // so return its sign as an int.
        return Long.signum(result);
    }
}
```

Example 6-1 uses a lot of the extended features of the type system that we introduced in Chapter 5. First, this example implements a parameterized, or generic, version of the `Comparable` interface. Second, it uses the `@Override` annotation to emphasize (and have the compiler verify) that certain methods override the implementations in `Object`.

## toString()

The purpose of the `toString()` method is to return a textual representation of an object. The

method is invoked automatically on objects during string concatenation and by methods such as `System.out.println()`. Giving objects a textual representation can be quite helpful for debugging or logging output, and a well-crafted `toString()` method can even help with tasks such as report generation.

> **WARNING**
>
> Consider carefully whether sensitive information in an object (i.e., passwords or API keys) should be included in `toString()` output or masked out. It's too easy for information like that to end up in logging and other output if it's included in this method by default.

The version of `toString()` inherited from `Object` returns a string that includes the name of the class of the object as well as a hexadecimal representation of the `hashCode()` value of the object (discussed later in this chapter). This default implementation provides basic type and identity information for an object but is not very useful.

The `toString()` method in Example 6-1 returns a human-readable string that includes the value of each of the fields of the `Circle` class.

## equals()

The `==` operator tests two references to see if they refer to the same object. If you want to test whether two distinct objects are equal to each other, you must use the `equals()` method instead. Any class can define its own notion of equality by overriding `equals()`. The `Object.equals()` method simply uses the `==` operator: this default implementation considers two objects equal only if they are actually in the exact same heap memory location.

The `equals()` method in Example 6-1 considers two distinct `Circle` objects to be equal if their fields are all equal. Note that it first does a quick identity test with `==` as an optimization and then checks the type of the other object with `instanceof`: a `Circle` can be equal only to another `Circle`, and it is not acceptable for an `equals()` method to throw a `ClassCastException`. Note that the `instanceof` test also rules out `null` arguments: `instanceof` always evaluates to `false` if its lefthand operand is `null`.

## hashCode()

This method returns an `int` for use by hash tables and related data structures. It is critical that two objects have the same hash code if they are equal according to the `equals()` method.

It is important but not required that unequal objects have unequal hash codes.[2] This second criterion can lead to `hashCode()` methods that involve mildly tricky arithmetic or bit manipulation.

The `Object.hashCode()` method works with the `Object.equals()` method and returns a hash code based on object identity rather than object equality. (If you ever need an identity-

based hash code, you can access the functionality of `Object.hashCode()` through the static method `System.identityHashCode()`.)

> **WARNING**
>
> When you override `equals()`, you must always override `hashCode()` to guarantee that equal objects have equal hash codes. Failing to do this can cause subtle bugs in your programs.

Because the `equals()` method in Example 6-1 bases object equality on the values of the three fields, the `hashCode()` method computes its hash code based on these three fields as well. It is clear from the code that if two `Circle` objects have the same field values, they will have the same hash code.

Note that the `hashCode()` method in Example 6-1 does not simply add the three fields and return their sum. Such an implementation would be legal but not efficient because two circles with the same radius but whose *x* and *y* coordinates were reversed would then have the same hash code. The repeated multiplication and addition steps "spread out" the range of hash codes and dramatically reduce the likelihood that two unequal `Circle` objects have the same code.[3]

In practice, modern Java programmers will either autogenerate `hashCode()`, `equals()`, and `toString()` from within their IDE (for classes) or use records—where the source code compiler inserts a standard form of these methods.

## Comparable::compareTo()

Example 6-1 includes a `compareTo()` method. This method is defined by the `java.lang.Comparable` interface rather than by `Object`, but it is such a common method to implement that we include it in this section. The purpose of `Comparable` and its `compareTo()` method is to allow instances of a class to be compared to one another in a similar way to how the `<`, `<=`, `>`, and `>=` operators compare numbers. If a class implements `Comparable`, we can call methods to allow us to say that one instance is less than, greater than, or equal to another instance. This also means that instances of a `Comparable` class can be sorted.

> **NOTE**
>
> The method `compareTo()` sets up a *total ordering* of the objects of the type. This is referred to as the *natural order* of the type, and the method is called the *natural comparison method*.

Because `compareTo()` is not declared by the `Object` class, it is up to each individual class to determine whether and how its instances should be ordered and to include a `compareTo()` method that implements that ordering.

The ordering defined by Example 6-1 compares `Circle` objects as if they were words on a page. Circles are first ordered from top to bottom: circles with larger *y* coordinates come before circles with smaller *y* coordinates. If two circles have the same *y* coordinate, they are ordered from left to right. A circle with a smaller *x* coordinate comes before a circle with a larger *x* coordinate. Finally, if two circles have the same *x* and *y* coordinates, they are compared by radius. The circle with the smaller radius comes before the other one.

Note that under this ordering, two circles are equal only if all three of their fields are equal. This means that the ordering defined by `compareTo()` is consistent with the equality defined by `equals()`. This is not strictly required but is very desirable, and you should aim for it wherever possible.

The `compareTo()` method returns an `int` value that requires further explanation. `compareTo()` should return a negative number if the `this` object is less than the object passed to it. It should return 0 if the two objects are equal. And `compareTo()` should return a positive number if `this` is greater than the method argument.

## clone()

`Object` defines a method named `clone()` whose purpose is to return an object with fields set identically to those of the current object. This is an unusual method for several reasons.

First, `clone()` is declared as `protected`. Therefore, if you want your object to be cloneable by other classes, you must override the `clone()` method, making it `public`.[4] Next, the default implementation of `clone()` in `Object` throws a checked exception, `CloneNotSupportedException`, unless the class implements the `java.lang.Cloneable` interface. Note that this interface does not define any methods (it is a marker interface), so implementing it is simply a matter of listing it in the `implements` clause of the class signature.

> **WARNING**
>
> The original intent of `clone()` was to provide a mechanism to produce "deep copies" of objects, but it is fundamentally flawed, and its use is not recommended. Legacy code that uses `clone()` should be investigated and remediated.

All array types implement the `Cloneable` interface, and any array can be copied by invoking its `clone()` method. Note that a cast may be required to convert the return value to the appropriate array type, but the `clone()` method of arrays is guaranteed not to throw a `CloneNotSupportedException`:

```
int[] data = { 1, 2, 3 };
int[] copy = data.clone();
```

The `clone()` method makes a shallow copy. If the element type of the array is a reference type, only the references are copied, not the referenced objects themselves. Because the copy is shallow, any array can be cloned, even if the element type is not itself `Cloneable`.

Modern best practice is for developers to instead declare a *copy constructor* if deep copying of their objects is required, for example:

```
Circle original = new Circle(1, 2, 3);  // regular constructor
Circle copy = new Circle(original);     // copy constructor
```

We will meet copy constructors again when we consider factory methods, later in this chapter.

## Working with Fields and Object State

Java provides a variety of access control keywords that can be used to define how fields can be accessed. There are three primary choices for field access that Java developers typically use:

*Constants (*`static final`*)*

> We met this case earlier in this chapter. It may have an additional access control keyword as well.

*Immutable fields (*`private final`*)*

> Fields with this combination cannot be altered after object creation.

*Mutable fields (*`private`*)*

> This combination should be used only if the programmer is sure that the field's value will change during the object's lifetime

When writing classes, we recommend restricting your design to use these three choices for field modifiers, depending on the circumstances. Furthermore, instance fields should always be initially written as `final` and made mutable only if necessary.

In addition, direct field access (e.g., nonprivate fields) should not be used, except for constants. Instead, getter methods (and setters, for the case of mutable state) should be preferred for accessing a field from outside the class where the field is defined.

This is because direct field access is a very tight coupling between the defining class and any client code, and it violates the principle of encapsulation, which we met in Chapter 4. Conversely, if accessor methods are used, then the implementation code for those methods can later be modified without changing the client code—this is impossible with direct field access.

We should also call out one common mistake in field handling: developers coming from C++ frequently make the mistake of not specifying any access modifiers for fields. This is a serious defect, because C++ has a default visibility of private, whereas Java's default access is considerably more open. This represents a failure of encapsulation in Java, and developers

should take care to avoid it.

## Field Inheritance

As well as the considerations we've discussed, Java offers multiple potential approaches to the design issue of the inheritance of state. The programmer could choose to mark fields as `protected` and allow them to be accessed directly by subclasses (including writing to them). Alternatively, we can provide *accessor methods* to read (and write, if necessary) the actual object fields while retaining encapsulation and leaving the fields as `private`.

Let's revisit our earlier `PositionedCircle` example from the end of Chapter 4 and explicitly show the field inheritance:

```
public class Circle {
  // This is a generally useful constant, so we keep it public
  public static final double PI = 3.14159;

  protected double r;     // State inheritance via a protected field

  // A method to enforce the restriction on the radius
  protected void checkRadius(double radius) {
    if (radius < 0.0)
      throw new IllegalArgumentException("radius may not be < 0");
  }

  // The nondefault constructor
  public Circle(double r) {
    checkRadius(r);
    this.r = r;
  }

  // Public data accessor methods
  public double getRadius() { return r; }
  public void setRadius(double r) {
    checkRadius(r);
    this.r = r;
  }

  // Methods to operate on the instance field
  public double area() { return PI * r * r; }
  public double circumference() { return 2 * PI * r; }
}

public class PositionedCircle extends Circle {
  // We automatically inherit the fields and methods of Circle,
  // so we only have to put the new definitions here.
  // New instance fields that store the center point of the circle
  private final double cx, cy;

  // A new constructor to initialize the new fields
  // It uses a special syntax to invoke the Circle() constructor
  public PositionedCircle(double r, double x, double y) {
    super(r);       // Invoke the constructor of the superclass
    this.cx = x;    // Initialize the instance field cx
    this.cy = y;    // Initialize the instance field cy
  }
```

```
  public double getCenterX() {
    return cx;
  }

  public double getCenterY() {
    return cy;
  }

  // The area() and circumference() methods are inherited from Circle
  // A new instance method that checks whether a point is inside the
  // circle; note that it uses the inherited instance field r
  public boolean isInside(double x, double y) {
    double dx = x - cx, dy = y - cy;
    // Pythagorean theorem
    double distance = Math.sqrt(dx * dx + dy * dy);
    return (distance < r);                  // Returns true or false
  }
}
```

Working with accessors is the alternative to direct field inheritance, so let's examine that approach next.

## Accessors

Instead of the code shown in the last section, we can rewrite `PositionedCircle` using accessor methods, like this:

```
public class PositionedCircle extends Circle {
  // Rest of class is the same as the previous code block; the field r
  // in the superclass Circle can be made private because
  // we no longer access it directly here

  // Note that we now use the accessor method getRadius()
  public boolean isInside(double x, double y) {
    // Distance to center
    double dx = x - cx, dy = y - cy;

    // Pythagorean theorem
    double distance = Math.sqrt(dx * dx + dy * dy);
    return (distance < getRadius());
  }
}
```

Both approaches are legal Java, but they have some differences.

As we discussed in "Data Hiding and Encapsulation", fields that are writable from outside the class are usually not a correct way to model object state. In fact, as we will see later in this chapter and again in Chapter 12, they can damage the running state of a program irreparably.

It is therefore unfortunate that the `protected` keyword in Java allows access to fields (and methods) from both subclasses *and* classes in the same packages as the declaring class. This, combined with the ability for anyone to write a class that belongs to any given package (except system packages), means that protected inheritance of state is potentially flawed in Java.

> **WARNING**
>
> Java does not provide a mechanism for a member to be visible only in the declaring class and its subclasses.

For all of these reasons, it is almost always better to use accessor methods (either `public` or `protected`) to provide access to state for subclasses—unless the inherited state is declared `final`, in which case protected inheritance of state is perfectly permissible.

## Mutability and Immutability

Having seen how the use of `final` allows us to make fields immutable, it makes sense to take a moment to discuss the implications of this and why it might be a good design practice.

> **WARNING**
>
> The `final` keyword applies to fields. It does *not* apply to the contents of objects that fields reference. You can (potentially) modify the contents of an object pointed to by a `final` field. You cannot make the `final` field point to a different object.

Java has been around for over 30 years, and the landscape of computing has changed significantly over that time. In the late 1990s, Java often ran on machines with heap sizes measured in the single digits of megabytes as opposed to the multi-gigabyte heaps of today. In such a restricted environment, using an immutable style is problematic, since creating a new object for each state change causes a high rate of garbage creation, which in turn causes frequent runs of the garbage collector and a significant performance impact.

For this reason, Java programmers originally favored a style where objects were mutated and reused instead of created frequently. Over time, the reasons for this design choice became less compelling—advances in hardware mean that the cost of object creation and reclamation is orders of magnitude lower than it was in Java's early days.

Instead, in recent years, many developers have adopted the practice of using immutable data wherever possible. There are several benefits to this approach, but the main one is that if objects can be designed so that they cannot be modified after creation, then they can be freely shared between threads.

We have already mentioned the divide between mutable and immutable data, and there are consequences for the design of interfaces depending on whether we choose to use the immutable style or not.

To see this, consider a simple interface like this:

```
interface Positioned {
    double getCenterX();
    double getCenterY();
```

```
}
```

The contract represented by this interface is that objects of classes that implement `Positioned` are guaranteed to have a position in two-dimensional space.

There are essentially two ways to model the ability for objects to move, by extending `Positioned`. We can define:

```
interface TranslateInPlace extends Positioned {
    void deltaX(double dx);
    void deltaY(double dy);
    void delta(double dx, double dy);
}
```

This interface operates by side effect and implies that the object state is modified in place (and therefore that the objects that conform to it must be mutable). Alternatively, we can define an interface that is amenable to immutable implementations:

```
interface Translatable<T extends Translatable> extends Positioned {
    T deltaX(double dx);
    T deltaY(double dy);
    T delta(double dx, double dy);
}
```

This interface design means an immutable implementation, such as a `TranslatableCircle` that extends `Translatable<TranslatableCircle>`, will have methods such as `deltaX()` that return a modified copy of the original object. Both the original and the copy are immutable, and the copy differs only by the changed value of `x`.

We should also note the use of the *recursive type bound* (`T extends Translatable`) is a common intermediate-to-advanced design feature. In this context it means that the return values of the methods such as `delta()` will be of whatever concrete type implements `Translatable`, and can often be an improvement over just returning `Translatable`.

> **NOTE**
>
> It is possible, but unusual, to write a mutable implementation of interfaces like `Translatable`—the methods will typically just return `this` when a modified copy is expected.

When defining our own interfaces, it is good practice to choose one or another of these approaches consistently, and the design choice will be visible in the signatures of the interface methods. We will meet an important example of this choice, and its consequences, in Chapter 8.

However, we should also be aware that most software makes heavy use of standard libraries, open source components, etc. When using such inherited, upstream code, it is important to be aware of the conventions and approaches in use and to conform to them where possible.

# Construction Patterns

In this section we will discuss some alternatives to the direct use of constructors to create new object instances. All objects must still, ultimately, be brought into existence by the `new` operator, but for reasons of encapsulation and good design, we may choose not to expose the base constructors as an API for users.

## Factory Methods

A simple alternative to using constructors directly is the *Factory Method* pattern. The basic form of this technique is to make the constructor private (or another nonpublic modifier, in some variants) and to provide a static method that returns the desired type. This static method is then used by client code that wants an instance of the type.

There are various reasons why, as a code author, we may not want to expose our constructors directly and may choose to use factories instead—for example, because we want to implement a caching factory pattern that does not necessarily need to create a new object on each invocation, or because there are several valid ways of constructing an object, so we want to provide overloads with different arguments.

> **WARNING**
>
> The static factory approach is not the same as the *Abstract Factory* pattern from the classic book *Design Patterns*.

Let's rewrite the constructor from Example 6-1 and introduce some factory methods:

```
public final class Circle implements Comparable<Circle> {
    private final int x, y, r;

    // Main constructor
    private Circle(int x, int y, int r) {
        if (r < 0) {
            throw new IllegalArgumentException("radius < 0");
        }
        this.x = x; this.y = y; this.r = r;
    }

    // Usual factory method
    public static Circle of(int x, int y, int r) {
        return new Circle(x, y, r);
    }

    // Factory method playing the role of the copy constructor
    public static Circle of(Circle original) {
        return new Circle(original.x, original.y, original.r);
    }

    // Third factory with intent given by name
    public static Circle ofOrigin(int r) {
        return new Circle(0, 0, r);
```

```
    }

    // other methods elided
}
```

This class contains a private constructor and three separate factory methods: a "usual" one with the same signature as the constructor, and two additional. One of these additional factories is effectively a copy constructor, and the other is used to handle a special case: circles at the origin.

One advantage of using factory methods is that, unlike constructors, the method has a name and so can indicate its intent using part of the name. In our example, the factory methods are `of()`, which is one very common choice, and we distinguish the case of the origin circles by using a name `ofOrigin()` that expresses it.

The general approach is to use overloads of the factory to handle the possibility that different choices of state to create an object may exist. In particular, you should not use the `Optional` type as a method parameter, even if the state element is not essential to object construction.

If you have enough potentially optional parameters that the overloads approach becomes unwieldy and causes a proliferation of factory methods, then a different approach is preferable. One of the best known alternatives is the Builder pattern, which is the subject of our next section.

## Builders

Factory methods are a useful technique for when you don't want to expose a constructor to client code. However, there are limitations to factories. They work well when only a few parameters, all of which are required, need to be passed. But in some circumstances, we need to model data where much of it is optional, or when there are many valid possible constructions for our domain objects. In this case, the number of factory methods can quickly multiply to represent all possible combinations and overwhelm us, cluttering the API.

An alternative approach is the *Builder* pattern. This pattern uses a secondary object (the builder) that exactly parallels the state of the real domain object, which is assumed to be immutable. For every field that the domain object has, the builder has the same field—the same name and the same type. However, while the domain object is immutable, the builder object is explicitly mutable. In fact, the builder has a setter method, named in the same way as the field (i.e., in "record convention") that the developer will use to set up a piece of state.

The overall intent of the Builder pattern is to start from a "blank" builder object and add state to it, until the builder is ready to be converted into an actual domain object, usually by calling the `build()` method on the builder.

> **WARNING**
>
> Builders provide less compile-time safety than factory methods can. For example, when using builders, client code may easily miss a mandatory parameter, which would be impossible when using a factory.

Let’s take a look at a simple example:

```
// Generic builder interface
public interface Builder<T> {
    T build();
}

public final class BCircle {
    private final int x, y, r;

    // The main constructor is now private
    private BCircle(CircleBuilder cb) {
        if (cb.r < 0) {
            throw new IllegalArgumentException("negative radius");
        }
        this.x = cb.x; this.y = cb.y; this.r = cb.r;
    }

    public static class CircleBuilder implements Builder<BCircle> {
        private int x = 0, y = 0, r = 0;

        public CircleBuilder x(int x) {
            this.x = x;
            return this;
        }

        public int x() {
            return x;
        }

        // Similarly for y and r

        @Override
        public BCircle build() {
            return new BCircle(this);
        }
    }

    // Other methods elided
}
```

Notice that the builder interface is typically generic. This is because in practice we may well have a large number of domain classes, all of which will require builders, so the use of a generic builder interface removes duplication. The `Builder` interface contains only one method, so it is technically a candidate for lambda target typing. But in practice this is almost never the intent, and so it is not normally tagged with `@FunctionalInterface`. The implementation of the `build()` method also shows a mandatory use of the `this` reference.

The builder can be driven by a simple bit of code like this:

```
var cb = new BCircle.CircleBuilder();
cb.x(1).y(2).r(3);
var circle = cb.build();
```

Note that first we instantiate the builder. Then, we call the methods to set the various parameters

on the builder. Finally, we create an immutable object from the builder, by calling `build()`.

You may notice that the methods on the builder that accrete state all return `this`. The point of this interface design is so that the calls can be *chained*—so that methods can be called one after another on the same mutable object—for example, as `cb.x(1).y(2).r(3)`. Another way of describing this style of interface design is a *fluent interface*. Since each method returns `this`, we know that all of these calls are safe: there can't be a `NullPointerException`.

Our example is very simple and is somewhat contrived; it has only three parameters, and all of them are required. In practice, builders are more useful with a larger number of object parameters and when there are multiple possibilities for "spanning sets" of valid object states. There exists an overlap between the cases where one should use a factory versus a builder; determining exactly where that boundary is for your own code is part of the development of OO design skills.

## Constructor Prologues

As discussed in Chapter 4, with the arrival of Java 25, a long-standing rule of the Java language has been removed. Before this version, when defining a subclass, the first statement of any constructor had to be a call to another constructor (i.e., `super(...)` or `this(...)`).

This rule helped to ensure that newly constructed instances are valid, but it comes at a cost—it disallows some programming patterns that are useful and perfectly safe.

For example, before the rule was relaxed, it was impossible for constructors to "fail-fast" by performing parameter validation before calling a (possibly expensive) superclass constructor, like this:

```
public abstract sealed class Pet permits Cat, Dog {
    private final String name;

    protected Pet(String name) {
        this.name = name;
    }

    public String name() {
        return name;
    }
}

public final class Dog extends Pet {
    public static final Set<String> DOG_NAMES =
        Set.of("Fido", "Bruce", "Rover");

    public Dog(String name) {
        super(name);
    }
}

public final class Cat extends Pet {
    public Cat(String name) {
        if (Dog.DOG_NAMES.contains(name)) {
            throw new IllegalArgumentException(name + " is for dogs");
```

```
            }
            super(name);
        }
    }
```

Fortunately, as of Java 25 these types of patterns are now supported, and as we will see when we discuss records, constructor prologues dovetail nicely with the compact constructors that records support.

We should note that the relaxation of the rules regarding constructors also leads naturally to a related idea—the *constructor epilogue*. However, as this simply refers to code that runs after any constructor calls, the concept is of limited usefulness.

## Singleton

The *Singleton* is a very well-known design pattern. It is intended to solve the design issue where only a single instance of a class is required or desired. Java provides a number of possible ways to implement the Singleton pattern. In our discussion, we will use a slightly more verbose form, which has the benefit of being very explicit about what needs to happen for a safe Singleton:

```
public class Singleton {
  private final static Singleton instance = new Singleton();
  private static boolean initialized = false;

  // Constructor
  private Singleton() {
    super();
  }

  private void init() {
    /* Do initialization */
  }

  // This method should be the only way to get a reference
  // to the instance
  public static synchronized Singleton getInstance() {
    if (initialized) {
        return instance;
    }
    instance.init();
    initialized = true;
    return instance;
  }
}
```

The crucial point is that for the Singleton pattern to be effective, it must be impossible to create more than one of the object in question, and it must be impossible to get a reference to the object in an uninitialized state (see later in this chapter for more on this important point).

To achieve this, we require a `private` constructor, which is called only once, ever. In our version of `Singleton`, we only call the constructor when we initialize the private static variable `instance`. We also separate the creation of the only `Singleton` object from its

initialization—which occurs in the private method `init()`.

With this mechanism in place, the only way to get a reference to the lone instance of our `Singleton` is via the static helper method, `getInstance()`.[5] This method checks the flag `initialized` to see if the object is already in an active state. If it is, then a reference to the Singleton object is returned. If not, then `getInstance()` calls `init()` to activate the object and flicks the flag to `true`, so that next time a reference to the `Singleton` is requested, an additional initialization will not occur.

Finally, we also note that `getInstance()` is a `synchronized` method. See Chapter 12 for full details of what this means and why it is necessary, but for now, know that it is present to guard against unintended consequences if the `Singleton` is used in a multithreaded program.

> **WARNING**
>
> Singleton, being one of the simplest patterns, is often overused. When used correctly, it can be a useful technique, but too many Singleton classes in a program is often a code smell.

The Singleton pattern has some drawbacks—in particular, it can be hard to test and to separate from other classes. It also requires care when used in multithreaded code. Nevertheless, it is important that developers are familiar with it and do not accidentally reinvent it.

The Singleton pattern is often used in configuration management, but modern code will typically use a framework (often a *dependency injection* framework) to provide the programmer with Singletons automatically, rather than via an explicit `Singleton` (or equivalent) class.

## Instance Methods or Class Methods?

Instance methods are one of the key features of object-oriented programming. That doesn't mean, however, that you should shun class methods. In many cases, it is perfectly reasonable to define class methods.

> **TIP**
>
> Remember that in Java, class methods are declared with the `static` keyword, and the terms *static method* and *class method* are used interchangeably.

For example, when working with the `Circle` class, you might find that you often want to compute the area of a circle with a given radius but don't want to bother creating a `Circle` object to represent that circle. In this case, a class method is more convenient:

```
public static double area(double r) { return PI * r * r; }
```

It is perfectly legal for a class to define more than one method with the same name, as long as the methods have different parameter lists. This version of the `area()` method is a class method, so it does not have an implicit `this` parameter and must have a parameter that specifies the radius of the circle. This parameter keeps it distinct from the instance method of the same name.

As another example of the choice between instance methods and class methods, consider defining a method named `bigger()` that examines two `Circle` objects and returns whichever has the larger radius. We can write `bigger()` as an instance method as follows:

```
// Compare the implicit "this" circle to the "that" circle passed
// explicitly as an argument and return the bigger one.
public Circle bigger(Circle that) {
  if (this.r > that.r) return this;
  else return that;
}
```

We can also implement `bigger()` as a class method as follows:

```
// Compare circles a and b and return the one with the larger radius
public static Circle bigger(Circle a, Circle b) {
  if (a.r > b.r) return a;
  else return b;
}
```

Given two `Circle` objects, `x` and `y`, we can use either the instance method or the class method to determine which is bigger. The invocation syntax differs significantly for the two methods, however:

```
// Instance method: also y.bigger(x)
Circle biggest = x.bigger(y);
Circle biggest = Circle.bigger(x, y);  // Static method
```

Both methods work well, and, from an OOD standpoint, neither of these methods is "more correct" than the other. The instance method is more formally object-oriented, but its invocation syntax suffers from a kind of asymmetry. In a case like this, the choice between an instance method and a class method is simply a design decision. Depending on the circumstances, one or the other will likely be the more natural choice.

## Exceptions and Exception Handling

We met checked and unchecked exceptions in "Checked and Unchecked Exceptions". In this section, we discuss some additional aspects of the design of exceptions and how to use them in your own code.

Recall that an exception in Java is an object. The type of this object is `java.lang.Throwable`, or more commonly, some subclass of `Throwable` that more specifically describes the type of exception that occurred. `Throwable` has two standard

subclasses: `java.lang.Error` and `java.lang.Exception`.

`Error` objects generally indicate unrecoverable problems: the virtual machine has run out of memory or a class file is corrupted and cannot be read, for example. Exception objects of this sort can be caught and handled, but it is rare to do so—these are one family of the unchecked exceptions previously mentioned.

Subclasses of `Exception`, on the other hand, indicate less severe conditions. These exceptions can be reasonably caught and handled. They include such exceptions as `EOFException`, which signals the end of a file, and `DataFormatException`, which indicates that a program has encountered an error in a datafile such as a ZIP file.

These are the checked exceptions from Chapter 2—except for subclasses of `RuntimeException`, which are also a form of unchecked exception. An example would be `ArrayIndexOutOfBoundsException`, which indicates that a program has tried to read past the end of an array.

In this book, we use the term *exception* to refer to any exception object, regardless of whether the type of that exception is `Exception` or `Error`.

Because an exception is an object, it can contain data, and its class can define methods that operate on that data. The `Throwable` class and all its subclasses include a `String` field that stores a human-readable error message that describes the exceptional condition. It's set when the exception object is created and can be read from the exception with the `getMessage()` method. Most exceptions contain only this single message, but a few add other data. The `java.io.InterruptedIOException`, for example, adds a field named `bytesTransferred` that specifies how much input or output was completed before the exceptional condition interrupted it.

When designing your own exceptions, you should consider what other additional modeling information is relevant to the exception object. This is usually situation-specific information about the aborted operation and the exceptional circumstance that was encountered (as we saw with `java.io.InterruptedIOException`).

There are some trade-offs in the use of exceptions in application design. Using checked exceptions means that the compiler can enforce the handling (or propagation up the call stack) of known conditions that have the potential of recovery or retry. It also means that it's more difficult to forget to actually handle errors—thus reducing the risk that a forgotten error condition causes a system to fail in production.

On the other hand, some applications will not be able to recover from certain conditions, even conditions that are theoretically modeled by checked exceptions.

For example, if an application requires a config file to be placed at a specific place in the filesystem and can't locate it at startup, it may have no option but to print an error message and exit—despite the fact that `java.io.FileNotFoundException` is a checked exception.

Forcing exceptions that cannot be recovered from to be either handled or propagated is, in these

circumstances, bordering on nonsensical; in this situation, printing the error and exiting is the only really sensible action.

When designing exception schemes, here are some good practices you should follow:

- Consider what additional state needs to be placed on the exception—remember that it's also an object like any other.
- Never create an exception until you're sure you need to throw it. Exception creation can be a costly operation.
- Don't use exceptions as flow control. An exception signifies an exceptional circumstance, not a normal condition of operation.
- `Exception` has four public constructors—under normal circumstances, custom exception classes should implement all of them—to initialize the additional state or to customize messages.[6]
- Don't create many fine-grained custom exception classes in your APIs—the Java I/O and reflection APIs both suffer from this, and it needlessly complicates working with those packages.
- Don't overburden a single exception type with describing too many conditions.

Finally, here are two exception-handling antipatterns you should avoid:

```
// Never just swallow an exception
try {
  someMethodThatMightThrow();
} catch(Exception e){
}

// Never catch, log, and rethrow an exception
try {
  someMethodThatMightThrow();
} catch(SpecificException e){
  log(e);
  throw e;
}
```

The first just ignores a condition that almost certainly required some action (even if just a notification in a log). This increases the likelihood of failure elsewhere in the system—potentially far from the original, real source.

The second just creates noise. We're logging a message but not actually doing anything about the issue; we still require some other code higher up in the system to actually deal with the problem.

## Interfaces Versus Abstract Classes

When defining a new type (e.g., `Shape`) that you expect to have many subtypes (e.g., `Circle`,

`Rectangle`, `Square`), you are faced with a choice between interfaces and abstract classes when designing the inheritance relationships. Remember that a class that extends an abstract class cannot extend any other class, and that interfaces (still) cannot contain any nonconstant fields. This means there are still some restrictions on how we can use inheritance in our Java programs.

Another important difference between interfaces and abstract classes has to do with compatibility. If you define an interface as part of a public API and then later add a new mandatory method to the interface, you break any classes that implemented the previous version of the interface—in other words, any new interface methods must be declared as default and an implementation provided.

Similarly, if you use an abstract class, you can safely add non-`abstract` methods to that class without requiring modifications to existing classes that extend the abstract class—but adding a new `abstract` method will force subclass implementers to modify their code.

In both cases, adding new method definitions can cause a clash with subclass methods of the same name and signature—with the subclass methods always winning. For this reason, we need to think carefully when adding new methods, especially when the method names are "obvious" for this type or where the method could have several possible meanings.

In general, the suggested approach is to prefer interfaces when an API specification is needed, and this can be paired with a partial (aka incomplete) implementation, as we will see in the next section.

## Partial Implementations

A simple coding pattern that is widely used in Java is the *Partial Implementation* that takes advantage of the fact that an abstract class can contain a partial implementation that subclasses can build upon. Any number of subclasses can rely on method implementations provided by an abstract superclass (also called an *abstract base*).

The pattern consists of an interface that contains the API specification for the basic methods, paired with a primary partial implementation as an abstract class. A good example would be `java.util.List`, which is paired with `java.util.AbstractList`. Two of the main implementations of `List` that ship with the JDK (`ArrayList` and `LinkedList`) are subclasses of `AbstractList`.

As another example:

```
// This interface represents a shape that fits inside
// of a rectangular bounding box. Any class that wants to serve as a
// RectangularShape can implement these methods from scratch.
public interface RectangularShape extends Shape {
    void setSize(double width, double height);
    void setPosition(double x, double y);
    void translate(double dx, double dy);
    boolean isInside();
}
```

```
// A partial implementation of the preceding interface.
public abstract class AbstractRectangularShape
                        implements RectangularShape {
    // The position and size of the shape
    protected double x, y, w, h;

    // Default implementations of some of the interface methods
    public void setSize(double width, double height) {
     w = width; h = height;
    }
    public void setPosition(double x, double y) {
     this.x = x; this.y = y;
    }
    public void translate (double dx, double dy) { x += dx; y += dy; }
}
```

Various implementations may find this a useful starting point, as only the `isInside()` method needs to be supplied by subclasses.

> **NOTE**
>
> It's important to note that even though `AbstractRectangularShape` has only one abstract method, it is not usable as the target type for a lambda expression because target types must be interfaces.

The arrival of default methods in Java 8 changed this landscape considerably. Interfaces can now contain implementation code, as we saw in "Default Methods".

## Do Default Methods Change Java's Inheritance Model?

Before Java 8, the inheritance model of the Java language was clear. Every class (except `Object`) had exactly one direct superclass, and method implementations could only either be defined in a class or inherited from the superclass hierarchy. Interfaces were pure API specification and contained no implementation.

This simplicity came with some downsides—for example, it could (and often did) lead to duplication of code when the interface had multiple implementations. Patterns such as Partial Implementation became widely used as workarounds in response to these downsides.

Java 8 fundamentally changed this simple OO programming model, by allowing default methods. This means that method implementations can be inherited from multiple places—either from the superclass hierarchy or from default implementations provided in interfaces. Any potential conflicts between different default methods from separate interfaces will result in a compile-time error.

The mandatory methods of the interface are nondefault, as they represent the part of the API that must be present for an implementation to be considered valid. Default methods should be used only if a method is truly optional, or if they are intended to have only a single possible implementation.

This means there is no possibility of conflicting multiple inheritance of implementations, since in any clash the programmer is required to manually disambiguate the methods.

There is also no multiple inheritance of state: interfaces still do not have non-constant fields.

Java's approach to multiple inheritance is different from the general multiple inheritance found in, for example, C++. In fact, default methods are effectively the *Mixin* pattern from C++ (for readers who are familiar with that language). Some developers also view default members as a form of the *trait* language feature that appears in some OO languages (e.g., Scala).

However, the official position from Java's language designers is that default methods fall short of being full traits. This view is somewhat undermined by the code that ships within the JDK—even the interfaces within `java.util.function` (such as `Function` itself) behave as simple traits.

For example, consider the `Function` type from `java.util.function`, which maps a value of type `T` to a value of type `R`:

```
@FunctionalInterface
public interface  Function<T, R> {
    R apply(T t);

    default <V> Function<V, R>
      compose(Function<? super V, ? extends T> before) {
        Objects.requireNonNull(before);
        return (V v) -> apply(before.apply(v));
    }

    default <V> Function<T, V>
      andThen(Function<? super R, ? extends V> after) {
        Objects.requireNonNull(after);
        return (T t) -> after.apply(apply(t));
    }

    static <T> Function<T, T> identity() {
        return t -> t;
    }
}
```

The usual case of using the `Function` interface is as a lambda expression, where the programmer supplies the body of the `apply()` method. However, as well as this single abstract method, the interface has two default methods (and a single static helper to create the identity mapping).

These are the functional composition methods (`compose()` and `andThen()`) and they show an important semantic point—it is highly implausible (or even inconceivable) that any sensible override of, for example, the default `compose()` method could exist. Put another way, the function objects defined by the `Function` interface will only ever be composed in the standard way. In turn, this shows that within the limited domain discussed, default methods can indeed be treated as a form of stateless trait.

Finally, to conclude this section, we should discuss an older (pre–Java 8) technique of dealing

with methods of an interface that are considered “optional.” This method is generally unsatisfactory and is not recommended for modern code, but you will almost certainly encounter it at some point and so it is important to be aware of and recognize it.

Before default methods were part of the Java language, library designers sometimes needed to convey to implementers that some methods of an interface were not necessary for a working implementation and could be omitted.

This was achieved by declaring in the library’s documentation which methods were optional and directing implementers to unconditionally throw `java.lang.UnsupportedOperationException` if the implementation does not support these optional capabilities.

This is deeply problematic for several reasons:

- It relies on consumers of the API to actually read the documentation and understand that some methods may not be supported by all implementations.
- There is no way to tell whether a given instance supports the optional methods or not without specific knowledge of the implementing class.
- `UnsupportedOperationException` is a runtime exception and thus is not present in the method signature of the optional methods.

In practice, this approach can cause unexpected runtime exceptions when dealing with objects of unknown provenance, and so it has always been best avoided.

## Delegation

Inheritance is not the only technique at our disposal in OOD. Objects can contain references to other objects, so a larger conceptual unit can be aggregated out of smaller component parts; this is known as *composition*.

An important related technique is *delegation*, where an object of a particular type holds a reference to a secondary object of a compatible type and forwards all operations to the secondary object.[7] This is frequently done using interface types, as shown in this example where we model the employment structure of software companies:

```
public interface Employee {
  void work();
}

public class Programmer implements Employee {
  public void work() { /* program a computer */ }
}

public class Manager implements Employee {
  private Employee report;

  public Manager(Employee staff) {
    report = staff;
```

```
  }

  public Employee setReport(Employee staff) {
    report = staff;
  }

  public void work() {
    report.work();
  }
}
```

The `Manager` class is said to *delegate* the `work()` operation to their direct report, and no actual work is performed by the `Manager` object. Variations of this pattern involve some work being done in the delegating class, with only some calls being forwarded to the delegate object. This variation is especially common in, for example, caching solutions where the cache is checked first before forwarding to an origin source.

Another useful and related technique to delegation is the *decorator pattern*. This provides the capability to extend objects with new functionality, including at runtime. It is very widely used, especially in the JDK utility classes. It's best explained via examples, so we will introduce it when we discuss Java I/O in "Decorators".

Now, let's move on to discuss lambdas and other recent language features and how to use them effectively in OOD.

# Modern OOD in Java

In this section we're going to discuss how lambda expressions, records, and sealed types can be used to bring some simple functional programming design patterns to Java programs.

> **TIP**
>
> One FP term that might be useful is that of *algebraic data types*. These are used to construct types from existing types via two possible mechanisms, known as *sum* and *product* types. In Java, the sum types are `sealed` types and the product types are records.

This is an evolving area—the community is still figuring out just how much functional programming feels natural within the Java ecosystem. Let's start with lambdas, which should hopefully already be familiar, as we met them in Chapters 2 and 5.

## OOD Using Lambdas

Consider this simple lambda expression:

```
Runnable r = () -> System.out.println("Hello World");
```

The type of the *lvalue* (lefthand side of the assignment) is `Runnable`, which is an interface

type. For this statement to make sense, the *rvalue* (righthand side of the assignment) must contain an instance of some class type (because interfaces cannot be instantiated) that implements `Runnable`. The minimal implementation that satisfies these constraints is a class type (of inconsequential name) that directly extends `Object` and implements `Runnable`.

Recall that the intention of lambda expressions is to allow Java programmers to express a concept that is as close as possible to the anonymous or inline methods seen in other languages.

Furthermore, given that Java is a statically typed language, this leads directly to the design of lambdas as implemented.

> **TIP**
>
> Lambdas are a shorthand for the construction of a new instance of a class type that is essentially an `Object` enhanced by a single method.

A lambda's single extra method has a signature provided by the interface type, and the Java source code compiler will check that the *rvalue* is consistent with this type signature.

### Lambdas versus nested classes

The addition of lambdas to the language in Java 8 was relatively late, as compared to other modern programming languages. As a consequence, the Java community had established patterns to work around the absence of lambdas. This manifested in a heavy use of very simple nested (aka inner) classes to fill the niche that lambdas usually occupy, and this can still be clearly seen in older Java code.

In modern Java projects developed from scratch, developers will typically use lambdas wherever possible. We also strongly suggest that, when refactoring old code, you take some time to convert inner classes to lambdas wherever possible—and some IDEs even provide an automatic conversion facility.

However, this still leaves the design question of when to use lambdas and when nested classes are still the correct solution.

Some cases are obvious; for example, when extending a default implementation for some functionality, a nested class approach is not only appropriate but necessary in two cases:

- The custom implementation has to override multiple methods
- The base implementation is a class, not an interface

Another major use case to consider is that of stateful lambdas. As there is nowhere to declare any fields, it would appear at first glance that lambdas cannot directly be used for anything that involves state—the syntax only gives the opportunity to declare a method body.

However, a lambda can refer to a variable defined in the scope that the lambda is created in, so we can create a closure, as discussed in Chapter 5, to fill the role of a stateful lambda. This

means that the choice of a closure versus a nested class is ultimately more a matter of style and code readability than anything else.

### Lambdas versus method references

The question of when to use a lambda and when to use a method reference is largely a matter of personal taste and style. There are, of course, some circumstances where it is essential to create a lambda. However, in many simple cases, a lambda can be replaced by a method reference.

One possible approach is to consider whether the lambda notation adds anything to the readability of the code. For example, in the Streams API, there is a potential benefit in using the lambda form, as it uses the `->` operator. This provides a form of visual metaphor—the Stream API is a lazy abstraction that can be visualized as data items "flowing through a functional pipeline."

For example, let's consider a `Person` object, which has standard characteristics, such as name, age, etc. We can compute the average using a pipeline, like this:

```
List<Person> persons = ... // obtained from somewhere
double aveAge = persons.stream()
        .mapToDouble(o -> o.getAge())
        .reduce(0, (x, y) -> x + y ) / persons.size();
```

The idea that the `mapToDouble()` method has an aspect of motion, or transformation, is strongly implied by the usage of an explicit lambda. For less experienced programmers, it also draws attention to the use of a functional API.

For other use cases, method references may well be more appropriate. For example, consider this simple *dispatch table*, where operations are looked up based on a string key:

```
public final class IntOps {
    private Map<String, IntBinaryOperator> table =
        Map.of("add", IntOps::add, "subtract", IntOps::sub);

    private static int add(int x, int y) {
        return x + y;
    }

    private static int sub(int x, int y) {
        return x - y;
    }

    public int eval(String op, int x, int y) {
        return table.get(op).apply(x, y);
    }
}
```

In situations where either notation could be used, you will come to develop a preference that fits you and your team's style over time. The key consideration is whether, when returning to reread code written several months (or years) ago, the choice of notation still makes sense and the code is easy to read.

Let’s discuss how records can play a major role in modern OO Java designs.

## OOD Using Records

Records were introduced in Chapter 4, and in their simplest form they represent a data entity that is “just fields” or a “bag of data.” In some other programming languages, this pattern is represented by a *tuple*, but Java’s records are different from tuples in two important ways:

1. Java records are named types, whereas tuples are anonymous.[8]
2. Java records can have methods, auxiliary constructors, and almost everything a class can

Both of these stem from the fact that records are a special type of class. This allows the programmer to start their design by using a record as a basic collection of fields and then to evolve from there.

For example, let’s rewrite Example 6-1 as a record (eliding the `Comparable` interface for simplicity):

```
public record Circle(int x, int y, int r) {
    // Primary (compact) constructor
    public Circle {
        // Validation code in the constructor
        // This would be impossible in a tuple
        if (r < 0) {
            throw new IllegalArgumentException("negative radius");
        }
    }

    // Factory method playing the role of the copy constructor
    public static Circle of(Circle original) {
        return new Circle(original.x, original.y, original.r);
    }
}
```

Note that we have introduced a new type of constructor, called a *compact constructor*. It is available only for records and is used in the case where we want to do a bit of extra work in the constructor as well as initialize the fields—in a similar way to a constructor prologue. Compact constructors don’t have (or need) a parameter list, because they always have the same parameter list as the declaration of the record.

This code is much shorter than Example 6-1 and clearly distinguishes the case of the primary constructor (the “true form”) of the record from the copy constructor and any other factories that may be present.

Let’s also take a quick look at how we would use a record with an interface that is designed for immutable implementations. Recall that our interfaces are defined like this:

```
interface Positioned extends Shape {
    double getCenterX();
    double getCenterY();
```

```
}


interface Translatable<T extends Translatable> extends Positioned {
    T deltaX(double dx);
    T deltaY(double dy);
    T delta(double dx, double dy);
}
```

and then we can define our record class to implement the interfaces:

```
public record TranslatableCircle(double x, double y, double r)
  implements Translatable<TranslatableCircle> {
    public static final double PI = 3.14159;

    // Constructors and factories elided

    @Override
    public double getCenterX() {
        return x();
    }

    @Override
    public double getCenterY() {
        return y();
    }

    @Override
    public double area() { return PI * r * r; }

    @Override
    public double circumference() { return 2 * PI * r; }

    @Override
    public TranslatableCircle deltaX(double dx) {
        return new TranslatableCircle(x() + dx, y(), r());
    }

    @Override
    public TranslatableCircle deltaY(double dy) {
        return new TranslatableCircle(x(), y() + dy, r());
    }

    @Override
    public TranslatableCircle delta(double dx, double dy) {
        return new TranslatableCircle(x() + dx, y() + dy, r());
    }
}
```

Note that we have to *adapt* the records' accessor methods (`x()` and `y()`) to the names that the `Positioned` interface expects.

This is a common occurrence: a lot of older Java code uses the Java Beans conventions that we met briefly in Chapter 4, but records (which, of course, lack setter methods) use the convention that getters are named the same as the field name. The end result is that when records need to conform to an existing interface, this type of *retrofitting* is often necessary.

As we can see, the design of Java's records means that they are a very flexible choice for the programmer. An entity can be initially modeled as just fields, and over time, can acquire more methods, implement interfaces, and so on.

Records can also be combined with another modern OO feature, sealed types, which are the subject of our next section.

## OOD Using Sealed Types

We met sealed classes for the first time in Chapter 4 and introduced sealed interfaces in Chapter 5. As well as the cases we've already met, there is also a simpler possibility, in which a sealed type can be extended only by classes that are defined inside the same compilation unit (i.e., Java source file), like this:

```
// Note the absence of a permits clause
public abstract sealed class Shape {

    public static final class Circle extends Shape {
        // ...
    }

    public static final class Rectangle extends Shape {
        // ...
    }
}
```

The classes `Shape.Circle` and `Shape.Rectangle` are the only permitted subclasses of `Shape`: any other attempt to extend `Shape` will result in a compilation error. This is really just additional detail, as the general concept remains the same; `sealed` indicates a type that has only a finite number of possible types that are compatible with it.

Sealing thus represents a constrained hierarchy of types that are known at design time. Each class that participates in a sealing relationship must have a keyword that describes the class's role—either `final` or `sealed` (with an additional `permits` clause) or `non-sealed`.

The last option (`non-sealed`) should not be used without a very good reason, as it undermines much of the semantic point of using class sealing in the first place.

> **WARNING**
>
> In general, `non-sealed` is very likely to be a code smell. It is occasionally legitimately encountered in, for example, systems built with *object-relational mapping* (ORM) or other techniques that use runtime generation of code. It should always provoke scrutiny from the code reviewer, however.

Sealed interfaces can also be used very effectively in combination with records (which are implicitly `final`, of course).

Let's take a look at an example—a delivery company that has exactly two types of orders: basic

orders (delivered free) and express orders (delivered more quickly but for an additional charge).

Two good choices for implementing this are:

- Model the entities as classes and use a `sealed abstract` base class
- Model the entities as records and use a `sealed` interface

In the second case, both order types will have the common fields (i.e., record components) hoisted up into the interface

Let's investigate the second approach—so that our basic interface for modeling the orders looks like this:

```
sealed interface Order permits BasicOrder, ExpressOrder {
    double price();
    String address();
    LocalDate delivery();
}
```

and the order types themselves are defined as:

```
public record BasicOrder(double price,
                         String address,
                         LocalDate delivery) implements Order {}

public record ExpressOrder(double price,
                           String address,
                           LocalDate delivery,
                           double deliveryCharge) implements Order {}
```

This is a very compact way to model our domain entities, but we can go even further, as we will see in the next section.

## Pattern Matching

There is an interesting duality in Java's version of OO:

- Enums have only a finite number of instances—any `enum` object is one of those instances.
- Sealed types have only a finite number of compatible classes—any `sealed` object belongs to one of those classes.

Now, let's consider a switch expression that accepts an enum, for example:

```
enum Season {
    SPRING, SUMMER, AUTUMN, WINTER
}

Season season = getSeason();
```

```
var temp = switch(season) {
    case WINTER -> 2.0;
    case SPRING -> 10.5;
    case SUMMER -> 24.5;
    case AUTUMN -> 16.0;
};
System.out.println("Average temp: " + temp);
```

All the possible enum constants for seasons are present in this switch expression, and so the match is said to be *total*. In this case, it is not necessary to include a `default`, as the compiler can use the *exhaustiveness* of the enum constants to infer that the default case would never be activated.

It isn't hard to imagine that we could do something similar with sealed types. Consider code like this:

```
Order order = getNextOrder();

if (order instanceof BasicOrder b) {
    IO.println("Total Price: " + b.price());
} else if (order instanceof ExpressOrder x) {
    IO.println("Total Price: " + (x.price() + x.deliveryCharge()));
}
```

It does work, but while it is obviously exhaustive to a human, it was not until Java 21 that the source code compiler was able to recognize this, via a new capability called *pattern matching*.

> **NOTE**
>
> The pattern matching language feature comes from the FP tradition—it does not have anything to do with either design patterns (which arose with OO programming) or regular expression and string pattern matching.

This feature enables developers to write code that, for example, "switches over a variable's type," like this:

```
Order order = getNextOrder();

var totalPrice = switch (order) {
    case BasicOrder b -> b.price();
    case ExpressOrder x -> x.price() + x.deliveryCharge();
};
IO.println("Total Price: " + totalPrice);
```

Note that, because `Order` is a sealed type, this match is exhaustive (aka "total"), so it does not need a default case, but it will still throw a `NullPointerException` if `order` is `null`.

When matching on a record object, a type pattern can *deconstruct* the record to extract values from within it, and if not all the fields on a record are used, then they can be ignored with the _ unnamed variable syntax:

```
Order order = getNextOrder();

totalPrice = switch (order) {
    case BasicOrder (double price, _, _) -> price;
    case ExpressOrder (double price, _, _, double delivery)
        -> price + delivery;
};
IO.println("Total Price: " + totalPrice);
```

In other languages, deconstruction is known by other names, including *destructuring* in JavaScript. Note that in Java we do repeat the type of the parameter in the deconstruction expression. This is for clarity and also to help protect against typos caused by too many (or too few) _ params.

Java also supports *case guards*, which allow the programmer to attach a condition to a case, like this:

```
var refundDue = switch (order) {
    case BasicOrder _ -> false;
    case ExpressOrder (_, _, LocalDate d, _)
        when LocalDate.now().isAfter(d)-> true;
    case ExpressOrder _ -> false;
};
IO.println("Order is eligible for shipping refund: "+ refundDue);
```

When constructing these types of more-complex pattern matches, we must take care that all the possible cases are covered, and the ordering of the cases does matter. For example, if we move the general case of the `ExpressOrder` above the guarded case, then the latter will never trigger, causing a compilation error.

The current version of pattern matching is very useful in OO design, but it is still under active development, with forthcoming enhancements intended to enable new design patterns inspired by FP, which have not been easy to achieve in Java until now.[9]

## Safe Java Programming

Programming languages are sometimes described as being *type safe*; however, this term is used rather loosely by working programmers. There are a number of viewpoints on and definitions for type safety, not all of which are mutually compatible.

The most useful view for our purposes is that *type safety* is the property of a programming language that prevents the type of data being incorrectly identified at runtime. This should be thought of as a sliding scale—it is more helpful to think of one language as being more (or less) type safe than another, rather than a simple binary property of safe/unsafe.

In Java, the static nature of the type system helps prevent a large class of possible errors by producing compilation errors if, for example, the programmer attempts to assign an incompatible value to a variable. However, Java is not perfectly type safe, as we can perform a cast between any two reference types—this will fail at runtime with a `ClassCastException` if the value

is not compatible.

In this book, we prefer to think of safety as inseparable from the broader topic of correctness. This means that we should think in terms of programs, rather than languages. This emphasizes the point that safe code is not guaranteed by any widely used language, and instead considerable programmer effort (and adherence to rigorous coding discipline) must be employed if the end result is to be truly safe and correct.

We approach our view of safe programs by working with the state model abstraction, as shown in Figure 6-1.

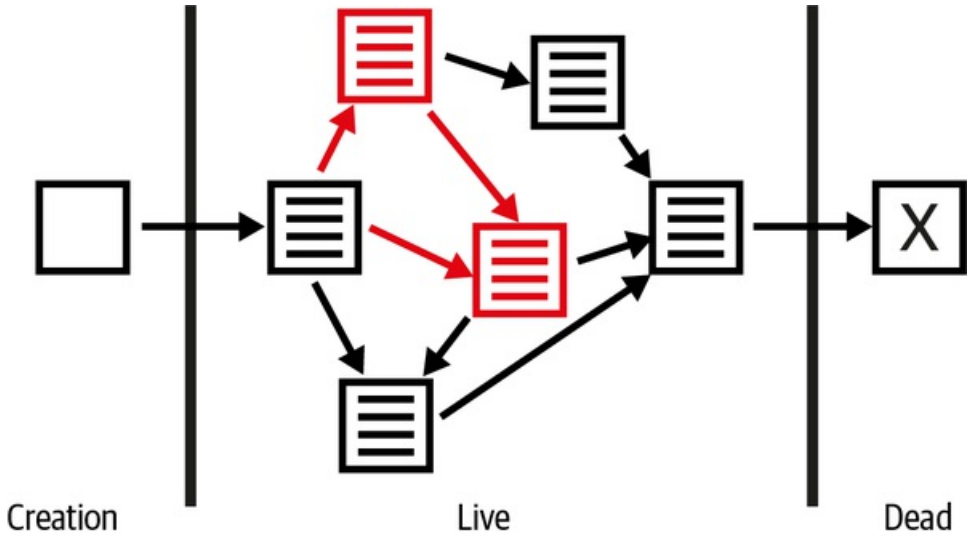



*Figure 6-1. Program state transitions*

More formally, a *safe* program is one in which:

- All objects start off in a legal state after creation.
- Externally accessible methods transition objects between legal states.
- Externally accessible methods must not return with objects in an inconsistent state.
- Externally accessible methods must reset objects to a legal state before throwing an exception.

In this context, *externally accessible* means `public`, package-private, or `protected`. This defines a reasonable model for safety of programs, and as it is bound up with defining our abstract types in such a way that their methods ensure consistency of state, it's reasonable to refer to a program satisfying these requirements as a "safe program," regardless of the language in which such a program is implemented.

> **WARNING**
>
> Private methods do not have to start or end with objects in a legal state, as they cannot be called by an external piece of code.

As you might imagine, actually engineering a substantial piece of code so that we can be sure that the state model and methods respect these properties can be quite an undertaking. In languages such as Java, in which programmers have direct control over the creation of preemptively multitasked execution threads, this problem is a great deal more complex.

Let's move on to discuss one of the most well-known problems in OO design that everyone learning the subject for the first time should be aware of.

## Case Study: Circle-Ellipse Problem

To conclude this chapter, we're going to consider a seemingly simple problem and follow the logical consequences of attempting to model it using OO techniques.

Our chosen domain is modeling circles and ellipses that are centered at the origin of the standard (i.e., Euclidean) plane. At first sight, it seems obvious that there should be some sort of subtyping relationship at work—after all, what is a circle if not a special case of an ellipse, mathematically speaking?

Let's start with the simple possible case of immutable objects and our familiar `Shape` interface:

```
public interface Shape {
    public static final double PI = 3.14159265358979323846;

    double area();
    double circumference();
}

public record Circle(int r) implements Shape {
    // Some methods may be elided ...

    @Override
    public double area() { return PI * r * r; }

    @Override
    public double circumference() { return 2 * PI * r; }
}

public record Ellipse(double a, double b) implements Shape {
    private double h() {
        return ((a - b) * (a - b)) / ((a + b) * (a + b));
    }

    @Override
    public double area() {
        return PI * a * b;
    }

    @Override
    public double circumference() {
        return PI * (a + b) * (1 + 3 * h() /
                               (10 + Math.sqrt(4 - 3 * h())));
    }
```

```
}
```

Writing down this extremely simple model first of all, without worrying about any inheritance/subtyping relationships between the classes directly exposes a simple fact:

Ellipses have *two* parameters (the lengths of the two axes), whereas a circle has only *one* (the radius).

> **NOTE**
>
> Perhaps surprisingly, the circumference of an ellipse does not have a simple closed-form expression, and the formula we use here is Ramanujan's second approximation.

A circle can be understood as a special case (or a limiting case) of an ellipse where both parameters are equal, which seems to imply that it would make sense to model the types with some sort of subtyping. However, to see the limitations of that approach, let's introduce resizing of the objects and see how the code needs to evolve, first with a design based on mutable state:

```
public interface ShearInPlace extends Shape {
    void shear(double dx, double dy);

    default void shearX(double dx) {
        shear(dx, 0.0);
    }

    default void shearY(double dy) {
        shear(0.0, dy);
    }
}

public final class ResizableCircle implements ShearInPlace {
    private double r;

    public ResizableCircle(double r) {
        this.r = r;
    }

    @Override
    public void shear(double dx, double dy) {
        r = r + Math.min(dx, dy);
    }

    @Override
    public void shearX(double dx) {
        shear(dx, dx);
    }

    @Override
    public void shearY(double dy) {
        shear(dy, dy);
    }

    // Other methods elided
```

```
}

public final class ResizeableEllipse implements ShearInPlace {
    private double a;
    private double b;

    public ResizeableEllipse(double a, double b) {
        this.a = a;
        this.b = b;
    }

    @Override
    public void shear(double dx, double dy) {
        a += dx;
        b += dy;
    }

    // Other methods elided
}
```

The types have to be classes rather than records, due to the mutable state, but the fundamental difference remains—circles have one field and ellipses have two.

It's very difficult to see how an ellipse could be considered an "augmented" form of a circle—the concept of two separate foci for the ellipse does not have an analogue for the circle. Conversely, the desire to represent a circle as an ellipse with an extra invariant (or constraint)—that the two axes must remain equal—also seems almost impossible to model.

There is no easy resolution to the problem by using immutable data either:

```
interface Shearable<T extends Shearable> extends Shape {
    T shear(double dx, double dy);

    default T shearX(double dx) {
        return shear(dx, 0.0);
    }

    default T shearY(double dy) {
        return shear(0.0, dy);
    }
}

public record ShearableCircle(double r)
        implements Shearable<ShearableCircle> {
    // Methods elided

    @Override
    public ShearableCircle shear(double dx, double dy) {
        return new ShearableCircle(r + Math.min(dx, dy));
    }

    @Override
    public ShearableCircle shearX(double dx) {
        return shear(dx, dx);
    }

    @Override
```

```
    public ShearableCircle shearY(double dy) {
        return shear(dy, dy);
    }
}

public record ShearableEllipse(double a, double b)
        implements Shearable<ShearableEllipse>{
    // Methods elided

    @Override
    public ShearableEllipse shear(double dx, double dy) {
        return new ShearableEllipse(a + dx, b + dy);
    }
}
```

We are forced to conclude that this problem is intractable—despite the superficial attractiveness, there is no good subtyping relationship between circles and ellipses. The best we can do is a "sibling" relationship, expressed by both types implementing `Shape`, and a couple of conversion methods:

```
public record ShearableCircle(double r)
        implements Shearable<ShearableCircle> {
    // ...

    public ShearableEllipse asEllipse() {
        return new ShearableEllipse(r, r);
    }

    // ...
}

public record ShearableEllipse(double a, double b)
        implements Shearable<ShearableEllipse>{
    // ...

    public ShearableCircle asCircle(boolean isInterior) {
        if (isInterior) {
            return new ShearableCircle(Math.min(a, b));
        }
        return new ShearableCircle(Math.max(a, b));
    }

    // ...
}
```

This example shows one good method for approaching OO designs—by starting with the fields (i.e., the data or state) first and using records as our primary modelling tool, we could immediately see a difference between the key entities of interest. The difficulty of finding a natural subtyping relationship and of preserving the circle's extra invariant were the extra clues we needed to conclude that the best way to solve the "circle-ellipse puzzle" was simply not to participate.

## Summary

In this chapter we have met a number of OO design techniques that bring together what we've learned about Java's type system and approach to object-oriented programming.

Mutability has been a major topic, which reflects the current interest in the Java community about the subject.

Moving on from our introduction of object-oriented design, and from Part I, we're ready to tackle the topics of Java programming and properly introduce the Java standard library (including the Collections, which so far we have met only informally).

---

1 Technically, this should probably be called `SCREAMING_SNAKE_CASE`.

2 For efficient operation of hash tables, it helps if unequal objects are at least unlikely to share a hash code.

3 The somewhat hazy description of hashing in this section can be made rigorous and is an important piece of some foundational techniques. A useful primer can be found in a *Communications of the ACM* article.

4 Overriding a method can broaden its visibility (e.g., from `protected` to `public`) but cannot narrow it.

5 This means that the Singleton can be thought of as a special case of the Factory we met earlier.

6 An IDE will usually autogenerate stubs for these constructors.

7 Java does not have explicit language-level support for delegates, unlike some languages—such as Kotlin.

8 Python does have `@namedtuple`, but it is less powerful than Java records.

9 Appendix A has more information about pattern matching enhancements and other future features.

# Part II. Working with the Java Platform

Part II is an introduction to some of the core libraries that ship with Java and some programming techniques that are common to intermediate and advanced Java programs.


Chapter 7, “Programming and Documentation Conventions”

Chapter 8, “Working with Java Collections”

Chapter 9, “Java and Memory”

Chapter 10, “Handling Common Data Formats”

Chapter 11, “File Handling and I/O”

Chapter 12, “Concurrency in Java”

Chapter 13, “Classloading, Reflection, and Method Handles”

Chapter 14, “Java Platform Modules”

# Chapter 7. Programming and Documentation Conventions

This chapter explains a number of important and useful Java programming and documentation conventions. It covers:

- General naming and capitalization conventions
- Portability tips and conventions
- `javadoc` documentation comment syntax and conventions

## Naming and Capitalization Conventions

The following widely adopted naming conventions apply to modules, packages, classes, interfaces, methods, fields, and constants in Java. Because these conventions are almost universally followed and because they affect the public API of the classes you define, you should adopt them as well:

*Modules*

As modules are the preferred unit of distribution for Java applications from Java 9 onward, you should take special care when naming them.

Module names must be globally unique—the modules system is essentially predicated on this assumption. As modules are effectively superpackages (or aggregates of packages), the module name should be closely related to the package names grouped into the module.

One recommended way to do this is to group the packages within a module and use the *root name* of the packages as the module name. For example, if an application's packages all live under `com.mycompany.*`, then `com.mycompany` is a good name for your module.

*Packages*

It is customary to ensure that your publicly visible package names are unique. One common way of doing this is by prefixing them with the inverted name of an internet domain that you own (e.g., `com.oreilly.javanutshell`).

This convention is now followed less strictly than it used to be, with some projects merely adopting a simple, recognizable, and unique prefix instead. All package names should be lowercase.

*Classes*

A type name should begin with a capital letter and be written in mixed case (e.g., `String`).

This is usually referred to as *Pascal case*. If a class name consists of more than one word, each word should begin with a capital letter (e.g., `StringBuffer`). If a type name, or one of the words of a type name, is an acronym, the acronym can be written in all capital letters (e.g., `URL`, `HTMLParser`).

Because classes and enumerated types are designed to represent objects, you should choose class names that are nouns (e.g., `Thread`, `Teapot`, `FormatConverter`).

Enum types are a special case of a class with a finite number of instances. They should be named as nouns in all but highly exceptional circumstances. The constants defined by `enum` types are also typically written in all capital letters, as per the rules for constants that follow.

*Interfaces*

Java programmers typically use interfaces in one of two ways: either to convey that a class has additional, supplementary aspects or behaviors or to indicate that the class is one possible implementation of an interface for which there are multiple valid choices.

When an interface is used to provide additional information about the classes that implement it, it is common to choose an interface name that is an adjective (i.e., `Runnable`, `Cloneable`, `Serializable`).

When an interface is intended to work more like an abstract superclass, use a name that is a noun (e.g., `Document`, `FileNameMap`, `Collection`). Unlike some languages such as C#, it is conventional to not indicate via the name that it is an interface (i.e., don't use `IDocument` or `DocumentInterface`).

*Methods*

A method name always begins with a lowercase letter. If the name contains more than one word, every word after the first begins with a capital letter (e.g., `insert()`, `insertObject()`, `insertObjectAt()`). This is usually referred to as *camel case*.

Method names are typically chosen so that the first word is a verb. Method names can be as long as is necessary to make their purpose clear, but choose succinct names where possible. Avoid overly general method names, such as `performAction()`, `go()`, or the dreadful `doIt()`.

*Fields and constants*

Nonconstant field names follow the same capitalization conventions as method names. A field name should be chosen to best describe the purpose of the field or the value it holds. Prefixes to indicate types or visibility of fields are discouraged.

If a field is a `static final` constant, it should be written in all uppercase. If the name of a constant includes more than one word, the words should be separated with underscores (e.g., `MAX_VALUE`).

*Parameters*

Method parameters follow the same capitalization conventions as nonconstant fields. Although not part of the binary contract of a method definition, the names of method parameters appear in the documentation (and in the auto-complete of many IDEs and code editors), so you should choose names that make the purpose of the parameters as clear as possible. Try to keep parameter names to a single word and use them consistently between methods on a class. For example, if a `WidgetProcessor` class defines many methods that accept a `Widget` object as the first parameter, name this parameter `widget`.

*Local variables*

Local variable names are an implementation detail and never visible outside your class. Nevertheless, choosing good names makes your code easier to read, understand, and maintain. Variables are typically named following the same conventions as methods and fields.

In addition to the conventions for specific types of names, there are conventions regarding the characters you should use in your names. Java allows the `$` character in any identifier, but, by convention, its use is reserved for synthetic names generated by source-code processors. For example, it is used by the Java compiler to make inner classes work. You should not use the `$` character in any name that you create.

Java allows names to use any alphanumeric characters from the entire Unicode character set. While this can be convenient for non-English-speaking programmers, Unicode use has never really taken off, and this usage is extremely rare.

## Practical Naming

The names we give to our constructs matter—a lot. Naming is a key part of the process that conveys our abstract designs to our peers. The process of transferring a software design from one human mind to another is hard—harder, in many cases, than the process of transferring our design from our mind into the machines that will execute it.

We must, therefore, do everything we can to ensure that this process is made easier. Names are a keystone of this. When reviewing code (and all code should be reviewed), pay particular attention to the names that have been chosen:

- Do the names of the types reflect the purpose of those types?
- Does each method do exactly what its name suggests? Ideally, no more and no less?
- Are the names descriptive enough? Could the name be more specific?
- Are the names well suited to the domain they describe?
- Are the names consistent across the domain?

- Do the names mix metaphors?
- Do the names reuse a common term of software engineering?
- Do the names of boolean-returning methods include negation? These often need more attention to understand when reading (e.g., `notEnabled()` versus `enabled()`)

Mixed metaphors are common in software, especially after several releases of an application. A system that starts off perfectly reasonably with components called `Receptionist` (for handling incoming connections), `Scribe` (for persisting orders), and `Auditor` (for checking and reconciling orders) can quite easily end up in a later release with a class called `Watchdog` for restarting processes. This isn't terrible, but it breaks the previously established pattern of people's job titles.

It is also incredibly important to realize that software changes a lot over time. A perfectly apposite name on version 1 can become highly misleading by version 4. Care should be taken that as the system focus and intent shift, the names are refactored along with the code. Modern IDEs have no problem with global search and replace of symbols, so there is no need to cling to outdated metaphors once they are no longer useful.

One final note of caution: an overly strict interpretation of these guidelines can lead the developer to some very odd naming constructs. There are a number of excellent descriptions of some of the absurdities that can result by taking these conventions to their extremes.

In other words, none of the conventions described here are mandatory. Following them will, in the vast majority of cases, make your code easier to read and maintain. However, you should not be afraid to deviate from these guidelines if it makes your code more understandable.

> *Break any of these rules sooner than say anything outright barbarous.*
>
> —George Orwell, *Politics and the English Language*

Above all, you should have a sense of the expected lifetime of the code you are writing. A risk calculation system in a bank may have a lifetime of a decade or more, whereas a prototype for a start-up may be relevant for only a few weeks. Document accordingly—the longer the code is likely to be live, the more robust its documentation and naming need to be.

## Java Documentation Comments

Most ordinary comments within Java code explain the implementation details of that code. By contrast, the Java Language Specification defines a special type of comment known as a *doc comment* that serves to document the API of your code.

Doc comments appear in two formats in modern Java. The original is an ordinary multiline comment that begins with `/**` (instead of the usual `/*`) and ends with `*/`. Documentation within the multiline comment can include simple HTML formatting and other special keywords we'll discuss in this chapter.

The second format added in recent Java versions supports Markdown (specifically, the CommonMark variant). Markdown doc comments are denoted with per-line comments of the form `///` (note the additional `/`, much like the multiline comments' added `*`). Along with standard Markdown constructs, the same special keywords as the multiline doc comments are supported.

Doc comments of either form appear immediately before a type or member definition and contain documentation for that type or member.

Doc comments are ignored by the compiler, but they can be extracted and automatically turned into online HTML documentation by the `javadoc` program. (See Appendix B for more information about `javadoc`.)

Here is an example class that contains appropriate multiline doc comments:

```
/**
 * This immutable class represents <i>complex numbers</i>.
 *
 * @author David Flanagan
 * @version 1.0
 * @see #Complex
 */
public class Complex {
    /**
     * Holds the real part of this complex number.
     * @see #y
     */
    protected double x;

    /**
     * Holds the imaginary part of this complex number.
     * @see #x
     */
    protected double y;

    /**
     * Creates a new Complex object that represents the complex number
     * x+yi.
     * @param x The real part of the complex number.
     * @param y The imaginary part of the complex number.
     */
    public Complex(double x, double y) {
        this.x = x;
        this.y = y;
    }

    /**
     * Adds two Complex objects and produces a third object that
     * represents their sum.
     * @param c1 A Complex object
     * @param c2 Another Complex object
     * @return  A new Complex object that represents the sum of
     *          <code>c1</code> and <code>c2</code>.
     * @exception java.lang.NullPointerException
     *            If either argument is <code>null</code>.
     */
```

```
    public static Complex add(Complex c1, Complex c2) {
        return new Complex(c1.x + c2.x, c1.y + c2.y);
    }
}
```

The following is the same class with Markdown doc comments. Note the more concise formatting using _, where the HTML in `<i>complex numbers</i>` becomes simply _complex numbers_, and `<code>c1</code>` becomes just `` `c1` ``:

```
/// This immutable class represents _complex numbers_.
///
/// @author Jason Clark
/// @version 9.0
public class ComplexMarkdown {

    /// Holds the real part of this complex number.
    /// @see #y
    protected double x;

    /// Holds the imaginary part of this complex number.
    /// @see #x
    protected double y;

    /// Creates a new Complex object that represents the complex number
    /// x+yi.
    /// @param x The real part of the complex number.
    /// @param y The imaginary part of the complex number.
    public ComplexMarkdown(double x, double y) {
        this.x = x;
        this.y = y;
    }

    /// Adds two Complex objects and produces a third object that
    /// represents their sum.
    /// @param c1 A Complex object
    /// @param c2 Another Complex object
    /// @return  A new Complex object that represents the sum of
    ///          `c1` and `c2`.
    /// @exception java.lang.NullPointerException
    ///            If either argument is `null`.
    public static ComplexMarkdown add(ComplexMarkdown c1,
                                      ComplexMarkdown c2) {
        return new ComplexMarkdown(c1.x + c2.x, c1.y + c2.y);
    }
}
```

## Structure of a Doc Comment

The body of a doc comment should begin with a one-sentence summary of the type or member being documented. This sentence may be displayed by itself as summary documentation, so it should be written to stand on its own. The initial sentence may be followed by any number of other sentences and paragraphs that describe the class, interface, method, or field in full detail.

After the descriptive paragraphs, a doc comment can contain any number of other paragraphs, each of which begins with a special doc-comment tag, such as `@author`, `@param`, or

`@returns`. These tagged paragraphs provide specific information about the class, interface, method, or field that the `javadoc` program displays in a standard way. The full set of doc-comment tags is listed in “Doc-Comment Tags”.

## Formatting Multiline Doc Comments

`/** ... */` delimited doc comments allow simple HTML markup tags in the descriptive material, such as `<i>` for emphasis; `<code>` for class, method, and field names; and `<pre>` for multiline code examples. It can also contain `<p>` tags to break the description into separate paragraphs and `<ul>`, `<li>`, and related tags to display bulleted lists and similar structures. Remember, however, that the material you write is embedded within a larger, more complex HTML document. For this reason, doc comments should not contain major structural HTML tags, such as `<h2>` or `<hr>`, that might interfere in the larger document.

Avoid the use of the `<a>` tag to include hyperlinks or cross-references in your doc comments. Instead, use the special `{@link}` doc-comment tag, which, unlike the other doc-comment tags, can appear anywhere within a doc comment. As described in “Doc-Comment Tags”, the `{@link}` tag has special support for specifying hyperlinks to other classes, interfaces, methods, and fields without knowing the HTML file naming conventions `javadoc` will use.

If you want to include an image in a doc comment, place the image file in a *doc-files* subdirectory of the source code directory. Give the image the same name as the class, with an integer suffix. For example, the second image that appears in the doc comment for a class named `Circle` can be included with this HTML tag:

```
<img alt="Image alt text" src="doc-files/Circle-2.gif">
```

Because the lines of a doc comment are embedded within a Java comment, any leading spaces and asterisks (`*`) are stripped from each line of the comment before processing. Thus, you don’t need to worry about the asterisks appearing in the generated documentation or about the indentation of the comment affecting the indentation of code examples included within the comment with a `<pre>` tag.

## Formatting Markdown Doc Comments

Markdown provides basic formatting for content with less ceremony than HTML markup. Most Markdown features are available in `///` doc comments, including emphasis via `_italics_`, code formatting with single backticks, and ``` for multiline code examples. Blank lines are used to introduce paragraph breaks as follows:

```
/// Paragraph one starts here.
///
/// Paragraph two carries on from there.
```

Lists are written by prepending `-` to the lines.

```
/// - First item
/// - Second item
/// - Third item
```

Multiline code blocks begin and end with ```. A language identifier can be provided optionally following the backticks for documentation-generating programs such as `javadoc` to consume:

```
/// ```
/// System.out.println("This code")
/// System.out.println("is blocking")
/// ```
///
/// ```bash
/// git add --all
/// git commit -m "yolo"
/// ```
```

As with multiline doc comments, the HTML generated will be embedded in a larger, more complex document, so Markdown heading markers (#, ##, and so forth) should not be used.

Markdown also provides a simpler table syntax inherited from GitHub Flavored Markdown:

```
/// |Method|Visibility|Notes|
/// |---|---|---|
/// |add|public|Just like `+`|
/// |getSecret|private|Shhh, it's secret|
```

As with multiline comments, avoid using `<a>` or Markdown linking (`[text](/destination.html)`) to explicitly cross-reference in your doc comments. `{@link}` is available as in multiline comments. A specialized Markdown link can be used as well, with `[Complex#add()]` being equivalent to `{@link Complex#add()}`. As described in "Doc-Comment Tags", these special link formats allow you to hyperlink other class, interface, method, and field documentation without knowing the precise HTML structure `javadoc` will emit.

Markdown images are handled like those in multiline comments. Place the image file in a *doc-files* subdirectory of the source code directory. Give the image the same name as the class with an integer suffix, and then use the Markdown syntax for images:

```
![Image alt text](doc-files/Circle-2.gif)
```

## Doc-Comment Tags

The `javadoc` program recognizes a number of special tags, each of which begins with an `@` character. These doc-comment tags allow you to encode specific information into your comments in a standardized way, and they allow `javadoc` to choose the appropriate output format for that information. For example, the `@param` tag lets you specify the name and meaning of a single parameter for a method. `javadoc` can extract this information and display it using an HTML `<dl>` list, an HTML `<table>`, or whatever it sees fit.

The following doc-comment tags are recognized by `javadoc`; a doc comment should typically use these tags in the order listed here:

### `@author` `name`

Adds an "Author:" entry that contains the specified name. This tag should be used for every class or interface definition but must not be used for individual methods and fields. If a class has multiple authors, use multiple `@author` tags on adjacent lines. For example:

```
@author David Flanagan
@author Ben Evans
@author Jason Clark
```

List the authors in chronological order, with the original author first. If the author is unknown, you can use "unascribed." `javadoc` does not output authorship information unless the `-author` command-line argument is specified.

### `@version` `text`

Inserts a "Version:" entry that contains the specified text. For example:

```
@version 1.32, 2004-08-26
```

This tag should be included in every class and interface doc comment but cannot be used for individual methods and fields. This tag is often used in conjunction with the automated version-numbering capabilities of a version control system, such as git, Perforce, or SVN. `javadoc` does not output version information in its generated documentation unless the `-version` command-line argument is specified.

### `@param` `parameter-name` `description`

Adds the specified parameter and its description to the "Parameters:" section of the current method. The doc comment for a method or constructor must contain one `@param` tag for each parameter the method expects. These tags should appear in the same order as the parameters specified by the method. The tag can be used only in doc comments for methods and constructors.

You are encouraged to use phrases and sentence fragments where possible to keep the descriptions brief. However, if a parameter requires detailed documentation, the description can wrap onto multiple lines and include as much text as necessary. For readability in source-code form, consider using spaces to align the descriptions with one another. For example:

```
@param o      the object to insert
```

```
@param index  the position at which to insert it
```

### `@return description`

Inserts a “Returns:” section that contains the specified description. This tag should appear in every doc comment for a method, unless the method returns `void` or is a constructor. The description can be as long as necessary, but consider using a sentence fragment to keep it short. For example:

```
@return <code>true</code> if the insertion is successful, or
        <code>false</code> if the list already contains the object.
```

### `@exception full-classname description`

Adds a “Throws:” entry that contains the specified exception name and description. A doc comment for a method or constructor should contain an `@exception` tag for every checked exception that appears in its `throws` clause—and these can usually be automatically generated. For example:

```
@exception java.io.FileNotFoundException
           If the specified file could not be found
```

The `@exception` tag can optionally be used to document unchecked exceptions (i.e., subclasses of `RuntimeException`) the method may throw, when these are exceptions that a user of the method may reasonably want to catch. If a method can throw more than one exception, use multiple `@exception` tags on adjacent lines and list the exceptions in alphabetical order. The description can be as short or as long as necessary to describe the significance of the exception. This tag can be used only for method and constructor comments.

### `@throws full-classname description`

This tag is a synonym for `@exception`.

### `@see reference`

Adds a “See Also:” entry that contains the specified reference. This tag can appear in any kind of doc comment. The syntax for the *`reference`* is explained in “Cross-References in Doc Comments”.

### `@deprecated explanation`

This tag specifies that the following type or member has been deprecated and that its use

should be avoided. `javadoc` adds a prominent “Deprecated” entry to the documentation and includes the provided *`explanation`* text. This text should specify when the class or member was deprecated and, if possible, suggest a replacement class or member and include a link to it. For example:

```
@deprecated As of Version 3.0, this method is replaced
            by {@link #setColor}.
```

The `@deprecated` tag is an exception to the general rule that `javac` ignores all comments. When this tag appears, the compiler notes the deprecation in the class file it produces. This allows it to issue warnings for other classes that rely on the deprecated feature.

### `@since version`

Specifies when the type or member was added to the API. This tag should be followed by a version number or other version specification. For example:

```
@since JNUT 3.0
```

Every doc comment for a type should include an `@since` tag, and any members added after the initial release of the type should have `@since` tags in their doc comments.

### `@serial description`

Technically, the way a class is serialized is part of its public API. If you write a class that you expect to be serialized, you should document its serialization format using `@serial` and the related tags listed next. `@serial` should appear in the doc comment for any field that is part of the serialized state of a `Serializable` class.

For classes that use the default serialization mechanism, this means all fields that are not declared `transient`, including fields declared `private`. The *`description`* should be a brief description of the field and of its purpose within a serialized object.

You can also use the `@serial` tag at the class and package level to specify whether a “serialized form page” should be generated for the class or package. The syntax is:

```
@serial include
@serial exclude
```

### `@serialField name type description`

A `Serializable` class can define its serialized format by declaring an array of `ObjectStreamField` objects in a field named `serialPersistentFields`. For such

a class, the doc comment for `serialPersistentFields` should include an `@serialField` tag for each element of the array. Each tag specifies the name, type, and description for a particular field in the serialized state of the class.

`@serialData description`

A `Serializable` class can define a `writeObject()` method to write data other than that written by the default serialization mechanism. An `Externalizable` class defines a `writeExternal()` method responsible for writing the complete state of an object to the serialization stream. The `@serialData` tag should be used in the doc comments for these `writeObject()` and `writeExternal()` methods, and the *`description`* should document the serialization format used by the method.

## Inline Doc-Comment Tags

In addition to the preceding tags, `javadoc` also supports several *inline tags* that may appear anywhere that HTML or Markdown appears in a doc comment. Because these tags appear directly within the text, they require curly braces as delimiters to separate the tagged text from the HTML or Markdown text. Supported inline tags include the following:

`{@link reference }`

The `{@link}` tag is like the `@see` tag except that instead of placing a link to the specified *`reference`* in a special "See Also:" section, it inserts the link inline. A `{@link}` tag can appear anywhere that text appears in a doc comment. In other words, it can appear in the initial description of the class, interface, method, or field and in the descriptions associated with the `@param`, `@returns`, `@exception`, and `@deprecated` tags. The *`reference`* for the `{@link}` tag uses the syntax described next in "Cross-References in Doc Comments". For example:

```
@param regexp The regular expression to search for. This string
              argument must follow the syntax rules described for
              {@link java.util.regex.Pattern}.
```

`{@linkplain reference }`

The `{@linkplain}` tag is just like the `{@link}` tag, except that the text of the link is formatted using the normal font rather than the code font used by the `{@link}` tag. This is most useful when *`reference`* contains both a *`feature`* to link to and a *`label`* that specifies alternate text to be displayed in the link. See "Cross-References in Doc Comments" for more on the *`feature`* and *`label`* portions of the *`reference`* argument.

`{@inheritDoc}`

When a method overrides a method in a superclass or implements a method in an interface, you can omit a doc comment and `javadoc` automatically inherits the documentation from

the overridden or implemented method. You can use the `{@inheritDoc}` tag to inherit the text of individual tags. This tag also allows you to inherit and augment the descriptive text of the comment. To inherit individual tags, use it like this:

```
@param index {@inheritDoc}
@return {@inheritDoc}
```

### `{@docRoot}`

This inline tag takes no parameters and is replaced with a reference to the root directory of the generated documentation. It is useful in hyperlinks that refer to an external file, such as an image or a copyright statement:

```
<img src="{@docroot}/images/logo.gif">
This is [Copyrighted]({@docRoot}/legal.html) material.
```

### `{@literal text }`

This inline tag displays *`text`* literally, escaping any HTML in it and ignoring any `javadoc` tags it may contain. It does not retain whitespace formatting but is useful when used within a `<pre>` tag.

### `{@code text }`

This tag is like the `{@literal}` tag but displays the literal *`text`* in code font. Equivalent to:

```
<code>{@literal <replaceable>text</replaceable>}</code>
```

In Markdown comments, enclosing text in backticks is equivalent to an `@code` tag.

### `{@snippet : text }`

The `snippet` tag allows for better handling of code blocks, especially those that are multiline. Text follows a `:` and newline and is included up to a closing `}`:

```
{@snippet :
  System.out.println("Hello world");
  System.out.println("Hello Java");
}
```

### `{@snippet attribute = value : text }`

The variant of `snippet` allows for additional attribute-value pairs that can be used by

document-generating tooling to interpret the contents of the snippet.

One example of this tag supported by `javadoc` is pulling in content from a file instead of from the inline content of the `snippet`:

```
{@snippet file="Example.java" region="ctor"}
```

When paired with a file containing the following, the documentation will show the constructor of the class in the highlighted region:

```
public class Example {
  // @start region="ctor"
  Example(String value) {
    this.value = value;
  }
  // @end
}
```

`{@value}`

The `{@value}` tag, with no arguments, is used inline in doc comments for `static final` fields and is replaced with the constant value of that field.

`{@value reference }`

This variant of the `{@value}` tag includes a *`reference`* to a `static final` field and is replaced with the constant value of that field.

## Cross-References in Doc Comments

The `@see` tag and the inline tags `{@link}`, `{@linkplain}`, and `{@value}` all encode a cross-reference to some other source of documentation, typically to the documentation comment for some other type or member.

*`reference`* can take three forms. If it begins with a quote character, it is taken to be the name of a book or some other printed resource and is displayed as is:

```
@see "Java in a Nutshell, 9th Edition"
```

If *`reference`* begins with a `<` character, it is taken to be an arbitrary HTML hyperlink that uses the `<a>` tag, and the hyperlink is inserted into the output documentation as is. This form of the `@see` tag can insert links to other online documents, such as a programmer's guide or user's manual:

```
@see <a href="https://github.com/kittylyst/javanut9">Code on GitHub</a>
```

If *`reference`* is not a quoted string or a hyperlink, it is expected to have the following form:

```
feature [label]
```

In this case, `javadoc` outputs the text specified by *`label`* and encodes it as a hyperlink to the specified *`feature`*. If *`label`* is omitted (as it usually is), `javadoc` uses the name of the specified *`feature`* instead.

*`feature`* can refer to a package, type, or type member using one of the following forms:

`pkgname`

A reference to the named package. For example:

```
@see java.lang.reflect
```

`pkgname.typename`

A reference to a class, interface, enumerated type, or annotation type specified with its full package name. For example:

```
@see java.util.List
```

`typename`

A reference to a type specified without its package name. For example:

```
@see List
```

`javadoc` resolves this reference by searching the current package and the list of imported classes for a class with this name.

`typename # methodname`

A reference to a named method or constructor within the specified type. For example:

```
@see java.io.InputStream#reset
@see InputStream#close
@see Complex#Complex
```

If the type is specified without its package name, it is resolved as described for *`typename`*.

This syntax is ambiguous if the method is overloaded or the class defines a field by the same name.

### `typename # methodname ( paramtypes )`

A reference to a method or constructor with the type of its parameters explicitly specified. This is useful when cross-referencing an overloaded method. For example:

```
@see InputStream#read(byte[], int, int)
@see Complex#Complex(double, double)
```

### `# methodname`

A reference to a nonoverloaded method or constructor in the current class or interface or one of the containing classes, superclasses, or superinterfaces of the current class or interface. Use this concise form to refer to other methods in the same class. For example:

```
@see #setBackgroundColor
@see #Complex
```

### `# methodname ( paramtypes )`

A reference to a method or constructor in the current class or interface or one of its superclasses or containing classes. This form works with overloaded methods because it lists the types of the method parameters explicitly. For example:

```
@see #setPosition(int, int)
@see #Complex(double, double)
```

### `typename # fieldname`

A reference to a named field within the specified class. For example:

```
@see java.io.BufferedInputStream#buf
```

If the type is specified without its package name, it is resolved as described for *`typename`*.

### `# fieldname`

A reference to a field in the current type or one of the containing classes, superclasses, or superinterfaces of the current type. For example:

```
@see #x
```

### Doc Comments for Packages

Documentation comments for classes, interfaces, methods, constructors, and fields appear in Java source code immediately before the definitions of the features they document. `javadoc` can also read and display summary documentation for packages. Because a package is defined in a directory, not in a single file of source code, `javadoc` looks for the package documentation in the directory that contains the source code for the classes of the package.

Package documentation, including package comments, can be placed in one of two files:

*package-info.java*

> This file can contain the package declaration, package annotations, package comments, and Javadoc tags. This file is preferred in modern Java.

*package.html*

> This file contains only package comments and Javadoc tags and does not permit package annotations.

Note that a package can have a *package.html* file or a *package-info.java* file but not both.

The package documentation file should contain simple HTML documentation for the package. It can also contain `@see`, `@link`, `@deprecated`, `@snippet`, and `@since` tags. Because *package.html* is not a file of Java source code, the documentation it contains should be HTML and should *not* be a Java comment (i.e., it should not be enclosed within `/**` and `*/` or `///` characters). Finally, any `@see` and `@link` tags that appear in *package.html* must use fully qualified class names.

In addition to defining a package documentation file for each package, you can also provide high-level documentation for a group of packages by defining an *overview.html* file in the source tree for those packages. When `javadoc` is run over that source tree, it uses *overview.html* as the highest-level overview it displays.

## Doclets

The `javadoc` tool that is used to generate HTML documentation is based upon a standard API. Since Java 9, this standard interface has been delivered in the module `jdk.javadoc`, and tools leveraging this API are typically called *doclets* (with `javadoc` being referred to as the standard doclet).

The standard doclet generates modern HTML5 by default, including implementing the WAI-ARIA standard for accessibility. This standard makes it easier for people with visual or other impairments to access `javadoc` output using tools such as screen readers.

> **NOTE**
>
> `javadoc` has also been enhanced to understand the new platform modules, and so the semantic meaning of what constitutes an API (and so what should be documented) is now aligned with the modular Java definition.

The standard doclet now also automatically indexes the code as documentation is generated and creates a client-side index in JavaScript. The resulting web pages have a search capability to allow developers to easily find some common program components, such as the names of:

- Modules
- Packages
- Types and members
- Method parameter types

The developer can also add search terms or phrases using an `@index` inline `javadoc` tag.

## Conventions for Portable Programs

One of the earliest slogans for Java was "Write once, run anywhere." This emphasizes that Java makes it easy to write portable programs, but it is still possible to write Java programs that do not automatically run successfully on any Java platform.

When writing code, you should pay attention to whether or not it needs to run across platforms. End-user applications or services may have just a single deployment intention, and that's absolutely fine—portability is not a concern for those codebases.

At the other extreme, reusable open source libraries should take into account the diversity of environments in which they may operate, and portability should be considered a first-class concern for such code.

The following tips help developers to avoid portability problems:

*Native methods*

Portable Java code can use any methods in the core Java APIs, including methods implemented as `native` methods. However, portable code must not define its own native methods. By their very nature, native methods must be ported to each new platform, so they directly subvert the "Write once, run anywhere" promise of Java.

*The* `Runtime.exec()` *method*

Calling the `Runtime.exec()` method to spawn a process and execute an external command on the native system is rarely allowed in portable code. This is because the native OS command to be executed is never guaranteed to exist or behave the same way on all platforms.

The only time it is legal to use `Runtime.exec()` in portable code is when the user is allowed to specify the command to run, either by typing the command at runtime or by specifying the command in a configuration file or preferences dialog box.

If the programmer wishes to control external processes, then this should be done through the enhanced `ProcessHandle` capability introduced in Java 9, rather than by using `Runtime.exec()` and parsing the output. This is not fully portable, but it at least reduces the amount of platform-specific logic necessary to control external processes.

*The* `System.getenv()` *method*

Different operating systems have differing casing conventions (e.g., Unix systems are case-sensitive, whereas Windows is not). Also, typical values found in an environment vary greatly between operating systems and organizations. Use of `System.getenv()` to parameterize specific values your application expects can be acceptable if well documented; this is frequently done with containerized applications. But reaching out to the broader environment can yield incompatible behavior.

*Undocumented classes*

Portable Java code must use only classes and interfaces that are a documented part of the Java platform. Most Java implementations ship with additional undocumented public classes that are part of the implementation but not part of the Java platform specification.

The modules system prevents a program from using and relying on these implementation classes, but even with the increased restrictions in Java 17 it is still possible to circumvent this protection by using reflection (although the exact runtime switches permitting reflection have changed in recent versions; see Chapter 14 for more details).

However, doing so is not portable because the implementation classes are not guaranteed to exist in all Java implementations or on all platforms, and they may change or disappear in future versions. Even if you don't care much about portability, use of undocumented classes can greatly complicate future JDK version upgrades.

Of particular note is the `sun.misc.Unsafe` class, which provides access to a number of "unsafe" methods, which can allow developers to circumvent key restrictions of the Java platform. Developers should not directly use the `Unsafe` class under any circumstances.

*Implementation-specific features*

Portable code must not rely on features specific to a single implementation. For example, in the early years of Java, Microsoft distributed a version of the Java runtime system that included a number of additional methods that were not part of the Java platform as defined by the specifications. Any program that depends on such extensions is obviously not portable to other platforms.

*Implementation-specific bugs*

Just as portable code must not depend on implementation-specific features, it must not depend on implementation-specific bugs. If a class or method behaves differently than the specification says it should, a portable program cannot rely on this behavior, which may be different on different platforms, and a future version may ultimately fix the bug, hindering JDK upgrades.

*Implementation-specific behavior*

Sometimes different platforms and different implementations present different behaviors, all of which are legal according to the Java specification. Portable code must not depend on any one specific behavior. For example, the Java specification does not indicate whether threads of equal priority share the CPU or if one long-running thread can starve another thread at the same priority. If an application assumes one behavior or the other, it may not run properly on all platforms.

*Defining system classes*

Portable Java code never attempts to define classes in any of the system or standard extension packages. Doing so violates the protection boundaries of those packages and exposes package-visible implementation details, even in those cases where it is not forbidden by the modules system.

*Hardcoded filenames*

A portable program contains no hardcoded file or directory names. This is because different platforms have significantly different filesystem organizations and use different directory separator characters. If you need to work with a file or directory, have the user specify the filename, or at least the base directory beneath which the file can be found. This specification can be done at runtime, in a configuration file, or as a command-line argument to the program. When concatenating a file or directory name to a directory name, use the `Path.of()` method, the `File()` constructor, or the `File.separator` constant.

*Line separators*

Different systems use different characters or sequences of characters as line separators. Do not hardcode `\n`, `\r`, or `\r\n` as the line separator in your program. Instead, use the `println()` method of `PrintStream` or `PrintWriter`, which automatically terminates a line with the line separator appropriate for the platform, or use the value of the `line.separator` system property. You can also use the `"%n"` format string with the `printf()` and `format()` methods of `java.util.Formatter` and related classes.

When writing open source libraries, portability can be an important factor in adoption by others in the ecosystem. If `MyGreatLibrary` on GitHub has incomplete or poor documentation, it is less likely to be used by others.

## Summary

In this chapter, we've seen the standard conventions around naming parts of our Java code. While the language allows many things beyond these conventions, your code will be easier for others to read and understand the more these are followed.

Good documentation is at the heart of creating maintainable systems. The `javadoc` tool allows us to write much of our documentation within our code, keeping it in context when things change. A variety of document tags allow for generating clear and consistent documentation.

Part of the appeal of the JVM is its broad install base across many operating systems and types of hardware. However, you can compromise the portability of your application if you're not careful in a few areas, so this chapter reviewed guidelines around the most typical of those stumbling blocks to avoid.

Next up, we'll take a look at one of the most commonly used parts of Java's standard libraries: the Java Collections.

# Chapter 8. Working with Java Collections

This chapter introduces Java's interpretation of fundamental data structures, known as the Java Collections. These abstractions are core to many (if not most) programming types and form an essential part of any programmer's basic toolkit. Accordingly, this is one of the most important chapters of the entire book and provides functionality that is essential to virtually all Java programmers.

In this chapter, we will introduce the fundamental interfaces and the type hierarchy, show how to use them, and discuss aspects of their overall design. Both the "classic" approach to handling the collections and the newer approach based on the Java Streams API will be covered.

## Introduction to Collections API

The Java Collections are a set of generic interfaces that describe the most common forms of data structure. Java ships with several implementations of each of the classic data structures, and because the types are represented as interfaces, development teams can, and have, developed their own, specialized implementations of the interfaces for use in their own projects or to share as libraries.

The Java Collections define two fundamental types of data structures. A `Collection` is a grouping of objects, while a `Map` is a set of mappings, or associations, between objects. The basic layout of the Java Collections is shown in Figure 8-1.

Note that there are two separate, disjoint type hierarchies represented in the diagram —`Collection` and `Map` do not have a common superinterface.

Within this basic setup, a `Set` is a type of `Collection` with no duplicates, and a `List` is a `Collection` in which the elements are ordered but may contain duplicates.

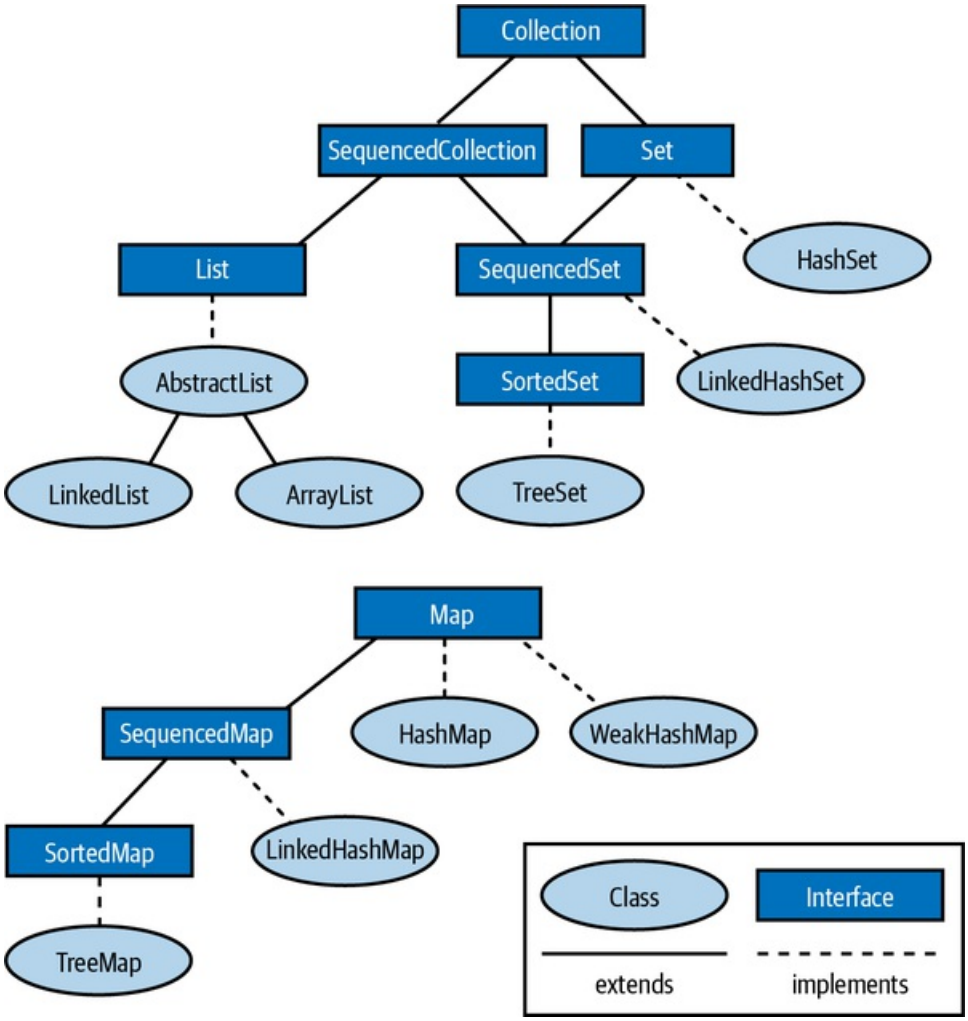



*Figure 8-1. Collections classes and inheritance*

Data structures possessing an order to their elements implement `SequencedCol lection`, `SequencedSet`, and `SequencedMap`, which standardize accessing the beginning and end of the collections via the `getFirst` and `getLast` methods. `SortedSet` and `SortedMap` are specialized sets and maps that maintain their elements in a sorted order.

`Collection`, `Set`, `List`, `Map`, `SortedSet`, and `SortedMap` are all interfaces, but the `java.util` package also defines various concrete implementations, such as lists based on arrays and linked lists, and maps and sets based on hash tables or binary trees. Other important interfaces are `Iterator` and `Iterable`, which allow you to loop through the objects in a collection, as we will see later on.

## Creating Collections

In many other languages, particularly Python and JavaScript, it is common for collections handling to have special syntax defined by the language. While Java does have array syntax built in, all other collections are strictly implemented and accessed through normal Java interfaces, classes and methods.

Given this, it is common to obtain a collection via the `new` keyword with a concrete class that implements one of the `Collection` interfaces we'll take a tour through next:

```
List<String> list = new ArrayList<>();
Map<Integer, String> map = new HashMap<>();
Set<Long> set = new HashSet<>();
```

We'll discuss in detail how to choose between the different concrete classes available in the Java Collection Framework in each of the following sections about the major interfaces.

While these constructors can be initialized with another full collection, they don't provide an easy way to list out individual elements, leading to an annoying pattern of having to instantiate the collection then repeatedly `add()` to it afterward on separate lines. To help address this friction, recent versions of Java have added an `of` method to the major interfaces for creating a collection from its constituent elements:

```
List<String> list = List.of("test", "strings");
Map<Integer, String> map = Map.of(1, "one", 2, "two");
Set<Long> set = Set.of(10L, 20L, 30L);
```

There are a couple of facts about the `of` methods to be aware of. The first is that they don't accept `null` as an entry and will throw a `NullPointerException` if one is passed.

Another restriction is that all collections returned from `of` methods are *immutable*, meaning that elements cannot be added or removed after the collection is created. For that reason, it may not be possible to swap arbitrary collection creation directly for an `of` method. Consider whether the code can be updated to be immutable, as it can be a desirable property on its own, as discussed in "Mutability and Immutability".

If a mutable collection is still required, it is often possible to tighten up the construction with `of` methods at the cost of an additional object allocation:

```
ArrayList<String> tidy = new ArrayList<>(List.of("test", "strings"));

// versus

ArrayList<String> wordy = new ArrayList<>();
wordy.add("test");
wordy.add("strings");
```

Now that we've created a collection, let's look closer at what we can do with it.

## The Collection Interface

`Collection<E>` is a parameterized interface that represents a generalized grouping of objects of type `E`. We can create a collection of any kind of reference type, but we cannot create collections of primitive types.[1]

> **NOTE**
>
> For your classes to work properly with the expectations of collections, you must take care when defining `hashCode()` and `equals()` methods on your classes, as discussed in Chapter 6.

Methods are defined for adding and removing objects from the group, testing an object for membership in the group, and iterating through all elements in the group. Additional methods return the elements of the group as an array and return the size of the collection.

> **NOTE**
>
> The grouping within a `Collection` may or may not allow duplicate elements and may or may not impose an ordering on the elements.

The Java Collections Framework provides `Collection` because it defines the features shared by many common forms of data structure. The JDK ships `Set`, `List`, and `Queue` as subinterfaces of `Collection`. (We'll meet `Queue` in Chapter 12 as it is commonly used for many concurrent applications).

The following code illustrates the operations you can perform on `Collection` objects:

```
// Create some collections to work with.
Collection<String> c = new HashSet<>();  // An empty set

// We'll see these utility methods later. Be aware that there are
// some subtleties to watch out for when using them
Collection<String> d = Arrays.asList("one", "two");
Collection<String> e = Collections.singleton("three");

// Add elements to a collection. These methods return true
// if the collection changes, which is useful with Sets which
// don't allow duplicates.
c.add("zero");           // Add a single element
c.addAll(d);             // Add all of the elements in d

// Copy a collection: most implementations have a copy constructor
Collection<String> copy = new ArrayList<String>(c);

// Remove elements from a collection.
// All but clear return true if the collection changes.
c.remove("zero");        // Remove first occurrence of an element
c.removeAll(e);          // Remove a collection of elements
c.retainAll(d);          // Remove all elements that are not in d
c.clear();               // Remove all elements from the collection

// Querying collection size
boolean b = c.isEmpty(); // c is now empty, so true
int s = c.size();        // Size of c is now 0.

// Restore collection from the copy we made
c.addAll(copy);
```

```
// Test membership in the collection. Membership is based on
// the equals method, not the == operator.
b = c.contains("zero");  // true
b = c.containsAll(d);    // true

// Most Collection implementations have a useful toString()  method
// In this example it prints "[zero, one, two]"
System.out.println(c);

// Obtain an array of collection elements.  If the iterator guarantees
// an order, this array has the same order. The Object array is a new
// instance, containing references to the same objects as the original
// collection `c` (aka a shallow copy).
Object[] elements = c.toArray();

// If we want the elements in a String[], we must pass one in
String[] strings = c.toArray(new String[c.size()]);

// Or we can pass an empty String[] just to specify the type and
// the toArray method will allocate an array for us
strings = c.toArray(new String[0]);
```

Remember that you can use any of the methods shown here with any `Set`, `List`, or `Queue`. These subinterfaces may impose membership restrictions or ordering constraints on the elements of the collection but still provide the same basic methods.

> **NOTE**
>
> Methods such as `addAll()`, `retainAll()`, `clear()`, and `remove()` that alter the collection were conceived of as optional parts of the API. Unfortunately, they were specified a long time ago, before Java had default methods. At that time the received wisdom was to indicate the absence of an optional method by throwing `UnsupportedOperationException`. Accordingly, some implementations (notably read-only forms) may throw this unchecked exception.

`Collection`, `Map`, and their subinterfaces do *not* extend the interfaces `Cloneable` or `Serializable`. All of the collection and map implementation classes provided in the Java Collections Framework, however, do implement these interfaces.

Some collection implementations place restrictions on the elements that they can contain. An implementation might prohibit `null` as an element, for example. And `EnumSet` restricts membership to the values of a specified enumerated type.

Attempting to add a prohibited element to a collection always throws an unchecked exception such as `NullPointerException` or `ClassCastException`. Checking whether a collection contains a prohibited element may also throw such an exception, or it may simply return `false`.

Now let’s take a deeper look at the common interfaces in the Java Collections Framework.

## The List Interface

A `List` is an ordered collection of objects. Each element of a list has a position in the list, and the `List` interface defines methods to query or set the element at a particular position, or *index*. In this respect, a `List` is like an array whose size changes as needed to accommodate the number of elements it contains. Lists allow duplicate elements.

In addition to its index-based `get()` and `set()` methods, the `List` interface defines methods to add or remove an element at a particular index, and also defines methods to return the index of the first or last occurrence of a particular value in the list. The `add()` and `remove()` methods inherited from `Collection` append to the list and remove the first occurrence of the specified value from the list. The inherited `addAll()` appends all elements in the specified collection to the end of the list, and another version inserts the elements at a specified index. The `retainAll()` and `removeAll()` methods behave as they do for any `Collection`, retaining or removing multiple occurrences of the same value, if needed.

For those coming from other languages like Python and JavaScript, some implementations of `List` in the Java Collection Framework are likely to feel more familiar than Java's statically sized arrays. Although in Java you can't interact with `List` objects via `[]` syntax, you'll find the common operations you'd expect are available.

The `List` interface doesn't define methods that operate on a range of list indexes. Instead, it defines a single `subList()` method that returns a `List` object that represents just the specified range of the original list. The sublist is backed by the parent list, and any changes made to the sublist are immediately visible in the parent list. Examples of `subList()` and the other basic `List` manipulation methods follow:

```
// Create lists to work with
List<String> l = new ArrayList<String>(Arrays.asList(args));
List<String> words = Arrays.asList("hello", "world");
List<String> words2 = List.of("hello", "world");

// Querying and setting elements by index
String first = l.get(0);             // First element of list
String last = l.get(l.size() - 1);   // Last element of list
l.set(0, last);                      // The last shall be first

// Adding and inserting elements.  add  can append or insert
l.add(first);       // Append the first word at end of list
l.add(0, first);    // Insert first at the start of the list again
l.addAll(words);    // Append a collection at the end of the list
l.addAll(1, words); // Insert collection after first word

// Sublists: backed by the original list
List<String> sub = l.subList(1, 3);  // second and third elements
sub.set(0, "hi");                   // modifies 2nd element of l

// Sublists can restrict operations to a subrange of backing list
String s = Collections.min(l.subList(0, 4));
Collections.sort(l.subList(0, 4));
```

```
// Independent copies of a sublist don't affect the parent list.
List<String> subcopy = new ArrayList<String>(l.subList(1, 3));
subcopy.clear();

// Searching lists
int p = l.indexOf(last);  // Where does the last word appear?
p = l.lastIndexOf(last);  // Search backward

// Removing elements from a list
l.remove(last);          // Remove first occurrence of the element
l.remove(0);             // Remove element at specified index
l.subList(0, 2).clear(); // Remove a range of elements using subList
l.retainAll(words);      // Remove all but elements in words
l.removeAll(words);      // Remove all occurrences of elements in words
l.clear();               // Remove everything
```

Because these `sublist` modifications alter the parent collection, they require that the parent be mutable. For an immutable collection such as one returned by `List.of`, an `UnsupportedOperationException` will be thrown when attempting to alter the parent through the sublist.

### Iteration with foreach loops

One very important way of working with collections is to process each element in turn, an approach known as *iteration*. We'll see other, more modern approaches to iteration in "Java Streams and Lambda Expressions", but basic iteration fits naturally with the `for` loop, as shown in this bit of code, and is easiest to illustrate using a `List`:

```
List<String> c = new ArrayList<String>();
// ... add some Strings to c

for (String word : c) {
    System.out.println(word);
}
```

The sense of the code should be clear—it takes the elements of `c` one at a time and uses them as a variable in the loop body. We've already met the *foreach loop* in "The foreach Variant".

Here are some more things you should know about the syntax of the foreach loop:

- As noted earlier, *`expression`* must be either an array or an object that implements the `java.lang.Iterable` interface. This type must be known at compile time so that the compiler can generate appropriate looping code.
- The type of the array or `Iterable` elements must be assignment-compatible with the type of the variable declared in the *`declaration`*. For example, to iterate through the elements of a `List<String>`, the variable must be declared `String` or its superclass `Object`, or one of the interfaces it implements: `CharSe quence`, `Comparable`, or `Serializable`. If you use an `Iterable` object that is not parameterized with an element type, the variable must be declared as an `Object`.

- The *`declaration`* usually consists of just a type and a variable name, but it may include a `final` modifier and any appropriate annotations (see Chapter 5). Using `final` prevents the loop variable from being reassigned within the loop.
- The loop variable of the foreach loop must be declared as part of the loop, with both a type (or `var`) and a variable name. You cannot use a variable declared outside the loop as you can with the `for` loop.

In "The Iterable and Iterator interfaces", we'll look closer at the mechanisms used to implement foreach with the `java.util.Iterable` and `java.util.Iterator` interfaces, but in the meantime let's talk about other ways of accessing lists.

### Random access to lists

A general expectation of `List` implementations is that they can be efficiently iterated, typically in time proportional to the size of the list. However, not all lists provide efficient random access to the elements at any index. Sequential-access lists, such as the `LinkedList` class, provide efficient insertion and deletion operations at the expense of random-access performance.

Implementations that provide efficient random access implement the `RandomAccess` marker interface, and you can test for this interface with `instanceof` if you need to ensure efficient list manipulations:

```
// Arbitrary list we're passed to manipulate
List<?> l = ...;

// Ensure we can do efficient random access.  If not, use a copy
// constructor to make a random-access copy of the list before
// manipulating it.
if (!(l instanceof RandomAccess)) l = new ArrayList<?>(l);
```

To iterate just a portion of a list, you can use the `subList()` method to create a sublist view:

```
List<String> words = ...;  // Get a list to iterate

// Iterate all elements of the list but the first
for(String word : words.subList(1, words.size()))
    System.out.println(word);
```

Table 8-1 summarizes the five general-purpose `List` implementations in the Java platform. `Vector` and `Stack` are legacy implementations and should not be used. `CopyOnWriteArrayList` is part of the `java.util.concurrent` package and is only really suitable for multithreaded use cases.

*Table 8-1. List implementations*

| Class | Representation | Random access | Notes |
|---|---|---|---|

| | | | |
|---|---|---|---|
| `ArrayList` | Array | Yes | Best all-around implementation. |
| `LinkedList` | Double-linked list | No | More efficient insertion and deletion in middle of list. |
| `CopyOnWriteArrayList` | Array | Yes | Threadsafe; fast traversal, slow modification. |
| `Vector` | Array | Yes | Legacy class; synchronized methods. Do not use. |
| `Stack` | Array | Yes | Extends `Vector`; adds `push()`, `pop()`, `peek()`. Legacy; use `Deque` instead. |

## The Map Interface

A *map* is a set of *key* objects and a mapping from each member of that set to a *value* object. The `Map` interface defines an API for defining and querying mappings. `Map` is part of the Java Collections Framework, but it does not extend the `Collection` interface, so a `Map` is a little-*c* collection, not a big-*C* `Collection`. `Map` is a parameterized type with two type variables, `Map<K, V>`. Type variable `K` represents the type of keys held by the map, and type variable `V` represents the type of the values that the keys are mapped to. A mapping from `String` keys to `Integer` values, for example, can be represented with a `Map<String, Integer>`.

The most important `Map` methods are `put()`, which defines a key-value pair in the map; `get()`, which queries the value associated with a specified key; and `remove()`, which removes the specified key and its associated value from the map. The general performance expectation for `Map` implementations is that these three basic methods are quite efficient: they should run in constant time and certainly no worse than in logarithmic time.

Those coming from Python will have seen similar data structures referred to as *dictionaries*. JavaScript's flexible base object type fulfills much of the same need for mapping. Many of the scripting languages allow accessing their map equivalent via `[]`, but in Java, access through the `Map` interface is via methods rather than those sorts of operators.

An important feature of `Map` is its support for "collection views." These can be summarized as:

- A `Map` is not a `Collection`.
- The keys of a `Map` can be viewed as a `Set`.
- The values can be viewed as a `Collection`.
- The mappings can be viewed as a `Set` of `Map.Entry` objects.

> **NOTE**
>
> `Map.Entry` is a nested interface defined within `Map`. It simply represents a single key-value pair.

The following sample code shows the `get()`, `put()`, `remove()`, and other methods of a `Map` and demonstrates some common uses of the collection views of a `Map`:

```
// New, empty map
Map<String, Integer> m = new HashMap<>();
Map<String, Integer> another = Map.of("another-key", -1);

// Populate the map using the put method to define mappings
// from array elements to the index at which each element appears
String[] words = { "this", "is", "a", "test" };
for(int i = 0; i < words.length; i++) {
    m.put(words[i], i);  // Note autoboxing of int to Integer
}

// Each key must map to a single value.
// Multiple keys may map to the same value.
for(int i = 0; i < words.length; i++) {
    m.put(words[i].toUpperCase(), i);
}

// The putAll() method copies mappings from another Map
m.putAll(another);

// Query the mappings with the get()  method
for(int i = 0; i < words.length; i++) {
    if (m.get(words[i]) != i) throw new AssertionError();
}

// Key and value membership testing
m.containsKey(words[0]);        // true
m.containsValue(words.length);  // false

// Map keys, values, and entries can be viewed as collections
Set<String> keys = m.keySet();
Collection<Integer> values = m.values();
Set<Map.Entry<String, Integer>> entries = m.entrySet();

// The Map and its collection views typically have useful
// toString  methods
System.out.printf("Map: %s%nKeys: %s%nValues: %s%nEntries: %s%n",
                  m, keys, values, entries);

// These collections can be iterated.
// Most maps have an undefined iteration order (but see SortedMap)
for(String key : m.keySet()) System.out.println(key);
for(Integer value: m.values()) System.out.println(value);

// The Map.Entry<K, V> type represents a single key-value pair in a map
for(Map.Entry<String, Integer> pair : m.entrySet()) {
    // Print out mappings
    System.out.printf("'%s' ==> %d%n", pair.getKey(), pair.getValue());
    // And increment the value of each Entry
```

```
    pair.setValue(pair.getValue() + 1);
}

// Removing mappings
m.put("testing", null);   // Mapping to null can "erase" a mapping:
m.get("testing");         // Returns null
m.containsKey("testing"); // Returns true: mapping still exists
m.remove("testing");      // Deletes the mapping altogether
m.get("testing");         // Still returns null
m.containsKey("testing"); // Now returns false.

// Deletions may also be made via the collection views of a map.
// Additions to the map may not be made this way, however.
m.keySet().remove(words[0]);  // Same as m.remove(words[0]);

// Removes one mapping to the value 2 - usually inefficient and of
// limited use
m.values().remove(2);
// Remove all mappings to 4
m.values().removeAll(List.of(4));
// Keep only mappings to 2 & 3
m.values().retainAll(List.of(2, 3));

// Deletions can also be done via iterators
Iterator<Map.Entry<String, Integer>> iter = m.entrySet().iterator();
while(iter.hasNext()) {
    Map.Entry<String,Integer> e = iter.next();
    if (e.getValue() == 2) iter.remove();
}

// Find values that appear in both of two maps.  In general, addAll()
// and retainAll() with keySet() and values() allow union and
// intersection
Set<Integer> v = new HashSet<>(m.values());
v.retainAll(another.values());

// Miscellaneous methods
m.clear();                // Deletes all mappings
m.size();                 // Returns number of mappings: currently 0
m.isEmpty();              // Returns true
m.equals(Map.of());       // true: Maps implementations override equals
```

As of Java 9, the `Map` interface was enhanced with factory methods for spinning up collections easily:

```
Map<String, Double> cities =
        Map.of(
          "Barcelona", 22.5,
          "New York", 28.3);
```

`Map.of()` is a little more complicated than `List.of()`, which we've met, because it has two types, keys and values, but Java allows only one varargs parameter in a method declaration. The solution is to have fixed argument size overloads, up to 10 entries and also to provide a new static method, `entry()`, that will construct an object to represent the key-value pair.

The code can then be written to use the varargs form, like this:

```
Map<String, Double> cities =
        Map.ofEntries(
          entry("Barcelona", 22.5),
          entry("New York", 28.3));
```

Note that the method name has to be different from `of()` due to the difference in type of the arguments—this is a varargs method of type `Map.Entry`. This difference is an example of a genuine problem caused by type erasure.

The `Map` interface includes a variety of general-purpose and special-purpose implementations, which are summarized in Table 8-2. As always, complete details are in the JDK's documentation and Javadoc. Javadoc refers to either a collection of documentation that is written inline alongside the code, or an executable tool that operates on that inline documentation.

All classes in Table 8-2 are in the `java.util` package except `ConcurrentHashMap` and `ConcurrentSkipListMap`, which are part of `java.util.concurrent`.

*Table 8-2. Map implementations*

| Class | Representation | Null key | Null values | Notes |
|---|---|---|---|---|
| `HashMap` | Hashtable | Yes | Yes | General-purpose implementation. |
| `ConcurrentHashMap` | Hashtable | No | No | General-purpose threadsafe implementation; see `ConcurrentMap` interface. |
| `ConcurrentSkipListMap` | Hashtable | No | No | Specialized threadsafe implementation; see `ConcurrentNavigableMap` interface. |
| `EnumMap` | Array | No | Yes | Keys are instances of an enum. |
| `LinkedHashMap` | Hashtable plus list | Yes | Yes | Preserves insertion or access order. |
| `TreeMap` | Red-black tree | No | Yes | Sorts by key value. Operations are O(log(n)). See `Sorte` |

| | | | | |
|---|---|---|---|---|
| | | | | dMap interface. |
| IdentityHashMap | Hashtable | Yes | Yes | Compares with == instead of equals(). |
| WeakHashMap | Hashtable | Yes | Yes | Doesn't prevent garbage collection of keys. |
| Hashtable | Hashtable | No | No | Legacy class; synchronized methods. Do not use. |
| Properties | Hashtable | No | No | Extends Hashtable with String methods. |

The ConcurrentHashMap and ConcurrentSkipListMap classes of the java.util.concurrent package implement the ConcurrentMap interface of the same package. ConcurrentMap extends Map and defines some additional atomic operations that are important in multithreaded programming. For example, the putIfAbsent() method is like put() but adds the key-value pair to the map only if the key is not already mapped.

TreeMap implements the SortedMap interface, which extends Map to add methods that take advantage of the sorted nature of the map. SortedMap is quite similar to the SortedSet interface we'll meet in the next section. The firstKey() and lastKey() methods return the first and last keys in the keySet(). And headMap(), tailMap(), and subMap() return a restricted range of the original map.

## The Set Interface

A *set* is a collection of objects that does not allow duplicates: it may not contain two references to the same object, two references to null, or references to two objects a and b such that a.equals(b). Most general-purpose Set implementations impose no ordering on the elements of the set, but ordered sets are not prohibited (see SortedSet and LinkedHashSet). Sets are further distinguished from ordered collections like lists by the general expectation that they have an efficient contains method that runs in constant or logarithmic time.

Set defines no methods of its own beyond those defined by Collection but places additional restrictions on some methods. The add() and addAll() methods of a Set are required to enforce the no-duplicates rules: they may not add an element to the Set if the set already contains that element. Recall that the add() and addAll() methods defined by the

`Collection` interface return `true` if the call resulted in a change to the collection, and `false` if it did not. This return value is relevant for `Set` objects because the no-duplicates restriction means that adding an element does not always result in a change to the set.

Table 8-3 lists the implementations of the `Set` interface and summarizes their internal representation, ordering characteristics, member restrictions, and the performance of the basic `add()`, `remove()`, and `contains()` operations as well as iteration performance. Note that `CopyOnWriteArraySet` is in the `java.util.concurrent` package; all the other implementations are part of `java.util`. Also note that `java.util.BitSet` is not a `Set` implementation. This legacy class is useful as a compact and efficient list of `boolean` values but is not part of the Java Collections Framework.

*Table 8-3. Set implementations*

| Class | Internal representation | Element order | Member restrictions | Basic operations | Iterati perfo |
|---|---|---|---|---|---|
| `HashSet` | Hashtable | None | None | O(1) | O(capa |
| `LinkedHashSet` | Linked hashtable | Insertion order | None | O(1) | O(n) |
| `EnumSet` | Bit fields | Enum declaration | Enum values | O(1) | O(n) |
| `TreeSet` | Red-black tree | Sorted ascending | Comparable | O(log(n)) | O(n) |
| `CopyOnWriteArraySet` | Array | Insertion order | None | O(n) | O(n) |

The `TreeSet` implementation uses a red-black tree data structure to maintain a set that is iterated in ascending order according to the natural ordering of `Comparable` objects or according to an ordering specified by a `Comparator` object. `TreeSet` actually implements the `SortedSet` interface, which is a subinterface of `Set`.

The `SortedSet` interface offers several interesting methods that take advantage of its sorted

nature. In particular, it can find the first and last element of the set, and can provide subsets before or after a specific element, which are called the head and tail. The following code illustrates:

```
// Create a SortedSet
SortedSet<Integer> s = new TreeSet<>(List.of(4, 3, 2, 1));

// Iterate set: elements are automatically sorted
for (Integer i : s) {
    System.out.println(i);
}

// Special elements
Integer first = s.first();  // First element, 1
Integer last = s.last();    // Last element, 4

// all elements but first
SortedSet<Integer> tail = s.tailSet(first + 1);
System.out.println(tail);   // 2, 3, 4 -- inclusive

// all elements but last
SortedSet<Integer> head = s.headSet(last);
System.out.println(head);   // 1, 2, 3 -- exclusive

SortedSet<Integer> middle = s.subSet(first + 1, last);
System.out.println(middle); // 2, 3 -- inclusive
```

Note that `tailSet` and `subSet` include the initial element requested, while `headSet` excludes it. Depending on the type of your set, this may complicate scenarios—for example, when you want the opposite behavior (e.g., all elements strictly after a given object).

From Java 9 onward, the API has also been upgraded with a helper static method on the `Set` interface, like this:

```
Set<String> set = Set.of("Hello", "World");
```

This API has several overloads that each take a fixed number of arguments, and also a varargs overload. The latter is used for the case where an arbitrary number of elements are wanted in the set and falls back to the standard varargs mechanism (marshaling the elements into an array before the call).

### The Iterable and Iterator interfaces

The `java.lang.Iterable` interface was introduced to enable foreach, but it also allows for general iteration across a collection of objects. A class implements this interface to advertise that it can provide a `java.util.Iterator` that can traverse through the collection:

```
public interface Iterable<E> {
     java.util.Iterator<E> iterator();
}
```

If an object is `Iterable<E>`, its `iterator()` method will return an `Iterator<E>`, which has a `next()` method that returns an object of type `E`. `java.lang.Iterable` is included in the base `Collection` interface, so implementations of that type can be walked through using a `java.util.Iterator`. Note that `Map` isn't directly `Iterable` but does provide methods to return the `keySet()`, `values()`, and `entrySet()`, which are `Iterable`.

> **WARNING**
>
> Don't get caught off guard! Despite their similar names and related purpose, `Iterable` is in the `java.lang` package, while the `Iterator` interface (returned by `Iterable.iterator()`) lives in the `java.util` package.

An `Iterator` tracks a single iteration through the elements of a collection. While there are more elements in the collection (`hasNext()` returns `true`), calling `next()` obtains the next element of the collection:

```
public interface Iterator<E> {
     boolean hasNext();
     E next();
     void remove();
}
```

When we've previously shown a *foreach* loop over an `Iterable` object, that is the equivalent of the following code:

```
// Iteration with a for loop
for(Iterator<String> i = c.iterator(); i.hasNext();) {
    System.out.println(i.next());
}
```

`Iterable` and `Iterator` can be used directly in addition to the syntactic sugar of *foreach* loops. It fits particularly well with `while` loops:

```
// Iterate through collection elements with a while loop.
Iterator<String> iterator = c.iterator();
while (iterator.hasNext()) {
    System.out.println(iterator.next());
}
```

> **WARNING**
>
> The `next()` method of `Iterator` performs two functions—it advances through the collection and also returns the element of the collection that we have just moved past. This combination of operations can cause problems when you are programming in a functional or immutable style, since it mutates the underlying collection.

Ordered collections, such as lists, typically have iterators that guarantee they'll return elements in

order. Unordered collections like `Set` simply guarantee that repeated calls to `next()` return all elements of the set without omissions or duplications, but they do not specify an ordering. Collections supporting ordered traversal will typically implement the `SequencedCollection`, `SequencedSet`, or `SequencedMap` interfaces as well, although this may not be true for all implementations of `Iterable`.

A common pitfall with iterators regards modification. If the collection is modified while iteration is in process, it may throw a `ConcurrentModificationException` error type:

```
List<String> l = new ArrayList<>(List.of("one", "two", "three"));
for (String x : l) {
    if (x.equals("one")) {
        l.remove("one");  // throws ConcurrentModificationException
    }
}
```

Avoiding this exception requires rethinking your algorithm so it doesn't modify the collection. This can often be accomplished by working against a local copy instead of the original collection. The newer `Stream` APIs for collections, which we'll discuss next, also provide a lot of useful helpers for these situations.

## Mutability and Immutability Revisited

The Java Collections Framework has been part of the standard library since Java 1.2 (1998), so it's unsurprising that the APIs now look a little bit dated. However, there are also some design issues with the Java Collections that were not fully appreciated when they were released. These issues related to the concept of mutability, which we discussed in "Mutability and Immutability".

Recall that prior to Java 8, interfaces could contain only mandatory methods, and the suggested approach to optional methods was to document methods as optional in the interface and for implementations to unconditionally throw `UnsupportedO perationException` if the optional method was not present.

The central problem is that the Java Collections interfaces are very large in terms of the number of methods defined.[2] These methods contain `add()`, `remove()`, and other methods that are obviously intended to mutate a collection in place. In other words, mutability is built into the definition of the Java Collections interfaces.

This raises the question of what to do if we want to create an immutable implementation of a collection. Any such implementation must implement the mutating methods, because they are mandatory.

It's worth noting that these methods are also required on the objects returned by the helper methods such as `List.of()` despite being immutable. A runtime exception is thrown on any attempts to add or remove from them after instantiation.

Other languages have different approaches to handling this problem. For example, Scala encourages users to use the Scala collections (which differentiate at type level between mutable

and immutable) wherever possible and to use Scala proxies (created by helper methods) if a Java-native collection needs to be used. Kotlin's approach uses *mapped types* to convert a Java collection into either a mutable *or* an immutable Kotlin collection at compile time.

Unfortunately, in Java, we are constrained by the previous design decisions, and, as yet, no one has come up with a satisfactory way to solve the problem of immutable implementations of the Java Collections—so being careful and keeping an eye out for `UnsupportedOperationException` is the best we can do for now.

## Java Streams and Lambda Expressions

In many modern programming languages, it has become a popular style to work with a collection as a whole, in a more declarative and functional style, versus iterating through with a `for` loop. To evolve to meet programmers' needs and to support these modern programming styles, Java 8 introduced lambda expressions, largely to facilitate extending the Collections API.

The new approach is referred to as *Java Streams*. Given Java's commitment to backward compatibility, support for streams was additive within the existing Collection API rather than a breaking change. A stream can be created from a collection simply by calling the `stream()` method.

> **NOTE**
>
> The desire to add new methods to existing interfaces was directly responsible for the new language feature referred to as *default methods* (see "Default Methods" for more details). Without this new mechanism, older implementations of the Collections interfaces would fail to compile under Java 8 and would fail to link if loaded into a Java 8 runtime.

In this section, we will introduce the use of streams and lambda expressions. For a fuller treatment, see *Java 8 Lambdas* by Richard Warburton (O'Reilly).

### The Streams API

The Java library designers faced a challenge in introducing streams. A large number of implementations of the core collections interfaces exist in the wild. If new stream methods were added to the `Collection` interfaces, implementations that predate Java 8 and lambdas would lack the methods corresponding to the new functional operations. Worse still, as method names such as `map()` and `filter()` have never been part of the interface of the Collections, implementations may already have methods with those names, leading to conflicts.

To work around this problem, a new abstraction called a `Stream` was introduced. A `Stream` object is generated from a collection object via the `stream()` method. This `Stream` type, being new and under the control of the library designers, is then guaranteed to be free of collisions. This then minimizes the risk of clashes, as only Collections implementations that

contained a `stream()` method would be affected.

A `Stream` object plays a similar role to an `Iterator` in the new approach to collections code. The overall idea is for the developer to build up a sequence (or "pipeline") of operations (such as `map`, `filter`, or `reduce`) that need to be applied to the collection as a whole. The actual content of the operations will usually be expressed as a lambda expression for each operation. Each stage of the pipeline returns another stream, so operations may be chained together.

At the end of the pipeline, the results usually need to be gathered up, or "materialized," either as a new collection or another value. This is done either by using a `Collector` or by finishing the pipeline with a "terminal method" such as `reduce()` that returns an actual value, rather than another stream. Overall, the new approach to collections looks like this:

```
        stream()   filter()   map()   collect()
Collection -> Stream -> Stream -> Stream -> Collection
```

The `Stream` class behaves as a sequence of elements that are accessed one at a time (although there are some types of streams that support parallel access and can be used to process larger collections in a naturally multithreaded way). In a similar way to an `Iterator`, the `Stream` is used to take each item in turn.

As is usual for generic classes in Java, `Stream` is parameterized by a reference type. However, in many cases, we actually want streams of primitive types, especially `int`s and `double`s. We cannot have `Stream<int>`, so instead in `java.util.stream` there are special (nongeneric) classes such as `IntStream` and `DoubleStream`. These are known as *primitive specializations* of the `Stream` class and have APIs that are very similar to the general `Stream` methods, except that they use primitives where appropriate.

## Functional Approaches

Java Streams enable an approach derived from functional programming languages and styles. We met some of these key patterns in "Lambdas and Functional Programming". Let's reintroduce them and look at some examples of each.

### Filter

The filter idiom applies a piece of code returning either true or false (known as a *predicate*) to each element in a stream. A new stream is built consisting of the elements that "passed the test" (i.e., the predicate returned `true` when applied to the element).

For example, let's look at some code to work with a collection of cats and pick out the tigers:

```
List<String> cats = List.of("tiger", "cat", "TIGER", "leopard");
String search = "tiger";
String tigers = cats.stream()
                    .filter(s -> s.equalsIgnoreCase(search))
                    .collect(Collectors.joining(", "));
```

```
// Prints "tiger, TIGER"
System.out.println(tigers);
```

The key piece is the call to `filter()`, which takes a lambda expression. The lambda takes in a string and returns a Boolean value. This is applied over the whole stream created from `cats`, and a new stream is created, which contains only tigers (no matter how they were capitalized).

The `filter()` method takes in an instance of the `Predicate` interface, from the package `java.util.function`. This is a functional interface with only a single nondefault method and so is a perfect fit for a lambda expression.

Note the final call to `collect()`; this is an essential part of the API and is used to "gather up" the results at the end of the lambda operations. We'll discuss it in more detail in the next section.

`Predicate` has some other very useful default methods, such as for constructing combined predicates by using logic operations. For example, if the tigers want to admit leopards into their group, this can be represented by using the `or()` method:

```
Predicate<String> p = s -> s.equalsIgnoreCase("tiger");
Predicate<String> combined = p.or(s -> s.equalsIgnoreCase("leopard"));
String pride = cats.stream()
                   .filter(combined)
                   .collect(Collectors.joining(", "));
System.out.println(pride);
```

Note that it's much clearer if the `Predicate<String>` object `p` is explicitly created so that the defaulted `or()` method can be called on it and the second lambda expression (which will also be automatically converted to a `Predicate<String>`) can be passed to it.

### Map

The map idiom is about transforming one stream into a new stream, where the new stream potentially has different types and values from the original. This uses the interface `Function<T, R>` in the package `java.util.function`. Like `Predicate<T>` we used with `filter()`, `Function<T, R>` is a functional interface and so has only one nondefaulted method, `apply()`. The type `T` is the type for the single parameter passed to the function, and `R` represents the return type of the function.

Let's look at a code example that uses `map()`:

```
List<Integer> namesLength = cats.stream()
                .map(String::length)
                .toList();
System.out.println(namesLength);
```

This gets called upon the previous `cats` variable (a `List<String>`) and applies the function `String::length` (a method reference) to each string in turn. The result is a new stream—but of `Integer` this time. We turn that stream into a `List` with the `toList()` method.

> **WARNING**
>
> Nothing prevents a `map()` or `filter()` call from mutating elements. It is only a convention that they must not mutate, but it's one that every Java programmer should adhere to.

Note that unlike the collections API, the `map()` method does not mutate the stream in place but returns a new value. This is key to the functional style as used here.

### forEach

The map and filter idioms are used to create one stream from another. In languages that are strongly functional, this would be combined with requiring that the original stream was not affected by the body of the lambda as it touched each element. In computer science terms, this means that the lambda body should be "side effect free."

In Java, of course, we often need to deal with mutable data, so the Streams API provides a way to mutate elements as the stream is traversed—the `forEach()` method. This takes an argument of type `Consumer<T>`, which is a functional interface that receives a single element of the stream and returns void. It is expected to operate by side effects, whether mutating the object itself or taking other actions like printing to the console or updating a database. A simple example of `forEach()` looks like this:

```
List<String> pets =
  List.of("dog", "cat", "fish", "iguana", "ferret");
pets.stream().forEach((pet) -> {
  System.out.println(pet);
});
```

Lambdas of type `(T t) → void` can be converted to `Consumer<T>`, which makes them convenient to use with `forEach`:

```
// Equivalent to prior example
pets.stream().forEach(System.out::println);
```

In this example, we are simply printing out each member of the collection. However, we're doing so by using a special kind of method reference as a lambda expression. This type of method reference is called a *bound method reference*, as it involves a specific object (in this case, the object `System.out`, which is a static public field of `System`). This is equivalent to the lambda expression:

```
s -> System.out.println(s);
```

This is, of course, eligible for conversion to an instance of a type that implements `Consumer<? super String>` as required by the method signature.

There's one final functional technique that we should look at before we move on. This is the

practice of aggregating a collection down to a single value, and it's the subject of our next section.

### Reduce

The reduce idiom is really a family of similar and related operations, some referred to as fold, or aggregation, operations. In the Streams API, it is implemented by the `reduce()` method.

In Java, `reduce()` takes two arguments. These are the initial value, which is often called the identity (or zero), and a function to apply step-by-step. This function is of type `BinaryOperator<T>`, which is another functional interface that takes in two arguments of the same type and returns another value of that type. This second argument to `reduce()` is a two-argument lambda. The signature of `reduce()` is:

```
T reduce(T identity, BinaryOperator<T> aggregator);
```

The easy way to think about the second argument to `reduce()` is that it creates a "running total" as it steps through the stream. It starts by combining the identity with the first element of the stream to produce the first result, then combines that result with the second element of the stream, and so on.

It can help to imagine that the implementation of `reduce()` works a bit like this:

```
public T reduce(T identity, BinaryOperator<T> aggregator) {
    T runningTotal = identity;
    for (T element : myStream) {
        runningTotal = aggregator.apply(runningTotal, element);
    }

    return runningTotal;
}
```

> **NOTE**
>
> In practice, implementations of `reduce()` can be more sophisticated than these and can even execute in parallel if the data structure and operations are amenable to this.

Let's look at a quick example of a `reduce()` and calculate the sum of some primes:

```
double sumPrimes = List.of(2, 3, 5, 7, 11, 13, 17, 19, 23)
        .stream()
        .reduce(0, (x, y) -> x + y);
System.out.println("Sum of some primes: " + sumPrimes);
```

### Stream gatherers

We've met many different operations that process elements in our stream pipeline. Those built-in operations, though, don't cover every possible scenario. The core stream methods largely work

one-to-one on elements, and pass them along immediately without buffering. Many situations, though, may require processing where a single element results in many downstream, or we want to aggregate or deduplicate elements before further processing. These were difficult to accomplish with the original Streams API.

The `Gatherer` interface was built to support these use cases and more, opening up the stream pipeline to customization. A full examination is beyond the scope of this book, but we'll briefly introduce them with a basic example.

Sometimes it may be useful to glimpse the intermediate values in a longer chain of methods. As a slightly contrived example, perhaps we want to know how often strings in a stream that we're processing for other reasons have an odd length. A naive implementation might look like this:

```
int oddCount = 0;
List<Integer> result = List.of("yes", "no", "odd", "even")
        .stream()
        .map((s) -> {
          if (s.length() % 2 != 0) oddCount++;
          return s;
        })
        .map((s) -> s.length())
        .toList();
```

While this gets the job done, it's also not very portable. If we wanted to do this tracking for another stream, we're stuck making yet another local variable and copying and pasting similar code in.

With a `Gatherer` we can make this more shareable. The core method is `integrator`, which is called for each element as it arrives in the gatherer for processing:

```
class OddCounter implements Gatherer<String, Integer, String> {
  public Integer count = 0;

  @Override
  public Gatherer.Integrator<Integer, String, String> integrator() {
    return Gatherer.Integrator.of((_, element, downstream) -> {
      if (element.length() % 2 != 0) count++;
      downstream.push(element);
      return true;
    });
  }
}

OddCounter counter = new OddCounter();
List<String> source = List.of("yes", "no", "odd", "even");
List<Integer> result = source
        .stream()
        .gather(counter)
        .map((s) -> s.length())
        .toList();

System.out.println("Odd entries seen: " + counter.count)
```

While this is obviously overkill for a single instance, it shows how the `Gatherer` interface extracts intermediate pipeline logic to be reused and abstracted.

The `Gatherer` interface has additional methods—`initializer`, `combiner`, and `finisher`—to allow flexibility in the stream processing. There is also a factory method for easy construction of ad hoc gatherers via `Gatherer.of`. Consult the JDK documentation for more details.

### Lazy evaluation

In fact, streams are more general than iterators (or even collections), as streams do not manage storage for data. In earlier versions of Java, there was always a presumption that all of the elements of a collection existed (usually in memory). It was possible to work around this in a limited way by insisting on the use of iterators everywhere, as well as by having the iterators construct elements on the fly. However, this was neither very convenient nor that common.

By contrast, streams are an abstraction for managing data, rather than being concerned with the details of storage. This makes it possible to handle more subtle data structures than just finite collections. For example, infinite streams can easily be represented by the `Stream` interface, and they can be used as a way, for example, to handle the set of all square numbers. Let's see how we could accomplish this using a `Stream`:

```
public class SquareGenerator implements IntSupplier {
    private int current = 1;

    @Override
    public synchronized int getAsInt() {
        int thisResult = current * current;
        current++;
        return thisResult;
    }
}

IntStream squares = IntStream.generate(new SquareGenerator());
PrimitiveIterator.OfInt stepThrough = squares.iterator();
for (int i = 0; i < 10; i++) {
    System.out.println(stepThrough.nextInt());
}
System.out.println("First iterator done...");

// We can go on as long as we like...
for (int i = 0; i < 10; i++) {
    System.out.println(stepThrough.nextInt());
}
```

Because our list of possible values is infinite, we must adopt a model in which elements do not all exist ahead of time. Essentially, a bit of code must return the next element as we demand it. The key technique used to accomplish this is *lazy evaluation*.

**NOTE**

> Lazy evaluation is a big change for Java, as until JDK 8 the value of an expression was always computed as soon as it was assigned to a variable (or passed into a method). This familiar model, where values are computed immediately, is called *eager evaluation*, and it is the default behavior for evaluation of expressions in most mainstream programming languages.

We can see this lazy evaluation in action in the preceding example if we modify `getAsInt()` slightly to provide output actively when it is called:

```
@Override
public synchronized int getAsInt() {
    int thisResult = current * current;
    System.out.print(String.format("%d... ", thisResult));
    current++;
    return thisResult;
}
```

When this modified program is run, we'll see output that shows each `getAsInt()` call immediately followed by the use of that value in the `for` loop:

```
1... 1
4... 4
9... 9
16... 16
25... 25
36... 36
49... 49
64... 64
81... 81
100... 100
First iterator done...
121... 121
...
```

One significant consequence of modeling the infinite stream is that terminal methods like `collect()` won't work. This is because we can't materialize the whole stream to a collection (we would run out of memory before we created the infinite amount of objects we would need).

Even when a stream isn't infinite, it's important to recognize what parts of the evaluation are lazy. For instance, the following code that tries to show us diagnostic information during a `map` operation doesn't actually yield any output:

```
List.of(1, 2, 3, 4, 5)
    .stream()
    .map((i) - > {
        System.out.println(i);
        return i;
    });
```

Only once we provide a terminal action such as `collect()` or `toList()` is our `map()` lambda actually executed.

Recognizing which intermediate results are lazy in their evaluation is a topic Java developers should be mindful of when working with the Stream API. The more complicated implementation details, though, fall to library writers rather than users of streams.

When debugging streams, it's often useful to inspect intermediate values, as we did previously with `map`. The `peek` method provides a more convenient way to look at values in the midst of a stream without altering the values passed along. The following prints out each intermediate value without having to return them in the lambda:

```
List.of(1, 2, 3, 4, 5)
    .stream()
    .peek((i) -> System.out.println(i))
    .toList();
```

While the combination of `filter`, `map`, and `reduce` can accomplish almost any stream-related task we're after, it isn't always the most convenient API. There are a wide variety of additional methods that build on top of these primitives to give us a richer vocabulary to work with streams.

### Further filtering

A common place where working with streams benefits from more elaborate methods is filtering. A number of methods on the `Stream` interface allow more expressive descriptions of how we want to cut down our streams for consumption:

```
// Distinct elements only
Stream.of(1, 2, 1, 2, 3, 4)
      .distinct();
// Results in  [1, 2, 3, 4]

// Ignores items as long as the predicate matches, then returns the
// entire remainder of the stream. After the first mismatch, later
// elements are not required to match the predicate.
Stream.of(1, 2, 3, 4, 5, 3)
      .dropWhile((i) -> i < 4);
// Results in [4, 5, 3]

// Returns items from the stream until the predicate stops matching.
// Note that later elements matching the predicate aren't returned.
Stream.of(1, 2, 3, 4, 3)
      .takeWhile((i) -> i < 4);
// Results in [1, 2, 3]

// Skips the first N items in the stream
Stream.of(1, 2, 3, 4, 5)
      .skip(2);
// Results in [3, 4, 5]

// Limits items taken from stream to an exact value
// Useful with infinite streams to set boundaries
Stream.of(1, 2, 3, 4, 5)
      .limit(3);
// Results in [1, 2, 3]
```

### Matching in streams

Another typical operation is to ask questions of an entire stream of elements, such as whether all (or none) match a given predicate, or alternatively if there's any single element that matches:

```
// Are all the items odd?
Stream.of(1, 1, 3, 5)
      .allMatch((i) -> i % 2 == 1);
// Returns true

// Are none of the items even?
Stream.of(1, 1, 3, 5)
      .noneMatch((i) -> i % 2 == 0);
// Returns true

// Is at least one item even?
Stream.of(1, 1, 3, 5, 6)
      .anyMatch((i) -> i % 2 == 0);
// Returns true
```

These matching checks are terminal operations, after which the stream is closed for further evaluation.

### Flattening

Once we've started down the path of modeling our data as streams, it's not unusual to find yet another layer of streams beneath. For instance, if we're processing multiple lines of text and wanted to gather the set of words from the entire block, we might reach first for code like this:

```
var lines = Stream.of(
  "For Brutus is an honourable man",
  "Give me your hands if we be friends and Robin shall restore amends",
  "Misery acquaints a man with strange bedfellows");

lines.map((s) -> s.split(" +"));
// Returns Stream.of(new String[] { "For", "Brutus",...},
//                   new String[] { "Give", "me", "your", ... },
//                   new String[] { "Misery", "acquaints", "a", ... },
```

This isn't quite the plain word list we're after, though. We have an extra layer of nesting, a `Stream<String[]>` instead of `Stream<String>`.

The `flatMap()` method is designed for exactly these situations. For each element in our original stream, the lambda provided to `flatMap()` returns not an individual value but another `Stream`. Then `flatMap()` gathers those multiple streams and joins them, flattening to a single stream of the contained type.

In our example, `split()` gives us arrays, which we can trivially convert to streams. From there, `flatMap()` will do the work of turning those multiple streams into the single stream of words we were after:

```
lines.flatMap((s) -> Arrays.stream(s.split(" +")));
```

```
// Returns Stream.of("For", "Brutus", "is", "an", ...)
```

### Parallel streams

With the introduction of the `Stream` interface in Java 8, the JDK also gained the ability to easily process a stream in parallel. The work of processing each element is passed off to a threadpool (which we'll meet in more detail in Chapter 12):

```
// Suspicious usage of parallelStream
List.of(1, 2, 3, 4, 5)
  .parallelStream()
  .map((i) -> i * 2)
  .toList();
```

At first this appears almost magical. Why wouldn't you make everything parallel to maximize the work getting done? The problem is that dispatching to the threadpool and gathering the results isn't free. For an example like our code—and for the vast majority of stream usages in real production code—making it parallel will actually slow your code down because of those coordination costs.

Parallel streams may be worth investigating if the following holds true:

- Ordering of the elements doesn't matter—processing out of sequence doesn't change the final results.
- Items are independent of one another, as they may be handled on separate threads.
- The number of simultaneous streams isn't too high, as they share a single threadpool, which can become a bottleneck.
- Measurement *on your live system* shows a bottleneck in stream computation.

That last point is key. If you simply swap out `stream()` for `parallelStream()`, hoping that it will make your program run faster, you're in for some nasty surprises in production. The bar is relatively high for showing that this convenient method is actually helping you out, so approach it with caution and real data.

### From streams to Collections

Defining a separate `Stream` interface was a pragmatic way to enable newer styles of development with Java while not breaking existing code. However, sometimes you still need the standard Java Collections, whether to pass to another API or for functionality that isn't present in streams. For the most common cases of returning a simple `List` or array of elements, the methods are provided directly on the `Stream` interface:

```
// Immutable list returned
List<Integer> list =
    Stream.of(1, 2, 3, 4, 5).toList();

// Note the return type is `Object[]`
```

```
Object[] array =
    Stream.of(1, 2, 3, 4, 5).toArray();
```

Transforming a stream into a nonstream collection or other object is primarily performed through the `collect()` method. This method receives an instance of the `Collector` interface, allowing for limitless possibilities to gather up our stream results without adding to the `Stream` interface itself.

Standard implementations for a variety of collectors are available on the `Collectors` class as static methods. For instance, we can turn our stream into any of our normal collection types:

```
// In earlier versions of Java, Stream#toList() didn't exist
// The next line was the commonly used approach, still seen today
List<Integer> list =
    Stream.of(1, 2, 3, 4, 5)
          .collect(Collectors.toList());

// Create a standard Set (no duplicates)
Set<Integer> set =
    Stream.of(1, 2, 3, 4, 5)
          .collect(Collectors.toSet());

// For Collection types that don't have a specific method, we can
// use toCollection with a function that creates our empty instance
// Each item will be added to that collection
TreeSet<Integer> collection =
    Stream.of(1, 2, 3, 4, 5)
          .collect(Collectors.toCollection(TreeSet::new));

// When creating maps, we must provide two functions
// The first constructs the key for each element, the second the value
// Here, each int is its own key and the value is its toString()
Map<Integer, String> map =
    Stream.of(1, 2, 3, 4, 5)
          .collect(Collectors.toMap(
                      (i) -> i,
                      Object::toString));
```

Unlike `Stream#toList()`, all of these options return a modifiable version of their collection type. `Collectors` also provides specific methods if you want to return an unmodifiable or immutable version. They follow a naming convention `toUnmodifiableX()`, where `X` is the collection type as seen in the preceding code block.

A final variation on gathering collections is when you want to group the elements by some property. In this example, we want to group the numbers by their first digit:

```
Map<Character, List<Integer>> grouped =
        Stream.of(10, 11, 12, 20, 30)
                .collect(Collectors.groupingBy((i) -> {
                    return i.toString().charAt(0);
                }));
// Returns map with {"1"=[10, 11, 12], "2"=[20], "3"=[30]}
```

### From streams to values

We don't always want to retrieve collections from our streams—sometimes we need a single value, much like the `reduce()` method gave us.

`Stream` has a few built-in methods for the most common values we might want from our stream:

```
var count = Stream.of(1, 2, 3).count();
var max = Stream.of(1, 2, 3).max(Integer::compareTo);
var min = Stream.of(1, 2, 3).min(Integer::compareTo);
```

The `collect()` method isn't limited to returning collection types either. A wide variety of result-gathering methods are available from `Collectors` to aid in common calculations, particularly on streams of numbers. These methods all require a function for turning the incoming item from the stream to a number, which allows it to be easily used with objects as well as primitive values:

```
var average =
    Stream.of(1, 2, 3)
          .collect(Collectors.averagingInt(Integer::intValue));

var sum =
    Stream.of(1, 2, 3)
          .collect(Collectors.summingInt(Integer::intValue));

var summary =
    Stream.of(1, 2, 3)
          .collect(Collectors.summarizingInt(Integer::intValue));
// IntSummaryStatistics{count=3, sum=6, min=1, average=2.0, max=3}
```

Similar methods are available for long and double types in addition to integers.

A final way of getting a result from a stream helps us with strings. A classic issue is turning a series of smaller strings into one larger delimited string. Streams make this quite simple.

```
var words = Stream.of("This", "is", "some", "text");
var csv = words.collect(Collectors.joining(", "));
// Returns string "This, is, some, text"
```

### Streams utility default methods

Java Streams took the opportunity to introduce a number of new methods to the Java Collections libraries. Using default methods, it was possible to add new methods to the Collections without breaking backward compatibility.

Some of these methods are *scaffold methods* for creating Streams from our existing collections. These include methods such as `Collection::stream`, `Collection::parallelStream`, and `Collection::spliterator` (which has specialized forms `List::spliterator` and `Set::spliterator`).

Other methods provide shortcuts to functionality that existed elsewhere in previous versions. For instance, the `List::sort` method essentially delegates to the more cumbersome version already available on the `Collections` class:

```
// Essentially just forwards to the helper method in Collections
public default void sort(Comparator<? super E> c) {
    Collections.<E>sort(this, c);
}
```

The remaining methods provide additional functional techniques using the interfaces of `java.util.function`:

`Collection::removeIf`

: This method takes a `Predicate` and iterates internally over the collection, removing any elements that satisfy the predicate object.

`Map::forEach`

: The single argument to this method is a lambda expression that takes two arguments (one of the key's type and one of the value's type) and returns `void`. This is converted to an instance of `BiConsumer` and applied to each key-value pair in the map.

`Map::computeIfAbsent`

: This takes a key and a lambda expression that maps the key type to the value type. If the specified key (first parameter) is not present in the map, then it computes a default value by using the lambda expression and puts it in the map.

(See also `Map::computeIfPresent`, `Map::compute`, and `Map::merge`.)

## Utility Methods

The `java.util.Collections` class is home to quite a few static utility methods designed for use with collections. One important group of these methods is the collection *wrapper* methods: they return a special-purpose collection wrapped around a collection you specify.

The purpose of the wrapper collection is to wrap additional functionality around a collection that does not provide it itself. Wrappers exist to provide thread safety, write protection, and runtime type checking. Wrapper collections are always *backed by* the original collection, which means that the methods of the wrapper simply dispatch to the equivalent methods of the wrapped collection. This means that changes made to the collection through the wrapper are visible through the wrapped collection, and vice versa.

The first set of wrapper methods provides threadsafe wrappers around collections. Except for the legacy classes `Vector` and `Hashtable`, the collection implementations in `java.util` do not have `synchronized` methods and are not protected against concurrent access by multiple threads. If you need threadsafe collections and don't mind the additional overhead of

synchronization, create them with code like this:

```
List<String> list =
    Collections.synchronizedList(new ArrayList<>());
Set<Integer> set =
    Collections.synchronizedSet(new HashSet<>());
Map<String, Integer> map =
    Collections.synchronizedMap(new HashMap<>());
```

A second set of wrapper methods provides collection objects through which the underlying collection cannot be modified. They return a read-only view of a collection: an `UnsupportedOperationException` will result from attempting to change the collection's content. These wrappers are useful when you must pass a collection to a method that must not be allowed to modify or mutate the content of the collection in any way:

```
List<Integer> primes = new ArrayList<>();
List<Integer> readonly = Collections.unmodifiableList(primes);
// We can modify the list through primes
primes.addAll(Arrays.asList(2, 3, 5, 7, 11, 13, 17, 19));
// But we can't modify through the read-only wrapper
readonly.add(23);  // UnsupportedOperationException
```

The `java.util.Collections` class also defines methods to operate on collections. Some of the most notable are methods to sort and search the elements of collections:

```
Collections.sort(list);
// list must be sorted first
int pos = Collections.binarySearch(list, "key");
```

Here are some other interesting `Collections` methods:

```
// Copy list2 into list1, overwriting list1 up to the length of list2
Collections.copy(list1, list2);
// Fill list with Object o
Collections.fill(list, o);
// Find the largest element in Collection c
Collections.max(c);
// Find the smallest element in Collection c
Collections.min(c);

Collections.reverse(list);      // Reverse list
Collections.shuffle(list);      // Mix up list
```

It is a good idea to familiarize yourself fully with the utility methods in `Collections` and `Arrays`, as they can save you from writing your own implementation of a common task.

### Special-case collections

In addition to its wrapper methods, the `java.util.Collections` class also defines utility methods for creating immutable collection instances that contain a single element, and other methods for creating empty collections. `singleton()`, `single tonList()`, and

`singletonMap()` return immutable `Set`, `List`, and `Map` objects that contain a single specified object or a single key-value pair. These methods are useful when you need to pass a single object to a method that expects a collection.

The `Collections` class also includes methods that return empty collections. If you are writing a method that returns a collection, it is usually best to handle the no-values-to-return case by returning an empty collection instead of a special-case value like `null`:

```
List<String> ls = Collections.emptyList();
Map<String, Integer> m = Collections.emptyMap();
Set<Integer> si = Collections.emptySet();
```

Since Java 9, though, these methods are frequently replaced by the `of()` methods on the `Set`, `List`, and `Map` interfaces:

```
List<String> ls = List.of();
Map<String, Integer> m = Map.of();
Set<Integer> si = Set.of();
```

These return immutable versions of their type and may also take elements through the same method:

```
List<String> ls = List.of("string");
Map<String, Integer> m = Map.of("one", 1);
Set<Integer> si = Set.of(1);
```

Finally, `nCopies()` returns an immutable `List` that contains a specified number of copies of a single specified object:

```
List<Integer> tenzeros = Collections.nCopies(10, 0);
```

Although these special cases aren't as useful in modern Java, you will see them in code, so it's good to be familiar with them.

## Arrays and Helper Methods

Arrays of objects and collections serve similar purposes. It is possible to convert from one to the other:

```
// An array
String[] a = { "this", "is", "a", "test" };
// View array as an ungrowable list
List<String> l = Arrays.asList(a);
// Make a growable copy of the view
List<String> m = new ArrayList<>(l);

// asList() is a varargs method so we can do this, too:
Set<Character> abc =
    new HashSet<Character>(Arrays.asList('a', 'b', 'c'));
```

```
// Collection defines a toArray method. The no-args version creates
// an Object[] array, copies collection elements to it and returns it
// Get set elements as an array
Object[] members = set.toArray();
// Get list elements as an array
Object[] items = list.toArray();
// Get map key objects as an array
Object[] keys = map.keySet().toArray();
// Get map value objects as an array
Object[] values = map.values().toArray();

// If you want the return value to be something other than Object[],
// pass in an array of the appropriate type. If the array is not
// big enough, another one of the same type will be allocated.
// If the array is too big, the collection elements copied to it
// will be null-filled
String[] c = l.toArray(new String[0]);
```

A stream can be directly created from an array via `Arrays.stream` without needing an intermediate collection:

```
// An array
String[] a = { "this", "is", "a", "test" };

// Create a stream from the array
Stream<String> stream = Arrays.stream(a);

// Mapping across the stream
List<Integer> lengths = stream.map(String::length).toList();
```

In addition, there are a number of useful helper methods for working with Java's arrays, which are included here for completeness.

The `java.lang.System` class defines an `arraycopy()` method that is useful for copying specified elements in one array to a specified position in a second array. The second array must be the same type as the first, and it can even be the same array:

```
char[] text = "Now is the time".toCharArray();
char[] copy = new char[100];
// Copy 10 characters from element 4 of text into copy,
// starting at copy[0]
System.arraycopy(text, 4, copy, 0, 10);

// Move some of the text to later elements, making room for
// insertions. If target and source are the same, this will involve
// copying to a temporary array
System.arraycopy(copy, 3, copy, 6, 7);
```

There are also a number of useful static methods defined on the `Arrays` class:

```
// An array of integers
int[] intArray = new int[] { 10, 5, 7, -3 };
```

```
// Sort it in place
Arrays.sort(intArray);
// Value 7 is found at index 2
// Requires that the array be sorted to function properly!
int pos = Arrays.binarySearch(intArray, 7);
// Not found: negative return value
pos = Arrays.binarySearch(intArray, 12);

// Arrays of objects can be sorted and searched too
String[] strArray = new String[] { "now", "is", "the", "time" };
Arrays.sort(strArray);   // sorted to: { "is", "now", "the", "time" }

// Arrays.equals compares all elements of two arrays
String[] clone = (String[]) strArray.clone();
boolean b1 = Arrays.equals(strArray, clone);  // Yes, they're equal

// Arrays.fill  initializes array elements
// An empty array; elements set to 0
byte[] data = new byte[100];
// Set them all to -1
Arrays.fill(data, (byte) -1);
// Set elements 5, 6, 7, 8, 9 to -2
Arrays.fill(data, 5, 10, (byte) -2);

// Creates a new array with elements copied into it.
// The second parameter determines the length of the resulting array.
int[] copied = Arrays.copyOf(new int[] { 1, 2, 3 }, 2);
```

Arrays can be treated and manipulated as objects in Java. Given an arbitrary object `o`, you can use code such as the following to find out if the object is an array and, if so, what type of array it is:

```
Class type = o.getClass();
if (type.isArray()) {
  Class elementType = type.getComponentType();
}
```

While it's typical for Java programs to favor collections over arrays in many cases, you will likely encounter APIs that require arrays, including some in the JDK itself. With these tools, it should be possible to move between the world of collections and the world of arrays seamlessly as needed.

## Summary

In this chapter, we've met the Java Collections libraries and seen how to start working with Java's implementations of fundamental and classic data structures. We've met the general `Collection` interface, as well as `List`, `Set`, and `Map`. We've seen the original, iterative way of handling collections and introduced the Java Streams style, based on ideas from fundamental programming. In the Streams API, we've seen how the approach is more general and can express more subtle programming concepts than the classic approach.

We've only scratched the surface—the Streams API is a fundamental shift in how Java code is

written and architected. There are inherent design limitations in how far the ideals of functional programming can be implemented in Java. Having said that, the possibility that Streams represents “just enough functional programming” is compelling.

Let’s move on. In the next chapter, we will look deeper at how Java handles memory management and garbage collection.

---

1 Project Valhalla may make this possible in future versions of Java. See Appendix A for more details.

2 To see this, just declare a class called `MyList` that implements `List<String>` in an IDE and look at how many methods are autogenerated.

# Chapter 9. Java and Memory

This chapter is an introduction to handling memory in the Java platform. We will cover:

- Introduction to automatic memory management
- The basic mark-and-sweep garbage collection (GC) algorithm
- How the HotSpot JVM optimizes GC based on the lifetime of the object
- Overview of HotSpot's default collector
- Alternate collectors

While it is possible to develop Java applications without any knowledge of the memory management subsystem, an awareness of the topic beyond "a wizard did it" is beneficial for most developers. The subject of garbage collection, in particular, is a deep one. It has a long history and yet is also the continued subject of active and novel academic research. Java's place within that history is an important one, as the JVM provides the best available general-purpose garbage collected runtime.

## Basic Concepts of Automatic Memory Management

The Java platform provides a form of *automatic memory management*, which means that the memory occupied by an object is automatically reclaimed when the object is no longer needed. Conceptually, there are two basic approaches to automatic memory management: *garbage collection* (which is what Java and JavaScript use) and *reference counting* (used by Python and Apple's Swift language).[1]

Some other popular languages do not provide automatic memory management—for example, C and C++ use manual management and Rust uses a novel compile-time approach called *ownership and borrowing*.[2]

For obvious reasons, we will discuss only GC from now on. Pioneered by languages such as Lisp, GC is a technique that has been around for many years, and Java was one of the first truly mainstream languages to support GC. For programmers accustomed to manual memory management, where you must call the `free()` function or the `delete` operator to reclaim memory, GC can be a refreshing change.

The fact that you don't need to remember to destroy every object you create is one of the features that makes Java a pleasant language to work with. It is also one of the features that makes programs written in Java less prone to bugs than those written in languages that don't support automatic GC.

Different JVM implementations handle GC in different ways, and the specifications do not

impose very stringent restrictions on how GC must be implemented.

Later in this chapter, we will discuss the GC algorithms available in the HotSpot JVM (which is the VM used in the OpenJDK-based implementations of Java, including Eclipse Adoptium and Oracle JDK). Although this is not the only JVM that you may encounter, it is by far the most common among server-side deployments and provides the reference example of a modern production JVM.

Let's take a moment to introduce a very simple form of GC, which is useful for grasping basic concepts, before we meet real, production GCs and learn about some of the detailed Java GC specifics.

## Introducing Mark-and-Sweep

Perhaps the most famous of GC algorithms is *mark-and-sweep*—which is really a family of closely related algorithms. Java's collectors all use a form of mark-and-sweep, and to understand these algorithms, let's look at the very simplest form.

Recall that all Java objects are created in the heap, and a reference (basically a pointer) to them is stored in a Java local variable (or field) when an object is created. Local variables live in the method's stack frame, and if an object is returned from a method, then the reference is passed back to the caller's stack frame when the method exits.

> **WARNING**
>
> The most basic form of GC is conceptually simple, but it is not practical for real applications for performance reasons. No real GC in a production-quality JVM uses anything close to this basic form.

All objects are allocated in the Java heap, so GC must trigger when the heap gets full (or slightly before, depending on the details). The basic idea of mark-and-sweep is to *trace* the heap and identify which objects are still in use.

This can be done by examining the stack frames of each Java thread (and a few other sources of references) and following any references into the heap. Each object located is *marked* as still alive and can then be checked to see if it has any fields that are reference types. If so, these references can be traced and marked as well. This process can be seen in Figure 9-1.

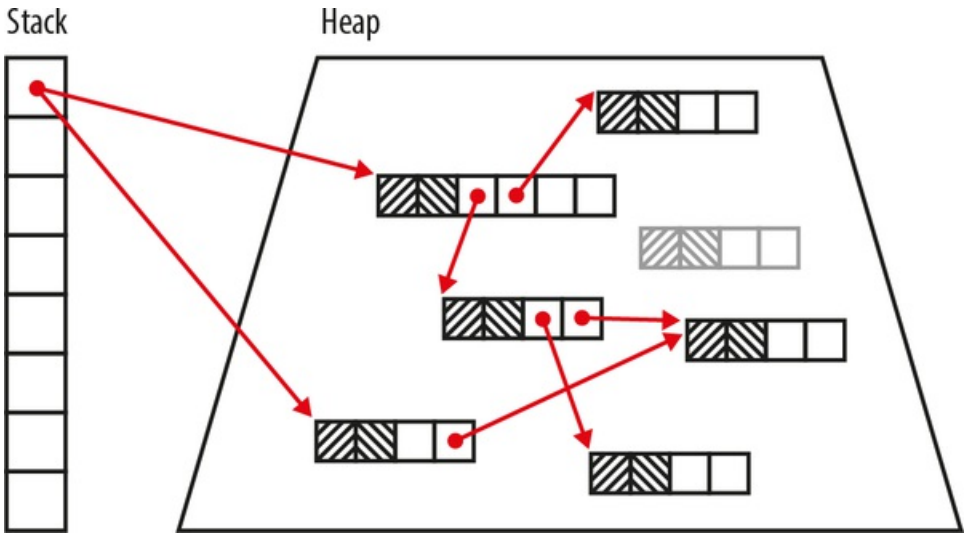



*Figure 9-1. Tracing through the heap*

When the tracing activity has completed, and there are no more references to be followed, then the heap has been *partitioned* into the *live* objects that the program still needs (because something, somewhere, has a reference to them—these objects are also referred to as *reachable*) and the *dead* (aka *unreachable*) objects that are no longer required.

> **TIP**
>
> The result of the partition is also known as the *transitive closure* of live objects, based on the mathematical principle of transitivity as applied to *graph theory*—two objects are connected if there is a path (of any length) from one to the other by following references.

The heap space occupied by the unmarked dead objects can be *swept* as garbage, i.e., the memory they used is reclaimed for use in further object allocations.

The fact that Java provides GC as a capability dramatically reduces the incidence of *memory leaks*. A memory leak occurs when memory is allocated and never reclaimed.

At first glance, it might seem that GC prevents all memory leaks because it reclaims all unused objects—but memory leaks can still occur in Java. The primary mechanism for memory leaks is: if a valid (but unused) reference to a no-longer-needed object is left hanging around.

For example, when a method runs for a long time (or forever), the local variables in that method can retain object references much longer than they are actually required. The following code illustrates this problem:

```
public static void main(String args[]) {
  int bigArray[] = new int[100000];

  // Do some computations with bigArray and get a result.
  int result = compute(bigArray);

  // We no longer need bigArray. It will get garbage collected when
```

```
    // there are no more references to it. Because bigArray is a local
    // variable, it refers to the array until this method returns. But
    // this method doesn't return.
    // If we explicitly sever the reference by assigning it to
    // null then the garbage collector knows it can reclaim the array.
    bigArray = null;

    // Loop forever, handling the user's input
    for(;;) handle_input(result);
  }
```

Memory leaks can also occur when you use a `HashMap` or a similar data structure to associate one object with another. Even when neither object is required anymore, the association remains in the map, preventing the objects from being reclaimed until the map itself is reclaimed. If the map has a substantially longer lifetime than the objects it holds, this can cause another common form of memory leak.

> **WARNING**
>
> Some older texts may refer to the `System.gc()` method to "force" the GC to run. However, the behavior of this method was never definitively specified—and it only advised the GC that the user had requested a collection cycle. It never guaranteed that a collection was actually performed. You should not use this method and should remove it if you encounter it in code you maintain.

Memory leaks can be especially problematic since they often grow over time and then manifest as crashes of your application. The crashes occur when a GC cycle is not recovering enough memory for the application to make meaningful forward progress. This results in an `OutOfMemoryError`, which is essentially an unrecoverable runtime error.

For these reasons, production systems usually monitor heap utilization to ensure that substantial memory leaks are not present. Let's move on to discuss some limitations of the naive form of mark-and-sweep before we see how to improve on it.

## Exactness and Limitations of Naive Mark-and-Sweep

If the heap partitioning step of a GC can be carried out exactly, then this type of collector is known, unsurprisingly enough, as an *exact garbage collector*.

On the other hand, if heuristics are used to determine whether objects are alive, they must be combined with some conservative assumptions about liveness, to avoid incorrect collections. These collectors are therefore known as *conservative garbage collectors*.

> **TIP**
>
> For practical purposes, all Java GCs can be considered to be exact, but this may not be true in other software environments (e.g., until relatively recently, some JavaScript implementations used a conservative GC).

To avoid collecting a still-live object (which would be catastrophic and must be avoided at all costs), a simple tracing GC like the one we have just described will cause a *stop-the-world* (STW) pause when it runs. This means that all application threads are stopped when possible, then GC occurs, and finally application threads are started up again.

This means that for the purposes of GC, the object graph is temporarily static—i.e., not being changed by executing application code—and so simple algorithms (such as depth-first traversal) can be used to mark the live objects.

The Java runtime enables this by halting each application thread as it reaches a *safepoint*—for example, the start of a loop or just before a method call returns. At these execution safepoints, the runtime knows that it can stop an application thread without damaging the data in the heap.

Performing a complete collection of the entire heap every time memory is exhausted is not performant enough for real JVMs—the pause times on nontrivial heap sizes would be prohibitive, even if multiple JVM threads were used for collection.

> **NOTE**
>
> The concept of Java pause times sometimes worries developers, but for most mainstream usages, Java is running on top of an operating system (and possibly multiple virtualization layers) that is constantly swapping processes on and off processor cores, so this slight additional stoppage is usually not a concern.

Instead of requiring a complete analysis of the heap on every collection (which would result in an object marking time that is linear in the heap size), a number of implementation enhancements are used to dramatically reduce pause time.

We will discuss several of these enhancements at a high level in this chapter. For production applications that may have to care about these details, specialist material such as *Optimizing Cloud Native Java* (O'Reilly) should be consulted, since a comprehensive discussion of the details of the GC subsystem is outside the scope of the current book.

## Improving on Basic Garbage Collection

One of the most important techniques available to improve on naive mark-and-sweep makes use of the fact that Java objects do not, in general, have constant lifetimes, and this is our next topic.

### The Weak Generational Hypothesis

The *weak generational hypothesis* (WGH) is a great example of one of the runtime facts about software that we introduced in Chapter 1. Simply put, it is that objects tend to have one of a small number of possible life expectancies (referred to as *generations*).

Usually objects are alive for only a very short amount of time (sometimes called *transient objects*) and then become eligible for garbage collection. However, some small fraction of objects live longer and are destined to become part of the longer-term state of the program

(sometimes referred to as the *working set*). This can be seen in Figure 9-2 where we see volume of memory (or number of objects created) plotted against expected lifetime.

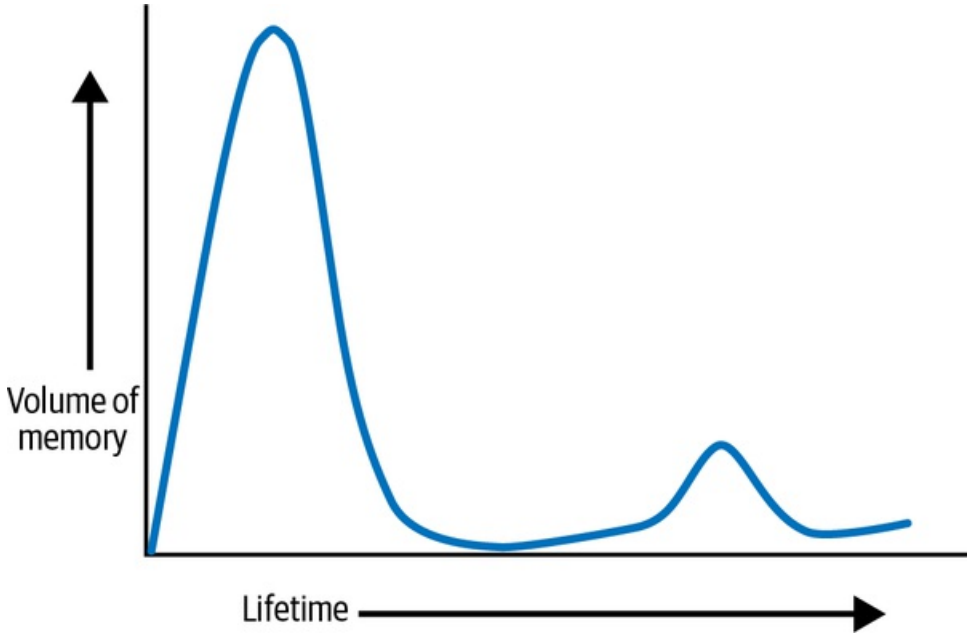



*Figure 9-2. Weak generational hypothesis*

This fact is not deducible from static analysis of programs, and yet when we measure the runtime behavior of software, we see that it is broadly true across a wide range of workloads.

The HotSpot JVM has a garbage collection subsystem that is designed specifically to take advantage of the WGH, and in this section, we will discuss how these techniques apply to short-lived objects (which is the majority case). This discussion is directly applicable to HotSpot, but other JVMs often employ similar or related techniques.

In its simplest form, a *generational garbage collector* is one that takes notice of the WGH. Such garbage collectors take the position that some extra bookkeeping to monitor memory will be more than paid for by gains obtained by being friendly to the WGH.

In the simplest forms of generational collectors, there are usually just two generations—often referred to as Young and Old (aka Tenured) generation. Even these are typically not performant enough for production use as is, and additional performance-related optimizations are used.

In a real JVM, there will be several different areas of heap memory, and real programs will use all of them in normal operation. In Figure 9-3 we show one possible layout of the heap, with two threads (T1 and T2) holding references that point into the heap.

Note that memory addresses are, of course, a one-dimensional space. The two-dimensional diagram is merely a visual aid for humans and has nothing to do with how memory is actually represented in the machine. The solid blocks represent surviving objects, and the hatched boxes represent allocated but now dead (and unreachable) objects.

The different areas are called *Eden*, *Survivor*, and *Tenured*; we'll meet each of these later in the chapter and see how they relate to one another. For now, let's note that Eden and the two

Survivor spaces make up the Young generation, and Tenured represents the Old generation.

The figure also illustrates that it would be dangerous to move objects that application threads have references to while the program is running. Consider what would happen if we were to relocate an object while a program was executing. We would be able to update the reference (i.e., the memory address) for the currently executing thread, but that thread would not know how many other threads also had a reference to the object that needed updating.

> **NOTE**
>
> For the sake of simplicity, the figures show an older form of the Java heap, where each memory area is a single, unbroken lump of memory. Modern collectors don't actually lay objects out this way, but it's easier to understand by thinking about it this way first!

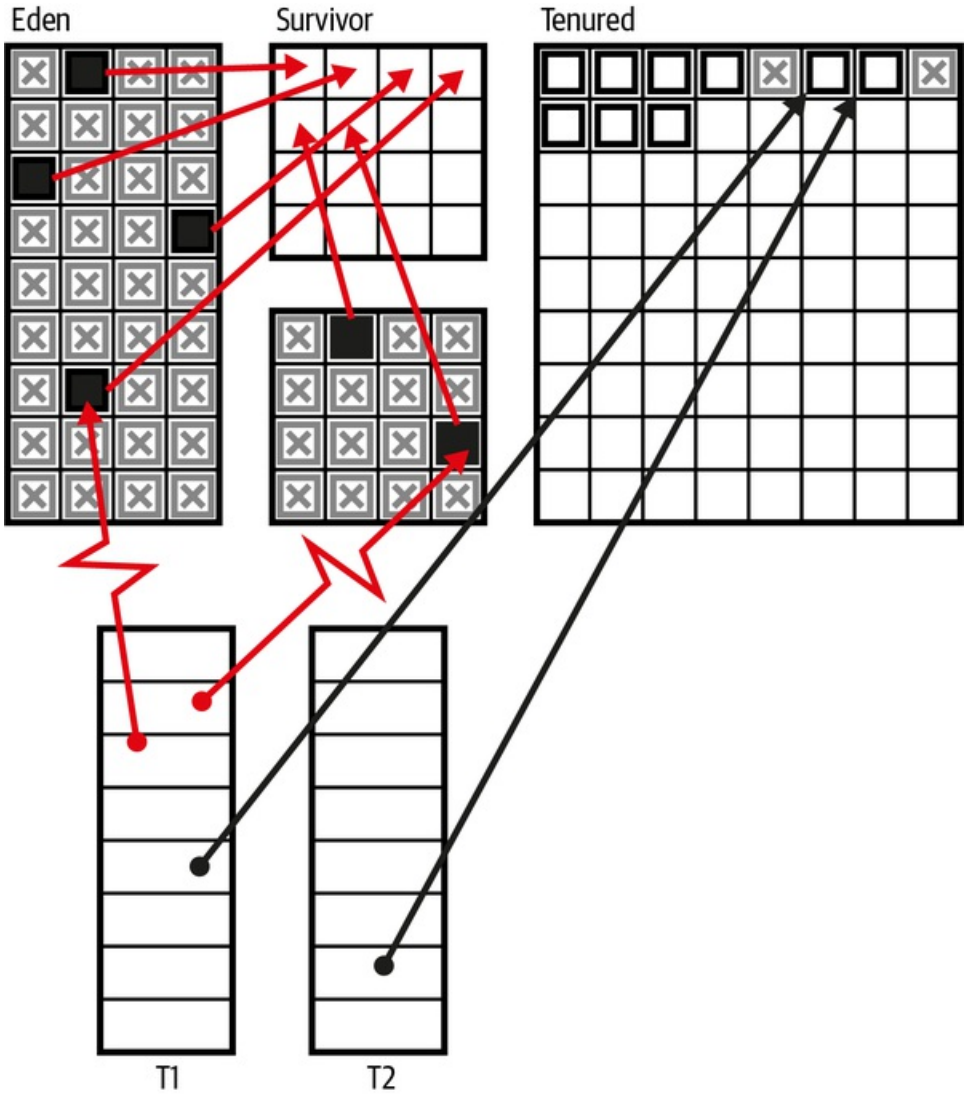



*Figure 9-3. Heap structure*

The WGH, and the observed fact of distinct groupings of object lifetimes, also provides a foundation for discussing techniques that trade the cost of moving certain objects with gains from additional optimizations. Let's take a closer look.

## Evacuation

In our original formulation of mark-and-sweep, during the cleanup phase, the GC reclaimed individual objects for reuse but did not touch the live objects. This is fine, as far as it goes, but it leads to issues such as memory fragmentation and the GC needing to maintain a "free list" of memory blocks that are available. However, if the WGH is true, and on any given GC cycle most objects are dead, then it may make sense to use an alternative approach to reclaiming space.

This works by dividing the heap into separate memory spaces; new objects are created in a space called *Eden*. Then, on each GC run, we locate only the live objects and move them to a different space, in a process called *evacuation*.

Collectors that do this are called *evacuating collectors*, and they have the property that the entire evacuated memory space can be wiped at the end of the collection, to be reused again and again. Figure 9-4 shows an evacuating collector in action.

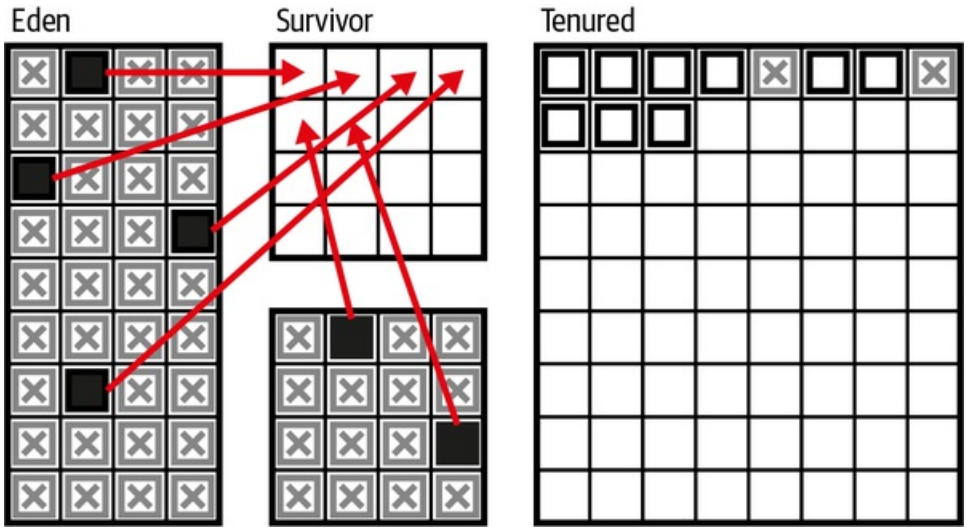



*Figure 9-4. Evacuating collectors*

This means that the GC time is proportional to the number of live objects, rather than the number of allocated objects. The only downside is slightly more bookkeeping—we have to pay the cost of copying the live objects, but this is almost always a very small price compared to the huge gains realized by evacuation strategies.

Using an evacuating collector also allows the use of *per-thread allocation*. This means each application thread can be given a contiguous chunk of memory (called a *thread-local allocation buffer*, or TLAB) for its exclusive use when allocating new objects.

When new objects are allocated, this just involves bumping a pointer in the allocation buffer, an extremely cheap (i.e., constant-time or O(1) operation). This is potentially much more efficient

than the naive collection approach, because the dead objects are never touched.

We should also look to address one potential problem: if an object is created just before a collection starts, then it will not have time to fulfill its purpose and will die before the GC cycle starts.

In a collector with only two generations, this short-lived object will be moved into the long-lived region, die almost immediately, and then stay there until the next full collection. This unnecessary promotion will cause full collections to happen more frequently than they would otherwise. As these are a lot more expensive, this is wasteful of CPU time and an obvious target for optimization in the JVM.

> **TIP**
>
> GC can be vastly cheaper than alternatives such as reference counting—a fact that surprises some developers.[3] This is due to the reality that short-lived objects are so cheap, because they pay the cost of a pointer bump to be allocated and nothing at all to be collected.

HotSpot uses a concept called *survivor spaces* to mitigate this—which is a holding area used to house objects that have survived previous collections of young objects. A surviving object is copied by the evacuating collector between survivor spaces until a *tenuring threshold* is reached, when the object will be *promoted* to the Old generation.

This solves (or greatly alleviates) the problem of short-lived objects cluttering up the Old generation, at the cost of more complexity in the GC subsystem. For the vast majority of workloads, this is a trade-off that is well worth making.

At this point, let's consolidate our description of the HotSpot heap as we have discussed it so far.

## The HotSpot Heap

The HotSpot JVM is a relatively complex piece of code, made up of an interpreter and a just-in-time compiler, as well as a user-space memory management subsystem. It is composed of a mixture of C, C++, and a fairly large amount of platform-specific assembly code.

HotSpot manages the JVM heap itself, more or less completely in user space, and does not need to perform system calls to allocate or free memory.[4] The basic features of HotSpot's memory management subsystem are:

- The Java heap is a contiguous block of memory, which is reserved at JVM startup.
- Only some of the heap is initially allocated to the various memory pools.
- As the application runs, memory pools are resized as needed.
- These resizes are performed by the GC subsystem.
- Objects are created in Eden by application threads and are removed by a

nondeterministic GC cycle.

- The GC cycle runs when necessary (i.e., when memory is getting low).
- The heap is divided into two generations, Young and Old.
- The Young generation is made up of Eden and Survivor spaces, whereas the Old generation is just one memory space.
- After surviving several GC cycles, objects get promoted to the Old generation.
- Collections that collect only the young generation are usually very cheap (in terms of computation required).
- Production JVMs use an evacuating strategy when collecting the Young generation.
- HotSpot uses an advanced form of mark-and-sweep and is prepared to do extra bookkeeping to improve GC performance.

When discussing garbage collectors, developers should know one other important terminology distinction:

*Parallel collector*

A garbage collector that uses multiple threads to perform collection

*Concurrent collector*

A garbage collector that can run at the same time that application threads are still running

In the discussion so far, the collection algorithms we have been describing have implicitly all been parallel, but not concurrent, collectors. For example, in Java 8 and earlier, the default collector for the Old generation is called *ParallelOld*, and we'll discuss it in more detail later in the chapter.

In modern approaches to GC, there is a growing trend toward using concurrent algorithms.[5] These types of algorithms are much more elaborate and computationally expensive than STW algorithms and involve trade-offs.

Today's applications are typically willing to trade some extra computation for reduced application pauses. In modern versions of HotSpot, a parallel and (partially) concurrent collection algorithm known as *Garbage First* (G1) has become the default, and it is the subject of the next section.

## G1

In legacy Java versions (i.e., 8 and older), the heap has a simple structure: each memory pool (Eden, Survivor, and Tenured) is a contiguous block of memory. This is the structure that we've shown in the diagrams, as it's easier for newcomers to visualize.

However, G1 is an example of a *region-based collector* and has a different heap layout from the old-style heap.

The reason for the change is that while the old default collector (ParallelOld) is incredibly efficient in terms of throughput, it is all-or-nothing. Once a ParallelOld GC cycle has started, it must run to completion, regardless of how long that takes. This leads to worst-case pause times that are roughly linear in the heap size, and with modern heap sizes, that can be a problem.

Region-based collectors have the option of collecting some garbage (by collecting some eligible regions but not all of them) and then allowing application threads to resume. Provided that the allocation rates do not overwhelm the collector, this can provide an upper limit on the pause time and is a major reason why G1 became the new default collector.

> **TIP**
>
> G1 uses a different version of the algorithm in each Java version, and there are some important differences in terms of performance and other behavior between versions.[6] You may find that when upgrading, e.g., from Java 11 to 21, you require fewer resources (and may even save money on your cloud bill).

A *region* is an area of memory (usually 1 MB in size, but larger heaps may have regions of 2, 4, 8, 16, or 32 MB) where all the objects belong to the same memory pool.[7] In a regional collector, the different regions that make up a pool are not necessarily located next to one another in memory, as we can see in Figure 9-5.

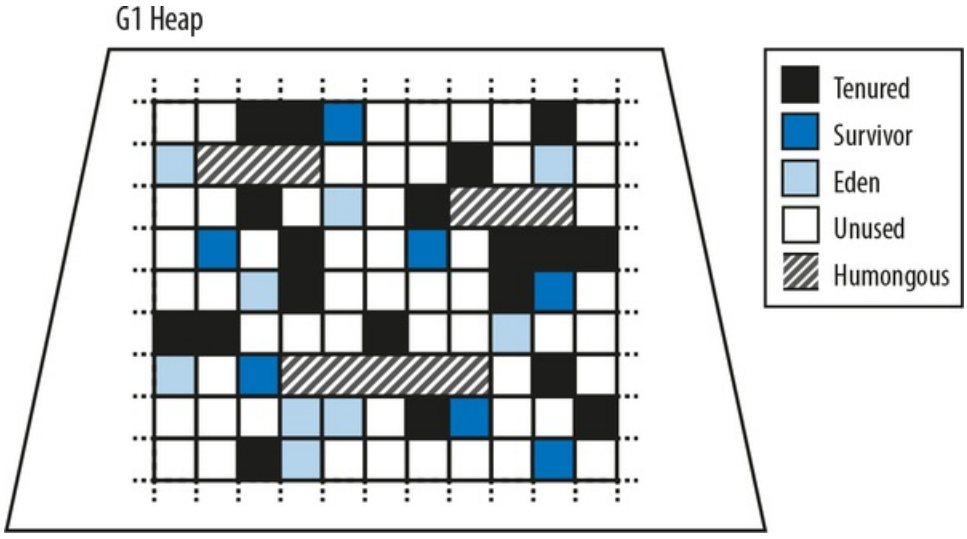



*Figure 9-5. Regional heap*

This is unlike the Java 8 heap, where each pool is contiguous, although in both cases the entire heap remains contiguous. G1 focuses its attention on regions that are mostly garbage, as they have the best free memory recovery. It is an evacuating collector and does two main types of collections—stop-the-world Young collections and mostly concurrent *mixed collections* that collect both generations.[8]

> **NOTE**
>
> The G1 collector was originally intended to take over from a previous collector (Concurrent-Mark-and-Sweep, or CMS) as the low-pause collector, but the overall characteristics of its behavior have meant that it has actually evolved into a more general-purpose collector (which is why it has now become the default).

The allocation rate of applications can be very unpredictable, even within the lifetime of an individual JVM process. This means it's difficult to produce a non-STW collection algorithm for short-lived objects, as the amount of memory headroom needed can vary so much, even during a single application run.

Instead, the idea is that young collections should be STW but very brief. This leads to the somewhat counterintuitive result that Java applications are performing young GC operations very frequently but that the duration of each pause is usually so short (submillisecond) that they aren't detectable above the background noise in real-world application performance.

The JVM provides a command-line switch that controls how long the G1 collector will aim to pause: `-XX:MaxGCPauseMillis=N`. It allows the user to specify *pause goals* in terms of how long and how often to pause when doing GC.[9]

The default pause time goal is 200 ms, but you can change this value depending on your needs. However, in practice, this switch is rarely needed—G1's pause times are usually much shorter than this. Those few applications that find G1's pause times to be a problem usually switch to one of the explicitly low-pause-time collectors (which we will meet later in this chapter) rather than extensively tune G1.[10]

In any case, there are limits to how far the collector can be pushed—you can't just set the pause goal to some extremely low value, as the JVM will not be able to accommodate it, and the result will very likely be much worse performance.

Two parameters that are occasionally tuned by application developers are the *Initiating Heap Occupancy Percent* (IHOP) threshold and the number of concurrent GC threads that will run alongside application threads during mixed collections.

G1 automatically determines IHOP based on earlier application behavior and has a default initial value of 45% (but this can be changed with a config switch `-XX:InitiatingHeapOccupancyPercent=45`), and the application will adjust up or down adaptively.[11]

One parameter that does occasionally need to be adjusted is `-XX:ConcGCThreads=n`, as it is possible that the autosizing can default to too many (or too few) threads. Too many threads can sometimes be created on a large, shared server (or a misconfigured container host). The opposite problem—too few GC threads—can arise in small containers where the available CPU is too low.[12]

To conclude our discussion of G1, we should note that the development of a new production-grade collector that is suitable for general use is not a quick process.

Garbage collectors are complex and general-purpose capabilities that must work acceptably over a very wide range of possible workloads and deployment parameters, and so the time and effort required to produce a new one is highly significant. It is a testament to Java's success that its GC capabilities have remained best in class for decades now, due to the investment and hard work provided by the OpenJDK team.

The original version of G1 shipped as Experimental with Java 6 and would cause occasional VM segfaults. With Java 7 it became officially supported, but its evolution would continue for a very long time after that. The version of G1 in Java 8 was "useful on some workloads" over Parallel but rarely competitive in practice for workloads that were already using the CMS low-pause collector—hence the evolution of G1 into a general-purpose collector.

A small piece of history: G1 became default in Java 9, so the first default version of G1 that came with an LTS release was the version in Java 11, which was substantially different from the version shipped in Java 8. In practice, there were still quite a few Java 11 workloads that struggled with G1, and that needed fallback to CMS or Parallel.

By Java 17 there were a much smaller number of workloads where G1 wasn't the best choice, and that number continued to decrease with Java 21 and 25. As of 2026, there are very few use cases for Parallel over G1 that aren't incredibly specific and niche—and CMS has been removed altogether.

Overall, the evolution and gradual development of G1 that we've just described took ~15 years.

In the next section, let's move on to discuss the alternative collectors that are provided by HotSpot (including ParallelOld, the default collector of Java 8).

## Alternate Collectors

The vast majority of Java workloads will not need to address the question of the choice of collector—or even need to adjust GC settings at all. However, a small number of workloads will benefit from choosing an alternative collector with different characteristics.

A detailed full treatment is outside the scope of the book, but it is worth knowing about the existence of alternate collectors. For non-HotSpot users, you should consult your JVM's documentation to see what options may be available for you.

### ParallelOld

By default, in Java 8 the collector for the Old generation is a parallel (but not concurrent) mark-and-sweep collector. This ParallelOld collector is the most efficient in terms of throughput, but it has two properties that make it less desirable for modern applications. Specifically, it is:

- Fully STW
- Linear in pause time with the size of the heap

This means that once GC has started, it cannot be aborted early, and the cycle must be allowed to

finish. As heap sizes increase, this makes ParallelOld a less attractive option than G1, which can often keep a constant pause time regardless of heap size (assuming the allocation rate is manageable).

In many ways ParallelOld seems, at first glance, to be similar to the collector used for the Young generation. However, it differs in one very important respect: it is *not* an evacuating collector. Instead, the old generation collector is what is known as a *compacting collector*.

The main feature of these collectors is that, at the end of the collection cycle, allocated memory (i.e., surviving objects) is arranged as a single contiguous area within the collected region. This is important so that the memory space does not become fragmented over time.

The normal case is that all the surviving objects have been "shunted up" within the memory pool (or region) usually to the start of the memory range, and there is now a pointer indicating the start of empty space available for objects to be written into once application threads restart.

Compacting collectors will avoid memory fragmentation but typically are much more expensive in terms of amount of CPU consumed than evacuating collectors. There are design trade-offs between the two approaches (the details of which are beyond the scope of this book), but both techniques are used in production collectors in Java (and in many other programming languages).[13]

The ParallelOld collector is still available as of Java 25, for those (hopefully few) apps that still need it, but the direction of the platform is clear—toward using G1 everywhere possible. This means that before using ParallelOld, careful testing should be performed as G1 typically gives better performance on a large majority of applications.

## Serial

The Serial and SerialOld collectors operate in a similar fashion to the Parallel collectors, with one important difference: they use only a single CPU core to perform fully STW GC.

On modern multicore systems, there is no benefit from using these collectors, and so they should not be used, as they are just an inefficient form of the parallel collectors. However, one place where you may still encounter these collectors is when running Java applications in containers. A full discussion of containerized Java is outside the scope of this book. However, if your application is run in too small a container (either too little memory or with only a single CPU), then the JVM will automatically select the Serial collector.[14]

Therefore, we do not recommend running Java in a single-core container, as the Serial collector performs noticeably worse than G1 under almost all realistic load scenarios.

## Shenandoah

Shenandoah is a new GC algorithm developed by Red Hat and other community members to work effectively with certain use cases where G1 and other algorithms do not perform well.

The aim of Shenandoah is to bring down pause times, especially on large heaps, and to guarantee

(as far as possible) that pause times will not exceed 1 ms, no matter the size of the heap.

> **WARNING**
>
> Developers sometimes overestimate their need for a low-pause collector. You should always start by benchmarking your application using G1, unless you have some external non-functional requirement (e.g., a very large heap size, such as > 20 GB+) that indicates you will need low-pause capabilities.

There is no such thing as a free lunch, however, and users of Shenandoah could experience a significant CPU overhead (i.e., reduction in application throughput) as well as increased memory usage. The exact numbers for the additional costs will depend on the details of the workload. For example, on some targeted benchmarks, you can observe up to 15% CPU overhead, toward the upper end of the expected range.

In general, the Java 17 version of Shenandoah should be considered production-ready (as are Red Hat's backports, if you're a Red Hat support customer). However, it's worth being reminded that it is not considered a general-purpose collector—it is intended for use on large heaps only.

Like G1, Shenandoah is an evacuating regional collector that performs concurrent marking. The evacuation of regions causes incremental compaction, but the key difference is that in G1, evacuation happens during an STW phase, whereas in Shenandoah the evacuation is concurrent with application threads.

Shenandoah can be activated with this command-line switch:

```
-XX:+UseShenandoahGC
```

One important point to note is that, at time of writing, Shenandoah is not a generational collector by default. However, for Java 25 onward, generational mode can be activated via a command-line switch, `-XX:ShenandoahGCMode=generational`.

## ZGC

As well as Shenandoah, Oracle has also created a new ultra-low-pause collector, known as ZGC. It is designed to appeal to the same sorts of workloads as Shenandoah and is broadly similar in intent, effect, and overhead.

ZGC is a generational, region-based, NUMA-aware, compacting collector.[15] However, the implementation of ZGC is quite different from Shenandoah.

ZGC can be activated with this command-line switch:

```
-XX:+UseZGC
```

ZGC needs only a stop-the-world pause to perform root scanning, which means that GC pause times do not increase with the size of the heap or the number of live objects. The official goal is

that pause time should never exceed 1 millisecond on heaps up to 1 TB (ZGC supports heap sizes up to 16 TB). Due to its intended domain of applicability (ultra-low pause on large heaps), ZGC is most commonly used by Oracle customers on the Oracle-supported builds of Java.

Just as for G1, the new low-pause collectors (Shenandoah and ZGC) have both been on an evolutionary journey of improvement. Some of the key technologies and techniques that low-pause collection relies upon were initially implemented by the ZGC team and then utilized by both collectors, for example:

- Java 14: Self-fixing barriers
- Java 14: Concurrent class unloading
- Java 15: Concurrent reference processing
- Java 17: Concurrent thread-stack processing

This ongoing delivery of low-level improvements to the JVM benefits not only the garbage collectors but other subsystems (e.g., profiling and telemetry) and is the subject of ongoing work in OpenJDK.

## Summary

In this chapter, we've discussed Java's view of memory and automatic memory management. Garbage collection is one of the major aspects of the platform that simplifies development by removing the need for programmers to manually manage memory in the Java heap.

We've explained core concepts such as reachability, pause time, and generational collection. We have seen how Java provides advanced GC capabilities and how modern versions of Java use the partially concurrent G1 collector by default. We've explored alternative collectors that may be useful for certain special-case workloads, such as ParallelOld, Shenandoah, and ZGC.

This is a huge subject, which the interested reader can explore at much greater length. For now, we should move on to our next topic, which covers handling a selection of common data formats.

---

1 Some texts regard reference counting as a form of GC, but we prefer the approach of regarding GC as an activity that is mostly independent from the core application processing, following Dijkstra's work.

2 Rust also provides the `Arc` capability for threadsafe reference counting if necessary.

3 The fact that tracing GC and automatic reference counting (RC) live in different complexity classes is one compelling reason why we don't subscribe to the argument that both should be called *garbage collection* when the term *automatic memory management* is available.

4 The heap is initially reserved but not necessarily fully mapped to physical memory. System calls will be required when new physical pages are needed.

5 Technically speaking, these are correctly referred to as *partially concurrent* collectors, as they do still have very short pauses but the vast majority of the GC work is done concurrently with application threads.

6 In particular, it is very important that, if upgrading from Java 8 to a later version and adopting G1, you undertake a full performance retest.

7 G1 also features special *humongous regions* that are used to store large objects (those that occupy more than half a region). The typical use case for this is large on-heap arrays.

8 G1's concurrent collections make use of a classic technique known as *tri-color marking* from the classic paper by Edsger W. Dijkstra, Leslie Lamport, A. J. Martin, C. S. Scholten, and E. F. M. Steffens, "On-the-Fly Garbage Collection: An Exercise in Cooperation," *Communications of the ACM* 21 (1978): 966–975.

9 This was part of the original intention for G1 to be a replacement low-pause collector.

10 There is no such thing as a free lunch, and those workloads that cannot tolerate G1's short pause times will pay for it in increased CPU utilization and ultimately in bigger cloud hosting bills.

11 Adaptive IHOP can also be switched off altogether, but this is an advanced tuning option and not generally recommended.

12 This can even cause the JVM to select the Serial collector rather than G1, which is extremely rarely the best choice.

13 For example, G1 can be considered as doing *incremental compaction* when evacuating individual regions.

14 See the Red Hat Developer website for more details.

15 Generational mode of ZGC is available from Java 21 onward, and is the only available mode for Java 25.

# Chapter 10. Handling Common Data Formats

Most of programming is handling data in various formats. In this chapter, we will introduce Java's support for handling two big classes of data—text and numbers.

The second half of the chapter will focus on handling date and time information. These APIs received a major update during Java 8, and we'll discuss the modern APIs in detail along with some pertinent information about the older form, which you may still encounter. We encourage updating to the new APIs for dates and times whenever possible.

Before we get to those complexities, let's talk about textual data and strings.

## Text

We have already met Java's strings on many occasions. They consist of sequences of Unicode characters and are represented as instances of the `String` class. Strings are one of the most common types of data that Java programs process (a claim you can investigate for yourself by using the `jmap` tool that we'll meet in Appendix B).

In this section, we'll meet the `String` class in some more depth and understand why it is in a unique position within the Java language. Later in the section, we'll introduce regular expressions, a very common abstraction for searching text for patterns (and a classic tool in the programmer's arsenal, regardless of language).

### Special Syntax for Strings

The `String` class is handled in a special way by the Java language. This is because, despite not being a primitive type, strings are so common that it makes sense for Java to have syntax features designed to make handling strings easy. Let's look at some examples of what Java provides.

#### String literals

As we saw in Chapter 2, Java allows a sequence of characters to be placed in double quotes to create a literal string object. Like this:

```
String pet = "Cat";
```

Without this syntax, we would have to write copious amounts of horrible code like this:

```
char[] pullingTeeth = {'C', 'a', 't'};
String pet = new String(pullingTeeth);
```

This would get tedious extremely quickly, so it's no surprise that Java, like all modern programming languages, provides a simple string literal syntax. The string literals are perfectly sound objects, so code like this is completely legal:

```
System.out.println("Dog".length());
```

Strings using basic double quotes cannot span multiple lines, but recent versions of Java have included multiline text blocks with the `"""` syntax. The resulting string objects are created at compile time and are no different from a `"` quoted string, just easier to express:

```
String lyrics = """
  This is the song that never ends
  This song goes on and on my friend
  ...""";
```

See "String literals" for complete coverage of string literals in Java.

### toString()

This method is defined on `Object` and is designed to allow easy conversion of any object to a string. This makes it easy to print out any object, by using the method `System.out.println()`. This method is actually `PrintStream::println` because `System.out` is a static field of type `PrintStream`. Let's see how this method is defined:

```
public void println(Object x) {
    String s = String.valueOf(x);
    synchronized (this) {
        print(s);
        newLine();
    }
}
```

This creates a new string by using the static method `String::valueOf()`:

```
public static String valueOf(Object obj) {
    return (obj == null) ? "null" : obj.toString();
}
```

> **NOTE**
>
> The static `valueOf()` method is used instead of `toString()` directly, to avoid a `NullPointerException` in the case where `obj` is `null`.

This construction means that `toString()` is always available for any object, and this comes in very handy for another major syntax feature that Java provides: string concatenation.

### String concatenation

Java allows us to create new strings by "adding" the characters from one string onto the end of another. This is called *string concatenation* and uses the operator `+`. Note that unlike other languages such as C++, operators cannot be overloaded in Java, so this support for `+` on strings is a special case in the language itself.

In versions of Java up to and including Java 8, the `+` operator works by first creating a "working area" in the form of a `StringBuilder` object that contains the same sequence of characters as the original string.

> **NOTE**
>
> Java 9 introduced a new mechanism that directly uses the `invokedynamic` instruction instead of `StringBuilder`. This is an advanced piece of functionality and out of scope for this discussion, but it doesn't change the behavior visible to the Java developer.

The builder object is then updated, and the characters from the additional string are added onto the end. Finally, `toString()` is called on the `StringBuilder` object (which now contains the characters from both strings). This gives us a new string with all the characters in it. All of this code is created automatically by `javac` whenever we use the `+` operator to concatenate strings.

The concatenation process returns a completely new `String` object, as we can see in this example:

```
String s1 = "AB";
String s2 = "CD";

String s3 = s1;
System.out.println(s1 == s3); // Same object? Yes.

s3 = s1 + s2;
System.out.println(s1 == s3); // Still same? Nope!
System.out.println(s1);       // Prints "AB"
System.out.println(s3);       // Prints "ABCD"
```

The concatenation example directly shows that the `+` operator is not altering (or *mutating*) `s1` in place. This is an example of a more general principle: Java's strings are immutable. This means that once the characters that make up the string have been chosen and the `String` object has been created, the `String` cannot be changed. This is an important language principle in Java, so let's look at it in a little more depth.

## String Immutability

To "change" a string, as we saw when we discussed string concatenation, we actually need to create an intermediate `StringBuilder` object to act as a temporary scratch area and then call `toString()` on it, to bake it into a new instance of `String`. Let's see how this works in

code:

```
String pet = "Cat";
StringBuilder sb = new StringBuilder(pet);
sb.append("amaran");
String boat = sb.toString();
System.out.println(boat);   // Prints "Catamaran"
```

Such code behaves like the following, although in Java 9 and above the actual bytecode sequences will differ:

```
String pet = "Cat";
String boat = pet + "amaran";
System.out.println(boat); // Prints "Catamaran"
```

As well as being used under the hood by `javac`, the `StringBuilder` class can also be used directly in application code, as we've seen.

> **WARNING**
>
> Along with `StringBuilder`, Java also has a `StringBuffer` class. This comes from the oldest versions of Java and synchronizes itself to protect against multithreaded scenarios, which has performance implications. `StringBuffer` should not be used for new development—use `StringBuilder` instead, unless you really need to share *construction* of a new string between multiple threads.

String immutability is an extremely useful language feature. For example, suppose the `+` changed a string instead of creating a new one; then, whenever any thread concatenated two strings, all other threads would also see the change. This is unlikely to be a useful behavior for most programs, and so immutability makes good sense.

### Hash codes and effective immutability

We have already met the `hashCode()` method in Chapter 6, where we described the contract that the method must satisfy. Let's take a look at the JDK source code and see how the method `String::hashCode()` is defined:

```
public int hashCode() {
    int h = hash;
    if (h == 0 && value.length > 0) {
        char val[] = value;

        for (int i = 0; i < value.length; i++) {
            h = 31 * h + val[i];
        }
        hash = h;
    }
    return h;
}
```

The field `hash` holds the hash code of the string, and the field `value` is a `char[]` that holds the characters that actually make up the string. As we can see from the code, Java computes the hash by looping over all the characters of the string. It therefore takes a number of machine instructions proportional to the number of characters in the string. For very large strings, this could take a bit of time. Rather than pre-compute the hash value, Java calculates it only when it is needed.

When the method runs, the hash is computed by stepping through the array of characters. At the end of the array, we exit the `for` loop and write the computed hash back into the field `hash`. Now, when this method is called again, the value has already been computed, so we can just use the cached value and subsequent calls to `hashCode()` return immediately.

> **NOTE**
>
> The computation of a string's hash code is an example of a *benign data race*. In a program with multiple threads, they could race to compute the hash code. However, they would all eventually arrive at exactly the same answer—hence the term *benign*.

All of the fields of the `String` class are final, except for `hash`. So Java's strings are not, strictly speaking, immutable. However, because the `hash` field is just a cache of a value that is deterministically computed from the other fields, which are all immutable then, provided `String` has been coded correctly, it will behave as if it were immutable. Classes that have this property are called *effectively immutable*, which means they are quite rare in practice, and working programmers can usually ignore the distinction between truly immutable and effectively immutable data.

## String Formatting

In "String concatenation", we saw how Java supports building strings from smaller strings by joining them. While this works, it can often be tedious and error prone when constructing more elaborate output strings. Java provides a number of other methods and classes for doing richer string formatting.

The static method `format` on the `String` class allows us to specify a template and then dynamically plug in various values:

```
// Result is the string "The 1 pet is a cat: true?"
var s = String.format("The %d pet is a %s: %b?%n", 1, "cat", true);

// Same result, but called on string instance instead of statically
s = "The %d pet is a %s: %b?%n".formatted(1, "cat", true);
```

Placeholders in the format string where values will be introduced start with the `%` character. In this example, we substitute an integer with `%d`, a string with `%s`, a boolean with `%b`, and finally end the string with a newline via `%n`. If a literal `%` is needed in the output, it can be included with

%%.

Those with a background in C or similar languages will recognize this format from the venerable `printf` function. Java supports many, though not all, of the same formats with a wide variety of options. Java's `printf` also provides more sophisticated date and time formatting as seen in C's `strftime` function. See the Java documentation on `java.util.Formatter` for the full list of options available.

Java also improves on the experience using these format strings by throwing exceptions on invalid conditions such as mismatched numbers of placeholders to values, or unrecognized % values.

`String.format()` provides powerful tools for constructing complex strings, but, particularly when making output correct across countries, more assistance is needed. `NumberFormat` is an example of classes Java provides to support more complex, locale-aware formatting of values. Other formatters are also available under `java.text`:

```
// Some common locales are available as constants
// A much longer list can be accessed at runtime
var locale = Locale.US;

NumberFormat.getNumberInstance(locale).format(1_000_000_000L)
// 1,000,000,000

NumberFormat.getCurrencyInstance(locale).format(1_000_000_000L)
// $1,000,000,000.00

NumberFormat.getPercentInstance(locale).format(0.1)
// 10%

NumberFormat.getCompactNumberInstance(locale, NumberFormat.Style.LONG)
            .format(1_000_000_000L)
// 1 billion

NumberFormat.getCompactNumberInstance(locale, NumberFormat.Style.SHORT)
            .format(1_000_000_000L)
// 1B
```

## Regular Expressions

Java has support for *regular expressions* (often shortened to *regex* or *regexp*). These are a representation of a search pattern used to scan and match text. A regex is a sequence of characters that we want to search for. Regex patterns can be very simple—for example, `abc` means that we're looking for *a*, followed immediately by *b*, followed immediately by *c*, anywhere within the text we're searching. Note that a search pattern may match an input text in zero, one, or more places.

The simplest regexes are just sequences of literal characters, like `abc`. However, the language of regexes can express more complex and subtle ideas than just literal sequences. For example, a regex can represent patterns to match:

- A numeric digit
- Any letter
- Any number of letters, which must all be in the range *a* to *j* but can be upper- or lowercase
- *a* followed by any four characters, followed by *b*

The syntax we use to write regular expressions is simple, but because we can build complex patterns, it is often possible to write an expression that does not implement precisely what we wanted. When using regexes, it is very important to always test them fully. This should include both test cases that should pass and cases that should fail.

To express these more-complex patterns, regexes use *metacharacters*. These are characters that indicate special processing is required. They can be thought of as similar to the use of the `*` character in operating system shells. In those circumstances, it is understood that the `*` is not to be interpreted literally but instead means "anything." If we wanted to list all the Java source files in the current directory on Unix, we would issue the command:

```
ls *.java
```

The metacharacters of regexes are similar, but there are far more of them, and they are far more flexible than the set available in shells. They also have different meanings from the ones in shell scripts, so don't get confused.

> **WARNING**
>
> Many different flavors of regular expression patterns exist in the world. Java's is PCRE-compatible, supporting a common set of metacharacters popularized by the Perl programming language. Be aware though that a random regex found online may or may not actually work, depending on which regex libraries you are using.

Let's meet a couple of examples. Suppose we want to have a spell-checking program that accommodates differences between British and American English. This means that *honor* and *honour* should both be accepted as valid spelling choices. This is easy to do with regular expressions.

Java uses a class called `Pattern` (from the package `java.util.regex`) to represent a regex. This class can't be directly instantiated, however. Instead, new instances are created by using a static factory method, `compile()`. Compiling a regex is a relatively expensive operation compared to using the resulting instance to match text, so it's common practice to compile a regex once into a static field rather than at every point of use.

From a pattern, we then derive a `Matcher` for a particular input string that we can use to explore the input string. For example, let's examine a bit of Shakespeare from the play *Julius Caesar*:

```
Pattern p = Pattern.compile("honou?r");

String caesarUK = "For Brutus is an honourable man";
Matcher mUK = p.matcher(caesarUK);

String caesarUS = "For Brutus is an honorable man";
Matcher mUS = p.matcher(caesarUS);

System.out.println("Matches UK spelling? " + mUK.find());
System.out.println("Matches US spelling? " + mUS.find());
```

> **WARNING**
>
> Be careful when using `Matcher`, as it has a method called `matches()`. However, this method indicates whether the pattern can cover the entire input string. It will return `false` if the pattern starts matching only in the middle of the string.

The last example introduces our first regex metacharacter `?`, in the pattern `honou?r`. This means "the preceding character is optional"—so both `honour` and `honor` will match. Let's look at another example. Suppose we want to match both *minimize* and *minimise* (the latter spelling is more common in British English). We can use square brackets to indicate that any character from a set (but only one alternative) `[]` can be used—like this:

```
Pattern p = Pattern.compile("minimi[sz]e");
```

Table 10-1 provides an expanded list of metacharacters available for Java regexes.

*Table 10-1. Regex metacharacters*

| Metacharacter | Meaning | Notes |
|---|---|---|
| `?` | Optional character—zero or one instance | |
| `*` | Zero or more of preceding character | |
| `+` | One or more of preceding character | |
| `{M,N}` | Between `M` and `N` instances (inclusive) of preceding character | |
| `\d` | A digit | |
| `\D` | A nondigit character | |

| | | |
|---|---|---|
| `\w` | A word character | Digits, letters, and _ |
| `\W` | A nonword character | |
| `\s` | A whitespace character | |
| `\S` | A nonwhitespace character | |
| `\n` | Newline character | |
| `\r` | Carriage return character | |
| `\R` | Any Unicode line break sequence | |
| `\t` | Tab character | |
| `.` | Any single character | Does not include newline in Java |
| `[ ]` | Any character contained with the brackets | Called a character class |
| `[^ ]` | Any character not contained with the brackets | Called a negated character class |
| `( )` | Build up a group of pattern elements | Called a group (or capturing group) |
| `\|` | Define alternative possibilities | Implements logical `OR` |
| `^` | Start of string | |
| `$` | End of string | |
| `\\` | Literal escape (`\`) char | |

There are a few more, but this is the basic list. The `java.util.regex.Pattern` Java documentation is a good source for all the details. From this, we can construct more complex expressions for matching, such as the examples given earlier in this section:

```
String text = "Apollo 13";

// A numeric digit. Note we must use \\ because we need a literal \
// and Java uses a single \ as an escape character, as per the table
Pattern p = Pattern.compile("\\d");
Matcher m = p.matcher(text);
// Prints "\d matches Apollo 13? true ; match: 1"
System.out.print(p + " matches " + text + "? " + m.find());
System.out.println(" ; match: " + m.group());

// A single letter
p = Pattern.compile("[a-zA-Z]");
m = p.matcher(text);
// Prints "[a-zA-Z] matches Apollo 13? true ; match: A"
System.out.print(p + " matches " + text + "? " + m.find());
System.out.println(" ; match: " + m.group());

// Any number of letters, which must all be in the range 'a' to 'j'
// but can be upper- or lowercase
p = Pattern.compile("([a-jA-J]*)");
m = p.matcher(text);
// Prints "([a-jA-J]*) matches Apollo 13? true ; match: A"
System.out.print(p + " matches " + text + "? " + m.find());
System.out.println(" ; match: " + m.group());

// 'a' followed by any four characters, followed by 'b'
text = "abacab";
p = Pattern.compile("a....b");
m = p.matcher(text);
// Prints "a....b matches abacab? true ; match: abacab"
System.out.print(p + " matches " + text + "? " + m.find());
System.out.println(" ; match: " + m.group());
```

Regexes are extremely useful for determining when a string matches a given pattern, but they also allow for extracting bits and pieces from the strings as well. This is done through the *group* mechanism, which is represented in the patterns by `()`:

```
String text = "Apollo 13";

Pattern p = Pattern.compile("Apollo (\\d*)");
Matcher m = p.matcher(text);
System.out.print(p + " matches " + text + "? " + m.find());
System.out.println("; mission: " + m.group(1));
```

The call to `Matcher.group(1)` returns the text that the regex matched in the `(\\d*)` of our pattern. Multiple groups are allowed, along with syntax for naming groups rather than using them by position. See the Java documentation for full details.

A common difficulty working with regular expressions is the need to use escape characters for both the Java string and the regular expression. Where text blocks have less escaping—such as quote characters—they can provide for less cluttered expressions:

```
// Detect if there are any double-quoted passages in string
// Note standard string literal requires escaping quotations
Pattern oldQuoted = Pattern.compile(".*\".*\".*");
```

```
Pattern newQuoted = Pattern.compile("""
                                    .*".*".*""");
```

Let's conclude our quick tour of regular expressions by meeting a method that was added to `Pattern` as part of Java 8: `asPredicate()`. This method is present to allow us to easily bridge from regular expressions to the Java Collections and their support for lambda expressions.

For example, suppose we have a regex and a collection of strings. It's very natural to ask the question "Which strings match against the regex?" We do this by using the filter idiom and by converting the regex to a `Predicate` using the helper method, like this:

```
// Contains a numeric digit
Pattern p = Pattern.compile("\\d");

List<String> ls = List.of("Cat", "Dog", "Ice-9", "99 Luftballoons");
List<String> containDigits = ls.stream()
        .filter(p.asPredicate())
        .toList();

System.out.println(containDigits);
```

Java's built-in support for text processing is more than adequate for the majority of text-processing tasks that business applications normally require. More advanced tasks, such as searching and processing very large data sets, or complex parsing (including formal grammars), are outside the scope of this book, but Java has a large ecosystem of helpful libraries and bindings to specialized technologies for text processing and analysis.

# Numbers and Math

In this section, we will discuss Java's support for numeric types in some more detail. In particular, we'll discuss the two's complement representation of integral types that Java uses. We'll introduce floating-point representations and touch on some of the problems they can cause. We'll also work through examples that use some of Java's library functions for standard mathematical operations.

## How Java Represents Integer Types

Java's integer types are all signed, as we first mentioned in "Primitive Data Types". This means that all integer types can represent both positive and negative numbers. As computers work with binary, this means that the only really logical way to represent this is to split the possible bit patterns and use half of them to represent negative numbers.

Let's work with Java's `byte` type to investigate how Java represents integers. This has 8 bits and thus can represent 256 different numbers (i.e., 128 negative and 128 nonnegative numbers). It's logical to use the pattern `0b0000_0000` to represent zero (recall that Java has the syntax `0b<binary digits>` to represent numbers as binary), and then it's easy to figure out the bit

patterns for the positive numbers:

```
byte b = 0b0000_0001;
System.out.println(b); // 1

b = 0b0000_0010;
System.out.println(b); // 2

b = 0b0000_0011;
System.out.println(b); // 3

// ...

b = 0b0111_1111;
System.out.println(b); // 127
```

When we set the most significant bit of the byte, the sign should change (as we have now used up all of the bit patterns that we've set aside for nonnegative numbers). So the pattern `0b1000_0000` should represent some negative number—but which one?

> **NOTE**
>
> As a consequence of how we've defined things, in this representation we have a very simple way to identify whether a bit pattern corresponds to a negative number: if the high-end bit of a bit pattern is a `1`, then the number being represented is negative.

Consider the bit pattern consisting of all set bits: `0b1111_1111`. If we add 1 to this number, then the result will overflow the 8 bits of storage that a `byte` has, resulting in `0b1_0000_0000`. If we want to constrain this to fit within the `byte` data type, then we should ignore the overflow, so this becomes `0b0000_0000`, otherwise known as zero. It is therefore natural to adopt the representation that "all set bits represent -1." This allows for natural arithmetic behavior, like this:

```
b = (byte) 0b1111_1111;
System.out.println(b);  // -1
b++;
System.out.println(b);  // 0 (0b0000_0000)

b = (byte) 0b1111_1110;
System.out.println(b);  // -2
b++;
System.out.println(b);  // -1 (0b1111_1111)
```

Finally, let's look at the number that `0b1000_0000` represents. It's the most negative number that the type can represent, so for `byte`:

```
b = (byte) 0b1000_0000;
System.out.println(b); // -128
```

This representation, called *two's complement*, is the most common representation for signed integers. To use it effectively, you need to remember only two points:

- A bit pattern of all `1`s is the representation for −1.
- If the high bit is set, the number is negative.

Java's other integer types (`short`, `int`, and `long`) behave in very similar ways but with more bits in their representation. The `char` data type is different because it represents a Unicode character, but in some ways it behaves as an unsigned 16-bit numeric type. It is not normally regarded as an integer type by Java programmers.

## Java and Floating-Point Numbers

Computers represent numbers using binary. We've seen how Java uses the two's complement representation for integers. But what about fractions or decimals? Java, like almost all modern programming languages, represents them using *floating-point arithmetic*. Let's take a look at how this works, first in base-10 (regular decimal) and then in binary. Java defines the two most important mathematical constants, *e* and π (pi), as constants in `java.lang.Math` like this:

```
public static final double E = 2.7182818284590452354;
public static final double PI = 3.14159265358979323846;
```

Of course, these constants are actually *irrational numbers* and cannot be precisely expressed as a fraction, or by any finite decimal number.[1] This means that whenever we try to represent them in a computer, there is always rounding error. Let's suppose we want to deal with only nine digits of π, and we want to represent the digits as a whole number. We can use a representation like this:

$314159265 \times 10^{-8}$

This starts to suggest the basis of how floating-point numbers work. We use some of the bits to represent the significant digits (`314159265`, in our example) of the number and some bits to represent the *exponent* of the base (-8, in our example). The collection of significant digits is called the *significand*, and the exponent describes whether we need to shift the significand up or down to get to the desired number.

Of course, in the examples we've met so far, we've been working in base-10. Computers use binary, so we need to use the number 2 as the base in our floating-point examples. This introduces some additional complications.

> **NOTE**
>
> The number `0.1` cannot be expressed as a finite sequence of binary digits. This means that virtually all calculations that humans care about will lose precision when performed in floating point, and rounding error is essentially inevitable.

Let's look at an example that shows the rounding problem:

```
double d = 0.3;
System.out.println(d); // Special-cased to avoid ugly representation

double d2 = 0.2;
// Should be -0.1 but prints -0.09999999999999998
System.out.println(d2 - d);
```

The official standard that describes floating-point arithmetic is IEEE 754, and Java's support for floating point is based on that standard. The standard uses 24 binary digits for standard precision and 53 binary digits for double precision.

As we mentioned briefly in Chapter 2, Java previously allowed deviation from this standard, resulting in greater precision when some hardware features were used to accelerate calculations. As of Java 17, this is no longer allowed, and all floating-point operations comply with the IEEE 754 standard.

### BigDecimal

Rounding error is a constant source of headaches for programmers who work with floating-point numbers. In response, Java has a class `java.math.BigDecimal` that provides arbitrary-precision arithmetic, in a decimal representation. This works around the problem of 0.1 not having a finite representation in binary, but there are still some edge conditions when converting to or from Java's primitive types, as you can see:

```
double d = 0.3;
System.out.println(d);  // Special-cased to show 0.3

BigDecimal bd = new BigDecimal(d);
System.out.println(bd); // 0.29999999999999998889...

bd = new BigDecimal("0.3");
System.out.println(bd); // 0.3
```

However, even with all arithmetic performed in base-10, there are still numbers, such as 1/3, that do not have a terminating decimal representation. Let's see what happens when we try to represent such numbers using `BigDecimal`:

```
bd = new BigDecimal(BigInteger.ONE);
bd.divide(new BigDecimal(3.0));
System.out.println(bd); // Should be 1/3
```

As `BigDecimal` can't represent 1/3 precisely, the call to `divide()` blows up with `ArithmeticException`. When you are working with `BigDecimal`, it is therefore necessary to be acutely aware of exactly which operations could result in a nonterminating decimal result. To make matters worse, `ArithmeticException` is an unchecked, runtime exception and so the Java compiler does not even warn about possible exceptions of this type.

As a final note on floating-point numbers, the paper "What Every Computer Scientist Should Know About Floating-Point Arithmetic" by David Goldberg should be considered essential reading for all professional programmers. It is easily and freely obtainable on the internet.

## Java's Standard Library of Mathematical Functions

To conclude this look at Java's support for numeric data and math, let's take a quick tour of the standard library of functions that Java ships with. These are mostly static helper methods that are located on the class `java.lang.Math` and include functions like:

`abs()`

: Returns the absolute value of a number. Has overloaded forms for various primitive types.

*Trigonometric functions*

: Basic functions for computing the sine, cosine, tangent, and so on. Java also includes hyperbolic versions and the inverse functions (such as arcsine).

`max()`, `min()`

: Overloaded functions to return the greater and smaller of two arguments (both of the same numeric type).

`ceil()`, `floor()`

: Used for rounding to integers. `floor()` returns the largest integer smaller than the argument (which is a `double`). `ceil()` returns the smallest integer larger than the argument.

`pow()`, `exp()`, `log()`

: Functions for raising one number to the power of another and for computing exponentials and natural logarithms. `log10()` provides logarithms to base-10, rather than the natural base.

Let's look at some simple examples of how to use these functions:

```
System.out.println(Math.abs(2));    // 2
System.out.println(Math.abs(-2));   // 2

double cosp3 = Math.cos(0.3);       // 0.955336489125606
double sinp3 = Math.sin(0.3);       // 0.29552020666133955
System.out.println((cosp3 * cosp3 + sinp3 * sinp3)); // Always 1.0

System.out.println(Math.max(0.3, 0.7));     // 0.7
System.out.println(Math.max(0.3, -0.3));    // 0.3
System.out.println(Math.max(-0.3, -0.7));   // -0.3

System.out.println(Math.min(0.3, 0.7));     // 0.3
System.out.println(Math.min(0.3, -0.3));    // -0.3
```

```
System.out.println(Math.min(-0.3, -0.7));   // -0.7

System.out.println(Math.floor(1.3));        // 1.0
System.out.println(Math.ceil(1.3));         // 2.0
System.out.println(Math.floor(7.5));        // 7.0
System.out.println(Math.ceil(7.5));         // 8.0

System.out.println(Math.round(1.3)); // Returns long, 1L
System.out.println(Math.round(7.5)); // Returns long, 8L

System.out.println(Math.pow(2.0, 10.0));          // 1024.0
System.out.println(Math.exp(1));                  // 2.71828...
System.out.println(Math.exp(2));                  // 7.38905...
System.out.println(Math.log(2.718281828459045));  // 1.0
System.out.println(Math.log10(100_000));          // 5.0
System.out.println(Math.log10(Integer.MAX_VALUE));  // 9.331...

System.out.println(Math.random());
System.out.println("Let's toss a coin: ");
if (Math.random() > 0.5) {
    System.out.println("It's heads");
} else {
    System.out.println("It's tails");
}
```

To conclude this section, let's briefly discuss Java's `random()` function. When this is first called, it sets up a new instance of `java.util.Random`. This is a *pseudorandom number generator* (PRNG)—a deterministic piece of code that produces numbers that *look* random but are actually produced by a mathematical formula.[2] In Java's case, the formula used for the PRNG is pretty simple, for example:

```
// From java.util.Random
public double nextDouble() {
    return (((long)(next(26)) << 27) + next(27)) * DOUBLE_UNIT;
}
```

If the sequence of pseudorandom numbers always starts at the same place, then exactly the same stream of numbers will be produced. To get around this problem, the PRNG is seeded by a value that should contain as much true randomness as possible. For this source of randomness for the seed value, Java uses a CPU counter value that is normally used for high-precision timing.

> **WARNING**
>
> While Java's built-in pseudorandom numbers are fine for most general applications, some specialist applications (notably cryptography and some types of simulations) have much more stringent requirements. If you are working on an application of that sort, seek expert advice from programmers who are already working in the area.

While it doesn't solve the limitations of pseudorandom numbers, the JDK also provides a `ThreadLocalRandom` class. As the name suggests, this avoids the global contention of `Math.random` by isolating the generation to the current thread. In general if `Math.random` is

acceptable for your use case, `ThreadLocalRandom` is also a good fit and often preferable.

Now that we've looked at text and numeric data, let's move on to look at another of the most frequently encountered kinds of data: date and time information.

## Date and Time

Almost all business software applications have some notion of date and time. When modeling real-world events or interactions, collecting a point at which the event occurred is critical for future reporting or comparison of domain objects.

Java 8 brought a complete overhaul to the way that developers work with date and time, and this section introduces those concepts.[3] In earlier versions, the only support is via classes such as `java.util.Date` that do not model the concepts. Code that uses the older APIs should be refactored as soon as possible, but we'll review the API for completeness starting on "Legacy Date and Time".

### Introducing the Java 8 Date and Time API

Modern date and time processing in Java uses the package `java.time`. This package contains the core classes that most developers work with. It also contains four subpackages:

`java.time.chrono`

: Alternative chronologies that developers using calendaring systems that do not follow the ISO standard will interact with. An example would be a Japanese calendaring system.

`java.time.format`

: Contains the `DateTimeFormatter` used for converting date and time objects into a `String` and also for parsing strings into the data and time objects.

`java.time.temporal`

: Contains the interfaces required by the core date and time classes and also abstractions (such as queries and adjusters) for advanced operations with dates.

`java.time.zone`

: Classes used for the underlying time zone rules; most developers won't require this package.

One of the most important concepts when representing time is the idea of an instantaneous point on the timeline of some entity. While this concept is well defined within, for example, special relativity, representing it within a computer requires us to make some assumptions. In Java, we represent a single point in time as an `Instant`, which has these key assumptions:

- We cannot represent more seconds than can fit into a `long`.

- We cannot represent time more precisely than nanosecond precision.

This means that we are restricting ourselves to modeling time in a manner that is consistent with the capabilities of current computer systems. However, another fundamental concept should also be introduced.

An `Instant` is about a single event in space-time. However, it is far from uncommon for programmers to have to deal with intervals between two events, and so Java also contains the `java.time.Duration` class. This class ignores calendar effects that might arise (e.g., from daylight saving time). With this basic concept of instants and durations between events, let's move on to unpack the possible ways of thinking about an instant.

### The parts of a timestamp

In Figure 10-1, we show the breakdown of the different parts of a timestamp in a number of possible ways.

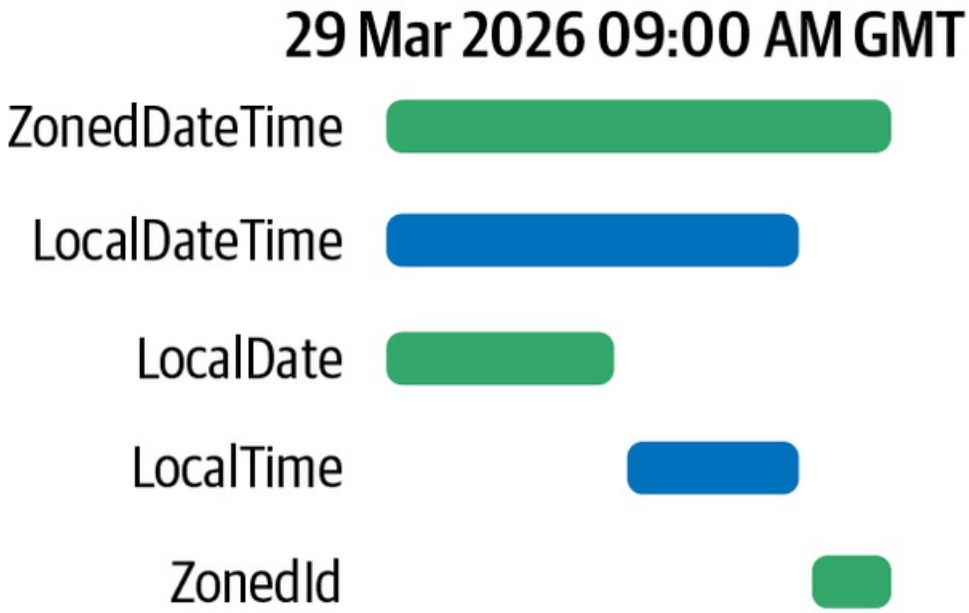



*Figure 10-1. Breaking apart a timestamp*

The key concept here is that a number of different abstractions might be appropriate at different times. For example, there are applications where a `LocalDate` is key to business processing, where the needed granularity is a business day. Alternatively, some applications require subsecond, or even microsecond, precision. Developers should be aware of their domain and use a suitable representation within their application.

### Example

The date and time API can be a lot to take in at first glance, so let's start by looking at an example and discussing a diary class that keeps track of birthdays. If you happen to be very forgetful about birthdays, then a class like this (and especially methods like `getBirthdaysInNextMonth()`) might be very helpful:

```
public class BirthdayDiary {
    private Map<String, LocalDate> birthdays;

    public BirthdayDiary() {
        birthdays = new HashMap<>();
    }

    public LocalDate addBirthday(String name, int day, int month,
                                 int year) {
        LocalDate birthday = LocalDate.of(year, month, day);
        birthdays.put(name, birthday);
        return birthday;
    }

    public LocalDate getBirthdayFor(String name) {
        return birthdays.get(name);
    }

    public int getAgeInYear(String name, int year) {
        Period period = Period.between(
              birthdays.get(name),
              birthdays.get(name).withYear(year));

        return period.getYears();
    }

    public Set<String> getFriendsOfAgeIn(int age, int year) {
        return birthdays.keySet().stream()
                .filter(p -> getAgeInYear(p, year) == age)
                .collect(Collectors.toSet());
    }

    public int getDaysUntilBirthday(String name) {
        int year = LocalDate.now().getYear();
        LocalDate birthday = birthdays.get(name).withYear(year);
        LocalDate today = LocalDate.now();
        if(today.isAfter(birthday)) {
            birthday = birthday.plusYears(1L);
        }
        return (int)(ChronoUnit.DAYS.between(today, birthday));
    }

    public Set<String> getBirthdaysIn(Month month) {
        return birthdays.entrySet().stream()
                .filter(p -> p.getValue().getMonth() == month)
                .map(p -> p.getKey())
                .collect(Collectors.toSet());
    }

    public Set<String> getBirthdaysInCurrentMonth() {
        return getBirthdaysIn(LocalDate.now().getMonth());
    }

    public int getTotalAgeInYears() {
        return birthdays.keySet().stream()
                .mapToInt(p -> getAgeInYear(p,
                      LocalDate.now().getYear()))
                .sum();
    }
}
```

This class shows how to use the low-level API to build up useful functionality. It also uses innovations such as the Java Streams API and demonstrates how to use `LocalDate` as an immutable class and how dates should be treated as values.

## Queries

Under a wide variety of circumstances, we may find ourselves wanting to answer a question about a particular temporal object. Some example questions we may want answers to are:

- Is the date before March 1st?
- Is the date in a leap year?
- How many days is it from today until my next birthday?

This is achieved by the use of the `TemporalQuery` interface, which is defined like this:

```
public interface TemporalQuery<R> {
    R queryFrom(TemporalAccessor temporal);
}
```

The parameter to `queryFrom()` should not be `null`, but if the result indicates that a value was not found, `null` could be used as a return value.

> **NOTE**
>
> The `Predicate` interface can be thought of as a query that can represent answers only to yes-or-no questions. Temporal queries are more general and can return a value of "How many?" or "Which?" instead of just "yes" or "no."

Let's look at an example of a query in action by considering a query that answers the following question: "Which quarter of the year is this date in?" Java does not support the concept of a quarter directly. Instead, code like this is used:

```
LocalDate today = LocalDate.now();
Month currentMonth = today.getMonth();
Month firstMonthOfQuarter = currentMonth.firstMonthOfQuarter();
```

This still doesn't give quarter as a separate abstraction, and instead special case code is still needed. So let's slightly extend the JDK support by defining this enum type:

```
public enum Quarter {
    FIRST, SECOND, THIRD, FOURTH;
}
```

Now, the query can be written as:

```
public class QuarterOfYearQuery implements TemporalQuery<Quarter> {
    @Override
    public Quarter queryFrom(TemporalAccessor temporal) {
        LocalDate now = LocalDate.from(temporal);

        if(now.isBefore(now.with(Month.APRIL).withDayOfMonth(1))) {
            return Quarter.FIRST;
        } else if(now.isBefore(now.with(Month.JULY)
                                .withDayOfMonth(1))) {
            return Quarter.SECOND;
        } else if(now.isBefore(now.with(Month.OCTOBER)
                                .withDayOfMonth(1))) {
            return Quarter.THIRD;
        } else {
           return Quarter.FOURTH;
        }
    }
}
```

`TemporalQuery` objects can be used directly or indirectly. Let's look at an example of each:

```
QuarterOfYearQuery q = new QuarterOfYearQuery();

// Direct
Quarter quarter = q.queryFrom(LocalDate.now());
System.out.println(quarter);

// Indirect
quarter = LocalDate.now().query(q);
System.out.println(quarter);
```

Under most circumstances, it is better to use the indirect approach, where the query object is passed as a parameter to `query()`. This is because it is normally a lot clearer to read in code.

## Adjusters

Adjusters modify date and time objects via implementing the `TemporalAdjuster` interface. Suppose, for example, that we want to return the first day of a quarter that contains a particular timestamp:

```
public class FirstDayOfQuarter implements TemporalAdjuster {
    @Override
    public Temporal adjustInto(Temporal temporal) {
        final int currentQuarter = YearMonth.from(temporal)
                .get(IsoFields.QUARTER_OF_YEAR);

        final Month firstMonthOfQuarter = switch (currentQuarter) {
            case 1 -> Month.JANUARY;
            case 2 -> Month.APRIL;
            case 3 -> Month.JULY;
            case 4 -> Month.OCTOBER;
            default -> throw new IllegalArgumentException("Invalid");
        };

        return LocalDate.from(temporal)
```

```
                .withMonth(firstMonthOfQuarter.getValue())
                .with(TemporalAdjusters.firstDayOfMonth());
    }
}
```

Let's look at an example of how to use an adjuster:

```
LocalDate now = LocalDate.now();
Temporal fdoq = now.with(new FirstDayOfQuarter());
System.out.println(fdoq);
```

The key here is the `with()` method, and the code should be read as taking in one `Temporal` object and returning another object that has been modified. This is completely normal for APIs that work with immutable objects.

While these APIs provide a wide variety of ways to manipulate dates and times, it's worth noting that some concepts aren't present, such as business days and holidays. Third-party libraries are available, though, that bridge that gap should your application require it.

## Time Zones

If you work with code that cares about dates, you will almost certainly encounter complications from time zones. Beyond the simple problems of presenting information clearly to users, time zones cause problems because they change. Whether from daylight saving moves or governments reassigning the zone for a given territory, the definition of time zones today isn't guaranteed to be the same next month.

The JVM brings its own copy of the standard Internet Assigned Numbers Authority (IANA) time zone data, so getting time zone updates typically requires a JDK upgrade. For those who need changes more frequently, Oracle publishes a `tzupdater` tool that can be used to modify a JDK installation in place with newer data.

## Legacy Date and Time

Unfortunately, many applications are not yet converted to use the superior date and time libraries that shipped with Java 8. So, for completeness, we briefly mention the legacy date and time support (which is based on `java.util.Date`).

> **WARNING**
>
> The legacy date and time classes, especially `java.util.Date`, should *not* be used in modern Java environments. Consider refactoring or rewriting any code that still uses the legacy classes.

In older versions of Java, `java.time` is not available. Instead, programmers rely upon the legacy and rudimentary support provided by `java.util.Date`. Historically, this was the only way to represent timestamps, and although named `Date`, this class actually consisted of both a

date and a time component—and this led to a lot of confusion for many programmers.

There are many problems with the legacy support provided by `Date`, for example:

- The `Date` class is incorrectly factored. It doesn't actually refer to a date and instead is more like a timestamp. It turns out that we need different representations for a date, versus a date and time, versus an instantaneous timestamp.
- `Date` is mutable. We can obtain a reference to a `Date` instance and then change the date it refers to.
- The `Date` class doesn't actually accept ISO 8601, the universal ISO date standard, as being a valid date.
- `Date` has a very large number of deprecated methods.

The current JDK uses two constructors for `Date`—the `void` constructor that is intended to be the "now constructor" and a constructor that takes a number of milliseconds since epoch.

If you cannot avoid `java.util.Date`, you can still take advantage of the newer APIs by converting with code, as in the following example:

```
// Defaults to timestamp when called
var oldDate = new java.util.Date();

// Note both forms require specifying time zone -
// part of the failing in the old API
var newDate = LocalDate.ofInstant(
                  oldDate.toInstant(),
                  ZoneId.systemDefault());

var newTime = LocalDateTime.ofInstant(
                  oldDate.toInstant(),
                  ZoneId.systemDefault());
```

## Summary

In this chapter, we've met several different classes of data. Textual data and numeric data are the most obvious examples, but as working programmers we will meet a large number of different sorts of data. Let's move on to look at whole files of data and new ways to work with I/O and networking. Fortunately, Java provides good support for dealing with many of these abstractions.

---

1 In fact, they are actually two examples of *transcendental numbers*.

2 It is very difficult to get computers to produce true random numbers, and in the rare cases where they do, specialized hardware is usually necessary.

3 The story is still evolving, however, because the date and time classes are in scope for conversion to value classes as part of Project Valhalla, which we will discuss in Appendix A.

# Chapter 11. File Handling and I/O

Java has had input/output (I/O) support since the very first version. However, due to Java's strong desire for platform independence, the earlier versions of I/O emphasized portability over functionality. As a result, these systems have undergone multiple significant revisions over the platform's history.

In this chapter we will introduce modern, recommended approaches to file I/O. After that, we'll touch on the "classic" APIs that you still may encounter in some code. Finally, we'll cover network communication.

## Path

When working with files, you have to know the location of the file you want to access. The `Path` interface in the `java.nio.file` package can be thought of as representing this file location whether it exists or not. A path is:

- System dependent
- Hierarchical
- Composed of a sequence of path elements
- Hypothetical (may not exist yet or may have been deleted)

`Path` is an interface, which enables different filesystem providers to each implement system-specific features while retaining the overall abstraction. As opposed to the legacy `File` class we'll see later, `Path` has no notion of the contents of a file and purely represents a file's location within the filesystem (which itself may be abstracted or virtualized).

> **NOTE**
>
> What's that NIO in the package name? As mentioned in the introduction, File I/O has seen several large revisions across Java's history. NIO stands for "New I/O." Java 7 in particular brought in a brand-new I/O API—usually called NIO.2—that `Path` is a part of.

The elements of a `Path` consist of an optional root component, which identifies the filesystem hierarchy that this instance belongs to. Note that, for example, relative `Path` instances may not have a root component. In addition to the root, all `Path` instances have zero or more directory names and a name element.

The name element is the element farthest from the root of the directory hierarchy and represents the name of the file or directory. The `Path` can be thought of as consisting of the path elements

joined by a special separator or delimiter.

`Path` is an abstract concept; it isn't necessarily bound to any physical filepath. This allows us to talk easily about the locations of files that don't exist yet. The `Path` interface provides static factory methods for creating `Path` instances.

> **NOTE**
>
> When NIO.2 was introduced in Java 7, static methods were not supported on interfaces, so a `Paths` class was introduced to hold the factory methods. With Java 17 the `Path` interface methods are recommended instead, and the `Paths` class may be deprecated in the future.

`Path` provides two `of()` methods for creating `Path` objects. The usual version takes one or more `String` instances and uses the default filesystem provider. A second `URI` version takes advantage of additional providers of bespoke filesystems. This is an advanced usage, and interested developers should consult the primary documentation. Let's look at some simple examples of how to use `Path`:

```
var p = Path.of("/Users/ben/cluster.txt");
var p2 = Path.of(new URI("file:///Users/ben/cluster.txt"));

// Returns true
System.out.println(p2.equals(p));
```

Note that our examples are given in a POSIX style with `/` as the file separator character. That is not the case for Windows-based operating systems, which may use \ instead. If you are constructing paths from individual components and work on multiple platforms, `Path.of` has a multi-argument form that will join the arguments using your current platform's requirements:

```
// Returns "tmp/joined" on POSIX operating systems
// "tmp\joined" on Windows
Path.of("tmp", "joined");
```

If required you can find out the current separator via `File.separator` or `FileSys tems.getDefault().getSeparator()`, but it is discouraged using that to concatenate paths yourself.

Knowing the location of a file is only part of what we need, though. Let's take a look at the `Files` class, which gives access to a wealth of attributes and operations for working with files.

## Files

The `Files` class has only static methods, and most of them are pretty self-explanatory. Many methods have return types that are similar to equivalent C code, but we excluded them here for brevity. Consult the JDK documentation for all the details.

```
Path source = Path.of("/tmp/source");
Path target = Path.of("/tmp/target");

// Example use of platform specific attributes
Set<PosixFilePermission> perms =
  PosixFilePermissions.fromString("rw-rw-rw-");
FileAttribute<?> attr = PosixFilePermissions.asFileAttribute(perms);

// Creating files
Files.createFile(target, attr);

// Deleting files
Files.delete(target);
boolean deleted = Files.deleteIfExists(target);

// Copying/moving files
Files.copy(source, target);
Files.move(source, target);

// Utility methods to retrieve information
long size = Files.size(target);

FileTime fTime = Files.getLastModifiedTime(target);
System.out.println(fTime.to(TimeUnit.SECONDS));

Map<String, ?> attrs = Files.readAttributes(target, "*");
System.out.println(attrs);

// Methods to deal with file types
boolean isDir = Files.isDirectory(target);
boolean isSym = Files.isSymbolicLink(target);

// It's recommended to always declare a character set
// rather than rely on defaults
Charset cs = StandardCharsets.UTF_8;

// Methods to deal with reading and writing.
// These read entire file contents into memory, which
// depending on file sizes can cause errors or performance
// issues in your application. Use with care.
String s = Files.readString(target, cs);
List<String> lines = Files.readAllLines(target, cs);
byte[] b = Files.readAllBytes(target);
```

Some of the methods on `Files` provide the opportunity to pass optional arguments, to provide additional (possibly implementation-specific) behavior for the operation.

Some of the API choices here produce occasionally annoying behavior. For example, by default, a copy operation will not overwrite an existing file, so we need to specify this behavior as a copy option:

```
Files.copy(Path.of("input.txt"), Path.of("output.txt"),
           StandardCopyOption.REPLACE_EXISTING);
```

`StandardCopyOption` is an enum that implements an interface called `CopyOption`. This is also implemented by `LinkOption`. So `Files.copy()` can take any number of either

`LinkOption` or `StandardCopyOption` arguments. `LinkOption` is used to specify how symbolic links should be handled (provided the underlying OS supports symlinks, of course).

The `Files` class also provides methods for handling temporary files and directories, which is a surprisingly common use case and can be a source of security bugs. For example, let's see how to load a resources file from within the classpath, copy it to a newly created temporary directory, and then safely clean up the temporary files:

```
Path tmpdir = Files.createTempDirectory(Path.of("/tmp"), "tmp-test");
try (InputStream in =
      FilesExample.class.getResourceAsStream("/res.txt")) {
    Path copied = tmpdir.resolve("copied-resource.txt");
    Files.copy(in, copied, StandardCopyOption.REPLACE_EXISTING);
    // ... work with the copy
}

// Clean up when done...
Files.walkFileTree(tmpdir, new Reaper());
```

`Files.walkFileTree` recursively walks through any directory structure starting from the provided path. The `Reaper` class, available in the book resources online, cleans up in those directories.

## FileSystem

One of the major advances that came with the NIO.2 work that introduced `Path` and `Files` was better handling for separate filesystems.

While alternate filesystems may be a niche concern, an example of how this may be commonly useful is the ability to access a JAR file as a `FileSystem` in its own right. (Recall that JAR files are actually just ZIP files, so this technique will also work for *.zip* archives.) With this capability, we can add a file directly into the JAR, and see how to work with the `FileSystems` class:

```
var tempJar = Path.of("sample.jar");
try (var workingFS =
      FileSystems.newFileSystem(tempJar)) {

  Path pathForFile = workingFS.getPath("/hello.txt");

  // Writes list of lines to the file within the JAR (aka ZIP)
  Files.write(pathForFile,
              List.of("Hello World!"),
              Charset.defaultCharset(),
              StandardOpenOption.WRITE, StandardOpenOption.CREATE);
}
```

This shows how we create a `FileSystem` object in order to create the `Path` objects that refer to files inside the jar, via the `getPath()` method. This enables the developer to essentially treat

`FileSystem` objects as black boxes: they are automatically created via a service provider interface (SPI) mechanism.

To see which filesystems are available on your machine, you can run some code like this:

```
for (FileSystemProvider f : FileSystemProvider.installedProviders()) {
    System.out.println(f.toString());
}
```

Once we have located or created a file of interest, we probably want to read the contents of it. That's where I/O streams come into play, which we'll examine in the next section.

## I/O Streams

The I/O stream abstraction (not to be confused with the streams that are used when dealing with the Java Collection APIs) was present in Java 1.0, as a way of dealing with sequential streams of bytes from disks or other sources.

The core of this API is a pair of abstract classes, `InputStream` and `OutputStream`. These are very widely used, and in fact the "standard" input and output streams, which are called `System.in` and `System.out`, are streams of this type. They are public, static fields of the `System` class, and they are often used in even the simplest programs:

```
System.out.println("Hello World!");
```

These stream classes and the reader and writer classes we'll see in the next section predate NIO additions to Java's APIs, but they remain in common use. Their older heritage is sometimes visible (for instance, `FileInputStream` does not allow `Path` as an argument type), but in general they're still acceptable to use versus the lower-level channel-based approaches that NIO introduced.

> **WARNING**
>
> Recall that in "Simple Standalone Java Programs" we met the class `java.lang.IO`, which is a recent addition to Java and provides a simplified interface to a small subset of the functionality provided by the standard streams. This is really intended only for the simplest use cases—any more complex usage should switch over to using the full capabilities of `System.in` and `System.out`. In particular, mixing `IO.readln()` with calls to methods on `System.in` can result in unpredictable or undefined behavior.

Specific subclasses of streams, including `FileInputStream` and `FileOutputStream`, can be used to operate on individual bytes in a file—for example, by counting all the times ASCII 97 (small letter *a*) occurs in a file:

```
try (var is = new FileInputStream("/Users/ben/cluster.txt")) {
  byte[] buf = new byte[4096];
```

```
    int len, count = 0;
    while ((len = is.read(buf)) > 0) {
      for (int i = 0; i < len; i += 1) {
        if (buf[i] == 97) {
          count = count + 1;
        }
      }
    }
    System.out.println("'a's seen: " + count);
  } catch (IOException e) {
    e.printStackTrace();
  }
```

This approach to dealing with on-disk data can lack some flexibility—most developers think in terms of characters, not bytes. To allow for this, the streams are usually combined with the higher-level `Reader` and `Writer` classes, which provide a character-stream level of interaction, rather than the low-level bytestream provided by `InputStream` and `OutputStream` and their subclasses.

## Readers and Writers

By moving to an abstraction that deals in characters, rather than bytes, developers are presented with an API that is much more familiar and that hides many of the issues with character encoding, Unicode, and so on.

The `Reader` and `Writer` classes are intended to overlay the bytestream classes and to remove the need for low-level handling of I/O streams. They have several subclasses that are often used to layer on top of one another, such as:

- `FileReader`
- `BufferedReader`
- `StringReader`
- `InputStreamReader`
- `FileWriter`
- `PrintWriter`
- `BufferedWriter`
- `OutputStreamWriter`

To read in all lines from a file and print them out, we use a `BufferedReader` layered on top of a `FileReader`, like this:

```
  try (var in = new BufferedReader(new FileReader(filename))) {
    String line;
```

```
    while((line = in.readLine()) != null) {
      System.out.println(line);
    }
  } catch (IOException e) {
    // Handle FileNotFoundException, etc. here
  }
```

The streaming approach these interfaces give supports reading and writing files that are much too large to fit in memory all at once.

If we need to read in lines from the console, rather than from a file, we will usually use an `InputStreamReader` applied to `System.in`.

Let's look at an example where we want to read in lines of input from the console but treat input lines that start with a special character as special—commands ("metas") to be processed, rather than regular text. This is a common feature of many chat programs, including IRC. We'll use regular expressions from Chapter 10 to help us:

```
// Meta example: "#info username"
var SHELL_META_START = Pattern.compile("^#(\\w+)\\s*(\\w+)?");

try (var console =
      new BufferedReader(new InputStreamReader(System.in))) {
  String line;

  while((line = console.readLine()) != null) {
    // Check for special commands ("metas")
    Matcher m = SHELL_META_START.matcher(line);
    if (m.find()) {
      String metaName = m.group(1);
      String arg = m.group(2);
      doMeta(metaName, arg);
    } else {
      System.out.println(line);
    }
  }
} catch (IOException e) {
  // Handle FileNotFoundException, etc. here
}
```

To output text to a file, we can use code like this:

```
var f = new File(System.getProperty("user.home")
 + File.separator + ".bashrc");
try (var out =
      new PrintWriter(new BufferedWriter(new FileWriter(f)))) {
  out.println("## Automatically generated config file. DO NOT EDIT");
  // ...
} catch (IOException iox) {
  // Handle exceptions
}
```

This style of Java I/O has a lot of other occasionally useful functionality. For example, to deal with text files, the `FilterInputStream` class is quite often useful. Or for threads that want

to communicate in a way similar to the classic “piped” I/O approach, `PipedInputStream`, `PipedReader`, and their write counterparts are provided.

## Decorators

The `Reader` and `Writer` classes demonstrate a pattern mentioned in “Delegation” that’s worth pausing to appreciate—the *Decorator pattern*. This pattern is used to extend an object without subclassing by wrapping it in another object with a matching signature.

To take a concrete example, the `BufferedReader` class we saw previously implements the `Reader` class and can be used anywhere that a `Reader` is required. However, it also takes a `Reader` as a parameter to its constructor. `BufferedReader` then overrides the methods on `Reader` necessary to do its I/O operations work in bulk—i.e., buffered—and delegates any other methods to the underlying `Reader` unchanged. Consumers of `BufferedReader` downstream should simply interact with it as a `Reader`.

> **TIP**
>
> When working with the I/O `Reader` and `Writer` classes, it’s recommended to use the most general class possible in arguments to your classes. Don’t declare a parameter as a `Buffer edReader`; instead use the more general `Reader` class. This will allow for flexibility by passing other, decorated instances should you need them in the future.

Taking a peek into the JDK code, let’s see part of how `BufferedReader` is implemented to familiarize ourselves with decorators. The structure starts with the constructor, where the `BufferedReader` is initialized with the `Reader` that it is intended to wrap and a size for the buffer it will hold:

```
public BufferedReader(Reader in, int sz) {
    super(in);
    if (sz <= 0)
        throw new IllegalArgumentException("Buffer size <= 0");
    this.in = in;
    cb = new char[sz];
    nextChar = nChars = 0;
}
```

`BufferedReader` fundamentally changes how the `read` methods work by pulling in a set buffer amount from the underlying reader rather than moving character by character. It’s no surprise, then, that the decorated `read` methods contain significant amounts of code to alter that behavior. If you’re unfamiliar with the `synchronized` keyword here, we’ll meet that properly in Chapter 12:

```
public int read() throws IOException {
    synchronized (lock) {
```

```
            ensureOpen();

            // Looping to repeatedly fill the buffer
            for (;;) {
                if (nextChar >= nChars) {
                    fill();
                    if (nextChar >= nChars)
                        return -1;
                }
                // Further code elided for length
            }
        }
    }
```

But other methods don't need additional work and serve mostly to pass through to the underlying `Reader`, like the `close` method:

```
    public void close() throws IOException {
        synchronized (lock) {
            if (in == null)
                return;
            try {
                in.close();
            } finally {
                in = null;
                cb = null;
            }
        }
    }
```

Decorators add a little complexity with the extra objects to allocate and method calls to override and delegate to the inner object, but when the pattern fits, it's remarkably powerful.

Throughout this chapter, we have used the language feature known as "try-with-resources" (TWR). This syntax was briefly introduced in "The try-with-resources Statement", but it is in conjunction with operations like I/O that it comes into its fullest potential, and it has granted a new lease on life to the older I/O style.

## try-with-resources Revisited

To make the most of Java's I/O capabilities, it is important to understand how to use TWR. TWR should be used whenever possible.

Before TWR, resources had to be closed manually; complex interactions between resources that failed to close led to buggy code that leaked resources.

> **NOTE**
>
> In fact, Oracle's engineers estimate that 60% of the resource handling code in the initial JDK 6 release was incorrect. So, if even the platform authors can't reliably get manual resource handling right, then all new code should definitely be using TWR.

The key to TWR is the `AutoCloseable` interface. This interface is a direct superinterface of `Closeable`. It marks a resource that must be automatically closed, and for which the compiler will insert special exception-handling code.

Inside a TWR resource clause, only declarations of objects that implement `AutoCloseable` objects may appear—but the developer may declare as many as required:

```
try (var in = new BufferedReader(
                          new FileReader("profile"));
     var out = new PrintWriter(
                        new BufferedWriter(
                          new FileWriter("profile.bak")))) {
  String line;
  while((line = in.readLine()) != null) {
    out.println(line);
  }
} catch (IOException e) {
  // Handle FileNotFoundException, etc. here
}
```

The consequences of this are that resources are automatically scoped to the `try` block. The resources (whether readable or writable) are automatically closed in the correct order (the reverse order of the way they were opened), and the compiler inserts exception handling that takes dependencies between resources into account.

TWR is related to similar concepts in other languages and environments, such as RAII (Resource Acquisition Is Initialization) in C++. However, TWR is limited to block scope. This minor limitation results because the feature is implemented by the Java source code compiler—it automatically inserts bytecode that calls the resource's `close()` method when the scope is exited (by whatever means).

As a result, the overall effect of TWR is more similar to C#'s `using` keyword, rather than the C++ version of RAII. For Java developers, the best way to regard TWR is as "finalization done right." Older code should be refactored to use TWR as soon as is practicable, as it provides real tangible benefits to resource handling code.

## Classic Java I/O

Java's original way to do file I/O centered around the `File` class. The `File` class has a very large number of methods on it, but some basic functionality (notably a way to read the actual contents of a file) is not, and never has been, provided directly. The `File` abstraction represented both files and directories but was sometimes a bit cumbersome to deal with, leading to code like this:

```
// Get a file object to represent the user's home directory
var homeDir = new File(System.getProperty("user.home"));

// Create an object to represent a config file (should
// already be present in the home directory)
```

```
var f = new File(homeDir, "app.conf");

// Check the file exists, really is a file, and is readable
if (f.exists() && f.isFile() && f.canRead()) {

  // Create a file object for a new configuration directory
  var configDir = new File(homeDir, ".configDir");
  // And create it
  configDir.mkdir();

  // Finally, move the config file to its new home
  f.renameTo(new File(configDir, ".config"));
}
```

This shows some of the flexibility possible with the `File` class, but it also demonstrates some of the problems with the abstraction. It is very general and thus requires a lot of methods to interrogate a `File` object in order to determine what it actually represents and what its capabilities are.

All of the functionality of the `File` class has been subsumed in recent Java versions by `Path` and `Files`, so reach for those when you need basic file operations.

The long history of code written to use `File`, though, does mean you may need to interoperate with those older APIs. To help in those situations, there are some helpful "bridge" methods, for example between `Path` and `File`:

```
var p = Path.of("/Users/ben/cluster.txt");

File f = p.toFile();
System.out.println(f.isDirectory());

Path p2 = f.toPath();
System.out.println(p2.equals(p));
```

This example also shows the easy interoperation between `Path` and `File` objects. The addition of a `toFile()` method to `Path` and a `toPath()` method to `File` allows the developer to move effortlessly between the two APIs and allows for a straightforward approach to refactoring the internals of code based on `File` to use `Path` instead.

`Files` also provides access to some of the classic stream and reader/writer objects that don't directly support `Path` in their construction:

```
var target = Path.of("/Users/ben/cluster.txt");
Charset cs = StandardCharsets.UTF_8;

var br = Files.newBufferedReader(target, cs);
var bwr = Files.newBufferedWriter(target, cs);

var is = Files.newInputStream(target);
var os = Files.newOutputStream(target);
```

One of the criticisms of Java's original I/O APIs was the lack of support for native and high-

performance I/O. A solution was initially added in Java 1.4, the Java New I/O (NIO) API, and it has been refined in later Java versions and forms the foundations of the modern approaches that we have already met.

## NIO Channels and Buffers

NIO buffers are a low-level abstraction for high-performance I/O. They provide a container for a linear sequence of elements of a specific primitive type. We'll work with the `ByteBuffer` (the most common case) in our examples.

### ByteBuffer

This is a sequence of bytes and can conceptually be thought of as a performance-critical alternative to working with a `byte[]`. To get the best possible performance, `ByteBuffer` provides support for dealing directly with the native capabilities of the platform the JVM is running on.

This approach is called the *direct buffers* case, and it bypasses the Java heap wherever possible. Direct buffers are allocated in native memory, not on the managed Java heap, which may have unexpected impact on an application's overall memory usage.

To obtain a direct `ByteBuffer`, call the `allocateDirect()` factory method. An on-heap version, `allocate()`, is also provided, but in practice this is not often used.

A third way to obtain a byte buffer is to `wrap()` an existing `byte[]`—this will give an on-heap buffer that serves to provide a more object-oriented view of the underlying bytes:

```
var b = ByteBuffer.allocateDirect(65536);
var b2 = ByteBuffer.allocate(4096);

byte[] data = {1, 2, 3};
ByteBuffer b3 = ByteBuffer.wrap(data);
```

Those coming from C/C++ may be surprised to find that `ByteBuffer` does not expose methods for deallocating or freeing the native memory. This operation is handled by the JVM automatically when the managed `ByteBuffer` object, which does live on the standard Java heap, is garbage collected.

Byte buffers are all about low-level access to the bytes. This means that developers have to deal with the details manually—including the need to handle the endianness of the bytes and the signed nature of Java's integral primitives:

```
b.order(ByteOrder.BIG_ENDIAN);

int capacity = b.capacity();
int position = b.position();
int limit = b.limit();
int remaining = b.remaining();
```

```
boolean more = b.hasRemaining();
```

To get data into or out of a buffer, we have two types of operations—single value, which reads or writes a single value, and bulk, which takes a `byte[]` or `ByteBuffer` and operates on a (potentially large) number of values as a single operation. It is from the bulk operations that we'd expect to realize performance gains. These writing operations all use the buffer's current position and shift that position after the write:

```
// Write various data types.
// Shifts buffer's current position after each write.
b.put((byte)42);
b.putChar('x');
b.putInt(0xc001c0de);

b.put(data);
b.put(b2);

double d = b.getDouble();
b.get(data, 0, data.length);
```

The single-value form also supports a form used for absolute positioning within the buffer:

```
b.put(0, (byte)9);
```

Buffers are an in-memory abstraction. To affect the outside world (e.g., the file or network), we need to use a `Channel`, from the package `java.nio.channels`. Channels represent connections to entities that can support read or write operations. Files and sockets are the usual examples of channels, but we could consider custom implementations used for low-latency data processing.

Channels are open when they're created and can subsequently be closed. Once closed, they cannot be reopened. Channels are usually either readable or writable but not both. The key to understanding channels is that:

- Reading from a channel puts bytes into a buffer
- Writing to a channel takes bytes from a buffer

For example, suppose we have a large file that we want to checksum in 16 MB chunks:

```
FileInputStream fis = getSomeStream();
boolean fileOK = true;

try (FileChannel fchan = fis.getChannel()) {
  var buffy = ByteBuffer.allocateDirect(16 * 1024 * 1024);
  while(fchan.read(buffy) != -1 || buffy.position() > 0 || fileOK) {
    fileOK = computeChecksum(buffy);
    buffy.compact();
  }
} catch (IOException e) {
  System.out.println("Exception in I/O");
}
```

This will use native I/O as far as possible and will avoid a lot of copying of bytes on and off the Java heap. If the `computeChecksum()` method has been well implemented, then this could be a very performant implementation.

### Mapped Byte Buffers

These are a type of direct byte buffer that contains a memory-mapped file (or a region of one). They are created from a `FileChannel` object, but note that the `File` object corresponding to the `MappedByteBuffer` must not be used after the memory-mapped operations or an exception will be thrown. To mitigate this, we again use `try`-with-resources to scope the objects tightly:

```
try (var raf =
  new RandomAccessFile(new File("input.txt"), "rw");
     FileChannel fc = raf.getChannel();) {

  MappedByteBuffer mbf =
    fc.map(FileChannel.MapMode.READ_WRITE, 0, fc.size());
  var b = new byte[(int)fc.size()];
  mbf.get(b, 0, b.length);
  for (int i = 0; i < fc.size(); i = i + 1) {
    b[i] = 0; // Won't be written back to the file, we're a copy
  }
  mbf.position(0);
  mbf.put(b); // Zeros the file
}
```

Note that `MappedByteBuffer` is read from and written to, making it an exception to most standard channel usage.

Even with buffers, there are limitations of what can be done in Java for large I/O operations (e.g., transferring 10 GB between filesystems) that perform synchronously on a single thread. Rather than writing custom multithreaded code and managing a separate thread for performing a background copy, Java provides an asynchronous I/O API that can automatically perform large operations without blocking the calling thread. Let’s take a look in our next section.

## Async I/O

The key to the asynchronous functionality is new subclasses of `Channel` that can deal with I/O operations that need to be handed off to a background thread. The same functionality can be applied to large, long-running operations and to several other use cases.

In this section, we’ll deal exclusively with `AsynchronousFileChannel` for file I/O. Asynchronous channels also exist for network I/O with the `AsynchronousSocket Ch annel` and `AsynchronousServerSocketChannel` classes, which are beyond the scope of this book.

There are two ways to interact with an asynchronous channel—`Future` style and callback style.

## Future-Based Style

A full discussion of the `Future` interface would take us too far into the details of Java concurrency. However, for the purpose of this chapter, it can be thought of as an ongoing task that may or may not have completed yet. It has two key methods:

`isDone()`

Returns a Boolean indicating whether the task has finished.

`get()`

Returns the result. If finished, returns immediately. If not finished, blocks until done.

Let's look at an example of a program that reads a large file (possibly as large as 100 MB) asynchronously:

```
try (var channel =
        AsynchronousFileChannel.open(Path.of("input.txt"))) {
  var buffer = ByteBuffer.allocateDirect(1024 * 1024 * 100);
  Future<Integer> result = channel.read(buffer, 0);

  while(!result.isDone()) {
    // Do some other useful work....
  }

  System.out.println("Bytes read: " + result.get());
}
```

The asynchronous nature of the `Future` in `result` allows other work to be accomplished while we're waiting for the reading to complete.

## Callback-Based Style

The callback style for asynchronous I/O is based on a `CompletionHandler`, which defines two methods, `completed()` and `failed()`, that will be called back when the operation either succeeds or fails.

This style is useful if you want immediate notification of events in asynchronous I/O—for example, if there are a large number of I/O operations in flight, but failure of any single operation is not necessarily fatal:

```
byte[] data = {2, 3, 5, 7, 11, 13, 17, 19, 23};
ByteBuffer buffy = ByteBuffer.wrap(data);

CompletionHandler<Integer, Object> h = new CompletionHandler<>() {
  public void completed(Integer written, Object o) {
    System.out.println("Bytes written: " + written);
  }
```

```
  public void failed(Throwable x, Object o) {
    System.out.println("Async write failed: " + x.getMessage());
  }
};

try (var channel = AsynchronousFileChannel.open(
        Path.of("primes.txt"),
        StandardOpenOption.CREATE,
        StandardOpenOption.WRITE)) {
  channel.write(buffy, 0, null, h);

  // Give the CompletionHandler time to run before foreground exit
  Thread.sleep(1000);
}
```

The `AsynchronousFileChannel` object is associated with a background threadpool, so that the I/O operation proceeds while the original thread gets on with other tasks.

> **NOTE**
>
> The `CompletionHandler` interface has two abstract methods, not one, so it cannot be the target type for a lambda expression, unfortunately.

By default, this uses a managed threadpool that is provided by the runtime. If required, it can be created to use a threadpool that is managed by the application (via an overloaded form of `AsynchronousFileChannel.open()`), but this is seldom necessary.

Finally, for completeness, let's touch upon NIO's support for multiplexed I/O. This enables a single thread to manage multiple channels and to examine those channels to see which are ready for reading or writing. The classes to support this are in the `java.nio.channels` package and include `SelectableChannel` and `Selector`.

These nonblocking multiplexed techniques can be extremely useful when you're writing advanced applications that require high scalability, but a full discussion is outside the scope of this book. In general, given its higher complexity, the nonblocking API should be used only for advanced use cases when high performance or other nonfunctional requirements demand it.

## Watch Services and Directory Searching

The last class of asynchronous services we will consider are those that watch a directory or visit a directory (or a tree). The watch services operate by recursively observing everything that happens within a directory—for example, the creation or modification of files:

```
var watcher = FileSystems.getDefault().newWatchService();

var dir = FileSystems.getDefault().getPath("/home/ben");
dir.register(watcher,
             StandardWatchEventKinds.ENTRY_CREATE,
```

```
                StandardWatchEventKinds.ENTRY_MODIFY,
                StandardWatchEventKinds.ENTRY_DELETE);

  while(!shutdown) {
    WatchKey key = watcher.take();
    for (WatchEvent<?> event: key.pollEvents()) {
      Object o = event.context();
      if (o instanceof Path) {
        System.out.println("Path altered: " + o);
      }
    }
    key.reset();
  }
```

By contrast, the directory streams provide a view into all files currently in a single directory. For example, to list all the Java source files and their size in bytes, we can use code like:

```
  try(DirectoryStream<Path> stream =
      Files.newDirectoryStream(Path.of("/opt/projects"), "*.java")) {
    for (Path p : stream) {
      System.out.println(p + ": " + Files.size(p));
    }
  }
```

One drawback of this API is that it will return only elements that match according to glob syntax, which is sometimes insufficiently flexible. We can go further by using the `Files.find()` and `Files.walk()` methods to address each element obtained by a recursive walk through the directory:

```
  // Looks for projects dir in your home directory
  var homeDir = Path.of(System.getProperty("user.home"), "projects");
  Files.find(homeDir, 255,
    (p, attrs) -> p.toString().endsWith(".java"))
       .forEach(q -> {System.out.println(q.normalize());});
```

It is possible to go even further and construct advanced solutions based on the `FileVisitor` interface in `java.nio.file`, but that requires the developer to implement all four methods on the interface, rather than just using a single lambda expression as done here.

In the last section of this chapter, we will discuss Java's networking support and the core JDK classes that enable it.

## Networking

The Java platform provides access to a large number of standard networking protocols, and these make writing simple networked applications quite easy. In practice, more complex needs are likely to be met by third-party libraries like Netty, but the basics, particularly around HTTP, are available within the JDK itself.

The core of Java's network support lives in the package `java.net`, with additional

extensibility provided by `javax.net` (and in particular, `javax.net.ssl`), all of which is in the module `java.base`.

One of the easiest protocols to use for building applications is Hypertext Transfer Protocol (HTTP), the protocol used as the basic communication protocol of the Web.

## HTTP

HTTP is the most common and popular high-level network protocol that Java supports out of the box. HTTP 1.1 is a very simple protocol, implemented on top of the standard TCP/IP stack. It can run on any network port but is usually found on port 443 when encrypted with TLS (known as HTTPS) or port 80 when running unencrypted. These days, HTTPS should be the default wherever possible.

Java has two separate APIs for handling HTTP—one of which dates back to the earliest days of the platform, and a more modern API that arrived fully in Java 11. The older API definitely shows its age and in fact implements only version 1.0 of the HTTP standard, which is very inefficient and considered archaic. Its only real virtue these days is its brevity for simple cases, and the more modern API should be favored for production applications.

The modern Java HTTP API was added as a result of Java needing to support the new HTTP/2 protocol. This modern HTTP API has been available in a fully supported module, `java.net.http`, since Java 11. Let's see a simple example of using the new API:

```
import java.net.http.*;
import static java.net.http.HttpResponse.BodyHandlers.ofString;

var client = HttpClient.newBuilder().build();
var uri = new URI("https://www.oreilly.com");
var request = HttpRequest.newBuilder(uri).build();

var response = client.send(request,
        ofString(Charset.defaultCharset()));
var body = response.body();
System.out.println(body);
```

Note that this API is designed to be extensible, with interfaces such as `Http Res ponse.BodySubscriber` available for implementing custom handling. The interface also seamlessly hides the differences between HTTP/2 and the older HTTP/1.1 protocol, meaning that Java applications will be able to migrate gracefully as web servers adopt the new version.

HTTP defines "request methods," which are the operations that a client can make on a remote resource. These methods are called GET, POST, HEAD, PUT, DELETE, OPTIONS, and TRACE.

Each has slightly different usages, for example:

- GET should only be used to retrieve a document and should *never* perform any side

effects.

- HEAD is equivalent to GET, except the body is not returned—useful if a program wants to quickly check via headers whether a URL has changed.
- POST is used when we want to send data to a server for processing.

By default, `HttpClient` uses GET but provides support for the other HTTP methods as well. In this next example, we're using the echo function provided by Postman to return a view of the data we posted:

```
import java.net.http.*;
import static java.net.http.HttpResponse.BodyHandlers.ofString;

var client = HttpClient.newBuilder().build();
var uri = new URI("https://postman-echo.com/post");
var request = HttpRequest.newBuilder(uri)
  .POST(HttpRequest.BodyPublishers.ofString("{\"java\":true}"))
  .build();

var response = client.send(request,
        ofString(Charset.defaultCharset()));
var body = response.body();
System.out.println(body);
```

Let's take a quick look at the older API, for the sake of completeness. In this API, `URL` is the key class—it supports URLs of the form `http://`, `ftp://`, `file://`, and `https://` out of the box. It is very easy to use, and the simplest example of Java HTTP support is to download a particular URL:

```
var url = new URL("https://www.google.com/");
try (InputStream in = url.openStream()) {
  Files.copy(in, Path.of("output.html"));
} catch(IOException ex) {
  ex.printStackTrace();
}
```

For more low-level control, including metadata about the request and response, the `URLConnection` can be used, but at this point most modern programs should be reaching for `java.net.http` instead.

Let's move on to look at the next layer down the networking stack, the Transmission Control Protocol (TCP).

## TCP

TCP is the basis of reliable network transport over the internet. It ensures that web pages and other internet traffic are delivered in a complete and comprehensible state. From a networking theory standpoint, the protocol properties that allow TCP to function as this "reliability layer" for internet traffic are:

*Connection based*

Data belongs to a single logical stream (a connection).

*Guaranteed in-order delivery*

Data packets will be resent until they arrive.

*Error checked*

Damage caused by network transit will be detected and fixed automatically.

TCP is a two-way (or bidirectional) communication channel and uses a special numbering scheme (TCP sequence numbers) for data chunks to ensure that both sides of a communication stream stay in sync. To support many different services on the same network host, TCP uses port numbers to identify services and ensures that traffic intended for one port does not go to a different one.

In Java, TCP is represented by the classes `Socket` and `ServerSocket`. `Socket` is used to provide the client side of a TCP connection, `ServerSocket` the server side, respectively—meaning that Java can be used both to connect to network services and as a language for implementing new services.

> **NOTE**
>
> Java's original socket support was reimplemented, without API changes, in Java 13. The classic socket APIs now share code with the more modern NIO infrastructure and will continue working well into the future as a result.

As an example, let's consider partially reimplementing HTTP 1.1. This is a relatively simple, text-based protocol. We'll implement both sides of the connection, so let's start with an HTTP client on top of a TCP socket. To accomplish this, we will actually need to implement the details of the HTTP protocol, but we do have the advantage that we have complete control over the TCP socket.

We will need to both read and write from the client socket, and we'll construct the actual request line in accordance with the HTTP 1.1 standard (which is known as RFC 2616 and uses explicit line-ending syntax). The resulting client code will look something like this:

```
var hostname = "www.example.com";
int port = 80;
var filename = "/index.html";

try (var sock = new Socket(hostname, port);
  var from = new BufferedReader(
      new InputStreamReader(sock.getInputStream()));
  var to = new PrintWriter(
      new OutputStreamWriter(sock.getOutputStream())); ) {
```

```
    // The HTTP protocol
    to.print("GET " + filename +
      " HTTP/1.1\r\nHost: " + hostname + "\r\n\r\n");
    to.flush();

    for (String l = null; (l = from.readLine()) != null; )
      System.out.println(l);
  }
```

On the server side, we'll need to receive possibly multiple incoming connections. To handle this, we'll kick off a main server loop, then use `accept()` to take a new connection from the operating system. The new connection is then quickly passed to a separate handler class on its own thread so that the main server loop can get back to listening for new connections. The code for this is a bit more involved than the client case:

```
  // Handler class
  private static class HttpHandler implements Runnable {
    private final Socket sock;
    HttpHandler(Socket client) { this.sock = client; }

    public void run() {
      try (var in =
             new BufferedReader(
               new InputStreamReader(sock.getInputStream()));
           var out =
             new PrintWriter(
               new OutputStreamWriter(sock.getOutputStream())); ) {
        out.print("HTTP/1.1 200\r\nContent-Type: text/plain\r\n\r\n");
        String line;
        while((line = in.readLine()) != null) {
          if (line.length() == 0) break;
          out.println(line);
        }
      } catch(Exception e) {
        // Handle exception
      }
    }
  }

  // Main server loop
  public static void main(String[] args) {
    try {
      var port = Integer.parseInt(args[0]);

      ServerSocket ss = new ServerSocket(port);
      while (true) {
        Socket client = ss.accept();
        var handler = new HTTPHandler(client);
        new Thread(handler).start();
      }
    } catch (Exception e) {
      // Handle exception
    }
  }
```

When designing a protocol for applications to communicate over TCP, there's a simple and

profound network architecture principle, known as Postel's Law (after Jon Postel, one of the founders of the internet) that you should always keep in mind. It is sometimes stated as "Be strict about what you send, and liberal about what you will accept." This simple principle means that communication can remain broadly possible in a network system, even in the event of quite imperfect implementations.

Postel's Law, when combined with the general principle that the protocol should be as simple as possible (sometimes called the KISS [keep it simple, stupid] principle), will make the developer's job of implementing TCP-based communication much easier than it otherwise would be.

Below TCP is the internet's general-purpose haulage protocol—the Internet Protocol (IP) itself.

### IP

IP, a "lowest common denominator" transport, provides a useful abstraction over the physical network technologies that are used to actually move bytes from A to B.

Unlike TCP, delivery of an IP packet is not guaranteed, and a packet can be dropped by any overloaded system along the path. IP packets do have a destination but usually no routing data—it's the responsibility of the (possibly many different) physical transports along the route to actually deliver the data.

It is possible to create "datagram" services in Java that are based around single IP packets (or those with a User Datagram Protocol [UDP] header, instead of TCP), but this is often not required except for extremely low-latency applications. Java uses the class `DatagramSocket` to implement this functionality, although few developers should ever need to venture this far down the network stack.

Finally, it's worth noting some changes currently in flight in the addressing schemes that are used across the internet. The current dominant version of IP in use is IPv4, which has a 32-bit space of possible network addresses. This space is now very crowded, and various mitigation techniques have been deployed to handle the depletion of free network addresses.

The next version of IP (IPv6) is being rolled out, but it is not fully accepted and has yet to displace IPv4, although steady progress toward it becoming the standard continues. As of this writing, IPv6 traffic is at about 48% of internet traffic and steadily rising. IPv6 will soon overtake IPv4 in terms of traffic volume, and low-level networking will need to adapt to this radically new version.

However, for Java programmers, the good news is that the language and platform have been working for many years on good support for IPv6 and the changes that it introduces. The transition between IPv4 and IPv6 is likely to be much smoother and less problematic for Java applications than for many other languages.

## Summary

In this chapter we've met the file handling, I/O, and networking capabilities provided in Java's SDK. However, these capabilities are not used equally often. The core file handling classes (especially `Path` and the rest of NIO.2) are used very often by Java developers, with the more advanced capabilities being less frequently encountered.

The story is different for the networking libraries. It's good to be aware of these capabilities, but they are fairly basic. In practice, higher-level libraries provided by third parties are often used instead (e.g., Netty). The one exception: the one low-level JDK networking library that Java developers can expect to encounter relatively often is the new HTTP library in `java.net.http`.

Let's move on to discuss how Java handles concurrency, one of the most powerful and complex features that the platform has to offer.

# Chapter 12. Concurrency in Java

This chapter is an introduction to the handling of concurrency (multithreading) in the Java platform. To fully understand this topic, it's important to have a good grasp of the discussion of memory in Chapter 9—especially because some of the diagrams in this chapter build upon those concepts. It's also useful, but not essential, to be familiar with the basic components of operating systems, such as the scheduler.

We will cover:

- Java's concurrency primitives
- Data visibility and mutability
- Executors and threadpools
- Concurrent data structures
- Virtual threads

Let's get started by defining the basic terms and concepts that we will use throughout this chapter.

## Basic Concepts of Java Concurrency

The idea of a *thread* is that of a lightweight unit of execution—smaller than a process but still capable of executing arbitrary code. The usual way this is implemented is for each thread to be a full-fledged unit of execution to the operating system but to belong to a process, with the address space of the process being shared between all threads that make up that process. This means each thread can be scheduled independently and has its own stack and program counter but shares memory and objects with other threads in the same process.

Java has supported multithreaded programming from the very first version, and the platform directly exposes the ability to create new threads of execution to the developer.

To understand this, first we must consider what happens when a Java program, represented by a `Program` class, starts up and the original application thread (usually referred to as the *main* thread) appears:

1. The programmer executes `java Program` (other startup cases are possible).
2. This causes the JVM, the context within which all Java programs run, to start up.
3. The JVM examines its arguments and sees that the programmer has requested execution starting at the entry point (the `main()` method) of `Program.class`.

4. Assuming that `Program.class` passes classloading checks, a dedicated thread for the execution of the program is started (the main thread).
5. The JVM bytecode interpreter is started on the main thread.
6. The main thread's interpreter reads the bytecode of `Program::main()` and execution begins, one bytecode at a time.

Every Java program starts this way, but this also means:

- Every Java program starts as part of a managed model with one interpreter per thread.
- Every Java program always runs as part of a multithreaded operating system process.
- The JVM has a certain ability to control a Java application thread.

When we create new threads in Java code, this can be as simple as:

```
Thread t = new Thread(() -> {System.out.println("Hello Thread");});
t.start();
```

This small piece of code creates and starts a new thread, which executes the body of the lambda expression. Technically speaking, the lambda is converted to an instance of the `Runnable` interface before being passed to the `Thread` constructor.

The threading mechanism allows new threads to execute concurrently with the original application thread and the threads that the JVM itself starts up for various purposes.

When we call `Thread::start()`, this call is delegated to the OS, and a new OS thread is created. This new OS thread uses `exec()` to start a new copy of the JVM bytecode interpreter. The interpreter starts executing at the `run()` method (or, equivalently, at the body of the lambda).

> **NOTE**
>
> Java's interpreter-per-thread model is different from some other languages—notably Python, which has only a single interpreter that is shared among threads, although work is underway to remove this and adopt a similar model to Java.[1]

This means that application threads' access to the CPU is controlled by the operating system *scheduler*—a built-in part of the OS that is responsible for managing timeslices of processor time (and that will not allow an application thread to exceed its allocated time).

In more recent versions of Java, an increasing trend toward *runtime-managed concurrency* has appeared. This is the idea that for many purposes it's not desirable for developers to explicitly manage threads. Instead, the environment should provide toolkits and "fire and forget" capabilities, whereby the program specifies what needs to be done, but the low-level details of how this is to be accomplished are left to the runtime. This viewpoint can be seen in the

concurrency toolkit contained in `java.util.concurrent`, which we will discuss later in this chapter.

In the final part of the chapter, we will also discuss the recent addition of virtual threads, which aim to provide more scalability of threads that perform some I/O handling, by rethinking the relationship between Java threads and OS threads.

> **WARNING**
>
> We strongly encourage you to understand both the low-level `Thread`-based and the runtime-managed approaches before doing any significant concurrent programming.

Let's get underway by introducing the low-level concurrency mechanisms that the Java platform provides and that every Java developer should be aware of.

## Thread Lifecycle

It's important to understand the lifecycle of an application thread. Every operating system has a view of threads that can differ in the details, but in most cases the view is broadly similar at a high level. Java tries hard to abstract these details away and has an enum called `Thread.State`, which wrappers over the operating system's view of the thread's state. The values of `Thread.State` provide an overview of the lifecycle of a thread:

`NEW`

: The thread has been created, but its `start()` method has not yet been called. All threads begin in this state.

`RUNNABLE`

: The thread is running or is available to run when the operating system schedules it.

`BLOCKED`

: The thread is not running because it is waiting to acquire a lock so that it can enter a `synchronized` method or block. We'll see more about `synchronized` methods and blocks later in this section.

`WAITING`

: The thread is not running because it has called `Object.wait()` or `Thread.join()`.

`TIMED_WAITING`

: The thread is not running because it has called `Thread.sleep()` or has called `Object.wait()` or `Thread.join()` with a timeout value.

`TERMINATED`

> The thread has completed execution. Its `run()` method has exited normally or by throwing an exception.

These states represent the view of a thread that is common (at least across mainstream operating systems), leading to a view like Figure 12-1.

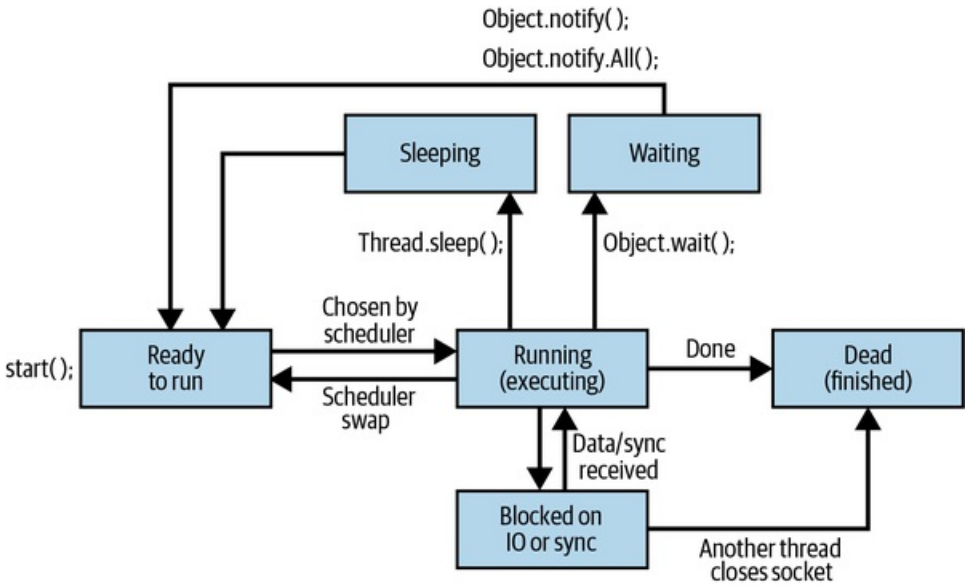



*Figure 12-1. Thread lifecycle*

Only some of the state transitions in the diagram are under the control of the user's code.

For example, threads can be made to sleep by using the `Thread.sleep()` method. This takes an argument in milliseconds, which indicates how long the thread would like to sleep, like this:

```
try {
    Thread.sleep(2000);
} catch (InterruptedException e) {
    e.printStackTrace();
}
```

Note that the call to sleep is a request to the operating system, not a demand. Your program may sleep for longer than requested, depending on load and other factors specific to the runtime environment. We will discuss the other methods of `Thread` (and the *Indefinite Waiting* pattern implemented by `wait()` and `notify()`) later in this chapter.

The other state transitions are handled automatically by the operating system. Java's approach to concurrency is therefore referred to as *preemptive multitasking*, and the OS is capable of *interrupting* or *preempting* user threads (and when it does, the user thread cannot ignore it).

This is why the `Thread.sleep()` call is surrounded by a `try { } catch { }` block—because the method can throw a checked exception (`InterruptedException`), which

occurs when the OS interrupts a thread (either for its own reasons, or on behalf of another user thread).

Preemption is related to, but somewhat different from, GC safepointing (which we met in Chapter 9). When a GC is required, the safepoint flag is raised, and each RUNNABLE thread stops when it reaches a safepoint in executing code.[2] The GC threads are then woken up to execute—but note that these are also userland threads owned by the JVM process.

Moving on, we next need to cover some important theory that deals with how threads access memory. This is fundamental to understanding why multithreaded programming is hard and can cause developers a lot of problems.

## Visibility and Mutability

All Java application threads in a process have their own call stacks (and local variables) but share a single heap. This makes it very easy to share objects between threads, as all that is required is to pass a reference from one thread to another. This is illustrated in Figure 12-2.

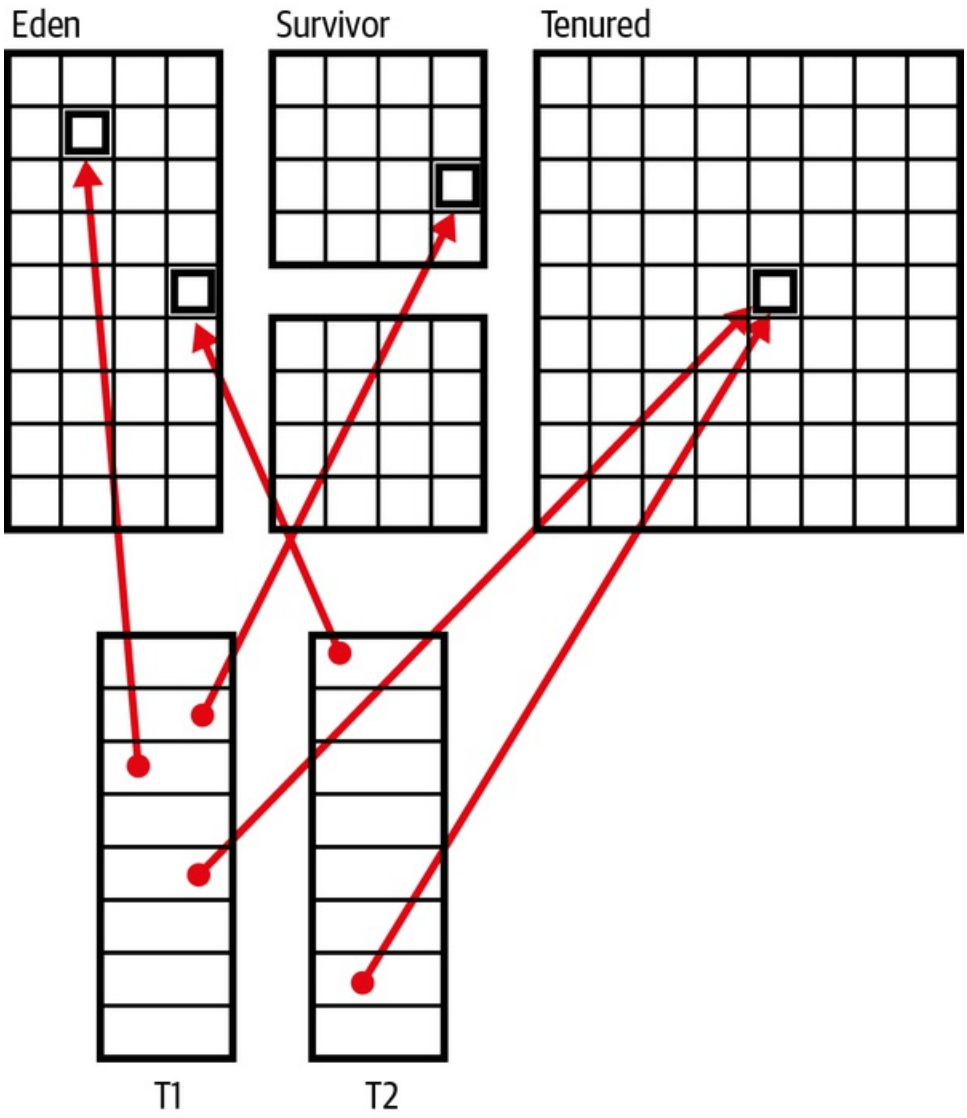



*Figure 12-2. Shared memory between threads*

This leads to a general design principle of Java—that objects are *visible by default*.[3] If I have a reference to an object, I can copy it and hand it off to another thread with no restrictions. A Java reference is essentially a typed pointer to a location in heap—and threads share the same heap, so visible by default is a natural model.

In addition to visible by default, Java has another property that is important to fully understand concurrency, which is that objects are *mutable*: the contents of an object instance's fields can usually be changed. We can make individual variables or references constant by using the `final` keyword, but this does not apply to the contents of the object.

As we will see throughout the rest of this chapter, the combination of these two properties—visibility across threads and object mutability—gives rise to a great many complexities when trying to reason about concurrent Java programs.

## Concurrent Safety

If we want to write correct multithreaded code, then we need our programs to satisfy a certain important property.

In Chapter 6, we defined a safe object-oriented program to be one where we move objects from legal state to legal state by calling their accessible methods. This definition works well for single-threaded code. However, there is a particular difficulty that arises when we try to extend it to concurrent programs.

> **TIP**
>
> A *safe multithreaded program* is one in which it is impossible for any object to be seen in an illegal or inconsistent state by any other object, no matter what methods are called, and no matter in what order the application threads are scheduled by the operating system.

For most mainstream cases, the operating system will schedule threads to run on particular processor cores at seemingly random times, depending on load and what else is running in the system. If load is high, then there may be other processes that also need to run.

The operating system will forcibly remove a Java thread from a CPU core if it needs to. The thread is suspended immediately, no matter what it's doing—including being partway through executing a method. However, as we discussed in Chapter 6, a method can temporarily put an object into an illegal state while it is working on it, providing it corrects it before the method exits.

This means that if a thread is swapped off before it has completed a long-running method, it may leave an object in an inconsistent state, *even if the program follows the safety rules*. Another way of saying this is that even data types that have been correctly modeled for the single-threaded case still need to protect against the effects of concurrency. Code that adds this extra layer of protection is called *concurrently safe* or (more informally) threadsafe.

In the next section, we'll discuss the primary means of achieving this safety, and at the end of the chapter, we'll meet some other mechanisms that can also be useful under some circumstances.

## Protecting State with Exclusion

Any code that either modifies or reads state that can become inconsistent must be protected. To achieve this, the Java platform provides only one mechanism: *exclusion*.

Consider a method that contains a sequence of operations that, if interrupted partway through, could leave an object in an inconsistent or illegal state. If this illegal state was visible to another object, incorrect code behavior could occur.

For example, consider an ATM or other cash-dispensing machine:

```
public class Account {
    private double balance = 0.0; // Must be >= 0
```

```
    // Assume the existence of other field (e.g., name) and methods
    // such as deposit(), checkBalance(), and dispenseNotes()

    public Account(double openingBal) {
        if (openingBal < 0.0) {
            throw new IllegalArgumentException("Balance must be >= 0");
        }
        balance = openingBal;
    }

    public boolean withdraw(double amount) {
        if (balance >= amount) {
            try {
                Thread.sleep(2000); // Simulate risk checks
            } catch (InterruptedException e) {
                return false;
            }
            balance = balance - amount;
            dispenseNotes(amount);
            return true;
        }
        return false;
    }
}
```

The sequence of operations that happens inside `withdraw()` can leave the object in an inconsistent state. In particular, after we've checked the balance, a second thread could come in while the first was sleeping (in simulated risk checks), and the account could be overdrawn, in violation of the constraint that `balance >= 0`.

This is an example of a system where the operations on the objects are single-threaded safe (because the objects cannot reach an illegal state (`balance < 0`) if called from a single thread) but not concurrently safe.

To allow the developer to make code like this concurrently safe, Java provides the `synchronized` keyword. This keyword can be applied to a block or to a method, and when it is used, the platform uses it to restrict access to the code inside the block or method.

> **NOTE**
>
> Because `synchronized` surrounds code, many developers are led to the conclusion that concurrency in Java is about code. Some texts even refer to the code that is inside the synchronized block or method as a *critical section* and consider that to be the crucial aspect of concurrency. This is not the case; instead, it is the inconsistency of *data* that we must guard against, as we will see.

The Java platform keeps track of a special token, called a *monitor*, for every object that it ever creates.[4] These monitors (also called *intrinsic locks*) are used by `synchronized` to indicate that the following code could temporarily render the object inconsistent. The sequence of events for a synchronized block or method is:

1. Thread needs to modify an object and may make it briefly inconsistent as an

intermediate step.

2. Thread acquires the monitor, indicating it requires temporary exclusive access to the object.
3. Thread modifies the object, leaving it in a consistent, legal state when done.
4. Thread releases the monitor.

If another thread attempts to acquire the lock while the object is being modified, then the attempt to acquire the lock blocks, until the holding thread releases the lock. It's also important to realize that the object being locked matters, not merely the presence of `synchronized`. Two methods can each be `synchronized` yet lock on different objects, and this will not provide any mutual exclusion.

Note that you do not have to use the `synchronized` statement unless your program creates multiple threads that share data. If only one thread ever accesses a data structure, there is no need to protect it with `synchronized`.

One point is of critical importance—acquiring the monitor does *not* prevent access to the object. It only prevents any other thread from claiming the lock. Correct concurrently safe code requires developers to ensure that all accesses that might either modify or read potentially inconsistent state acquire the object monitor before operating on or reading that state.

Put another way, if a `synchronized` method is working on an object and has placed it into an illegal state, and another method (which is not synchronized) reads from the object, it can still see the inconsistent state.

> **WARNING**
>
> Synchronization is a cooperative mechanism for protecting state, and it is very fragile as a result. A single bug (such as missing a single `synchronized` keyword from a method it's required on) can have catastrophic results for the safety of the system as a whole.

The reason we use the word `synchronized` as the keyword for "requires temporary exclusive access" is that in addition to acquiring the monitor, the JVM also rereads the current state of the object from main memory when the block is entered. Similarly, when the `synchronized` block or method is exited, the JVM flushes any modified state of the object back to main memory.

Without synchronization, different CPU cores in the system may not see the same view of memory, and memory inconsistencies can damage the state of a running program, as we saw in our ATM example.

The simplest example of this is known as *Lost Update*, as demonstrated in the following code:

```
public class Counter {
    private int i = 0;
```

```
    public int increment() {
        return i = i + 1;
    }
    public int getCounter() { return i; }
}
```

This can be driven via a simple control program:

```
    Counter c = new Counter();
    int REPEAT = 10_000_000;
    Runnable r = () -> {
        for (int i = 0; i < REPEAT; i++) {
            c.increment();
        }
    };
    Thread t1 = new Thread(r);
    Thread t2 = new Thread(r);

    t1.start();
    t2.start();
    t1.join();
    t2.join();

    int anomaly = (2 * REPEAT) - c.getCounter();
    double perc = ((double) anomaly * 100) / (2 * REPEAT);
    System.out.println("Lost updates: " + anomaly + " ; % = " + perc);
```

If this concurrent program were correct, then the value for the anomaly (number of lost updates) would be exactly zero. It is not, and so we may conclude that unsynchronized access is fundamentally unsafe.

> **WARNING**
>
> If the Lost Update example is run in JShell, then it will appear to be anomaly-free. This is an artifact of JShell being an interactive environment rather than a "real" Java program. If the contents of the control program are placed into a static method `m()`, then the real behavior of the code can be seen, even in JShell.

By contrast, we also see that the addition of the keyword `synchronized` to the increment method is sufficient to reduce the Lost Update anomaly to zero—that is, to make the method correct, even in the presence of multiple threads.

## Volatile State

Java provides another keyword for dealing with concurrent access to data. This is the `volatile` keyword, and it indicates that before being used by application code, the value of the field or variable must be reread from main memory. Likewise, after a volatile value has been modified, as soon as the write to the variable has completed, it must be written back to main memory.

One common usage of the `volatile` keyword is in the “run-until-shutdown” pattern. This is used in multithreaded programming where an external user or system needs to signal to a processing thread that it should finish the current job being worked on and then shut down gracefully. This is sometimes called the “graceful completion” pattern. Let’s look at a typical example, supposing that this code for our processing thread is in a class that implements `Runnable`:

```
private volatile boolean shutdown = false;

public void shutdown() {
    shutdown = true;
}

public void run() {
    while (!shutdown) {
        // ... process another task
    }
}
```

All the time that the `shutdown()` method is not called by another thread, the processing thread continues to sequentially process tasks (this is often combined very usefully with a work queue to deliver new jobs to execute—we’ll discuss this pattern in full later in this chapter).

However, once `shutdown()` is called by another thread, the processing thread immediately sees the `shutdown` flag change to `true`. This does not affect the running job, but once the task finishes, the processing thread will not accept another task and instead will shut down gracefully.

Unfortunately, as useful as the `volatile` keyword is, it does not provide a complete protection of state—as we can see by using it to mark the field in `Counter` as `volatile`. We might naively assume this would protect the code in `Counter`—but it does not. To see this, modify the previous `Counter` example and add the word `volatile` to the field `i` and rerun the example. The observed nonzero value of the anomaly (and therefore the presence of the Lost Update problem) tells us that by itself, `volatile` does not make code threadsafe.

## Working with Threads

In this section, we’ll take a closer look at the main methods provided by `Thread` that relate to concurrent programming. We’ll also discuss the Indefinite Waiting pattern and then some older thread methods that are considered unsafe for modern applications.

### Useful Methods of Thread

The `Thread` class has a number of methods to make your life easier when you’re working with multiple application threads. This is not an exhaustive list—there are many other methods on `Thread`, but this is a description of some of the more common methods.

#### getId()

This method returns the ID number of the thread as a `long`. This ID will stay the same for the lifetime of the thread and is guaranteed to be unique within this instance of the JVM.

### getPriority() and setPriority()

These methods are used to control the priority of threads. The scheduler decides how to handle thread priorities; for example, one strategy could be to not have any low-priority threads run while there are high-priority threads waiting. In most cases, there is no way to influence how the scheduler will interpret priorities. Thread priorities are represented as an integer between 1 and 10, with 10 being the highest.

### setName() and getName()

These methods allow the developer to set or retrieve a name for an individual thread. Naming threads is good practice, as it can make debugging much easier, especially in a tool such as JDK Mission Control (which we will discuss briefly in Appendix B).

### getState()

This returns a `Thread.State` object that indicates which state this thread is in using the values defined in "Thread Lifecycle".

### isAlive()

This method is used to test whether a thread is still alive.

### start()

This method is used to create a new application thread and to schedule it, with the `run()` method being the entry point for execution. A thread terminates normally when it reaches the end of its `run()` method or when it executes a `return` statement in that method.

### interrupt()

If a thread is blocked in a `sleep()`, `wait()`, or `join()` call, then calling `interrupt()` on the `Thread` object that represents the thread will cause the thread to be sent an `InterruptedException` (and to wake up).

If the thread was involved in interruptible I/O, then the I/O will be terminated and the thread will receive a `ClosedByInterruptException`. The interrupt status of the thread will be set to `true`, even if the thread was not engaged in any activity that could be interrupted.

### join()

The current thread waits until the thread corresponding to the `Thread` object has died. It can be thought of as an instruction not to proceed until the other thread has completed.

### setDaemon()

A *user thread* is a thread that will prevent the process from exiting if it is still alive—this is the

default for threads. Sometimes programmers want threads that will not prevent an exit from occurring—these are called *daemon threads*. The status of a thread as a daemon or user thread can be controlled by the `setDaemon()` method and checked using `isDaemon()`.

### setUncaughtExceptionHandler()

When a thread exits by throwing an exception (i.e., one that the program did not catch), the default behavior is to print the name of the thread, the type of the exception, the exception message, and a stack trace. If this isn't sufficient, you can install a custom handler for uncaught exceptions in a thread. For example:

```
// This thread just throws an exception
Thread handledThread =
  new Thread(() -> { throw new UnsupportedOperationException(); });

// Giving threads a name helps with debugging
handledThread.setName("My Broken Thread");

// Here's a handler for the error.
handledThread.setUncaughtExceptionHandler((t, e) -> {
    System.err.printf("Exception in thread %d '%s':" +
        "%s at line %d of %s%n",
        t.getId(),    // Thread id
        t.getName(),  // Thread name
        e.toString(), // Exception name and message
        e.getStackTrace()[0].getLineNumber(),
        e.getStackTrace()[0].getFileName()); });
handledThread.start();
```

This can be useful in some situations; for example, if one thread is supervising a group of other worker threads, then this pattern can be used to restart any threads that die.

There is also `setDefaultUncaughtExceptionHandler()`, a `static` method that sets a backup handler for catching any thread's uncaught exceptions.

## Dangerous Methods of Thread

In addition to the useful methods of `Thread`, there are a number of dangerous methods you should not use. These methods formed part of the original Java thread API but were quickly found to be unsuitable for developer use. Developers simply need to be aware of them and avoid using them under *all* circumstances.

### stop()

The intent of `Thread.stop()` was to kill the thread immediately, without giving it any opportunity to recover objects to legal states. This design is fundamentally flawed—it is essentially impossible to use correctly without violating concurrent safety.

As of Java 25, `stop()` unconditionally throws `UnsupportedOperationException`.

### suspend(), resume(), countStackFrames(), and destroy()

These methods all had flawed designs (such as not releasing monitors, causing deadlock issues), and as of Java 25 have been removed from the `Thread` class.

If you ever encounter code that tries to call these methods, then you should immediately flag it up to your team lead for removal.

## Indefinite Waiting

We've seen that threads can be asked to sleep for a period of time using `Thread.sleep()`. It is also useful to go to sleep for an unspecified amount of time and wait until a condition is met. In Java, this is handled by the `wait()` and `notify()` methods that are present on `Object`.

As well as the lock that every Java object has associated with it, each object maintains a list of threads that are waiting for a change notification on the object. When a thread calls the `wait()` method of an object, any locks the thread holds are temporarily released, and the thread is added to the list of waiting threads for that object and it stops running.

Indefinite waiting is important for multithreaded programming, because it solves the problem in which a thread cannot make progress with a specific object in its current state. The thread enters a `wait()` and will not run again until the object state has changed.

When another thread calls the `notify()` or `notifyAll()` methods of the same object, then the waiting threads are woken up by the OS scheduler and are allowed to continue running. When the original thread is woken up again, it restarts where it began waiting and reacquires its monitor.

Let's look at an example of a simple data structure that is safe for multithreaded use. This is a version of a queue, which will we meet in more depth later in this chapter. For reasons of simplicity, it does not implement the Java Collections `Queue` interface:

```
/*
 * A thread can call put() to put an object on the queue.
 * Another can call take() to get an object off the queue.
 *
 * If there is no data, take() waits until there is some,
 * using the wait() and notify() mechanism.
 */
public final class WaitingQueue<E> {
    LinkedList<E> q = new LinkedList<>(); // storage

    public synchronized void put(E o) {
        q.add(o);         // Append the object to the end of the list
        this.notifyAll(); // Tell waiting threads that data is ready
    }

    public synchronized E take() {
        while(q.size() == 0) {
            try { this.wait(); }
            catch (InterruptedException _) {}
        }
        return q.remove();
    }
```

```
}
```

This class uses the `wait()` method if the backing list is empty (which would make the `take()` fail). The taking thread temporarily releases its monitor, allowing another thread to claim it—a thread that might `put()` something new onto the queue. There is no guarantee that other threads actually have `put()` items into the queue, however, so we must recheck the state of the queue (which is why `take()` contains a `while` loop) before progressing.

> **NOTE**
>
> `wait()` and `notify()` must be used inside a `synchronized` method or block, because of the temporary relinquishing of locks required for them to work properly. Calling these methods outside of a `synchronized` context will cause `IllegalMoni torStateException` (a runtime exception) to be thrown.

In general, most developers shouldn't roll their own classes like the one in this example. Instead, use the libraries and components that the Java platform provides for you—we'll have more to say about this later in the chapter.

## Synchronization Checklist

To work effectively with multithreaded code, you need the basic facts about monitors and locks at your command. This checklist contains the main facts you should know:

- Synchronization is about protecting object state and memory, not code.
- Synchronization is a cooperative mechanism between threads. One bug can break the cooperative model and have far-reaching consequences.
- Acquiring a monitor only prevents other threads from acquiring the monitor—it does not protect the object.
- Unsynchronized methods can see (and modify) inconsistent state, even while the object's monitor is locked.
- Locking an `Object[]` doesn't lock the individual objects.
- Primitive values are not mutable, so they can't be (and don't need to be) locked.
- `synchronized` can't appear on a method declaration in an interface.
- Inner classes are just syntactic sugar, so locks on inner classes have no effect on the enclosing class (and vice versa).
- Java's monitors are *reentrant*. This means that if a thread holding a monitor encounters a synchronized block for the same monitor, it can enter the block.[5]

Synchronization is a vital building block of Java concurrency, but it is not the only capability

that we need. Let's move on to discuss some of the data structures provided by the package `java.util.concurrent` that are safe for use in multithreaded programming.

## Concurrent Data Structures

Most of the standard implementations of the Java Collections interfaces are not suitable for use when access is required from multiple threads. In this section, we'll meet some of the most common implementations that can be used for concurrent programming when shared access to data is required.

### Atomic Counters

We met the wrapper classes (such as `Integer` and `Long`) in Chapter 2. Recall that they represent immutable objects that are used to "stand in" for primitive values in circumstances where a primitive may not be used and an object is required. Being immutable, they are, of course, threadsafe—but what if we require shared *mutable* state for counting?

It would be possible to use a fully synchronized class as a holder for the mutable state, but this is inefficient. Instead, the JDK provides `AtomicInteger` and `AtomicLong` in the package `java.util.concurrent.atomic`. These classes provide fast and threadsafe shared state for arithmetic operations.[6]

If we modify the code in "Protecting State with Exclusion" to use an atomic integer instead, then it is easy to verify that the atomics are indeed free from the Lost Update problem:

```
var c = new AtomicInteger();
Runnable r = () -> {
    for (int i = 0; i < REPEAT; i++) {
        c.incrementAndGet();
    }
};
```

In fact, the code can be changed to handle any number of threads and any number of repetitions without causing any lost updates.

> **NOTE**
>
> `AtomicInteger` does not extend `Integer` for several reasons—first, because `Integer` is a final class and second, because they represent very different concepts. `AtomicInteger` is shared mutable state and `Integer` is immutable state; there is also the fact that the classes have completely different performance characteristics.

In addition to the `AtomicInteger` and `AtomicLong` classes, the package also contains the `AtomicBoolean`, which effectively acts as an object wrapper over a volatile boolean field. In high-contention circumstances, it may be preferable to use atomic adder and accumulator classes instead of the basic atomics. These must also be used for floating-point operations, as Java does

not provide an `AtomicDouble` class.

There are also classes available to handle atomic updates to references—these are a powerful building block for more advanced concurrency components, but a discussion of them is outside the scope of this book. Let's move on to talk about another of Java's fundamental concurrent data structures.

## Concurrent Maps

In general, maps are a very useful (and common) data structure for building concurrent applications. Unfortunately, `HashMap` suffers from Lost Update even in the simplest case (when the only mutations are adding new entries).

There are three straightforward, though naive, approaches to fixing this and making `HashMap` safe for concurrent use.

First, we could imagine an immutable map (like that produced by `Map.of()`) that delegates read operations and throws `UnsupportedOperationException` on any methods that attempt to mutate the map. This works, but it doesn't permit shared mutable state, and it also has the problems associated with throwing runtime exceptions that we have previously discussed.

Second, we could use a fully synchronized approach, whereby we make all the methods of the class `synchronized`.

Finally, we can use a synchronized wrapper to delegate to an underlying object that actually houses the dictionary. In fact, the JDK provides the `synchronizedMap()` method on the `Collections` class, which does exactly this.

Unfortunately, fully synchronized approaches are not feasible for most practical systems due to performance overhead. Instead, we need a concurrent version of the standard `HashMap` that includes the possibility of shared mutable state. This is provided by the `ConcurrentHashMap` class (CHM) that works as a drop-in replacement for `HashMap`. You should note that there are still additional runtime costs associated with using a CHM, and it is a bad idea to use CHM unless you know that the data structure will need to be accessed by multiple threads.

To see how this works, let's recall the basic structure of `HashMap`, as shown in Figure 12-3.

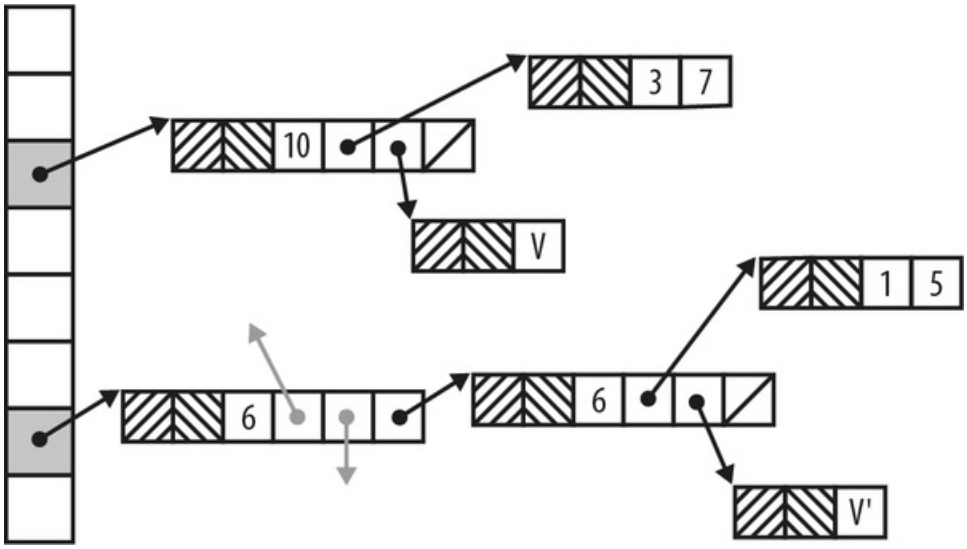



*Figure 12-3. Simple map implementation*

The `HashMap` contains an array of singly linked lists (known as *hash chains*), and the key is converted to an index (an `int`) by calling `hashCode()`. This index is used to look up the hash chain where the key-value pair resides (or will do, in the case of an insertion).

This structure is not threadsafe as it stands, but the fully synchronized approach makes it safe—by locking the entire data structure on every operation.

However, we can do better. Instead of needing to lock the whole structure when accessing the map, it's only necessary to lock the single hash chain that's being altered or read, as shown in Figure 12-4.

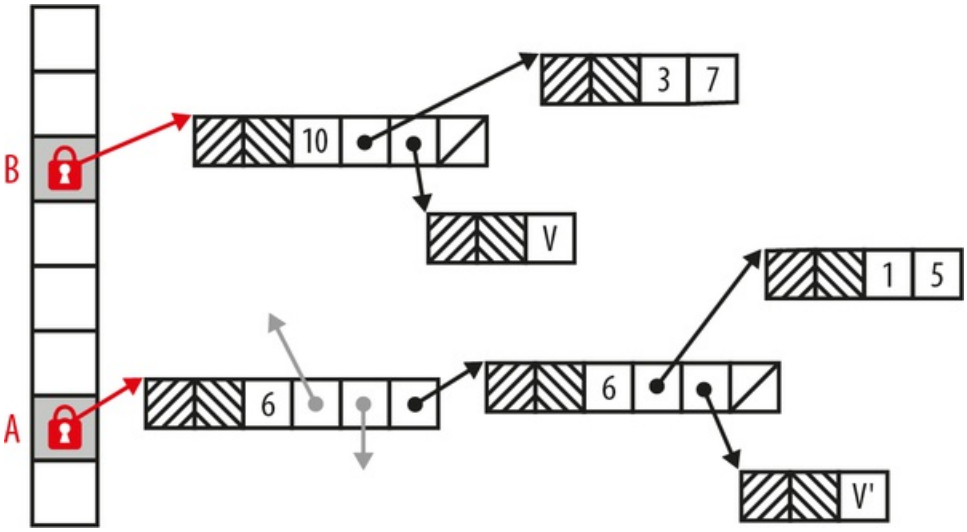



*Figure 12-4. Lock striping in a map*

The implementation has effectively moved the lock down onto the individual hash chains. This technique is known as *lock striping*, and it enables multiple threads to access the map, provided they are operating on different chains.

> **NOTE**
>
> Recall that as the number of elements in the map increases, the table of buckets will resize, meaning that as more and more elements are added to a `ConcurrentHashMap`, it will be able to deal with more and more threads in an efficient manner.

Of course, if two threads need to operate on the same chain, then they will still exclude each other, but in general, lock striping provides much better throughput than synchronizing the entire map.

Unfortunately, while maps generalize nicely to a concurrent use case, linear data structures can be somewhat more difficult.

Java does provide the `CopyOnWriteArrayList` class, and, as the name suggests, this type is a replacement for the standard `ArrayList` class that has been made thread-safe by the addition of *copy-on-write semantics*. This means that any operations that mutate the list will create a new copy of the array backing the list.

This also means that any iterators created don't have to worry about any modifications that they didn't expect. The iterators are guaranteed not to throw `ConcurrentModificationException` and will not reflect any additions, removals, or changes to the list since the iterator was created.

The problem is that `CopyOnWriteArrayList` is not a drop-in replacement for `ArrayList`, and changing to copy-on-write semantics will often require a major rethink of algorithms to make them effective in a multithreaded context.

Instead, let's move on to discuss another linear data structure that is very important for concurrent programming and much more widely used.

## Queues and BlockingQueues

A *queue* is an ordered collection of elements with methods for extracting elements, in order, from the *head* of the queue. Unlike a `Set`, `Queue` implementations typically allow duplicate elements. `Queue` implementations are commonly based on insertion order as either first-in, first-out (FIFO) queues or last-in, first-out (LIFO) queues.

> **NOTE**
>
> LIFO queues are also known as *stacks*, and Java does provide a `Stack` class, but as we mentioned in Chapter 8, its use is strongly discouraged—instead modern code should use implementations of the `Deque` interface instead.

Let's take a look at the definition of the `Queue` interface:

```
public interface Queue<E> extends Collection<E> {
    boolean add(E e);
```

```
    boolean offer(E e);
    E remove();
    E poll();
    E element();
    E peek();
}
```

Unlike `List`, the `Queue` interface does not define methods for manipulating queue elements at arbitrary positions. For example, only the element at the head of the queue is available for examination. For this reason, the `Queue` interface does not extend `List` (but it does extend `Collection`).

There are three `Queue` implementations in the `java.util` package.

`PriorityQueue` is an alternative to both FIFO and LIFO orderings and instead orders its elements according to a `Comparator` or orders `Comparable` elements according to the *natural* order defined by their `compareTo()` methods. The head of a `PriorityQueue` is always the smallest element according to the defined ordering. Note that this implies that a `PriorityQueue` may, in the worst case, potentially need to reorder the entire queue on every insertion.

The `LinkedList` class also implements the `Queue` interface. It provides unbounded FIFO ordering, and insertion and removal operations require constant time. `LinkedList` allows `null` elements, although their use is discouraged when the list is being used as a queue.

Finally, `ArrayDeque` is a double-ended queue implementation. It is often used when a stack implementation is needed.

The `WaitingQueue` that we introduced earlier in the chapter is an example of an effectively *unbounded* queue—it is backed by an instance of `LinkedList` and can hold a number of elements that are restricted only by the memory available to the JVM.

However, it is very common for `Queue` implementations to have a fixed capacity, so that when a queue is full, it is not possible to add more elements (and similarly, when a queue is empty, it is not possible to remove any more elements). Such queues are known as *bounded* queues, and they raise the question of how the queue should behave when an operation cannot be completed.

There are broadly two sets of behaviors that the `Queue` interface supports—returning a special value to indicate failure or throwing a (runtime) exception. So, in lieu of example code here, we'll enumerate the possible queue insertion and removal operations.

The mutating methods available on `Queue` are:

`add()`

> This `Collection` method adds an element in the normal way. In bounded queues, this method may throw an exception (usually `IllegalStateExcep tion`) if the queue is full.

`offer()`

This `Queue` method is like `add()` but returns `false` instead of throwing an exception if the element cannot be added because a bounded queue is full.

`remove()`

In addition to the `Collection.remove()` method, which removes a specified element from the queue, the `Queue` interface defines a no-argument version of `remove()` that removes and returns the element at the head of the queue. If the queue is empty, this method throws a `NoSuchElementException`.

`poll()`

This `Queue` method removes and returns the element at the head of the queue, like `remove()` does, but returns `null` if the queue is empty instead of throwing an exception.

Querying refers to examining the element at the head without removing it from the queue, and this capability is provided by two methods:

`element()`

This `Queue` method returns the element at the head of the queue but does not remove that element from the queue. It throws `NoSuchElementException` if the queue is empty.

`peek()`

This `Queue` method is like `element` but returns `null` if the queue is empty.

When using queues, it is usually a good idea to pick one particular group of methods with a consistent style for dealing with a failure. For example, if you want to examine the return code of a method to see if the queue operation succeeded, then `offer()`, `poll()`, and `peek()` are appropriate choices.

> **WARNING**
>
> Full and empty conditions are a normal part of many queue-based algorithms, so the exception-throwing methods are generally not as useful as the value-returning forms. Java's exceptions should not be used to model normal program conditions (sometimes expressed as the principle "Don't use exceptions for flow control").

We should also note that the `peek()` and `poll()` methods return `null` to indicate that the queue is empty. For this reason, most `Queue` implementations do not allow `null` elements (because if `null` was allowed, then how could we distinguish a failed operation from one that succeeded and returned a `null` element?).

The basic `Queue` implementations are not threadsafe and are of limited applicability even in single-threaded code. To generalize to data structures that are useful for concurrent programming, Java provides the `BlockingQueue` interface (which extends `Queue`) as part of

the `java.util.concurrent` package.

This interface adds operations that will block indefinitely until they can be completed. This is known as a *blocking queue*, and they are an important part of many multithreaded algorithms. There are seven methods defined by `BlockingQueue`, two of which are new overloads of existing `Queue` methods:

`put()`

: This `BlockingQueue` method blocks: if the element cannot be inserted because the queue is full, `put()` waits until some other thread removes an element from the queue and space becomes available for the new element.

`take()`

: This `BlockingQueue` method removes and returns the element at the head of the queue. If the queue is empty, it blocks until some other thread adds an element to the queue.

`offer()`

: `BlockingQueue` defines a version of `offer()` that waits up to a specified amount of time for space to become available in a full queue. Like the basic version of the method, it returns `true` if the element was inserted and `false` otherwise.

`poll()`

: `BlockingQueue` defines a timeout overload of `poll()` that waits up to a specified amount of time for an element to be added to an empty queue.

`remainingCapacity()`

: This method returns an estimate of the number of additional elements that this queue can currently accept (in the absence of memory or resource constraints) without blocking, or `Integer.MAX_VALUE` if there is no intrinsic limit.

`drainTo()`

: Removes all available elements from the queue and adds them to a specified `Collection`. It does not block to wait for elements to be added to the queue. A variant of the method accepts a maximum number of elements to drain.

The `java.util.concurrent` package contains a number of `BlockingQueue` implementations, which are designed for use in multithreaded programs. The overall intent is to remove synchronization and provide safe handoff between threads.

**WARNING**

Regardless of the result of calling `remainingCapacity()`, it is impossible to tell whether an attempt to insert

an element will succeed because it may be the case that (an unbounded number of) other threads are about to insert an element.

The `BlockingQueue` is a key building block for concurrent code, but it still leaves the problem that manually managing threads by hand is difficult.

Java addresses this issue by providing managed threadpools to handle the details of thread lifecycle management and freeing up the programmer to think about higher-level design and concerns—and these capabilities are the subject of our next section.

## Executors and Threadpools

Java's preferred approach is to emphasize the computation that needs to be done as tasks that can be executed without spinning up a new thread for each one. This means that the tasks should be modeled as code that can be called, for example, as instances of `Runnable` or `Callable`.

Then these individual tasks can be scheduled on a shared resource—a pool of threads—that executes a task to completion and then moves on to the next task.

The key interface here is `ExecutorService` from `java.util.concurrent`, and the package also defines a general-purpose implementation—the `ThreadPoolExecutor` class (TPE) that is extremely capable.

**WARNING**

There is an `Executor` interface available, but in practice it lacks important lifecycle capabilities and so is not widely used.

A key feature of TPE is that it includes a *task queue*—a blocking queue where work units can be stored if they cannot be executed immediately. This means that the thread that is submitting a job for execution can do so in a nonblocking and threadsafe manner (provided the task queue has space).

TPE has some other useful properties—for example, it will detect if a worker thread dies and automatically create a replacement if required. It is very possible to use TPE directly, but it is very configurable—and some of the config parameters have some subtlety associated with them.

To help developers who are new to concurrency avoid dealing with this complexity, Java provides some convenience factory methods in `Executors` that return threadpools that are pre-configured to work in specific roles. A full discussion of each option is, unfortunately, outside the scope of this book, but let's deal with two very common choices—the *fixed threadpool* and the *cached threadpool*.

The fixed threadpool is created from the `Executors.newFixedThreadPool()` factory method. The user provides an explicit thread count, and the pool is created with the

corresponding number of threads.

These threads will be reused to run multiple tasks, so individual tasks don't have to pay the cost of creating a new thread. When a task is submitted, if all threads are in use, then the new task will be added to the queue until a thread becomes free.

> **WARNING**
>
> The pool threads will take turns accepting tasks from the queue in a nondeterministic manner. Even if there is a strict temporal ordering of when tasks are submitted, there is no guarantee of which pool thread will handle a specific task.

The fixed threadpool is a good baseline choice—especially when the workload is known and fairly stable. However, it is not the only possibility, and some situations will benefit from a different threadpool implementation. For example, if the profile of the work is more erratic, then a pool with a fixed number of threads is not necessarily the best choice, as some jobs will be left waiting at busy periods.

The other implementation that we'll mention is the `CachedThreadPool`. It is usually created via `Executors.newCachedThreadPool()` and is an unbounded pool. This means it will reuse threads if they are available but will create as many new threads as required to handle incoming tasks.

Once created, threads that have finished executing a job are kept around (in the *idle cache*) for 60 seconds, in case new jobs arrive. If more tasks arrive, the idle threads can be reused, avoiding the cost of thread creation. On the other hand, if they are still in the cache at the end of that period, they will be removed from the cache and shut down.

Finally, a word of warning—the tasks that are submitted to Java's threadpools must actually terminate. If not, then the cached threadpool will, over time, create more and more threads and consume more and more of the machine's resources and eventually crash or become unresponsive. In the case of the fixed threadpool, nonterminating jobs will "hoard" a thread from the pool, and the automatic recovery mechanism will not kick in.

This means that the effective number of pool worker threads will permanently decrease by one. If this happens a number of times, the throughput of the threadpool will drop to zero and no further jobs will be processed—a serious concurrency bug that must be avoided.

In this section we have only scratched the surface of what is possible with the highly capable toolkit provided by `java.util.concurrent`. The interested reader who wants to go further should refer to more advanced texts, such as *Java Concurrency in Practice* (Addison-Wesley) by Brian Goetz et al.

One obvious next step is to look at the `CompletableFuture` class, as it is a very common workhorse class in modern concurrent Java programs. However, it does represent a significant increase in sophistication, and so we prefer to refer the reader to more advanced material rather than attempt a very limited discussion here.

Let's move on to our final topic for this chapter—a very quick introduction to the new virtual threads feature that arrived with Java 21. This is a relatively advanced topic and can be skipped on a first reading.

## Virtual Threads

Before Java 21, every executing Java language thread was an OS thread, because calling `Thread.start()` triggered a system call that created an OS thread. This creates a constraint between the number of available Java execution contexts and the limits of the OS. Specifically, the OS has data structures (e.g., fixed-size execution stacks) that it creates for each thread, and it individually schedules execution of each thread via the OS run queue.

As the Java ecosystem and the demands of modern systems have evolved, this constraint has become more problematic. This naturally leads to the question: how many OS threads can an application start? 1,000? Perhaps 10,000? Regardless of the exact number, there is definitely a hard limit in this approach that will eventually be reached.

To transcend this limitation, a reimagining of Java's concurrency model was needed—and the key concept is *virtual threads* (or vthreads), a new construct that does not have a one-to-one relationship with OS threads. Java's existing threads are now referred to as *platform threads* to distinguish them from virtual threads.

> **WARNING**
>
> Virtual threads are very different from approaches such as `async`/`await` that developers may have used in other languages (e.g., JavaScript). There is no current intention to bring those approaches to Java.

From a Java programming perspective, virtual threads are instances of `VirtualTh read`, which is a final subclass of `Thread`, but they are managed by the JVM, not the OS. This means that no OS-level data structures (e.g., for the thread's stack frames) are created when a vthread is started, and all the thread management metadata is handled by the JVM. This includes the scheduling behavior—rather than the OS scheduler, a Java execution scheduler (i.e., a threadpool) is used.

When a virtual thread wants to execute, it does so on an OS *carrier thread* and runs until a blocking call (such as I/O) is made.[7] The carrier thread is *yielded* to another vthread, while the blocking call is handed off to run in the background. This arrangement means that a vthread may execute on several different carriers over its lifetime.

> **WARNING**
>
> The connection to blocking calls means that vthreads are not suitable for purely CPU-bound tasks, as there are no suitable execution points where the vthread can relinquish its carrier thread.

Java programmers are required to specifically create a vthread if they want one—there is no "automatic virtualization" of platform threads, nor is there planned to be. This is important for several reasons, but one of the most important is that the arrival of virtual threads should not change the meaning—or performance characteristics—of any existing code.

To obtain a virtual thread, new static methods have been added to `Thread ofPlat form()` and `ofVirtual()`) that dovetail into a Builder pattern that can be used like this:

```
Thread.Builder tb = Thread.ofVirtual();
tb.name("MyVThread");
Thread t = tb.unstarted(() -> System.out.println("Hello World!"));
System.out.println(t);
t.start();
```

Thread builders can set a name and then be built into either a started or unstarted thread by supplying a `Runnable` task and then calling `start()` or `unstarted()`. There are also thread factories available from the builders via the `factory()` method to provide additional flexibility and customization if needed.

When started, vthreads look just like platform threads from the outside, and care has been taken to ensure this isolation. For example, a vthread cannot directly observe its current carrier, so `Thread.currentThread()` will return the vthread, and the stack frames from the carrier will not show up.

There is also a new executor available—`newVirtualThreadPerTaskExecutor()`. It starts a new vthread for each submitted job, for a type of Fire-and-Forget pattern (which is similar to Go's *goroutines*, for readers who are familiar with them). The new executor is not based on TPE, and it can be used with `try`-with-resources clauses, because the executor implements `AutoCloseable`, for example:

```
public static void handle(URL[] urls) {
    Future<String>[] fs = new Future[urls.length];
    try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
        for (var i = 0; i < urls.length; i = i + 1) {
            var url = urls[i];
            fs[i] = executor.submit(() -> fetchURL(url));
        }
        var sb = new StringBuilder();
        for (var f : fs) {
            sb.append(f.get());
        }
        IO.println(sb);
    } catch (ExecutionException | InterruptedException e) {
        e.printStackTrace();
    }
}

public static String fetchURL(URL url) throws IOException {
    try (var in = url.openStream()) {
        return new String(in.readAllBytes(), StandardCharsets.UTF_8);
    }
}
```

In this example, each URL is fetched by a separate vthread, and the results are combined after all requests have completed.

It's important not to just apply vthreads blindly, as there are some important caveats regarding their use. We've already mentioned that you should not use vthreads for compute-bound tasks, but here are some other things to consider.

First, due to the extremely lightweight nature of vthreads, they should not be treated as costly objects that need to be saved and reused (as platform threads sometimes are, e.g., in the fixed threadpool pattern)—in other words, do not use vthreads with an object pooling pattern.

Second, tools, especially monitoring and analysis tools such as JDK Mission Control, may not be equipped to deal with potentially vast numbers of vthreads—they may need to be upgraded to new versions that are vthread-aware.

Third, don't think of vthreads as a free lunch. Developers must learn what types of problems they'll help with. In particular, there may be useful examples from how goroutines are used in the Go ecosystem that can be used as a starting point for using vthreads in Java.

Ultimately, virtual threads are still a work in progress, and we should expect further enhancements over time. For example, Java 25 introduced *Scoped Values* and provided an unfinalized preview of *Structured Concurrency* (both advanced topics outside the scope of this book).

It also remains to be seen how much vthreads will directly impact end-user code. There is a lot of interest from framework and library authors, but that may be the main way that application writers make use of them, rather than directly.

## Summary

In this chapter, we've discussed Java's view of concurrency and how, as CPUs develop more and more cores, we will need to use concurrent programming techniques to use those cores effectively. In other words, concurrency is key to the future of well-performing applications.

Java's threading model is based on three fundamental concepts:

*Shared, visible-by-default mutable state*

> Objects are easily shared between different threads in a process, and they can be changed ("mutated") by any thread holding a reference to them.

*Preemptive thread scheduling*

> The OS thread scheduler can swap threads on and off cores at more or less any time.

*Object state can only be protected by locks*

> Locks can be hard to use correctly, and state is quite vulnerable—even in unexpected places such as read operations.

Taken together, these three aspects of Java's approach to concurrency explain why multithreaded programming can cause so many headaches for developers.

We've also explored high-level concurrency concepts and learned that:

- `java.util.concurrent` classes should be your preferred toolkit for all new multithreaded Java code.
- Executors, threadpools, and concurrent data structures are essential building blocks.
- Virtual threads were added with Java 21 and enhanced in Java 25 and are a major modernization of Java's approach to concurrency.

Let's move on to discuss our next topic—the technologies that provide the foundations of many of Java's dynamic and runtime features.

---

1 JavaScript has a completely different model from both Java and Python and works purely by an *event loop*, which can be the root cause of major headaches in complex JavaScript code.

2 `BLOCKED` and `WAITING` threads are implicitly at a safepoint.

3 This concept has nothing to do with Java's access control keywords, such as `private`.

4 Work is underway in Project Valhalla to define a new type of Java object that does not have identity, or monitors—see Appendix A for details.

5 Not all implementations of locks have this property, but Java's intrinsic monitors do.

6 The details of how atomics achieve better performance than synchronization are outside the scope of this book—specialist material such as *Optimizing Cloud Native Java* (O'Reilly) or *Java Concurrency in Practice* (Addison-Wesley) should be consulted if desired.

7 The Java socket library implementation was rewritten in Java 17 to use nonblocking I/O internally even for blocking calls.

# Chapter 13. Classloading, Reflection, and Method Handles

In Chapter 4, we met Java's `Class` objects, a way of representing a live type in a running Java process. In this chapter, we will build on this foundation to discuss how the Java environment loads and makes new types available at runtime.

In the second half of the chapter, we'll introduce Java's introspection capabilities, both the original Reflection API and the newer Method Handles capabilities.

These capabilities are important because they provide the dynamic features of the Java platform. Remember back in Chapter 1 we said that although Java source code looks like C/C++ and it seems to be a compiled language, at runtime it is a different story?

This chapter will provide an introduction to how this is achieved—and why Java is capable of being as flexible and dynamic as Python or JavaScript or Ruby. Let's get started by taking a closer look at class files and `Class` objects.

## Class Files, Class Objects, and Metadata

Class files, as we saw in Chapter 1, are the result of compiling Java source files (or, potentially, other languages) into the intermediate form used by the JVM. These are binary files that are not designed to be human-readable. At runtime these files are represented by class objects that contain metadata about the Java type defined in the file.

### Examples of Class Objects

You can obtain a class object in Java in several ways. The simplest is:

```
Class<?> myClass = getClass();
```

This returns the class object of the instance that it is called from. However, as we know from our survey of the public methods of `Object`, the `getClass()` method on `Object` is public, so we can also obtain the class of an arbitrary object `o`:

```
Class<?> c = o.getClass();
```

Class objects for known types can also be written as "class literals":

```
// Express a class literal as a type name followed by ".class"
c = String.class; // Same as "a string".getClass()
c = byte[].class; // Type of byte arrays
```

For primitive types and `void`, we also have class objects that are represented as literals:

```
// Obtain a Class object for primitive types with various
// predefined constants
c = Void.TYPE; // The special "no-return-value" type
c = Byte.TYPE; // Class object that represents a byte
c = Integer.TYPE; // Class object that represents an int
c = Double.TYPE; // etc.; see also Short, Character, Long, Float
```

There is also the possibility of using the `.class` syntax directly on a primitive type, like this:

```
c = int.class; // Same as Integer.TYPE
```

At first glance, this might seem a bit odd—how can we have a class object for `int` when it is not a reference type? However, as we will see, there is a need to be able to talk about and represent the primitive type `int` at runtime, and this is the (somewhat imperfect) solution.

The relationship between `.class` and `.TYPE` can be seen with some simple tests:

```
// outputs true
System.out.printf("%b%n", Integer.TYPE == int.class);

// outputs false
System.out.printf("%b%n", Integer.class == int.class);

// outputs false
System.out.printf("%b%n", Integer.class == Integer.TYPE);
```

Note that the wrapper types (`Integer`, etc.) have a `.TYPE` property, but in general classes do not.

> **NOTE**
>
> For almost all types, there is only one class object of each type—e.g., `String.class` is the only object of type `Class<String>` (and there is no way to create additional objects of this type). As we have seen, the wrapper types are the exception—there are *two* class objects for each wrapper type (e.g., `int.class` and `Integer.class` are the only two objects of type `Class<Integer>`).

Also, all of this works only for types that are known at compile time; for unknown types, we will have to use more sophisticated methods.

## Class Objects and Metadata

The class objects contain *metadata* about the given type. This includes the methods, fields, constructors, and the like that are defined on the class in question. This metadata can be accessed by the programmer to investigate the class, even if nothing is known about the class when it is loaded.

For example, we can find all the deprecated methods in the class file (they will be marked with the `@Deprecated` annotation):

```
Class<?> clz =  ... // Get class from somewhere, e.g., loaded from disk
for (Method m : clz.getMethods()) {
  for (Annotation a : m.getAnnotations()) {
    if (a.annotationType() == Deprecated.class) {
      System.out.println(m.getName());
    }
  }
}
```

We could also find the common ancestor of a pair of classes. This simple form will work when both classes have been loaded by the same classloader (we'll meet classloaders properly in the next section):

```
public static Class<?> commonAncestor(Class<?> cl1, Class<?> cl2) {
    if (cl1 == null || cl2 == null) return null;
    if (cl1.equals(cl2)) return cl1;
    if (cl1.isPrimitive() || cl2.isPrimitive()) return null;
    if (cl1.isInterface() || cl2.isInterface()) return null;
    Set<Class<?>> ancestors = new HashSet<>();
    Class<?> c = cl1;
    while (!c.equals(Object.class)) {
        if (c.equals(cl2)) return c;
        ancestors.add(c);
        c = c.getSuperclass();
    }
    c = cl2;
    while (!c.equals(Object.class)) {
        if(ancestors.contains(c)) return c;
        c = c.getSuperclass();
    }
    return Object.class;
}
```

Note that this will also deal with the cases where a supplied class object is, for example, `int.class` or an interface.

Class files have a very specific layout they must conform to if they are to be legal and loadable by the JVM. The sections of the class file are (in order):

- Magic number (all class files starting with the four bytes `CA FE BA BE` in hexadecimal)[1]
- Version of class file standard in use
- Constant pool for this class
- Access flags (`abstract`, `public`, etc.)
- Name of this class

- Inheritance info (e.g., name of superclass)
- Implemented interfaces
- Fields
- Methods
- Attributes

The class file is a simple binary format, but as it is not human-readable, tools like `javap` (see Appendix B) should be used to comprehend the contents.

One of the most frequently used sections in the class file is the *constant pool*, which contains representations of all the methods, classes, fields, and constants that the class needs to refer to (whether they are in this class or another). It is designed so that bytecodes can simply refer to a constant pool entry by its index number—which saves space in the bytecode representation.

A number of different class file versions are created by various Java versions. However, one of Java's backward-compatibility rules is that JVMs (and tools) from newer versions can always use older class files.

Now, let's look at how the classloading process can be used to add a new type to a running JVM process.

## Phases of Classloading

Classloading is the process by which a collection of bytes (located on disk or elsewhere) is taken and turned into a new class object.

> **NOTE**
>
> Classloading is the only way that additional code can enter the system after startup, and the only way to turn data into code in the Java platform. There is no equivalent of `eval`, which you might have met in JavaScript or other languages.

There are several phases to the process of classloading. Let's examine them in turn.

### Loading

The classloading process starts with loading a byte array. This is often read in from a filesystem, but it also can be read from a URL or other location (often represented as a `Path` object). It can even be read from a Java byte array—meaning that Java programs can define brand-new classes at runtime in an entirely self-contained way.

The `ClassLoader::defineClass()` method is responsible for turning a class file (represented as a byte array) into a class object. It is a protected method and so is not accessible

without subclassing.

The first job of `defineClass()` is loading. This produces the skeleton of a class object, corresponding to the class you're attempting to load. By this stage, some basic checks have been performed on the class (e.g., the constants in the constant pool have been checked to ensure that they're self-consistent).

However, loading doesn't produce a complete class object by itself, and the class isn't yet usable. Instead, after loading, the class must be linked. This step breaks down into separate subphases:

- Verification
- Preparation and resolution
- Initialization

## Verification

Verification confirms that the class file conforms to expectations and that it doesn't try to violate the JVM's security model (see "Secure Programming and Classloading" for details).

JVM bytecode is designed so it can be (mostly) checked statically. This has the effect of slowing down the classloading process but speeding up runtime (since checks can be omitted).

> **NOTE**
>
> The default methods mechanism works via classloading. When an implementation of an interface is being loaded, the class file is examined to see if implementations for default methods are present. If they are, classloading continues normally. If some are missing, the implementation is patched to add in the default implementation of the missing methods.

The verification step is designed to prevent the JVM from executing bytecodes that might crash it or put it into an undefined and untested state where it might be vulnerable to other attacks by malicious code. Bytecode verification is a defense against malicious handcrafted Java bytecodes and untrusted Java compilers that might output invalid bytecodes.

## Preparation and Resolution

After successful verification, the class is prepared for use. Memory is allocated, and static variables in the class are readied for initialization.

At this stage, variables aren't initialized, and no bytecode from the new class has been executed. Before we run any code, the JVM checks that every type referred to by the new class file is known to the runtime. If the types aren't known, they may also need to be loaded—which can kick off the classloading process again, as the JVM loads the new types.

This process of loading and discovery can execute iteratively until a stable set of types is reached. This is the transitive closure of the original type that was loaded, just as in Chapter 9 we

met the concept of transitive closure as it relates to live objects.

Let's look at a quick example by examining the dependencies of `java.lang.Object`.

Figure 13-1 shows a simplified dependency graph for `Object`.

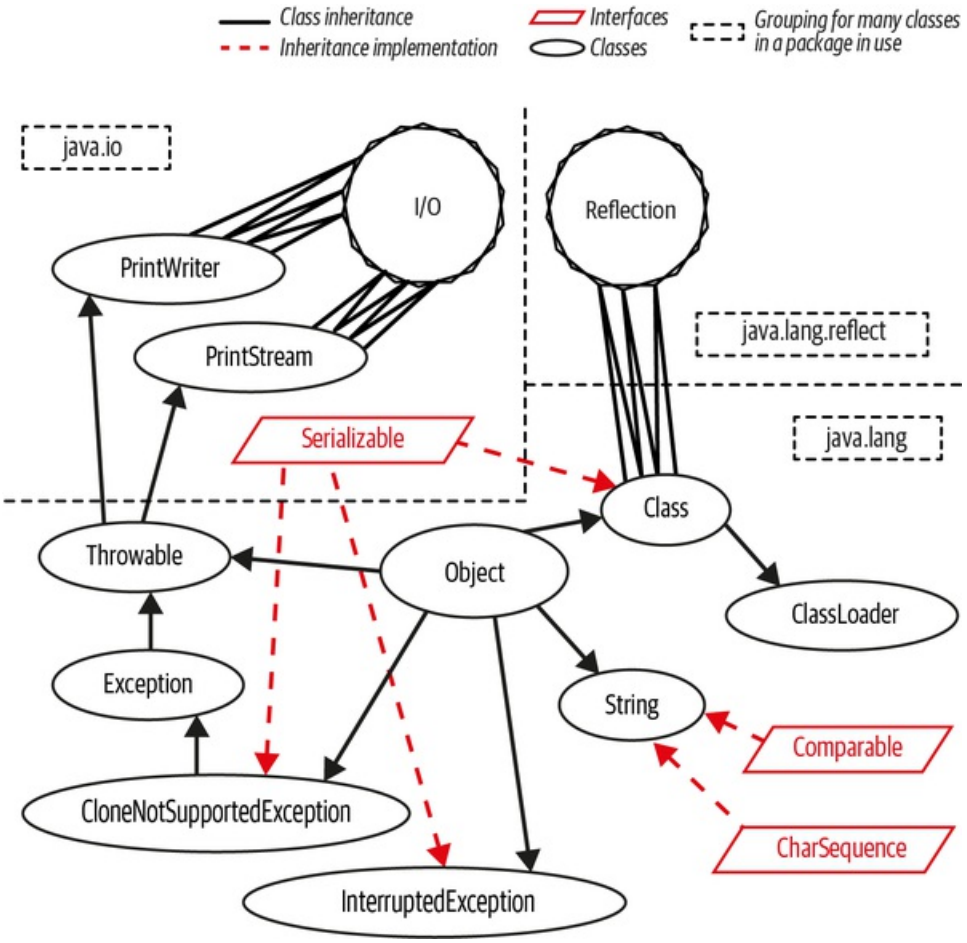



*Figure 13-1. Transitive closure of types*

Figure 13-1 shows only the direct dependencies of `Object` that are visible in the public API of `Object` and the direct, API-visible dependencies of those dependencies. In addition, the dependencies of `Class` on the reflection subsystem, and of `PrintStream` and `PrintWriter` on the I/O subsystems, are shown in very simplified form.

Thus, in Figure 13-1, we can see part of the transitive closure of `Object`.

## Initialization

Once resolved, the JVM can finally initialize the class. Static variables can be initialized, and static initialization blocks are run.

This is the first time that the JVM is executing bytecode from the newly loaded class. When the static blocks complete, the class is fully loaded and ready to go.

## Secure Programming and Classloading

Java programs can dynamically load Java classes from a variety of sources, including untrusted sources, such as websites reached across an insecure network. The ability to create and work with such dynamic sources of code is one of the great strengths and features of Java. To make it work successfully, however, Java puts great emphasis on a security architecture that allows untrusted code to run safely, without fear of damage to the host system.

Java's classloading subsystem is where a lot of safety features are implemented. The central idea of the security aspects of the classloading architecture is that there is only one way to get new executable code into the process: a class.

This provides a "pinch point"—the only way to create a new class is to use the functionality provided by `ClassLoader` to load a class from a stream of bytes. By concentrating on making classloading secure, we can shrink the attack surface that needs to be protected.

One extremely helpful aspect of the JVM's design is that the JVM is a stack machine, so all operations are evaluated on a stack, rather than in registers. The stack state can be deduced at every point in a method, and this can be used to ensure that the bytecode doesn't attempt to violate the security model.

Some of the security checks implemented by the JVM are:

- All the bytecode of the class has valid parameters.
- All methods are called with the right number of parameters of the correct static types.
- Bytecode never tries to underflow or overflow the JVM stack.
- Local variables are not used before they are initialized.
- Variables are assigned only suitably typed values.
- Field, method, and class access control modifiers must be respected.
- There are no unsafe casts (e.g., attempts to convert an `int` to a pointer).
- All branch instructions are to legal points within the same method.

Of fundamental importance is the approach to memory, and pointers. In assembly and C/C++, integers and pointers are interchangeable, so an integer can be used as a memory address. We can write it in assembly like this:

```
mov eax, [STAT] ; Move 4 bytes from addr STAT into eax
```

The lowest level of the Java security architecture involves the design of the JVM and the bytecodes it executes. The JVM does not allow any kind of direct access to individual memory addresses of the underlying system, which prevents Java code from interfering with the native hardware and operating system. These intentional restrictions on the JVM are reflected in the Java language itself, which does not support pointers or pointer arithmetic.

In fact, neither the language nor the JVM bytecode allows an integer to be converted to an object reference, or vice versa, and there is no way whatsoever in bytecode to obtain an object's address in memory. Without capabilities like these, malicious code that leads to this class of bugs simply cannot gain a foothold.

Recall from Chapter 2 that Java has two types of values—primitives and object references. These are the only things that can be put into variables. Note that "object contents" cannot be put into variables. Java has no equivalent of C's `struct` and always has pass-by-value semantics. For reference types, what is passed is a copy of the reference—which is a value.

References are represented in the JVM as pointers, but those pointers are not directly manipulated by the bytecode. In fact, bytecode does not even have opcodes for "access memory at location X."

Instead, all we can do is access fields and methods; bytecode cannot call an arbitrary memory location. This means that the JVM always knows the difference between code and data. As another example, bytecode has "local goto within a method" and "call method" opcodes but no way of calling arbitrary memory locations. The careful design of these types of features prevents entire categories of stack buffer overflow and other types of attack.

## Applied Classloading

The Java class `java.lang.ClassLoader` is absolutely central to fully understanding the classloading mechanism. The first thing to notice is that it is, indeed, a Java type and so it can be accessed and used by ordinary Java code.

In addition to the `defineClass()` method that we've already met, we can also load classes via a public `loadClass()` method. This is commonly used by the `URLClassLoader` subclass, which can load classes from a URL or filepath.

We can use `URLClassLoader` to load classes from the local disk like this:

```
var current = new File( "." ).getCanonicalPath();
var urls = new URL[] {new URL("file://" + current + "/")};
try (URLClassLoader loader = new URLClassLoader(urls)) {
  Class<?> clz = loader.loadClass("com.example.DFACaller");
  System.out.println(clz.getName());
}
```

The argument to `loadClass()` is the binary name of the class file. Note that for the `URLClassLoader` to find the classes correctly, they need to be in the expected place on the filesystem. In this example, the class `com.example.DFACaller` would need to be found in the file *com/example/DFACaller.class* relative to the working directory.

Alternatively, `Class` provides `Class.forName()`, a static method that can load classes that are present on the classpath but haven't been referred to yet.

This method takes a fully qualified class name. For example:

```
Class<?> jdbcClz = Class.forName("oracle.jdbc.driver.OracleDriver");
```

It throws a `ClassNotFoundException` if the class can't be found. As the example indicates, this was commonly used in older versions of Java Database Connectivity (JDBC) to ensure that the correct driver was loaded while avoiding a direct `import` dependency on the driver classes. With the advent of JDBC 4.0, this initialization step is no longer required.

`Class.forName()` has an alternative, three-argument form, which is sometimes used in conjunction with alternative classloaders:

```
Class.forName(String name, boolean init, ClassLoader classloader);
```

There are a host of subclasses of `ClassLoader` that deal with individual special cases of classloading—all of which fit into the classloader hierarchy.

## Classloader Hierarchy

The JVM has a hierarchy of classloaders; each classloader in the system (apart from the initial, "bootstrap" classloader) has a parent that it can delegate to.

> **NOTE**
>
> The arrival of modules in Java 9 has affected the details of the way that classloading operates. In particular, the classloaders that load the Java Runtime Environment (JRE) classes are now *modular classloaders*.

The convention is that a classloader will ask its parent to resolve and load a class, and it will perform the job itself only if the parent classloader is unable to comply. Some common classloaders are shown in Figure 13-2.

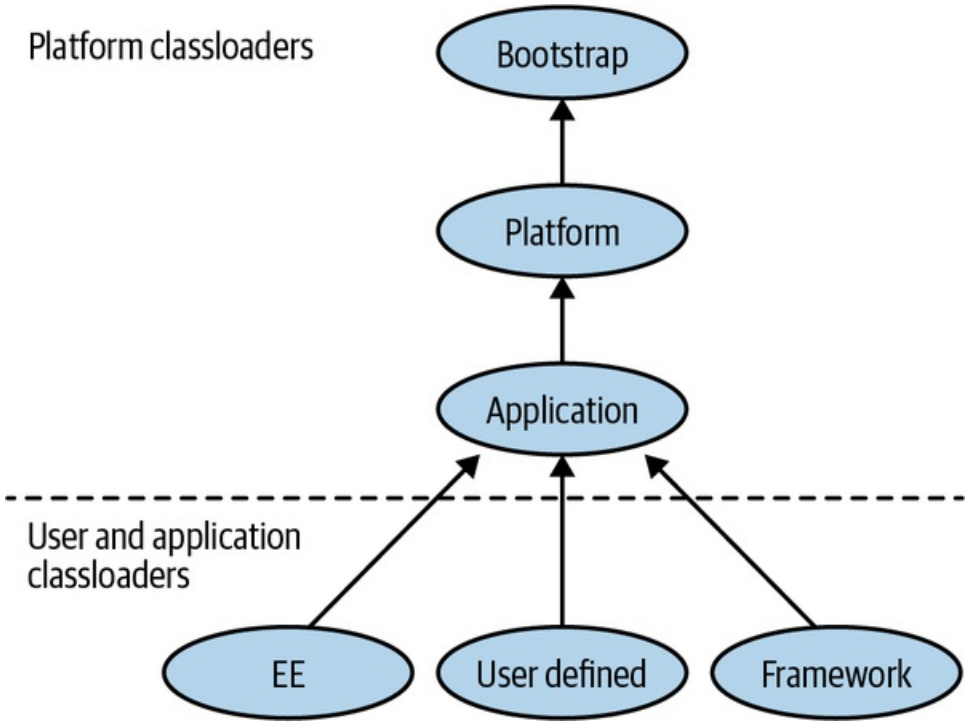



*Figure 13-2. Classloader hierarchy*

### Bootstrap classloader

This is the first classloader to appear in any JVM process and is only used to load the core system classes. In older texts, it is sometimes referred to as the *primordial classloader*, but modern usage favors the bootstrap name.

For performance reasons, the bootstrap classloader does no verification and relies on the boot classpath being secure. Types loaded by the bootstrap classloader are implicitly granted all security permissions, and so this group of modules is kept as restricted as possible.

### Platform classloader

This level of the classloader hierarchy was originally used as the *extension classloader*, but this mechanism has now been removed.

In its new role, this classloader (which has the bootstrap classloader as its parent) is now known as the *platform classloader*. It is available via the method `ClassLoader::getPlatformClassLoader` and appears in (and is required by) the Java specification from version 9 onward. It loads the remaining modules from the base system (the equivalent of the old `rt.jar` used in version 8 and earlier).

Since the arrival of modular JDKs (with Java 9, in 2017), far less code is required to bootstrap a Java process; accordingly, as much JDK code (now represented as modules) as possible has been moved out of the scope of the bootstrap loader and into the platform loader instead.

### Application classloader

Historically, this was sometimes called the *system classloader*, but this is a bad name, as it doesn't load the system (the bootstrap and platform classloaders do). Instead, it is the classloader that loads application code from either the module path or the classpath. It is the most commonly encountered classloader, and it has the platform classloader as its parent.

To perform classloading, the application classloader first searches the named modules on the module path (the modules known to any of the three built-in classloaders). If the requested class is found in a module known to one of these classloaders, then that classloader will load the class. If the class is not found in any known named module, the application classloader delegates to its parent (the platform classloader). If the parent fails to find the class, the application classloader searches the classpath. If the class is found on the classpath, it is loaded as a member of the application classloader's unnamed module.

The application classloader is very widely used, but many advanced Java frameworks require functionality that the main classloaders do not supply. Instead, extensions to the standard classloaders are required.

### Custom classloader

`ClassLoader` is a fully functional abstract class and has no abstract methods. This might seem like an odd choice, perhaps even a design flaw, but the `abstract` modifier is used to ensure users must subclass `ClassLoader` to extend it.

Subclassing forms the basis of what is known as *custom classloading*—in which additional functionality is supplied by the programmer. Recall that when performing classloading, sooner or later we have to turn data into code. As we saw earlier, the `defineClass()` (actually a group of related methods) is what is responsible for actually converting a `byte[]` into a class object.

The existing JDK classloaders allow us to source that byte array from a variety of sources, but they aren't comprehensive. For example, suppose that we want the user to supply the location of a class to be loaded at runtime—this would be the case for plug-ins, etc. If the location is not on the classpath, then the standard JDK classloaders will not be able to find it.

The solution is to write a custom classloader that can read from a nonstandard location. As it is a subclass of `Classloader`, it will have access to the protected method `defineClass()`, which we can call from a public method, as shown in this example:

```
public final class DiskLoader extends ClassLoader {
  public DiskLoader() {
    super(DiskLoader.class.getClassLoader());
  }

  public Class<?> loadFromDisk(String clzPath) throws IOException {
    byte[] b = Files.readAllBytes(Path.of(clzPath));

    return defineClass(null, b, 0, b.length);
  }
}
```

One important detail in the example is that we need to provide a classloader to act as parent for any custom classloader. In this example, we provided the classloader that loaded the `DiskLoader` class (which would usually be the application classloader).

Custom classloading is a very common technique in Java, and it provides the basis of some sophisticated techniques. We'll see a more extensive example of custom classloading later in this chapter.

One drawback of dynamic classloading is that when working with a class object that we loaded dynamically, we typically have little or no information about the class. To work effectively with these sorts of classes, we therefore have to use an indirect style of programming.

Java provides *two* separate APIs to facilitate this style of programming—*reflection* and *method handles*. The first of these is older but is much more common in examples and tutorials. Some people also find it easier to understand, so we'll start by discussing it before moving on to the more modern alternative.

## Reflection

Reflection (or *reflective programming*) is the capability of examining, operating on, and modifying objects at runtime without compile-time knowledge of their types or structure.

> **WARNING**
>
> The Java modules system introduces major changes to how reflection works on the platform.[2] It is important to reread this section after you have gained an understanding of how modules work and how the two capabilities interact. More details on how modules restrict reflection are available in "Open Modules".

Reflection is capable of working even when type and method names are not known at compile time. It uses the essential metadata provided by class objects and can discover method or field names from the class object—and then acquire an object representing the method or field. These objects are of type `Method` and `Field`, respectively.

Instances can also be constructed reflectively (by obtaining a `Constructor` object from `Class::getConstructor()` and invoking it). With a reflectively constructed object and a `Method` object, we can then call that method on an object of a previously unknown type.

This makes reflection a very powerful technique, so it's important to understand when we should use it and when it's overkill.

### When to Use Reflection

Many, if not most, Java frameworks use reflection in some capacity. Writing architectures that are flexible enough to cope with code that is unknown until runtime usually requires reflection. For example, plug-in architectures, debuggers, code browsers, and read-evaluate-print loop

(REPL)–like environments are usually implemented on top of reflection.

Reflection is also widely used in testing (e.g., by the JUnit and TestNG libraries) and for mock object creation. If you've used any kind of Java framework, you have almost certainly been using reflective code, even if you didn't realize it. In some ways, that's as it should be—reflection is a somewhat advanced technique, and a good library writer will create something that can be used by everyone. You shouldn't need to know a bunch of advanced techniques to use frameworks—the magic should be hidden, and the cognitive burden placed on those who create and maintain frameworks, not on their users.

Having said that, as you progress as a Java programmer, it's useful to have at least met reflection and to understand at a high level how it works. So let's take a look.

To start using the Reflection API in your own code, the most important thing to realize is that it is about accessing objects where virtually no information is known, and that the interactions with those objects can be cumbersome because of this.

> **NOTE**
>
> If some static information *is* known about dynamically loaded classes (e.g., that the classes loaded all implement a known interface), this can greatly simplify the interaction with the classes and reduce the burden of operating reflectively.

It is a common mistake to try to create a reflective framework that attempts to account for all possible circumstances, instead of dealing only with the cases that are immediately applicable to the problem domain.

## How to Use Reflection

The first step in any reflective operation is to get a `Class` object representing the type to be operated on. From this, other objects, representing fields, methods, or constructors, can be accessed and applied to instances of the unknown type.

If we already have an instance of an unknown type, we can retrieve its class via the `getClass()` method.[3] Alternatively, the static `Class.forName()` method demonstrated in "Applied Classloading" for classloading can also perform lookup of a `Class` object by name:

```
var clzForInstance = "Hi".getClass();
var clzForName = Class.forName("java.lang.String");
```

Once we have an instance of a `Class` object, the next reasonable step is calling a method reflectively. The `Method` objects are some of the most commonly used objects provided by the Reflection API. We'll discuss them in detail—the `Constructor` and `Field` objects are similar in many respects.

### Method objects

A class object contains a `Method` object for each method on the class.

> **NOTE**
>
> Method objects are created lazily after classloading, and so they aren't immediately visible in an IDE's debugger.

Methods on `Class` allow us to retrieve (and if necessary lazily initialize) these `Method` objects:

```
var clz = Class.forName("java.lang.String");

// Returns list of all publicly visible methods on clz
var publicMethods = clz.getMethods();

// Returns named method from clz, or throws
var toString = clz.getMethod("toString", new Class[] {});
```

The second parameter to `getMethod()` takes an array of `Class` objects representing the method's parameters to distinguish between method overrides.

The code demonstrated here will list and find only public methods on our `Class` objects. There are alternative methods of the form `getDeclaredMethod` that parallel what we've shown that allow access to protected and private methods. We'll have more to say shortly about using these mechanisms to circumvent Java's access model, though.

Like any good Java object, `Method` provides accessors for all the relevant information about the method. Let's look at the most critical metadata about a method that we can retrieve:

```
var clz = Class.forName("java.lang.String");
var toString = clz.getMethod("toString", new Class[] {});

// The method's name
String name = toString.getName();

// Generic type information for the method
TypeVariable[] typeParams = toString.getTypeParameters();

// Array of method annotations with RUNTIME retention
Annotation[] ann = toString.getAnnotations();

// Array of checked exception types declared by method
Class[] exceptions = toString.getExceptionTypes();

// Array of Parameter objects for calling the method
Parameter[] params = toString.getParameters();

// Array of just the `Class` for each parameter to the method
Class[] paramTypes = toString.getParameterTypes();

// Class of the method's return type
Class ret = toString.getReturnType();
```

We can explore the metadata of a `Method` object by calling accessor methods, but by far the single biggest use case for `Method` is reflective invocation.

The methods represented by these objects can be executed by reflection using the `invoke()` method on `Method`.

An example of invoking `hashCode()` on a `String` object follows:

```
Object rcvr = "a";
try {
  Class<?>[] argTypes = new Class[] { };
  Object[] args = null;

  Method meth = rcvr.getClass().getMethod("hashCode", argTypes);
  Object ret = meth.invoke(rcvr, args);
  System.out.println(ret);

} catch (IllegalArgumentException | NoSuchMethodException |
        SecurityException e) {
  e.printStackTrace();
} catch (IllegalAccessException | InvocationTargetException x) {
  x.printStackTrace();
}
```

Note that the static type of `rcvr` was declared to be `Object`, and so no static type information was used during the reflective invocation. The `invoke()` method also returns `Object`, so the actual return type of `hashCode()`, which is `int`, has been autoboxed to `Integer`.

This autoboxing (because `int` is not a subtype of `Object`) is one of the aspects of Reflection where you can see some of the slight awkwardness of the API—which we'll discuss in an upcoming section.

### Creating instances with Reflection

If you're looking to create new instances of a `Class` object, you'll find that the method lookups don't help—because constructors don't really have names (at least, not names that those APIs are able to find).

The simplest case—that of a no-argument constructor—is handled via a helper available provided by the `Class` object:

```
Class<?> clz = ... // Get some class object
Object rcvr = clz.getDeclaredConstructor().newInstance();
```

For constructors that take arguments, `Class` has methods like `getConstructor` that allow for finding the exact override you need. While they return a different type— `Constructor`— using them is otherwise very similar to what we've already seen for interacting with `Method` objects.

Let's put together what we've learned and look at an extended example. This will involve combining reflection with custom classloading to inspect a class file on disk for any deprecated

methods (i.e., those marked with `@Deprecated`):

```
public class CustomClassloadingExamples {
    public static class DiskLoader extends ClassLoader {

        public DiskLoader() {
            super(DiskLoader.class.getClassLoader());
        }

        public Class<?> loadFromDisk(String clzName)
          throws IOException {
            byte[] b = Files.readAllBytes(Paths.get(clzName));

            return defineClass(null, b, 0, b.length);
        }
    }

    public void findDeprecatedMethods(Class<?> clz) {
        for (Method m : clz.getMethods()) {
            for (Annotation a : m.getAnnotations()) {
                if (a.annotationType() == Deprecated.class) {
                    System.out.println(m.getName());
                }
            }
        }
    }

    public static void main(String[] args)
      throws IOException, ClassNotFoundException {
        var rfx = new CustomClassloadingExamples();

        if (args.length > 0) {
            DiskLoader dlr = new DiskLoader();
            Class<?> clzToTest = dlr.loadFromDisk(args[0]);
            rfx.findDeprecatedMethods(clzToTest);
        }
    }
}
```

In the example code, the user provides the location of a file on disk and the program attempts to classload it, using a custom classloader. If this is successful, the `findDeprecatedMethods()` method uses reflection to examine the methods of the class object one by one and prints out the names of any deprecated methods it finds.

This example showcases some of the power of reflective techniques, but there are also problems that come with using the API.

### Problems with Reflection

Java's Reflection API is one of the primary approaches to dealing with dynamically loaded code, but a number of annoyances in the API can make it somewhat awkward to deal with:

- `Object[]` is used heavily to represent call arguments and other instances.
- `Class[]` is used when talking about types.

- Methods can be overloaded on name, so we need an array of types to distinguish between methods.
- Representing primitive types can be problematic—we have to manually box and unbox.

`void` is a particular problem—there is a `void.class`, but it's not used consistently. Java has a slightly odd relationship with `void`—it is not actually a type at all (technically speaking, `void` is a JVM-level concept that indicates that the method's temporary evaluation stack is empty when the method returns)—and some places in the Reflection API use `void.class` and some use `null` instead.

This is cumbersome and can be error-prone—in particular, the slight verbosity of Java's array syntax can lead to errors.

One further problem is the treatment of non-`public` methods. As mentioned before, instead of using `getMethod()`, we must use `getDeclaredMethod()` to get a reference to a non-`public` method. Additionally, to call non-`public` methods, we must override the Java access control subsystem, calling `setAccessible()` to allow it to be executed:

```
public class MyCache {
  private void flush() {
    // Flush the cache...
  }
}

Class<?> clz = MyCache.class;
try {
  Object rcvr = clz.newInstance();
  Class<?>[] argTypes = new Class[] { };
  Object[] args = null;

  Method meth = clz.getDeclaredMethod("flush", argTypes);
  meth.setAccessible(true);
  meth.invoke(rcvr, args);
} catch (IllegalArgumentException | NoSuchMethodException |
        InstantiationException | SecurityException e) {
  e.printStackTrace();
} catch (IllegalAccessException | InvocationTargetException x) {
  x.printStackTrace();
}
```

Because reflection always involves unknown information, we just have to live with some of this verbosity. It's the price of using the dynamic, runtime power of reflective invocation.

## Method Handles

The second API that Java provides for indirect runtime method access is called *method handles*.

It was originally conceived as a way to help non-Java dynamic languages run efficiently on the JVM. Languages of that type (such as JavaScript or Ruby) face some technical hurdles when targeting the JVM. In particular, they often need to participate in *method dispatch decisions* (i.e.,

exactly which method needs to get called) at runtime.

For example, JavaScript code can change the definition of a method on a specific, individual object—which is impossible in Java.

So, to support this type of flexibility at the JVM level, a new bytecode—`invokedy namic`—was introduced with Java 7 (back in 2011). This bytecode was not used by Java 7 itself (and was not emitted by `javac`) but only by non-Java languages running on the JVM. When Java 8 (in 2014) arrived, `invokedynamic` was used to implement lambda expressions, and in modern Java it is an essential tool in an increasing number of areas of the Java standard library and runtime.

Method handles are a critical part of the `invokedynamic` capability, but they can also be used standalone. They provide an alternative way to invoke methods at runtime, for example, when method names are not known at compile time. But they do not offer a complete replacement for all of the Reflection API.

For example, there is no method handles equivalent to the `Class::getMethods()` method to get a list of available methods.

Instead, method handles is focused on method invocation—and in particular a cleaner, more complete and more JVM-friendly approach to invocation than that provided by Reflection.

> **NOTE**
>
> For new (aka greenfield) development, if your use case allows you to choose between Reflection and method handles, you should prefer the latter.[4]

Let's get started by meeting one concrete way that the Method Handles API improves on Reflection.

## MethodType

In the Reflection API, method signatures are represented as `Class[]`. This is quite cumbersome. By contrast, method handles rely on `MethodType` objects. These are a typesafe and object-oriented way to represent the type signature of a method.

Method handles include the return type and argument types but not the receiver type or name of the method. The name is not present, as this allows any method of the correct signature to be bound to any name (as per the functional interface behavior of lambda expressions).

A type signature for a method is represented as an immutable instance of `MethodType`, as acquired from the factory method `MethodType.methodType()`. The zeroth argument to `methodType()` is the return type of the method, with the types of the method arguments following it.

For example:

```
// Method type for toString()
MethodType m2Str = MethodType.methodType(String.class);

// Method type for Integer.parseInt()
MethodType mtParseInt =
  MethodType.methodType(Integer.class, String.class);

// Method type for defineClass() from ClassLoader
MethodType mtdefClz = MethodType.methodType(Class.class, String.class,
                                            byte[].class, int.class,
                                            int.class);
```

This single piece of the puzzle provides significant gains over Reflection, as it makes method signatures significantly easier to represent and discuss. Given the API's focus on invocation, the next step is to acquire a handle on a method, which is achieved by a lookup process.

## Method Lookup

Method *lookup queries* are performed on the class where a method is defined and are dependent on the context that the lookup is executed from:

```
// String.toString has only a return type and takes no parameters
MethodType mtToString = MethodType.methodType(String.class);

try {
  var l = MethodHandles.lookup();
  MethodHandle mh = l.findVirtual(String.class, "toString",
                                  mtToString);
  IO.println(mh);
} catch (NoSuchMethodException | IllegalAccessException e) {
  e.printStackTrace();
}
```

We always need to start by calling `MethodHandles.lookup()`—this gives us a lookup context object based on the currently executing method.

Lookup objects have several methods (which all start with `find`) declared on them for method resolution. These include `findVirtual()`, `findConstructor()`, and `findStatic()`. We need to provide the method name and signature (represented as a `MethodType` object) for the method resolution to find the method we need.

> **TIP**
>
> Method handles give us a way to get the current class from a static context without using a literal—via the static call `MethodHandles.lookup().lookupClass()`.

One big difference between the Reflection and Method Handles APIs is access control. A `Lookup` object will only return methods that are accessible to the context where the lookup was created—and there is no way to subvert this (i.e., no equivalent of Reflection's

`setAccessible()` hack).

For example, we can see that when we attempt to look up the protected `ClassLoader::defineClass()` method from a general lookup context, we fail to resolve it with an `IllegalAccessException`, as the protected method is not accessible:

```
public static void lookupDefineClass(Lookup l) {
  MethodType mt = MethodType.methodType(Class.class, String.class,
                                        byte[].class, int.class,
                                        int.class);

  try {
    MethodHandle mh =
      l.findVirtual(ClassLoader.class, "defineClass", mt);
    System.out.println(mh);
  } catch (NoSuchMethodException | IllegalAccessException e) {
    e.printStackTrace();
  }
}

Lookup l = MethodHandles.lookup();
lookupDefineClass(l);
```

Method handles therefore always comply with the Java security model, even when equivalent reflective code does not. They are access-checked at the point where the lookup context is constructed—the lookup object will not return handles to any methods to which it does not have proper access.

The lookup object, or method handles derived from it, can be returned to other contexts, including ones where access to the method would no longer be possible. Under those circumstances, the handle is still executable—access control is checked at lookup time and not at runtime, as we can see in this example:

```
public class SneakyLoader extends ClassLoader {
  public SneakyLoader() {
    super(SneakyLoader.class.getClassLoader());
  }

  public Lookup getLookup() {
    return MethodHandles.lookup();
  }
}

SneakyLoader snLdr = new SneakyLoader();
l = snLdr.getLookup();
lookupDefineClass(l);
```

With a `Lookup` object, we're able to produce method handles to any method we have access to. We can also produce a way of accessing fields that may not have a method that gives access. The `findGetter()` and `findSetter()` methods on `Lookup` produce method handles that can read or update fields as needed.

### Invoking Method Handles

A method handle represents the ability to call a method. They are strongly typed and as typesafe as possible. Instances are all of some subclass of `java.lang.invoke.MethodHandle`, which is a class that needs special treatment from the JVM.

There are two ways to invoke a method handle—`invoke()` and `invokeExact()`. Both of these take the receiver and call arguments as parameters. `invokeExact()` tries to call the method handle directly as is, whereas `invoke()` will massage call arguments if needed.

In general, `invoke()` performs an `asType()` conversion if necessary—this converts arguments according to these rules:

- A primitive argument will be boxed if required.
- A boxed primitive will be unboxed if required.
- Primitives will be widened if necessary.
- A `void` return type will be massaged to 0 or `null`, depending on whether the expected return was primitive or of reference type.
- `null` values are passed through, regardless of static type.

With these potential conversions in place, invocation looks like this:

```
Object rcvr = "a";
try {
  MethodType mt = MethodType.methodType(int.class);
  MethodHandles.Lookup l = MethodHandles.lookup();
  MethodHandle mh = l.findVirtual(rcvr.getClass(), "hashCode", mt);

  int ret;
  try {
    ret = (int)mh.invoke(rcvr);
    System.out.println(ret);
  } catch (Throwable t) {
    t.printStackTrace();
  }
} catch (IllegalArgumentException |
  NoSuchMethodException | SecurityException e) {
  e.printStackTrace();
} catch (IllegalAccessException x) {
  x.printStackTrace();
}
```

This code is somewhat similar to the reflective invocation code we met earlier in the chapter, but the presence of the lookup provides a clear sign that this is using method handles.

## Summary

In this chapter, we've met Java's key capabilities that support dynamism and flexible

programming. From class introspection and runtime classloading, we have met both Reflection and Method Handles.

The second of these is particularly important for modern Java as it provides a clearer and more coherent way to access the same dynamic invocation capabilities as Reflection. In addition, method handles are designed to work well with the low-level execution model of the JVM and can potentially improve performance over the older approach provided by Reflection.

Let's move on to meet the last major topic of the book—Java modules.

---

1 This spells "Cafe Babe," which is a rather sad reflection on the standards of the less-enlightened times when Java was created. It's basically impossible to change now, unfortunately.

2 Further changes are continuing in the "Integrity by default" project, which we will discuss in Appendix A.

3 There is also a method to look up the current class object from a static context—we will meet it presently.

4 In fact, from Java 21 onward, Java reflection has a new implementation that is based on method handles.

# Chapter 14. Java Platform Modules

In this chapter, we will provide a basic introduction to the *Java Platform Modules System* (JPMS). However, this is a large, complex subject—interested readers may well require a more in-depth reference, such as *Java 9 Modularity* by Sander Mak and Paul Bakker (O'Reilly).

Modules, a relatively advanced feature, are primarily about packaging and deploying entire applications and their dependencies. They were added to the platform roughly 20 years after the first version of Java and so can be seen as mostly orthogonal to the rest of the language syntax.

Java's strong promotion of backward compatibility also plays a role here, as non-modular applications must continue to run. This has led the architects and stewards of the Java platform to adopt a pragmatic view of the necessity for teams to adopt modules for existing code. This is perhaps best expressed by this quote from Oracle's Java Platform Architect:

> *There is no need to switch to modules.*
>
> *There has never been a need to switch to modules.*
>
> *Java 9 and later releases support traditional JAR files on the traditional classpath, via the concept of the unnamed module, and will likely do so until the heat death of the universe.*
>
> *Whether to start using modules is entirely up to you.*
>
> —Mark Reinhold

Due to the advanced nature of modules, this chapter assumes you are familiar with a modern Java build tool, such as Gradle or Maven.

If you are new to Java, you can safely ignore references to those tools and just read the chapter to get a first, high-level overview of JPMS. It is not necessary for a new Java programmer to fully understand this topic while still learning how to write Java programs.

> **NOTE**
>
> Because modules are more advanced than many of the topics we cover in the book, if you are new to Java, we recommend coming back to reread this chapter again after you have more hands-on Java experience.

## Why Modules?

There were several major motivating reasons for wanting to add modules to the Java platform. These included a desire for:

- Strong encapsulation
- Well-defined interfaces

- Explicit dependencies

These are all language (and application) design-level concerns, and they were combined with the promise of new platform-level capabilities as well:

- Scalable development
- Improved performance (especially startup time) and reduced footprint
- Reduced attack surface and better security
- Evolvable internals

The encapsulation point was driven by the fact that the original language specification supports only private, public, protected, and package-private visibility levels. There is no way to control access in a more fine-grained way to express concepts such as:

- Only specified packages are available as an API—others are internal and may not be accessed.
- Certain packages can be accessed by this list of packages but no others.
- Only specific types are exported from a package.

The lack of these and related capabilities has been a significant shortcoming when architecting larger Java systems. Not only that, but without a suitable protection mechanism, it would be very difficult to evolve the internals of the JDK—as nothing would prevent user applications from directly accessing implementation classes.

> *With a sufficient number of users of an API, it does not matter what you promise in the contract: all observable behaviors of your system will be depended on by somebody.*
>
> —Hyrum Wright

The modules system attempts to address all of these concerns at once and to provide a solution that works both for the JDK and for user applications.

## Modularizing the JDK

The monolithic JDK that shipped with Java 8 was the first target for the modules system, and the familiar `rt.jar` was broken up into modules.

> **NOTE**
>
> Java 8 had begun the work of modularization by shipping a feature called *Compact Profiles* that tidied up the code and made it possible to ship a reduced runtime footprint.

`java.base` is the module that represents the minimum that's actually needed for a Java application to start up. It contains core packages, such as:

```
java.io
java.lang
java.math
java.net
java.nio
java.security
java.text
java.time
java.util
javax.crypto
javax.net
javax.security
```

along with some subpackages (e.g., `java.lang.invoke` and `java.util.concurrent`) and nonexported implementation packages such as `sun.net`.

Let's look at an example—a simple program that extends a public but internal class, which is contained in `java.base` for modular JDKs:

```
import sun.net.URLCanonicalizer;

public class ExposedCanonicalizer extends URLCanonicalizer {

    public boolean isSimple(String url) {
        return isSimpleHostName(url);
    }
}
```

Attempting to compile the code on Java 11 produces this error message:

```
$ javac javanut/ch14/ExposedCanonicalizer.java
javanut/ch14/ExposedCanonicalizer.java:3:
        error: package sun.net is not visible
import sun.net.URLCanonicalizer;
          ^
  (package sun.net is declared in module java.base, which does not
   export it to the unnamed module)
javanut/ch14/ExposedCanonicalizer.java:8: error: cannot find symbol
        return isSimpleHostName(url);
               ^
  symbol:   method isSimpleHostName(String)
  location: class ExposedCanonicalizer
2 errors
```

As we can see, when using a modular Java, even classes that are public cannot be accessed unless they are explicitly exported by the module they are defined in. We can temporarily force the compiler to make the internal package visible to us (basically reasserting the old access rules) with the `--add-exports` switch, like this:

```
$ javac --add-exports java.base/sun.net=ALL-UNNAMED \
        javanut/ch14/ExposedCanonicalizer.java
javanut/ch14/ExposedCanonicalizer.java:3:
        warning: URLCanonicalizer is internal proprietary API and may
        be removed in a future release
```

```
import sun.net.URLCanonicalizer;
              ^
javanut/ch14/ExposedCanonicalizer.java:5:
        warning: URLCanonicalizer is internal proprietary API and may
        be removed in a future release
public class ExposedCanonicalizer extends URLCanonicalizer {
                                          ^
2 warnings
```

We have to specify that the export is being granted to the *unnamed module*, as we are compiling our class standalone and not as part of a module. The compiler warns us that we're using an internal API and that this might break with a future release of Java, but it does compile.

When running this code under Java 17 and later versions, we have to add the `--add-exports java.base/sun.net=ALL-UNNAMED` flag to the runtime invocation as well as compilation. This is because Java 17 now enforces additional runtime checks as part of the tightening up of encapsulation of the internals.

The requirement to include flags such as `--add-exports` when adopting modules is an unfortunate fact of life when modernizing applications. If you encounter it, then you should take it as an indication that some design decisions should be rethought—perhaps directly interacting with the JDK internals is not the best approach for code that needs to be maintained into the future.

## The Java SE Module Graph

Although `java.base` is the absolute runtime minimum that an application needs to start up, at compile time we want the visible platform to be as close to the old (Java 8) experience as possible.

This means that we use a much larger set of modules, contained under an *umbrella* module, `java.se`. This module has a dependency graph, shown in Figure 14-1.

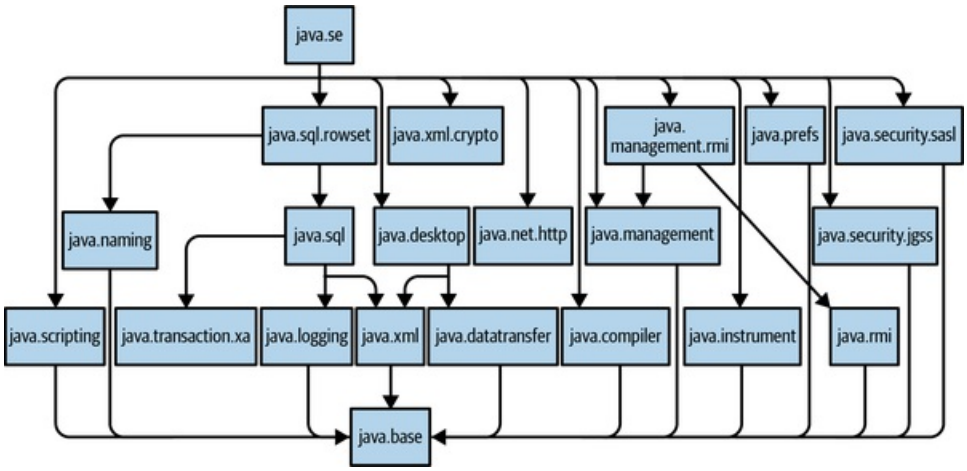



*Figure 14-1. Module dependency graph of java.se*

This brings in almost all the classes and packages that most Java developers expect to be

available.

However, one important exception is that the Java 8 packages defining the CORBA and Java EE APIs (now known as Jakarta EE) have been removed and are not part of `java.se`. This means that any project that depends on those APIs will not compile by default on Java 11 onward, and a new build config must be used that explicitly depends upon external libraries that provide these APIs.

Along with these changes to compilation visibility, due to the modularization of the JDK, the modules system is also intended to allow developers to modularize their own code, and that's the subject of our next section.

### Module Imports

Java 25 introduced a new refinement of the `import` statement to simplify working with modules. Writing `import module M;` imports all the public top-level classes and interfaces in the packages exported by `M` to the current module, along with any required transitive dependencies (which we'll meet properly in the next section).

This mechanism can greatly reduce the number of import statements required. For example, `import module java.base` has the same effect as writing 54 on-demand package imports (one for each package exported by `java.base`).

One problem that emerges from the broad scope that module imports can cause is ambiguity of class short names. For example, these imports have a collision:

```
import module java.base;      // exports java.util, which has a public
                              // Date class
import module java.sql;       // exports java.sql, which has a public
                              // Date class
```

To resolve this ambiguity, we use another import declaration: `import java.sql.Date;` which declares which of the two possible `Date` classes will be allowed to use the short name.

## Writing Your Own Modules

In this section, we will discuss the basic concepts needed to start writing modular Java applications.

### Basic Modules Syntax

The key to modularizing is the new file *module-info.java*, which contains a description of a module. This is referred to as a *module descriptor*.

A module is laid out for compilation correctly on the filesystem in the following way:

- Below the source root of the project (*src*), there needs to be a directory named the same

as the module (the *moduledir*).

- Inside the *moduledir* is the *module-info.java*, at the same level as where the packages start from.

The module info is compiled to a binary format, *module-info.class*, which contains the metadata that will be used when a modular runtime attempts to link and run our application. Let's look at a simple example of a *module-info.java*:

```
module httpchecker {
    requires java.net.http;

    exports httpchecker.main;
}
```

This introduces some new syntax: `module`, `exports`, and `requires`—but these are not really full keywords in the accepted sense. As stated in the Java Language Specification:

> *A further ten character sequences are restricted keywords:* `open`, `module`, `requires`, `transitive`, `exports`, `opens`, `to`, `uses`, `provides`, *and* `with`. *These character sequences are tokenized as keywords solely where they appear as terminals in the* `ModuleDeclaration` *and* `ModuleDirective` *productions.*
>
> —*Java Language Specification Java SE* 11th ed., Section 3.9

This means that these restricted keywords can appear only in the module metadata and are compiled into the binary format by `javac`.

> **NOTE**
>
> The concept of *restricted keyword* is considerably expanded in Java 17 and beyond. As a result, the description is much longer and less clear in later versions of Java. We use the older wording here because it refers specifically to the modules system and is more suited to our purposes.

The meanings of the major restricted keywords are:

`module`

Starts the module's metadata declaration

`requires`

Lists a module on which this module depends

`exports`

Declares which packages are exported as an API

The remaining (module-related) restricted keywords will be introduced throughout the rest of the

chapter.

In our example, this means that we're declaring a module `httpchecker` that depends upon the module `java.net.http` that was standardized in Java 11 (as well as an implicit dependency on `java.base`).

Requiring a module `M` means that `M` must be available to the source code compiler, or a compile-time error will occur. Assuming that it is, then the compiler will allow code in the module being compiled to read `M` and access the public members in the exported packages of `M`.

It is possible that `M` might export a package containing a type containing a method whose signature refers to a package in a second module `D`. In this case, then the declaration of `M` should include a `requires transitive D`. This will ensure that other modules that depend upon `M` will automatically be able to read `D` and that the method signatures provided by `M` will make sense.

The example module exports a single package, `httpchecker.main`, which is the only package in this module that will be accessible from other modules at compile time.

## Building a Simple Modular Application

As an example, let's build a simple tool that checks whether websites are using HTTP/2 yet, using the API that we met in Chapter 11:

```
package httpchecker.main;

import static java.net.http.HttpResponse.BodyHandlers.ofString;

public final class HTTP2Checker {
    public static void main(String[] args) throws Exception {
        if (args.length == 0) {
            System.err.println("Provide URLS to check");
        }
        for (final var location : args) {
            var client = HttpClient.newBuilder().build();
            var uri = new URI(location);
            var req = HttpRequest.newBuilder(uri).build();

            var response = client.send(req,
                    ofString(Charset.defaultCharset()));
            System.out.println(location + ": " + response.version());
        }
    }
}
```

This relies on two modules—`java.net.http` and the ubiquitous `java.base`. The module file for the app is very simple:

```
module httpchecker {
    requires java.net.http;
    exports httpchecker.main;
}
```

Assuming a simple, standard module layout, this can be compiled like this:

```
$ javac -d out/httpchecker \
        httpchecker/httpchecker/main/HTTP2Checker.java \
        httpchecker/module-info.java
```

This creates a compiled module in the *out/* directory. For use, it needs to be packaged as a JAR file:

```
$ jar --create --file httpchecker.jar \
        --main-class httpchecker.main.HTTP2Checker \
        -C out/httpchecker .
```

The `--create` switch tells `jar` to create a new jar, which will include the classes contained in the directory. The final `.` at the end of the command is mandatory and signifies that all of the class files (relative to the path specified with `-C`) should be packaged into the JAR.

We used the `--main-class` switch to set an *entry point* for the module—that is, a class to be executed when we use the module as an application. Let's see it in action:

```
$ java -jar httpchecker.jar http://www.google.com
http://www.google.com: HTTP_1_1
$ java -jar httpchecker.jar https://www.google.com
https://www.google.com: HTTP_2
```

This shows that, at the time of writing, Google's website was using HTTP/2 to serve its main page over HTTPS but still using HTTP/1.1 for the legacy unencrypted HTTP service.[1]

Now that we have seen how to compile and run a simple modular application, let's meet some more of the core features of modularity that are needed to build and run full-size applications.

## The Module Path

Many Java developers are familiar with the concept of the classpath. When working with modular Java applications, we instead need to work with the *module path*. This is a new concept for modules that replaces the classpath wherever possible.

Modules carry metadata about their exports and dependencies—they are not just a long list of types. This means a graph of module dependencies can be built easily and module resolution can proceed efficiently.

Code that is not yet modularized continues to be placed on the classpath. This code is loaded into the *unnamed module*, which is special and can read all other modules that can be reached from `java.se`. Using the unnamed module happens automatically when classes are placed on the classpath.

This provides a migration path to adopting a modular Java runtime without having to migrate to a fully modular application path. However, it does have two major drawbacks: the benefits of modules will not be fully available until the end of the app migration process, and the self-

consistency of the classpath must be maintained by hand until modularization is complete.

## Automatic Modules

One of the constraints of the modules system is that we can't reference JARs on the classpath from named modules. This is a safety feature—the designers of the module system wanted the module dependency graph to utilize full metadata and be able to rely on the completeness of that metadata.

However, there may be times when modular code needs to reference packages that have not yet been modularized. The solution for this is to place the unmodified JAR onto the module path directly (and remove it from the classpath). A JAR placed on the module path like this becomes an *automatic module*.

This has the following features:

- Module name derived from JAR name (or read from *MANIFEST.MF*)
- Exports every package
- Requires all other modules (including the unnamed module)

This is another feature designed to mitigate and help with migration, but some safety is still being given up by using automatic modules.

## Open Modules

In earlier versions of Java, whether a public method on a public class was accessible to code from other packages depends on the way the package is deployed on the host system. For example, when the class files that constitute a package are stored in a directory, then a user must have read access to the directory and the files within it to have access to the package.

However, as noted in this chapter, simply marking a class and method as `public` no longer guarantees that the element will be accessible everywhere. Instead, accessibility also now depends upon whether the package containing that element is exported by its defining module. Another issue (somewhat secondary but still important for modern Java applications) in the design of modules is the use of reflection to access classes.

Reflection is such a wide-ranging, general-purpose mechanism that it is difficult to see, at first glance, how it can be reconciled with the strong encapsulation goals of JPMS. Worse yet, so many of the Java ecosystem's most important libraries and frameworks rely on reflection (e.g., unit testing, dependency injection, and many more) that not having a solution for reflection would make modules impossible to adopt for any real application.

The solution provided is twofold. First, a module can declare itself to be an `open` module, like this:

```
open module jin {
    exports jin.api;
```

```
}
```

This declaration has the effect that:

- All packages in the module can be accessed via reflection.
- Compile-time access is *not* provided for nonexported packages.

This means that the configuration behaves like a standard module at compile time. The overall intent is to provide simple compatibility with existing code and frameworks and ease migration pain. With an open module, the previous expectation of being able to reflectively access code is restored. In addition, the `setAccessible()` hack that allows access to `private` and other methods that would not normally permit access is preserved for open modules.

Finer-grained control over reflective access is also provided via the `opens` restricted keyword. This does not create an open module but instead selectively opens specific packages for reflective access by explicit declaration:

```
module jin {
    exports jin.api;
    opens jin.domain;
}
```

This type of usage is likely to be useful when, for example, you are providing a domain model to be used by a module-aware *object-relational mapping* (ORM) system that needs full reflective access to the core domain types of a module.

It is possible to go further and restrict reflective access to specific client packages, using the `to` restricted keyword. Where possible, this can be a good design principle—but such a technique will not work well with a general-purpose framework such as an ORM.

> **NOTE**
>
> In a similar way, it is possible to restrict the export of a package to only specific external packages. However, this feature was added largely to help with the modularization of the JDK itself, and it has limited applicability to user modules.

It is possible to both export and open an internal package, but this is not recommended. Instead, during migration to modules, access to a package should be analyzed and determined to be either compile-time or reflective but not both.

In the case where reflective access to a package now contained in a module is required, the platform provides some switches to act as a stopgap for the transitional period.

In particular, the `java` option `--add-opens module/package=ALL-UNNAMED` can be used to open a specific package of a module for reflective access to all code from the classpath, overriding the behavior of the modules system. For code that is already modular, this option can

also be used to allow reflective access to a specific module.

When you are migrating to modular Java, any code that reflectively accesses internal code of another module should be run with that switch first, until the situation can be remediated.

Related to this issue of reflective access (and a special case of it) is the issue of widespread use of internal platform APIs by frameworks. This is usually characterized as the “Unsafe problem,” and we will look at it more closely toward the end of the chapter.

### Providing Services

The modules system includes the *services* mechanism to mitigate another problem with an advanced form of encapsulation. This problem is simply explained by considering a familiar piece of code:

```
import com.example.Service;

Service s = new ServiceImpl();
```

Even if `Service` lives in an exported API package, this line of code will not compile unless the package containing `ServiceImpl` is also exported. What we need is a mechanism to allow fine-grained access to classes implementing service classes without needing the entire package to be imported. For example, we could write something like:

```
module jin {
    exports jin.api;
    requires othermodule.services;

    provides services.Service with jin.services.ServiceImpl;
}
```

Now the `ServiceImpl` class is accessible at compile time as an implementation of the `Service` interface. Note that the `services` package must be contained in another module, which is required by the current module for this provision to work.

A module can also declare that it `uses` a specific service, by specifying a type name (which is either the interface or an abstract class of the service, not the implementation class). This provides a more fine-grained way to specify the dependency relationship for a service than a `requires` clause does.

In the next section, we’ll address the question of whether (and when) Java applications should be migrated to use modules.

## Migrating to Modules

Let’s start by reiterating that many applications will not need to be migrated at all.

However, modularizing existing codebases can be worthwhile—because the better encapsulation

and overall architectural benefits do pay off over the longer term. This allows new developers to be brought onto the team faster and provides a clear structure that is easier to understand and maintain.

> **TIP**
>
> Modules should be the default for all new, greenfield apps—especially those that are architected in a microservices style. Modules have significant benefits and are normally quite easy to adopt for new projects.

When considering migration of an existing app (especially a monolithic design), you can use the following roadmap:

1. First upgrade the application runtime to the in-use Java version (running from the classpath initially).
2. Identify any application dependencies that have been modularized and migrate those dependencies to modules.
3. Retain any nonmodularized dependencies as automatic modules.
4. Introduce a single *monolithic module* of all application code.

At this point, a minimally modularized application should be ready for production deployment. This module will usually be an `open` module at this stage of the process. The next step is architectural refactoring; at this point, applications can be broken out into individual modules as needed.

Once the application code runs in modules, it can make sense to limit reflective access to your code via `opens`. This access can be restricted to specific modules (such as ORM or dependency injection modules) as a first step toward removing any unnecessary access.

> **NOTE**
>
> The `jdeps` tool (see Appendix B) can help with determining which packages and modules your code depends upon. This is very helpful for migrations from Java 8 and the use of `jdeps` when rearchitecting from Java 8 is recommended.

For Maven users, it's worth remembering that Maven is not a modules system, but it does have dependencies—and (unlike JPMS dependencies) they are versioned. The Maven ecosystem is still evolving to fully integrate with JPMS—some plug-ins have not caught up at the time of this writing. However, some general guidelines for modular Maven projects do exist:

- Aim to produce one module per Maven POM.
- Don't modularize a Maven project until you are ready (or have an immediate need to).

- Remember that running on a modular Java runtime is still possible even if you are still building on a Java 8 toolchain.

The last point indicates that one path for migration of Java 8 Maven projects is to start by building as a Java 8 project and ensuring that those Maven artifacts can deploy cleanly (as automatic modules) on a modern Java runtime. Once that first step is working properly, then a full modularization can be undertaken.

> **TIP**
>
> Java modules are intended to be delivered in a number of different ways—but currently they are most commonly encountered as JAR files. Recall from Chapter 3 that a traditional JAR file is just a ZIP file with possibly some additional metadata (in a directory called `META-INF`).

JAR files containing modules will include the `module-info.class` file, which is the compiled form of the `module-info.java` file for the module. You should note that this file is different from other class files because the module formats did not exist prior to Java 9.

Let's move on and look at a technology that helps library projects migrate from Java 8 to a modular Java.

## Multi-Release JARs

To explain the problem that is solved by multi-release JARs, let's consider a simple example: finding the process ID (PID) of the currently executing process (i.e., the JVM that's executing our code).

> **WARNING**
>
> We don't reuse the HTTP/2 example from earlier on in this chapter, as Java 8 doesn't have an HTTP/2 API—so we would have had to do a huge amount of work (essentially a full backport!) to provide the equivalent functionality for 8.

This may seem like a simple task, but on Java 8 this requires a surprising amount of boilerplate code:

```
public class GetPID {
    public static long getPid() {
        // This rather clunky call uses JMX to return the name that
        // represents the currently running JVM. This name is in the
        // format <pid>@<hostname>—on OpenJDK and Oracle VMs only—there
        // is no guaranteed portable solution for this on Java 8
        final String jvmName =
            ManagementFactory.getRuntimeMXBean().getName();
        final int index = jvmName.indexOf('@');
        if (index < 1)
            return -1;
```

```
            try {
                return Long.parseLong(jvmName.substring(0, index));
            } catch (NumberFormatException nfe) {
                return -1;
            }
        }
    }
```

As we can see, this is nowhere near as straightforward as we might like. Worse still, it is not supported in a standard way across all Java 8 implementations. Fortunately, modern Java provides the `ProcessHandle` API, like this:

```
public class GetPID {
    public static long getPid() {
        // Use the modern Process API introduced in Java 9 (2017)
        ProcessHandle processHandle = ProcessHandle.current();
        return processHandle.getPid();
    }
}
```

This now utilizes a standard API, but it leads to an essential problem: how can the developer write code that is guaranteed to run on all current Java versions?

What we want is to build and run a project correctly in multiple Java versions. We want to depend on library classes that are only available in later versions but still run on an earlier version by using some code "shims." The end result must be a single JAR, and we do not want to require the project to switch to a multimodule format—in fact, the JAR must work as an automatic module.

Let's look at an example project that has to run correctly in both Java 8 and Java 11 or higher. The main codebase is built with Java 8, and the Java 11 portion must be built with Java 11. This part of the build must be isolated from the main codebase to prevent compilation failures, although it can depend on the build artifacts of the Java 8 build.

To keep the build configuration simple, this feature is controlled using an entry in *MANIFEST.MF* within the JAR file:

```
Multi-Release: True
```

The variant code (i.e., that for a later version) is then stored in a special directory in *META-INF*. In our case, this is *META-INF/versions/11*.

For a Java runtime that implements this feature, any classes in the version-specific directory override the versions in the content root. On the other hand, for Java 8 and earlier versions, both the manifest entry and the *versions/* directory are ignored and only the classes in the content root are found. This arrangement using version-specific classes is the exception rather than the rule—and is really used only when needing to support older Java versions (or when backporting to them).

## Converting to a Multi-Release JAR

To start deploying your software as a multi-release JAR, follow this outline:

1. Isolate code that is JDK-version-specific.
2. If possible, place that code into a package or group of packages.
3. Get the version 8 project building cleanly.
4. Create a new, separate project for the supplementary classes.
5. Set up a single dependency for the new project (the version 8 artifact).

For Gradle, you can also use the concept of a *source set* and compile the Java 11 code using a different (later) compiler. This can then be built into a JAR using a stanza like this:

```
jar {
  into('META-INF/versions/11') {
     from sourceSets.java11.output
  }

  manifest.attributes(
     'Multi-Release': 'true'
  )
}
```

For Maven, the current easiest route is to use the Maven Dependency Plug-in and add the modular classes to the overall JAR as part of the separate `generate-resources` phase.

## Custom Runtime Images

One of the key goals of JPMS is to provide for the possibility that applications may not need every class present in the traditional monolithic runtime of Java 8 and instead can manage with a smaller subset of modules. Such applications can have a much smaller footprint in terms of startup time and memory overhead. This can be taken further: if not all classes are needed, then why not ship an application with a reduced, custom runtime image that includes only what's necessary?

To demonstrate the idea, let's package the HTTP/2 checker into a standalone tool with a custom runtime. We can use the `jlink` tool (which has been part of the platform since Java 9) to achieve this:

```
$ jlink --module-path httpchecker.jar:$JAVA_HOME/jmods \
      --add-modules httpchecker,jdk.crypto.ec \
      --launcher http2chk=httpchecker \
      --output http2chk-image
```

Note that this assumes that the JAR file *httpchecker.jar* was created with a main class (aka entry point). The result is an output directory, *http2chk-image*, which is about 39 MB in size, much

less than the full image. This invocation also indicates that because the tool uses the new HTTP module, it requires the libraries for security, crypto, and so on when connecting using HTTPS.

From within the custom image directory, we can run the `http2chk` tool directly and see that it works even when the machine does not have the required version of `java`:

```
$ java -version
openjdk version "1.8.0_422"
OpenJDK Runtime Environment (Zulu 8.80.0.17-CA-macos-aarch64)
    (build 1.8.0_422-b05)
OpenJDK 64-Bit Server VM (Zulu 8.80.0.17-CA-macos-aarch64)
    (build 25.422-b05, mixed mode)
$ ./bin/http2chk https://www.google.com
https://www.google.com: HTTP_2
```

The deployment of custom runtime images has great potential to reduce your code footprint and help Java remain competitive in the age of microservices, although it is not as widely known and deployed as it might be. In the future, `jlink` could even be combined with new approaches to compilation, including an ahead-of-time (AOT) compiler—see Appendix B for more details.

## Issues with Modules

The modules system, despite being the result of a large amount of engineering effort—both as the subject of a major release and plenty of follow-up work since—is not without its problems. This was, perhaps, inevitable—the feature fundamentally changes how Java applications are architected and delivered. It would have been almost impossible for modules to avoid running up against some problems when trying to retrofit the large, mature ecosystem that is Java. Let's take a quick look at some of the issues, starting with the versioning problem.

### Lack of Versioning

The JPMS standard as of Java 25 does not include the versioning of dependencies.

> **NOTE**
>
> Not supporting versioned modules was a deliberate design decision to reduce the complexity of the delivered system. This does not preclude the possibility that modules could be versioned in the future, although this is not currently under discussion.

The current situation therefore requires external tools to handle the versioning of module dependencies. In the case of Maven, this will be within the Project Object Model (POM)—and similarly for Gradle. An advantage to this approach is that the download and management of versions are also handled within the local repository of the build tool.

However it is done, though, the simple fact is that the dependency version information must be stored out of the module and does not form part of the JAR artifact or the module resolution

process.

There's no getting away from it—this is pretty ugly, but the counterpoint is that the situation is no worse than it was with dependencies being deduced from the classpath.

## The Unsafe Class

One internal API deserves a particular mention—`sun.misc.Unsafe`.

It is a class that has historically been used widely by framework writers and other implementers within the Java world. However, it is an internal implementation class and is not part of the standard API of the Java platform (as the package name clearly indicates). The class name also provides a fairly strong clue that this is not intended for use by Java applications.

Officially, `Unsafe` is an unsupported, internal API and so could be withdrawn or modified by any new Java version, without regard to the effect on user applications. Any code that does use it is technically directly coupled to the HotSpot JVM internals and so is also potentially nonstandard and may not run on other implementations.

However, although not an official part of Java Standard Edition (SE) in any way, `Unsafe` became a de facto standard and key part of the implementation of basically every major framework in one way or another. Over subsequent versions it evolved into a kind of dumping ground for nonstandard but necessary features.

> **TIP**
>
> More details about `Unsafe` and related low-level platform techniques can be found in *Optimizing Cloud Native Java* (O'Reilly).

As a result, many frameworks and libraries were unable to move to a modular JDK without replacing some `Unsafe` features. In turn, this impacted everyone using any libraries from a wide range of frameworks—basically every application in the Java ecosystem.

To fix this problem, new, supported APIs were created for some of the necessary functionality, with segregated APIs that could not be encapsulated in time being moved into a module, `jdk.unsupported`.[2] This name also makes it clear this is not a supported API and that developers use it at their own risk. This gave `Unsafe` a (strictly limited-time) temporary pass while encouraging library and framework developers to move to the new APIs.

As of Java 25, `Unsafe` is finally terminally deprecated for external use—there are supported APIs to replace all of the functionality, and access will be withdrawn over the next few Java versions. However, versions prior to Java 25 may still have important usages of `Unsafe`—and not all libraries support the new APIs yet.

## Adoption Rates

As we discussed way back in Chapter 1, with the release of Java 9, the Java release model fundamentally changed. Java 8 and 9 used the "keystone" release model—where one keystone (or landmark) feature, such as lambdas for Java 8 or modules for Java 9, essentially defined the release and so the ship date was determined by when the feature was complete.

The problem with this model is that it can cause inefficiencies due to uncertainty about when versions will ship. In particular, a small feature that just misses a release will have to wait a long time for the next major release. As a result, a new release model was adopted, which introduces *strict time-based versioning*.

This means:

- Java releases are now classified as *feature* releases, which occur at a regular cadence of once every six months.
- Features are not merged into the platform until they are essentially complete.
- The mainline repo is in a releasable state at all times.

These releases are good for only six months, after which time they are no longer supported. Certain releases are designated by Oracle as *long-term support* (LTS) releases. These have extended, paid-for support available from Oracle.

The release cadence of these LTS releases at time of writing is every two years. This means that Oracle LTS releases are currently 8 (retrospectively added), 11, 17, 21, and 25—which was released in September 2025.

However, as well as Oracle, builds of OpenJDK are available from other providers including Amazon, Azul Systems, Eclipse Adoptium, IBM, Microsoft, Red Hat, and SAP. These vendors offer various ways to get JDK updates (including security)—at zero cost.

There are also new and existing paid support models available from several of these vendors.

> **NOTE**
>
> For an in-depth write-up of this topic, please see the guide "Java Is Still Free" by the Java Champions community, an independent body of Java leaders in the software industry.

In practice, only the LTS releases are seeing widespread adoption. This may be due to the desire of teams to have longer support cycles, rather than upgrading to each feature release after only six months.

It is also the case that the upgrade from Java 8 to a modular JDK is not a perfect drop-in replacement. The modules subsystem fundamentally changes many aspects of the Java platform, and this can result in small issues that need to be fixed—even if the end-user applications do not take advantage of modules.

## Summary

The modules feature, first introduced in Java 9, aims to solve several problems at once. The feature aims to deliver shorter startup times, lower footprints, and reduced complexity by tightening access to platform internals. All of these objectives have been met as of Java 25. The longer-term goals of enabling better architecture of applications and starting to think about new approaches for compilation and deployment are still in progress, however.

The reality is that, as of the release of Java 25, not all teams and projects have migrated completely to the modular world. This is to be expected, as modularity is a long-term project that has a slow payoff and relies on network effects within the ecosystem to achieve the full benefit.

New applications should definitely default to building in a modular way from the get-go, but the overall story of platform modularity within the Java ecosystem is still developing, and the overall trend is positive.

In these 14 chapters, we have covered a wide range of subjects. This is a testament to the continued vitality of Java—the language and ecosystem have changed enormously over the last 30+ years, and yet the platform and community remain vibrant. To have achieved this, while remaining recognizable as a language and platform, is no small accomplishment.

Ultimately, Java's continued existence and viability depend upon the individual developer. On that basis, the future looks bright, and we look forward to the next wave and beyond.

---

1 The HTTP standard has a version 3 as well, but support for it is still being deployed at time of writing.

2 An example of the new, supported APIs is *VarHandles*—an extension to the Method Handles concept from Chapter 13 with additional new functionality, such as concurrency barrier modes.

# Appendix A. Future Java

This appendix discusses the future of Java, i.e., versions of Java beyond Java 25. Historically, we resisted adding forward-looking material to *Java in a Nutshell*, but the model of Feature and LTS Java releases (which we discussed in Chapter 1) has prompted a change of tack. As we mentioned there, our approach is to restrict the main text to fully delivered features and refer forward to this appendix for important features that are still in Incubating or Preview state.

Let's start by covering how major development efforts are arranged within OpenJDK and then discuss what is known of forthcoming releases.

## Long-Term JDK Projects

OpenJDK is organized into projects that cover specific major areas of ongoing work. This includes projects that are centered on the development of major future language or JVM features that can take multiyear efforts to deliver. They are usually known by their project codenames, and the projects that are currently most active are:

- Loom
- Panama
- Valhalla
- Leyden

We'll discuss the projects in that order, especially because the second half of Project Panama depends upon Project Valhalla.

The first, Project Loom, is a new concurrency model for Java, and much of the expected functionality (including Virtual Threads and Scoped Values) has already been delivered, but a number of important aspects are still ongoing—including *Structured Concurrency*. As all the remaining parts of Loom are basically advanced enhancements to the vthreads capability, we won't discuss it any further here.

Project Panama's mission is to provide two major improvements: a modern foreign-function interface for Java—which was delivered and is available as of Java 25—and support for vector CPU instructions in a platform-independent way.

Project Valhalla is related to next-generation memory utilization, but that description doesn't do full justice to what is actually the most ambitious, wide-ranging, and highest-impact of all the projects we are considering. It is also the most complex and the furthest from delivery as a shipping product.

The last of the four projects we'll discuss is Project Leyden. The motivation for this project is

that in some modern environments, especially containers, the powerful flexibility and dynamism that Java provides by default is not fully needed. Leyden seeks to answer the question “Can we constrain the dynamism somewhat and avoid paying a runtime cost for it?”

Let’s meet these three projects properly, starting with Panama.

## Panama

Project Panama aims to provide a modern *foreign* (i.e., non-Java) interface for connecting to native code. The codename comes from the isthmus of Panama, a narrow strip connecting two larger “landmasses,” understood to be the JVM and native memory (aka “off-heap”).

Panama provides two main components to assist in the interoperation of Java and native code:

- Foreign Function and Memory API (FFM)
- Vector API

FFM is concerned with the allocation, manipulation, and freeing of structured foreign memory and the lifecycle management of foreign resources. This goes beyond the existing capabilities of the `ByteBuffer` class and, for example, can address more than 2 GB of memory as a single `MemorySegment`. It is also intended to provide a full and complete modern replacement for the Java Native Interface (JNI), which is Java’s original (and now ancient) foreign call interface.

As of Java 25, FFM has been delivered and is ready for use in production applications. A full discussion is outside the scope of this book, but IBM has published an excellent introduction.

Panama also provides support for vector computations by shipping an API that has these main goals:

- Clear and concise API
- Platform agnostic
- Reliable JIT compilation and performance
- Graceful degradation of vector mode back to linear instructions

Panama is initially shipping an implementation for the x64 and AArch64 CPU architectures. However, as expressed in the goals, the API does not—and must not—rule out possible implementations for other CPUs.

The Vector API (not to be confused with the ancient and heavily deprecated `java.util.Vector` class) has been in Incubator status for many releases now, but this should not be taken to imply any immaturity. Instead, the Vector API is waiting for Valhalla, as much of the potential that the API offers will be fully realized only with the new capabilities of Valhalla.

*The Vector API will incubate until necessary features of Project Valhalla become available as*

*preview features. At that time, we will adapt the Vector API and its implementation to use them and then promote the Vector API from incubation to preview.*

—JEP 529

These projects are especially useful for enabling Java for AI workloads. Python attracts the most attention and has the single biggest market share for AI projects, but an increasing number of enterprises are finding that Java has a very strong role to play in their AI adoption. Panama and Valhalla are both enabling technologies to help Java compete effectively with Python while not sacrificing Java's advantages in safety and robustness.

With this in mind, let's take a look at Project Valhalla as our next topic.

## Valhalla

Project Valhalla is a very ambitious OpenJDK project that has been running since 2014. The goal, "to align JVM memory layout behavior with the cost model of modern hardware," seems simple and innocuous enough.

However, this is deeply deceptive.

The main technical mechanism to achieve the goal is to divide Java objects into two subcases: the identity objects we're used to using and a new kind of *value object* that has only final fields and does not have a unique identity.

The intended use case for this new data value is small, immutable, identity-free types. This allows these new identity-free values to "bridge" the use cases of both of the existing kinds of values—object reference and primitives. Ideally, value objects should be able to get the best of both worlds and should enable several possible use cases—including "user-defined primitives," which is one use case that the Vector API in particular will benefit from.[1]

> **TIP**
>
> Users should think of values as objects without identity and that they will get performance benefits from the JIT (such as enhanced *escape analysis*), which will be automatically provided by the runtime.

The fact that these new data values lack object identity implies that there is no object monitor, so `wait()`, `notify()`, and `synchronized` are not possible for these types. A related question is whether these value classes will permit `null` as a valid value or not—which leads to the idea of *null-restricted types*, which is a related but separate project.

There is also a potentially surprising connection between Valhalla and Java generics because only reference types are permissible as the value for a type parameter. Valhalla therefore proposes to extend generics to allow abstraction over all types, including these new data values and even the existing primitives.

In addition to the extensive work to plumb these new forms of data through the JVM, it is also

necessary to create a usage model in the Java language that seems natural to Java programmers. Valhalla must also enable existing libraries (including, but not limited to, the JDK) to compatibly evolve as these changes are delivered. This includes changes in which the current implementation of some capabilities (e.g., `Optional` or the date and time classes) will become value classes.

Valhalla's new types have been known by several names during the project's history, including *value types*, *inline types*, and *primitive classes*. The JDK Enhancement Proposals (JEPs) that cover the implementation of Valhalla are not, at the time of writing, targeted at any specific Java version, and it may be some time before most Java programmers encounter them in day-to-day work.

## Leyden and Cloud Native Java

One of the ongoing mega trends in the software industry is the transition to workloads that run "in the cloud," which means on time-leased servers owned by infrastructure providers such as Amazon, Microsoft, and Google.

Modern programming environments increasingly need to ensure that they are economic and easy to use in cloud deployments, and Java is no exception. However, Java's design does have certain aspects that are potentially less friendly to cloud applications than we would like.

These largely stem from the dynamic aspects of the runtime, such as reflection, classloading, and JIT compilation. The advantages of dynamism, principally flexibility and high performance over the lifetime of a single JVM process, have been critical to the success of Java over its 30-year history.

However, in the cloud, this dynamic nature can have side effects such as:

- Slow application startup time
- Long time to peak performance
- Potentially high memory overhead

In particular, the lifetime of cloud processes (especially for "Serverless" and *Function-as-a-Service* deployments) may be too short for the performance benefits of Java to pay off. This can be seen as the costs required to get the gains not being fully amortized by the time the process exits.

There are ongoing attempts to solve these long-term pain points and ensure that Java remains a competitive and attractive programming environment as Cloud-First becomes the dominant mode of delivery for server-side applications.

One of the major approaches is *native compilation*: the conversion of a Java program from bytecode into compiled machine code. Since this compilation occurs before program execution starts (as it does for languages like Rust and C++), it is also known as *ahead-of-time compilation* (AOT). un This technique is unusual in the Java space, but it aims to provide a faster startup time

as programs do not need to be classloaded or JIT compiled. However, it does not generally give better peak performance than the same application would have when running in dynamic VM mode. This is because peak performance isn't the point here—AOT and JIT represent different strategies and different trade-offs.

There are two main efforts that take somewhat different approaches to reducing startup time: first, native compiled Java from Oracle's GraalVM project. This was developed as a separate research project in Oracle Labs and is available in two editions, an open source edition and a proprietary Enterprise Edition, which has a licensing and support cost.

> **NOTE**
>
> GraalVM includes Truffle, a framework for generating interpreters for languages to run on the JVM. Supported languages written on top of Truffle include JavaScript, Python, Ruby, and R. This support is the basis of the polyglot capabilities that GraalVM provides.

The core of the GraalVM approach is a compiler called Graal that can operate in either JIT or AOT mode.[2] The AOT mode of Graal is the basis of GraalVM's Native Image technology that can produce a standalone, compiled machine code binary from a Java application.

Another project working on improving cloud native support is Quarkus, a Java microservices framework designed specifically for the Kubernetes cloud orchestration and deployment stack. Quarkus attempts to reduce the impact of the cloud native pain points by extensive use of build-time processing. Expensive computations and startup that would normally be handled reflectively during startup are instead performed ahead of time wherever possible.

Quarkus also emphasizes developer experience and provides both reactive and imperative styles of programming microservices.

The framework is open source and production ready, and it has support available from Red Hat, which is the primary maintainer of the project. It includes support for native compilation (based on the open source edition of GraalVM), but Quarkus also runs in dynamic VM mode on top of an OpenJDK runtime.[3]

The other major approach to improving cloud native behavior is Project Leyden.[4] This is an OpenJDK project that seeks to improve the startup time, time to peak performance, and footprint of Java programs.

The project is adopting a different approach from GraalVM—using *static runtime images* to enhance the Java platform. The starting point for this path is a technology called *class-data sharing* (CDS) archives.

Originally designed for sharing class metadata between processes on the same machine, it turns out that CDS also offers a significant startup-time improvement. In fact, Java 17 and later include a built-in CDS archive containing the metadata of over a thousand commonly used JDK classes. CDS is therefore in extremely wide use even though many Java developers have never heard of it

(and very few have used it directly).

As of Java 25, it is possible to create and reuse an *ahead-of-time cache*. This is similar to a CDS archive and can store classes after reading, parsing, loading, and linking them. Once a cache is created for a specific application, it can be reused in subsequent runs of that application to improve startup time.

Creating the cache has two steps—first a *training run* to collect the data and record the configuration required to create the cache, and second, a creation step that doesn't run the app but just builds the cache.

This process is currently quite manual, but the overall intent is to evolve toward creating useful artifacts that function similarly to the AOT-compiled native image binaries created by GraalVM. One of the next steps is to include AOT-compiled code (not just metadata and loaded and linked classes) corresponding to methods in the AOT cache. The techniques from Project Leyden are incrementally appearing in Java releases, with more steps to come along the way.

## Integrity By Default

Another major effort within OpenJDK is known as *integrity by default*, although it does not have a formal project codename like the others we have met. The project is related to encapsulation and seeks to address any systemic defects that relate to aspects such as:

- Correctness
- Codebase scalability (and program composability)
- Security

The project also has some secondary goals, such as unlocking new performance optimizations and improving the long-term maintainability of the OpenJDK codebase.

At a high level, integrity in Java applications is undermined by four main factors:

- Deep Reflection / `setAccessible()`
- `Unsafe`
- Instrumentation
- JNI

Each of these areas can cause arbitrary or *undefined behavior* (UB) in the sense used by the C programming language.

The C and C++ programming languages are famous for popularizing the concept of UB and Java's design explicitly tried to reduce sources of UB—but this was always an imperfectly achieved goal. Integrity by default aims to improve significantly on the current state and reduce (hopefully substantially) sources of unsoundness and UB.

A number of techniques are in scope to help strengthen integrity in Java:

- Strong encapsulation via modules
- Replacement and restriction of `Unsafe` and other critical internal APIs
- Opt-in for dynamic agent attach
- Opt-in for native code
- Removal of `Thread.stop()`

Of these, we discussed the changes to Reflection that modules bring in Chapter 14. We also mentioned the replacement of internal, but commonly used, APIs such as `Unsafe`.

Requiring opt-in for dynamic attach achieves the goal of making agent usage explicit. Application owners need to either specify their choice of agent at startup or explicitly opt-in to dynamic attach. This closes the instrumentation gap somewhat—by restricting it to only those applications that consciously choose it.

For native code, the same "warn" and then "restrict with opt-in" progression is being applied—just as it was with Deep Reflection. In Java 25, Java Native Interface (JNI) code now warns unless a runtime switch is passed to suppress the warning, and use of the FFM API was built so that it warns right from the start.

We discussed the removal of `Thread.stop()` briefly in Chapter 12, but due to how rarely this functionality was ever used, it represents only a very small reduction in UB.

To conclude, let's discuss one way in which we can observe the effect of the improvements in integrity: once teams have upgraded to a modular JDK (11+), then upgrading from one Java release to another is generally significantly less painful than for earlier, pre-modular releases.

This process is becoming easier and easier with each LTS release, especially as more and more libraries adopt replacements to Deep Reflection and the use of unsupported APIs. Integrity is, ultimately, a long-term project that the ecosystem will adopt gradually—but the current indications are positive, and the benefits are beginning to be realized.

---

1 This will, in turn, help greatly with AI applications.

2 One interesting aspect of the Graal compiler is that it is written in Java, unlike the JIT compilers in OpenJDK, which are implemented in native code.

3 In fact, this is the default deployment configuration for Quarkus—native compiled mode is actually used by only a minority of users.

4 The project name comes from a "Leyden jar," a device from the 1700s used for storing static electrical charge.

# Appendix B. Platform Tools

This chapter discusses the tools that ship with the OpenJDK version of the Java platform. The tools covered are all command-line tools. If you are using a different version of Java, you may find different tools (with similar function) as part of your distribution. Many third parties build on the foundations provided by these tools in the JDK.

Later in the chapter, we devote a section to Java Flight Recorder (JFR) tooling for deep profiling of Java applications.

## Command-Line Tools

The command-line tools we cover are the most commonly used tools and those of greatest utility —they are not a complete description of every available tool.

> **NOTE**
>
> In some cases, we need to discuss switches that take filesystem paths. As elsewhere in the book, we use Unix conventions for such cases. Other aspects of these tools, such as the command-line switches available, should be the same across operating systems.

Here we'll discuss the following tools, including their basic usage, description, and common switches:

- `javac`
- `java`
- `jar`
- `javadoc`
- `jdeps`
- `jps`
- `jstat`
- `jstatd`
- `jinfo`
- `jstack`
- `jmap`

- `javap`
- `jlink`
- `jmod`
- `jcmd`
- `jwebserver`

> **NOTE**
>
> Options described throughout are targeted at Java 25 and may vary in older Java versions. For example, `--class-path` was introduced when `--module-path` became an option but won't work on Java 8 and earlier (which require `-cp` or `--classpath`).

## javac

#### Basic usage

`javac` *`some/package/`*`MyClass.java`

#### Description

`javac` is the Java source code compiler—it produces bytecode (in the form of *.class* files) from *.java* source files.

For modern Java projects, `javac` is not often used directly, as it is rather low-level and unwieldy, especially for larger codebases. Instead, modern integrated development environments (IDEs) either drive `javac` automatically for the developer or have built-in compilers for use while code is being written. For deployment, most projects will use a separate build tool, most commonly Maven or Gradle. Discussion of these tools is outside the scope of this book.

Nevertheless, it is useful for developers to understand how to use `javac`, as there are cases when compiling small codebases by hand is preferable to having to install and manage a production-grade build tool such as Maven.

#### Common switches

`-cp`, `--class-path` *`<path>`*

Supply classes we need for compilation.

`-p`, `--module-path` *`<path>`*

Supply application modules for compilation. See Chapter 14 for a full discussion of Java modules.

`-d` *`some/dir`*

Tell `javac` where to output class files.

`@project.list`

Load options and source files from the file *project.list*.

`-help`

Help on options.

`-X`

Help on nonstandard options.

`-source <`*`version`*`>`

Control the Java version that `javac` will accept.

`-target <`*`version`*`>`

Control the version of class files that `javac` will output.

`-profile <`*`profile`*`>`

Control the profile that `javac` will use when compiling the application.

`-Xlint`

Enable detail about warnings.

`-Xstdout <`*`path`*`>`

Redirect output of compilation run to a file.

`-g`

Add debug information to class files.

#### Notes

`javac` has traditionally accepted switches (`-source` and `-target`) that control the version of the source language that the compiler accepts and the version of the class file format used for the outputted class files. From JDK 9 onward, setting these versions is better accomplished with the `-release` flag, which ensures the APIs used align with the version selected, not just the bytecode format output.

This functionality introduces additional compiler complexity (as multiple language syntaxes must be supported internally) for very little developer benefit. In Java 8, this capability was slightly tidied up and placed on a more formal basis.

The older versions supported by `javac` vary with each JDK. Consult `javac --help` for a listing of the supported versions.

C and C++ developers may find that the `-g` switch is less helpful to them than it is in those other languages. This is largely due to the widespread use of IDEs in the Java ecosystem—integrated debugging is simply a lot more useful, and easier to use, than additional debug symbols in class files.

The use of `-Xlint` remains somewhat controversial among developers. Many Java developers produce code that triggers a large number of compilation warnings, which they then simply ignore. However, experience on larger codebases (especially on the JDK codebase itself) suggests that in a substantial percentage of cases, code that triggers warnings is code in which subtle bugs may lurk. Use of the lint feature, or static analysis tools (such as SpotBugs), is strongly recommended.

## java

#### Basic usage

```
java some.package.MyClass
java -jar my-package.jar
```

#### Description

`java` is the executable that starts up a Java Virtual Machine. The initial entry point into the program is the `main()` method, which can exist with a number of signatures:

```
public static void main(String[] args)
public static void main()
void main(String[] args)
void main()
```

The detected main method is run on the single application thread created by the JVM startup. The JVM process will exit once this method returns (and any additional nondaemon application threads that were started have terminated).

If the form takes a JAR file rather than a class (the executable JAR form), the JAR file must contain a piece of metadata that tells the JVM which class to start from.

This bit of metadata is the `Main-Class`: attribute, and it is contained in the *MANIFEST.MF* file in the *META-INF/* directory. See the description of the `jar` tool for more details.

#### Common switches

`-cp`, `--class-path <path>`

Define the classpath to read from.

`-p`, `--module-path <path>`

Define the path to find modules.

`--list-modules`

List modules found with current settings and exits.

`-X`, `-?`, `-help`

Provide help about the `java` executable and its switches.

`-D<property=value>`

Set a Java system property that can be retrieved by the Java program. Any number of such properties can be specified this way.

`-jar`

Run an executable JAR (see the entry for `jar`).

`-Xbootclasspath(/a or /p)`

Run with an alternative system classpath (very rarely used).

`-client`, `-server`

Select a HotSpot JIT compiler (see "Notes" for this entry).

`-Xint`, `-Xcomp`, `-Xmixed`

Control JIT compilation (very rarely used).

`-Xms<size>`

Set the minimum committed heap size for the JVM.

`-Xmx<size>`

Set the maximum committed heap size for the JVM.

`-agentlib:<agent>`, `-agentpath:<path to agent>`

Specify a JVM Tooling Interface (JVMTI) agent to attach to the process being started. Agents are typically used for instrumentation or monitoring.

`-verbose`

Generate additional output. Sometimes useful for debugging.

`--dry-run`

Loads the JVM for your application without executing its `main` method. Useful for debugging complex command-line flags such as module configuration.

#### Notes

The HotSpot VM contains two separate JIT compilers—known as the client (or C1) compiler and the server (or C2) compiler. These were designed for different purposes, with the client compiler offering more predictable performance and quicker startup, at the expense of not performing aggressive code optimization.

Traditionally, the JIT compiler that a Java process used was chosen at process startup via the `-client` or `-server` switch. However, as hardware advances have made compilation ever cheaper, a new possibility has become available—to use the client compiler early on, while the Java process is warming up, and then to switch to the high-performance optimizations available in the server compiler when they are available. This scheme is called *Tiered Compilation*, and it is the default starting with Java 8. Most processes will no longer need explicit `-client` or `-server` switches.

On the Windows platform, a slightly different version of the `java` executable is often used —`javaw.exe`. This version starts up a JVM without forcing a Windows console window to appear.

In older Java versions, a number of different legacy interpreters and virtual machine modes were supported. These have now mostly been removed, and any remaining should be regarded as vestigial.

Switches that start with `-X` were intended to be nonstandard switches. However, the trend has been to standardize a number of these switches (particularly `-Xms` and `-Xmx`). In parallel, Java versions have introduced an increasing number of `-XX`: switches. These were intended to be experimental and not for production use. However, as the implementations have stabilized, some of these switches are now suitable for some advanced users (even in production deployments).

In general, a full discussion of switches is outside the scope of this book. Configuration of the JVM for production use is a specialist subject, and developers are urged to take care, especially when modifying any switches related to the garbage collection subsystem.

## jar

#### Basic usage

```
jar cvf my.jar someDir/
```

#### Description

The `jar` utility is used to create and manipulate Java archive (*.jar*) files. These are ZIP format files that contain Java classes, additional resources, and (usually) metadata. The tool has five major modes of operation—Create, Update, Index, List, and Extract—on a JAR file.

These are controlled by passing a command option character (not a switch) to `jar`. Only one command character can be specified, but optional modifier characters can also be used.

#### Command options

- `c`: Create a new archive
- `u`: Update archive
- `i`: Index an archive
- `t`: List an archive
- `x`: Extract an archive

#### Modifiers

- `v`: Verbose mode
- `f`: Operate on a named file, rather than standard input
- `0`: Store, but do not compress, files added to the archive
- `m`: Add the contents of the specified file to the JAR metadata manifest
- `e`: Make this JAR executable, with the specified class as the entry point

#### Notes

The syntax of the `jar` command is intentionally very similar to that of the Unix `tar` command. This similarity is the reason `jar` uses command options, rather than switches (as the other Java platform commands do). More typical explicit switches (e.g., `--create`) are also available, and documentation for them can be found via `jar --help`.

When you create a JAR file, the `jar` tool will automatically add a directory called *META-INF* that contains a file called *MANIFEST.MF*—this is metadata in the form of headers paired with values. By default, *MANIFEST.MF* contains just two headers:

```
Manifest-Version: 1.0
Created-By: 17.0.4 (Eclipse Adoptium)
```

Using the `m` option allows additional metadata to be added into *MANIFEST.MF* at JAR creation time. One frequently added piece is the `Main-Class:` attribute, which indicates the entry point into the application contained in the JAR. A JAR with a specified `Main-Class:` can be directly executed by the JVM, via `java -jar` or by double-clicking the JAR in a graphical file browser.

The addition of the `Main-Class:` attribute is so common that `jar` has the `e` option to create it directly in *MANIFEST.MF*, rather than having to create a separate text file for this purpose. Contents of a JAR, including the manifest, may be inspected easily using the `--extract` option.

## javadoc

#### Basic usage

`javadoc` *`some.package`*

#### Description

`javadoc` produces documentation from Java source files. It does so by reading a special comment format (known as Javadoc comments) and parsing it into a standard documentation format, which can then be output into a variety of document formats (although HTML is by far the most common).

For a full description of Javadoc syntax, refer to Chapter 7.

#### Common switches

`-cp`, `--class-path` *`<path>`*

Define the classpath to use

`-p`, `--module-path` *`<path>`*

Define the path to find modules

`-D` *`<directory>`*

Tell `javadoc` where to output the generated docs

`-quiet`

Suppress output except for errors and warnings

#### Notes

The platform API docs are all written in Javadoc.

`javadoc` is built on top of the same classes as `javac` and uses some of the source compiler infrastructure to implement Javadoc features.

The typical way to use `javadoc` is to run it against a whole package, rather than just a class.

`javadoc` has a very large number of switches and options that can control many aspects of its behavior. Detailed discussion of all the options is outside the scope of this book.

## jdeps

The `jdeps` tool is a static analysis tool for analyzing the dependencies of packages or classes. The tool has a number of usages, from identifying developer code that makes calls into the internal, undocumented JDK APIs (such as the `sun.misc` classes) to helping trace transitive dependencies.

`jdeps` can also be used to confirm whether a JAR file can run under a Compact Profile.

#### Basic usage

```
jdeps com.me.MyClass
```

#### Description

`jdeps` reports dependency information for the classes it is asked to analyze. The classes can be specified as any class on the classpath, a filepath, a directory, or a JAR file.

#### Common switches

`-cp`, `--class-path` <*path*>

Define the classpath to use.

`-p`, `--module-path` <*path*>

Define the path to find modules.

`-s`, `-summary`

Print dependency summary only.

`-m <module-name>`

Target a module for analysis

`-v`, `-verbose`

Print all class-level dependencies.

`-verbose:package`

Print package-level dependencies, excluding dependencies within the same archive.

`-verbose:class`

Print class-level dependencies, excluding dependencies within the same archive.

`-p <pkg name>`, `-package <pkg name>`

Find dependencies in the specified package. You can specify this option multiple times for different packages. The `-p` and `-e` options are mutually exclusive.

`-e <regex>`, `-regex <regex>`

Find dependencies in packages matching the specified regular expression pattern. The `-p` and `-e` options are mutually exclusive.

`-include <regex>`

Restrict analysis to classes matching pattern. This option filters the list of classes to be analyzed. It can be used together with `-p` and `-e`.

`-jdkinternals`

Find class-level dependencies in JDK internal APIs (which may change or disappear even in minor platform releases).

`-apionly`

Restrict analysis to APIs—for example, dependencies from the signature of public and protected members of public classes including field type, method parameter types, returned type, and checked exception types.

`-R`, `-recursive`

Recursively traverse all dependencies.

`-h`, `-?`, `--help`

Print help message for `jdeps`.

#### Notes

`jdeps` is a useful tool for making developers aware of their dependencies on the JRE not as a monolithic environment but as something more modular.

## jps

#### Basic usage

```
jps
jps <remote URL>
```

#### Description

`jps` provides a list of all active JVM processes on the local machine (or a remote machine, if a suitable instance of `jstatd` is running on the remote side). Remote URL support requires Remote Method Invocation (RMI); this configuration is explained in more detail in the `jstatd` section.

#### Common switches

`-m`

Output the arguments passed to the main method

`-l`

Output the full package name for the application's main class (or the full pathname to the application's JAR file)

`-v`

Output the arguments passed to the JVM

#### Notes

This command is not strictly necessary, as the standard Unix `ps` command could suffice. However, it does not use the standard Unix mechanism for interrogating the process, so there are circumstances in which a Java process stops responding (and looks dead to `jps`) but is still listed as alive by the operating system.

## jstat

#### Basic usage

```
jstat -options
jstat <report type such as -class> <PID>
```

#### Description

This command displays some basic statistics about a given Java process. This is usually a local process but can be located on a remote machine, provided the remote side is running a suitable `jstatd` process.

#### Common switches

`-options`

List report types that `jstat` can produce

`-class`

Report on classloading activity to date

`-compiler`

JIT compilation of the process so far

`-gcutil`

Detailed garbage collection report

`-printcompilation`

More detail on compilation

#### Notes

The general syntax `jstat` uses to identify a process (which may be remote) is:

```
[<protocol>://]<vmid>[@hostname][:port][/servername]
```

This syntax is used to specify a remote process (which is usually connected to via Java Management Extensions [JMXs] over RMI), but in practice, the more common local syntax simply uses the VM ID, which is the operating system process ID (PID) on mainstream platforms (Linux, Windows, Unix, macOS, etc.).

## jstatd

#### Basic usage

`jstatd <options>`

#### Description

`jstatd` has been deprecated as of JDK 24. It is recommended to use Java Flight Recorder instead for gathering remote JVM information. Related tools like `jps` will continue to work for local JVMs. This deprecation impacts only the RMI-based remote capabilities.

`jstatd` makes information about local JVMs available over the network. It achieves this using RMI and can make these otherwise-local capabilities accessible to JMX clients. This requires special security settings, which differ from the JVM defaults. To start `jstatd`, first we need to create the following file and name it *jstatd.policy*:

```
grant codebase "jrt:/jdk.jstatd" {
   permission java.security.AllPermission;
};

grant codebase "jrt:/jdk.internal.jvmstat" {
   permission java.security.AllPermission;
};
```

This policy file grants all security permissions to any class loaded from the JDK modules that implement `jstatd`. The precise policy requirements changed with the introduction of modules in JDK 9 and may vary in future JDK versions.

To launch `jstatd` with this policy, use this command line:

```
jstatd -J-Djava.security.policy=<path to jstat.policy>
```

#### Common switches

`-p <port>`

Look for an existing RMI registry on that port and create one if not found

`-Joption`

Passes option through to the JVM on startup

#### Notes

It is recommended that `jstatd` is always switched on in production environments but not over the public internet. For most corporate and enterprise environments, this is nontrivial to achieve and will require the cooperation of Operations and Network Engineering staff. However, the benefits of having telemetry data from production JVMs, especially during outages, are difficult

to overstate.

A full discussion of JMX and monitoring techniques is outside the scope of this book.

## jinfo

#### Basic usage

```
jinfo <PID>
jinfo <core file>
```

#### Description

This tool displays the system properties and JVM options for a running Java process (or a core file).

#### Common switches

`-flags`

Display JVM flags only

`-sysprops`

Display system properties only

#### Notes

In practice, this is very rarely used—although it can occasionally be helpful as a sanity check that the expected program is actually the one that is executing.

## jstack

#### Basic usage

```
jstack <PID>
```

#### Description

The `jstack` utility produces a stack trace for each Java thread in the process.

#### Common switches

`-e`

Extended mode (contains additional information about threads)

`-l`

Long mode (contains additional information about locks)

#### Notes

Producing the stack trace does not stop or terminate the Java process. The files that `jstack` produces can be very large, and some postprocessing of the file is usually necessary.

## jmap

#### Basic usage

`jmap <output option> <process>`

#### Description

`jmap` provides a view of memory allocation for a running Java process.

#### Common switches

`-dump:<option>,file=<location;>`

Produce a heap dump from the running process.

`-histo`

Produce a histogram of the current state of allocated memory.

`-histo:live`

This version of the histogram displays information only for live objects.

#### Notes

The histogram form walks the JVM's allocation list. This includes both live and dead (but not yet collected) objects. The histogram is organized by the type of objects using memory and is ordered from greatest to least number of bytes used by a particular type. The standard form does

not pause the JVM.

The live form ensures that it is accurate by performing a full, stop-the-world (STW) garbage collection before executing. As a result, it should not be used on a production system at a time when a full GC would appreciably impact users.

For the `-dump` form, note that the production of a heap dump can be a time-consuming process and is STW. The size of the resulting file is proportional to the currently allocated heap and hence may be extremely large for some processes.

javap

#### Basic usage

```
javap <classname>
javap <path/to/ClassFile.class>
```

#### Description

`javap` is the Java class disassembler—effectively a tool for peeking inside class files. It can show the bytecode that Java methods have been compiled into, as well as the constant pool information (which contains information similar to that of the symbol table of Unix processes).

By default, `javap` shows signatures of `public`, `protected`, and default methods. The `-p` switch will also show `private` methods.

#### Common switches

`-c`

Decompile bytecode

`-v`

Verbose mode (include constant pool information)

`-p`

Include `private` methods

`-cp`, `--class-path`

Location of classes if loading by class name

`-p`, `--module-path`

Location of modules if loading by class name

#### Notes

The `javap` tool will work with any class file, provided `javap` is from a JDK version that is the same as (or later than) the one that produced the file.

> **NOTE**
>
> Some Java language features may have surprising implementations in bytecode. For example, as we saw in Chapter 10, Java's `String` class has effectively immutable instances, and the JVM implements the string concatenation operator `+` in a different way in Java 9 and later. This difference is clearly visible in the disassembled bytecode shown by `javap`.

jlink

#### Basic usage

```
jlink [options] --module-path modulepath --add-modules module
```

#### Description

`jlink` is the custom runtime image linker for the Java platform—a tool for linking and packaging Java classes, modules, and their dependencies into a custom runtime image. The image created by the `jlink` tool will comprise a linked set of modules, along with their transitive dependencies.

#### Common switches

`--add-modules <module> [, module1]`

Add modules to the root set of modules to be linked

`--endian {little|big}`

Specify the endianness of the target architecture

`--module-path <path>`

Specify the path where the modules for linking can be found

`--save-opts <file>`

Save the options to the linker in the specified file

`--help`

Print help information

`@<filename>`

Read options from the given file instead of the command line

#### Notes

The `jlink` tool will work with any class file or module, and linking will require the transitive dependencies of the code to be linked.

> **NOTE**
>
> Custom runtime images don't have any support for automatic updates by default. This means developers are responsible for rebuilding and updating their own applications in the field when necessary. Some Java language features may have restrictions, as the runtime image may not include the full JDK; therefore, reflection and other dynamic techniques may not be fully supported.

jmod

#### Basic usage

`jmod create [options] my-new.jmod`

#### Description

`jmod` prepares Java software components for use by the custom linker (`jlink`). The result is a *.jmod* file. This should be considered an intermediate file, not a primary artifact for distribution.

#### Basic modes

`create`

Create a new JMOD file

`extract`

Extract all files from a JMOD file (explode it)

`list`

List all files from a JMOD file

`describe`

Print details about a JMOD file

#### Common switches

`--module-path <path>`

Specify the module path where the core contents of the module can be found

`--libs <path>`

Specify the path where native libraries for inclusion can be found

`--help`

Print help information

`<@filename>`

Read options from the specified file instead of the command line

#### Notes

`jmod` reads and writes the JMOD format, but please note that this is different from the modular JAR format and is not intended as an immediate replacement for it.

> **NOTE**
>
> The `jmod` tool is currently intended only for modules that are to be linked into a runtime image (using the `jlink` tool). One other possible use case is for packaging modules that have native libraries or other configuration files that must be distributed along with the module.

## jcmd

#### Basic usage

```
jcmd <PID>
jcmd <PID> <command>
```

#### Description

`jcmd` issues commands to a running Java process. The precise commands may vary between Java versions and may be listed by running `jcmd` with the process ID and no command.

#### Common switches

`-f <path>`

Read commands from a file rather than command-line arguments

`-l`

List Java processes (similar to `jps`)

`--help`

Print help information

#### Common commands

`GC.heap_dump <path>`

Generate a heap dump like `jmap`. Note the path is relative to the Java process, *not* relative to where `jcmd` is run!

`GC.heap_info`

Display statistics and sizing information about the Java process heap.

`JFR.start`

Begin a Java Flight Recorder (JFR) session. JFR is the JVM's built-in performance monitoring and profiling tool.

`JFR.stop name=<name from start> filename=<path>`

Stop named JFR session and record to a file.

`VM.system_properties`

Output Java process system properties.

#### Notes

**NOTE**

Commands to `jcmd` are grouped by the subsystem they interact with—for instance, `GC` or `JFR`. There are many

more commands than the examples we've given here. It's worth exploring what's available on your Java installation for help in operating the JVM in production.

jwebserver

#### Basic usage

```
jwebserver
```

#### Description

`jwebserver` runs a minimal web server for serving static files. It is intended for use in prototyping and testing and does not provide dynamic functionality.

#### Common switches

`-b <address>`

Address to bind to if the default of 127.0.0.1 is not desired.

`-p <port>`

Port to listen on. Defaults to 8000 if omitted.

`-d <dir>`

Location to serve files from. Defaults to current directory if omitted.

`-o <none|info|verbose>`

Output format from the running server. `info` logs a line per request; `verbose` also includes headers.

`--help`

Print help information.

#### Notes

**NOTE**

Prior to the addition of `jwebserver` in Java 18, developers had to look to other languages or write a meaningful amount of code to get even basic HTTP serving capabilities. This serves as an alternative to popular approaches such as `python -m http.server`.

## Introduction to Java Flight Recorder (JFR)

Java Flight Recorder (JFR) is a powerful, low-latency profiling system built directly into the JVM. It has existed for years but prior to Java 11 was available only with a commercial license. Now this rich source of information is available with OpenJDK and worth exploring.

The typical JFR workflow involves starting a profile against a running JVM, downloading the results as a file, and then inspecting that file offline with the JDK Mission Control (JMC) GUI application. While JFR is embedded directly within OpenJDK, JMC isn't distributed with the JDK but can be downloaded from the JDK Java site.

JFR recording can be started either via options at JVM startup or interactively with the `jcmd` tool shown earlier in this chapter. The following `java` invocation starts with JFR recording for two minutes, writing the results to a file when finished:

```
java -XX:StartFlightRecording=duration=120s,filename=flight.jfr \
   Application
```

Options allow for tight control over the volume of data JFR will hold in memory, either by specifying how long a recording to generate or by the size of the file that will be generated. When combined with its low overhead, JFR can be plausibly run persistently in production so data is always ready should you wish to capture it (sometimes referred to as *ring buffer* mode). This opens up a world of possibilities for debugging in amazing detail with the JFR profiles, even minutes to hours after a problem has occurred.

Along with sizing limits, JFR recording can also be configured to gather only specific information of interest. Typical areas (but only a few of the things JFR measures) include:

- Object allocation
- Garbage collection
- Threads and locks
- Method profiling

With Java 17, APIs are available in-process to consume JFR events in a streaming fashion, evolving away from the file-based profiling approaches. This opens the door for monitoring tooling to tap into this rich source of data, without the hassle of logging onto servers to ask for a profile to be dumped.

In the future, we may expect JFR to act as a data source for the new generation of observability tools that are being adopted by the Java ecosystem, such as OpenTelemetry.

# Index

## Symbols


! (exclamation point)

!= (not equals) operator, Operator summary table, Comparison Operators

boolean NOT operator, Operator summary table, Boolean Operators

" (quotes, double)

enclosing string literals, String literals

in string literals, String literals

literals in, Literals

$ character, Naming and Capitalization Conventions

% (percent sign)

%= (modulo assignment) operator, Operator summary table

modulo operator, Operator summary table, Arithmetic Operators

& (ampersand)

&& (conditional AND) operator, Operator summary table, Boolean Operators, Boolean Operators

&= (bitwise AND assignment) operator, Operator summary table

bitwise AND operator, Operator summary table, Bitwise and Shift Operators

boolean AND operator, Operator summary table, Boolean Operators

' (quotes, single)

enclosing character literals, The char Type

in char literals, String literals

in string literals, String literals

literals in, Literals

() (parentheses)

cast operator, Operator summary table, Special Operators

enclosing expressions in if statements, The if/else Statement

enclosing method parameter list, Defining Methods

method invocation operator, Operator summary table, Side effects, Special Operators

overriding operator precedence, Precedence

separators (tokens), Punctuation

* (asterisk)

*= (multiply assignment) operator, Operator summary table

in doc comments, Formatting Multiline Doc Comments

in multiline comments, Comments

multiplication operator, Operator summary table, Arithmetic Operators

+ (plus sign)

++ (increment) operator, Operator summary table, Increment and Decrement Operators

+= (add assignment) operator, Operator summary table, Arithmetic Operators

addition operator, Operator summary table, Arithmetic Operators

string concatenation operator, Operator summary table, Arithmetic Operators, String Concatenation Operator, String concatenation

unary plus operator, Operator summary table

, (comma) operator, Generic Methods

, (comma) separators (tokens), Punctuation

- (minus sign)

-- (decrement) operator, Operator summary table, Increment and Decrement Operators

-= (subtract assignment) operator, Operator summary table

subtraction operator, Operator summary table, Operand number and type, Arithmetic Operators

unary minus operator, Operator summary table, Operand number and type, Arithmetic Operators

-> (lambda arrow) operator, Operator summary table

. (dot)

member access operator, Special Operators

object member access operator, Operator summary table

separators (tokens), Punctuation

/ (slash)

/* */ in multiline comments, Comments

/** */ in doc comments, Comments, Java Documentation Comments

// in single-line comments, Comments

/// in doc comments, Comments, Formatting Markdown Doc Comments-Formatting Markdown Doc Comments

/= (divide assignment) operator, Operator summary table

division operator, Operator summary table, Arithmetic Operators

0 (zero)

division by zero, Arithmetic Operators

negative and positive zero, Floating-Point Types

represented by float and double types, Floating-Point Types

represented by integer types, How Java Represents Integer Types

:: (colons) separators (tokens), Punctuation

; (semicolon)

for loops and, The for Statement

in abstract methods, Method Modifiers

in break statements, The break Statement

separators (tokens), Punctuation

terminating do loops, The do Statement

< > (angle brackets)

< (less than) operator, Operator summary table, Comparison Operators

<< (left shift) operator, Bitwise and Shift Operators

<< (signed left shift) operator, Operator summary table

<<= (left shift assignment) operator, Operator summary table

<= (less than or equal to) operator, Operator summary table, Comparison Operators

> (greater than) operator, Operator summary table, Comparison Operators

>= (greater than or equal to) operator, Operator summary table, Comparison Operators

>> (signed right shift) operator, Operator summary table, Bitwise and Shift Operators

>>= (right shift assignment) operator, Operator summary table

>>> (unsigned right shift) operator, Operator summary table, Bitwise and Shift Operators

>>>= (unsigned right shift assignment) operator, Operator summary table

homogeneous collections and, Introduction to Generics

= (equals sign)

= (assignment) operator, confusion with == (equal to) operator, Assignment Operators

== (equal to) operator, Operator summary table, Comparison Operators, Comparing Objects

assignment operator, Operator summary table, Assignment Operators

? (question mark)

? : (conditional) operator, Operator summary table, Operand number and type, Return type, The Conditional Operator

regular expression metacharacter, Regular Expressions

@ (at sign)

in doc-comment tags, Structure of a Doc Comment

separators (tokens), Punctuation

[ ] (brackets)

accessing array elements, Special Operators, Accessing array elements

after array element type, Special Operators, Array Types

array access operator, Operator summary table

in regular expressions, Regular Expressions

\ (backslash)

in escape sequences, The Unicode Character Set

\f (form feed) escape sequence, The char Type

\n (newlines, escaping), The char Type

\r (carriage return) escape sequence, The char Type

\t (tab), The char Type

\\ (backslash)

\\ escape sequence in char literals, The char Type

^ (caret)

?= (bitwise XOR assignment) operator, Bitwise and Shift Operators

bitwise XOR operator, Operator summary table, Bitwise and Shift Operators

boolean XOR operator, Operator summary table, Boolean Operators

_ (underscore), Identifiers

{ } (curly braces)

in nested if/else statements, The if/else Statement

in switch statements, The switch Statement

in try/catch/finally statements, The try/catch/finally Statement

members of a class in, Basic OO Definitions

optional for small lambda expressions, Special Operators

separators (tokens), Punctuation

| (vertical bar)

bitwise OR operator, Operator summary table, Bitwise and Shift Operators

boolean OR operator, Operator summary table, Boolean Operators

|= (bitwise OR assignment) operator, Operator summary table

|| (conditional OR) operator, Operator summary table, Boolean Operators

~ (tilde), bitwise complement operator, Operator summary table, Bitwise and Shift Operators

… (ellipses)

in variable-length argument lists, Variable-Length Argument Lists

separators (tokens), Punctuation

## A

abs(), Java’s Standard Library of Mathematical Functions

abstract classes, Abstract Classes and Methods-Abstract Classes and Methods

defined, Class Definition Syntax

interfaces versus, Interfaces Versus Abstract Classes-Delegation

partial implementations, Partial Implementations-Partial Implementations

abstract methods, Method Modifiers, Abstract Classes and Methods-Abstract Classes and Methods

AbstractList, Partial Implementations

access control, Access Control-Member access summary

access to classes, Access to classes

access to members of a class, Access to members-Access to members

access to modules, Access to modules

access to packages, Access to packages

data accessor methods, Data Accessor Methods-Data Accessor Methods

inheritance and, Access control and inheritance

member access summary, Member access summary

modules, Access Control-Member access summary

access modifiers, Method Modifiers, Basic OO Definitions

accessor methods, field inheritance and, Accessors

Account class, Basic OO Definitions, Reference Type Conversions

ActionListener, Lambda Expressions

add(), The Set Interface

addAll(), The Set Interface

addition operator (+), Arithmetic Operators

adjusters, date and time, Adjusters

ahead-of-time (AOT) compilation, Leyden and Cloud Native Java

ahead-of-time cache, Leyden and Cloud Native Java

AI, Javas role in adoption of, Panama

allocateDirect(), ByteBuffer

Android, What Is the Java Ecosystem?

annotations (annotated types), Annotations-Annotations

anonymous classes, Lambda Expression Conversion, Anonymous Classes-Anonymous Classes

AOT (ahead-of-time) compilation, Leyden and Cloud Native Java

application classloader, Application classloader

arithmetic operators, Arithmetic Operators-Arithmetic Operators

ArithmeticException, Integer Types

array, Arrays-Multidimensional Arrays

- accessing array elements, Special Operators, Accessing array elements
- array bounds, Array bounds
- array types, Array Types-Array type widening conversions
- as operand type, Increment and Decrement Operators, The instanceof Operator
- conversion rules, Reference Type Conversions
- converting to/from objects, Arrays and Helper Methods
- copying, Copying arrays
- covariance, Array type widening conversions
- creating/initializing, Creating and Initializing Arrays-Array initializers
- creation with new operator, Special Operators
- initializers, Array initializers
- iterating, Iterating arrays
- Java Collections and, Arrays and Helper Methods
- Java Streams, Arrays and Helper Methods
- literal, Special Operators
- multidimensional, Multidimensional Arrays-Multidimensional Arrays
- type widening conversions, Array type widening conversions
- using, Using Arrays-Array utilities
- utilities, Array utilities

arrayCopy(), Arrays and Helper Methods

ArrayDeque, Queues and BlockingQueues

Arrays class, Array utilities

ASCII escape sequences, The char Type

asPredicate(), Regular Expressions

assert statements, The assert Statement-Enabling assertions

AssertionError, The assert Statement

assertions, The assert Statement

assignment operators, Assignment Operators-Assignment Operators

associativity, Associativity

Async I/O, Async I/O-Watch Services and Directory Searching

- callback-based style, Callback-Based Style-Callback-Based Style
- future-based style, Future-Based Style
- watch services and directory searching, Watch Services and Directory Searching

atomic counters, Atomic Counters-Atomic Counters

AtomicBoolean class, Atomic Counters

AtomicInteger class, Atomic Counters

AtomicLong class, Atomic Counters

@author doc-comment tag, Doc-Comment Tags

autoboxing, Integer Types, Boxing and Unboxing Conversions

AutoCloseable interface, try-with-resources Revisited

autocompletion, Introduction to JShell

automatic imports, Importing Types

automatic memory management

- basic concepts, Basic Concepts of Automatic Memory Management-Exactness and Limitations of Naive Mark-and-Sweep
- mark-and-sweep algorithm, Introducing Mark-and-Sweep-Exactness and Limitations of Naive Mark-and-Sweep

automatic modules, Automatic Modules

## B

backward compatibility
    Java and, Backward compatibility
    type erasure and, Type Erasure
base types, Reference Type Conversions
benign data race, Hash codes and effective immutability
BigDecimal, BigDecimal
binary operators, Operand number and type
bitwise AND (&) operator, Bitwise and Shift Operators
bitwise complement (~) operator, Operator summary table, Bitwise and Shift Operators
bitwise operators, Bitwise and Shift Operators-Bitwise and Shift Operators
bitwise OR (|) operator, Operator summary table, Bitwise and Shift Operators
BlockingQueues, Queues and BlockingQueues-Queues and BlockingQueues
body, in class definition, Basic OO Definitions
Boolean operators, Boolean Operators-Boolean Operators
    (see also specific operators)
boolean type, The boolean Type
bootstrap classloader, Bootstrap classloader
bound method reference, Method References, forEach
bounded queue, Queues and BlockingQueues
bounded type parameters, Bounded Type Parameters-Bounded Type Parameters
bounded wildcards, Bounded wildcards-Bounded wildcards
boxing conversions, Boxing and Unboxing Conversions
break statements, The switch Statement, The break Statement
BufferedReader class, Readers and Writers, Decorators
buffers, NIO, NIO Channels and Buffers-Mapped Byte Buffers
builder pattern, Builders-Builders
byte type, Integer Types
ByteBuffer class

basics, ByteBuffer-ByteBuffer

MappedByteBuffer, Mapped Byte Buffers

bytecode

class files and, The Lifecycle of a Java Program

defined, What is bytecode?

endianness, Is bytecode really machine independent? What about things like endianness?

machine-independence of, Is bytecode really machine independent? What about things like endianness?

optimization and, Is bytecode optimized?

security and, Insecure

terminology, Why is it called “bytecode”?

verification, Insecure

versioning, The Source Code Compiler

byte[], ByteBuffer

## C

C/C++

access modifiers for fields, Constants

enum types in, Enums

increment/decrement operators, Increment and Decrement Operators

interchangeability of integers/pointers, Secure Programming and Classloading

Java compared to, Java Compared to C

model for Java language syntax, What Is the Java Language?

operator precedence, Precedence

pointers or memory addresses, references as, Reference Versus Primitive Types

switch statement, The switch Statement

using native methods to interface Java code to C/C++ libraries, Method Modifiers

virtual keyword, Virtual method lookup

CachedThreadPool, Executors and Threadpools

callback style, for async I/O, Callback-Based Style-Callback-Based Style

camel case, Identifiers, Naming and Capitalization Conventions

capitalization conventions, Naming and Capitalization Conventions-Naming and Capitalization Conventions

captured variable, Lexical Scoping and Local Variables

case guards, Pattern Matching

case labels (switch statements), The switch Statement

case sensitivity, Case Sensitivity and Whitespace

casts

() (cast) operator, Special Operators

conversion rules and, Reference Type Conversions

of primitive types, Primitive Type Conversions

catch clause, catch

CDS (class-data sharing) archives, Leyden and Cloud Native Java

ceil(), Java's Standard Library of Mathematical Functions

channels

ByteBuffer, ByteBuffer-ByteBuffer

MappedByteBuffer, Mapped Byte Buffers

char type, The char Type-String literals

Character class, static method, The char Type

conversion to and from integer and floating-point types, Primitive Type Conversions-Primitive Type Conversions

conversion to other primitive types, Primitive Type Conversions

escape characters in char literals, The char Type

surrogate pair, Unicode supplementary characters, The char Type

checked exceptions, Checked and Unchecked Exceptions-Working with checked exceptions

in throws clause of method signature, Defining Methods

working with, Working with checked exceptions

CHM (ConcurrentHashMap) class, The Map Interface, Concurrent Maps

chrono package, Introducing the Java 8 Date and Time API

circle-ellipse problem (OOD case study), Case Study: Circle-Ellipse Problem-Case Study: Circle-Ellipse Problem

class, Introduction to Classes and Objects-Lambda Expressions

(see also specific classes)

access control, Access to classes

access control for members of a class, Access to members-Access to members

basic object-oriented definitions, Basic OO Definitions

class definition syntax, Class Definition Syntax-Class Definition Syntax

constructor chaining, Constructor Chaining

core classes of Java platform, Packages and the Java Namespace

defined, Basic OO Definitions

defining a class, Defining a Class

definition syntax, Class Definition Syntax-Class Definition Syntax

enums, Enums and Annotations-Enums

extending, Extending a Class-Extending a Class

fields and methods, Fields and Methods-How the this Reference Works

final, Extending a Class

hiding data within (see encapsulation)

hierarchy, Superclasses, Object, and the Class Hierarchy

implicitly declared (Java 25), Simple Standalone Java Programs

initializer blocks, Field Defaults and Initializers

lambda expressions and, Lambda Expressions-Lambda Expressions

name collisions, preventing, Globally Unique Package Names

naming conventions, Naming and Capitalization Conventions

object-oriented programming, Class Definition Syntax-Class Definition Syntax

sealed, Sealed Classes-Sealed Classes

simple/fully qualified names, Packages and the Java Namespace

subclasses and inheritance, Subclasses and Inheritance-Sealed Classes

superclasses, Superclasses, Object, and the Class Hierarchy

Class class, Type literals

class declarations, Java Source File Structure

class fields, Class Fields

class files, What Is the JVM?, The Source Code Compiler

class hierarchy, Introduction to JShell, Superclasses, Object, and the Class Hierarchy

class keyword, Class Definition Syntax

class methods, Class Methods

instance methods versus, Instance Methods or Class Methods?-Instance Methods or Class Methods?

static modifier, Method Modifiers

static synchronized, The synchronized Statement

synchronized modifier, Method Modifiers

Class objects

creating new instances with reflection, Creating instances with Reflection-Creating instances with Reflection

reflection, Reflection-Problems with Reflection

type literals, Type literals

class objects, Class Files, Class Objects, and Metadata-Custom classloader

(see also classloading)

examples of, Examples of Class Objects-Examples of Class Objects

metadata with, Class Objects and Metadata-Class Objects and Metadata

class-data sharing (CDS) archives, Leyden and Cloud Native Java

Class.forName(), Applied Classloading

classic Java I/O, Classic Java I/O-Classic Java I/O

ClassLoader, Applied Classloading

classloading, Class Files, Class Objects, and Metadata-Class Objects and Metadata

- application classloader, Application classloader
- applied, Applied Classloading-Custom classloader
- bootstrap classloader, Bootstrap classloader
- custom classloader, Custom classloader-Custom classloader
- hierarchy of classloaders, Classloader Hierarchy-Custom classloader
- initialization phase, Initialization
- loading phase, Loading
- phases of, Phases of Classloading-Initialization
- platform classloader, Platform classloader
- preparation and resolution, Preparation and Resolution
- secure programming, Secure Programming and Classloading-Secure Programming and Classloading
- verification phase, Verification

ClassNotFoundException, Applied Classloading

clone() method, Array Types, clone()-clone()

Cloneable interface, Array Types

closures, Lexical Scoping and Local Variables, Lexical Scoping and Local Variables

cloud-native Java, Leyden and Cloud Native Java-Leyden and Cloud Native Java

- Leyden, Leyden and Cloud Native Java
- Quarkus, Leyden and Cloud Native Java

@code doc-comment tag, Inline Doc-Comment Tags

collect(), From streams to Collections-From streams to Collections, From streams to Collections

Collection interface, The Collection Interface-The Collection Interface

collection views, The Map Interface

collections, Working with Java Collections-Summary

- arrays and helper methods, Arrays and Helper Methods

basics, Introduction to Collections API-Mutability and Immutability Revisited

Collection interface, The Collection Interface-The Collection Interface

creating, Creating Collections

Iterable/Iterator interfaces, The Iterable and Iterator interfaces-The Iterable and Iterator interfaces

iterating over with foreach loops, The foreach Variant-What foreach cannot do

Java Streams and lambda expressions, Java Streams and Lambda Expressions-Arrays and Helper Methods

List interface, The List Interface-Random access to lists

Map interface, The Map Interface-The Map Interface

mutability/immutability, Mutability and Immutability Revisited-Mutability and Immutability Revisited

Set interface, The Set Interface-The Set Interface

special-case collections, Special-case collections

utility methods, Utility Methods-Special-case collections

Collections

special-case collections, Special-case collections

utility methods, Utility Methods-Special-case collections

wrapper methods, Utility Methods

colliding default method, Implementation of default methods

command-line tools, Command-Line Tools-Command-Line Tools

comments, in Java programs, Comments-Comments

compact constructor, Record Constructors, OOD Using Records

compacting collector, ParallelOld

compareTo(), Comparable::compareTo()-Comparable::compareTo()

comparison operators, Comparison Operators-Comparison Operators

compilation units, Java Programs from the Top Down

compile-time typing, Compile and Runtime Typing

compilers

javac and, Is javac a compiler?, Command-Line Tools-Command-Line Tools

source code compiler, The Source Code Compiler-The Source Code Compiler

CompletionHandler interface, Callback-Based Style-Callback-Based Style

compound declaration, Generic Methods

compound statements, Compound Statements

concatenation, of strings, String concatenation-String concatenation

concurrency, Concurrency in Java-Summary

(see also memory management; threads)

basic concepts, Basic Concepts of Java Concurrency-Volatile State

concurrent data structures, Concurrent Data Structures-Queues and BlockingQueues

executors and threadpools, Executors and Threadpools-Executors and Threadpools

Loom, Long-Term JDK Projects

primitives, The synchronized Statement

protecting state with exclusion, Protecting State with Exclusion-Protecting State with Exclusion

safety, Concurrent Safety

synchronize statement, The synchronized Statement

thread lifecycle, Thread Lifecycle-Thread Lifecycle

virtual threads, Virtual Threads-Virtual Threads

visibility and mutability, Visibility and Mutability-Visibility and Mutability

volatile state, Volatile State

working with threads, Working with Threads-Synchronization Checklist

concurrent data structures

atomic counters, Atomic Counters-Atomic Counters

concurrent maps, Concurrent Maps-Concurrent Maps

queues and BlockingQueues, Queues and BlockingQueues-Queues and BlockingQueues

concurrent maps, Concurrent Maps-Concurrent Maps

concurrent safety, Concurrent Safety

ConcurrentHashMap (CHM) class, The Map Interface, Concurrent Maps

ConcurrentMap interface, The Map Interface

ConcurrentModificationException, The Iterable and Iterator interfaces

ConcurrentSkipListMap class, The Map Interface

conditional AND operator (&&), Boolean Operators, Boolean Operators

conditional operator (? :), The Conditional Operator

conservative garbage collectors, Exactness and Limitations of Naive Mark-and-Sweep

constant pool, Class Objects and Metadata

constants

- field access and, Working with Fields and Object State
- importing into code, Importing Static Members
- naming conventions, Naming and Capitalization Conventions
- object-oriented design and, Constants

construction patterns

- builder pattern, Builders-Builders
- constructor prologues, Constructor Prologues-Constructor Prologues
- factory methods, Factory Methods-Factory Methods
- singleton pattern, Singleton-Singleton

constructors

- constructor chaining, Constructor Chaining
- default constructor, The Default Constructor
- defined, Defining Methods
- defining a single constructor, Defining a Constructor-Defining a Constructor
- defining multiple constructors, Defining Multiple Constructors
- definition of term, Special Operators
- invoking one constructor from another, Invoking One Constructor from Another
- purpose of, Creating and Initializing Objects
- record constructors, Record Constructors

subclass constructors, Subclass Constructors

continue statements, Labeled Statements, The continue Statement

control transfer with finally clause, finally

conventions, programming/documentation, Programming and Documentation Conventions-Summary

classes, Naming and Capitalization Conventions

constants, Naming and Capitalization Conventions

doclets, Doclets

enums, Naming and Capitalization Conventions

fields, Naming and Capitalization Conventions

interfaces, Naming and Capitalization Conventions

Java documentation comments, Java Documentation Comments-Doc Comments for Packages

(see also doc comments)

local variables, Naming and Capitalization Conventions

methods, Naming and Capitalization Conventions

modules, Naming and Capitalization Conventions

naming and capitalization conventions, Naming and Capitalization Conventions-Naming and Capitalization Conventions

packages, Naming and Capitalization Conventions

parameters, Naming and Capitalization Conventions

portable code conventions, Conventions for Portable Programs-Conventions for Portable Programs

practical naming, Practical Naming-Practical Naming

static final constant, Naming and Capitalization Conventions

types, Naming and Capitalization Conventions

variables, Naming and Capitalization Conventions

conversions

boxing/unboxing, Boxing and Unboxing Conversions

reference type, Reference Type Conversions-Reference Type Conversions

copy constructor, clone()

CopyOnWriteArrayList class, Random access to lists, Concurrent Maps

CopyOnWriteArraySet class, The Set Interface

CopyOption interface, Files

counters for loops, incrementing, The while Statement

countStackFrames(), suspend(), resume(), countStackFrames(), and destroy()

covariance, generics and, Introducing Covariance-Introducing Covariance

covariant return, Overriding Superclass Methods

cross-references, in doc comments, Cross-References in Doc Comments-Cross-References in Doc Comments

currency symbols in identifiers, Identifiers

custom classloader, Custom classloader-Custom classloader

custom runtime images, JMPS, Custom Runtime Images

## D

daemon thread, setDaemon()

data encapsulation (see encapsulation)

data formats, handling of common, Handling Common Data Formats-Summary

date and time, Date and Time-Legacy Date and Time

numbers and math, Numbers and Math-Java’s Standard Library of Mathematical Functions

text, Text-Regular Expressions

data hiding (see encapsulation)

data types

array index expressions, Accessing array elements

array types, Array Types-Array type widening conversions

boolean types, The boolean Type

char types, The char Type-String literals

conversions, Primitive Type Conversions-Primitive Type Conversions
floating-point types, Arithmetic Operators-Arithmetic Operators
instanceof operator, The instanceof Operator
integer types, Integer Types-Integer Types
of operands, Operand number and type
primitive, Primitive Type Conversions
type conversion or casting with (), Special Operators
date and time API, Introducing the Java 8 Date and Time API-Example
adjusters, Adjusters
diary class example, Example-Example
legacy issues, Legacy Date and Time
queries, Queries-Queries
time zones, Time Zones
timestamp, The parts of a timestamp
date and time formats, Date and Time-Legacy Date and Time
debugging
assertions for, The assert Statement
streams, Lazy evaluation
deconstruction, Pattern Matching
Decorator pattern, Delegation, Decorators-Decorators
decrement operator (--), Increment and Decrement Operators
default access, Access to members
default constructor, The Default Constructor
default implementation, Backward compatibility
default keyword, The switch Statement, Interfaces
default methods, Method Modifiers
backward compatibility and, Backward compatibility
collections and, Streams utility default methods

implementation of, Backward compatibility

inheritance and, Do Default Methods Change Java's Inheritance Model?-Do Default Methods Change Java's Inheritance Model?

interfaces and, Default Methods-Implementation of default methods

defineClass(), Loading

delegation, Delegation-Delegation

@deprecated doc-comment tag, Doc-Comment Tags

destroy(), suspend(), resume(), countStackFrames(), and destroy()

destructuring, Pattern Matching

diamond inheritance, Implementation of default methods

diamond syntax, Diamond Syntax

direct buffers, ByteBuffer

directory streams, Async I/O, Watch Services and Directory Searching

division operator, Arithmetic Operators

do statements, The do Statement

doc comments, Comments, Java Documentation Comments-Doc Comments for Packages

cross-references in, Cross-References in Doc Comments-Cross-References in Doc Comments

for packages, Doc Comments for Packages

formatting Markdown doc comments, Formatting Markdown Doc Comments-Formatting Markdown Doc Comments

formatting multiline comments, Formatting Multiline Doc Comments

inline tags, Inline Doc-Comment Tags-Inline Doc-Comment Tags

structure of, Structure of a Doc Comment

tags, Doc-Comment Tags-Doc-Comment Tags

doc-comment tags, Doc-Comment Tags-Doc-Comment Tags

doclets, Doclets

@docRoot doc-comment tag, Inline Doc-Comment Tags

documentation (see conventions, programming/documentation; javadoc)

double type, Floating-Point Types, Floating-Point Types
    return type for operators, Return type
Duration package, Introducing the Java 8 Date and Time API
dynamically typed language, The Java Type System

## E

eager evaluation, Lazy evaluation
ecosystem, Java, What Is the Java Ecosystem?
Eden space, Evacuation
effectively final variable, Lexical Scoping and Local Variables
effectively immutable classes, Hash codes and effective immutability
else clause, The if/else Statement
else if clause, The else if clause
empty collections, Collections class methods for, Special-case collections
empty statements, The Empty Statement
encapsulation, Basic OO Definitions, Data Hiding and Encapsulation-Data Accessor Methods
    access control, Access Control-Member access summary
    data accessor methods, Data Accessor Methods-Data Accessor Methods
endianness, bytecode and, Is bytecode really machine independent? What about things like endianness?
entry point, main() as, Defining Methods
enums (enumerated types), Enums and Annotations-Enums, Naming and Capitalization Conventions
equality operator (==), Comparison Operators
    comparing reference types, Comparing Objects
    confusion with = (assignment) operator, Assignment Operators
equals(), Comparing Objects, equals()
Error, Exceptions and Exception Handling
errors (see exception entries)

escape sequences, The Unicode Character Set, The char Type, String literals

evacuating collectors, Evacuation

evacuation, Evacuation-Evacuation

evaluation of expressions, Order of evaluation

exact garbage collector, Exactness and Limitations of Naive Mark-and-Sweep

Exception, Exceptions and Exception Handling

@exception doc-comment tag, Doc-Comment Tags

exception handling, The throw Statement, Exceptions and Exception Handling-Exceptions and Exception Handling

exceptions

- checked/unchecked, Checked and Unchecked Exceptions-Working with checked exceptions
- good practices for exception schemes, Mutability and Immutability
- object-oriented design and, Exceptions and Exception Handling-Exceptions and Exception Handling
- throwing, The throw Statement-The throw Statement
- try/catch construction, Simple Standalone Java Programs

exclusion, protecting state with, Protecting State with Exclusion-Protecting State with Exclusion

execution environment, JVM as, What Is the JVM?

executors, Executors and Threadpools-Executors and Threadpools

exp(), Java’s Standard Library of Mathematical Functions

exponential notation, Floating-Point Types

expression statements, Expression Statements

expressions, Expressions and Operators-Special Operators

- defined, Expressions and Operators
- following return statements, The break Statement
- in if statements, The if/else Statement
- statements versus, Statements
- switch statements, The switch Statement

synchronize statements, The synchronized Statement
throw statements, The throw Statement
while statements, The while Statement
extending interfaces, Extending Interfaces
extends clause, Extending Interfaces
extension classloader, Platform classloader

## F

factory methods, Factory Methods-Factory Methods
fall-through, The switch Statement
FFM (Foreign Function and Memory) API, Panama
field hiding, overriding versus, Overriding is not hiding-Overriding is not hiding
fields, Fields and Methods-How the this Reference Works
class fields, Class Fields
declaration syntax, Field Declaration Syntax-Field Declaration Syntax
field defaults/initializers, Field Defaults and Initializers-Field Defaults and Initializers
hiding superclass fields, Hiding Superclass Fields-Hiding Superclass Fields
inheritance, Field Inheritance-Field Inheritance
initializer blocks, Field Defaults and Initializers
instance fields, Instance Fields
modifiers, Field Declaration Syntax-Field Declaration Syntax
naming conventions, Naming and Capitalization Conventions
working with, Working with Fields and Object State-Mutability and Immutability
File class, Importing Types, Classic Java I/O-Classic Java I/O
file handling and I/O, File Handling and I/O-Summary
Async I/O, Async I/O-Watch Services and Directory Searching
classic Java I/O, Classic Java I/O-Classic Java I/O
decorators, Decorators-Decorators

Files class, Files-Files
FileSystem class, FileSystem
HTTP, HTTP-HTTP
IP, IP
networking, Networking-IP
NIO channels/buffers, NIO Channels and Buffers-Mapped Byte Buffers
path, Path-Path
Readers and Writers, Readers and Writers-Readers and Writers
stream abstraction, I/O Streams-I/O Streams
TCP, TCP-TCP
try-with-resources statement, try-with-resources Revisited-try-with-resources Revisited
FileChannel class, Mapped Byte Buffers
FileInputStream class, I/O Streams
filenames
hardcoded, Conventions for Portable Programs
public class and, Java Source File Structure
FileOutputStream class, I/O Streams
FileReader class, Readers and Writers
Files class, Files-Files
FileSystem class, FileSystem
FileVisitor interface, Watch Services and Directory Searching
filter(), Lambdas and Functional Programming
Predicate interface, Filter
Stream interface, Filter, Further filtering
FilterInputStream class, Readers and Writers
final classes, Class Definition Syntax, Extending a Class
final keyword, Local Variable Declaration Statements, Constants, Mutability and Immutability
final methods, Method Modifiers

final modifier, Field Declaration Syntax

finally clause, finally

flatMap(), Flattening

flattening, Flattening

floating-point numbers, Java and Floating-Point Numbers-BigDecimal

floating-point representations, Floating-Point Types

floating-point types, Floating-Point Types-Floating-Point Types

- as approximations, Floating-Point Types
- conversions, Primitive Type Conversions-Primitive Type Conversions
- division by zero, Arithmetic Operators
- Double class, Floating-Point Types
- double type, Floating-Point Types
- Float class, Floating-Point Types
- float type, Floating-Point Types
- floating-point arithmetic, Floating-Point Types, Arithmetic Operators-Arithmetic Operators
- floating-point literals, Floating-Point Types
- return type for operators, Return type
- strictfp modifier for methods, Method Modifiers
- testing if value is NaN, Comparison Operators

floor(), Java’s Standard Library of Mathematical Functions

flow-control statements, Statements

fluent interface, Builders

for statements, The for Statement-The for Statement

- break statement in, The break Statement
- foreach variant, The foreach Variant-What foreach cannot do
- initialize, test, and update expressions in, The for Statement
- iterating arrays, Iterating arrays

foreach, The foreach Variant-What foreach cannot do, Iteration with foreach loops

forEach(), forEach

Foreign Function and Memory (FFM) API, Panama

format package, Introducing the Java 8 Date and Time API

format(), Conventions for Portable Programs

free list, Evacuation

Function<T, R> interface, Map

functional interfaces, Lambda Expressions

functional programming

- key patterns, Functional Approaches-Special-case collections
- lambda expressions and, Lambdas and Functional Programming-Lambdas and Functional Programming

@FunctionalInterface annotation, Lambda Expression Conversion, Method References

Future interface, Future-Based Style

## G

G1 (Garbage First) collector, G1-G1

garbage collection (GC), Performance Problems

- (see also memory management)
- evacuation, Evacuation-Evacuation
- G1 collector, G1-G1
- HotSpot heap, The HotSpot Heap-The HotSpot Heap
- improving on basic GC, Improving on Basic Garbage Collection-The HotSpot Heap
- mark-and-sweep algorithm, Introducing Mark-and-Sweep-Exactness and Limitations of Naive Mark-and-Sweep
- ParallelOld collector, ParallelOld-ParallelOld
- Serial collector, Serial
- Shenandoah, Shenandoah-Shenandoah
- WGH and, The Weak Generational Hypothesis-The Weak Generational Hypothesis
- ZGC, ZGC

Garbage First (G1) collector, G1-G1

Gatherer interface, Stream gatherers

GC (see garbage collection)

generational garbage collector, The Weak Generational Hypothesis

generic methods, Defining Methods, Generic Methods-Generic Methods

generics, Java Generics-Using and Designing Generic Types

- basics, Introduction to Generics-Introduction to Generics
- bounded type parameters, Bounded Type Parameters-Bounded Type Parameters
- bounded wildcards, Bounded wildcards-Bounded wildcards
- compile-time/runtime typing, Compile and Runtime Typing
- covariance, Introducing Covariance-Introducing Covariance
- diamond syntax, Diamond Syntax
- generic methods, Generic Methods-Generic Methods
- Project Valhalla and, Valhalla
- type erasure, Type Erasure-Type Erasure
- types/type parameters, Generic Types and Type Parameters
- using and designing, Using and Designing Generic Types
- wildcards, Wildcards-Bounded wildcards

GET method (HTTP), HTTP

get(), Future-Based Style

getClass(), Examples of Class Objects

getId(), getId()

getInstance(), Singleton

getName(), setName() and getName()

getPriority(), getPriority() and setPriority()

getState(), getState()

GraalVM, Leyden and Cloud Native Java

Gradle, Converting to a Multi-Release JAR

gradual typing, The Java Type System

greater than operator (>), Comparison Operators

greater than or equal to operator (>=), Comparison Operators

## H

hash chains, Concurrent Maps

hash codes, Hash codes and effective immutability

hashCode(), hashCode()-hashCode(), Hash codes and effective immutability

HashMap, Concurrent Maps

HEAD method (HTTP), HTTP

heap (see HotSpot heap)

heterogeneous collections, Introduction to Generics

hidden fields, overriding versus field hiding, Overriding is not hiding-Overriding is not hiding

homogeneous collections, Introduction to Generics

HotSpot heap, The HotSpot Heap-The HotSpot Heap

HotSpot JVM, What Is the JVM?, Command-Line Tools

HTTP, HTTP-HTTP

## I

I/O (input/output) (see file handling and I/O)

I/O stream abstraction, I/O Streams-I/O Streams

identifiers, Identifiers

IEEE-754 floating-point arithmetic standard, Method Modifiers, Java and Floating-Point Numbers

if statements, Comparison Operators, The if/else Statement-The else if clause

if/else statements, The if/else Statement-The else if clause

conditional operator (? :) as version of, The Conditional Operator

else clause, The if/else Statement

else if clause, The else if clause

nested, The if/else Statement

IHOP (Initiating Heap Occupancy Percent) threshold, G1

immutability, Mutability and Immutability-Mutability and Immutability

collections and, Creating Collections, Mutability and Immutability Revisited-Mutability and Immutability Revisited

hash codes and, Hash codes and effective immutability

string, String Immutability-Hash codes and effective immutability

immutable fields, Mutability and Immutability-Mutability and Immutability

implementation-specific code, Conventions for Portable Programs

implements keyword, Implementing an Interface

implicitly declared classes, Simple Standalone Java Programs, Simple Standalone Java Programs

import declarations, Importing Types

import statements, Module Imports

import static declarations, Importing Static Members-Static member imports and overloaded methods

increment operator (++), Increment and Decrement Operators

indexes, The List Interface

infinite loops

creating with syntax while(true), The while Statement

writing with for(;;), The for Statement

infinity

division by zero, Arithmetic Operators

modulo operator (%) and, Arithmetic Operators

inheritance

access control and, Access control and inheritance

default methods and, Do Default Methods Change Java’s Inheritance Model?-Do Default Methods Change Java’s Inheritance Model?

@inheritDoc doc-comment tag, Inline Doc-Comment Tags

initialization, classloading and, Initialization

initialize expressions (for loops), The for Statement

initializers

array, Array initializers

in variable declarations, Local Variable Declaration Statements

initializing objects

defining a constructor, Defining a Constructor-Defining a Constructor

defining multiple constructors, Defining Multiple Constructors

field defaults and initializers, Field Defaults and Initializers-Field Defaults and Initializers

invoking one constructor from another, Invoking One Constructor from Another

Initiating Heap Occupancy Percent (IHOP) threshold, G1

inline doc-comment tags, Inline Doc-Comment Tags-Inline Doc-Comment Tags

inner classes (see nested types)

InputStream class, I/O Streams

InputStreamReader class, Readers and Writers

instance fields, Instance Fields

instance methods, Instance Methods, Instance Methods or Class Methods?-Instance Methods or Class Methods?

instanceof operator, Operator summary table

basics, The instanceof Operator

pattern matching and type patterns, Pattern Matching and Type Patterns

Instant class, Introducing the Java 8 Date and Time API

int type

32-bit int values, Integer Types

conversions to other primitive types, Primitive Type Conversions

return type for operators, Return type

integer literals, Integer Types

integer types, Integer Types-Integer Types, How Java Represents Integer Types-How Java Represents Integer Types

conversions, Primitive Type Conversions-Primitive Type Conversions
integer arithmetic, Integer Types, Arithmetic Operators
integer overflow, Integer Types
wrapper classes, Integer Types
integrity by default (OpenJDK project), Integrity By Default-Integrity By Default
interface segregation, Extending Interfaces
interfaces, Interfaces, Interfaces-Marker Interfaces
abstract classes versus, Interfaces Versus Abstract Classes-Delegation
annotations, Annotations-Annotations
backward compatibility, Backward compatibility
default methods, Default Methods-Implementation of default methods
defining, Defining an Interface
definition of term, Interfaces
delegation, Delegation-Delegation
extending, Extending Interfaces
implementing, Implementing an Interface-Implementing an Interface
marker, Marker Interfaces
naming conventions, Naming and Capitalization Conventions
records and, Records and Interfaces
restrictions on, Defining an Interface
sealed, Sealed Interfaces-Sealed Interfaces
intermediate representation, bytecode as, What is bytecode?
interpreted languages, Is Java an interpreted language?
interrupt(), interrupt()
InterruptedIOException, Exceptions and Exception Handling
introspection (see Method Handles; Reflection)
invoke(), Invoking Method Handles
invokedynamic bytecode, Method Handles

invokeExact(), Invoking Method Handles

IO class, Simple Standalone Java Programs, Java Source File Structure, I/O Streams

IO.println(), Simple Standalone Java Programs

IOException objects, Working with checked exceptions, Working with checked exceptions

IP, IP

irrational numbers, Java and Floating-Point Numbers

isAlive(), isAlive()

isDone(), Future-Based Style

isJavaIdentifierPart(): Character, Identifiers

isJavaIdentifierStart(): Character, Identifiers

isNaN(), Comparison Operators

Iterable, Iteration with foreach loops, The Iterable and Iterator interfaces

Iterable interface, Iteration with foreach loops, The Iterable and Iterator interfaces-The Iterable and Iterator interfaces

iteration

- arrays, Iterating arrays
- foreach and, The foreach Variant-What foreach cannot do
- lists, Random access to lists-Random access to lists

Iterator, Nonstatic Member Classes, The Iterable and Iterator interfaces

Iterator interface, The Iterable and Iterator interfaces-The Iterable and Iterator interfaces

Iterator object, foreach loop and, What foreach cannot do

iterator(), The Iterable and Iterator interfaces

## J

JAR (Java archive) files

- jar utility, Command-Line Tools-Command-Line Tools
- manipulating as a FileSystem, FileSystem
- multi-release, Multi-Release JARs-Converting to a Multi-Release JAR

jar command, Command-Line Tools-Command-Line Tools

Java (generally)

- answering criticisms of, Answering Some Criticisms of Java-Too Corporate
- basics, Introduction to the Java Environment-Summary
- corporateness of, Too Corporate
- history, A Brief History of Java and the JVM-A Brief History of Java and the JVM
- interpreted languages and, Is Java an interpreted language?
- other programming languages compared to, Comparing Java to Other Languages-Too Corporate
- overview of versions 1.0 through 17, A Brief History of Java and the JVM-A Brief History of Java and the JVM
- pace of change, Slow to Change
- performance problems, Performance Problems
- release model, Java's Release Model
- security issues, Insecure
- syntax (see syntax, Java)
- verbosity of, Overly Verbose

Java 8

- Date and Time API, Introducing the Java 8 Date and Time API-Example
- default methods, Do Default Methods Change Java's Inheritance Model?
- interfaces in, Interfaces, Backward compatibility
- lamdba expressions (see lambda expressions)
- language changes in, What Is the Java Language?
- method references, Method References-Method References
- updates in, A Brief History of Java and the JVM

Java 9

- doclet upgrade, Doclets
- keystone release model, Adoption Rates

new features, A Brief History of Java and the JVM

platform modules, Access to modules

Set interface upgrade, The Set Interface

Java 11

migrating to modules, Migrating to Modules-Custom Runtime Images

multi-release JARs, Multi-Release JARs

new features, A Brief History of Java and the JVM

OpenJDK and, What Is the JVM?

Java 17

instanceof and pattern matching, Pattern Matching and Type Patterns

JFR and, Introduction to Java Flight Recorder (JFR)

modularizing the JDK, Modularizing the JDK

new features, A Brief History of Java and the JVM

Path interface, Path

pattern matching and type patterns, Pattern Matching and Type Patterns

record support, Records

sealed classes, Sealed Classes

Shenandoah, Shenandoah

strictfp modifier, Method Modifiers

switch expressions, The switch Expression

Java 21

case null, The switch Expression

new features, A Brief History of Java and the JVM

Java 25

implicitly declared classes, Simple Standalone Java Programs

JNI, Integrity By Default

module imports, Module Imports

new features, A Brief History of Java and the JVM

versions beyond, Future Java-Integrity By Default

Java collections (see collections)

java command, Command-Line Tools-Command-Line Tools

Java ecosystem, What Is the Java Ecosystem?

Java Flight Recorder (JFR), Introduction to Java Flight Recorder (JFR)

Java generics (see generics)

Java language (see syntax, Java)

Java Language Specification (JLS), What Is the Java Language?

Java Native Interface (JNI), Integrity By Default

Java Platform Modules System (JPMS), Java Platform Modules-Summary

adoption rates, Adoption Rates-Adoption Rates

automatic modules, Automatic Modules

basic modules syntax, Basic Modules Syntax-Basic Modules Syntax

building a simple modular application, Building a Simple Modular Application-Building a Simple Modular Application

custom runtime images, Custom Runtime Images

deploying software as multi-release JAR, Converting to a Multi-Release JAR

issues with modules, Issues with Modules-Adoption Rates

Java SE module graph, The Java SE Module Graph

lack of versioning, Lack of Versioning

migrating to modules, Migrating to Modules-Custom Runtime Images

modularizing the JDK, Modularizing the JDK-Modularizing the JDK

module imports, Module Imports

module path, The Module Path

multi-release JARs, Multi-Release JARs-Converting to a Multi-Release JAR

open modules, Open Modules-Open Modules

providing services, Providing Services

reasons for creation of, Why Modules?-Why Modules?

Unsafe class and related problems, The Unsafe Class

writing your own modules, Writing Your Own Modules-Providing Services

Java programming environment, Introduction to the Java Environment-Summary

Java ecosystem, What Is the Java Ecosystem?

Java language, What Is the Java Language?-What Is the Java Language?

Java/JVM history, A Brief History of Java and the JVM-A Brief History of Java and the JVM

JVM, What Is the JVM?-What Is the JVM?

lifecycle of Java program, The Lifecycle of a Java Program

open ecosystem, The Language, the JVM, and the Ecosystem

Java programs

lexical structure of, Lexical Structure-Punctuation

lifecycle of, The Lifecycle of a Java Program

Java SE, version 25, as default version for discussion, The Language, the JVM, and the Ecosystem

.java source file, Java Source File Structure-Java Source File Structure

Java Streams (see Streams)

Java Virtual Machine (JVM), What Is the JVM?-What Is the JVM?

classloader hierarchy, Classloader Hierarchy-Custom classloader

history, A Brief History of Java and the JVM-A Brief History of Java and the JVM

java command, Command-Line Tools-Command-Line Tools

running non-Java languages on, Can other languages run on the JVM?

security checks implemented by, Secure Programming and Classloading

verification of class file, Verification

java.base module, Modularizing the JDK

java.lang package, Packages and the Java Namespace, Importing Types, Annotations

java.net package, Networking

java.se module, The Java SE Module Graph

java.time package, Introducing the Java 8 Date and Time API

java.util package

- Map interface implementations, The Map Interface-The Map Interface
- Set implementations, The Set Interface

javac, Command-Line Tools-Command-Line Tools

- class initialization method generation, Field Defaults and Initializers
- compiler flags, The Source Code Compiler, The Source Code Compiler
- lambda expression conversion, Lambda Expression Conversion-Lambda Expression Conversion
- optimized bytecode, Is bytecode optimized?
- similarity to compiler, Is javac a compiler?
- virtual method lookup, Virtual method lookup

javadoc, Comments

- basics, Command-Line Tools
- defined, The Map Interface
- doc comments, Java Documentation Comments
- doclets, Doclets
- package documentation, Doc Comments for Packages

javap tool, Command-Line Tools

JavaScript, Java compared to, Java Compared to JavaScript

javax, package names beginning with, Packages and the Java Namespace

javax.net package, Networking

jcmd tool, Command-Line Tools-Command-Line Tools

jdeps tool, Command-Line Tools-Command-Line Tools

JDK (Java Development Kit)

- long-term projects, Long-Term JDK Projects
- modularizing of, Modularizing the JDK-Modularizing the JDK

JDK Enhancement Proposals (JEPs), Valhalla

JFR (Java Flight Recorder), Introduction to Java Flight Recorder (JFR)

jinfo tool, Command-Line Tools

jlink tool, Command-Line Tools-Command-Line Tools

JLS (Java Language Specification), What Is the Java Language?

jmap utility, Command-Line Tools

jmod tool, Command-Line Tools-Command-Line Tools

JNI (Java Native Interface), Integrity By Default

join(), join()

JPMS (see Java Platform Modules System)

jps command, Command-Line Tools

JShell, Introduction to JShell-Introduction to JShell, Protecting State with Exclusion

jstack utility, Command-Line Tools

jstat command, Command-Line Tools

jstatd tool, Command-Line Tools-Command-Line Tools

JVM (see Java Virtual Machine)

jwebserver, Command-Line Tools-Command-Line Tools

## K

Kanji character (in Java identifier), Identifiers

keystone release model, Adoption Rates

Kotlin, Mutability and Immutability Revisited

## L

labeled statements, Labeled Statements

lambda expressions, Lambda Expressions-Lambda Expressions, Lambda Expressions-Lexical Scoping and Local Variables

- anonymous classes and, Lambda Expression Conversion
- basics, Lambda Expressions-Lambda Expressions
- conversion, Lambda Expression Conversion-Lambda Expression Conversion

defined, Special Operators, Lambda Expressions

functional programming and, Lambdas and Functional Programming-Lambdas and Functional Programming

Java Streams and, Java Streams and Lambda Expressions-Arrays and Helper Methods

lexical scoping and local variables, Lexical Scoping and Local Variables-Lexical Scoping and Local Variables

method references and, Method References-Method References

method references versus, Lambdas versus method references-Lambdas versus method references

nested classes versus, Lambdas versus nested classes

nominal typing and, Lambda Expression Conversion

object-oriented design and, OOD Using Lambdas-Lambdas versus method references

large language models (LLMs), Local Variable Declaration Statements

Latin-1 character set, The char Type

lazy evaluation, Lazy evaluation-Lazy evaluation

left shift operator (<<), Bitwise and Shift Operators

less than operator (<), Comparison Operators

less than or equal to operator (<=), Comparison Operators

lexical scoping, Lexical Scoping and Local Variables

lexical structure of Java program, Lexical Structure-Punctuation

case sensitivity/whitespace, Case Sensitivity and Whitespace

comments, Comments-Comments

identifiers, Identifiers

literals, Literals

punctuation, Punctuation

reserved words, Reserved Words

Unicode character set, The Unicode Character Set

whitespace, Case Sensitivity and Whitespace

Leyden, Leyden and Cloud Native Java

lifecycle, of Java program, The Lifecycle of a Java Program

line separators, Conventions for Portable Programs

@link doc-comment tag, Formatting Multiline Doc Comments, Formatting Markdown Doc Comments, Inline Doc-Comment Tags

LinkedList class, Queues and BlockingQueues

LinkOption class, Files

@linkplain doc-comment tag, Inline Doc-Comment Tags, Cross-References in Doc Comments

List, Naming conflicts and shadowing, Naming conflicts and shadowing

List interface, The List Interface-Random access to lists

- generics and, Introduction to Generics
- iteration with foreach loops, Iteration with foreach loops
- random access to lists, Random access to lists-Random access to lists

lists, iterating through using foreach loop, What foreach cannot do

@literal doc-comment tag, Inline Doc-Comment Tags

literals, Literals

LLMs (large language models), Local Variable Declaration Statements

loadClass(), Applied Classloading

loading (classloading phase), Loading

local classes, Local Classes-Scope of a local class

local variable declaration statements, Local Variable Declaration Statements-Local Variable Declaration Statements

local variable type inference (LVTI), Type Inference, An Aside: Nondenotable Types

local variables, Local Variable Declaration Statements

- lambda expressions and, Lexical Scoping and Local Variables-Lexical Scoping and Local Variables
- naming conventions, Naming and Capitalization Conventions

LocalDate class, The parts of a timestamp

lock striping, Concurrent Maps

log analyzer program, Executing Full Java Programs

log(), Java's Standard Library of Mathematical Functions

log10(), Java's Standard Library of Mathematical Functions

logical operators, Boolean Operators-Boolean Operators

long type

- 64-bit long values, Integer Types
- conversions between char values and, Primitive Type Conversions
- return type for operators, Return type

long-term support (LTS) releases, What Is the Java Language?, A Brief History of Java and the JVM, Java's Release Model, Adoption Rates

Lookup object, Method Lookup

lookup queries, Method Lookup-Method Lookup

Loom, Long-Term JDK Projects

looping

- for statements, The for Statement
- foreach, Iteration with foreach loops
- while statements, The while Statement

LTS (long-term support) releases, What Is the Java Language?, A Brief History of Java and the JVM, Java's Release Model, Adoption Rates

lvalue, Assignment Operators

LVTI (local variable type inference), Type Inference, An Aside: Nondenotable Types

## M

main thread, Basic Concepts of Java Concurrency

main(), The throw Statement, Defining Methods, Executing Full Java Programs

Map interface, The Map Interface-The Map Interface

map(), Lambdas and Functional Programming, Map

Map.Entry interface, The Map Interface

mapped types, Mutability and Immutability Revisited

MappedByteBuffer, Mapped Byte Buffers

maps, concurrent, Concurrent Maps-Concurrent Maps

mark-and-sweep algorithm, Introducing Mark-and-Sweep-Introducing Mark-and-Sweep

basics, Introducing Mark-and-Sweep-Introducing Mark-and-Sweep

exactness/limitations, Exactness and Limitations of Naive Mark-and-Sweep

Markdown, Inline Doc-Comment Tags

/// in doc comments, Comments

doc comment formatting, Formatting Markdown Doc Comments-Formatting Markdown Doc Comments

marker interfaces, Marker Interfaces

Matcher class, Regular Expressions

Math, Java's Standard Library of Mathematical Functions

Math class, Primitive Type Conversions

Math.random(), Java's Standard Library of Mathematical Functions

mathematical functions, standard library of, Java's Standard Library of Mathematical Functions-Java's Standard Library of Mathematical Functions

Maven, What Is the Java Ecosystem?, Executing Full Java Programs, Migrating to Modules, Converting to a Multi-Release JAR, Lack of Versioning

max(), Java's Standard Library of Mathematical Functions

MAX_VALUE constant, Integer Types, Floating-Point Types

member access operator (.), Special Operators

member classes

features, Features of member classes

nonstatic, Nonstatic Member Classes-Syntax for member classes

static, Static Member Types-Features of static member types

syntax for, Syntax for member classes

memory leaks, Introducing Mark-and-Sweep, Introducing Mark-and-Sweep

memory management, Basic Concepts of Automatic Memory Management-Summary

(see also concurrency; garbage collection (GC))

basic concepts of automatic memory management, Basic Concepts of Automatic Memory Management-Exactness and Limitations of Naive Mark-and-Sweep

G1 collector, G1-G1

HotSpot heap, The HotSpot Heap-The HotSpot Heap

improving on basic garbage collection, Improving on Basic Garbage Collection-The HotSpot Heap

mark-and-sweep algorithm, Introducing Mark-and-Sweep-Introducing Mark-and-Sweep

ParallelOld collector, ParallelOld-ParallelOld

memory, primitive and reference type requirements, Reference Versus Primitive Types

metacharacters in regular expressions, Regular Expressions

metadata, class objects and, Class Objects and Metadata-Class Objects and Metadata

Method Handles, Method Handles-Invoking Method Handles

invoking, Invoking Method Handles

lookup queries, Method Lookup-Method Lookup

MethodType objects, MethodType

method invocation operator (( )), Side effects, Special Operators

Method objects, Reflection and, Method objects-Method objects

method overloading, Defining Methods, Static member imports and overloaded methods

method references, lambdas and, Method References-Method References, Lambdas versus method references-Lambdas versus method references

method signature, Defining Methods

MethodHandles.lookup(), Method Lookup

methods, Methods-Variable-Length Argument Lists

(see also specific methods)

abstract, Abstract Classes and Methods-Abstract Classes and Methods

checked/unchecked exceptions, Checked and Unchecked Exceptions-Working with checked exceptions

class methods, Class Methods

data accessor methods, Data Accessor Methods-Data Accessor Methods

defining, Defining Methods-Defining Methods

implementing native methods, Method Modifiers

instance methods, Instance Methods

instance methods versus class methods, Instance Methods or Class Methods?-Instance Methods or Class Methods?

invoking an overridden method, Invoking an overridden method-Invoking an overridden method

modifiers, Method Modifiers-Method Modifiers

naming conventions, Naming and Capitalization Conventions

native, Method Modifiers, Conventions for Portable Programs

overriding superclass methods, Overriding Superclass Methods-Invoking an overridden method

overriding versus field hiding, Overriding is not hiding-Overriding is not hiding

parameter list, Defining Methods

synchronized, The synchronized Statement

this keyword, How the this Reference Works-How the this Reference Works

variable-length argument lists, Variable-Length Argument Lists-Variable-Length Argument Lists

virtual lookup, Virtual method lookup

void, The return Statement

when executing full Java programs, Executing Full Java Programs-Executing Full Java Programs

MethodType objects, MethodType

min(), Java’s Standard Library of Mathematical Functions

MIN_VALUE constant, Integer Types, Floating-Point Types

modifiers

defined, Defining Methods

method, Method Modifiers-Method Modifiers

summary of, Modifier Summary-Summary

module descriptor, Basic Modules Syntax-Basic Modules Syntax

module path, The Module Path

module-info.java file, Basic Modules Syntax-Basic Modules Syntax

modules

- access control, Access to modules
- JMPS (see Java Platform Modules System)
- naming conventions, Naming and Capitalization Conventions

modulo operator (%), Arithmetic Operators

multidimensional arrays, Multidimensional Arrays-Multidimensional Arrays

multiple inheritance, Implementation of default methods

multiplication operator (*), Arithmetic Operators

multithreading (see concurrency)

mutability, Mutability and Immutability-Mutability and Immutability

- collections and, Creating Collections, Mutability and Immutability Revisited-Mutability and Immutability Revisited
- concurrency and, Visibility and Mutability-Visibility and Mutability
- defined, Visibility and Mutability

mutable fields, Mutability and Immutability-Mutability and Immutability

## N

namespaces

- globally unique package names, Globally Unique Package Names
- importing types, Importing Types

naming conflicts, Naming conflicts and shadowing

naming conventions, Practical Naming-Practical Naming

NaN (Not a number), Floating-Point Types

- floating-point calculations, division by zero, Arithmetic Operators
- modulo operator (%) and, Arithmetic Operators

narrowing conversions, Primitive Type Conversions

native compilation, Leyden and Cloud Native Java

native methods, Method Modifiers, Conventions for Portable Programs

natural comparison method, Comparable::compareTo()

negative infinity, Floating-Point Types

negative zero, Floating-Point Types

NEGATIVE_INFINITY constant, Floating-Point Types

nested classes, lambdas versus, Lambdas versus nested classes

nested types, Nested Types-Anonymous Classes

- anonymous classes, Anonymous Classes-Anonymous Classes
- local classes, Local Classes-Scope of a local class
- nonstatic member class, Nonstatic Member Classes-Syntax for member classes
- static member types, Static Member Types-Features of static member types

networking, Networking-IP

- HTTP, HTTP-HTTP
- IP, IP
- TCP, TCP-TCP

new keyword, Operator summary table, Side effects, Special Operators, Creating and Initializing Arrays

newVirtualThreadPerTaskExecutor(), Virtual Threads

next(), The Iterable and Iterator interfaces

NIO channels and buffers, NIO Channels and Buffers-Mapped Byte Buffers

- ByteBuffer, ByteBuffer-ByteBuffer
- MappedByteBuffer, Mapped Byte Buffers

NIO.2, Path

nominal typing, Lambda Expression Conversion, Nominal Typing-Nominal Typing

non-denotable types, An Aside: Nondenotable Types-An Aside: Nondenotable Types

nonstatic members, Basic OO Definitions, Nonstatic Member Classes-Syntax for member classes

not equals operator (!=), Operator summary table, Comparison Operators

NOT operator

bitwise NOT (), Bitwise and Shift Operators

boolean NOT (!), Boolean Operators

notify(), Indefinite Waiting, Indefinite Waiting

notifyAll(), Indefinite Waiting

null reference, The null reference

numbers, Numbers and Math-Java's Standard Library of Mathematical Functions

floating-point, Java and Floating-Point Numbers-BigDecimal

integer types, How Java Represents Integer Types-How Java Represents Integer Types

library of mathematical functions, Java's Standard Library of Mathematical Functions-Java's Standard Library of Mathematical Functions

## O

Object, Array Types

clone() method, clone()-clone()

compareTo() method, Comparable::compareTo()-Comparable::compareTo()

equals() method, equals()

hashCode() method, hashCode()-hashCode()

Object class, Reference Type Conversions

object literals, Object Literals

null keyword, The null reference

null reference, The null reference

string literals, String literals-String literals

type literals, Type literals

object references, Reference Type Conversions-Reference Type Conversions

object state, The synchronized Statement, Working with Fields and Object State-Mutability and Immutability

object-oriented (OO) programming, Object-Oriented Programming in Java-Summary

abstract classes/methods, Abstract Classes and Methods-Abstract Classes and Methods

basic OO definitions, Basic OO Definitions-Basic OO Definitions

class definition syntax, Class Definition Syntax-Class Definition Syntax

constructor chaining, Constructor Chaining

creating/initializing objects, Creating and Initializing Objects-Record Constructors

data hiding/encapsulation, Data Hiding and Encapsulation-Data Accessor Methods

default constructor, The Default Constructor

defining a single constructor, Defining a Constructor-Defining a Constructor

defining multiple constructors, Defining Multiple Constructors

extending a class, Extending a Class-Extending a Class

field defaults/initializers, Field Defaults and Initializers-Field Defaults and Initializers

fields and methods, Fields and Methods-How the this Reference Works

hiding superclass fields, Hiding Superclass Fields-Hiding Superclass Fields

interfaces, Interfaces

invoking one constructor from another, Invoking One Constructor from Another

modifier summary, Modifier Summary-Summary

overriding superclass methods, Overriding Superclass Methods-Invoking an overridden method

record constructors, Record Constructors

records, Records-Records

reference type conversions, Reference Type Conversions-Reference Type Conversions

sealed classes, Sealed Classes-Sealed Classes

subclass constructors, Subclass Constructors

subclasses/inheritance, Subclasses and Inheritance-Sealed Classes

superclasses/object/class hierarchy, Superclasses, Object, and the Class Hierarchy

object-oriented design (OOD), Introduction to Object-Oriented Design in Java-Summary

accessors, Accessors

builder pattern, Builders-Builders

by reference versus pass by reference, Is Java “Pass-by-Reference”?-Is Java “Pass-by-

Reference”?

case study: circle-ellipse problem, Case Study: Circle-Ellipse Problem-Case Study: Circle-Ellipse Problem

constants, Constants

construction patterns, Construction Patterns-Singleton

default methods and inheritance, Do Default Methods Change Java’s Inheritance Model?-Do Default Methods Change Java’s Inheritance Model?

delegation, Delegation-Delegation

exception handling, Exceptions and Exception Handling-Exceptions and Exception Handling

factory methods, Factory Methods-Factory Methods

field inheritance, Field Inheritance-Field Inheritance

fields and object state, Working with Fields and Object State-Mutability and Immutability

instance methods versus class methods, Instance Methods or Class Methods?-Instance Methods or Class Methods?

interfaces versus abstract classes, Interfaces Versus Abstract Classes-Delegation

Java values, Java Values-Constants

lambdas and, OOD Using Lambdas-Lambdas versus method references

modern OOD in Java, Modern OOD in Java-Pattern Matching

mutability/immutability, Mutability and Immutability-Mutability and Immutability

pattern matching, Pattern Matching-Pattern Matching

records, OOD Using Records-OOD Using Records

safe programming, Safe Java Programming-Safe Java Programming

sealed types, OOD Using Sealed Types-OOD Using Sealed Types

singleton pattern, Singleton-Singleton

object-relational mapping (ORM), Open Modules

objects

arrays as, Arrays-Multidimensional Arrays

converting to/from arrays, Arrays and Helper Methods

creating, Creating an Object
creating/initializing, Creating and Initializing Objects-Record Constructors
creation with new operator, Special Operators
defined, Basic OO Definitions
defining a single constructor, Defining a Constructor-Defining a Constructor
defining multiple constructors, Defining Multiple Constructors
field defaults/initializers, Field Defaults and Initializers-Field Defaults and Initializers
invoking one constructor from another, Invoking One Constructor from Another
literals, Object Literals
memory requirements for storing, Reference Versus Primitive Types
null keyword, The null reference
null reference, The null reference
operand type, Increment and Decrement Operators, The instanceof Operator
record constructors, Record Constructors
string literals, String literals-String literals
type literals, Type literals
using, Using an Object
on-demand import declaration, Importing Types
OOD (see object-oriented design)
OOP (see object-oriented programming)
open modules, Open Modules-Open Modules
OpenJDK, Long-Term JDK Projects
integrity by default, Integrity By Default-Integrity By Default
Leyden and cloud-native Java, Leyden and Cloud Native Java-Leyden and Cloud Native Java
Panama, Panama-Panama
Valhalla, Valhalla-Valhalla
operand, number and type of, Associativity

operators, Punctuation, Expressions and Operators
    arithmetic, Arithmetic Operators-Arithmetic Operators
    assignment, Assignment Operators-Assignment Operators
    associativity, Associativity
    bitwise/shift, Bitwise and Shift Operators-Bitwise and Shift Operators
    Boolean, Boolean Operators-Boolean Operators
        (see also specific operators)
    comparison, Comparison Operators-Comparison Operators
    conditional, The Conditional Operator
    increment/decrement, Increment and Decrement Operators
    instanceof, The instanceof Operator
    operand number and type, Operand number and type
    order of evaluation, Order of evaluation
    precedence, Precedence
    return type, Return type
    shift, Bitwise and Shift Operators-Bitwise and Shift Operators
    side effects, Side effects
    special, Special Operators-Special Operators
    statements and, Statements
    string concatenation, String Concatenation Operator
    summary, Operator Summary-Order of evaluation
    summary table, Operator summary table
optimization, bytecode and, Is bytecode optimized?
Optional type, Factory Methods
or(), Filter
Oracle Corporation
    GraalVM project, Leyden and Cloud Native Java
    Java and, The Language, the JVM, and the Ecosystem

order of evaluation, Order of evaluation

ORM (object-relational mapping), Open Modules

OutputStream class, I/O Streams

overflow, integer arithmetic and, Integer Types

overloading, static member imports and, Static member imports and overloaded methods

@Override annotation, Annotations

overriding superclass methods, Overriding Superclass Methods-Invoking an overridden method

- field hiding versus, Overriding is not hiding-Overriding is not hiding
- invoking an overridden method, Invoking an overridden method-Invoking an overridden method
- overriding versus field hiding, Overriding is not hiding-Overriding is not hiding
- virtual method lookup, Virtual method lookup

**P**

package access, Access to members

package declarations, Package Declaration

package keyword, Package Declaration

package-info.java, Doc Comments for Packages

package-level constants, Constants

package.html, Doc Comments for Packages

packages, Packages and the Java Namespace-Static member imports and overloaded methods

- access control, Access to packages
- declarations, Package Declaration
- doc comments for, Doc Comments for Packages
- globally unique names, Globally Unique Package Names
- importing static members, Importing Static Members-Static member imports and overloaded methods
- importing types, Importing Types-Static member imports and overloaded methods
- naming conflicts/shadowing, Naming conflicts and shadowing

naming conventions, Naming and Capitalization Conventions

static member imports and overloaded methods, Static member imports and overloaded methods

Panama, Panama-Panama

parallel processing, Parallel streams

ParallelOld collector, ParallelOld-ParallelOld

@param doc-comment tag, Doc-Comment Tags

parameterized type, Generic Types and Type Parameters

parameters

naming conventions for, Naming and Capitalization Conventions

passing from command line into scripts, Simple Standalone Java Programs

partial implementation pattern, Partial Implementations-Partial Implementations

pass-by-reference language, Is Java “Pass-by-Reference”?-Is Java “Pass-by-Reference”?

pass-by-value language, Is Java “Pass-by-Reference”?-Is Java “Pass-by-Reference”?

Path interface, Path

Path object, Simple Standalone Java Programs

Paths class, Path

pattern matching, A Brief History of Java and the JVM, Pattern Matching and Type Patterns, The switch Statement, Pattern Matching-Pattern Matching

pause goals, G1

payload type, Generic Types and Type Parameters

PECS (Producer Extends, Consumer Super) principle, Bounded wildcards

peek method, Lazy evaluation

per-thread allocation, Evacuation

permits keyword, Sealed Classes

platform classloader, Platform classloader

platform tools, Platform Tools-Introduction to Java Flight Recorder (JFR)

polymorphism, Introduction to Object-Oriented Design in Java

positive infinity, Floating-Point Types

positive zero, Floating-Point Types

POSITIVE_INFINITY constant, Float and Double classes, Floating-Point Types

post-decrement operator (--), Increment and Decrement Operators

post-increment operator (++), Increment and Decrement Operators

Postel's Law, TCP

pow(), Java’s Standard Library of Mathematical Functions

pre-decrement operator (--), Increment and Decrement Operators

pre-increment operator (++), Increment and Decrement Operators

precedence, Precedence

Predicate interface, temporal queries versus, Queries

preemptive multitasking, Thread Lifecycle

primary expressions, Expressions and Operators

primitive types, Primitive Data Types-Special Operators

- boolean, The boolean Type
- boxing/unboxing conversions, Boxing and Unboxing Conversions
- char, The char Type-String literals
- comparing objects, Comparing Objects
- conversions between integer values and floating-point values, Primitive Type Conversions-Primitive Type Conversions
- conversions to strings, String Concatenation Operator
- equals operator (==), testing operand values, Comparison Operators
- floating-point types, Floating-Point Types-Floating-Point Types
- integer types, Integer Types-Integer Types
- manipulating objects/reference copies, Manipulating Objects and Reference Copies-Manipulating Objects and Reference Copies
- reference types versus, Reference Versus Primitive Types-Reference Versus Primitive Types
- String literals, String literals

wrapper classes, Boxing and Unboxing Conversions

println(), Conventions for Portable Programs, toString()

PriorityQueue, Queues and BlockingQueues

private constants, Constants

private constructor, The Default Constructor

private modifier, Field Declaration Syntax

methods, Method Modifiers

rules governing, Access control and inheritance

PRNG (pseudorandom number generator), Java's Standard Library of Mathematical Functions

ProcessHandle API, Multi-Release JARs

Producer Extends, Consumer Super (PECS) principle, Bounded wildcards

programming conventions (see conventions)

programming, new technologies for, Your First Java Programs-Summary

executing full Java programs, Executing Full Java Programs-Executing Full Java Programs

Java source file structure, Java Source File Structure-Java Source File Structure

JShell, Introduction to JShell-Introduction to JShell

simple standalone Java programs, Simple Standalone Java Programs-Simple Standalone Java Programs

source code compiler, The Source Code Compiler-The Source Code Compiler

Project Leyden, Leyden and Cloud Native Java

Project Loom, Long-Term JDK Projects

Project Panama, Panama-Panama

Project Quarkus, Leyden and Cloud Native Java

Project Valhalla, Valhalla-Valhalla

protected access, Access to members

protected modifier, Field Declaration Syntax

methods, Method Modifiers

rules governing, Access control and inheritance

pseudorandom number generator (PRNG), Java’s Standard Library of Mathematical Functions

public class, Java Source File Structure

public constants, Constants

public modifier, Field Declaration Syntax, Access to members

methods, Method Modifiers

rules governing, Access control and inheritance

punctuation characters as tokens, Punctuation

Python, Java compared to, What Is the JVM?, Java Compared to Python

## Q

Quarkus, Leyden and Cloud Native Java

Queues, Queues and BlockingQueues-Queues and BlockingQueues

## R

Random package, Java’s Standard Library of Mathematical Functions

random(), Java’s Standard Library of Mathematical Functions

RandomAccess, Marker Interfaces

RandomAccess interface, Random access to lists

raw type, Type Erasure

Reader class, Readers and Writers-Readers and Writers

receiver object, Special Operators

receiver of a method, Special Operators

record constructors, Record Constructors

records, Records-Records

constructors, Record Constructors

interfaces and, Records and Interfaces

OOD and, OOD Using Records-OOD Using Records

record constructors, Record Constructors

rectangular arrays, Multidimensional Arrays

recursive type bound, Mutability and Immutability

reduce(), Lambdas and Functional Programming, Reduce

reference types, Reference Types-Boxing and Unboxing Conversions

- array types, Array Types-Array type widening conversions
- boxing/unboxing conversions, Boxing and Unboxing Conversions
- comparing objects, Comparing Objects
- conversions, Reference Type Conversions-Reference Type Conversions
- manipulating objects/reference copies, Manipulating Objects and Reference Copies-Manipulating Objects and Reference Copies
- null, The null reference
- object reference conversions, Reference Type Conversions-Reference Type Conversions
- operand type, The instanceof Operator
- operands, testing with == operator, Comparison Operators
- pointers in C/C++ versus, Reference Versus Primitive Types
- primitive types versus, Reference Versus Primitive Types-Reference Versus Primitive Types

reflection (reflective programming), Reflection-Problems with Reflection

- creating instances with, Creating instances with Reflection-Creating instances with Reflection
- how to use, How to Use Reflection-Problems with Reflection
- Method objects, Method objects-Method objects
- problems with, Problems with Reflection-Problems with Reflection
- when to use, When to Use Reflection

Reflection API, When to Use Reflection

region-based collector, G1

regular expressions (regex), Regular Expressions-Regular Expressions

relational operators, Comparison Operators

request methods (HTTP), HTTP

reserved words, Reserved Words

restricted keywords, Basic Modules Syntax

resume(), suspend(), resume(), countStackFrames(), and destroy()

@return doc-comment tag, Doc-Comment Tags

return statements, The return Statement

return types, Return type

right shift operators, Bitwise and Shift Operators

rounding numbers, Primitive Type Conversions

runtime typing, Compile and Runtime Typing

runtime-managed concurrency, Basic Concepts of Java Concurrency

RuntimeException, Checked and Unchecked Exceptions

rvalue, Assignment Operators

## S

safe multithreaded program, Concurrent Safety

safepoint, Exactness and Limitations of Naive Mark-and-Sweep

safety

- classloading and secure programming, Secure Programming and Classloading-Secure Programming and Classloading
- concurrent, Concurrent Safety
- object-oriented design and, Safe Java Programming-Safe Java Programming

SAM (single abstract method) type, Lambda Expressions

scaffold methods, Streams utility default methods

Scala, Mutability and Immutability Revisited

scientific notation, Floating-Point Types

scope, of local variables, Local Variable Declaration Statements

sealed classes, Class Definition Syntax, Sealed Classes-Sealed Classes

sealed interfaces, Sealed Interfaces-Sealed Interfaces

sealed keyword, Sealed Classes-Sealed Classes

sealed types, OOD Using Sealed Types-OOD Using Sealed Types

search, regular expressions for, Regular Expressions-Regular Expressions

secure programming, classloading and, Secure Programming and Classloading-Secure Programming and Classloading

security, Insecure

@see doc-comment tag, Doc-Comment Tags, Cross-References in Doc Comments

separators, Punctuation

Serial collector, Serial

@serial doc-comment tag, Doc-Comment Tags

@serialData doc-comment tag, Doc-Comment Tags

@serialField doc-comment tag, Doc-Comment Tags

SerialOld collector, Serial

ServerSocket class, TCP

services mechanism, in JPMS, Providing Services

Set interface, The Set Interface-The Set Interface

setDaemon(), setDaemon()

setName(), setName() and getName()

setPriority(), getPriority() and setPriority()

setUncaughtExceptionHandler(), setUncaughtExceptionHandler()

Shenandoah collector, Shenandoah-Shenandoah

shift operators, Bitwise and Shift Operators-Bitwise and Shift Operators

short type, Primitive Type Conversions

shutdown(), Volatile State

side effects, operators and, Side effects

signature, in class definition, Basic OO Definitions

@since doc-comment tag, Doc-Comment Tags

single abstract method (SAM) type, Lambda Expressions

single type import declaration, Importing Types

singleton pattern, Singleton-Singleton

Socket class, TCP

sort()

    as default method, Implementation of default methods

    static member imports and, Static member imports and overloaded methods

SortedMap interface, The Map Interface

SortedSet interface, The Set Interface

source code compiler, The Source Code Compiler-The Source Code Compiler

source file, Java, Java Source File Structure-Java Source File Structure

special imports, Introduction to JShell

special operators (language constructs), Special Operators-Special Operators

StandardCopyOption enum, Files

start(), start()

state, protecting with exclusion, Protecting State with Exclusion-Protecting State with Exclusion

statements, Statements-Enabling assertions

    (see also specific statements)

static (class) fields, Class Fields

static final constant, naming conventions for, Naming and Capitalization Conventions

static import declarations, Importing Static Members-Static member imports and overloaded methods

static initializer, Field Defaults and Initializers

static keyword, Instance Methods or Class Methods?

static member types, Static Member Types-Features of static member types

static members, Basic OO Definitions

    importing, Importing Static Members-Static member imports and overloaded methods

    imports and overloaded methods, Static member imports and overloaded methods

static methods, Method Modifiers

static modifier, Field Declaration Syntax

static runtime images, Leyden and Cloud Native Java

static synchronized method, The synchronized Statement

statically typed language, The Java Type System

stop(), stop()

stop-the-world (STW) garbage collection, Exactness and Limitations of Naive Mark-and-Sweep

storage location, Assignment Operators

stream abstraction, I/O Streams-I/O Streams

Stream object, The Streams API

stream(), The Streams API

Streams, Java Streams and Lambda Expressions-Arrays and Helper Methods

- arrays and helper methods, Arrays and Helper Methods
- default methods, Streams utility default methods-Streams utility default methods
- filtering, Filter, Further filtering
- flattening, Flattening
- forEach, forEach
- functional approaches, Functional Approaches-Streams utility default methods
- Gatherer interface, Stream gatherers
- lambda expressions and, Java Streams and Lambda Expressions-Arrays and Helper Methods
- lazy evaluation, Lazy evaluation-Lazy evaluation
- map idiom, Map
- matching, Matching in streams
- parallel stream processing, Parallel streams
- reduce, Reduce
- Streams API, The Streams API
- transforming to nonstream collection, From streams to Collections-From streams to Collections
- transforming to values, From streams to values

Streams API, The Streams API

(see also Java Streams)

strictfp modifier, Method Modifiers, Class Definition Syntax

String, Extending a Class

String class

reference type conversions and, Reference Type Conversions

special syntax for, Special Syntax for Strings-String Formatting

string concatenation operator (+), String Concatenation Operator, String literals, String concatenation

string literals, String literals, String literals-String literals, String literals

String type, String literals versus, String literals

StringBuffer class, String Immutability

StringBuilder class, String concatenation

strings, Text-String Formatting

concatenation, String concatenation-String concatenation

conversions for all primitive types, String Concatenation Operator

converting to integer values, Integer Types

formatting, String Formatting

hash codes and effective immutability, Hash codes and effective immutability

immutability, String Immutability-Hash codes and effective immutability

literals, String literals

String class, String literals

toString() method, toString()

STW (stop-the-world) garbage collection, Exactness and Limitations of Naive Mark-and-Sweep

subclasses

access control, Access control and inheritance

constructor chaining, Constructor Chaining

default constructor, The Default Constructor

defined, Basic OO Definitions

extending a class, Extending a Class-Extending a Class

inheritance and, Subclasses and Inheritance-Sealed Classes

overriding superclass methods, Overriding Superclass Methods-Invoking an overridden method

subclass constructors, Subclass Constructors

superclasses/object/class hierarchy, Superclasses, Object, and the Class Hierarchy

subList(), The List Interface

subtraction operator (-), Arithmetic Operators

super keyword

as reserved word, Subclass Constructors

invoking an overridden method, Invoking an overridden method-Invoking an overridden method

super() versus, Invoking an overridden method

super(), Subclass Constructors, Invoking an overridden method

superclasses, Superclasses, Object, and the Class Hierarchy

default constructor, The Default Constructor

defined, Basic OO Definitions

hiding superclass fields, Hiding Superclass Fields-Hiding Superclass Fields

overriding superclass methods, Overriding Superclass Methods-Invoking an overridden method

reference type conversions, Reference Type Conversions

surrogate pairs (Unicode supplementary characters), The char Type

survivor space, Evacuation

suspend(), suspend(), resume(), countStackFrames(), and destroy()

switch statements, The switch Statement-The switch Expression

synchronized keyword, The synchronized Statement

synchronized methods, Method Modifiers

synchronized statements, The synchronized Statement

syntax, Java, Java Syntax from the Ground Up-Summary
    arrays, Arrays-Multidimensional Arrays
    case sensitivity, Case Sensitivity and Whitespace
    classes/objects, Introduction to Classes and Objects-Lambda Expressions
    comments, Comments-Comments
    expressions and operators, Expressions and Operators-Expressions and Operators
    field declaration syntax, Field Declaration Syntax-Field Declaration Syntax
    identifiers, Identifiers
    lexical structure, Lexical Structure-Punctuation
    literals, Literals
    methods, Methods-Variable-Length Argument Lists
    packages and the Java namespace, Packages and the Java Namespace-Static member imports and overloaded methods
    primitive data types, Primitive Data Types-Special Operators
    punctuation, Punctuation
    reference types, Reference Types-Boxing and Unboxing Conversions
    reserved words, Reserved Words
    statements, Statements-Enabling assertions
    top-down overview, Java Programs from the Top Down
    Unicode character set, The char Type
    whitespace, Case Sensitivity and Whitespace
System, Arrays and Helper Methods
system (application) classloader, Application classloader
system classes, Conventions for Portable Programs
System.arraycopy(), Copying arrays
System.err object, Java Source File Structure
System.exit(), finally
System.gc(), Introducing Mark-and-Sweep

System.in, I/O Streams

System.out, Java Source File Structure, I/O Streams

System.out.println(), Defining Methods, toString()

## T

tab completion, Introduction to JShell

tags

- doc-comment, Doc-Comment Tags-Doc-Comment Tags
- inline doc-comment, Inline Doc-Comment Tags-Inline Doc-Comment Tags

task queue, Executors and Threadpools

TCP, TCP-TCP

temporal package, Introducing the Java 8 Date and Time API

TemporalAdjuster interface, Adjusters

TemporalQuery interface, Queries

tenuring threshold, Evacuation

test double, Interfaces

test expressions (for loops), The for Statement

text, Text-Regular Expressions

- regular expressions and, Regular Expressions-Regular Expressions
- strings, Text-String Formatting
  - (see also strings)

text blocks, String literals

this keyword, How the this Reference Works-How the this Reference Works, Invoking One Constructor from Another

this(), Subclass Constructors

thread-local allocation buffer (TLAB), Evacuation

Thread.sleep(), Thread Lifecycle

Thread.State, Thread Lifecycle

ThreadLocalRandom class, Java’s Standard Library of Mathematical Functions

ThreadPoolExecutor (TPE) class, Executors and Threadpools-Executors and Threadpools
threadpools, Executors and Threadpools-Executors and Threadpools
threads, Working with Threads-Synchronization Checklist
(see also concurrency)
dangerous methods of, Dangerous Methods of Thread
executors and threadpools, Executors and Threadpools-Executors and Threadpools
indefinite waiting, Indefinite Waiting-Indefinite Waiting
lifecycle of, Thread Lifecycle-Thread Lifecycle
Project Loom and, Long-Term JDK Projects
synchronization checklist, Synchronization Checklist
useful methods of, Useful Methods of Thread-setUncaughtExceptionHandler()
virtual threads, Virtual Threads-Virtual Threads
working with, Working with Threads-Synchronization Checklist
throw statements, The throw Statement-The throw Statement
methods using to throw checked exceptions, Defining Methods
stopping switch statements, The switch Statement
Throwable, Exceptions and Exception Handling
Throwable class, catch, Exceptions and Exception Handling
Throwable objects, Checked and Unchecked Exceptions
throws clause (method signature), Defining Methods, Working with checked exceptions
@throws doc-comment tag, Doc-Comment Tags
time zones, Time Zones
timestamp, The parts of a timestamp
TLAB (thread-local allocation buffer), Evacuation
top-level directory, Java Source File Structure
top-level types, Nested Types
toString(), String Concatenation Operator, toString(), toString()
TPE (ThreadPoolExecutor) class, Executors and Threadpools-Executors and Threadpools

transient modifier, Field Declaration Syntax

transitive closure, Introducing Mark-and-Sweep, Preparation and Resolution

TreeMap class, The Map Interface

TreeSet class, The Set Interface

Truffle, Leyden and Cloud Native Java

try block, The try/catch/finally Statement

try clause, try

try-with-resources (TWR) statements, The try-with-resources Statement, try-with-resources Revisited-try-with-resources Revisited

try/catch/finally statements, The try/catch/finally Statement-finally

tuples, Java records versus, OOD Using Records

two's complement, How Java Represents Integer Types

TWR (try-with-resources) statements, The try-with-resources Statement, try-with-resources Revisited-try-with-resources Revisited

type conversion or casting operator (( )), Special Operators

type erasure, Type Erasure-Type Erasure

type inference, Lambda Expressions, Type Inference-Type Inference

type literals, Type literals

type parameter, Bounded Type Parameters-Bounded Type Parameters

type patterns, Pattern Matching and Type Patterns

type safety, Safe Java Programming

type system, The Java Type System-Summary

- annotations, Annotations-Annotations
- characteristics of, Summary
- describing, Describing the Java Type System-An Aside: Nondenotable Types
- enums, Enums and Annotations-Enums
- generics, Java Generics-Using and Designing Generic Types
- interfaces, Interfaces-Marker Interfaces

JShell and, Introduction to JShell

lambda expressions, Lambda Expressions-Lexical Scoping and Local Variables

nested types, Nested Types-Anonymous Classes

nominal typing, Nominal Typing-Nominal Typing

non-denotable types, An Aside: Nondenotable Types-An Aside: Nondenotable Types

type inference, Type Inference-Type Inference

type variance, Bounded wildcards

types, Importing Types

(see also generics)

importing, Importing Types-Static member imports and overloaded methods

naming conventions, Naming and Capitalization Conventions

wildcards, Wildcards-Bounded wildcards

## U

UB (undefined behavior), Integrity By Default

umbrella module, The Java SE Module Graph

unary operators, Operand number and type

associativity, Associativity

unary minus (-) operator, Arithmetic Operators

unbound method reference, Method References

unbounded queue, Queues and BlockingQueues

unboxing conversions, Boxing and Unboxing Conversions

unchecked exceptions, Checked and Unchecked Exceptions-Working with checked exceptions

undefined behavior (UB), Integrity By Default

underflow, integer arithmetic and, Integer Types

Unicode character set, The Unicode Character Set, The char Type

escaping in char literals, The char Type

supplementary characters, The char Type

unknown type, Wildcards

(see also wildcard types)

Unsafe class, The Unsafe Class

unsigned right shift (>>>) operator, Operator summary table, Bitwise and Shift Operators

update expressions (for loops), The for Statement

URL class, Working with checked exceptions

URLClassLoader, loadClass(), Applied Classloading

user thread, setDaemon()

UTF-8 identifier, Identifiers

## V

Valhalla, Valhalla-Valhalla

@value doc-comment tag, Inline Doc-Comment Tags

valueof(), Integer Types, Floating-Point Types

values, transforming streams to, From streams to values

var (pseudo-keyword), An Aside: Nondenotable Types-An Aside: Nondenotable Types

varargs (variable-length argument) methods, Variable-Length Argument Lists-Variable-Length Argument Lists

variable declaration statements, Local Variable Declaration Statements-Local Variable Declaration Statements

variable-length argument methods (varargs), Variable-Length Argument Lists-Variable-Length Argument Lists

variables

declaring, Local Variable Declaration Statements

local variable declaration statements, Local Variable Declaration Statements-Local Variable Declaration Statements

naming conventions, Naming and Capitalization Conventions

Vector API, Panama, Panama

verification (classloading phase), Verification

@version doc-comment tag, Doc-Comment Tags

versioning, as JPMS deficiency, Lack of Versioning

virtual method lookup, Virtual method lookup

virtual threads (vthreads), A Brief History of Java and the JVM

visibility, concurrency and, Visibility and Mutability-Visibility and Mutability

void keyword, The return Statement, Defining Methods, Problems with Reflection

volatile keyword, Field Declaration Syntax, Volatile State

## W

wait(), Indefinite Waiting, Indefinite Waiting

WaitingQueue, Queues and BlockingQueues

warnings, The Source Code Compiler

watch services, Watch Services and Directory Searching

weak generational hypothesis (WGH), The Weak Generational Hypothesis-The Weak Generational Hypothesis

while statements, The while Statement

whitespace, Case Sensitivity and Whitespace

widening conversions, Primitive Type Conversions, Primitive Type Conversions, Array type widening conversions

wildcard types, Wildcards

working set, The Weak Generational Hypothesis

wrapper classes, Integer Types, Boxing and Unboxing Conversions

wrapper collections, Utility Methods

write once, run anywhere (WORA), What Is the JVM?

Writer class, Readers and Writers-Readers and Writers

## X

XOR operator

bitwise XOR (^), Bitwise and Shift Operators

boolean XOR (^), Boolean Operators

## Z

zero (0)

division by zero, Arithmetic Operators

positive and negative zero, Floating-Point Types

represented by float and double types, Floating-Point Types

zero extension, Bitwise and Shift Operators

ZGC collector, ZGC

zone package, Introducing the Java 8 Date and Time API

## About the Authors

**Benjamin J. Evans** is a Java Champion and senior principal software engineer at Red Hat. Previously he was lead architect for instrumentation at New Relic, and cofounded jClarity, a performance tools startup acquired by Microsoft. He has also worked as chief architect for listed derivatives at Deutsche Bank and as senior technical instructor for Morgan Stanley. He served for six years on the Java Community Process Executive Committee, helping define new Java standards. Ben is the author of six books—including *Optimizing Java*, the new editions of *Java in a Nutshell*, and the recently updated *The Well-Grounded Java Developer*—and he writes regularly for Oracle's *Java Magazine* and other industry publications.

**Jason Clark** is a staff software engineer at GitHub, where he has worked on everything from remote development environments to cloud-based agentic systems. Previously, he worked at New Relic, instrumenting user applications and then ingesting petabyte-scale telemetry data with JVM data processing pipelines.

**David Flanagan** is a computer programmer who spends most of his time writing about JavaScript and Java. His books with O'Reilly include *Java in a Nutshell*, *Java Examples in a Nutshell*, *Java Foundation Classes in a Nutshell*, *JavaScript: The Definitive Guide*, and *JavaScript Pocket Reference*. David has a degree in computer science and engineering from the Massachusetts Institute of Technology.

## Colophon

The animal on the cover of *Java in a Nutshell*, ninth edition, is a Javan tiger (*Panthera tigris sondaica*), a subspecies unique to the island of Java. Although this tiger's genetic isolation once presented an unrivaled opportunity for evolutionary study to biologists and other researchers, the subspecies has all but disappeared in the wake of human encroachment on its habitat: in a worst-case scenario for the tiger, Java developed into the most densely populated island on Earth, and awareness of the Javan tiger's precarious position apparently came too late to secure the animal's survival even in captivity.

The last confirmed sighting of the tiger occurred in 1976, and it was declared extinct by the World Wildlife Fund in 1994. However, reports of sightings around Meru Betiri National Park in East Java and in the Muria mountain range persist. Camera traps have been used as recently as 2012 in efforts to verify the Javan tiger's continued existence.

Many of the animals on O'Reilly covers are endangered; all of them are important to the world.

The cover image is based on a 19th-century engraving from the *Dover Pictorial Archive*. The cover fonts are Gilroy Semibold and Guardian Sans. The text font is Adobe Minion Pro; the heading font is Adobe Myriad Condensed; and the code font is Dalton Maag's Ubuntu Mono.